Huawei openEuler Certification Training

HCIE-openEuler

Lab Guide

Issue: 1.0



Huawei Technologies Co., Ltd

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

The Huawei certification system is a platform for shared growth, part of a thriving partner ecosystem. There are three types of certification: one for ICT infrastructure, one for base hardware and software, and one for cloud services and platforms.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Huawei certification courses cover the entire ICT domain, with a focus on how today's architecture generates cloud-pipe-device synergy. The courses present the latest developments of all essential ICT aspects to foster a thriving ICT talent ecosystem for the digital age.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. HCIE-openEuler certification covers the Ceph cloud storage management, the cloud operating system (OS) cluster management, advanced practices of cloud native platform Kubernetes, advanced automation management, continuous integration and continuous delivery (CI/CD), intelligent operation and maintenance (O&M) platform A-Ops, and log management system EFK.

Huawei certification helps you unlock opportunities to advance your career and take one more step towards the top of the industry.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Overview

This document is intended for candidates who are preparing for the HCIE-openEuler exam and readers who want to know how to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management on openEuler.

Description

This document consists of seven exercises, including Ceph installation, Ceph resource pool management, Ceph cluster configuration, Ceph object storage gateway configuration and usage, Ceph block storage and file storage configuration and usage, and CRUSH map configuration.

- Exercise 1: Ceph installation. In this exercise, three Elastic Cloud Servers (ECSs) are used to install and initialize the Ceph cluster, and common service management operations, such as adding and deleting services, are performed.
- Exercise 2: Basic operations of Ceph resource pool management, including creating a resource pool, setting replicas and specifications for the resource pool, and creating snapshots.
- Exercise 3: Ceph user management, including user creation, user permission authorization, and authorization file import and export.
- Exercise 4: Ceph object storage practices, including creating an object storage gateway, configuring object bucket quotas, and configuring multi-region disaster recovery (DR).
- Exercise 5: Ceph block storage practices, including creating block storage, mapping and mounting clients, and creating, deleting, and restoring RADOS block device (RBD) snapshots.
- Exercise 6: Ceph file storage practices, including creating the file storage resource pool and mounting the file system to the client.
- Exercise 7: The CRUSH map. In this exercise, you will be expected to manually change the drive type in the cluster to solid-state drive (SSD) by modifying the CRUSH map.

Background Knowledge Required

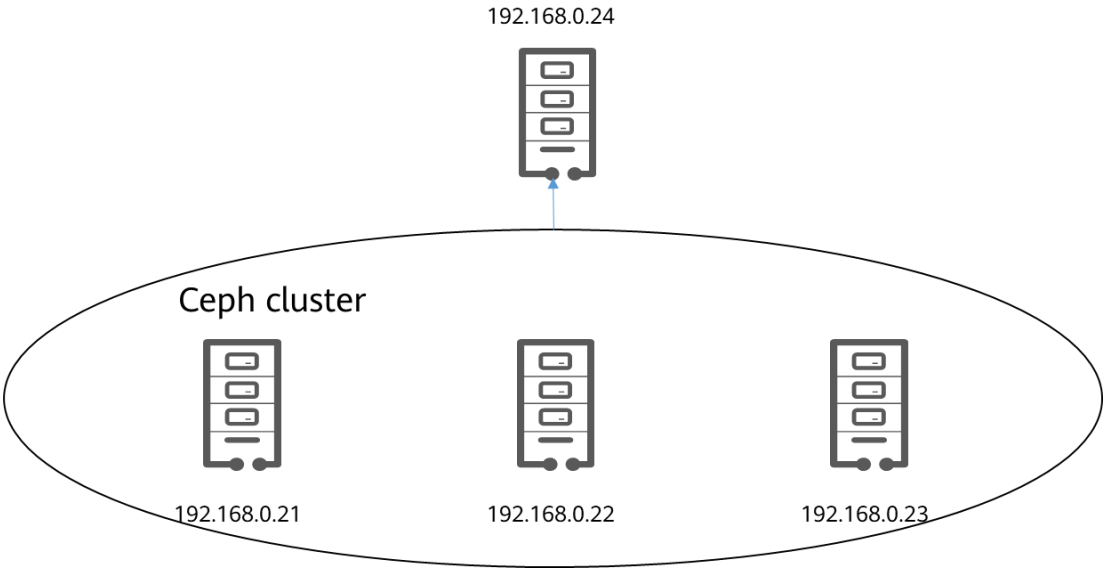
This course is for Huawei certification. To better understand this course, the intended audiences are advised to meet the following requirement:

- Have basic Linux knowledge. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Lab Environment Overview

This lab environment requires four ECSs: ceph01, ceph02, ceph03, and Client. The corresponding addresses are 192.168.0.21, 192.168.0.22, 192.168.0.23, and 192.168.0.24. ceph0*x* are the Ceph nodes and are connected to three 10 GB data disks, respectively, where c7.xlarge.2 (4U8G) is used as the flavor. Client is the client node and uses c7.large.2 (2U4G) as the flavor. To view the lab result on the browser, bind elastic IP addresses (EIPs) to the ECSs where active MGR is located. Because the EIPs are not manually specified, use the actual EIPs in your labs.

In this environment, ceph01 is the bootstrap node and its EIP is 116.63.109.200. If MGR is ready for a switchover, you need to manually bind the EIP to the corresponding node. See the following figure.



For the ECS security group, allow both inbound and outbound traffic.

Lab Environment Preparation

Checking Devices

Before starting the lab practices, each trainee should apply for ECSs on Huawei Cloud according to the following table.

ECS Name	Specifications	Remarks
ceph01	4 vCPUs | 8GiB | c7.xlarge.2	Add three 10 GB data disks.
ceph02	4 vCPUs | 8 GiB | c7.xlarge.2	Add three 10 GB data disks.
ceph03	4 vCPUs | 8 GiB | c7.xlarge.2	Add three 10 GB data disks.
Client	2 vCPUs | 4 GiB | c7.large.2	

Contents

About This Document 4
Overview 4
Description 4
Background Knowledge Required 4
Lab Environment Overview 5
Lab Environment Preparation 5
1 Installing Ceph 1
1.1 Introduction 1
1.1.1 About This Exercise 1
1.1.2 Objectives 1
1.2 Preparations 1
1.3 Install Ceph 4
1.4 Other Common Ceph Operations 7
1.4.1 Removing Services 7
1.4.2 Removing OSDs 8
1.4.3 Removing a Node 9
1.4.4 Manually Deploying a Service 10
2 Managing Resource Pools 12
2.1 Introduction 12
2.1.1 About This Exercise 12
2.1.2 Objectives 12
2.2 Creating and Configuring a Replicated Pool 12
2.3 Creating an Erasure-coded Pool 18
3 Practicing User Management in Ceph 20
3.1 Introduction 20
3.1.1 About This Exercise 20
3.1.2 Objectives 20
3.2 Creating a User and Granting Permissions 20
4 Practicing Object Storage in Ceph 23
4.1 Introduction 23
4.1.1 About This Exercise 23
4.1.2 Objectives 23
4.2 Creating an RGW 23
4.3 Preparing the Client 26

4.4 Limiting the Bucket Quota....28
4.5 Practicing Multi-Zone Configuration....30
5 Practicing Ceph Block Storage RBD....38
5.1 Introduction....38
5.1.1 About This Exercise....38
5.1.2 Objectives....38
5.2 Creating RBDs and Mounting Them to a Client....38
5.3 Expanding RBD Capacity....41
5.4 Managing RBD Snapshots....42
6 Practicing Ceph File Storage CephFS....45
6.1 Introduction....45
6.1.1 About This Exercise....45
6.1.2 Objectives....45
6.2 Resources Required for Creating CephFS....45
6.3 Mounting the File System to the Client....46
7 Manually Editing the CRUSH Map....48
7.1 Introduction....48
7.1.1 About This Exercise....48
7.1.2 Objectives....48
7.2 Manually Configuring the CRUSH Map....48

1 Installing Ceph

1.1 Introduction

1.1.1 About This Exercise

In this exercise, Ceph running on openEuler 22.03 LTS is installed on the ECS, and the hard disks of all nodes are configured as OSDs for subsequent labs.

1.1.2 Objectives

This exercise corresponds to the Ceph Architecture and Installation slides. Though performing basic operations, you will compound your knowledge of Ceph. Operations include installing Ceph, adding and deleting components, and installing and deleting hosts.

1.2 Preparations

Prepare your environment as follows:

1. Configure host name resolution.
2. Disable the firewall and SELinux.
3. Configure time synchronization.
4 Download and Install Cephadm.
5. Configure the Yum source required by Ceph.
6. Install the container engine recommended by Ceph.

Step 1 Configure host names and host name resolution.

Set the host names of the three hosts to **ceph01. novalocal**, **ceph02. novalocal**, **ceph03. novalocal**

Log in to all hosts and add the following content to the hosts file:

```
192.168.0.21 ceph01.novalocal ceph01
192.168.0.22 ceph02.novalocal ceph02
192.168.0.23 ceph03.novalocal ceph03
```

Then, ping each host name on ceph01 to ensure normal network communication.

```
[root@ceph01 ~]# ping ceph01
PING ceph01.novalocal (192.168.0.21) 56(84) bytes of data.
64 bytes from ceph01.novalocal (192.168.0.21): icmp_seq=1 ttl=64 time=0.038 ms

--- ceph01.novalocal ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.038/0.038/0.038/0.000 ms
[root@ceph01 ~]# ping ceph02
PING ceph02.novalocal (192.168.0.22) 56(84) bytes of data.
64 bytes from ceph02.novalocal (192.168.0.22): icmp_seq=2 ttl=64 time=0.196 ms
64 bytes from ceph02.novalocal (192.168.0.22): icmp_seq=3 ttl=64 time=0.141 ms

--- ceph02.novalocal ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 0.141/0.225/0.338/0.083 ms
[root@ceph01 ~]# ping ceph03
PING ceph03.novalocal (192.168.0.23) 56(84) bytes of data.
64 bytes from ceph03.novalocal (192.168.0.23): icmp_seq=1 ttl=64 time=0.304 ms
64 bytes from ceph03.novalocal (192.168.0.23): icmp_seq=2 ttl=64 time=0.172 ms

--- ceph03.novalocal ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1061ms
rtt min/avg/max/mdev = 0.172/0.238/0.304/0.066 ms
```

Step 2 Disable the firewall and SELinux.

Log in to all hosts and run **systemctl disable firewalld** to disable the firewall from automatically starting upon system startup.

Then, disable SELinux.

Step 3 Configure time synchronization.

Run the following command on all hosts to install chrony and set it to automatically start upon system startup:

```
yum install -y chrony
systemctl enable chronyd --now
```

Run **chronyc sources** to check the clock sources of the current ECS. Ensure that the clock sources are the same before performing subsequent operations.

```
[root@ceph01 ~]# chronyc sources
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^? ntp1.flashdance.cx            2   6     1     1  -1343ms[-1343ms] +/-  132ms
^? ntp7.flashdance.cx            2   6     1     1  -1342ms[-1342ms] +/-  132ms
^? tick.ntp.infomaniak.ch        1   6     1     1  -1348ms[-1348ms] +/-   95ms
^? 119.28.183.184                2   6     1     1  -1341ms[-1341ms] +/-   38ms
```

Step 4 Download and install Cephadm.

This step only needs to be performed on one host. In this exercise, this step is performed on ceph01.

Run the following command to download Cephadm adapted to openEuler:

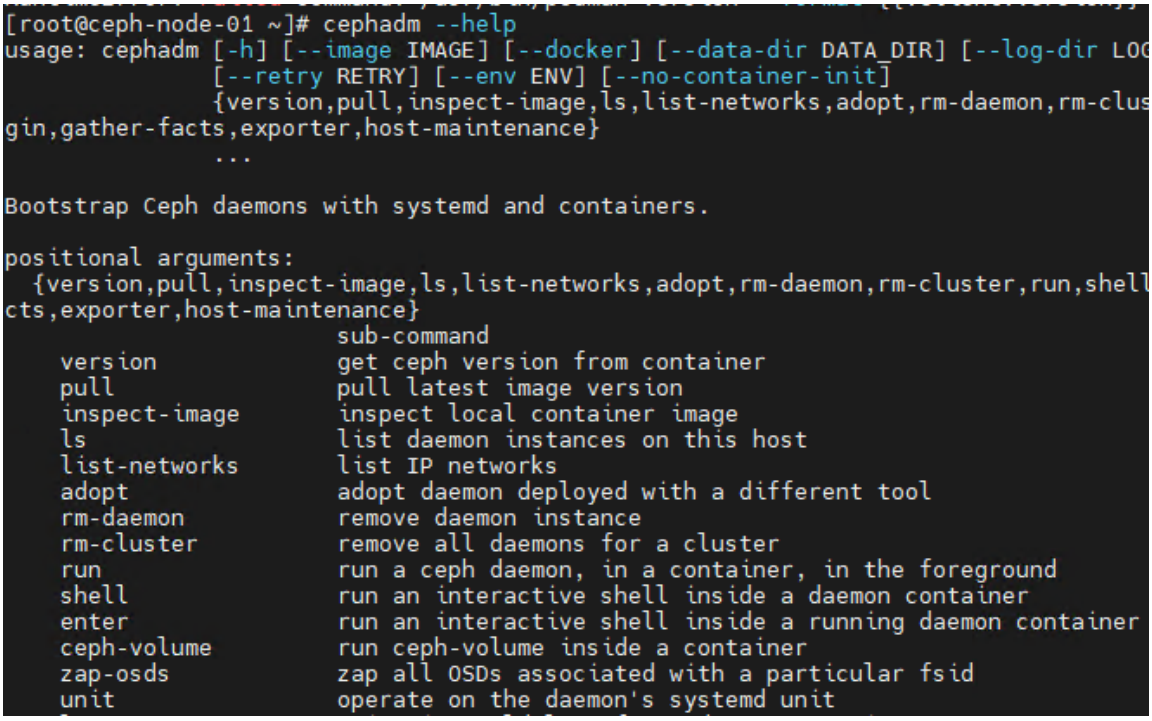
```
git clone https://gitee.com/yftyxa/openeuler-cephadm.git
```

If the git command does not exist, install Cephadm manually.

Once it is downloaded, copy the Cephadm file to the **/usr/sbin/** directory and grant the execute permission to the file. The commands are as follows:

```
cp openeuler-cephadm/cephadm /usr/sbin && chmod a+x /usr/sbin/cephadm
```

After the installation is complete, confirm the Cephadm installation is correct by checking the help information. The following figure shows the help information.



Step 5 Add the Yum source required by Ceph.

Run the following commands on all hosts to add the Ceph Yum source:

```
cat >> /etc/yum.repos.d/ceph.repo <<EOF
[ceph]
name=ceph x86_64
baseurl=https://repo.huaweicloud.com/ceph/rpm-pacific/el8/x86_64
enabled=1
gpgcheck=0

[ceph-noarch]
name=ceph noarch
baseurl=https://repo.huaweicloud.com/ceph/rpm-pacific/el8/noarch
enabled=1
gpgcheck=0

[ceph-source]
name=ceph SRPMS
baseurl=https://repo.huaweicloud.com/ceph/rpm-pacific/el8/SRPMS
enabled=1
gpgcheck=0
EOF
```

Step 6 Install the container engine recommended by Ceph.

Run the following commands on all hosts to add the Yum source required by the container engine:

```
wget -O /etc/yum.repos.d/CentOS-Base.repo https://repo.huaweicloud.com/repository/conf/CentOS-8-reg.repo
```

Run the following commands to install the corresponding container engine:

```
yum install podman-3.3.1-9.module_el8.5.0+988+b1f0b741.x86_64 lvm2 -y
```

During the installation, if there is a software package conflict, uninstall the software packages and then continue the installation. Once installed, set the container engine to start upon system startup.

Then, restart all ECS hosts.

1.3 Install Ceph

After the ECSs are restarted, install and deploy the Ceph cluster.

Step 1 Initialize the Ceph cluster on ceph01.

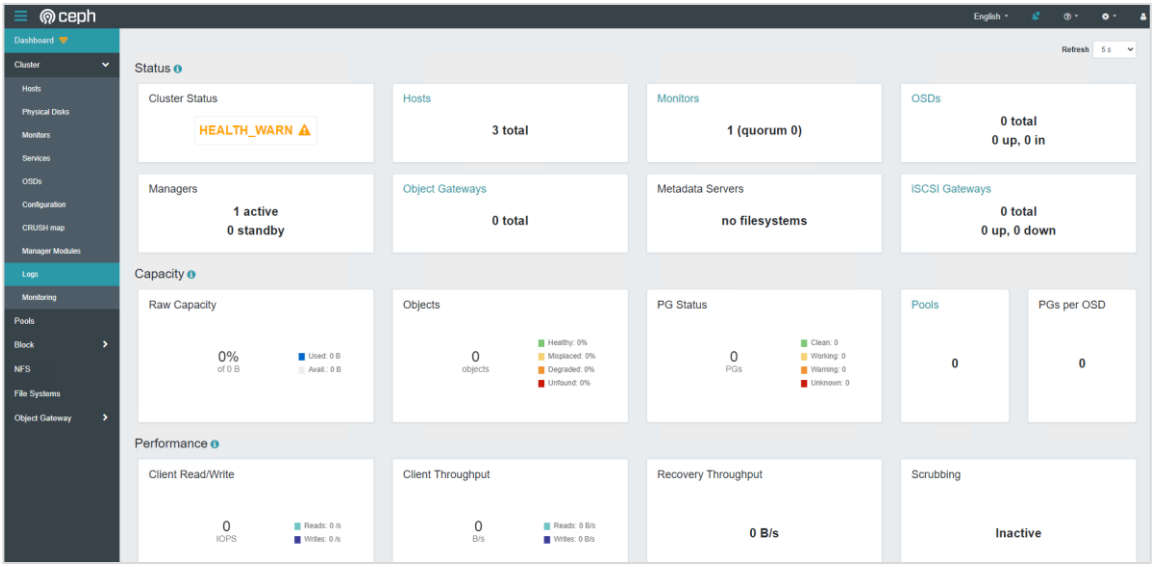
Run the following commands to initialize the Ceph cluster:

```
cephadm bootstrap --mon-ip 192.168.0.21 --allow-fqdn-hostname --initial-dashboard-user admin --initial-dashboard-password Huawei@123 --dashboard-password-noupdate
```

After the initialization is complete, the system returns the graphical login interface of the cluster. See the following figure.



Use the EIP bound to ceph01 to log in to the Ceph web page. See the following figure.



Step 2 Add a node to the cluster.

Create a Ceph management container.

```
cephadm shell
```

```
[root@ceph01 ~]# cephadm shell
Inferring fsid c5c8019c-5697-11ee-8a12-fa163edb3348
Using recent ceph image swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/ceph@sha256:cb51f1ed84198177a48ff2c370008462eb3cb537c9a0e66acbda1724b6f4c14
```

Generate a cluster public key and copy it to the remaining hosts.

```
ceph cephadm get-pub-key > ~/ceph.pub
ssh-copy-id -f -i ~/ceph.pub root@ceph02.novalocal
ssh-copy-id -f -i ~/ceph.pub root@ceph03.novalocal
```

When adding the key, enter the password of the root user of the host as prompted. See the following figure.

```
[ceph: root@ceph01 /]# ssh-copy-id -f -i ~/ceph.pub root@ceph03.novalocal
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/ceph.pub"
The authenticity of host 'ceph03.novalocal (192.168.0.23)' can't be established.
ECDSA key fingerprint is SHA256:wA1yHugkLw0z1+YrE1jbVsFEbU6k7HwlB/3HbTskpfY.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Authorized users only. All activities may be monitored and reported.
root@ceph03.novalocal's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'root@ceph03.novalocal'"
and check to make sure that only the key(s) you wanted were added.
```

Add all hosts to the cluster.

```
ceph orch host add ceph02.novalocal
ceph orch host add ceph03.novalocal
```

After the hosts are added, check the host status in the current cluster.

```
ceph orch host ls
```

```
[root@ceph01 ~]# ceph orch host ls
HOST              ADDR          LABELS  STATUS
ceph01.novalocal  192.168.0.21  _admin
ceph02.novalocal  192.168.0.22
ceph03.novalocal  192.168.0.23
3 hosts in cluster
```

Wait for the system to automatically deploy services to all nodes. After the deployment is complete, the status is as follows:

```
[root@ceph01 ~]# ceph orch ls
NAME           PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager   ?:9093,9094      1/1  9m ago     3h   count:1
crash                           3/3  9m ago     3h   *
grafana        ?:3000           1/1  9m ago     3h   count:1
mgr                             2/2  9m ago     3h   count:2
mon                             3/5  9m ago     3h   count:5
node-exporter  ?:9100           3/3  9m ago     3h   *
prometheus     ?:9095           1/1  9m ago     3h   count:1
```

Step 3 Initialize the Ceph cluster.

Disable the automatic extension function of the MON daemon.

```
[root@ceph01 ~]# ceph orch apply mon --unmanaged=true
Scheduled mon update...
```

Once disabled, the value of **PLACEMENT** corresponding to **mon** changes to **unmanaged**. See the following figure.

```
[ceph: root@ceph01 /]# ceph orch ls
NAME           PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager   ?:9093,9094      1/1  35s ago    6m   count:1
crash                           3/3  97s ago    6m   *
grafana        ?:3000           1/1  35s ago    6m   count:1
mgr                             2/2  36s ago    16s  count:2
mon                             3/5  97s ago    38s  <unmanaged>
node-exporter  ?:9100           3/3  97s ago    6m   *
prometheus     ?:9095           1/1  35s ago    6m   count:1
```

Add the **_admin** label to ceph02 and ceph03.

```
[root@ceph01 ~]# ceph orch host label add ceph02.novalocal _admin
Added label _admin to host ceph02.novalocal
[root@ceph01 ~]# ceph orch host label add ceph03.novalocal _admin
Added label _admin to host ceph03.novalocal
```

Deploy the MON and MGR components on the specified node.

```
[root@ceph01 ~]# ceph orch apply mon --placement="label:_admin"
Scheduled mon update...
[root@ceph01 ~]# ceph orch apply mgr --placement="label:_admin"
Scheduled mgr update...
```

```
[root@ceph01 ~]# ceph orch host ls
HOST               ADDR           LABELS  STATUS
ceph01.novalocal   192.168.0.21   _admin
ceph02.novalocal   192.168.0.22   _admin
ceph03.novalocal   192.168.0.23   _admin
3 hosts in cluster
[root@ceph01 ~]# ceph orch ls
NAME            PORTS         RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager    ?:9093,9094       1/1  6s ago     4h   count:1
crash                             3/3  4m ago     4h   *
grafana         ?:3000            1/1  6s ago     4h   count:1
mgr                               3/3  4m ago     18s  label:_admin
mon                               3/3  4m ago     27s  label:_admin
node-exporter   ?:9100            3/3  4m ago     4h   *
prometheus      ?:9095            1/1  7s ago     4h   count:1
```

Step 4 Add OSDs.

Add the hard disks on all hosts as OSDs.

```
ceph orch apply osd --all-available-devices
```

Run **ceph orch ps** to display the components on all hosts.

```
[root@ceph01 ~]# ceph orch ps
NAME                          HOST              PORTS        STATUS          REFRESHED  AGE  MEM USE  MEM LIM  VERSION  IMAGE ID      CONTAINER ID
alertmanager.ceph01           ceph01.novalocal  *:9093,9094  running (102s)    42s ago    4h    15.3M        -  0.23.0   ba2b418f427c  31bd1fe1167a
crash.ceph01                  ceph01.novalocal               running (3h)      42s ago    4h    13.2M        -  16.2.13  e08a45948779  909db008b2aa
crash.ceph02                  ceph02.novalocal               running (3h)      42s ago    3h    6643k        -  16.2.13  e08a45948779  6cdb125e4873
crash.ceph03                  ceph03.novalocal               running (3h)      43s ago    3h    6651k        -  16.2.13  e08a45948779  1323edd41ee7
grafana.ceph01                ceph01.novalocal  *:3000       running (77m)     42s ago   78m    57.2M        -  8.3.5    dad864ee21e9  bdc9d3e5827a
mgr.ceph01.novalocal.pmrwyv   ceph01.novalocal  *:9283       running (3h)      42s ago    4h     522M        -  16.2.13  e08a45948779  878cda687e90
mgr.ceph02.bmmlba             ceph02.novalocal  *:8443,9283  running (109s)    42s ago  109s     390M        -  16.2.13  e08a45948779  4cf9c13dc38b
mgr.ceph03.zoarwo             ceph03.novalocal  *:8443       running (78m)     43s ago   78m     393M        -  16.2.13  e08a45948779  6260ed083108
mon.ceph01.novalocal          ceph01.novalocal               running (3h)      42s ago    4h     603M    2048M  16.2.13  e08a45948779  f8ebff2838ff
mon.ceph02                    ceph02.novalocal               running (78m)     42s ago   78m     185M    2048M  16.2.13  e08a45948779  f3a4ba55ec83
mon.ceph03                    ceph03.novalocal               running (78m)     43s ago   78m     184M    2048M  16.2.13  e08a45948779  4a242b7b98f7
node-exporter.ceph01          ceph01.novalocal  *:9100       running (78m)     42s ago   78m    13.1M        -  1.3.1    1dbe0e931976  4abee19ee088
node-exporter.ceph02          ceph02.novalocal  *:9100       running (78m)     42s ago   78m    13.1M        -  1.3.1    1dbe0e931976  5f9b332d1355
node-exporter.ceph03          ceph03.novalocal  *:9100       running (78m)     43s ago   78m    13.1M        -  1.3.1    1dbe0e931976  6ddd0d1ce149
osd.0                         ceph02.novalocal               running (54s)     42s ago   54s    23.7M    4096M  16.2.13  e08a45948779  febb2d85a36a
osd.1                         ceph03.novalocal               running (54s)     43s ago   54s    23.6M    4096M  16.2.13  e08a45948779  0798455f9547
osd.2                         ceph01.novalocal               running (53s)     42s ago   53s    31.5M    4096M  16.2.13  e08a45948779  596c1777a609
osd.3                         ceph02.novalocal               running (51s)     42s ago   51s    26.3M    4096M  16.2.13  e08a45948779  e8f335ee6ffe
osd.4                         ceph03.novalocal               running (51s)     43s ago   51s    25.3M    4096M  16.2.13  e08a45948779  df7c5c058c64
osd.5                         ceph01.novalocal               running (50s)     42s ago   50s    22.7M    4096M  16.2.13  e08a45948779  af7b24d9801c
osd.6                         ceph02.novalocal               running (48s)     42s ago   48s    19.2M    4096M  16.2.13  e08a45948779  7d2381c3943f
osd.7                         ceph03.novalocal               running (48s)     43s ago   48s    21.8M    4096M  16.2.13  e08a45948779  2a26a7dc30c9
osd.8                         ceph01.novalocal               running (47s)     42s ago   47s    18.5M    4096M  16.2.13  e08a45948779  41f93021e10c
prometheus.ceph02             ceph02.novalocal  *:9095       running (101s)    42s ago   77m    37.3M        -  2.33.4   514e6a882f6e  64981a041e95
```

1.4 Other Common Ceph Operations

1.4.1 Removing Services

Run **ceph orch daemon rm** *<names>* to remove a specified service. The following commands remove all services except OSD from ceph02:

Filter out the services on ceph02:

```
ceph orch ps | grep ceph02
```

```
[root@ceph01 ~]# ceph orch ps | grep ceph02
crash.ceph02                  ceph02.novalocal               running (3h)      7m ago   3h    6643k        -  16.2.13  e08a45948779  6cdb125e4873
mgr.ceph02.bmmlba             ceph02.novalocal  *:8443,9283  running (10m)     7m ago  10m     390M        -  16.2.13  e08a45948779  4cf9c13dc38b
mon.ceph02                    ceph02.novalocal               running (87m)     7m ago  87m     187M    2048M  16.2.13  e08a45948779  f3a4ba55ec83
node-exporter.ceph02          ceph02.novalocal  *:9100       running (87m)     7m ago  87m    13.1M        -  1.3.1    1dbe0e931976  5f9b332d1355
osd.0                         ceph02.novalocal               running (9m)      7m ago   9m    25.4M    4096M  16.2.13  e08a45948779  febb2d85a36a
osd.3                         ceph02.novalocal               running (9m)      7m ago   9m    27.7M    4096M  16.2.13  e08a45948779  e8f335ee6ffe
osd.6                         ceph02.novalocal               running (9m)      7m ago   9m    25.2M    4096M  16.2.13  e08a45948779  7d2381c3943f
prometheus.ceph02             ceph02.novalocal  *:9095       running (10m)     7m ago  86m    55.4M        -  2.33.4   514e6a882f6e  64981a041e95
```

Delete the **ceph orch apply** *<name>* **--unmanaged=true** commands to disable the automatic expansion of cluster components.

```
ceph orch apply crash --unmanaged=true
ceph orch apply mon --unmanaged=true
ceph orch apply mgr --unmanaged=true
ceph orch apply node-exporter --unmanaged=true
ceph orch apply prometheus --unmanaged=true
```

```
[root@ceph01 ~]# ceph orch apply crash --unmanaged=true
Scheduled crash update...
[root@ceph01 ~]# ceph orch apply mon --unmanaged=true
Scheduled mon update...
[root@ceph01 ~]# ceph orch apply mgr --unmanaged=true
Scheduled mgr update...
```

Run the following commands to delete specified services:

```
ceph orch daemon rm crash.ceph02
ceph orch daemon rm mgr.ceph02.bmmlba
ceph orch daemon rm mon.ceph02 --force
```

```
[root@ceph01 ~]# ceph orch daemon rm crash.ceph02
Removed crash.ceph02 from host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch daemon rm mgr.ceph02.bmmlba
Removed mgr.ceph02.bmmlba from host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch daemon rm mon.ceph02 --force
Removed mon.ceph02 from host 'ceph02.novalocal'
```

Note: If ceph02 is running OSD in addition to other services, stop the other services before performing the next step.

1.4.2 Removing OSDs

Run the following commands to stop OSDs on ceph02:

```
ceph orch daemon stop osd.0
ceph orch daemon stop osd.3
ceph orch daemon stop osd.6
```

```
[root@ceph01 ~]# ceph orch daemon stop osd.0
Scheduled to stop osd.0 on host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch daemon stop osd.3
Scheduled to stop osd.3 on host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch daemon stop osd.6
Scheduled to stop osd.6 on host 'ceph02.novalocal'
```

After all OSDs are stopped, run the following commands to remove them:

```
ceph osd rm 0
ceph osd rm 3
ceph osd rm 6
```

```
[root@ceph01 ~]# ceph osd rm 0
removed osd.0
[root@ceph01 ~]# ceph osd rm 3
removed osd.3
[root@ceph01 ~]# ceph osd rm 6
removed osd.6
```

Then, remove the data on the corresponding disks.

```
ceph orch device zap ceph02.novalocal /dev/sdb --force
ceph orch device zap ceph02.novalocal /dev/sdc --force
ceph orch device zap ceph02.novalocal /dev/sdd --force
```

```
[root@ceph01 ~]# ceph orch device zap ceph02.novalocal /dev/sdb --force
zap successful for /dev/sdb on ceph02.novalocal
[root@ceph01 ~]# ceph orch device zap ceph02.novalocal /dev/sdc --force
zap successful for /dev/sdc on ceph02.novalocal
[root@ceph01 ~]# ceph orch device zap ceph02.novalocal /dev/sdd --force
zap successful for /dev/sdd on ceph02.novalocal
```

Delete the OSD mapping in CRUSH.

```
ceph osd crush rm osd.0
ceph osd crush rm osd.3
ceph osd crush rm osd.6
```

```
[root@ceph01 ~]# ceph osd crush rm osd.0
removed item id 0 name 'osd.0' from crush map
[root@ceph01 ~]# ceph osd crush rm osd.3
removed item id 3 name 'osd.3' from crush map
[root@ceph01 ~]# ceph osd crush rm osd.6
removed item id 6 name 'osd.6' from crush map
```

After the OSD mapping is deleted from CRUSH, run the following command to disable the automatic OSD deployment:

```
ceph orch apply osd --all-available-devices --unmanaged=true
```

```
[ceph: root@ceph01 /]# ceph orch apply osd --all-available-devices --unmanaged=true
Scheduled osd.all-available-devices update...
[ceph: root@ceph01 /]# ceph orch ls
NAME                       PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager               ?:9093,9094      1/1  8m ago     3h   count:1
crash                                       2/3  10m ago    10m  <unmanaged>
grafana                    ?:3000           1/1  8m ago     3h   count:1
mgr                                         2/2  10m ago    10m  <unmanaged>
mon                                         2/5  10m ago    10m  <unmanaged>
node-exporter              ?:9100           3/3  10m ago    3h   *
osd.all-available-devices                     6  10m ago    11s  <unmanaged>
prometheus                 ?:9095           1/1  57s ago    3h   count:1
```

1.4.3 Removing a Node

Run the following command to remove the ceph02 node:

```
ceph orch host rm ceph02.novalocal
```

```
[ceph: root@ceph01 /]# ceph orch host rm ceph02.novalocal
Removed  host 'ceph02.novalocal'
```

Enable automatic deployment for all services.

```
ceph orch apply mon --unmanaged=false
ceph orch apply mgr --unmanaged=false
ceph orch apply node-exporter --unmanaged=false
ceph orch apply crash --unmanaged=false
ceph orch apply prometheus --unmanaged=false
```

```
[ceph: root@ceph01 /]# ceph orch ls
NAME                       PORTS        RUNNING  REFRESHED  AGE   PLACEMENT
alertmanager               ?:9093,9094      1/1  42m ago    3h    count:1
crash                                       2/2  42m ago    81s   *
grafana                    ?:3000           1/1  42m ago    3h    count:1
mgr                                         2/2  42m ago    113s  count:2
mon                                         2/5  42m ago    2m    count:5
node-exporter              ?:9100           2/2  42m ago    99s   *
osd.all-available-devices                     6  42m ago    33m   <unmanaged>
prometheus                 ?:9095           0/1  -          4m    count:1
```

1.4.4 Manually Deploying a Service

Refer to Step 2 in section 1.3 to add ceph02 back to the cluster. The details are as follows:

```
[root@ceph01 ~]# ceph orch host add ceph02.novalocal
Added host 'ceph02.novalocal' with addr '192.168.0.22'
[root@ceph01 ~]# ceph orch host ls
HOST              ADDR          LABELS  STATUS
ceph01.novalocal  192.168.0.21  _admin
ceph02.novalocal  192.168.0.22
ceph03.novalocal  192.168.0.23  _admin
3 hosts in cluster
```

Add the **_admin** label to the new node.

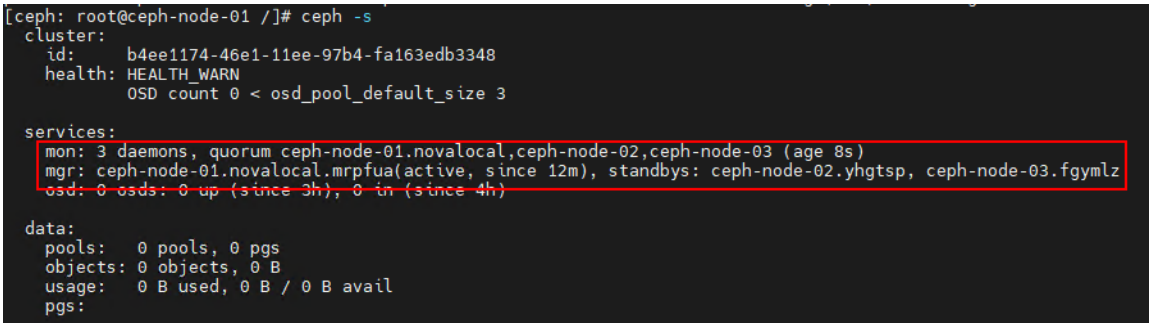
```
ceph orch   host   label add ceph02.novalocal _admin
```

```
[root@ceph01 ~]# ceph orch daemon add node-exporter --placement=ceph02.novalocal
Deployed node-exporter.ceph02 on host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch daemon add crash --placement=ceph02.novalocal
Deployed crash.ceph02 on host 'ceph02.novalocal'
[root@ceph01 ~]# ceph orch  host  label add ceph02.novalocal _admin
Added label _admin to host ceph02.novalocal
```

```
[root@ceph01 ~]# ceph orch host ls
HOST              ADDR          LABELS  STATUS
ceph01.novalocal  192.168.0.21  _admin
ceph02.novalocal  192.168.0.22  _admin
ceph03.novalocal  192.168.0.23  _admin
3 hosts in cluster
```

Deploy management-related components on the host labeled **_admin**.

```
ceph orch apply mgr --placement="label:_admin"
ceph orch apply mon --placement="label:_admin"
```



Manually add the hard disk of ceph02 as an OSD.

```
[ceph: root@ceph01 /]# ceph orch daemon add osd ceph02.novalocal:/dev/sdb
Created osd(s) 2 on host 'ceph02.novalocal'
[ceph: root@ceph01 /]# ceph orch daemon add osd ceph02.novalocal:/dev/sdc
Created osd(s) 5 on host 'ceph02.novalocal'
[ceph: root@ceph01 /]# ceph orch daemon add osd ceph02.novalocal:/dev/sdd
Created osd(s) 8 on host 'ceph02.novalocal'
```

2 Managing Resource Pools

2.1 Introduction

2.1.1 About This Exercise

This exercise gives trainees an opportunity to perform preliminary operations, such as creating, deleting, and configuring resource pools, based on the Ceph cluster created in the previous exercise, and create snapshots for data recovery.

2.1.2 Objectives

This exercise corresponds to the Ceph Resource Pool Management slides. Here, you will practice creating, viewing, and configuring replicated pools and erasure-coded pools, and creating resource pool snapshots to restore pool data.

2.2 Creating and Configuring a Replicated Pool

This section covers basic resource pool management, and operations include: create and rename a resource pool, modify its configurations, upload and download objects, and create snapshots.

Step 1 Create a replicated pool named test.

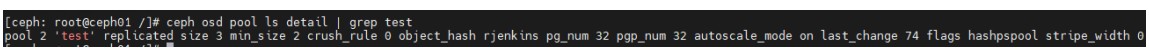
Create the replicated pool test and retain the default settings.

```
[ceph: root@ceph01 /]# ceph osd pool create test
pool 'test' created
```



After the resource pool is created, query the pool information.

```
[ceph: root@ceph01 /]# ceph osd pool ls detail | grep test
pool 2 'test' replicated size 3 min_size 2 crush_rule 0 object_hash rjenkins pg_num 32 pgp_num 32
autoscale_mode on last_change 74 flags hashpspool stripe_width 0
```



Or, run the following command to query more detailed information about the resource pool:

```
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 32
pgp_num: 32
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

Step 2 Modify a resource pool.

According to Ceph suggestions, the recommended number of PGs is 256. Run the following command to change the number of PGs to **256**:

```
[ceph: root@ceph01 /]# ceph osd pool set test pg_num 256
set pool 2 pg_num to 256
```

Run the **ceph osd pool get test all** command multiple times to view the adjustment process of **pg_num** and **pgp_num**.

```
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 99
pgp_num: 93
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

After a period of time, the number of PGs is adjusted.

```
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 256
pgp_num: 256
```

```
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

Question: When adjusting the value of **pg_num**, should **pgp_num** or **pg_num** be adjusted first? And Why?

Answer: **pgp_num** should take priority, because it is the permutation and combination of OSDs. PGs can find the corresponding OSD for data storage only after **pgp_num** is adjusted.

Question: Will **pg_num** remain unchanged after it is changed to **256**?

Answer: Because **pg_autoscale_mode** is enabled, the Ceph cluster automatically changes **pg_num** to an appropriate value based on the amount of data stored in the resource pool.

Step 3 Change the value of **min_size**.

The default value of **min_size** is **2**, indicating that data can be read and written only when at least two replicas are available. This ensures data consistency but reduces data availability. Run the following command to change the value to **1**:

```
[ceph: root@ceph01 /]# ceph osd pool set test min_size 1
set pool 2 min_size to 1
```

```
[ceph: root@ceph01 /]# ceph osd pool set test min_size 1
set pool 2 min_size to 1
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 1
pg_num: 121
pgp_num: 118
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

Step 4 Rename the resource pool.

Rename the resource pool test to test_rbd.

```
[ceph: root@ceph01 /]# ceph osd pool rename test test_rbd
pool 'test' renamed to 'test_rbd'
```

```
[ceph: root@ceph01 /]# ceph osd pool rename test test_rbd
pool 'test' renamed to 'test_rbd'
[ceph: root@ceph01 /]# ceph osd pool ls
device_health_metrics
test_rbd
```

Step 5 Use RADOS commands to perform simple operations on objects.

Upload the **/etc/hosts** file to test_rbd.

```
[ceph: root@ceph01 /]# rados -p test_rbd put hosts /etc/hosts
```

Next, run the following command to query the existing objects in the resource pool:

```
[ceph: root@ceph01 /]# rados -p test_rbd ls
hosts
```

```
[ceph: root@ceph01 /]# rados -p test_rbd put hosts /etc/hosts
[ceph: root@ceph01 /]# rados -p test_rbd ls
hosts
```

Step 6 (Optional) Query the disk where the uploaded object resides.

Run the following commands to view the OSD mapped to the hosts object:

```
[ceph: root@ceph01 /]# ceph osd map test_rbd hosts
osdmap e3478 pool 'test_rbd' (2) object 'hosts' -> pg 2.ea1b298e (2.e) -> up ([4,2,3], p4) acting
([4,2,3], p4)
```

In the command output, **[4,2,3]** indicates the OSD ID. You can view the corresponding hard disk based on the OSD ID.

```
[ceph: root@ceph01 /]# ceph device ls
```

```
[ceph: root@ceph01 /]# ceph device ls
DEVICE                                                HOST:DEV              DAEMONS              WEAR  LIFE EXPECTANC
Huawei_VBS_fileIO_dsw347393408612973612334 15  ceph03.novalocal:sda  mon.ceph03
Huawei_VBS_fileIO_dsw347393408612973612334 18  ceph01.novalocal:sda  mon.ceph01.novalocal
Huawei_VBS_fileIO_dsw347393408612973612334 20  ceph02.novalocal:sda  mon.ceph02
Huawei_VBS_fileIO_dsw347393408612973612334 25  ceph01.novalocal:sdb  osd.0
Huawei_VBS_fileIO_dsw347393408612973612334 26  ceph02.novalocal:sdb  osd.2
Huawei_VBS_fileIO_dsw347393408612973612334 27  ceph03.novalocal:sdb  osd.1
Huawei_VBS_fileIO_dsw347393408612973612334 28  ceph01.novalocal:sdc  osd.3
Huawei_VBS_fileIO_dsw347393408612973612334 29  ceph03.novalocal:sdc  osd.4
Huawei_VBS_fileIO_dsw347393408612973612334 30  ceph02.novalocal:sdc  osd.5
Huawei_VBS_fileIO_dsw347393408612973612334 31  ceph03.novalocal:sdd  osd.7
Huawei_VBS_fileIO_dsw347393408612973612334 32  ceph02.novalocal:sdd  osd.8
Huawei_VBS_fileIO_dsw347393408612973612334 33  ceph01.novalocal:sdd  osd.6
```

Step 7 Create a snapshot for the resource pool.

Run the following commands to create snapshot snapshot1 for resource pool test_rbd:

```
[ceph: root@ceph01 /]# ceph osd pool mksnap test_rbd snapshot1
created pool test_rbd snap snapshot1
```

```
[ceph: root@ceph01 /]# ceph osd pool mksnap test_rbd snapshot1
created pool test_rbd snap snapshot1
[ceph: root@ceph01 /]# rados -p test_rbd lssnap
1       snapshot1
1 snaps
```

Step 8 Delete objects.

Run the following command to delete the object:

```
[ceph: root@ceph01 /]# rados -p test_rbd rm hosts
[ceph: root@ceph01 /]# rados -p test_rbd ls
hosts
[ceph: root@ceph01 /]# rados -p test_rbd rm hosts
error removing test_rbd>hosts: (2) No such file or directory
```

Question: Why the hosts object is still displayed in the **ls** command output, even after it is deleted?

Answer: After the hosts object is deleted, Ceph does not delete it immediately. Instead, Ceph marks it as "deleted". The hosts object stays in the **ls** command output, but it cannot be read or written.

Step 9 Restore an object using a snapshot.

Run the following command to check if there is a corresponding object in the snapshot:

```
[ceph: root@ceph01 /]# rados -p test_rbd -s snapshot1 ls
```

```
[ceph: root@ceph01 /]# rados -p test_rbd -s snapshot1 ls
selected snap 3 'snapshot1'
hosts
```

If there is, download the object from the snapshot to the local host and name it hosts-obj:

```
[ceph: root@ceph01 /]# rados -p test_rbd -s snapshot1 get hosts hosts-obj
selected snap 3 'snapshot1'
```

```
[ceph: root@ceph01 /]# rados -p test_rbd -s snapshot1 get hosts hosts-obj
selected snap 3 'snapshot1'
[ceph: root@ceph01 /]# ls
bin  dev  etc  home  hosts-obj  lib  lib64  lost+found  media  mnt  opt  pro
```

Run the RADOS **put** command to upload the object to the resource pool and name the object hosts.

```
[ceph: root@ceph01 /]# rados -p test_rbd put hosts /hosts-obj
```

```
[ceph: root@ceph01 /]# rados -p test_rbd put hosts /hosts-obj
[ceph: root@ceph01 /]# rados -p test_rbd ls
hosts
```

Run the RADOS **get** command to download object hosts and name it hosts_test. Confirm the object is restored.

```
[ceph: root@ceph01 /]# rados -p test_rbd get hosts hosts_test
```

```
[ceph: root@ceph01 /]# rados -p test_rbd get hosts hosts_test
[ceph: root@ceph01 /]# ls
bin  dev  etc  home  hosts-obj  hosts test  lib  lib64  lost+fo
```

Step 10 Set resource pool quotas.

Set the maximum number of objects that can be uploaded to test_rbd to **5**.

```
[ceph: root@ceph01 /]# ceph osd pool set-quota test_rbd max_objects 5
set-quota max_objects = 5 for pool test_rbd
```

Upload three more objects to the test_rbd resource pool.

```
[ceph: root@ceph01 /]#  ceph osd pool set-quota test_rbd max_objects 5
set-quota max_objects = 5 for pool test_rbd
[ceph: root@ceph01 /]# rados -p test_rbd put hosts1 /etc/hosts
[ceph: root@ceph01 /]# rados -p test_rbd put hosts2 /etc/hosts
[ceph: root@ceph01 /]# rados -p test_rbd put hosts3 /etc/hosts
[ceph: root@ceph01 /]# rados -p test_rbd put hosts4 /etc/hosts
[ceph: root@ceph01 /]# rados -p test_rbd put hosts5 /etc/hosts
```

When the sixth object is uploaded, the system is in the uploading state, but will report the following error:

```
[ceph: root@ceph01 /]# ceph health detail
HEALTH_WARN 1 pool(s) do not have an application enabled; 1 pool(s) full
[WRN] POOL_APP_NOT_ENABLED: 1 pool(s) do not have an application enabled
    application not enabled on pool 'test_rbd'
    use 'ceph osd pool application enable <pool-name> <app-name>', where <app-na
[WRN] POOL_FULL: 1 pool(s) full
    pool 'test_rbd' is full (running out of quota)
```

Step 11 Delete a resource pool.

Enable the option of deleting resource pools.

```
[ceph: root@ceph01 /]# ceph config set mon mon_allow_pool_delete true
```

```
[ceph: root@ceph01 /]# ceph config set mon mon_allow_pool_delete true
[ceph: root@ceph01 /]# ceph config get mon mon_allow_pool_delete
true
```

Delete the test_rbd resource pool.

```
[ceph: root@ceph01 /]# ceph osd pool rm test_rbd test_rbd --yes-i-really-really-mean-it
```

```
[ceph: root@ceph01 /]# ceph osd pool rm test_rbd test_rbd --yes-i-really-really-mean-it
pool 'test_rbd' removed
[ceph: root@ceph01 /]# ceph osd pool ls
device_health_metrics
.rgw.root
default.rgw.log
default.rgw.control
default.rgw.meta
```

2.3 Creating an Erasure-coded Pool

Create an erasure-coded pool object_test and set the number of data chunks to **4** and the number of coding chunks to **2**.

Step 1 Create the erasure coding rule for the pool.

Create an erasure coding rule test and set the number of data chunks to **4** and the number of coding chunks to **2**. To avoid errors, you are advised to set the failure domain to **osd** in this lab environment. The specific command is as follows:

```
[ceph: root@ceph01 /]# ceph osd erasure-code-profile set test k=4 m=2 crush-failure-domain=osd
```

```
[ceph: root@ceph01 /]# ceph osd erasure-code-profile get test
crush-device-class=
crush-failure-domain=osd
crush-root=default
jerasure-per-chunk-alignment=false
k=4
m=2
plugin=jerasure
technique=reed_sol_van
w=8
```

Step 2 Create the erasure-coded pool.

Create the pool using the created erasure coding rule.

```
[ceph: root@ceph01 /]# ceph osd pool create object_test erasure test
```

```
[ceph: root@ceph01 /]# ceph osd pool create object_test erasure test
pool 'object_test' created
[ceph: root@ceph01 /]# ceph osd pool get object_test all
size: 6
min_size: 5
pg_num: 32
pgp_num: 32
crush_rule: object_test
hashpspool: true
allow_ec_overwrites: false
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
erasure_code_profile: test
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

Step 3 Set the resource pool type.

Set the type of the created resource pool to **rgw**.

```
[ceph: root@ceph01 /]# ceph osd pool application enable object_test rgw
```

```
[ceph: root@ceph01 /]# ceph osd pool application enable object_test rgw
enabled application 'rgw' on pool 'object_test'
```

3 Practicing User Management in Ceph

3.1 Introduction

3.1.1 About This Exercise

All operations on the Ceph cluster in this section until the end of this lab guide are to be performed on the client. In this exercise, the hcie_openeuler user will be created and permissions will be configured so that the client can manage the Ceph cluster.

3.1.2 Objectives

This exercise corresponds to the Ceph Cluster Configuration and User Authorization slides. Here you will practice creating users, assigning permissions, and authenticating related files to enhance your understanding of the slides.

3.2 Creating a User and Granting Permissions

Step 1 Create a user.

Run the following command to create the hcie_openeuler user:

```
[ceph: root@ceph01 /]# ceph auth add client.hcie_openeuler
```

```
[ceph: root@ceph01 /]# ceph auth add client.hcie_openeuler
added key for client.hcie_openeuler
[ceph: root@ceph01 /]# ceph auth get client.hcie_openeuler
[client.hcie_openeuler]
        key = AQCbAw1lFidyBxAA5EVSJLb/6kneMAiKAb+QhA==
exported keyring for client.hcie_openeuler
```

Step 2 Grant permissions to the created user.

The hcie_openeuler user needs to have the permission to manage the cluster. Run the following command to grant the MON read and write permissions to the user:

```
[ceph: root@ceph01 /]# ceph auth caps client.hcie_openeuler mon 'allow rw'
```

```
[ceph: root@ceph01 /]# ceph auth caps client.hcie_openeuler mon 'allow rw'
updated caps for client.hcie_openeuler
[ceph: root@ceph01 /]# ceph auth get client.hcie_openeuler
[client.hcie_openeuler]
        key = AQCbAw1lFidyBxAA5EVSJLb/6kneMAiKAb+QhA==
        caps mon = "allow rw"
exported keyring for client.hcie_openeuler
```

Step 3 Install the client software.

Run the following command to install the client software on the client:

```
[root@client ~]# yum install -y ceph-2:16.2.7-14.oe2203.x86_64
```

Step 4 Export authentication file.

Export the keyring of the hcie_openeuler user and copy it to the client host.

```
[ceph: root@ceph01 /]# ceph auth get client.hcie_openeuler -o client.hcie_openeuler.keyring
[ceph: root@ceph01 /]# scp client.hcie_openeuler.keyring root@192.168.0.24:/etc/ceph/
```

Copy the configuration file of the Ceph cluster to the client host.

```
[ceph: root@ceph01 /]# scp /etc/ceph/ceph.conf root@192.168.0.24:/etc/ceph/
```

```
[ceph: root@ceph01 /]# ceph auth get client.hcie_openeuler -o client.hcie_openeuler.keyring
exported keyring for client.hcie_openeuler
[ceph: root@ceph01 /]# scp client.hcie_openeuler.keyring root@192.168.0.24:/etc/ceph/
The authenticity of host '192.168.0.24 (192.168.0.24)' can't be established.
ECDSA key fingerprint is SHA256:pGnGv3qj7BeVXZe+Cn4mz9Zh6Rb7Ztkk5+CplD6/PyY.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.24' (ECDSA) to the list of known hosts.

Authorized users only. All activities may be monitored and reported.
root@192.168.0.24's password:
client.hcie_openeuler.keyring
[ceph: root@ceph01 /]# scp /etc/ceph/ceph.conf root@192.168.0.24:/etc/ceph/

Authorized users only. All activities may be monitored and reported.
root@192.168.0.24's password:
ceph.conf
```

Step 5 Perform a test.

Run the following command on the client to check whether the Ceph cluster information can be viewed:

```
[root@client ~]# ceph -s --id hcie_openeuler --keyring /etc/ceph/client.hcie_openeuler.keyring
```

```
[root@client ~]# ceph -s --id hcie_openeuler --keyring /etc/ceph/
ceph.conf                    client.hcie_openeuler.keyring  rbdmap
[root@client ~]# ceph -s --id hcie_openeuler --keyring /etc/ceph/client.hcie_openeuler.keyring
  cluster:
    id:     d5faf9ba-56c0-11ee-9ffe-fa163edb3348
    health: HEALTH_OK

  services:
    mon: 3 daemons, quorum ceph01.novalocal,ceph03,ceph02 (age 90m)
    mgr: ceph01.novalocal.udalan(active, since 43m), standbys: ceph03.uessau, ceph02.puzvmw
    osd: 9 osds: 9 up (since 90m), 9 in (since 2d)

  data:
    pools:   6 pools, 161 pgs
    objects: 26 objects, 1.5 KiB
    usage:   407 MiB used, 90 GiB / 90 GiB avail
    pgs:     161 active+clean
```

- Question 1: Why do you need to specify a user name when performing operations on the client?
- Answer: If a user name is not specified, by default the system uses admin to access the cluster, and because there is no authentication file for the current client, the system will report an error.

```
[root@client ~]# ceph -s
2024-02-22T03:16:48.960+0000 7f01b14e2640 -1 auth: unable to find a keyring on /etc/ceph/ceph.client.admin.keyring,/etc/ceph/ceph.keyring,/
2024-02-22T03:16:48.960+0000 7f01b14e2640 -1 AuthRegistry(0x7f01ac05eaa0) no keyring found at /etc/ceph/ceph.client.admin.keyring,/etc/ceph
2024-02-22T03:16:48.960+0000 7f01b14e2640 -1 auth: unable to find a keyring on /etc/ceph/ceph.client.admin.keyring,/etc/ceph/ceph.keyring,/
2024-02-22T03:16:48.960+0000 7f01b14e2640 -1 AuthRegistry(0x7f01b14e10d0) no keyring found at /etc/ceph/ceph.client.admin.keyring,/etc/ceph
2024-02-22T03:16:48.961+0000 7f01ab7fe640 -1 monclient(hunting): handle_auth_bad_method server allowed_methods [2] but i only support [1]
2024-02-22T03:16:48.961+0000 7f01aa7fc640 -1 monclient(hunting): handle_auth_bad_method server allowed_methods [2] but i only support [1]
2024-02-22T03:16:48.961+0000 7f01aaffd640 -1 monclient(hunting): handle_auth_bad_method server allowed_methods [2] but i only support [1]
2024-02-22T03:16:48.961+0000 7f01b14e2640 -1 monclient: authenticate NOTE: no keyring found; disabled cephx authentication
[errno 13] RADOS permission denied (error connecting to the cluster)
```

- Question 2: Can the preceding operations prove that user hcie_openeuler has the permission to manage the Ceph cluster?
- Answer: No. The preceding operations only show that the user has the read permission on the Ceph cluster and can view the Ceph status. The user has the management permission only after Ceph is configured and managed.

4 Practicing Object Storage in Ceph

4.1 Introduction

4.1.1 About This Exercise

In this exercise, Swift is used to interconnect with ceph-rgw to upload and download objects, and multiple sites are configured for DR protection of the object storage.

4.1.2 Objectives

This exercise corresponds to the Ceph Object Storage Gateway RGW slides. Here, you will practice deploying object storage gateways (RGW), uploading and downloading objects, and deploying multisite object gateways to enhance your understanding of the slides.

4.2 Creating an RGW

Create an RGW and define related resources such as realms and zones as for_swift.

Step 1 Create a realm.

Run the following command to create realm for_swift:

```
[ceph: root@ceph01 /]# radosgw-admin realm create --rgw-realm=for_swift --default
```

```
[ceph: root@ceph01 /]# radosgw-admin realm create --rgw-realm=for_swift --default
{
    "id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "name": "for_swift",
    "current_period": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "epoch": 1
}
```

Step 2 Create a zonegroup.

Run the following command to create zonegroup for_swift and set it as the master zonegroup:

```
[ceph: root@ceph01 /]# radosgw-admin zonegroup create --rgw-realm=for_swift --rgw-zonegroup=for_swift --master --default
```

```
[ceph: root@ceph01 /]# radosgw-admin zonegroup create --rgw-realm=for_swift --rgw-zonegroup=for_swift --master --default
{
    "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
    "name": "for_swift",
    "api_name": "for_swift",
    "is_master": "true",
    "endpoints": [],
    "hostnames": [],
    "hostnames_s3website": [],
    "master_zone": "",
    "zones": [],
    "placement_targets": [],
    "default_placement": "",
    "realm_id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "sync_policy": {
        "groups": []
    }
}
```

Step 3 Create a zone.

Run the following command to create realm for_swift:

```
[ceph: root@ceph01 /]# radosgw-admin zone create --rgw-realm=for_swift --rgw-zonegroup=for_swift --rgw-zone=for_swift --master --default
```

```
{
    "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
    "name": "for_swift",
    "domain_root": "for_swift.rgw.meta:root",
    "control_pool": "for_swift.rgw.control",
    "gc_pool": "for_swift.rgw.log:gc",
    "lc_pool": "for_swift.rgw.log:lc",
    "log_pool": "for_swift.rgw.log",
    "intent_log_pool": "for_swift.rgw.log:intent",
    "usage_log_pool": "for_swift.rgw.log:usage",
    "roles_pool": "for_swift.rgw.meta:roles",
    "reshard_pool": "for_swift.rgw.log:reshard",
    "user_keys_pool": "for_swift.rgw.meta:users.keys",
    "user_email_pool": "for_swift.rgw.meta:users.email",
    "user_swift_pool": "for_swift.rgw.meta:users.swift",
    "user_uid_pool": "for_swift.rgw.meta:users.uid",
    "otp_pool": "for_swift.rgw.otp",
    "system_key": {
        "access_key": "",
        "secret_key": ""
    },
    "placement_pools": [
        {
            "key": "default-placement",
            "val": {
                "index_pool": "for_swift.rgw.buckets.index",
                "storage_classes": {
                    "STANDARD": {
                        "data_pool": "for_swift.rgw.buckets.data"
                    }
                },
                "data_extra_pool": "for_swift.rgw.buckets.non-ec",
                "index_type": 0
            }
        }
    ],
    "realm_id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "notif_pool": "for_swift.rgw.log:notif"
}
```

Run the command to ensure the changes are applied.

```
[ceph: root@ceph01 /]# radosgw-admin period update --rgw-realm=for_swift --commit
```

```
[ceph: root@ceph01 /]# radosgw-admin period update --rgw-realm=for_swift --commit
{
    "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 1,
    "predecessor_uuid": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "sync_status": [],
    "period_map": {
        "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
        "zonegroups": [
            {
                "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
                "name": "for_swift",
                "api_name": "for_swift",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                "zones": [
                    {
                        "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                        "name": "for_swift",
                        "endpoints": [],
                        "log_meta": "false",
```

Step 4 Create an RGW.

Create two RGW instances and associate them with for_swift:

```
[ceph: root@ceph01 /]# ceph orch apply rgw for_swift --realm=for_swift --zone=for_swift --placement="2" --port=8080
```

```
[ceph: root@ceph01 /]# ceph orch apply rgw for_swift --realm=for_swift --zone=for_swift --placement="2" --port=8080
Scheduled rgw.for_swift update...
```

Step 5 Create a user.

Run the following command to create the swift user:

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=swift --display-name=swift
```

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=swift --display-name=swift
{
    "user_id": "swift",
    "display_name": "swift",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "swift",
            "access_key": "S3EJN3JBDFYZ0BMW9Z76",
            "secret_key": "jnTI9PFUsdmtkDGinn0j0un0d8cWkzqzAH8ogYEM"
        }
    ],
    "swift_keys": [],
    "caps": [],
```

Create the swift_rgw sub-user:

```
[ceph: root@ceph01 /]# radosgw-admin subuser create --uid=swift --subuser=swift:swift_rgw --access=full
```

```
[ceph: root@ceph01 /]# radosgw-admin subuser create --uid=swift --subuser=swift:swift_rgw --access=full
{
    "user_id": "swift",
    "display_name": "swift",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [
        {
            "id": "swift:swift_rgw",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "swift",
            "access_key": "S3EJN3JBDFYZ0BMW9Z76",
            "secret_key": "jnTI9PFUsdmtkDGinn0j0unOd8cWkzqzAH8ogYEM"
        }
    ],
    "swift_keys": [
        {
            "user": "swift:swift_rgw",
            "secret_key": "gJieBxnbLPuMpHskqLoxWAZBdNlUnJJWyzC0vhwR"
```

Generate a key that can be used by Swift for the created sub-user:

```
[ceph: root@ceph01 /]# radosgw-admin key create --subuser=swift:swift_rgw --key-type=swift --gen-secret
```

```
[ceph: root@ceph01 /]# radosgw-admin key create --subuser=swift:swift_rgw --key-type=swift --gen-secret
{
    "user_id": "swift",
    "display_name": "swift",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [
        {
            "id": "swift:swift_rgw",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "swift",
            "access_key": "S3EJN3JBDFYZ0BMW9Z76",
            "secret_key": "jnTI9PFUsdmtkDGinn0j0unOd8cWkzqzAH8ogYEM"
        }
    ],
    "swift_keys": [
        {
            "user": "swift:swift_rgw",
            "secret_key": "mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7"
        }
```

After the configuration is complete, record the generated key, which will be used for authentication.

4.3 Preparing the Client

Step 1 Install Swift on the client.

Run the following command to install Swift on the client:

```
[root@client ~]# yum install -y python-setuptools python-pip
```

```
[root@client ~]# yum install -y python-setuptools python-pip
Last metadata expiration check: 1:05:21 ago on [redacted]
Package python-setuptools-59.4.0-3.oe2203.noarch is already installed.
Package python3-pip-21.3.1-1.oe2203.noarch is already installed.
Dependencies resolved.
================================================================================
 Package                                                 Architecture
================================================================================
Upgrading:
 python-setuptools                                       noarch
 python3-pip                                             noarch

Transaction Summary
================================================================================
Upgrade  2 Packages

Total download size: 2.7 M
Downloading Packages:
(1/2): python-setuptools-59.4.0-5.oe2203.noarch.rpm
(2/2): python3-pip-21.3.1-2.oe2203.noarch.rpm
--------------------------------------------------------------------------------
```

```
[root@client ~]# pip install --upgrade python-swiftclient
```

```
[root@client ~]# pip install --upgrade python-swiftclient
WARNING: Running pip install with root privileges is generally not a good idea. Try `pip install --user` instead.
Collecting python-swiftclient
  Downloading python_swiftclient-4.4.0-py3-none-any.whl (88 kB)
     |[redacted]| 88 kB 66 kB/s
Requirement already satisfied: requests>=2.4.0 in /usr/lib/python3.9/site-packages (from python-swiftclient) (2.26.0)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/lib/python3.9/site-packages (from requests>=2.4.0->python-swiftclient) (1.26.7)
Requirement already satisfied: charset_normalizer~=2.0.0 in /usr/lib/python3.9/site-packages (from requests>=2.4.0->python-swiftclient) (2.0.8)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3.9/site-packages (from requests>=2.4.0->python-swiftclient) (3.2)
Installing collected packages: python-swiftclient
Successfully installed python-swiftclient-4.4.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recomm
nings/venv
```

Check whether the Swift client is successfully installed or not.

```
[root@client ~]# swift --version
```

```
[root@client ~]# swift --version
python-swiftclient 4.4.0
```

Step 2 Authenticate the user.

Use the user created in Step 5 in section 4.2 to connect to Ceph and run the following command to check whether the user name and key are available:

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K
mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 stat -v --insecure
```

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 stat -v --insecure
                              StorageURL: http://192.168.0.21:8080/swift/v1
                              Auth Token: AUTH_rgwtk0f0000007376966743a7376966745f7267779087d951ac5451b6e0f01365ed851520e5d0c5ed30997797c67f71207a238fca74663883
                                 Account: v1
                              Containers: 0
                                 Objects: 0
                                   Bytes: 0
Objects in policy "default-placement-bytes": 0
  Bytes in policy "default-placement-bytes": 0
   Containers in policy "default-placement": 0
      Objects in policy "default-placement": 0
        Bytes in policy "default-placement": 0
                              X-Timestamp: 1695719264.54063
             X-Account-Bytes-Used-Actual: 0
                            X-Trans-Id: tx0000003d43b7e35caf1ff-0065129f60-16fa8-for_swift
                 X-Openstack-Request-Id: tx0000003d43b7e35caf1ff-0065129f60-16fa8-for_swift
                          Accept-Ranges: bytes
                           Content-Type: text/plain; charset=utf-8
                             Connection: Keep-Alive
```

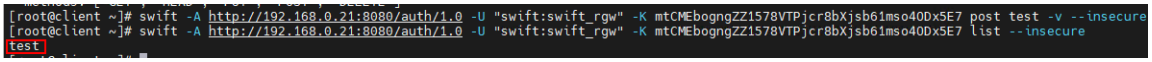
Step 3 Use Swift to perform basic object operations.

Create a container (bucket) test using Swift.

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K
mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 post test --insecure
```

Run the following command to display the new container (bucket):

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 list --insecure
```

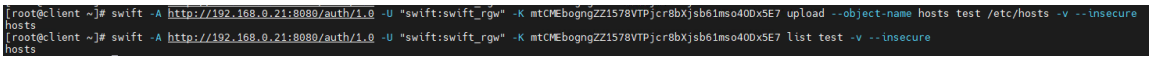


Upload the **/etc/hosts** file on the client to the test container (bucket) and name the file **hosts**:

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name hosts test /etc/hosts -v --insecure
```

Check whether the upload is complete.

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 list test -v --insecure
```



4.4 Limiting the Bucket Quota

Bucket quotas can be configured for users or specified buckets, which will be practiced in this exercise separately.

Step 1 Limit quotas for users.

Enable the quota limit for the swift user.

```
[ceph: root@ceph01 /]# radosgw-admin quota enable --quota-scope=user --uid=swift
```

Set the maximum number of objects that can be uploaded by the swift user to **3**.

```
[ceph: root@ceph01 /]# radosgw-admin quota set --quota-scope=user --uid=swift --max-objects=3
```

Then, check whether the configurations take effect.

```
[ceph: root@ceph01 /]# radosgw-admin user info --uid=swift
```

```
[ceph: root@ceph01 /]# radosgw-admin user info --uid=swift
{
    "user_id": "swift",
    "display_name": "swift",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [
        {
            "id": "swift:swift_rgw",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "swift",
            "access_key": "S3EJN3JBDFYZ0BMW9Z76",
            "secret_key": "jnTI9PFUsdmtkDGinn0j0unOd8cWkzqzAH8ogYEM"
        }
    ],
    "swift_keys": [
        {
            "user": "swift:swift_rgw",
            "secret_key": "mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7"
        }
    ],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": true,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": 3
    },
    "temp_url_keys": [],
    "type": "rgw",
    "mfa_ids": []
}
```

Run the **upload** command on the client to test the quota limit. When the third file is uploaded, the client receives an error message.

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name hosts1 test /etc/hosts --insecure
hosts1
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name hosts2 test /etc/hosts --insecure
hosts2
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name hosts3 test /etc/hosts --insecure
Object PUT failed: http://192.168.0.21:8080/swift/v1/test/hosts3 413 Request Entity Too Large b'QuotaExceeded' (txn: tx00000930feab127b4cc8d-0065138409-196c5-for_swift)
Consider using the --segment-size option to chunk the object
[root@client ~]#
```

To cancel the limit on the object quantity quota, set **--max-objects** to **-1**.

```
[ceph: root@ceph01 /]# radosgw-admin quota set --quota-scope=user --uid=swift --max-objects=-1
```

Then, upload the third object again on the client. The upload is successful. The details are as follows:

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K
mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name hosts3 test /etc/hosts --
insecure
hosts3
```

- Question: In addition to setting the number of objects, are there any other methods for limiting quotas?
- Answer: Yes, the maximum upload capacity can be limited as well. This method is demonstrated in limiting quotas for buckets.

Step 2 Limit quotas for buckets.

Enable the quota limit for the test bucket.

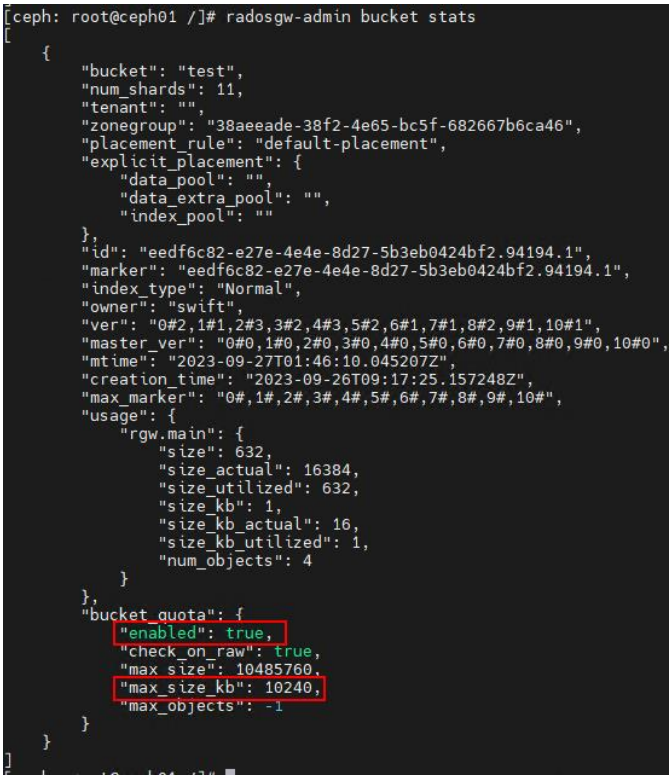
```
[ceph: root@ceph01 /]# radosgw-admin quota enable --quota-scope=bucket --bucket=test
```

Set the maximum capacity of the test bucket to 10 MB.

```
[ceph: root@ceph01 /]# radosgw-admin quota set --quota-scope=bucket --bucket=test --max-size=10M
```

Check the changes have been applied.

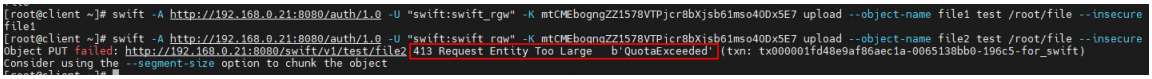
```
[ceph: root@ceph01 /]# radosgw-admin bucket stats
```



Create a 6 MB file on the client to test the quota limit.

```
[root@client ~]# dd if=/dev/zero of=file bs=1M count=6
```

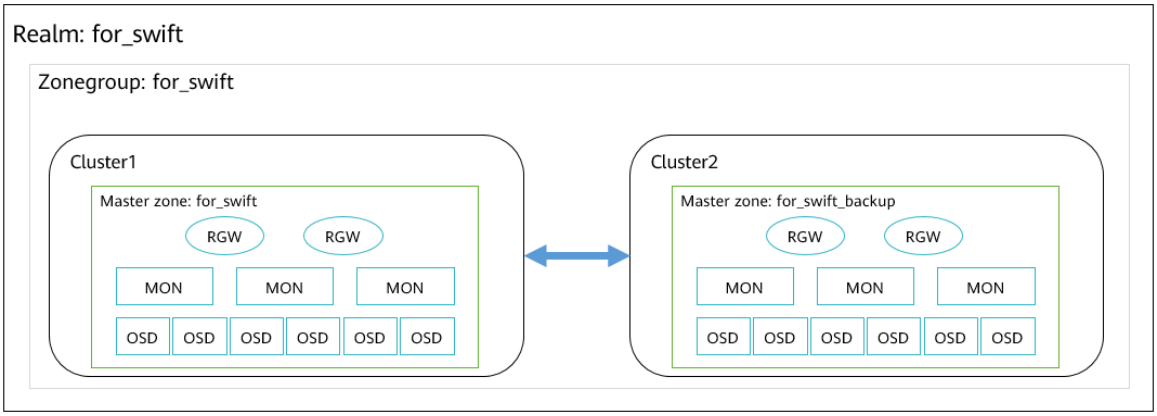
Upload the file to the test container (bucket). When the second file is uploaded, the system returns an error message.



- Task: Design an exercise to cancel the quota limit on the bucket capacity.

4.5 Practicing Multi-Zone Configuration

Upgrade the object storage created in Section 4.2 to the multisite architecture. The following figure shows the architecture.



Step 1 In the master zone, create a system user.

Create a system user for synchronizing data.

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system
```

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system
{
    "user_id": "system-user",
    "display_name": "system-user",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "system-user",
            "access_key": "Huawei@123",
            "secret_key": "Huawei12#$"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "system": "true",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
```

Bind the created system user to the master zone.

```
[ceph: root@ceph01 /]# radosgw-admin zone modify --rgw-realm=for_swift --rgw-zonegroup=for_swift --rgw-zone=for_swift --endpoints 'http://ceph01:8080,http://ceph02:8080' --access-key=Huawei@123 --secret=Huawei12#$ --master --default
```

```
[ceph: root@ceph01 /]# radosgw-admin zone modify --rgw-realm=for_sw

{
    "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
    "name": "for_swift",
    "domain_root": "for_swift.rgw.meta:root",
    "control_pool": "for_swift.rgw.control",
    "gc_pool": "for_swift.rgw.log:gc",
    "lc_pool": "for_swift.rgw.log:lc",
    "log_pool": "for_swift.rgw.log",
    "intent_log_pool": "for_swift.rgw.log:intent",
    "usage_log_pool": "for_swift.rgw.log:usage",
    "roles_pool": "for_swift.rgw.meta:roles",
    "reshard_pool": "for_swift.rgw.log:reshard",
    "user_keys_pool": "for_swift.rgw.meta:users.keys",
    "user_email_pool": "for_swift.rgw.meta:users.email",
    "user_swift_pool": "for_swift.rgw.meta:users.swift",
    "user_uid_pool": "for_swift.rgw.meta:users.uid",
    "otp_pool": "for_swift.rgw.otp",
    "system_key": {
        "access_key": "Huawei@123",
        "secret_key": "Huawei12#$"
    },
    "placement_pools": [
        {
            "key": "default-placement",
            "val": {
                "index_pool": "for_swift.rgw.buckets.index",
                "storage_classes": {
                    "STANDARD": {
                        "data_pool": "for_swift.rgw.buckets.data"
                    }
                },
                "data_extra_pool": "for_swift.rgw.buckets.non-ec",
                "index_type": 0
            }
        }
    ],
    "realm_id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "notif_pool": "for_swift.rgw.log:notif"
}
```

Step 2 In the master zone, configure the RGW.

Run the following command to display the RGW information:

```
[ceph: root@ceph01 /]# ceph orch ps | grep rgw
```

```
[ceph: root@ceph01 /]# ceph orch ps | grep rgw
rgw.for_swift.ceph01.cjavrh  ceph01.novalocal  *:80
rgw.for_swift.ceph02.gfipgo  ceph02.novalocal  *:80
```

Set the RGW information.

```
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zone for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zone for_swift
```

```
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zone for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zone for_swift
```

Step 3 In the master zone, commit the configuration.

Run the following command to commit the configuration:

```
[ceph: root@ceph01 /]# radosgw-admin period update --commit
```

```
[ceph: root@ceph01 /]# radosgw-admin period update --commit
{
    "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 2,
    "predecessor_uuid": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "sync_status": [],
    "period_map": {
        "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
        "zonegroups": [
            {
                "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
                "name": "for_swift",
                "api_name": "for_swift",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                "zones": [
                    {
                        "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                        "name": "for_swift",
                        "endpoints": [
                            "http://ceph01:8080",
                            "http://ceph02:8080"
                        ],
```

Step 4 In the master zone, restart the RGW.

Run the following command to restart the RGW.

```
[ceph: root@ceph01 /]# ceph orch restart rgw.for_swift
```

```
[ceph: root@ceph01 /]# ceph orch restart rgw.for_swift
Scheduled to restart rgw.for_swift.ceph01.cjavrh on host 'ceph01.novalocal'
Scheduled to restart rgw.for_swift.ceph02.gfipgo on host 'ceph02.novalocal'
```

Step 5 Deploy Cluster2.

Apply for another two ECSs with the same configurations as the cluster. Then, install and initialize the Ceph cluster. During host domain name resolution, the hosts file must contain all hosts in Cluster1 and Cluster2. In addition, you need to add information about the hosts in Cluster2 to the hosts file in Cluster1.

Step 6 In the secondary zone, pull the realm configuration of the master zonegroup.

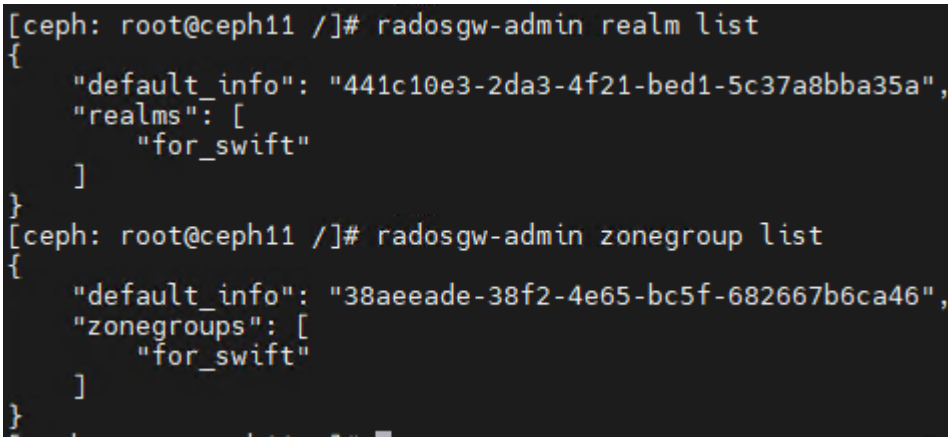
Run the following command to pull the realm configuration information of the master zonegroup:

```
[ceph: root@ceph11 /]# radosgw-admin realm pull --url=http://192.168.0.21:8080 --access-key=Huawei@123 --secret=Huawei12#$
```



Check whether the realm and zonegroup information is obtained.

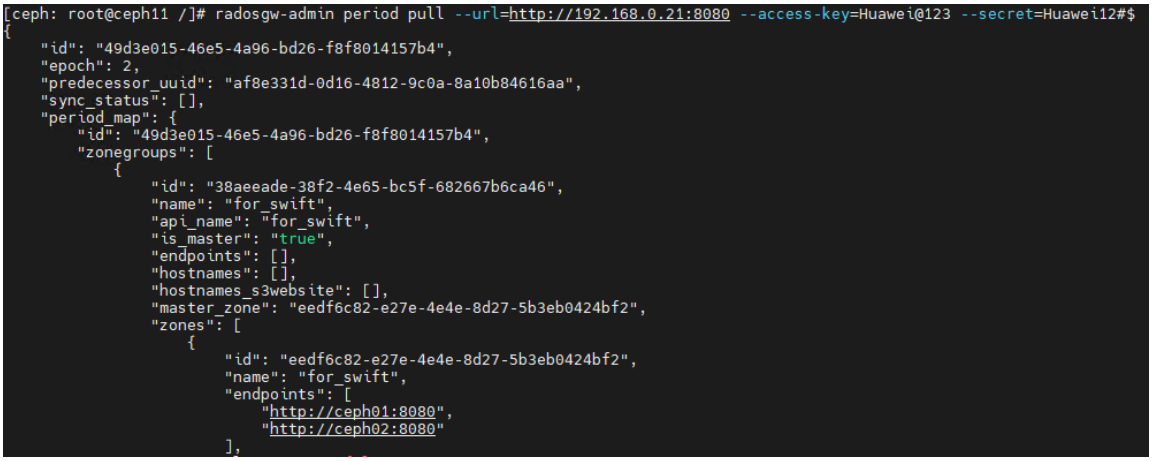
```
[ceph: root@ceph11 /]# radosgw-admin realm list
[ceph: root@ceph11 /]# radosgw-admin zonegroup list
```



Step 7 In the secondary zone, obtain the period.

Run the following command to obtain the period information:

```
[ceph: root@ceph11 /]# radosgw-admin period pull --url=http://192.168.0.21:8080 --access-key=Huawei@123 --secret=Huawei12#$
```



Check whether the obtained information is correct.

```
[ceph: root@ceph11 /]# radosgw-admin period list
```

```
[ceph: root@ceph11 /]# radosgw-admin period list
{
    "periods": [
        "49d3e015-46e5-4a96-bd26-f8f8014157b4"
    ]
}
```

Step 8 In the secondary zone, create a non-master zone.

```
[ceph: root@ceph11 /]# radosgw-admin zone create --rgw-zonegroup=for_swift --rgw-zone=for_swift_backup --endpoints "http://ceph11:8080, http://ceph12:8080" --access-key=Huawei@123 --secret=Huawei12#$
```

```
[ceph: root@ceph11 /]# radosgw-admin zone create --rgw-zonegroup
2023-09-27T09:23:28.782+0000 7f469d25b4c0  0 failed reading obj
2023-09-27T09:23:28.782+0000 7f469d25b4c0  0 WARNING: could not
{
    "id": "ac2f559d-70c1-4d25-92fe-42c96f53aab0",
    "name": "for_swift_backup",
    "domain_root": "for_swift_backup.rgw.meta:root",
    "control_pool": "for_swift_backup.rgw.control",
    "gc_pool": "for_swift_backup.rgw.log:gc",
    "lc_pool": "for_swift_backup.rgw.log:lc",
    "log_pool": "for_swift_backup.rgw.log",
    "intent_log_pool": "for_swift_backup.rgw.log:intent",
    "usage_log_pool": "for_swift_backup.rgw.log:usage",
    "roles_pool": "for_swift_backup.rgw.meta:roles",
    "reshard_pool": "for_swift_backup.rgw.log:reshard",
    "user_keys_pool": "for_swift_backup.rgw.meta:users.keys",
    "user_email_pool": "for_swift_backup.rgw.meta:users.email",
    "user_swift_pool": "for_swift_backup.rgw.meta:users.swift",
    "user_uid_pool": "for_swift_backup.rgw.meta:users.uid",
    "otp_pool": "for_swift_backup.rgw.otp",
    "system_key": {
        "access_key": "Huawei@123",
        "secret_key": "Huawei12#$"
    },
    "placement_pools": [
```

Step 9 In the secondary zone, create an RGW.

Run the following command to create an RGW:

```
[ceph: root@ceph11 /]# ceph orch apply rgw for_swift --realm=for_swift --zone=for_swift_backup --placement="2" --port=8080
```

```
[ceph: root@ceph11 /]# ceph -s
  cluster:
    id:     9b0381fc-5d0e-11ee-b606-fa163edb3352
    health: HEALTH_WARN
            Degraded data redundancy: 184/552 objects degraded (33

  services:
    mon: 2 daemons, quorum ceph11,ceph12 (age 6m)
    mgr: ceph11.zjjbgc(active, since 6m), standbys: ceph12.mtqqip
    osd: 6 osds: 6 up (since 6m), 6 in (since 106m)
    rgw: 2 daemons active (2 hosts, 1 zones)

  data:
    pools:   5 pools, 129 pgs
    objects: 184 objects, 6.7 KiB
    usage:   41 MiB used, 60 GiB / 60 GiB avail
    pgs:     184/552 objects degraded (33.333%)
             78 active+undersized
             51 active+undersized+degraded
```

Step 10 In the secondary zone, set the RGW information.

Run the following command to set the RGW information:

```
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph11.ykrfbf rgw_realm for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph11.ykrfbf rgw_zonegroup for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph11.ykrfbf rgw_zone for_swift_backup
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_realm for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_zonegroup for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_zone for_swift_backup
```

```
[ceph: root@ceph11 /]# ceph config set client.rgw.realm.zone.ceph11 rgw_realm for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.realm.zone.ceph11 rgw_zonegroup for_swift
[ceph: root@ceph11 /]# ceph config set client.rgw.realm.zone.ceph11 rgw_zone for_swift_backup
```

Step 11 In the secondary zone, update the realm.

Run the following command to update the realm:

```
[ceph: root@ceph11 /]# radosgw-admin period update --rgw-realm=for_swift --commit
```

```
[ceph: root@ceph11 /]# radosgw-admin period update --rgw-realm=for_swift --commit
2023-09-27T10:31:47.586+0000 7f9eae7c84c0  1 Cannot find zone id=edbc1161-e26b-4214-8184-5a66e3fb0f62 (name=for_swift_backup), switching to local zonegroup configuration
Sending period to new master zone eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2
{
    "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 3,
    "predecessor_uuid": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "sync_status": [],
    "period_map": {
        "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
        "zonegroups": [
            {
                "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
                "name": "for_swift",
                "api_name": "for_swift",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                "zones": [
                    {
```

Step 12 In the secondary zone, restart the RGW daemon.

Run the following command to restart the RGW.

```
[ceph: root@ceph01 /]# ceph orch restart rgw.for_swift
```

```
[ceph: root@ceph11 /]# ceph orch restart rgw.for_swift
Scheduled to restart rgw.for_swift.ceph11.ykrfbf on host 'ceph11'
Scheduled to restart rgw.for_swift.ceph12.rhraxx on host 'ceph12'
[ceph: root@ceph11 /]#
```

Step 13 Perform a test.

In the secondary zone, check whether the bucket in the master zone is synchronized.

```
[ceph: root@ceph11 /]# radosgw-admin bucket list
[
    "test"
]
```

Run the following command to display detailed information about the data synchronization:

```
[ceph: root@ceph11 /]# radosgw-admin sync status
```

```
[ceph: root@ceph11 /]# radosgw-admin sync status
          realm 441c10e3-2da3-4f21-bed1-5c37a8bba35a (for_swift)
      zonegroup 38aeeade-38f2-4e65-bc5f-682667b6ca46 (for_swift)
           zone edbc1161-e26b-4214-8184-5a66e3fb0f62 (for_swift_backup)
  metadata sync syncing
                full sync: 0/64 shards
                incremental sync: 64/64 shards
                metadata is caught up with master
      data sync source: eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2 (for_swift)
                        syncing
                        full sync: 0/128 shards
                        incremental sync: 128/128 shards
                        data is caught up with source
```

Step 14 Perform a data synchronization test.

On the client, run the following command to upload an object named ceph11 to for_swift_backup:

```
[root@client ~]# swift -A http://192.168.0.31:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 upload --object-name ceph11 test /etc/hosts -v --insecure
```

In Cluster1, check whether ceph11 is synchronized. The details are as follows:

```
[ceph: root@ceph01 /]# radosgw-admin bucket radoslist
```

```
[ceph: root@ceph01 /]# radosgw-admin bucket radoslist
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_ceph11
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1__shadow_.QpTBE7lozVxclCjX0oSKFB9yqwjvqPO_1
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_file1
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_hosts
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_hosts1
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_hosts2
eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2.94194.1_hosts3
```

Or, run the **swift** command to check whether ceph11 exists in the for_swift zone.

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 list test --insecure
```

```
[root@client ~]# swift -A http://192.168.0.21:8080/auth/1.0 -U "swift:swift_rgw" -K mtCMEbogngZZ1578VTPjcr8bXjsb61mso4ODx5E7 list test --insecu
ceph11
file1
hosts
hosts1
hosts2
hosts3
```

5 Practicing Ceph Block Storage RBD

5.1 Introduction

5.1.1 About This Exercise

In this exercise, you will create and mount an RBD resource pool in Ceph to the client node as a block device. Operations such as formatting and data read/write are also practiced on the client.

5.1.2 Objectives

This exercise corresponds to the Ceph Block Storage RBD and File Storage CephFS slides, which focuses on block devices operations such as creating and mounting pools.

5.2 Creating RBDs and Mounting Them to a Client

In this exercise, you will create an RBD resource pool named block, and two RBDs, rbd1 and rbd2. The capacities of rbd1 and rbd2 are 3 GB and 5 GB, respectively, and they are mounted to the **/mnt/rbd1** and **/mnt/rbd2** directories on the client host.

Step 1 Create an RBD resource pool.

Run the following commands to create a resource pool block and set it to **rbd**:

```
[ceph: root@ceph01 /]# ceph osd pool create block
[ceph: root@ceph01 /]# ceph osd pool application enable block rbd
```

```
[ceph: root@ceph01 /]# ceph osd pool create block
pool 'block' created
[ceph: root@ceph01 /]# ceph osd pool application enable block rbd
enabled application 'rbd' on pool 'block'
```

Step 2 Create RBDs.

Run the following commands to create two RBDs with capacities of 3 GB and 5 GB:

```
[ceph: root@ceph01 /]# rbd create --size 3G block/rbd1
[ceph: root@ceph01 /]# rbd create --size 5G block/rbd2
```

```
[ceph: root@ceph01 /]# rbd create --size 3G block/rbd1
[ceph: root@ceph01 /]# rbd create --size 5G block/rbd2
[ceph: root@ceph01 /]# rbd info /block/rbd1
rbd: error opening default pool 'rbd'
Ensure that the default pool has been created or specify an alternate pool name.
[ceph: root@ceph01 /]# rbd info block/rbd1
rbd image 'rbd1':
        size 3 GiB in 768 objects
        order 22 (4 MiB objects)
        snapshot_count: 0
        id: 1e5821c66de6e
        block_name_prefix: rbd_data.1e5821c66de6e
        format: 2
        features: layering, exclusive-lock, object-map, fast-diff, deep-flatten
        op_features:
        flags:
        create_timestamp: Thu Sep 28 03:11:27 2023
        access_timestamp: Thu Sep 28 03:11:27 2023
        modify_timestamp: Thu Sep 28 03:11:27 2023
[ceph: root@ceph01 /]# rbd info block/rbd2
rbd image 'rbd2':
        size 5 GiB in 1280 objects
        order 22 (4 MiB objects)
        snapshot_count: 0
        id: 1e588d4c95686
        block_name_prefix: rbd_data.1e588d4c95686
        format: 2
        features: layering, exclusive-lock, object-map, fast-diff, deep-flatten
        op_features:
        flags:
        create_timestamp: Thu Sep 28 03:11:34 2023
        access_timestamp: Thu Sep 28 03:11:34 2023
        modify_timestamp: Thu Sep 28 03:11:34 2023
```

Step 3 Create a user.

Create the block user and grant the read and write permissions on all RBDs to it.

```
[ceph: root@ceph01 /]# ceph auth add client.block mon 'allow r' osd 'allow rwx pool=block'
```

```
[ceph: root@ceph01 /]# ceph auth get client.block
[client.block]
        key = AQAO8BRltjJnKhAAMJnOMQSckxorqHbQXAB7cQ==
        caps mon = "allow r"
        caps osd = "allow rwx pool=block"
exported keyring for client.block
```

Export the keyring file of the block user as **ceph.client.block.keyring**.

```
[ceph: root@ceph01 /]# ceph auth get client.block -o ceph.client.block.keyring
```

```
[ceph: root@ceph01 /]# ceph auth get client.block -o ceph.client.block.keyring
exported keyring for client.block
[ceph: root@ceph01 /]# cat ceph.client.block.keyring
[client.block]
        key = AQAO8BRltjJnKhAAMJnOMQSckxorqHbQXAB7cQ==
        caps mon = "allow r"
        caps osd = "allow rwx pool=block"
```

Send the file to the **/etc/ceph** directory on the client.

```
[ceph: root@ceph01 /]# scp ceph.client.block.keyring root@192.168.0.24:/etc/ceph/

Authorized users only. All activities may be monitored and reported.
root@192.168.0.24's password:
ceph.client.block.keyring
```

Step 4 Mount the RBDs to the client.

Install the Ceph client software on the client.

```
[root@client ~]# yum install -y ceph
```

Run the **rbd** commands to map the created RBDs to the client.

```
[root@client ~]#   rbd map block/rbd1 --name client.block
[root@client ~]#   rbd map block/rbd2 --name client.block
```

```
[root@client ~]#  rbd map block/rbd1 --name client.block
/dev/rbd0
[root@client ~]#  rbd map block/rbd2 --name client.block
/dev/rbd1
```

After the mapping is complete, check whether the corresponding devices can be viewed.

```
[root@client ~]# lsblk
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  40G  0 disk
├─sda1   8:1    0   2G  0 part /boot
└─sda2   8:2    0  38G  0 part /
rbd0   253:0    0   3G  0 disk
rbd1   253:16   0   5G  0 disk
```

Format and mount the mapped devices to the specified directory as follows:

```
[root@client ~]# mkdir -p /mnt/rbd{1,2}
[root@client ~]# mkfs.xfs /dev/rbd0
[root@client ~]# mkfs.xfs /dev/rbd1
[root@client ~]# mount /dev/rbd0 /mnt/rbd1
[root@client ~]# mount /dev/rbd1 /mnt/rbd2
```

Once mounted, the directory information is as follows:

```
[root@client ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        1.8G     0  1.8G   0% /dev
tmpfs           1.8G     0  1.8G   0% /dev/shm
tmpfs           710M   17M  693M   3% /run
tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
/dev/sda2        38G  3.1G   33G   9% /
tmpfs           1.8G     0  1.8G   0% /tmp
/dev/sda1       2.0G   96M  2.0G   5% /boot
/dev/rbd0       3.0G   54M  3.0G   2% /mnt/rbd1
/dev/rbd1       5.0G   69M  5.0G   2% /mnt/rbd2
```

Step 5 Set the RBDs to be automatically mounted.

Add the following content to the **/etc/fstab** file:

```
/dev/rbd/block/rbd1     /mnt/rbd1     xfs     defaults,_netdev 0 0
/dev/rbd/block/rbd2     /mnt/rbd2     xfs     defaults,_netdev 0 0
```

```
UUID=ae2cd454-a1aa-4683-8c9d-711a1d610f40    /     ext4    defaults  0 0
UUID=5930-B586 /boot vfat defaults 0 0
/dev/rbd/block/rbd1           /mnt/rbd1        xfs     defaults,_netdev 0 0
/dev/rbd/block/rbd2           /mnt/rbd2        xfs     defaults,_netdev 0 0
```

Add the following content to the **/etc/ceph/rbdmap** file:

```
block/rbd1      id=block,keyring=/etc/ceph/ceph.client.block.keyring
block/rbd2      id=block,keyring=/etc/ceph/ceph.client.block.keyring
```

```
# RbdDevice             Parameters
#poolname/imagename     id=client,keyring=/etc/ceph/ceph.client.keyring
block/rbd1          id=block,keyring=/etc/ceph/ceph.client.block.keyring
block/rbd2          id=block,keyring=/etc/ceph/ceph.client.block.keyring
```

Set the rbdmap service to automatically start upon system startup.

```
[root@client ~]# systemctl enable rbdmap
Created symlink /etc/systemd/system/multi-user.target.wants/rbd
```

Restart the client and check whether the client automatically mounts the two RBDs upon startup.

If the mounting fails, check whether the RBDs have the exclusive lock. If so, delete it and mount them again.

```
[ceph: root@ceph01 /]# rbd lock list block/rbd2
There is 1 exclusive lock on this image.
Locker         ID                        Address
client.134260  auto 18446462598732840963  192.168.0.24:0/3840007873
[ceph: root@ceph01 /]# rbd lock rm block/rbd2 "auto 18446462598732840963" "client.134260"
```

5.3 Expanding RBD Capacity

In this exercise, rbd1 created in section 5.2 will be expanded to 5 GB.

Step 1 Expand the capacity of rbd1 to 5 GB.

Run the following command to expand the capacity of rbd1 from 3 GB to 5 GB:

```
[ceph: root@ceph01 /]# rbd --pool block resize rbd1 --size 5G
```

Check whether the RBD capacity has been expanded:

```
[ceph: root@ceph01 /]# rbd --pool block du rbd1
```

```
[ceph: root@ceph01 /]# rbd --pool block resize rbd1 --size 5G
Resizing image: 100% complete...done.
[ceph: root@ceph01 /]# rbd --pool block du rbd1
NAME  PROVISIONED  USED
rbd1        5 GiB  36 MiB
[ceph: root@ceph01 /]#
```

Step 2 Expand the client capacity.

The RBD capacity expansion cannot take effect directly on the client. You need to expand the partition capacity on the client to complete the capacity expansion. The details are as follows:

```
[root@client ~]# xfs_growfs /dev/rbd0
```

Check whether the partition capacity has been expanded:

```
[root@client ~]# xfs_growfs /dev/rbd0
meta-data=/dev/rbd0              isize=512    agcount=14, agsize=98304 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=0 inobtcount=0
data     =                       bsize=4096   blocks=1310720, imaxpct=25
         =                       sunit=16     swidth=16 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=2560, version=2
         =                       sectsz=512   sunit=16 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
[root@client ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        1.8G     0  1.8G   0% /dev
tmpfs           1.8G     0  1.8G   0% /dev/shm
tmpfs           710M   17M  693M   3% /run
tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
/dev/sda2        38G  3.2G   33G   9% /
tmpfs           1.8G     0  1.8G   0% /tmp
/dev/sda1       2.0G   96M  2.0G   5% /boot
/dev/rbd0       5.0G   69M  5.0G   2% /mnt/rbd1
/dev/rbd1       5.0G   69M  5.0G   2% /mnt/rbd2
```

5.4 Managing RBD Snapshots

In this exercise, you will create snapshot snap1 for rbd2, create a new RBD based on snap1, and name the RBD rbd3.

Step 1 Make preparations.

Create a test file named **test** in **/mnt/rbd2** on the client and enter "this is a test file". The detailed operations are as follows:

```
[root@client ~]# echo "this is a test file" > /mnt/rbd2/test
[root@client ~]# cat /mnt/rbd2/test
this is a test file
```

Step 2 Create a snapshot.

Create snapshot snap1 for rbd2.

```
[ceph: root@ceph01 /]# rbd snap create block/rbd2@snap1
```

```
[ceph: root@ceph01 /]# rbd snap create block/rbd2@snap1
Creating snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap ls block/rbd2
SNAPID  NAME   SIZE   PROTECTED  TIMESTAMP
     4  snap1  5 GiB             Thu Sep 28 07:36:45 2023
```

Step 3 Test the snapshot function.

Delete the file test from **/mnt/rbd2**, roll back the snapshot, and check whether test can be restored.

Delete test on the client and cancel the mapping and mounting of the RBD. The detailed operations are as follows:

```
[root@client ~]# umount /mnt/rbd2
[root@client ~]# rbdmap unmap block/rbd2
```

```
[root@client ~]# umount /mnt/rbd2
[root@client ~]# rbdmap unmap block/rbd2
[root@client ~]# lsblk
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  40G  0 disk
├─sda1   8:1    0   2G  0 part /boot
└─sda2   8:2    0  38G  0 part /
```

Run the following command on the Ceph management node to roll back the snapshot:

```
[ceph: root@ceph01 /]# rbd snap rollback  block/rbd2@snap1
```

```
[ceph: root@ceph01 /]# rbd snap rollback  block/rbd2@snap1
Rolling back to snapshot: 100% complete...done.
```

Map and mount the RBD on the client again and check whether the test file exists.

```
[root@client ~]# rbdmap map block/rbd2 --name client.block
[root@client ~]# lsblk
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  40G  0 disk
├─sda1   8:1    0   2G  0 part /boot
└─sda2   8:2    0  38G  0 part /
rbd0   253:0    0   5G  0 disk /mnt/rbd1
rbd1   253:16   0   5G  0 disk /mnt/rbd2
[root@client ~]# mount -a
[root@client ~]# ls /mnt/rbd2
test
```

Step 4 Create a clone volume using the snapshot.

Set the snapshot to the protection mode.

```
[ceph: root@ceph01 /]# rbd snap protect block/rbd2@snap1
```

Create the clone volume rbd3 for snap1.

```
[ceph: root@ceph01 /]# rbd clone block/rbd2@snap1 block/rbd3
```

Convert the clone volume to an independent RBD.

```
[ceph: root@ceph01 /]# rbd flatten block/rbd3
```

```
[ceph: root@ceph01 /]# rbd snap protect block/rbd2@snap1
[ceph: root@ceph01 /]# rbd clone block/rbd2@snap1 block/rbd3
[ceph: root@ceph01 /]# rbd -p block ls
rbd1
rbd2
rbd3
[ceph: root@ceph01 /]# rbd flatten block/rbd3
Image flatten: 100% complete...done.
```

On the client, map the converted rbd3 to the client.

```
[root@client ~]# rbd map block/rbd3 --name client.block
```

Unmount the base volume of the clone volume and mount the clone volume to the **/mnt/rbd3** directory.

```
[root@client ~]# umount /mnt/rbd2
[root@client ~]# mkidr /mnt/rbd3
[root@client ~]# mount /dev/rbd2 /mnt/rbd3
```

After the mounting is complete, check whether the test file exists.

```
[root@client ~]# lsblk
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  40G  0 disk
├─sda1   8:1    0   2G  0 part /boot
└─sda2   8:2    0  38G  0 part /
rbd0   253:0    0   5G  0 disk /mnt/rbd1
rbd1   253:16   0   5G  0 disk
rbd2   253:32   0   5G  0 disk /mnt/rbd3
[root@client ~]# ls /mnt/rbd3
test
[root@client ~]#
```

- Question: Is it mandatory to unmount the base volume? And Why?
- Answer: This is a mandatory operation in this exercise because the base volume is formatted to the XFS format and its UUID is the same as that of the clone volume. When the clone volume is being mounted, the system considers the clone volume has been mounted and will report an error. For other file systems, you do not need to unmount the base volume.

6 Practicing Ceph File Storage CephFS

6.1 Introduction

6.1.1 About This Exercise

In this exercise, you will create two replicated resource pools; one to store CephFS metadata, and the other to store CephFS data. You will also create a response user for authentication during client mounting.

6.1.2 Objectives

This exercise corresponds to the Ceph Block Storage RBD and File Storage CephFS slides, which focuses on practicing using CephFS, including resource pool creation and mounting, to enhance your understanding of CephFS.

6.2 Resources Required for Creating CephFS

Step 1 Create resource pools.

Create two replicated pools named data_cephfs and metadata_cephfs. data_cephfs is the data pool of CephFS, and metadata_cephfs is the metadata pool of CephFS.

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_cephfs
[ceph: root@ceph01 /]# ceph osd pool create data_cephfs
```

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_cephfs
pool 'metadata_cephfs' created
[ceph: root@ceph01 /]# ceph osd pool create data_cephfs
pool 'data_cephfs' created
```

Step 2 Format the resource pools.

Run the following command to format the created resource pools:

```
[ceph: root@ceph01 /]# ceph fs new fs01 metadata_cephfs data_cephfs
```

```
[ceph: root@ceph01 /]# ceph fs new fs01 metadata_cephfs data_cephfs
new fs with metadata pool 15 and data pool 16
```

Step 3 Deploy the MDS.

Deploy two MDSs in the Ceph cluster.

```
[ceph: root@ceph01 /]# ceph orch apply mds fs01 --placement="2"
```

```
[ceph: root@ceph01 /]# ceph orch apply mds fs01 --placement="2"
Scheduled mds.fs01 update...
```

After the deployment is complete, check the MDS status. The details are as follows:

```
[ceph: root@ceph01 /]# ceph orch ls
NAME                       PORTS        RUNNING  REFRESHED  AGE   PLACEMENT
alertmanager               ?:9093,9094      1/1  6m ago     2w    count:1
crash                                       3/3  6m ago     2w    *
grafana                    ?:3000           1/1  6m ago     2w    count:1
mds.fs01                                    2/2  96s ago    101s  count:2
mgr                                         3/3  6m ago     2w    label:_admin
mon                                         3/3  6m ago     2w    label:_admin
node-exporter              ?:9100           3/3  6m ago     2w    *
osd                                           3  96s ago    -     <unmanaged>
osd.all-available-devices                     6  6m ago     2w    <unmanaged>
prometheus                 ?:9095           1/1  6m ago     2w    count:1
rgw.for_swift              ?:8080           2/2  6m ago     2w    count:2
```

Step 4 Create a user.

Create user01 and grant it the read and write permissions on fs01.

```
[ceph: root@ceph01 /]# ceph fs authorize fs01 client.user01 / rwps -o ceph.client.user01.keyring
```

Check whether the configuration is correct.

```
[ceph: root@ceph01 /]# ceph auth ls | grep -A 4 user
installed auth entries:

client.user01
        key: AQC7rSNlBesFBxAAeBhPod/SfnJxJNdrLo66YA==
        caps: [mds] allow rwps fsname=fs01
        caps: [mon] allow r fsname=fs01
        caps: [osd] allow rw tag cephfs data=fs01
```

6.3 Mounting the File System to the Client

Mount CephFS to the **/mnt/cephfs** directory on the client and enable automatic mounting upon system startup.

Step 1 Copy the keyring of user01 to the client.

On the Ceph server, copy the keyring file of user01 generated in 6.2 to the **/etc/ceph** directory on the client.

```
[ceph: root@ceph01 /]# scp ceph.client.user01.keyring root@192.168.0.24:/etc/ceph/

Authorized users only. All activities may be monitored and reported.
root@192.168.0.24's password:
ceph.client.user01.keyring
```

Step 2 Create the authentication file.

Create the authentication file **user.txt** and save the user information and key information in the file.

```
[root@client ~]# echo "AQC7rSNlBesFBxAAeBhPod/SfnJxJNdrLo66YA==" > user.txt
```

```
[root@client ~]# echo "AQC7rSNlBesFBxAAeBhPod/SfnJxJNdrLo66YA==" > user.txt
[root@client ~]# cat user.txt
AQC7rSNlBesFBxAAeBhPod/SfnJxJNdrLo66YA==
```

Step 3 Modify the **/etc/fstab** file.

Add the following content to the **/etc/fstab** file:

```
192.168.0.22:6789,192.168.0.23:6789:/ /mnt/cephfs ceph defaults,_netdev,name=user01,secretfile=/root/user.txt 0 0
```

```
[root@client ~]# cat /etc/fstab
UUID=ae2cd454-a1aa-4683-8c9d-711a1d610f40    /    ext4    defaults  0 0
UUID=5930-B586 /boot vfat defaults 0 0
/dev/rbd/block/rbd1          /mnt/rbd1        xfs     defaults,_netdev 0 0
/dev/rbd/block/rbd2          /mnt/rbd2        xfs     defaults,_netdev 0 0
192.168.0.22:6789,192.168.0.23:6789:/          /mnt/cephfs      ceph     defaults,_netdev,name=user01,secretfile=/root/user.txt 0 0
```

Run **mount -a** to mount the file system, and then run **mount** to check whether the mounting is successful. The details are as follows:

```
[root@client ~]# mount -a
[root@client ~]# mount | grep cephh
[root@client ~]# mount | grep ceph
192.168.0.22:6789,192.168.0.23:6789:/ on /mnt/cephfs type ceph (rw,relatime,seclabel,name=user01,secret=<hidden>,acl,_netdev)
[root@client ~]#
```

Try to create and delete files in **/mnt/cephfs** to check whether user01 has the read and write permissions on the resource pool.

- Question: What are 192.168.0.22 and 192.168.0.23 in the **/etc/fstab** file? Can they be replaced by other addresses?
- Answer: These IP addresses relate to the nodes where the MDS resides. These IP addresses need to be obtained from the Ceph management node after the MDS is deployed. In this exercise, the two addresses cannot be replaced. If a node is specified during MDS deployment or an MDS process is faulty, the system deploys the MDS on another node. In this case, the address needs to be replaced to ensure that the client can use CephFS.

7 Manually Editing the CRUSH Map

7.1 Introduction

7.1.1 About This Exercise

When creating ECSs, you select the general-purpose SSD as the disk type, but Ceph identifies the SSDs as HDDs. In this exercise, you need to manually modify the CRUSH map to change the type of all data disks to SSD.

7.1.2 Objectives

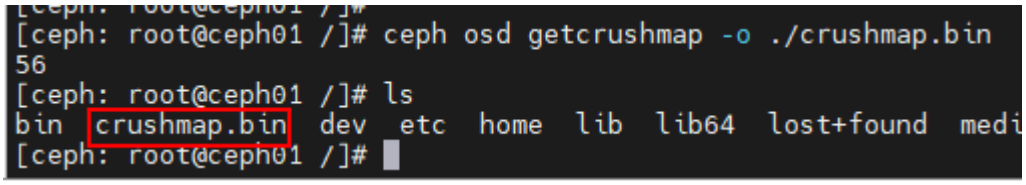
This exercise corresponds to the CRUSH Map Overview and Configurations slides. Here, you will practice manually editing the CRUSH map to configure Ceph to enhance your understanding of the slides.

7.2 Manually Configuring the CRUSH Map

Step 1 Download the CRUSH map.

Run the following command to download the CRUSH map to the specified location:

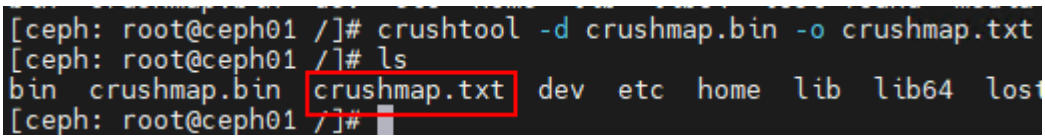
```
[ceph: root@ceph01 /]# ceph osd getcrushmap -o ./crushmap.bin
```



Step 2 Decompile the CRUSH map.

Run the following command to decompile the exported CRUSH map into an editable file:

```
[ceph: root@ceph01 /]# crushtool -d crushmap.bin -o crushmap.txt
```



Step 3 Edit the CRUSH map.

Change **hdd** to **ssd** in the decompiled **crushmap.txt** file.

```
# devices
device 0 osd.0 class ssd
device 1 osd.1 class ssd
device 2 osd.2 class ssd
device 3 osd.3 class ssd
device 4 osd.4 class ssd
device 5 osd.5 class ssd
device 6 osd.6 class ssd
device 7 osd.7 class ssd
device 8 osd.8 class ssd
```

In addition, modify the definition of buckets. See the following figure.

```
# buckets
host ceph03 {
        id -3           # do not change unnecessarily
        id -4 class ssd         # do not change unnecessarily
        # weight 0.029
        alg straw2
        hash 0  # rjenkins1
        item osd.1 weight 0.010
        item osd.4 weight 0.010
        item osd.7 weight 0.010
}
host ceph01 {
        id -5           # do not change unnecessarily
        id -6 class ssd         # do not change unnecessarily
        # weight 0.029
        alg straw2
        hash 0  # rjenkins1
        item osd.0 weight 0.010
        item osd.3 weight 0.010
        item osd.6 weight 0.010
}
host ceph02 {
        id -7           # do not change unnecessarily
        id -8 class ssd         # do not change unnecessarily
        # weight 0.029
        alg straw2
        hash 0  # rjenkins1
        item osd.2 weight 0.010
        item osd.5 weight 0.010
        item osd.8 weight 0.010
}
root default {
        id -1           # do not change unnecessarily
        id -2 class ssd         # do not change unnecessarily
        # weight 0.088
        alg straw2
        hash 0  # rjenkins1
        item ceph03 weight 0.029
        item ceph01 weight 0.029
        item ceph02 weight 0.029
}
```

Step 4 Recompile the CRUSH map.

Run the following command to recompile the edited file into a new CRUSH map file:

```
[ceph: root@ceph01 /]# crushtool -c ./crushmap.txt -o ./crushmap2.bin
```

```
[ceph: root@ceph01 /]# crushtool -c ./crushmap.txt -o ./crushmap2.bin
[ceph: root@ceph01 /]# ls
bin  crushmap.bin  crushmap.txt  crushmap2.bin  dev  etc  home  lib
```

Step 5 Import the CRUSH map again.

Run the following command to import the recompiled file to the system:

```
[ceph: root@ceph01 /]# ceph osd setcrushmap -i crushmap2.bin
```

```
[ceph: root@ceph01 /]# ceph osd setcrushmap -i crushmap2.bin
58
```

After the file is imported, check the OSD status to determine whether the CRUSH map takes effect.

```
[ceph: root@ceph01 /]# ceph osd tree
ID  CLASS  WEIGHT   TYPE NAME         STATUS  REWEIGHT  PRI-AFF
-1         0.08698  root default
-5         0.02899      host ceph01
 0    ssd  0.00999          osd.0         up   1.00000  1.00000
 3    ssd  0.00999          osd.3         up   1.00000  1.00000
 6    ssd  0.00999          osd.6         up   1.00000  1.00000
-7         0.02899      host ceph02
 2    ssd  0.00999          osd.2         up   1.00000  1.00000
 5    ssd  0.00999          osd.5         up   1.00000  1.00000
 8    ssd  0.00999          osd.8         up   1.00000  1.00000
-3         0.02899      host ceph03
 1    ssd  0.00999          osd.1         up   1.00000  1.00000
 4    ssd  0.00999          osd.4         up   1.00000  1.00000
 7    ssd  0.00999          osd.7         up   1.00000  1.00000
```

In the preceding figure, the OSD **CLASS** has been updated to **ssd**.

---------End---------

Huawei openEuler Certification Training

HCIE-openEuler

Lab Guide

ISSUE: 1.0



Huawei Technologies Co., Ltd.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

 and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China

Website: https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Huawei Certification covers all ICT fields and follows the industry's trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talents in the digital era, and building a sound ICT talent ecosystem.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. HCIE-openEuler certification covers the Ceph cloud storage management, the cloud operating system (OS) cluster management, advanced practices of cloud native platform Kubernetes, advanced automation management, continuous integration and continuous delivery (CI/CD), intelligent operation and maintenance (O&M) platform A-Ops, and log management system EFK.

Huawei Certification helps you unlock opportunities to advance your career and take one more step towards the top of the industry.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Overview

This document is intended for candidates who are preparing for the HCIE-openEuler exam and readers who want to know how to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management on openEuler.

Description

This lab guide consists of seven exercises, including setup, networking, storage, and scheduling of the Kubernetes cluster.

- Exercise 1: Automated Kubernetes cluster creation. This exercise describes how to use kubeadm to automatically create a Kubernetes cluster.
- Exercise 2: Network components. This exercise describes network mode switching, IP address pool configuration, and network policies, helping you master Kubernetes network principles.
- Exercise 3: Backend storage. This exercise describes how to connect Kubernetes to Ceph for block storage and file storage.
- Exercise 4: Scheduler. This exercise describes how to schedule pods using the default Kubernetes scheduler.
- Exercise 5: Autoscaling. This exercise describes how to use metrics such as CPU and memory to automatically scale pods.
- Exercise 6: Container OS upgrade. This exercise describes how to perform OS upgrade using KubeOS and how to implement kernel live upgrade.
- Exercise 7: Hybrid deployment of containers. This exercise describes how to use Rubik to deploy online and offline services together.

Background Knowledge Required

The intended audiences should:

- Know Linux basics. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Lab Environment Overview

It is recommended that the lab environment uses the following configurations. The following table lists the mappings among the devices, models, and versions.

Hostname	Kubernetes Cluster IP Address	IP Address for Communicating with the Storage Device	Virtual IP Address (VIP)
k8smaster1	192.168.1.11/24	192.168.0.11/24	192.168.1.10/24
k8smaster2	192.168.1.12/24	192.168.0.12/24	
k8smaster3	192.168.1.13/24	192.168.0.13/24	
k8snode1	192.168.1.15/24	192.168.0.14/24	
k8snode2	192.168.1.16/24	192.168.0.15/24	
k8snode3	192.168.1.17/24	192.168.0.16/24	

Unified specifications of Kubernetes nodes:

OS: openEuler 22.03 LTS

CPU architecture: x86

CPU: 4 vCPUs

Memory: 8 GiB

Drive: 40 GiB. The **k8smaster1** node requires 80 GiB.

NIC: Two NICs. Each node is configured with an elastic IP address (EIP) or can access the Internet through route configuration.

Lab Environment Preparation

Checking Devices

Before starting the exercises, candidates of each group should check whether the devices in the following table are available.

Device	Quantity	Remarks
Kubernetes node	6	
Ceph cluster	1	

Check that the following resources are ready:

- One laptop or desktop computer
- Six Kubernetes nodes
- One Ceph cluster

Contents

About This Document **3**
Overview 3
Description 3
Background Knowledge Required 3
Lab Environment Overview 3
1 Automated Kubernetes Cluster Creation **1**
1.1 Introduction 1
1.1.1 About This Exercise 1
1.1.2 Objectives 1
1.2 Environment Setup 1
1.3 Master Nodes in HA Mode 4
1.4 Kubernetes Cluster Creation 7
1.5 Result Verification 12
2 Network Components **14**
2.1 Introduction 14
2.1.1 About This Exercise 14
2.1.2 Objectives 14
2.2 Basic Operations of Calico 14
2.3 IP Address Pool 19
2.4 VXLAN Mode 22
2.5 Fixed IP Address 24
2.6 BGP Mode 25
2.7 Namespace-based Access Control 28
2.8 Label-based Access Control 30
3 Persistent Storage **34**
3.1 Introduction 34
3.1.1 About This Exercise 34
3.1.2 Objectives 34
3.2 Resource Pool Creation 34
3.3 ceph-csi File Preparation 35
3.4 Kubernetes Call to RBD 35
3.5 Kubernetes Call to CephFS 40
4 Scheduler **44**
4.1 Introduction 44

4.1.1 About This Exercise 44
4.1.2 Objectives 44
4.2 nodeSelector-based Scheduling 44
4.3 nodeName-based Scheduling 47
4.4 One Topology Spread Constraint 48
4.5 Multiple Topology Spread Constraints 50
4.6 Hard Node Affinity 51
4.7 Soft Node Affinity 53
4.8 Inter-Pod Affinity 54
4.9 Inter-Pod Anti-affinity 56
4.10 Taints and Tolerations 56
4.11 Taint Eviction 59
4.12 Multiple Taints and Tolerations 60
4.13 Node Marked as Schedulable 61
4.14 Master Node Scheduling 63
5 Autoscaling 66
5.1 Introduction 66
5.1.1 About This Exercise 66
5.1.2 Objectives 66
5.2 Metrics Server Installation 66
5.3 HPA on CPU Utilization 67
5.4 HPA on Memory Utilization 70
6 Container OS Upgrade 74
6.1 Introduction 74
6.1.1 About This Exercise 74
6.1.2 Objectives 74
6.2 Container OS Image Build 74
6.3 k8snode4 Deployment 78
6.4 CRD, Operator, and Proxy Deployment 80
6.5 Container OS Upgrade for k8snode4 82
6.6 Kernel Live Upgrade 84
7 Hybrid Container Deployment 87
7.1 Introduction 87
7.1.1 About This Exercise 87
7.1.2 Objectives 87
7.2 Rubik Installation and Deployment 87
7.3 Hybrid Deployment Test 88

1 Automated Kubernetes Cluster Creation

1.1 Introduction

1.1.1 About This Exercise

This lab describes how to use kubeadm to automatically create a Kubernetes cluster, and use NGINX+Keepalived to implement high availability (HA).

1.1.2 Objectives

- Understand the Kubernetes cluster architecture.
- Set up an HA cluster.
- Set up a Kubernetes cluster using kubeadm.

1.2 Environment Setup

Step 1 Set node names.

Run the following commands on the six nodes in sequence to set the node names to **k8smaster1**, **k8smaster2**, **k8smaster3**, **k8snode1**, **k8snode2**, and **k8snode3**:

```
[root@localhost ~]# hostnamectl hostname k8smaster1
[root@localhost ~]# sudo -i
[root@k8smaster1 ~]#
```

Step 2 Configure node name resolution for **hosts**.

Run the following commands on all nodes:

```
cat <<EOF>>/etc/hosts
192.168.1.11 k8smaster1
192.168.1.12 k8smaster2
192.168.1.13 k8smaster3
192.168.1.14 k8snode1
192.168.1.15 k8snode2
192.168.1.16 k8snode3
EOF
```

Step 3 Configure password-free login.

Run the following command on the **k8smaster1** node and press **Enter** repeatedly to generate a key file:

```
[root@k8smaster1 ~]# ssh-keygen
```

```
[root@k8smaster1 ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:MO96bEtGqS6AWpnAZizEO0hhjrqJjb05mF7EF6bohbU root@k8smaster1.novalocal
The key's randomart image is:
+---[RSA 3072]----+
|.o.              |
|++               |
|B... oo          |
|=B= + .+ .       |
|==.E .  S        |
|+*B .  +         |
|*=oo  ..+        |
|+ oo.. ++        |
|..o. .oo..       |
+----[SHA256]-----+
[root@k8smaster1 ~]#
```

Copy the key on the **k8smaster1** node to the other five nodes, for example, **k8smaster2**.

```
[root@k8smaster1 ~]# ssh-copy-id k8smaster2
```

Type **yes** and the node login password as prompted.

Step 4 Clear iptables and disable the firewall and SELinux.

Run the following commands on all nodes:

```
iptables -F
setenforce 0
sed -i 's/^SELINUX=.*/SELINUX=disabled/g' /etc/selinux/config
systemctl stop firewalld && systemctl disable firewalld
```

Step 5 Disable the swap partition.

Run the following commands on all nodes:

```
swapoff -a
sed -i 's/.*swap.*/#&/' /etc/fstab
```

Step 6 Enable ipvs.

Run the following commands on all nodes:

```
cat > /etc/sysconfig/modules/ipvs.modules <<END
#!/bin/bash
```

```
ipvs_modules="ip_vs ip_vs_lc ip_vs_wlc ip_vs_rr ip_vs_wrr ip_vs_lblc ip_vs_lblcr ip_vs_dh ip_vs_sh
ip_vs_nq ip_vs_sed ip_vs_ftp nf_conntrack"
for kernel_module in ${ipvs_modules}; do
 /sbin/modinfo -F filename ${kernel_module} > /dev/null 2>&1
 if [ 0 -eq 0 ]; then
 /sbin/modprobe ${kernel_module}
 fi
done
END
chmod 755 /etc/sysconfig/modules/ipvs.modules
bash /etc/sysconfig/modules/ipvs.modules
```

Step 7 Set time synchronization.

Run the following commands on all nodes:

```
yum install -y chrony
systemctl enable --now chronyd
chronyc sources
```

Step 8 Configure a Yum repository.

Run the following command on all nodes to configure the Yum repository of Kubernetes:

```
cat > /etc/yum.repos.d/kubernetes.repo<<EOF
[kubernetes]
name=Kubernetes
baseurl=https://repo.huaweicloud.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=1
repo_gpgcheck=0
gpgkey=https://repo.huaweicloud.com/kubernetes/yum/doc/yum-key.gpg
https://repo.huaweicloud.com/kubernetes/yum/doc/rpm-package-key.gpg
EOF
```

Step 9 Install Docker.

Run the following commands on all nodes to install and start Docker:

```
yum install -y docker
systemctl enable docker && systemctl start docker
```

Run the following command on all nodes to change **cgroupdriver** of Docker to **systemd** and add the configuration to **/etc/docker/daemon.json**:

```
cat <<EOF > /etc/docker/daemon.json
{
  "exec-opts": ["native.cgroupdriver=systemd"]
}
EOF
```

Run the following commands on all nodes to restart Docker:

```
systemctl daemon-reload
systemctl restart docker
```

Run the following command on all nodes to check the Docker status:

```
systemctl status docker
```

```
[root@k8smaster1 ~]# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)
     Active: active (running) since Wed 2023-11-15 06:58:24 UTC; 38s ago
       Docs: https://docs.docker.com
   Main PID: 2057 (dockerd)
      Tasks: 21 (limit: 49015)
     Memory: 46.8M
     CGroup: /system.slice/docker.service
             ├─2057 /usr/bin/dockerd --live-restore
             └─2067 containerd --config /var/run/docker/containerd/containerd.toml --log-level info
```

1.3 Master Nodes in HA Mode

Step 1 Install NGINX and Keepalived.

Run the following command on the three master nodes to install NGINX and Keepalived:

```
yum install -y nginx keepalived nginx-all-modules
```

Step 2 Modify the NGINX configuration file.

Add the following content in bold to the **/etc/nginx/nginx.conf** configuration file of the three master nodes:

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;
include /usr/share/nginx/modules/*.conf;
events {
    worker_connections 1024;
}

stream {
    log_format main '$remote_addr $upstream_addr - [$time_local] $status
$upstream_bytes_sent';
    access_log /var/log/nginx/k8s-access.log main;
    upstream k8s-apiserver {
        server 192.168.1.11:6443;
        server 192.168.1.12:6443;
        server 192.168.1.13:6443;
    }
    server {
        listen 16443;
        proxy_pass k8s-apiserver;
    }
}
```

Step 3 Modify the Keepalived configuration file.

Overwrite the **/etc/keepalived/keepalived.conf** configuration file of the **k8smaster1** node.

```
cat > /etc/keepalived/keepalived.conf<<EOF
! Configuration File for keepalived

global_defs {
    router_id master1
}

vrrp_instance Nginx {
      state MASTER
      interface ens3
      virtual_router_id 51
      priority 200
      advert_int 1
      authentication {
            auth_type PASS
            auth_pass Huawei@1
      }
      virtual_ipaddress {
            192.168.1.10/24
      }
}
EOF
```

The configurations of **k8smaster2** and **k8smaster3** are the same as the preceding configurations. Note that the value of **state** must be changed to **BACKUP** and that of **priority** must be reduced. For example, the **priority** value of **k8smaster2** is **150**, and **k8smaster3**, **100**.

Step 4 Start NGINX and Keepalived.

Run the following commands on the three master nodes:

```
systemctl enable nginx --now
systemctl enable keepalived --now
```

Step 5 Verify the configuration results.

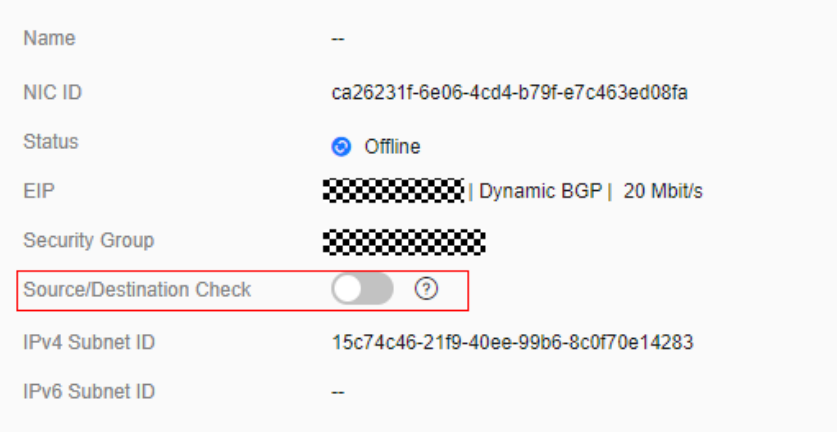
Run the following commands on the three master nodes and verify that the NGINX and Keepalived processes are in the **active(running)** state:

```
systemctl status nginx
systemctl status keepalived
```

The VIP **192.168.1.10** is displayed in the IP address of the **k8smaster1** node.

```
[root@k8smaster1 ~]# ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:f0:49:53 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute ens3
       valid_lft 107998743sec preferred_lft 107998743sec
    inet 192.168.1.10/24 scope global secondary ens3
       valid_lft forever preferred_lft forever
    inet6 fe80::f816:3eff:fef0:4953/64 scope link
       valid_lft forever preferred_lft forever
3: enp4s4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:f0:48:53 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.11/24 brd 192.168.0.255 scope global dynamic noprefixroute enp4s4
       valid_lft 107998743sec preferred_lft 107998743sec
    inet6 fe80::f816:3eff:fef0:4853/64 scope link
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:a5:00:fb:04 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

The VIP **192.168.1.10** can be pinged on all nodes. Note that if Huawei Cloud Elastic Cloud Servers (ECSs) are used, disable **Source/Destination Check** for the NICs of the ECSs.



Step 6 Verify HA switchover.

Stop the Keepalived service on **k8smaster1** and switch the VIP **192.168.1.10** to **k8smaster2**.

```
[root@k8smaster1 ~]# systemctl stop keepalived
```

```
[root@k8smaster2 ~]# ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:f0:49:54 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.1.12/24 brd 192.168.1.255 scope global dynamic noprefixroute ens3
       valid_lft 107998583sec preferred_lft 107998583sec
    inet 192.168.1.10/24 scope global secondary ens3
       valid_lft forever preferred_lft forever
    inet6 fe80::f816:3eff:fef0:4954/64 scope link
       valid_lft forever preferred_lft forever
3: enp4s4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 100
    link/ether fa:16:3e:f0:48:54 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.12/24 brd 192.168.0.255 scope global dynamic noprefixroute enp4s4
       valid_lft 107998583sec preferred_lft 107998583sec
    inet6 fe80::f816:3eff:fef0:4854/64 scope link
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:64:fa:dd:4c brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

Start the Keepalived service on **k8smaster1**. The VIP **192.168.1.10** returns to **k8smaster1** and can be pinged on all nodes.

```
[root@k8smaster1 ~]# systemctl start keepalived
```

```
[root@k8smaster1 ~]# ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:f0:49:53 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute ens3
       valid_lft 107998743sec preferred_lft 107998743sec
    inet 192.168.1.10/24 scope global secondary ens3
       valid_lft forever preferred_lft forever
    inet6 fe80::f816:3eff:fef0:4953/64 scope link
       valid_lft forever preferred_lft forever
3: enp4s4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:f0:48:53 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.11/24 brd 192.168.0.255 scope global dynamic noprefixroute enp4s4
       valid_lft 107998743sec preferred_lft 107998743sec
    inet6 fe80::f816:3eff:fef0:4853/64 scope link
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:a5:00:fb:04 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

1.4 Kubernetes Cluster Creation

Step 1 Install the Kubernetes software package.

Run the following commands on all nodes:

```
yum install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
systemctl enable --now kubelet
```

Step 2 Create a working directory.

```
mkdir /root/deploy
cd /root/deploy
```

Step 3 Create a **kubeadm-init.yaml** initialization file.

Create a YAML file for cluster initialization on all master nodes and set key configurations. Ensure that the indentation of each field is the same.

```
cat > kubeadm-init.yaml < EOF
apiVersion: kubeadm.k8s.io/v1beta2
kind: ClusterConfiguration
kubernetesVersion: v1.23.1
imageRepository: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler
controlPlaneEndpoint: 192.168.1.10:16443
dns:
  type: CoreDNS
apiServer:
  certSANs:
  - 192.168.1.10
networking:
  podSubnet: 10.0.0.0/8
  serviceSubnet: 172.16.0.0/16
---
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: ipvs
EOF
```

Step 4 Pull the Kubernetes image.

Run the following command on all nodes:

```
kubeadm config images pull --config kubeadm-init.yaml
```

```
[root@k8smaster1 deploy]# kubeadm config images pull --config kubeadm-init.yaml
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-apiserver:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-controller-manager:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-scheduler:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-proxy:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/pause:3.6
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/etcd:3.5.1-0
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/coredns:v1.8.6
```

Step 5 Initialize the cluster.

Run the following initialization command on the **k8smaster1** node:

```
kubeadm init --config kubeadm-init.yaml
```

After the command is executed, the following two commands are generated for subsequent use. The first one is used to add the other two master nodes to the cluster, and the second one is used to add three nodes to the cluster.

```
To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

  kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63 \
        --control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63
[root@k8smaster1 deploy]#
```

If the initialization fails, perform step 12 to reset the initialization after the fault is rectified.

Step 6 Copy the **config** file.

Run the following commands on the **k8smaster1** node to copy the **config** file generated during kubectl initialization. The file records the access address of the API Server, allowing you to run the **kubectl** command to connect to the API Server.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 7 Copy certificates.

Run the following commands on the **k8smaster2** and **k8smaster3** nodes to create folders for storing certificates:

```
mkdir -p /etc/kubernetes/pki/
mkdir -p /etc/kubernetes/pki/etcd
```

Run the following commands on the **k8smaster1** node to transfer certificates to the **k8smaster2** and **k8smaster3** nodes:

```
scp /etc/kubernetes/pki/ca.* /etc/kubernetes/pki/sa.* /etc/kubernetes/pki/front-proxy-ca.*
k8smaster2:/etc/kubernetes/pki/
scp /etc/kubernetes/pki/etcd/ca.* k8smaster2:/etc/kubernetes/pki/etcd/
scp /etc/kubernetes/admin.conf k8smaster2:/etc/kubernetes/
scp /etc/kubernetes/pki/ca.* /etc/kubernetes/pki/sa.* /etc/kubernetes/pki/front-proxy-ca.*
k8smaster3:/etc/kubernetes/pki/
scp /etc/kubernetes/pki/etcd/ca.* k8smaster3:/etc/kubernetes/pki/etcd/
scp /etc/kubernetes/admin.conf k8smaster3:/etc/kubernetes/
```

Step 8 Add other master nodes to the cluster.

Run the **kubeadm join** command generated during initialization to add a node to the cluster. The command contains the token and can be classified into the command for

adding the master node to the control plane and the command for adding the node to the cluster.

```
You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

  kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63 \
        --control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63
```

Run the **kubeadm join** command on the **k8smaster2** and **k8smaster3** nodes to add them to the cluster. Do not directly copy the following example command but copy the command generated during kubeadm initialization.

```
kubeadm join 192.168.1.10:16443 --token aipo4t.mfr0sigv4cfx7rud \
        --discovery-token-ca-cert-hash sha256:XXXXXX \
        --control-plane
```

```
This node has joined the cluster and a new control plane instance was created:

* Certificate signing request was sent to apiserver and approval was received.
* The Kubelet was informed of the new secure connection details.
* Control plane (master) label and taint were applied to the new node.
* The Kubernetes control plane instances scaled up.
* A new etcd member was added to the local/stacked etcd cluster.

To start administering your cluster from this node, you need to run the following as a regular user:

        mkdir -p $HOME/.kube
        sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
        sudo chown $(id -u):$(id -g) $HOME/.kube/config

Run 'kubectl get nodes' to see this node join the cluster.
```

If the join operation fails, perform step 12 to reset the initialization after the fault is rectified.

Run the following commands on the **k8smaster2** and **k8smaster3** nodes to copy the **kubeconfig** file:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 9 Add nodes to the cluster.

Run the command generated during initialization on the three nodes to add them to the cluster. Note that the **--control-plane** option is not required and you need to copy the command generated during kubeadm initialization instead of directly copying the following example command:

```
kubeadm join 10.0.0.10:16443 --token aipo4t.mfr0sigv4cfx7rud \
        --discovery-token-ca-cert-hash sha256:XXXXXX
```

```
kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

If the join operation fails, perform step 12 to reset the initialization after the fault is rectified.

Run the following command on **k8smaster1** to check the node status:

```
[root@k8smaster1 ~]# kubectl get nodes
```

```
[root@k8smaster1 deploy]# kubectl get nodes
NAME         STATUS     ROLES                  AGE    VERSION
k8smaster1   NotReady   control-plane,master   10m    v1.23.1
k8smaster2   NotReady   control-plane,master   102s   v1.23.1
k8smaster3   NotReady   control-plane,master   102s   v1.23.1
k8snode1     NotReady   <none>                 31s    v1.23.1
k8snode2     NotReady   <none>                 31s    v1.23.1
k8snode3     NotReady   <none>                 19s    v1.23.1
```

Step 10 Install the network plug-in Calico.

Run the following commands on the **k8smaster1** node:

```
curl -O https://docs.projectcalico.org/archive/v3.23/manifests/calico.yaml
kubectl apply -f calico.yaml
```

```
[root@k8smaster1 deploy]# curl -O https://docs.projectcalico.org/archive/v3.23/manifests/calico.yaml
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  226k  100  226k    0     0   109k      0  0:00:02  0:00:02 --:--:--  109k
[root@k8smaster1 deploy]# kubectl apply -f calico.yaml
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipreservations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.org created
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrole.rbac.authorization.k8s.io/calico-node created
clusterrolebinding.rbac.authorization.k8s.io/calico-node created
daemonset.apps/calico-node created
serviceaccount/calico-node created
deployment.apps/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
poddisruptionbudget.policy/calico-kube-controllers created
```

Step 11 Check the node status.

Wait several minutes after Calico is deployed. Run the following command on **k8smaster1** to verify that the status of all nodes is **Ready**:

```
kubectl get nodes
```

```
[root@k8smaster1 deploy]# kubectl get nodes
NAME         STATUS   ROLES                  AGE     VERSION
k8smaster1   Ready    control-plane,master   16m     v1.23.1
k8smaster2   Ready    control-plane,master   7m8s    v1.23.1
k8smaster3   Ready    control-plane,master   7m8s    v1.23.1
k8snode1     Ready    <none>                 5m57s   v1.23.1
k8snode2     Ready    <none>                 5m57s   v1.23.1
k8snode3     Ready    <none>                 5m45s   v1.23.1
```

Verify that the status of all pods is **Running**.

```
kubectl get pods -n kube-system -o wide
```

```
[root@k8smaster1 deploy]# kubectl get pods -n kube-system -o wide
NAME                                       READY   STATUS    RESTARTS      AGE   IP              NODE         NOMINATED NODE   READINESS GATES
calico-kube-controllers-54756b744f-lhdbt   1/1     Running   0             10m   10.38.249.2     k8snode1     <none>           <none>
calico-node-4z7p6                          1/1     Running   0             10m   192.168.1.13    k8smaster3   <none>           <none>
calico-node-6qj8b                          1/1     Running   0             10m   192.168.1.16    k8snode3     <none>           <none>
calico-node-f67dj                          1/1     Running   0             10m   192.168.1.15    k8snode2     <none>           <none>
calico-node-jhwcv                          1/1     Running   0             10m   192.168.1.12    k8smaster2   <none>           <none>
calico-node-ljqxq                          1/1     Running   0             10m   192.168.1.14    k8snode1     <none>           <none>
calico-node-t69zf                          1/1     Running   0             10m   192.168.1.11    k8smaster1   <none>           <none>
coredns-665d974787-5wcnk                   1/1     Running   0             24m   10.38.249.1     k8snode1     <none>           <none>
coredns-665d974787-jt95v                   1/1     Running   0             24m   10.38.249.3     k8snode1     <none>           <none>
etcd-k8smaster1                            1/1     Running   0             24m   192.168.1.11    k8smaster1   <none>           <none>
etcd-k8smaster2                            1/1     Running   0             15m   192.168.1.12    k8smaster2   <none>           <none>
etcd-k8smaster3                            1/1     Running   0             14m   192.168.1.13    k8smaster3   <none>           <none>
kube-apiserver-k8smaster1                  1/1     Running   0             24m   192.168.1.11    k8smaster1   <none>           <none>
kube-apiserver-k8smaster2                  1/1     Running   0             15m   192.168.1.12    k8smaster2   <none>           <none>
kube-apiserver-k8smaster3                  1/1     Running   1 (14m ago)   15m   192.168.1.13    k8smaster3   <none>           <none>
kube-controller-manager-k8smaster1         1/1     Running   1 (14m ago)   24m   192.168.1.11    k8smaster1   <none>           <none>
kube-controller-manager-k8smaster2         1/1     Running   0             15m   192.168.1.12    k8smaster2   <none>           <none>
kube-controller-manager-k8smaster3         1/1     Running   0             15m   192.168.1.13    k8smaster3   <none>           <none>
kube-proxy-8dfbs                           1/1     Running   0             14m   192.168.1.15    k8snode2     <none>           <none>
kube-proxy-ckvgv                           1/1     Running   0             14m   192.168.1.14    k8snode1     <none>           <none>
kube-proxy-qmkbp                           1/1     Running   0             15m   192.168.1.13    k8smaster3   <none>           <none>
kube-proxy-spsll                           1/1     Running   0             24m   192.168.1.11    k8smaster1   <none>           <none>
kube-proxy-v7vdb                           1/1     Running   0             13m   192.168.1.16    k8snode3     <none>           <none>
kube-proxy-wvwjp                           1/1     Running   0             15m   192.168.1.12    k8smaster2   <none>           <none>
kube-scheduler-k8smaster1                  1/1     Running   1 (14m ago)   24m   192.168.1.11    k8smaster1   <none>           <none>
kube-scheduler-k8smaster2                  1/1     Running   0             15m   192.168.1.12    k8smaster2   <none>           <none>
kube-scheduler-k8smaster3                  1/1     Running   0             15m   192.168.1.13    k8smaster3   <none>           <none>
```

Step 12 (Optional) Perform reset upon an initialization error.

If an error is reported when you run the **kubeadm init** command, reset the initialization and then perform the initialization again. If an error is reported when you run the **kubeadm join** command, reset the join operations, but this will delete the certificate, requiring you to transfer the certificate from the **k8smaster1** node again.

```
kubeadm reset -f
```

Step 13 (Optional) Regenerate a token if the token expires.

```
kubeadm token create --print-join-command
```

1.5 Result Verification

Step 1 Create a Deployment.

On the **k8smaster1** node, use the following YAML file to create a Deployment:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  creationTimestamp: null
  labels:
    app: busybox
  name: busybox
spec:
  replicas: 2
  selector:
    matchLabels:
      app: busybox
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: busybox
    spec:
      containers:
      - image: busybox:latest
        name: busybox
        command: ["sleep","99d"]
```

After the creation is complete, check whether the pods are normal. If the following information is displayed, the pods are normal:

```
[root@master01 ~]# kubectl get pod -o wide
NAME                       READY   STATUS    RESTARTS   AGE   IP
busybox-9dc75d45c-6dlw7    1/1     Running   0          50m   10.31.185.193
busybox-9dc75d45c-dfqdb    1/1     Running   0          50m   10.89.193.67
```

2 Network Components

2.1 Introduction

2.1.1 About This Exercise

This lab describes how to change the Calico network mode, use an IP address pool on the Kubernetes cluster network, and adjust network policies to implement namespace- and label-based traffic control.

2.1.2 Objectives

- Change the Calico network mode.
- Create and use an IP address pool.
- Master the namespace-based traffic control.
- Master the label-based traffic control.

2.2 Basic Operations of Calico

Step 1 View basic information about Calico.

Run the following command to view the resource object **ippool** related to Calico:

```
[root@master01 ~]# kubectl get ippool
```

```
[root@master01 ~]# kubectl get ippool
NAME                  AGE
default-ipv4-ippool   40h
```

Run the following command to view the details about **ippool**:

```
[root@master01 ~]# kubectl get ippool default-ipv4-ippool -o yaml
```

```
[root@master01 ~]# kubectl get ippool default-ipv4-ippool -o yaml
apiVersion: crd.projectcalico.org/v1
kind: IPPool
metadata:
  annotations:
    projectcalico.org/metadata: '{"uid":"ca33f0d2-54d2-499e-b957-c
  creationTimestamp: "2024-01-22T08:43:39Z"
  generation: 1
  name: default-ipv4-ippool
  resourceVersion: "5117"
  uid: 5c49a4ba-4f04-46c9-bfad-ba4424944ac0
spec:
  allowedUses:
  - Workload
  - Tunnel
  blockSize: 26
  cidr: 10.0.0.0/8
  ipipMode: Always
  natOutgoing: true
  nodeSelector: all()
  vxlanMode: Never
```

Step 2 Capture packets to check the data packet forwarding path.

Run the **ping pod busybox-9dc75d45c-6dlw7** command on the **k8smaster1** node and run the following command on the pod node to capture packets (install the tcpdump utility by yourself):

```
[root@node1 ~]# tcpdump -i eth0 -nn ip host 192.168.1.15 and host 192.168.1.11
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens192, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:54:36.138722 IP 192.168.1.11 > 192.168.1.15: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 5, length 64
09:54:36.138811 IP 192.168.1.15 > 192.168.1.11: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 5, length 64
09:54:37.162851 IP 192.168.1.11 > 192.168.1.15: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 6, length 64
09:54:37.162951 IP 192.168.1.15 > 192.168.1.11: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 6, length 64
09:54:38.186733 IP 192.168.1.11 > 192.168.1.15: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 7, length 64
09:54:38.186826 IP 192.168.1.15 > 192.168.1.11: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 7, length 64
```

- Question: What conclusion can you draw from the captured packets?
- Answer: In IP-in-IP (IPIP) mode, data packets are encapsulated on the host network and then routed. After reaching the destination node, the data packets are decapsulated and then forwarded to the corresponding pod.

Step 3 Use the calicoctl tool.

Run the following command on the **k8smaster1** node and view the Calico version:

```
[root@k8smaster1 ~]# kubectl describe deployment calico-kube-controllers -n kube-system
```

```
[root@k8smaster1 ~]# kubectl describe deployment calico-kube-controllers -n kube-system
Name:                   calico-kube-controllers
Namespace:              kube-system
CreationTimestamp:      Wed, 15 Nov 2023 07:30:19 +0000
Labels:                 k8s-app=calico-kube-controllers
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               k8s-app=calico-kube-controllers
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           Recreate
MinReadySeconds:        0
Pod Template:
  Labels:               k8s-app=calico-kube-controllers
  Service Account:      calico-kube-controllers
  Containers:
   calico-kube-controllers:
    Image:      docker.io/calico/kube-controllers:v3.23.5
    Port:       <none>
    Host Port:  <none>
    Liveness:   exec [/usr/bin/check-status -l] delay=10s timeout=10s period=10s #success=1 #failure=6
    Readiness:  exec [/usr/bin/check-status -r] delay=0s timeout=1s period=10s #success=1 #failure=3
    Environment:
      ENABLED_CONTROLLERS:  node
      DATASTORE_TYPE:       kubernetes
    Mounts:                 <none>
  Volumes:                  <none>
  Priority Class Name:      system-cluster-critical
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   calico-kube-controllers-54756b744f (1/1 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  28m   deployment-controller  Scaled up replica set calico-kube-controllers-54756b744f to 1
```

Step 4 Install calicoctl.

Download the calicoctl whose version is the same as the Calico version and copy it to the **/usr/local/bin** directory.

```
[root@k8smaster1 ~]# git clone https://gitee.com/yftyxa/hcie-calicoctl.git
[root@k8smaster1 ~]# cd hcie-calicoctl
[root@k8smaster1 ~]# cp calicoctl /usr/local/bin/
```

```
[root@k8smaster1 bin]# curl -L https://github.com/projectcalico/calico/releases/download/v3.23.5/calicoctl-linux-amd64 -o calicoctl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 56.5M  100 56.5M    0     0   728k      0  0:01:19  0:01:19 --:--:-- 5803k
```

Add the execute permission.

```
[root@k8smaster1 ~]# chmod +x /usr/local/bin/calicoctl
```

View the version.

```
[root@k8smaster1 ~]# calicoctl version
```

```
[root@k8smaster1 bin]# calicoctl version
Client Version:    v3.23.5
Git commit:        9e0398360
Cluster Version:   v3.23.5
Cluster Type:      k8s,bgp,kubeadm,kdd
```

Step 5 View node information.

```
[root@k8smaster1 ~]# calicoctl get node
```

```
[root@k8smaster1 bin]# calicoctl get node
NAME
k8smaster1
k8smaster2
k8smaster3
k8snode1
k8snode2
k8snode3
```

You can see that Calico is installed on each Kubernetes node.

Step 6 View the current IP address pool.

```
[root@k8smaster1 ~]# calicoctl get ippool
```

```
[root@k8smaster1 ~]# calicoctl get ippool
NAME                  CIDR          SELECTOR
default-ipv4-ippool   10.0.0.0/8    all()
```

The default IP address pool is the CIDR network segment configured during Kubernetes installation.

Step 7 View the current network mode.

```
[root@k8smaster1 ~]# calicoctl get ippool -o wide
```

The current network mode is IPIP.

```
[root@k8smaster1 ~]# calicoctl get ippool -o wide
NAME                  CIDR          NAT    IPIPMODE   VXLANMODE   DISABLED   DISABLEBGPEXPORT   SELECTOR
default-ipv4-ippool   10.0.0.0/8    true   Always     Never       false      false              all()
```

Step 8 Check the current node status.

```
[root@k8smaster1 ~]# calicoctl node status
```

```
[root@k8smaster1 ~]# calicoctl node status
Calico process is running.

IPv4 BGP status
+--------------+-------------------+-------+----------+-------------+
| PEER ADDRESS |     PEER TYPE     | STATE |  SINCE   |    INFO     |
+--------------+-------------------+-------+----------+-------------+
| 192.168.0.14 | node-to-node mesh | up    | 07:31:25 | Established |
| 192.168.0.16 | node-to-node mesh | up    | 07:31:25 | Established |
| 192.168.0.15 | node-to-node mesh | up    | 07:31:34 | Established |
| 192.168.0.13 | node-to-node mesh | up    | 07:31:53 | Established |
| 192.168.0.12 | node-to-node mesh | up    | 07:36:09 | Established |
+--------------+-------------------+-------+----------+-------------+

IPv6 BGP status
No IPv6 peers found.
```

node-to-node mesh indicates that Calico uses the BGP full-mesh mode by default.

Step 9 View node connection information.

```
[root@k8smaster1 ~]# netstat -anp | grep ESTABLISH | grep bird
```

```
[root@k8smaster1 ~]# netstat -anp | grep ESTABLISH | grep bird
tcp        0      0 192.168.0.11:179        192.168.0.16:60631      ESTABLISHED 10486/bird
tcp        0      0 192.168.0.11:179        192.168.0.14:58873      ESTABLISHED 10486/bird
tcp        0      0 192.168.0.11:179        192.168.0.13:38153      ESTABLISHED 10486/bird
tcp        0      0 192.168.0.11:179        192.168.0.15:41729      ESTABLISHED 10486/bird
tcp        0      0 192.168.0.11:179        192.168.0.12:41141      ESTABLISHED 10486/bird
```

The Kubernetes cluster has six nodes. Therefore, five connections can be queried on each node, indicating that every two nodes are connected.

Step 10 View IP address pool details.

```
[root@k8smaster1 ~]# kubectl describe ippools
```

```
[root@k8smaster1 ~]# kubectl describe ippools
Name:         default-ipv4-ippool
Namespace:
Labels:       <none>
Annotations:  projectcalico.org/metadata: {"uid":"5cf8d98f-1
API Version:  crd.projectcalico.org/v1
Kind:         IPPool
Metadata:
  Creation Timestamp:  2023-11-15T07:30:55Z
  Generation:          1
  Managed Fields:
    API Version:  crd.projectcalico.org/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:metadata:
        f:annotations:
          .:
          f:projectcalico.org/metadata:
      f:spec:
        .:
        f:allowedUses:
        f:blockSize:
        f:cidr:
        f:ipipMode:
        f:natOutgoing:
        f:nodeSelector:
        f:vxlanMode:
    Manager:         Go-http-client
    Operation:       Update
    Time:            2023-11-15T07:30:55Z
  Resource Version:  2232
  UID:               42970772-0d99-4ce4-a826-6f46ecc0d969
Spec:
  Allowed Uses:
    Workload
    Tunnel
  Block Size:     26
  Cidr:           10.0.0.0/8
  Ipip Mode:      Always
  Nat Outgoing:   true
  Node Selector:  all()
  Vxlan Mode:     Never
Events:           <none>
```

You can use kubectl to view details about the IP address pool, including the IP address range and Calico mode.

2.3 IP Address Pool

Step 1 View **kind**.

```
[root@k8smaster1 ~]# kubectl api-resources -o wide|grep calico|grep IPPool
```

```
[root@k8smaster1 bin]# kubectl api-resources -o wide|grep calico|grep IPPool
ippools                                        crd.projectcalico.org/v1              false        IPPool
```

The **crd.projectcalico.org/v1** queried can be used to create an IP address pool using kubectl.

Step 2 Create a working directory.

```
[root@k8smaster1 ~]# mkdir /root/network
[root@k8smaster1 ~]# cd /root/network
[root@k8smaster1 network]#
```

Step 3 Use calicoctl to create an IP address pool for a pod.

Note that the network segment cannot conflict with the existing pod network segment. The YAML file is as follows:

```
cat > pool1.yaml <<EOF
apiVersion: projectcalico.org/v3
kind: IPPool
metadata:
  name: pool1
spec:
  cidr: 11.0.0.0/8
  ipipMode: Always
  natOutgoing: true
  nodeSelector: all()
EOF
calicoctl create -f pool1.yaml
```

```
[root@k8smaster1 network]# calicoctl create -f pool1.yaml
Successfully created 1 'IPPool' resource(s)
```

Step 4 Use kubectl to create an IP address pool for a pod.

The YAML file is as follows. The value of **aipVersion** must be the same as that of **kind**.

```
cat > pool2.yaml <<EOF
apiVersion: crd.projectcalico.org/v1
kind: IPPool
metadata:
  name: pool2
spec:
  cidr: 12.0.0.0/8
  ipipMode: Always
  natOutgoing: true
  nodeSelector: all()
EOF
kubectl create -f pool2.yaml
```

```
[root@k8smaster1 network]# kubectl create -f pool2.yaml
ippool.crd.projectcalico.org/pool2 created
```

Step 5 View the created IP address pools.

```
[root@k8smaster1 network]# calicoctl get ippool
```

```
[root@k8smaster1 network]# calicoctl get ippool
NAME                  CIDR          SELECTOR
default-ipv4-ippool   10.0.0.0/8    all()
pool1                 11.0.0.0/8    all()
pool2                 12.0.0.0/8    all()
```

Check the IP address usage of the IP address pools.

```
[root@k8smaster1 network]# calicoctl ipam show
```

```
[root@k8smaster1 network]# calicoctl ipam show
+----------+------------+------------+------------+--------------------+
| GROUPING |    CIDR    | IPS TOTAL  | IPS IN USE |      IPS FREE      |
+----------+------------+------------+------------+--------------------+
| IP Pool  | 10.0.0.0/8 | 1.6777e+07 | 9 (0%)     | 1.6777e+07 (100%) |
| IP Pool  | 11.0.0.0/8 | 1.6777e+07 | 0 (0%)     | 1.6777e+07 (100%) |
| IP Pool  | 12.0.0.0/8 | 1.6777e+07 | 0 (0%)     | 1.6777e+07 (100%) |
+----------+------------+------------+------------+--------------------+
```

Step 6 Create Deployments for specified IP address pools.

Create two Deployments that use IP address pools **pool1** and **pool2**, respectively.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: busybox
  name: busybox-11
spec:
  replicas: 2
  selector:
    matchLabels:
      app: busybox
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: busybox
      annotations:
        cni.projectcalico.org/ipv4pools : "[\"pool1\"]"
    spec:
      containers:
      - image: busybox:latest
        name: busybox
```

```
          command: ["sleep","99d"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: busybox
  name: busybox-12
spec:
  replicas: 2
  selector:
    matchLabels:
      app: busybox
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: busybox
      annotations:
        cni.projectcalico.org/ipv4pools : "[\"pool2\"]"
    spec:
      containers:
      - image: busybox:latest
        name: busybox
        command: ["sleep","99d"]
```

Verify that the Deployments belong to different IP address pools.

```
NAME                          READY   STATUS    RESTARTS   AGE    IP
busybox-11-547c4d448-cpx42    1/1     Running   0          104s   11.224.169.192
busybox-11-547c4d448-lctbl    1/1     Running   0          104s   11.89.193.64
busybox-12-6b54df5b95-2pbrz   1/1     Running   0          104s   12.31.185.192
busybox-12-6b54df5b95-dmdcs   1/1     Running   0          104s   12.224.169.192
busybox-9dc75d45c-6dlw7       1/1     Running   0          90m    10.31.185.193
busybox-9dc75d45c-dfqdb       1/1     Running   0          90m    10.89.193.67
```

Step 7 Test network connectivity.

Log in to a pod and ping pods in other network segments.

```
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: tunl0@NONE: <NOARP> mtu 1480 qdisc noop qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
4: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1480 qdisc noqueue
    link/ether 06:cb:f1:18:20:d6 brd ff:ff:ff:ff:ff:ff
    inet 10.31.185.193/32 scope global eth0
       valid_lft forever preferred_lft forever
/ # ping 11.224.169.192
PING 11.224.169.192 (11.224.169.192): 56 data bytes
64 bytes from 11.224.169.192: seq=0 ttl=62 time=0.486 ms
64 bytes from 11.224.169.192: seq=1 ttl=62 time=0.321 ms
^C
--- 11.224.169.192 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.321/0.403/0.486 ms
/ # ping 12.31.185.192
PING 12.31.185.192 (12.31.185.192): 56 data bytes
64 bytes from 12.31.185.192: seq=0 ttl=63 time=0.103 ms
64 bytes from 12.31.185.192: seq=1 ttl=63 time=0.089 ms
^C
--- 12.31.185.192 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.089/0.096/0.103 ms
```

Step 8 View the current routes.

```
default via 192.168.1.1 dev ens192 proto static metric 100
10.31.185.192/26 via [redacted]5 dev tunl0 proto bird onlink
10.89.193.64/26 via [redacted] dev tunl0 proto bird onlink
10.146.95.192/26 via [redacted]3 dev tunl0 proto bird onlink
10.183.209.192/26 via [redacted]12 dev tunl0 proto bird onlink
10.224.169.192/26 via [redacted]14 dev tunl0 proto bird onlink
blackhole 10.239.112.128/26 proto bird
11.89.193.64/26 via [redacted]6 dev tunl0 proto bird onlink
11.224.169.192/26 via [redacted]14 dev tunl0 proto bird onlink
12.31.185.192/26 via [redacted]5 dev tunl0 proto bird onlink
12.224.169.192/26 via [redacted]14 dev tunl0 proto bird onlink
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.1.0/24 dev ens192 proto kernel scope link src 192.168.1.111 metric 100
```

In the current mode, **tun10** is used as the next-hop interface to other network segments.

2.4 VXLAN Mode

Step 1 Change modes.

Run the following command to change the network modes of **pool1** and **pool2**. Change the value of **ipipMode** to **Never** and that of **vxlanMode** to **Always**. Manually add the **vxlanMode** field of **pool2**.

```
[root@k8smaster1 network]# kubectl edit ippool
```

```
- apiVersion: crd.projectcalico.org/v1
  kind: IPPool
  metadata:
    annotations:
      projectcalico.org/metadata: '{"uid":"a92914a
    creationTimestamp: "2023-11-15T09:55:12Z"
    generation: 1
    name: pool1
    resourceVersion: "19184"
    uid: b6fe7285-bc66-42a8-961f-faaa474ee515
  spec:
    allowedUses:
    - Workload
    - Tunnel
    blockSize: 26
    cidr: 11.0.0.0/8
    ipipMode: Never
    natOutgoing: true
    nodeSelector: all()
    vxlanMode: Always
- apiVersion: crd.projectcalico.org/v1
  kind: IPPool
  metadata:
    creationTimestamp: "2023-11-15T09:59:42Z"
    generation: 1
    name: pool2
    resourceVersion: "19959"
    uid: 99202bf8-4ed1-4b25-a823-0ca6a259268b
  spec:
    cidr: 12.0.0.0/8
    ipipMode: Never
    natOutgoing: true
    nodeSelector: all()
    vxlanMode: Always
kind: List
metadata: {}
```

Step 2 View NIC information.

Run the following command and view the NIC information:

```
[root@k8smaster1 network]# ip add
```

```
11: vxlan.calico: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 14
    link/ether 66:96:f8:39:99:45 brd ff:ff:ff:ff:ff:ff
    inet 11.90.93.64/32 scope global vxlan.calico
       valid_lft forever preferred_lft forever
    inet6 fe80::6496:f8ff:fe39:9945/64 scope link
       valid_lft forever preferred_lft forever
```

The **vxlan.calico** information is generated in the NIC.

Step 3 View route information.

Run the following command and verify that the egress of the **pool1** and **pool2** network segments is **vxlan.calico**:

```
[root@k8smaster1 network]# route -n
```

```
[root@k8smaster1 network]# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.1.1     0.0.0.0         UG    101    0        0 ens3
10.4.185.0      192.168.0.16    255.255.255.192 UG    0      0        0 tunl0
10.38.249.0     192.168.0.14    255.255.255.192 UG    0      0        0 tunl0
10.76.185.192   192.168.0.15    255.255.255.192 UG    0      0        0 tunl0
10.86.137.64    192.168.0.12    255.255.255.192 UG    0      0        0 tunl0
10.90.93.64     0.0.0.0         255.255.255.192 U     0      0        0 *
10.99.63.192    192.168.0.13    255.255.255.192 UG    0      0        0 tunl0
11.4.185.0      11.4.185.5      255.255.255.192 UG    0      0        0 vxlan.calico
11.38.249.0     11.38.249.3     255.255.255.192 UG    0      0        0 vxlan.calico
11.86.137.64    11.86.137.64    255.255.255.192 UG    0      0        0 vxlan.calico
11.90.93.64     0.0.0.0         255.255.255.192 U     0      0        0 *
11.99.63.192    11.99.63.192    255.255.255.192 UG    0      0        0 vxlan.calico
12.76.185.192   12.76.185.198   255.255.255.192 UG    0      0        0 vxlan.calico
169.254.169.254 192.168.1.1     255.255.255.255 UGH   101    0        0 ens3
172.17.0.0      0.0.0.0         255.255.0.0     U     0      0        0 docker0
192.168.0.0     0.0.0.0         255.255.255.0   U     100    0        0 enp4s4
192.168.1.0     0.0.0.0         255.255.255.0   U     101    0        0 ens3
```

Step 4 Test network connectivity.

View pods.

```
[root@k8smaster1 network]# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS        AGE    IP              NODE
ippool-pod1   1/1     Running   4 (2m50s ago)   4h2m   11.4.185.1      k8snode3
ippool-pod2   1/1     Running   4 (2m50s ago)   4h2m   12.76.185.193   k8snode2
```

Log in to a pod and ping pods in different network segments of different nodes.

```
[root@k8smaster1 network]# kubectl exec -it ippool-pod1 sh
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a
/ #
/ # ping 12.76.185.193
PING 12.76.185.193 (12.76.185.193): 56 data bytes
64 bytes from 12.76.185.193: seq=0 ttl=62 time=0.338 ms
64 bytes from 12.76.185.193: seq=1 ttl=62 time=0.334 ms
^C
--- 12.76.185.193 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.334/0.336/0.338 ms
/ # exit
```

Delete the test pods.

```
[root@k8smaster1 network]# calicoctl delete -f pool1.yaml
[root@k8smaster1 network]# kubectl delete -f pool2.yaml
```

2.5 Fixed IP Address

Step 1 Deploy a pod with a fixed IP address.

Deploy a pod with an idle fixed IP address. Example:

```
root@k8smaster1 network]# cat > fixed-ip-pod.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: fixed-ip-pod
```

```
  annotations:
    cni.projectcalico.org/ipAddrs : "[\"11.1.1.11\"]"
spec:
  containers:
  - name: fixed-ip-pod
    image: busybox
    imagePullPolicy: IfNotPresent
    command: ["sleep", "3600"]
EOF
kubectl apply -f fixed-ip-pod.yaml
```

Step 2 View the result.

```
[root@k8smaster1 network]# kubectl get pods -o wide
NAME           READY   STATUS    RESTARTS      AGE     IP              NODE
fixed-ip-pod   1/1     Running   0             16m     11.1.1.11       k8snode2
```

Delete the test pod and IP address pool.

```
[root@k8smaster1 network]# kubectl delete -f fixed-ip-pod.yaml
[root@k8smaster1 network]# kubectl delete -f ippool-pod.yaml
```

2.6 BGP Mode

Step 1 Change the mode.

Modify **pool1.yaml** in step 3 in section 2.3 to the following content in bold:

```
apiVersion: projectcalico.org/v3
kind: IPPool
metadata:
  name: pool1
spec:
  cidr: 11.0.0.0/8
  ipipMode: CrossSubnet
  natOutgoing: true
  nodeSelector: all()
```

Run the **calicoctl apply -f /root/network/pool1.yaml** command to make the configuration take effect.

```
[root@master01 network]# kubectl apply -f pool1.yaml
ippool.crd.projectcalico.org/pool1 configured
```

Step 2 Check the modification.

Run the **ip route list** command and check the corresponding route as follows:

```
[root@master01 network]# ip route list
default via 192.168.1.1 dev ens192 proto static metric 100
10.31.185.192/26 via [redacted] dev tunl0 proto bird onlink
10.89.193.64/26 via [redacted] dev tunl0 proto bird onlink
10.146.95.192/26 via [redacted] dev tunl0 proto bird onlink
10.183.209.192/26 vi[redacted]2 dev tunl0 proto bird onlink
10.224.169.192/26 vi[redacted]4 dev tunl0 proto bird onlink
blackhole 10.239.112[redacted] bird
11.89.193.64/26 via [redacted] dev ens192 proto bird
11.224.169.192/26 vi[redacted]4 dev ens192 proto bird
12.31.185.192/26 via [redacted] dev tunl0 proto bird onlink
12.224.169.192/26 vi[redacted]4 dev tunl0 proto bird onlink
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.1.0/24 dev ens192 proto kernel scope link src 192.168.1.111 metric 100
```

The preceding output indicates that the next-hop interface of the route related to **pool1** becomes the physical interface of the node, not **tunl0**.

Step 3 Capture packets to check the data forwarding path.

On the master node, ping the pod in the 11.0.0.0/24 network segment and capture packets on the node according to step 2 in section 2.2. The following traffic is obtained:

```
[root@node01 ~]# tcpdump -i ens192 -nn ip host 11.224.169.192
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens192, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:23:45.578527 IP 192.168.1.11 > 11.224.169.192: ICMP echo request, id 5031, seq 10, length 64
14:23:45.578595 IP 11.224.169.192 > 192.168.1.11: ICMP echo reply, id 5031, seq 10, length 64
14:23:46.602534 IP 192.168.1.11 > 11.224.169.192: ICMP echo request, id 5031, seq 11, length 64
14:23:46.602608 IP 11.224.169.192 > 192.168.1.11: ICMP echo reply, id 5031, seq 11, length 64
14:23:47.626564 IP 192.168.1.11 > 11.224.169.192: ICMP echo request, id 5031, seq 12, length 64
14:23:47.626639 IP 11.224.169.192 > 192.168.1.11: ICMP echo reply, id 5031, seq 12, length 64
```

The data does not need to be encapsulated and can be directly forwarded.

Step 4 Deploy a route reflector (RR).

Compile the following YAML file for deploying an RR:

```
apiVersion: projectcalico.org/v3
kind: Node
metadata:
  labels:
    route-reflector: true
  name: k8smaster1
spec:
  bgp:
    ipv4Address: 192.168.1.11/24
    ipv4IPIPTunnelAddr: 10.239.112.128
    routeReflectorClusterID: 1.1.1.1
```

Run the **calico apply -f rr.yaml** command to deploy the RR.

Step 5 Change the mode of all nodes.

Compile the following YAML file for changing the mode of all nodes:

```
kind: BGPPeer
apiVersion: projectcalico.org/v3
metadata:
  name: bgppeer
spec:
  nodeSelector: all()
  peerSelector: route-reflector=="true"
```

Run the **calico apply -f peer.yaml** command to make the configuration take effect.

Step 6 Check the node status after deployment.

Check the status of all nodes on the RR node. The details are as follows:

```
[root@master01 network]# calicoctl node status
Calico process is running.

IPv4 BGP status
+--------------+--------------+-------+----------+-------------+
| PEER ADDRESS |   PEER TYPE  | STATE |  SINCE   |    INFO     |
+--------------+--------------+-------+----------+-------------+
| 192.168.1.2  | node specific | up    | 06:56:58 | Established |
| 192.168.1.3  | node specific | up    | 06:56:58 | Established |
| 192.168.1.4  | node specific | up    | 06:56:58 | Established |
| 192.168.1.5  | node specific | up    | 06:56:58 | Established |
| 192.168.1.6  | node specific | up    | 06:56:58 | Established |
+--------------+--------------+-------+----------+-------------+
```

The command output shows that **PEER TYPE** has been changed from **node-to-node mesh** to **node specific**, indicating that the mode has been changed to RR.

Run the same command on non-RR nodes. The details are as follows:

```
IPv4 BGP status
+--------------+-------------------+-------+------------+-------------+
| PEER ADDRESS |     PEER TYPE     | STATE |   SINCE    |    INFO     |
+--------------+-------------------+-------+------------+-------------+
| 192.168.1.2  | node-to-node mesh | up    | 2024-01-22 | Established |
| 192.168.1.3  | node-to-node mesh | up    | 2024-01-22 | Established |
| 192.168.1.5  | node-to-node mesh | up    | 2024-01-22 | Established |
| 192.168.1.6  | node-to-node mesh | up    | 2024-01-22 | Established |
| 192.168.1.1  | node specific     | up    | 06:56:58   | Established |
+--------------+-------------------+-------+------------+-------------+
```

The command output shows that the local node has established a peer relationship with the RR node in RR mode, but the local node still learns routes from other nodes in node-to-node mesh mode.

Step 7 Disable the default mode.

Compile the following YAML file:

```
apiVersion: projectcalico.org/v3
kind: BGPConfiguration
metadata:
  name: default
```

```
spec:
  logSeverityScreen: Info
  nodeToNodeMeshEnabled: false
  asNumber: 63400
```

Run the **calicoctl apply -f no-default.yaml** command to make the configuration take effect. Then, check the node status on non-RR nodes.

```
+---------------+---------------+-------+----------+-------------+
| PEER ADDRESS  |   PEER TYPE   | STATE |  SINCE   |    INFO     |
+---------------+---------------+-------+----------+-------------+
| 192.168.1.[redacted]1 | node specific | up    | 07:16:02 | Established |
+---------------+---------------+-------+----------+-------------+
```

The command output shows that the peer relationships between the node and other nodes are interrupted and only the peer relationship between the node and the RR node is retained.

- Question: What are the advantages of establishing BGP peering only between non-RR nodes and RR nodes?
- Answer: This mode greatly reduces the number of peer entries and the performance loss caused by peer maintenance.

2.7 Namespace-based Access Control

Step 1 Create test namespaces.

Run the following commands on the **k8smaster1** node to create two namespaces:

```
[root@k8smaster1 network]# kubectl create ns test1
[root@k8smaster1 network]# kubectl create ns test2
```

Step 2 Create two pods.

```
[root@k8smaster1 network]# kubectl run nginx1 --image=nginx:1.15-alpine -n test1
[root@k8smaster1 network]# kubectl run nginx2 --image=nginx:1.15-alpine -n test2
```

View the IP addresses of the pods.

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test1
```

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test1
NAME     READY   STATUS    RESTARTS   AGE   IP              NODE
nginx1   1/1     Running   0          10s   10.76.185.196   k8snode2
```

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test2
```

```
[root@k8smaster1 network]#  kubectl get pods -o wide -n test2
NAME     READY   STATUS    RESTARTS   AGE   IP           NODE
nginx2   1/1     Running   0          23s   10.4.185.2   k8snode3
```

Step 3 Connect the network of pods in different namespaces.

Test the pod network connectivity. Change the command based on the IP addresses queried in the previous step. Use the actual IP address in the **ping** command in subsequent operations of this lab. Do not directly copy the example command. Example:

```
[root@k8smaster1 network]# kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
```

```
[root@k8smaster1 network]# kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
PING 10.4.185.2 (10.4.185.2): 56 data bytes
64 bytes from 10.4.185.2: seq=0 ttl=62 time=0.352 ms
64 bytes from 10.4.185.2: seq=1 ttl=62 time=0.287 ms
```

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes
64 bytes from 10.76.185.196: seq=0 ttl=62 time=0.290 ms
64 bytes from 10.76.185.196: seq=1 ttl=62 time=0.259 ms
```

Step 4 Create a network policy that denies incoming traffic.

```
root@k8smaster1 network]# cat > netpolicy1.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: netpolicy1
spec:
  podSelector: {}
  policyTypes:
  - Ingress
EOF
kubectl apply -f netpolicy1.yaml -n test1
```

View the network policy.

```
[root@k8smaster1 network]# kubectl get netpol -n test1
```

```
[root@k8smaster1 network]# kubectl get netpol -n test1
NAME         POD-SELECTOR   AGE
netpolicy1   <none>         8s
```

Step 5 Verify that outgoing traffic is allowed.

Verify that the **test1** pod can ping the **test2** pod. For **test1**, outgoing traffic is allowed.

```
[root@k8smaster1 network]# kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
```

```
[root@k8smaster1 network]# kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
PING 10.4.185.2 (10.4.185.2): 56 data bytes
64 bytes from 10.4.185.2: seq=0 ttl=62 time=0.296 ms
64 bytes from 10.4.185.2: seq=1 ttl=62 time=0.288 ms
```

Step 6 Verify that incoming traffic is denied.

Verify that the **test2** pod cannot ping the **test1** pod. For **test1**, incoming traffic is denied.

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes

--- 10.76.185.196 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Step 7 Create a network policy that denies outgoing traffic.

Add the egress restriction to the **netpolicy1.yaml** file.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: netpolicy 1
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Redeploy the policy.

```
[root@k8smaster1 network]# kubectl apply -f netpolicy1.yaml -n test1
```

Step 8 Verify that outgoing traffic is denied.

```
[root@k8smaster1 network]# kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
```

```
[root@k8smaster1 network]#  kubectl exec -it nginx1 -n test1 -- ping -c 2 10.4.185.2
PING 10.4.185.2 (10.4.185.2): 56 data bytes

--- 10.4.185.2 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Step 9 Delete the network policy.

Delete the network policy. Otherwise, the subsequent tasks will be affected.

```
[root@k8smaster1 network]# kubectl delete -f netpolicy1.yaml -n test1
```

2.8 Label-based Access Control

Step 1 Add two pods.

```
[root@k8smaster1 network]# kubectl run nginx3 --image=nginx:1.15-alpine -n test1
[root@k8smaster1 network]# kubectl run nginx4 --image=nginx:1.15-alpine -n test2
```

View their IP addresses.

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test1
```

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test1
NAME     READY   STATUS    RESTARTS   AGE     IP              NODE
nginx1   1/1     Running   0          116m    10.76.185.196   k8snode2
nginx3   1/1     Running   0          5m9s    10.4.185.3      k8snode3
```

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test2
```

```
[root@k8smaster1 network]# kubectl get pods -o wide -n test2
NAME     READY   STATUS    RESTARTS   AGE     IP              NODE
nginx2   1/1     Running   0          116m    10.4.185.2      k8snode3
nginx4   1/1     Running   0          5m21s   10.76.185.197   k8snode2
```

Step 2 Add a label to a pod.

Add a label to the **nginx1** pod.

```
[root@k8smaster1 network]# kubectl label pod nginx1 app=nginx1 -n test1
```

Step 3 Configure an allowlist.

Create a network policy that matches the **nginx1** label, and add the network segment setting at the ingress to allow access from the **10.0.0.0** network segment but deny access from **nginx2**. The IP address in **except** must be changed to the queried IP address. Do not directly copy the example configuration.

```
[root@k8smaster1 network]# cat > netpolicy2.yaml<<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: netpolicy2
spec:
  podSelector:
    matchLabels:
      app: nginx1
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 10.0.0.0/8
        except:
        - 10.4.185.2/32
EOF
kubectl apply -f netpolicy2.yaml -n test1
```

Step 4 Verify the allowed IP address range at the ingress.

nginx4 can ping **nginx1**.

```
[root@k8smaster1 network]# kubectl exec -it nginx4 -n test2 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]# kubectl exec -it nginx4 -n test2 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes
64 bytes from 10.76.185.196: seq=0 ttl=63 time=0.062 ms
64 bytes from 10.76.185.196: seq=1 ttl=63 time=0.060 ms

--- 10.76.185.196 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.060/0.061/0.062 ms
```

Step 5 Verify the denied IP address at the ingress.

nginx2 cannot ping **nginx1**.

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes

--- 10.76.185.196 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Step 6 Verify the label.

nginx2 can ping **nginx3**. Only **nginx1** has the **app=nginx1** label, and **nginx3** does not have this label.

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.4.185.3
```

```
[root@k8smaster1 network]# kubectl exec -it nginx2 -n test2 -- ping -c 2 10.4.185.3
PING 10.4.185.3 (10.4.185.3): 56 data bytes
64 bytes from 10.4.185.3: seq=0 ttl=63 time=0.085 ms
64 bytes from 10.4.185.3: seq=1 ttl=63 time=0.063 ms

--- 10.4.185.3 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.063/0.074/0.085 ms
```

- Question: Can **nginx1** ping **nginx2**? Why?
- Answer: No, it cannot because the egress policy is enabled for the network policy but no allowlist is configured.

Step 7 Modify the policy.

Modify the content in bold and delete the original **ipBlock** field from the **netpolicy2.yaml** file, so that only pods with the **access_nginx1=true** label in the same namespace can be accessed.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: netpolicy2
spec:
  podSelector:
    matchLabels:
      app: nginx1
  policyTypes:
  - Ingress
  - Egress
  ingress:
```

```
  - from:
    - podSelector:
        matchLabels:
          access_nginx1: "true"
```

Redeploy the **netpolicy2** policy.

```
kubectl apply -f netpolicy2.yaml -n test1
```

Step 8 Verify the label.

nginx3 cannot ping **nginx1**.

```
[root@k8smaster1 network]# kubectl exec -it nginx3 -n test1 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]# kubectl exec -it nginx3 -n test1 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes

--- 10.76.185.196 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Add a label to **nginx3**.

```
[root@k8smaster1 network]# kubectl label pod nginx3 access_nginx1=true -n test1
```

nginx3 can ping **nginx1**.

```
[root@k8smaster1 network]# kubectl exec -it nginx3 -n test1 -- ping -c 2 10.76.185.196
```

```
[root@k8smaster1 network]#  kubectl exec -it nginx3 -n test1 -- ping -c 2 10.76.185.196
PING 10.76.185.196 (10.76.185.196): 56 data bytes
64 bytes from 10.76.185.196: seq=0 ttl=62 time=0.293 ms
64 bytes from 10.76.185.196: seq=1 ttl=62 time=0.250 ms

--- 10.76.185.196 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.250/0.271/0.293 ms
```

Step 9 Clear the lab environment.

Delete the network policy and pods.

```
[root@k8smaster1 network]# kubectl delete -f netpolicy2.yaml -n test1
[root@k8smaster1 network]# kubectl delete pods nginx1 -n test1
[root@k8smaster1 network]# kubectl delete pods nginx2 -n test2
[root@k8smaster1 network]# kubectl delete pods nginx3 -n test1
[root@k8smaster1 network]# kubectl delete pods nginx4 -n test2
```

3 Persistent Storage

3.1 Introduction

3.1.1 About This Exercise

This lab describes how to connect Kubernetes to Ceph, including RADOS Block Devices (RBD) and Ceph File System (CephFS).

3.1.2 Objectives

- Mount RBD to a volume.
- Mount RBD to PersistentVolume (PV) and PersistentVolumeClaim (PVC).
- Mount RBD to StorageClass.
- Mount CephFS to StorageClass.

3.2 Resource Pool Creation

Step 1 Create an RBD resource pool.

Run the following commands to create an RBD resource pool **kubernetes**:

```
ceph osd pool create kubernetes
ceph osd pool application enable kubernetes rbd
ceph auth get-or-create client.kubernetes mon 'profile rbd' osd 'profile rbd pool=kubernetes' mgr
'profile rbd pool=kubernetes'
```

Step 2 Create a CephFS resource pool.

Run the following commands to create the resource pool and MDS required by CephFS:

```
ceph osd pool create kubernetes_data
ceph osd pool create kubernetes_metadata
ceph fs new kubernetes kubernetes_metadata kubernetes_data
ceph orch apply mds kubernetes --placement="2"
ceph fs authorize kubernetes client.kubernetes_ceph / rwps
```

3.3 ceph-csi File Preparation

Step 1 Download the standard ceph-csi file.

Run the following commands on **k8smaster1** to create a working directory:

```
[root@k8smaster1 ~]# mkdir /root/ceph
[root@k8smaster1 ~]# cd /root/ceph
[root@k8smaster1 ceph]# git clone https://gitee.com/yftyxa/ceph-csi.git
```

Step 2 Prepare required files.

Create a dedicated directory (for example, to connect to RBD) and copy required files to the directory. The commands are as follows:

```
[root@k8smaster1 ceph]# mkdir ~/ceph-rbd
[root@k8smaster1 ceph]# cp /root/ceph/ceph-csi/deploy/rbd/kubernetes/* ~/ceph-rbd/
[root@k8smaster1 ceph]# cp /root/ceph/ceph-csi/examples/rbd/secret.yaml ~/ceph-rbd/
[root@k8smaster1 ceph]# cd ~/ceph-rbd/
```

3.4 Kubernetes Call to RBD

Step 1 Create a namespace.

To facilitate subsequent observation, run the following command to create a dedicated namespace:

```
kubectl create ns csi
```

Step 2 Create a ConfigMap.

Run the following command on the Ceph node to query the information (in bold) required by the ConfigMap:

```
[ceph: root@ceph01 /]# ceph mon dump
epoch 3
fsid 88ab034c-a622-11ee-9a8c-000c2905539b
last_changed 2023-12-29T08:27:15.558834+0000
created 2023-12-29T08:16:48.302982+0000
min_mon_release 16 (pacific)
election_strategy: 1
0: [v2:192.168.1.11:3300/0,v1:192.168.1.11:6789/0] mon.ceph01
1: [v2:192.168.1.12:3300/0,v1:192.168.1.12:6789/0] mon.ceph02
2: [v2:192.168.1.13:3300/0,v1:192.168.1.13:6789/0] mon.ceph03
```

In the **ceph-rbd** directory, modify the **csi-config-map.yaml** file based on the following content (set **ceph-csi-config** based on the query result and do not directly copy the example value):

```
---
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: "ceph-csi-config"
data:
  config.json: |-
    [
      {
        "clusterID": "88ab034c-a622-11ee-9a8c-000c2905539b",
        "monitors": [
          "192.168.1.11:6789",
          "192.168.1.12:6789",
          "192.168.1.13:6789"
        ]
      }
    ]

---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    {}
metadata:
  name: ceph-csi-encryption-kms-config

---
apiVersion: v1
kind: ConfigMap
data:
  ceph.conf: |
    [global]
    auth_cluster_required = cephx
    auth_service_required = cephx
    auth_client_required = cephx
  keyring: |
metadata:
  name: ceph-config
```

After the modification, run the following command to deploy the ConfigMap:

```
kubectl apply -f csi-config-map.yaml -n csi
```

After the deployment is complete, you can query the following information:

```
[root@master01 ceph-csi]# kubectl get configmap -n csi
NAME                            DATA   AGE
ceph-config                     2      92m
ceph-csi-config                 1      92m
ceph-csi-encryption-kms-config  1      92m
kube-root-ca.crt                1      140m
```

Step 3 Create a secret.

On the Ceph node, query the information about the user of the connected RBD resource pool. The user is created in step 1 in section 3.2. The detailed information is as follows:

```
[ceph: root@ceph01 /]# ceph auth get client.kubernetes
[client.kubernetes]
        key = AQAtBbNl/mm7AhAAXeVQDRVQ+oK2JIv95ZbHHQ==
        caps mgr = "profile rbd pool=kubernetes"
        caps mon = "profile rbd"
        caps osd = "profile rbd pool=kubernetes"
exported keyring for client.kubernetes
```

userID and **userKey** need to be specified in the **secret.yaml** file. The following is an example **secret.yaml** file:

```
---
apiVersion: v1
kind: Secret
metadata:
  name: csi-rbd-secret
  namespace: csi
stringData:
  # Key values correspond to a user name and its key, as defined in the
  # ceph cluster. User ID should have required access to the 'pool'
  # specified in the storage class
  userID: kubernetes
  userKey: AQAtBbNl/mm7AhAAXeVQDRVQ+oK2JIv95ZbHHQ==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

Add a namespace. Then, use this file to deploy the secret. After the deployment is complete, verify that the secret information can be queried. The details are as follows:

```
[root@master01 ceph-csi]# kubectl get secret -n csi
NAME                  TYPE                 DATA   AGE
csi-rbd-secret        Opaque               3      115m
```

Step 4 Deploy ceph-csi.

Modify the namespace information in **csi-provisioner-rbac.yaml**, **csi-nodeplugin-rbac.yaml**, **csi-rbdplugin-provisioner.yaml**, and **csi-rbdplugin.yaml**. Change **default** to **csi**. You are advised to use sed to modify the namespace information in batches because there are multiple settings to be modified. After the modification, run the **grep** command and check whether the files contain **namespace: default**. If the modifications are correct and complete, run the following command to deploy ceph-csi:

```
kubectl apply -f csi-nodeplugin-rbac.yaml -f csi-provisioner-rbac.yaml -f csi-rbdplugin-provisioner.yaml -f csi-rbdplugin.yaml
```

After the operation is complete, you can query the Deployment related to ceph-csi. The details are as follows:

```
[root@master01 ceph-csi]# kubectl get deploy -n csi
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
csi-rbdplugin-provisioner    3/3     3            3           60m
```

You can also query the following DaemonSet:

```
[root@master01 ceph-csi]# kubectl get daemonset -n csi
NAME           DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
csi-rbdplugin  3         3         3       3            3           <none>          60m
```

Verify that the information is correct before proceeding to the next step.

Step 5 Mount storage.

Mount a raw device (block) or a file system to a pod. In this lab, a raw device is used as an example.

Create a StorageClass file **csi-rbd-sc.yaml** as follows:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
    name: csi-rbd-sc
    namespace: csi
provisioner: rbd.csi.ceph.com
parameters:
    clusterID: 88ab034c-a622-11ee-9a8c-000c2905539b
    pool: kubernetes
    imageFeatures: layering
    csi.storage.k8s.io/provisioner-secret-name: csi-rbd-secret
    csi.storage.k8s.io/provisioner-secret-namespace: csi
    csi.storage.k8s.io/controller-expand-secret-name: csi-rbd-secret
    csi.storage.k8s.io/controller-expand-secret-namespace: csi
    csi.storage.k8s.io/node-stage-secret-name: csi-rbd-secret
    csi.storage.k8s.io/node-stage-secret-namespace: csi
reclaimPolicy: Delete
allowVolumeExpansion: true
mountOptions:
    - discard
```

Note: The namespace in this file must be the same as that of other object resources. **secret** must be the same as that created in step 3 of this lab.

Run the **kubectl apply -f csi-rbd-sc.yaml** command for deployment.

Create a PVC file **raw-block-pvc.yaml** as follows:

```
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: raw-block-pvc
  namespace: csi
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
```

```
  resources:
    requests:
      storage: 1Gi
  storageClassName: csi-rbd-sc
```

Note: The value of **storageClassName** in this file must be the same as that in **csi-rbd-sc.yaml**.

Run the **kubectl apply -f raw-block-pvc.yaml** command for deployment.

Create a pod file **raw-block-pod.yaml** as follows:

```
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-raw-block-volume
  namespace: csi
spec:
  containers:
    - name: fc-container
      image: nginx
      volumeDevices:
        - name: data
          devicePath: /dev/xvda
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: raw-block-pvc
```

Run the **kubectl apply -f raw-block-pod.yaml** command for deployment.

Step 6 Verify the configuration results.

On the Kubernetes node, verify that the PVC and PV are bound. The details are as follows:

```
[root@master01 ceph-csi]# kubectl get pvc -n csi
NAME           STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   AGE
raw-block-pvc  Bound    pvc-bd53dba8-8f2c-4ebc-a6ba-20f1ac34d57a   1Gi        RWO            csi-rbd-sc     88m
```

On the Ceph node, verify that an image is created in the resource pool. The details are as follows:

```
[ceph: root@ceph01 /]# rbd -p kubernetes ls
csi-vol-98f25147-ffc6-4d14-aafa-51a21d4923cd
[ceph: root@ceph01 /]# rbd info kubernetes/csi-vol-98f25147-ffc6-4d14-aafa-51a21d4923cd
rbd image 'csi-vol-98f25147-ffc6-4d14-aafa-51a21d4923cd':
        size 1 GiB in 256 objects
        order 22 (4 MiB objects)
        snapshot_count: 0
        id: 7ca2dd30f0e9c
        block_name_prefix: rbd_data.7ca2dd30f0e9c
        format: 2
        features: layering
        op_features:
        flags:
        create_timestamp: Fri
        access_timestamp: Fri
        modify_timestamp: Fri
```

According to the results, the RBD resource is successfully connected to Kubernetes.

- Question: After the created pod and PVC are deleted, will the PV and the image in the corresponding RBD pool be automatically deleted?
- Answer: After the pod and PVC are deleted from Kubernetes, the image is automatically deleted. The details are as follows:

```
[root@master01 ceph-csi]# kubectl delete -f raw-block-pvc.yaml
persistentvolumeclaim "raw-block-pvc" deleted
[root@master01 ceph-csi]# kubectl   get pvc -n   csi
No resources found in csi namespace.
[root@master01 ceph-csi]# kubectl   get pv
No resources found
```

On the Ceph node, the corresponding image is automatically deleted.

3.5 Kubernetes Call to CephFS

Step 1 Create a namespace.

To facilitate subsequent observation, run the following command to create a dedicated namespace:

```
kubectl create ns cephfs
```

Step 2 Prepare required files.

Create a dedicated directory for storing required files by following instructions in section 3.3. The commands are as follows:

```
mkdir /root/cephfs
cd /root/cephfs
cp /root/ceph/ceph-csi/deploy/cephfs/kubernetes/* /root/cephfs/
cp /root/ceph/ceph-csi/examples/cephfs/secret.yaml /root/cephfs/
```

Step 3 Create a ConfigMap.

Reuse the ConfigMap file in step 2 in section 3.4. Specify the newly created namespace when creating the ConfigMap file.

```
kubectl apply -f /root/ceph-rbd/csi-config-map.yaml -n cephfs
```

Step 4 Create a secret.

On the Ceph node, query the information about the user of the connected RBD resource pool. The user is created in step 2 in section 3.2. The detailed information is as follows:

```
[ceph: root@ceph01 /]# ceph auth get client.kubernetes_ceph
[client.kubernetes_ceph]
        key = AQDuSrNlBAsIHBAAO1WOmXhTzHMDl+m3WpyRVg==
        caps mds = "allow rwps fsname=kubernetes"
        caps mon = "allow r fsname=kubernetes"
```

```
        caps osd = "allow rw tag cephfs data=kubernetes"
exported keyring for client.kubernetes_ceph
```

In the following part, PVs will be created dynamically. You need to obtain the information about the Ceph administrator. In this lab, the **admin** user is used as an example.

userID and **userKey** need to be specified in the **secret.yaml** file. The following is an example **secret.yaml** file:

```
---
apiVersion: v1
kind: Secret
metadata:
  name: csi-cephfs-secret
  namespace: cephfs
stringData:
  # Required for statically provisioned volumes
  userID: kubernetes_ceph
  userKey: AQDuSrNlBAsIHBAAO1WOmXhTzHMDl+m3WpyRVg==

  # Required for dynamically provisioned volumes
  adminID: admin
  adminKey: AQBvgI5lqanTKhAAwyKQaG7i340SPTMTLBF7kQ==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

Add a namespace. Then, use this file to deploy the secret. After the deployment is complete, verify that the secret information can be queried. The details are as follows:

```
[root@master01 cephfs]# kubectl get secret -n cephfs
NAME                  TYPE                                  DATA   AGE
csi-cephfs-secret     Opaque                                5      15s
```

Step 5 Deploy ceph-csi.

Modify the namespace information in **csi-provisioner-rbac.yaml**, **csi-nodeplugin-rbac.yaml**, **csi-rbdplugin-provisioner.yaml**, and **csi-rbdplugin.yaml**. Change **default** to **cephfs** or manually add **cephfs** for the namespaces that have not specified the setting. You are advised to use sed to modify the namespace information in batches because there are multiple settings to be modified. After the modification, run the **grep** command and check whether the files contain **namespace: default**. If the modifications are correct and complete, run the following command to deploy ceph-csi:

```
kubectl apply -f csi-nodeplugin-rbac.yaml -f csi-provisioner-rbac.yaml -f csi-cephfsplugin-provisioner.yaml -f csi-cephfsplugin.yaml
```

After the operation is complete, you can query the Deployment related to ceph-csi. The details are as follows:

```
[root@master01 cephfs]# kubectl get deploy -n cephfs
NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
csi-cephfsplugin-provisioner   3/3     3            3           36s
```

You can also query the following DaemonSet:

```
[root@master01 cephfs]# kubectl get daemonset -n cephfs
NAME              DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
csi-cephfsplugin  3         3         3       3            3           <none>          32s
```

Verify that the information is correct before proceeding to the next step.

Step 6 Mount storage.

PVs can be created statically or dynamically. This lab creates a PV dynamically as an example.

Create a StorageClass file **csi-cephfs-sc.yaml** as follows:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: csi-cephfs-sc
  namespace: cephfs
provisioner: cephfs.csi.ceph.com
parameters:
  clusterID: 88ab034c-a622-11ee-9a8c-000c2905539b
  fsName: kubernetes
  pool: kubernetes_data
  csi.storage.k8s.io/provisioner-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/provisioner-secret-namespace: cephfs
  csi.storage.k8s.io/controller-expand-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/controller-expand-secret-namespace: cephfs
  csi.storage.k8s.io/node-stage-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/node-stage-secret-namespace: cephfs
  mounter: kernel
reclaimPolicy: Delete
allowVolumeExpansion: true
```

Note: The namespace in this file must be the same as that of other object resources. **secret** must be the same as that created in step 4 of this lab.

Run the **kubectl apply -f csi- cephfs-sc.yaml** command for deployment.

Create a PVC file **csi-cephfs-pvc.yaml** as follows:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: csi-cephfs-pvc
  namespace: cephfs
spec:
  storageClassName: csi-cephfs-sc
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: 2Gi
```

Note: The value of **storageClassName** in this file must be the same as that in **csi-cephfs-sc.yaml**.

Run the **kubectl apply -f csi-cephfs-pvc.yaml** command for deployment.

Create a pod file **csi-cephfs-pod.yaml** as follows:

```
---
apiVersion: v1
kind: Pod
metadata:
  name: csi-cephfs-pod1
  namespace: cephfs
spec:
  containers:
    - name: csi-cephfs-pod1
      image: nginx
      volumeMounts:
        - name: csi-cephfs-pod1
          mountPath: /mnt
          readOnly: false
  volumes:
    - name: csi-cephfs-pod1
      persistentVolumeClaim:
        claimName: csi-cephfs-pvc
```

Run the **kubectl apply -f csi-cephfs-pod.yaml** command for deployment.

Step 7 Verify the configuration results.

On the Kubernetes node, verify that the PVC and PV are bound. The details are as follows:

```
[root@master01 cephfs]# kubectl get pvc -n cephfs
NAME            STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS    AGE
csi-cephfs-pvc  Bound    pvc-9b4051dc-847b-474a-b22c-cd78d17412aa   2Gi        RWX            csi-cephfs-sc   7m59s
```

4 Scheduler

4.1 Introduction

4.1.1 About This Exercise

This lab describes how to use the default scheduler for pod affinity and anti-affinity scheduling.

4.1.2 Objectives

- Use **nodeSelector** for scheduling.
- Use **nodeName** for scheduling.
- Use topology spread constraints.
- Understand node affinity scheduling.
- Understand inter-pod affinity scheduling.
- Use taints and tolerations.
- Understand taint eviction.
- Master the method of scheduling a specified node.

4.2 nodeSelector-based Scheduling

Step 1 Create a working directory.

Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 ~]# mkdir scheduler
[root@k8smaster1 ~]# cd scheduler/
[root@k8smaster1 scheduler]#
```

Step 2 Add a label to a node.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class=a
```

View the label.

```
[root@k8smaster1 scheduler]# kubectl get nodes --show-labels k8snode1
```

```
[root@k8smaster1 scheduler]# kubectl get nodes --show-labels k8snode1
NAME       STATUS   ROLES    AGE     VERSION   LABELS
k8snode1   Ready    <none>   4d19h   v1.23.1   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux class=a
```

Alternatively, run the **kubectl describe node k8snode1** command to view the label.

Step 3 Create a pod with a specified node label.

```
[root@k8smaster1 scheduler]# cat > nodeselector-pod.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: nodeselector-pod
spec:
  containers:
  - name: nodeselector-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
EOF
[root@k8smaster1 scheduler]# kubectl apply -f nodeselector-pod.yaml
```

Step 4 Check the scheduling result.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
```

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide | grep nodeselector-pod
nodeselector-pod                1/1     Running   0          82s     10.38.249.15    k8snode1
```

Step 5 View the scheduling process.

According to the event, the pod has been scheduled by **default-scheduler**.

```
[root@k8smaster1 scheduler]# kubectl describe pod nodeselector-pod
Name:         nodeselector-pod
Namespace:    default
Priority:     0
Node:         k8snode1/192.168.1.14
Start Time:   Mon, 20 Nov 2023 02:13:31 +0000
Labels:       <none>
Annotations:  cni.projectcalico.org/containerID: 706e444caa28b6ea2190c33e883394bb24722051e314dc4714dfc975377e3fd6
              cni.projectcalico.org/podIP: 10.38.249.15/32
              cni.projectcalico.org/podIPs: 10.38.249.15/32
Status:       Running
IP:           10.38.249.15
IPs:
  IP:  10.38.249.15
Containers:
  nodeselector-pod:
    Container ID:   docker://b68fc191d04f99de7f305ba80b61f3e354711510679a6c6f9e91a3e9a0395825
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:86e53c4c16a6a276b204b0fd3a8143d86547c967dc8258b3d47c3a21bb68d3c6
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 20 Nov 2023 02:13:41 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-p9xcq (ro)
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  kube-api-access-p9xcq:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              class=a
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age    From               Message
  ----    ------     ----   ----               -------
  Normal  Scheduled  2m18s  default-scheduler  Successfully assigned default/nodeselector-pod to k8snode1
  Normal  Pulling    2m17s  kubelet            Pulling image "nginx"
  Normal  Pulled     2m8s   kubelet            Successfully pulled image "nginx" in 9.072625522s
  Normal  Created    2m8s   kubelet            Created container nodeselector-pod
  Normal  Started    2m8s   kubelet            Started container nodeselector-pod
```

Step 6 Use common label commands.

Query a node with a specified label.

```
[root@k8smaster1 scheduler]# kubectl get nodes -l class=a
```

```
[root@k8smaster1 scheduler]# kubectl get nodes -l class=a
NAME       STATUS   ROLES    AGE    VERSION
k8snode1   Ready    <none>   4d18h  v1.23.1
```

Modify a label.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class=b --overwrite
```

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class=b --overwrite
node/k8snode1 labeled
```

Delete a label.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class-
```

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class-
node/k8snode1 unlabeled
```

Step 7 Delete the test pod.

```
[root@k8smaster1 scheduler]# kubectl delete -f nodeselector-pod.yaml
```

4.3 nodeName-based Scheduling

Step 1 Create a pod with a specified node name.

```
[root@k8smaster1 scheduler]# cat > nodename-pod.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: nodename-pod
spec:
  containers:
  - name: nodename-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeName: k8snode2
EOF
[root@k8smaster1 scheduler]# kubectl apply -f nodename-pod.yaml
```

Step 2 Check the scheduling result.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide | grep nodename-pod
nodename-pod                1/1     Running   0          25s     10.76.185.202   k8snode2
```

Step 3 View the scheduling process.

According to the event, the pod has not been scheduled by **default-scheduler**.

```
[root@k8smaster1 scheduler]# kubectl describe pods nodename-pod
Name:         nodename-pod
Namespace:    default
Priority:     0
Node:         k8snode2/192.168.1.15
Start Time:   Mon, 20 Nov 2023 02:17:15 +0000
Labels:       <none>
Annotations:  cni.projectcalico.org/containerID: c3bf60e3fff187a8eae0f4cd9f1dffcf9e36f42fbf9780e0dd2f4617603d14d4
              cni.projectcalico.org/podIP: 10.76.185.202/32
              cni.projectcalico.org/podIPs: 10.76.185.202/32
Status:       Running
IP:           10.76.185.202
IPs:
  IP:  10.76.185.202
Containers:
  nodename-pod:
    Container ID:   docker://3409c5c2fd064d546b8e3f402a7ef0644f5bd04c9e8b6f58999e53f600ee9b5d
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:86e53c4c16a6a276b204b0fd3a8143d86547c967dc8258b3d47c3a21bb68d3c6
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 20 Nov 2023 02:17:16 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-sfjfn (ro)
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  kube-api-access-sfjfn:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason   Age   From     Message
  ----    ------   ----  ----     -------
  Normal  Pulled   68s   kubelet  Container image "nginx" already present on machine
  Normal  Created  68s   kubelet  Created container nodename-pod
  Normal  Started  68s   kubelet  Started container nodename-pod
```

Step 4 Clear the environment.

```
[root@k8smaster1 scheduler]# kubectl delete -f nodename-pod.yaml
```

4.4 One Topology Spread Constraint

Step 1 Set the master node to be schedulable.

```
[root@k8smaster1 scheduler]# kubectl taint node k8smaster1 node-role.kubernetes.io/master-
[root@k8smaster1 scheduler]# kubectl taint node k8smaster2 node-role.kubernetes.io/master-
```

Step 2 Add labels to nodes.

Label **zoneA** for **k8smaster1** and **k8smaster2**, and label **zoneB** for **k8snode1** and **k8snode2**.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8smaster1 zone=zoneA
[root@k8smaster1 scheduler]# kubectl label nodes k8smaster2 zone=zoneA
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 zone=zoneB
[root@k8smaster1 scheduler]# kubectl label nodes k8snode2 zone=zoneB
```

Step 3 Create pods with topology spread constraints.

Create the **topo-pod1.yaml**, **topo-pod2.yaml**, **topo-pod3.yaml**, and **topo-pod4.yaml** files for four pods. Change the pod name in the files. The following is an example of **topo-pod1.yaml**:

```
[root@k8smaster1 scheduler]# cat > topo-pod1.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: topo-pod1
  labels:
    foo: bar
spec:
  containers:
  - name: topo-pod
    image: nginx
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
EOF
```

Step 4 Deploy **topo-pod1** to **topo-pod3**.

```
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod1.yaml
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod2.yaml
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod3.yaml
```

Check the deployment status. The three pods are deployed in **zoneA** and **zoneB**, respectively.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide -l foo
NAME        READY   STATUS    RESTARTS   AGE   IP              NODE
topo-pod1   1/1     Running   0          12s   10.76.185.205   k8snode2
topo-pod2   1/1     Running   0          10s   10.90.93.66     k8smaster1
topo-pod3   1/1     Running   0          7s    10.76.185.206   k8snode2
```

Step 5 Deploy **topo-pod4**.

```
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod4.yaml
```

Step 6 Check the scheduling of **topo-pod4**.

topo-pod4 is scheduled to the zone with fewer pods.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide -l foo
NAME        READY   STATUS    RESTARTS   AGE   IP              NODE
topo-pod1   1/1     Running   0          36s   10.76.185.205   k8snode2
topo-pod2   1/1     Running   0          34s   10.90.93.66     k8smaster1
topo-pod3   1/1     Running   0          31s   10.76.185.206   k8snode2
topo-pod4   1/1     Running   0          6s    10.90.93.67     k8smaster1
```

Step 7 Delete the test pods.

```
[root@k8smaster1 scheduler]# kubectl delete pods -l foo
```

4.5 Multiple Topology Spread Constraints

Step 1 Add topology spread constraints.

Add node name constraints to the YAML files of **topo-pod1** to **topo-pod4**. The following is an example of **topo-pod1.yaml**:

```
[root@k8smaster1 scheduler]# cat > topo-pod1.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: topo-pod1
  labels:
    foo: bar
spec:
  containers:
  - name: topo-pod
    image: nginx
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
EOF
```

Step 2 Deploy **topo-pod1** to **topo-pod3**.

```
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod1.yaml
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod2.yaml
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod3.yaml
```

Check the deployment status. The three pods are deployed in **zoneA** and **zoneB** respectively, and the two in the same zone belong to different nodes.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide -l foo
NAME        READY   STATUS    RESTARTS   AGE   IP              NODE
topo-pod1   1/1     Running   0          33s   10.76.185.207   k8snode2
topo-pod2   1/1     Running   0          30s   10.90.93.68     k8smaster1
topo-pod3   1/1     Running   0          28s   10.38.249.16    k8snode1
```

Step 3 Deploy **topo-pod4**.

```
[root@k8smaster1 scheduler]# kubectl apply -f topo-pod4.yaml
```

Step 4 Check the scheduling of **topo-pod4**.

topo-pod4 is scheduled to the zone with fewer pods and to another node.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide -l foo
NAME        READY   STATUS    RESTARTS   AGE   IP              NODE
topo-pod1   1/1     Running   0          57s   10.76.185.207   k8snode2
topo-pod2   1/1     Running   0          54s   10.90.93.68     k8smaster1
topo-pod3   1/1     Running   0          52s   10.38.249.16    k8snode1
topo-pod4   1/1     Running   0          5s    10.86.137.66    k8smaster2
```

Step 5 Delete the test pods.

```
[root@k8smaster1 scheduler]# kubectl delete pods -l foo
```

4.6 Hard Node Affinity

Step 1 Create a pod with hard node affinity.

Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 scheduler]# cat > node-affinity-pod1.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: node-affinity-pod1
spec:
  containers:
  - name: node-affinity-pod1
    image: nginx
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
            - k8smaster4
EOF
[root@k8smaster1 scheduler]# kubectl apply -f node-affinity-pod1.yaml
```

Step 2 Check the scheduling result.

The pod failed to be scheduled and is in the **Pending** state. The event shows no matched label is found.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP       NODE
node-affinity-pod1  0/1     Pending   0          16m   <none>   <none>
```

```
[root@k8smaster1 scheduler]# kubectl describe pods node-affinity-pod1
Name:         node-affinity-pod1
Namespace:    default
Priority:     0
Node:         <none>
Labels:       <none>
Annotations:  <none>
Status:       Pending
IP:
IPs:          <none>
Containers:
  node-affinity-pod1:
    Image:        nginx
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-xzsz6 (ro)
Conditions:
  Type           Status
  PodScheduled   False
Volumes:
  kube-api-access-xzsz6:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason            Age                 From               Message
  ----     ------            ----                ----               -------
  Warning  FailedScheduling  16s (x18 over 17m)  default-scheduler  0/6 nodes are available: 1 node(s) had t
aint {node-role.kubernetes.io/master: }, that the pod didn't tolerate, 5 node(s) didn't match Pod's node aff
inity/selector.
```

Step 3 Change the label value.

```
[root@k8smaster1 scheduler]# cat > node-affinity-pod1.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: node-affinity-pod1
spec:
  containers:
  - name: node-affinity-pod1
    image: nginx
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
            - k8smaster1
            - k8smaster2
EOF
```

Re-create a pod.

```
[root@k8smaster1 scheduler]# kubectl delete pods node-affinity-pod1
[root@k8smaster1 scheduler]# kubectl apply -f node-affinity-pod1.yaml
```

Step 4 Check the scheduling result.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP             NODE
node-affinity-pod1  1/1     Running   0          7s    10.90.93.69    k8smaster1
```

Step 5 Delete the test pod.

```
[root@k8smaster1 scheduler]# kubectl delete pods node-affinity-pod1
```

4.7 Soft Node Affinity

Step 1 Create a pod with soft node affinity.

```
[root@k8smaster1 scheduler]# cat > node-affinity-pod2.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: node-affinity-pod2
spec:
  containers:
  - name: node-affinity-pod2
    image: nginx
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
            - k8smaster4
EOF
[root@k8smaster1 scheduler]# kubectl apply -f node-affinity-pod2.yaml
```

Step 2 Check the scheduling result.

The scheduler did not find **k8smaster4**, but still performs scheduling and allocates the pod to another random node.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE     IP              NODE
node-affinity-pod2  1/1     Running   0          4m44s   10.76.185.208   k8snode2
```

```
[root@k8smaster1 scheduler]# kubectl describe pods node-affinity-pod2
Name:         node-affinity-pod2
Namespace:    default
Priority:     0
Node:         k8snode2/192.168.1.15
Start Time:   Mon, 20 Nov 2023 03:03:13 +0000
Labels:       <none>
Annotations:  cni.projectcalico.org/containerID: e8cfc2113bddc915ce390b05a32db0b636ba8c605fe5aecbb442d6fef24
              cni.projectcalico.org/podIP: 10.76.185.208/32
              cni.projectcalico.org/podIPs: 10.76.185.208/32
Status:       Running
IP:           10.76.185.208
IPs:
  IP:  10.76.185.208
Containers:
  node-affinity-pod2:
    Container ID:   docker://e1a19ce611fe476c0d9c88956d65f8fca6e8515bdb932aa08675b57d27501179
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:86e53c4c16a6a276b204b0fd3a8143d86547c967dc8258b3d47c3a21b
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 20 Nov 2023 03:03:17 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hch6c (ro)
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  kube-api-access-hch6c:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  43s   default-scheduler  Successfully assigned default/node-affinity-pod2 to k8snode2
  Normal  Pulling    43s   kubelet            Pulling image "nginx"
  Normal  Pulled     40s   kubelet            Successfully pulled image "nginx" in 2.94047812s
  Normal  Created    40s   kubelet            Created container node-affinity-pod2
  Normal  Started    40s   kubelet            Started container node-affinity-pod2
```

Step 3 Delete the test pod.

```
[root@k8smaster1 scheduler]# kubectl delete pods node-affinity-pod2
```

4.8 Inter-Pod Affinity

Step 1 (Optional) View node labels.

Check whether **zone** has been set for nodes. If not, set it.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8smaster1 zone=zoneA
[root@k8smaster1 scheduler]# kubectl label nodes k8smaster2 zone=zoneA
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 zone=zoneB
[root@k8smaster1 scheduler]# kubectl label nodes k8snode2 zone=zoneB
```

Step 2 Create a pod with a specified label.

```
[root@k8smaster1 scheduler]# cat > pod-affinity-pod1.yaml <<EOF
apiVersion: v1
```

```
kind: Pod
metadata:
  name: pod-affinity-pod1
  labels:
    foo: bar
spec:
  containers:
  - name: pod-affinity-pod1
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeName: k8smaster1
EOF
[root@k8smaster1 scheduler]# kubectl apply -f pod-affinity-pod1.yaml
```

Verify that the pod has been scheduled to the specified node **k8smaster1**.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP             NODE
pod-affinity-pod1   1/1     Running   0          4s    10.90.93.71    k8smaster1
```

Step 3 Create a pod with inter-pod affinity.

```
[root@k8smaster1 scheduler]# cat > pod-affinity-pod2.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: pod-affinity-pod2
spec:
  containers:
  - name: pod-affinity-pod2
    image: nginx
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: foo
            operator: In
            values:
            - bar
        topologyKey: zone
EOF
[root@k8smaster1 scheduler]# kubectl apply -f pod-affinity-pod2.yaml
```

Step 4 Check the scheduling result.

Verify that **pod-affinity-pod2** has been scheduled to the zone where **pod-affinity-pod1** (with the **foo:bar** label) is located.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP             NODE
pod-affinity-pod1   1/1     Running   0          97s   10.90.93.72    k8smaster1
pod-affinity-pod2   1/1     Running   0          94s   10.86.137.69   k8smaster2
```

4.9 Inter-Pod Anti-affinity

Step 1 Create a pod with inter-pod anti-affinity.

```
[root@k8smaster1 scheduler]# cat > pod-affinity-pod3.yaml <<EOF
kind: Pod
apiVersion: v1
metadata:
  name: pod-affinity-pod3
spec:
  containers:
  - name: pod-affinity-pod3
    image: nginx
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: foo
              operator: In
              values:
              - bar
          topologyKey: zone
EOF
[root@k8smaster1 scheduler]# kubectl apply -f pod-affinity-pod3.yaml
```

Step 2 Check the scheduling result.

pod1 with the **foo:bar** label exists in **zoneA**. Therefore, **pod3** is preferentially scheduled to the node in **zoneB**.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE
pod-affinity-pod1   1/1     Running   0          85s   10.90.93.105    k8smaster1
pod-affinity-pod2   1/1     Running   0          81s   10.90.93.108    k8smaster1
pod-affinity-pod3   1/1     Running   0          5s    10.76.185.250   k8snode2
```

Step 3 Delete the test pods.

```
[root@k8smaster1 scheduler]# kubectl delete pods pod-affinity-pod1
[root@k8smaster1 scheduler]# kubectl delete pods pod-affinity-pod2
[root@k8smaster1 scheduler]# kubectl delete pods pod-affinity-pod3
```

4.10 Taints and Tolerations

Step 1 Create a taint for a node.

```
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoSchedule
```

View the node taint.

```
[root@k8smaster1 scheduler]# kubectl get nodes k8snode1 -o go-template={{.spec.taints}}
```

```
[root@k8smaster1 scheduler]# kubectl get nodes k8snode1 -o go-template={{.spec.taints}}
[map[effect:NoSchedule key:key1 value:value1]][root@k8smaster1 scheduler]#
```

Alternatively, run the **kubectl describe nodes k8snode1** command to view the taint.

```
[root@k8smaster1 scheduler]# kubectl describe nodes k8snode1
Name:               k8snode1
Roles:              <none>
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=k8snode1
                    kubernetes.io/os=linux
                    network=k8snode1
                    zone=zoneB
Annotations:        kubeadm.alpha.kubernetes.io/cri-socket: /var/
                    node.alpha.kubernetes.io/ttl: 0
                    projectcalico.org/IPv4Address: 192.168.0.14/2
                    projectcalico.org/IPv4IPIPTunnelAddr: 10.38.2
                    volumes.kubernetes.io/controller-managed-atta
CreationTimestamp:  Wed, 15 Nov 2023 07:26:29 +0000
Taints:             key1=value1:NoSchedule
Unschedulable:      false
Lease:
  HolderIdentity:  k8snode1
  AcquireTime:     <unset>
  RenewTime:       Mon, 20 Nov 2023 03:28:05 +0000
```

Add a label to **k8snode1**.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8snode1 class=a
```

Step 2 Deploy test pods.

```
[root@k8smaster1 scheduler]# cat > tolerations-pod1.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod1
spec:
  containers:
  - name: tolerations-pod1
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
EOF
cat > tolerations-pod2.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod2
spec:
  containers:
  - name: tolerations-pod2
    image: nginx
    imagePullPolicy: IfNotPresent
```

```
  nodeSelector:
    class: a
EOF
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod1.yaml
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod2.yaml
```

Verify that the pods are in the **Pending** state.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP       NODE
tolerations-pod1    0/1     Pending   0          9s    <none>   <none>
tolerations-pod2    0/1     Pending   0          5s    <none>   <none>
```

```
[root@k8smaster1 scheduler]# kubectl describe pods tolerations-pod1
Name:         tolerations-pod1
Namespace:    default
Priority:     0
Node:         <none>
Labels:       <none>
Annotations:  <none>
Status:       Pending
IP:
IPs:          <none>
Containers:
  tolerations-pod1:
    Image:        nginx
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-849jp (ro)
Conditions:
  Type           Status
  PodScheduled   False
Volumes:
  kube-api-access-849jp:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    ConfigMapOptional:        <nil>
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               class=a
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  80s   default-scheduler  0/6 nodes are available: 1 node(s) had taint {key1: va
lue1}, that the pod didn't tolerate, 5 node(s) didn't match Pod's node affinity/selector.
```

Step 3 Add tolerations to pods.

Delete the test pods, add tolerations to the YAML files, and re-deploy the pods.

```
[root@k8smaster1 scheduler]# kubectl delete -f tolerations-pod1.yaml
[root@k8smaster1 scheduler]# kubectl delete -f tolerations-pod2.yaml
[root@k8smaster1 scheduler]# cat > tolerations-pod1.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod1
spec:
  containers:
  - name: tolerations-pod1
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
  tolerations:
  - key: "key1"
```

```
      operator: "Exists"
      effect: "NoSchedule"
EOF
cat > tolerations-pod2.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod2
spec:
  containers:
  - name: tolerations-pod2
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
  tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoSchedule"
EOF
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod1.yaml
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod2.yaml
```

Step 4 Check the scheduling result.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
```

The tolerations of the two pods match the taint. Therefore, the pod can be scheduled to **k8snode1**.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME               READY   STATUS    RESTARTS   AGE   IP              NODE
tolerations-pod1   1/1     Running   0          12s   10.38.249.12    k8snode1
tolerations-pod2   1/1     Running   0          8s    10.38.249.13    k8snode1
```

Step 5 Delete the node taint.

```
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoSchedule-
```

4.11 Taint Eviction

Step 1 Create a taint with eviction.

```
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoExecute
```

pod1 and **pod2** are evicted because they do not match the taint. They are not printed in the **kubectl get pods** command output.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
No resources found in default namespace.
```

Step 2 Create pods with tolerations.

```
[root@k8smaster1 scheduler]# cat > tolerations-pod3.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod3
spec:
  containers:
  - name: tolerations-pod3
    image: nginx
    imagePullPolicy: IfNotPresent
  tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoExecute"
EOF
cat > tolerations-pod4.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod4
spec:
  containers:
  - name: tolerations-pod4
    image: nginx
    imagePullPolicy: IfNotPresent
  tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoExecute"
    tolerationSeconds: 30
EOF
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod3.yaml
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod4.yaml
```

Step 3 Check the scheduling result.

```
tolerations-pod3   1/1     Running   0          8s    10.38.249.17   k8snode1
tolerations-pod4   1/1     Running   0          4s    10.38.249.18   k8snode1
```

After 30 seconds, check the scheduling result. Only **pod3** exists and **pod4** is evicted after timeout.

```
tolerations-pod3   1/1     Running   0          70s   10.38.249.17   k8snode1
```

4.12 Multiple Taints and Tolerations

Step 1 Create multiple taints.

```
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoSchedule
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key2=value2:NoSchedule
```

Step 2 Create a pod with multiple tolerations.

```
[root@k8smaster1 scheduler]# cat > tolerations-pod5.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: tolerations-pod5
spec:
  containers:
  - name: tolerations-pod5
    image: nginx
    imagePullPolicy: IfNotPresent
  tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoExecute"
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoSchedule"
EOF
[root@k8smaster1 scheduler]# kubectl apply -f tolerations-pod5.yaml
```

Step 3 Check the scheduling result.

pod5 cannot match all taints of **k8snode1** and is scheduled to another node.

```
tolerations-pod5   1/1     Running   0          13s     10.4.185.16     k8snode3
```

Step 4 Clear taints.

Clear taints. Otherwise, other pods cannot be scheduled to the node.

```
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoExecute-
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key1=value1:NoSchedule-
[root@k8smaster1 scheduler]# kubectl taint nodes k8snode1 key2=value2:NoSchedule-
```

Step 5 Delete the test pods.

```
[root@k8smaster1 scheduler]# kubectl delete pods tolerations-pod3
[root@k8smaster1 scheduler]# kubectl delete pods tolerations-pod5
```

4.13 Node Marked as Schedulable

Step 1 Mark a node as unschedulable.

```
[root@k8smaster1 scheduler]# kubectl cordon k8snode1
```

Check the setting.

```
[root@k8smaster1 scheduler]# kubectl describe nodes k8snode1
Name:               k8snode1
Roles:              <none>
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=k8snode1
                    kubernetes.io/os=linux
                    network=k8snode1
                    zone=zoneB
Annotations:        kubeadm.alpha.kubernetes.io/cri-socket: /var/
                    node.alpha.kubernetes.io/ttl: 0
                    projectcalico.org/IPv4Address: 192.168.0.14/2
                    projectcalico.org/IPv4IPIPTunnelAddr: 10.38.2
                    volumes.kubernetes.io/controller-managed-atta
CreationTimestamp:  Wed, 15 Nov 2023 07:26:29 +0000
Taints:             node.kubernetes.io/unschedulable:NoSchedule
Unschedulable:      true
```

```
[root@k8smaster1 scheduler]# kubectl get nodes
NAME         STATUS                     ROLES
k8smaster1   Ready                      control-plane,master
k8smaster2   Ready                      control-plane,master
k8smaster3   Ready                      control-plane,master
k8snode1     Ready,SchedulingDisabled   <none>
k8snode2     Ready                      <none>
k8snode3     Ready                      <none>
```

Step 2 Create a test pod.

Create a pod.

```
[root@k8smaster1 scheduler]# cat > cordon-pod.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: cordon-pod
spec:
  containers:
  - name: cordon-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
EOF
[root@k8smaster1 scheduler]# kubectl apply -f cordon-pod.yaml
```

Check the node scheduling status. Only **k8snode1** has the **class:a** label and has been marked as unschedulable. In this case, the pod can only be pending.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP       NODE
cordon-pod   0/1     Pending   0          3s    <none>   <none>
```

Step 3 Mark a node as schedulable.

```
[root@k8smaster1 scheduler]# kubectl uncordon k8snode1
```

```
[root@k8smaster1 ~]# kubectl get nodes
NAME         STATUS   ROLES                  AGE   VERSION
k8smaster1   Ready    control-plane,master   9d    v1.23.1
k8smaster2   Ready    control-plane,master   9d    v1.23.1
k8smaster3   Ready    control-plane,master   9d    v1.23.1
k8snode1     Ready    <none>                 9d    v1.23.1
k8snode2     Ready    <none>                 9d    v1.23.1
k8snode3     Ready    <none>                 9d    v1.23.1
```

Verify that the pod can be scheduled properly.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP             NODE
cordon-pod   1/1     Running   0          25s   10.38.249.21   k8snode1
```

Step 4 Force a node to evict a pod.

Evict the pod from **k8snode1**.

```
[root@k8smaster1 scheduler]# kubectl drain k8snode1 --ignore-daemonsets --force
```

Check the node status.

```
[root@k8smaster1 scheduler]# kubectl get nodes
NAME         STATUS                     ROLES
k8smaster1   Ready                      control-plane,master
k8smaster2   Ready                      control-plane,master
k8smaster3   Ready                      control-plane,master
k8snode1     Ready,SchedulingDisabled   <none>
k8snode2     Ready                      <none>
k8snode3     Ready                      <none>
```

Verify that the pod has been evicted from **k8snode1**.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide | grep k8snode1
[root@k8smaster1 scheduler]#
```

Mark the node as schedulable.

```
[root@k8smaster1 scheduler]# kubectl uncordon k8snode1
```

```
[root@k8smaster1 scheduler]# kubectl get nodes
NAME         STATUS   ROLES                  AGE
k8smaster1   Ready    control-plane,master   4d22h
k8smaster2   Ready    control-plane,master   4d22h
k8smaster3   Ready    control-plane,master   4d22h
k8snode1     Ready    <none>                 4d22h
k8snode2     Ready    <none>                 4d22h
k8snode3     Ready    <none>                 4d22h
```

4.14 Master Node Scheduling

Step 1 Set the master node to MasterOnly.

```
[root@k8smaster1 scheduler]# kubectl taint node k8smaster3 node-
role.kubernetes.io/master="":NoSchedule
```

```
[root@k8smaster1 scheduler]# kubectl describe nodes k8smaster3
Name:               k8smaster3
Roles:              control-plane,master
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=k8smaster3
                    kubernetes.io/os=linux
                    node-role.kubernetes.io/control-plane=
                    node-role.kubernetes.io/master=
                    node.kubernetes.io/exclude-from-external-loa
Annotations:        kubeadm.alpha.kubernetes.io/cri-socket: /var
                    node.alpha.kubernetes.io/ttl: 0
                    projectcalico.org/IPv4Address: 192.168.0.13/
                    projectcalico.org/IPv4IPIPTunnelAddr: 10.99.
                    volumes.kubernetes.io/controller-managed-att
CreationTimestamp:  Wed, 15 Nov 2023 07:25:18 +0000
Taints:             node-role.kubernetes.io/master:NoSchedule
Unschedulable:      false
```

Step 2 Create a test pod.

Create a node label to ensure that the pod is scheduled to the specified node.

```
[root@k8smaster1 scheduler]# kubectl label nodes k8smaster3 class=b
```

Create a pod.

```
[root@k8smaster1 scheduler]# cat > master-pod.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: master-pod
spec:
  containers:
  - name: master-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: b
EOF
[root@k8smaster1 scheduler]# kubectl apply -f master-pod.yaml
```

Check the node scheduling status. Only **k8smaster3** has the **class:b** label and has been marked as MasterOnly. In this case, the pod can only be pending.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP       NODE
master-pod   0/1     Pending   0          3s    <none>   <none>
```

Step 3 Mark the master node as schedulable.

```
[root@k8smaster1 scheduler]# kubectl taint node k8smaster3 node-role.kubernetes.io/master-
```

Verify that the pod has been scheduled to **k8smaster3** and is running properly.

```
[root@k8smaster1 scheduler]# kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP              NODE
master-pod   1/1     Running   0          34s   10.99.63.195    k8smaster3
```

Step 4 Delete the test pod.

```
[root@k8smaster1 scheduler]# kubectl delete pods master-pod
```

5 Autoscaling

5.1 Introduction

5.1.1 About This Exercise

This lab describes how to use a HorizontalPodAutoscaler (HPA) to perform autoscaling.

5.1.2 Objectives

- Use the HPA on CPU utilization.
- Use the HPA on memory utilization.

5.2 Metrics Server Installation

Step 1 Create a working directory.

Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 ~]# mkdir hpa
[root@k8smaster1 ~]# cd hpa
[root@k8smaster1 hpa]#
```

Step 2 Obtain the YAML file.

Run the following command to download **metrics-server**:

```
[root@k8smaster1 hpa]# git clone https://gitee.com/yftyxa/metrics-server.git
```

Step 3 Install a Metrics Server.

```
[root@k8smaster1 hpa]# kubectl apply -f metrics-server/metrics-server.yaml
```

Check the running status of the Metrics Server.

```
[root@k8smaster1 hpa]# kubectl get pods -n kube-system
```

```
metrics-server-7dc78f6f-tlxd7              0/1     Running   0              25s
```

5.3 HPA on CPU Utilization

Step 1 Create a test Deployment.

```
[root@k8smaster1 hpa]# cat > nginx-hpa.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-hpa
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-hpa
        image: nginx
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
EOF
[root@k8smaster1 hpa]# kubectl apply -f nginx-hpa.yaml
```

Step 2 Create an HPA.

```
[root@k8smaster1 hpa]# kubectl autoscale deployment nginx-hpa --cpu-percent=10 --min=1 --
max=10
```

Step 3 Check the HPA status.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   <unknown>/10%   1         10        2          25s
```

The default collection period of the Metrics Server is 1 minute. After scale-out or update is complete, the Metrics Server cannot obtain monitoring data for a period of time. Wait 1 minute and view the monitoring data.

```
[root@k8smaster1 hpa]#  kubectl get hpa
NAME        REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   0%/10%    1         10        2          87s
```

```
[root@k8smaster1 hpa]# kubectl describe hpa nginx
```

```
[root@k8smaster1 hpa]# kubectl describe hpa nginx
Warning: autoscaling/v2beta2 HorizontalPodAutoscaler is deprecated in v1.23+, unavailable in v1.
Name:                                                  nginx-hpa
Namespace:                                             default
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Mon, 20 Nov 2023 07:07:39 +0000
Reference:                                             Deployment/nginx-hpa
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  0% (0) / 10%
Min replicas:                                          1
Max replicas:                                          10
Deployment pods:                                       2 current / 2 desired
Conditions:
  Type            Status  Reason              Message
  ----            ------  ------              -------
  AbleToScale     True    ScaleDownStabilized  recent recommendations were higher than current o
  ScalingActive   True    ValidMetricFound     the HPA was able to successfully calculate a repl
  ScalingLimited  False   DesiredWithinRange   the desired count is within the acceptable range
Events:           <none>
```

Step 4 Perform scale-out to increase loads.

View the IP addresses of the pods.

```
[root@k8smaster1 hpa]# kubectl get pods -o wide
```

```
[root@k8smaster1 hpa]# kubectl get pods -o wide
NAME                          READY   STATUS    RESTARTS   AGE   IP              NODE
nginx-hpa-74b4446cf6-b58kd    1/1     Running   0          18s   10.99.63.219    k8smaster3
nginx-hpa-74b4446cf6-pdnth    1/1     Running   0          18s   10.90.93.109    k8smaster1
```

Access the pods from other nodes cyclically. Replace the IP address in the following command with the actual IP address queried in the preceding step. Before running the command, ensure that wget has been installed on the node.

```
[root@k8smaster2 ~]# while true;do wget -q -O- http://10.99.63.219 > /dev/null;done
```

Step 5 View HPA results.

Check the HPA. The CPU utilization increases by more than 10%.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME        REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   16%/10%   1         10        2          16m
```

Check the scale-out status.

```
[root@k8smaster1 hpa]# kubectl get pods
```

```
[root@k8smaster1 hpa]# kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-hpa-74b4446cf6-b58kd    1/1     Running   0          4m54s
nginx-hpa-74b4446cf6-fmdzg    1/1     Running   0          11s
nginx-hpa-74b4446cf6-fshj4    1/1     Running   0          11s
nginx-hpa-74b4446cf6-pdnth    1/1     Running   0          4m54s
```

View HPA details.

```
[root@k8smaster1 hpa]# kubectl describe hpa nginx-hpa
```

```
[root@k8smaster1 hpa]# kubectl describe hpa nginx-hpa
Warning: autoscaling/v2beta2 HorizontalPodAutoscaler is deprecated in v1.23+, unavailable in v1.26+; use autoscaling/v2 HorizontalPodAutoscaler
Name:                                                  nginx-hpa
Namespace:                                             default
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Mon, 20 Nov 2023 07:07:39 +0000
Reference:                                             Deployment/nginx-hpa
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  16% (16m) / 10%
Min replicas:                                          1
Max replicas:                                          10
Deployment pods:                                       4 current / 4 desired
Conditions:
  Type            Status  Reason              Message
  ----            ------  ------              -------
  AbleToScale     True    ReadyForNewScale    recommended size matches current size
  ScalingActive   True    ValidMetricFound    the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage
  ScalingLimited  False   DesiredWithinRange  the desired count is within the acceptable range
Events:
  Type    Reason             Age   From                       Message
  ----    ------             ----  ----                       -------
  Normal  SuccessfulRescale  11m   horizontal-pod-autoscaler  New size: 1; reason: All metrics below target
  Normal  SuccessfulRescale  74s   horizontal-pod-autoscaler  New size: 2; reason: cpu resource utilization (percentage of request) above target
  Normal  SuccessfulRescale  29s   horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization (percentage of request) above target
```

Step 6 Perform scale-in to decrease loads.

Stop cyclically accessing the pods and check the HPA. The CPU utilization decreases to **0**.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME        REFERENCE                TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa     0%/10%    1         10        4          18m
```

The default timeout interval for scale-in is 5 minutes. After the scale-in times out, check the HPA results again. The number of pods decreases to **1**.

```
[root@k8smaster1 hpa]# kubectl get pods -o wide
```

```
[root@k8smaster1 hpa]# kubectl get pods -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP              NODE
nginx-hpa-74b4446cf6-pdnth   1/1     Running   0          11m   10.90.93.109    k8smaster1
```

View HPA details.

```
[root@k8smaster1 hpa]# kubectl describe hpa nginx-hpa
```

```
[root@k8smaster1 hpa]#  kubectl describe hpa nginx-hpa
Warning: autoscaling/v2beta2 HorizontalPodAutoscaler is deprecated in v1.23+, unavailable in v1.26+; use autoscaling/v2 HorizontalPodAutoscaler
Name:                                                  nginx-hpa
Namespace:                                             default
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Mon, 20 Nov 2023 07:07:39 +0000
Reference:                                             Deployment/nginx-hpa
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  0% (0) / 10%
Min replicas:                                          1
Max replicas:                                          10
Deployment pods:                                       1 current / 1 desired
Conditions:
  Type            Status  Reason            Message
  ----            ------  ------            -------
  AbleToScale     True    ReadyForNewScale  recommended size matches current size
  ScalingActive   True    ValidMetricFound  the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of request)
  ScalingLimited  True    TooFewReplicas    the desired replica count is less than the minimum replica count
Events:
  Type    Reason             Age                  From                       Message
  ----    ------             ----                 ----                       -------
  Normal  SuccessfulRescale  11m                  horizontal-pod-autoscaler  New size: 2; reason: cpu resource utilization (percentage of request) above target
  Normal  SuccessfulRescale  10m                  horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization (percentage of request) above target
  Normal  SuccessfulRescale  3m56s (x2 over 21m)  horizontal-pod-autoscaler  New size: 1; reason: All metrics below target
```

Step 7 Create an HPA based on an absolute value metric.

Create an HPA and set an absolute value metric.

```
[root@k8smaster1 hpa]# cat > hpa-test.yaml <<EOF
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
   name: hpa-test
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-hpa
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: AverageValue
        averageValue: 10
EOF
[root@k8smaster1 hpa]# kubectl apply -f hpa-test.yaml
```

Perform step 4 to increase loads and check the HPA results.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME         REFERENCE                TARGETS    MINPODS   MAXPODS   REPLICAS   AGE
hpa-test     Deployment/nginx-hpa     27m/10     1         10        1          103s
nginx-hpa    Deployment/nginx-hpa     27%/10%    1         10        1          29m
[root@k8smaster1 hpa]# kubectl get pods -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP              NODE
nginx-hpa-74b4446cf6-9jgp6   1/1     Running   0          27s   10.76.185.216   k8snode2
nginx-hpa-74b4446cf6-tncq8   1/1     Running   0          30m   10.4.185.19     k8snode3
nginx-hpa-74b4446cf6-tr5zs   1/1     Running   0          27s   10.38.249.27    k8snode1
```

Step 8 Clear the environment.

Stop the loads, and delete the HPA and test Deployment.

```
[root@k8smaster1 hpa]# kubectl delete hpa hpa-test
[root@k8smaster1 hpa]# kubectl delete hpa nginx-hpa
[root@k8smaster1 hpa]# kubectl delete -f nginx-hpa.yaml
```

5.4 HPA on Memory Utilization

Step 1 Create a test Deployment.

```
[root@k8smaster1 hpa]# cat > nginx-mem.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-mem-demo
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
```

```
        app: nginx
    spec:
      volumes:
      - name: increase-mem-script
        configMap:
          name: increase-mem-config
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
        volumeMounts:
        - name: increase-mem-script
          mountPath: /etc/script
        resources:
          requests:
            memory: 50Mi
            cpu: 50m
        securityContext:
          privileged: true
EOF
```

Step 2 Create a ConfigMap.

```
[root@k8smaster1 hpa]# cat > increase-mem-cm.yaml <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: increase-mem-config
data:
  increase-mem.sh: |
    #!/bin/bash
    mkdir /tmp/memory
    mount -t tmpfs -o size=40M tmpfs /tmp/memory
    dd if=/dev/zero of=/tmp/memory/block
    sleep 60
    rm /tmp/memory/block
    umount /tmp/memory
    rmdir /tmp/memory
EOF
```

Step 3 Start the Deployment and ConfigMap.

```
[root@k8smaster1 hpa]# kubectl apply -f increase-mem-cm.yaml
[root@k8smaster1 hpa]# kubectl apply -f nginx-mem.yaml
```

Step 4 Create an HPA.

```
[root@k8smaster1 hpa]# cat > hpa-mem.yaml <<EOF
apiVersion: autoscaling/v2beta1
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-mem-demo
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: memory
      targetAverageUtilization: 60
EOF
[root@k8smaster1 hpa]# kubectl apply -f hpa-mem.yaml
```

Step 5 Perform scale-out to increase loads.

Run the **increase-mem.sh** script in the mounted container to increase the memory.

```
[root@k8smaster1 hpa]# kubectl get pods
```

```
[root@k8smaster1 hpa]# kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
hpa-mem-demo-74cf7f674f-7kl2d     1/1     Running   0          7m23s
```

Log in to the pod and run the following command with the pod name obtained in the preceding step. Do not directly copy the example command.

```
[root@k8smaster1 hpa]# kubectl exec -it hpa-mem-demo-74cf7f674f-7kl2d /bin/bash
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version. Use
kubectl exec [POD] -- [COMMAND] instead.
root@hpa-mem-demo-74cf7f674f-7kl2d:/# ls /etc/script/
increase-mem.sh
root@hpa-mem-demo-74cf7f674f-7kl2d:/# source /etc/script/increase-mem.sh
dd: writing to '/tmp/memory/block': No space left on device
81921+0 records in
81920+0 records out
41943040 bytes (42 MB, 40 MiB) copied, 0.0486124 s, 863 MB/s
```

Step 6 View HPA results.

Start another terminal to view HPA results. The memory utilization exceeds 60%, so the HPA triggers auto scale-out and the number of replicas increases.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME        REFERENCE                  TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/hpa-mem-demo    89%/60%   1         5         1          8m51s
```

```
[root@k8smaster1 hpa]# kubectl get pods
```

```
[root@k8smaster1 hpa]# kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
hpa-mem-demo-74cf7f674f-7kl2d     1/1     Running   0          9m41s
hpa-mem-demo-74cf7f674f-jx4xp     1/1     Running   0          45s
```

Step 7 Perform scale-in to decrease loads.

Return to the original terminal and perform scale-in to decrease loads. The memory utilization decreases.

```
[root@k8smaster1 hpa]# kubectl get hpa
```

```
[root@k8smaster1 hpa]# kubectl get hpa
NAME        REFERENCE                  TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/hpa-mem-demo    8%/60%    1         5         2          10m
```

Step 8 Clear the environment.

Delete the HPA and test Deployment.

```
[root@k8smaster1 hpa]# kubectl delete -f hpa-mem.yaml
[root@k8smaster1 hpa]# kubectl delete -f nginx-mem.yaml
```

6 Container OS Upgrade

6.1 Introduction

6.1.1 About This Exercise

This lab uses KubeOS to implement automatic upgrade of Kubernetes nodes and kernel live upgrade.

6.1.2 Objectives

- Master the KubeOS upgrade process.
- Master the kernel live upgrade process.

6.2 Container OS Image Build

Step 1 Install KubeOS.

Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 ~]# sudo yum install golang make git rust cargo openssl-devel
[root@k8smaster1 ~]# cat << EOF >> ~/.cargo/config
[source.crates-io]
replace-with='rsproxy'

[source.rsproxy]
registry="https://rsproxy.cn/crates.io-index"

[registries.rsproxy]
index = "https://rsproxy.cn/crates.io-index"

[net]
git-fetch-with-cli = true
EOF
[root@k8smaster1 ~]# sudo git clone https://gitee.com/openeuler/KubeOS.git
[root@k8smaster1 ~]# cd KubeOS
[root@k8smaster1 KubeOS]# sudo make
```

Step 2 Modify the image configuration file.

Go to the **scripts** directory.

```
[root@k8smaster1 KubeOS]# cd /root/KubeOS/scripts
```

If the node is a Huawei Cloud ECS, modify the **/root/KubeOS/scripts/grub.cfg** configuration file in **/dev/vda2** and **/dev/vda3**.

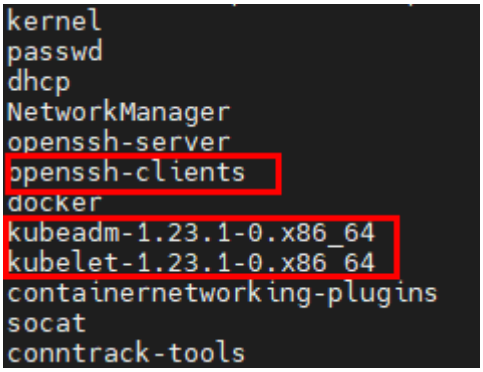
```
### BEGIN /etc/grub.d/10_linux ###
menuentry 'A' --class KubeOS --class gnu-linux --class gnu --class os --unrestricted
$menuentry_id_option 'KubeOS-A' {
        load_video
        set gfxpayload=keep
        insmod gzio
        insmod part_msdos
        insmod ext2
        set root='hd0,msdos2'
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --file --set=root --hint-bios=hd0,msdos2 --hint-efi=hd0,msdos2 --hint-
baremetal=ahci0,msdos2 /vmlinuz
        else
          search --no-floppy --file --set=root /vmlinuz
        fi
        linux   /vmlinuz root=/dev/vda2 ro rootfstype=ext4 nomodeset quiet oops=panic
softlockup_panic=1 nmi_watchdog=1 rd.shell=0 selinux=0 crashkernel=256M panic=3
        initrd  /initramfs.img
}

menuentry 'B' --class KubeOS --class gnu-linux --class gnu --class os --unrestricted
$menuentry_id_option 'KubeOS-B' {
        load_video
        set gfxpayload=keep
        insmod gzio
        insmod part_msdos
        insmod ext2
        set root='hd0,msdos3'
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --file --set=root --hint-bios=hd0,msdos3 --hint-efi=hd0,msdos3 --hint-
baremetal=ahci0,msdos3 /vmlinuz
        else
          search --no-floppy --file --set=root /vmlinuz
        fi
        linux   /vmlinuz root=/dev/vda3 ro rootfstype=ext4 nomodeset quiet oops=panic
softlockup_panic=1 nmi_watchdog=1 rd.shell=0 selinux=0 crashkernel=256M panic=3
        initrd  /initramfs.img
}

### END /etc/grub.d/10_linux ###
```

Step 3 Modify the configuration file of extensions.

Open the image extension configuration file **/root/KubeOS/scripts/rpmlist**, modify the kubeadm and kubelet versions, and add openssh-clients. If other software tools are required, add them to the configuration file. No more software tools will be installed on the nodes in subsequent operations.



Step 4 Install dependencies.

```
[root@k8smaster1 scripts]# yum install -y qemu-img bc parted
```

Step 5 Create a base container OS.

In the **scripts** directory, create a Yum repository file **oe2203lts.repo** for the OS to be upgraded.

```
[OS]
name=OS
baseurl=http://repo.openeuler.org/openEuler-22.03-LTS/OS/$basearch/
enabled=1
gpgcheck=1
gpgkey=http://repo.openeuler.org/openEuler-22.03-LTS/OS/$basearch/RPM-GPG-KEY-openEuler

[everything]
name=everything
baseurl=http://repo.openeuler.org/openEuler-22.03-LTS/everything/$basearch/
enabled=1
gpgcheck=1
gpgkey=http://repo.openeuler.org/openEuler-22.03-LTS/everything/$basearch/RPM-GPG-KEY-
openEuler

[EPOL]
name=EPOL
baseurl=http://repo.openeuler.org/openEuler-22.03-LTS/EPOL/main/$basearch/
enabled=1
gpgcheck=1
gpgkey=http://repo.openeuler.org/openEuler-22.03-LTS/OS/$basearch/RPM-GPG-KEY-openEuler

[kubernetes]
name=Kubernetes
baseurl=https://repo.huaweicloud.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=1
repo_gpgcheck=0
gpgkey=https://repo.huaweicloud.com/kubernetes/yum/doc/yum-key.gpg
https://repo.huaweicloud.com/kubernetes/yum/doc/rpm-package-key.gpg
```

Create a base container OS. The **.repo** file is the Yum repository of openEuler 22.03 LTS, and **1xyz$RdLyKTL32WEvK3lg8CXID0** is encrypted from the password

(**openEuler12#$**) of the **root** user. You can also use the custom password generated by OpenSSL.

```
[root@k8smaster1 scripts]# bash kbimg.sh create vm-image -p oe2203lts.repo -v v1 -b ../bin/os-agent -e "'$1$xyz$RdLyKTL32WEvK3lg8CXID0'"
```

The **system.img**, **system.qcow2**, and **update.img** files are generated.

```
[root@k8smaster1 scripts]# ll
total 5235012
drwxr-xr-x 2 root root        4096 Nov 23 10:46 00bootup
drwxr-xr-x 2 root root        4096 Nov 23 10:46 admin-container
-rw-r--r-- 1 root root        1382 Nov 23 10:46 bootloader.sh
drwxr-xr-x 2 root root        4096 Nov 23 10:46 common
drwxr-xr-x 2 root root        4096 Nov 23 10:46 create
-rw-r--r-- 1 root root          43 Nov 23 10:46 Dockerfile
-rw-r--r-- 1 root root        5156 Nov 23 11:12 grub.cfg
-rw-r--r-- 1 root root       10109 Nov 23 10:46 kbimg.sh
-rw-r--r-- 1 root root         898 Nov 23 11:16 oe2203lts.repo
-rw-r--r-- 1 root root         722 Dec  1 02:09 oe2203sp2.repo
-rw-r--r-- 1 root root         255 Nov 23 11:13 rpmlist
-rw-r--r-- 1 root root         944 Nov 23 10:46 set_in_chroot.sh
-rw-r--r-- 1 root root 21474836480 Dec  1 02:12 system.img
-rw-r--r-- 1 root root  1481965568 Dec  1 02:12 system.qcow2
-rw-r--r-- 1 root root  2202009088 Dec  1 02:12 update.img
```

Download **system.qcow2** to the local PC and change its name to **oe2203lts.qcow2**.

After the download is complete, delete the three OS image files.

```
[root@k8smaster1 scripts]# rm -rf system.img
[root@k8smaster1 scripts]# rm -rf system.qcow2
[root@k8smaster1 scripts]# rm -rf update.img
```

Step 6 (Optional) Upload the base container OS.

If the Kubernetes cluster uses Huawei Cloud ECSs, use **oe2203lts.qcow** to create a private image.

1. Log in to Huawei Cloud Object Storage Service (OBS), create a standard bucket, and upload **oe2203lts.qcow2** to the bucket.

2. Log in to Huawei Cloud Image Management Service (IMS) and create a private image. Set **Type** to **Import Image**, select the **oe2203lts.qcow2** file in the bucket, set **System Disk (GiB)** to **20**, and enter a custom image name.

Step 7 Create a container OS for upgrade.

In the **scripts** directory, create a Yum repository file **oe2203sp2.repo** for upgrade.

```
[openEuler]
name=openEuler
baseurl=http://repo.openeuler.org/openEuler-22.03-LTS-SP2/everything/$basearch/
enabled=1
gpgcheck=1
gpgkey=http://repo.openeuler.org/openEuler-22.03-LTS-SP2/everything/$basearch/RPM-GPG-KEY-openEuler

[Epol]
```

```
name=Epol
baseurl=http://repo.openeuler.org/openEuler-22.03-LTS-SP2/EPOL/main/$basearch/
enabled=1
gpgcheck=1
gpgkey=http://repo.openeuler.org/openEuler-22.03-LTS-SP2/OS/$basearch/RPM-GPG-KEY-openEuler

[kubernetes]
name=Kubernetes
baseurl=https://repo.huaweicloud.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=1
repo_gpgcheck=0
gpgkey=https://repo.huaweicloud.com/kubernetes/yum/doc/yum-key.gpg
https://repo.huaweicloud.com/kubernetes/yum/doc/rpm-package-key.gpg
```

Create a container OS for upgrade.

```
 [root@k8smaster1 scripts]# bash kbimg.sh create vm-image -p oe2203sp2.repo -v v2 -b ../bin/os-agent -e '"$1$xyz$RdLyKTL32WEvK3lg8CXID0"'
```

The **system.img**, **system.qcow2**, and **update.img** files are generated.

6.3 k8snode4 Deployment

Step 1 Deploy a **k8snode4** node.

Install a new node for the upgrade lab (if server resources are limited, you can use an existing node in the original Kubernetes cluster). Select the base container OS image **oe2203lts.qcow2** created in step 5 in section 6.2, set the private IP address to **192.168.1.17**. After the OS is started, perform the following steps to add the node to the Kubernetes cluster:

1. Change **hostname** to **k8snode4**.

```
[root@host-192-168-1-17 ~]# hostnamectl hostname k8snode4
```

Exit SSH and log in to **k8snode4** again.

2. Configure host name resolution on **k8smaster1** and **k8snode4**.

```
cat <<EOF>>/etc/hosts
192.168.1.11 k8smaster1
192.168.1.12 k8smaster2
192.168.1.13 k8smaster3
192.168.1.14 k8snode1
192.168.1.15 k8snode2
192.168.1.16 k8snode3
192.168.1.17 k8snode4
EOF
```

3. Modify the cgroup configuration of Docker and restart Docker.

```
[root@k8snode4 ~]# cat <<EOF > /etc/docker/daemon.json
{
  "exec-opts": ["native.cgroupdriver=systemd"]
}
EOF
[root@k8snode4 ~]# systemctl daemon-reload
[root@k8snode4 ~]# systemctl restart docker
[root@k8snode4 ~]# systemctl status docker
```

4. Run the **kubeadm token create --print-join-command** command on **k8smaster1** to generate the **kubeadm join** command for adding the node to the cluster, and run the command on **k8snode4**.

Step 2 Modify the network configuration file.

Change the **cni** path on the node because only the **/etc** and **/var** directories are writable for the images created by KubeOS. You can run the **docker images** command to view the pause image name. Example:

```
[root@k8snode4 ~]# cat > /var/lib/kubelet/kubeadm-flags.env <<EOF
KUBELET_KUBEADM_ARGS="--network-plugin=cni --pod-infra-container-image=swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/pause:3.6 --cni-bin-dir=/etc/cni/bin"
EOF
[root@k8snode4 ~]# systemctl daemon-reload && systemctl restart kubelet
```

Step 3 Modify the YAML file of Calico.

Modify the Calico configuration file on the **k8smaster1** node.

```
[root@k8smaster1 ~]# cd /root/deploy
[root@k8smaster1 deploy]# vim calico.yaml
```

Change **cni-bin-dir** to **/etc/cni/bin**.

```
# Used to install CNI.
- name: cni-bin-dir
  hostPath:
    path: /etc/cni/bin
```

Re-deploy Calico.

```
[root@k8smaster1 deploy]# kubectl apply -f calico.yaml
```

Wait until all pods of Calico are in the **Running** state and verify that the **k8snode4** node is in the **Ready** state.

```
[root@k8smaster1 deploy]# kubectl get nodes
```

```
[root@k8smaster1 ~]# kubectl get nodes
NAME         STATUS   ROLES                  AGE   VERSION
k8smaster1   Ready    control-plane,master   15d   v1.23.1
k8smaster2   Ready    control-plane,master   15d   v1.23.1
k8smaster3   Ready    control-plane,master   15d   v1.23.1
k8snode1     Ready    <none>                 15d   v1.23.1
k8snode2     Ready    <none>                 15d   v1.23.1
k8snode3     Ready    <none>                 15d   v1.23.1
k8snode4     Ready    <none>                 16m   v1.23.1
```

6.4 CRD, Operator, and Proxy Deployment

Step 1 Create a Dockerfile.

```
[root@k8smaster1 deploy]# cd /root/KubeOS
[root@k8smaster1 KubeOS]# cat > Dockerfile1<<EOF
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/openeuler-22.03-lts-sp3
COPY ./bin/proxy /proxy
ENTRYPOINT ["/proxy"]
EOF
cat > Dockerfile2<<EOF
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/openeuler-22.03-lts-sp3
COPY --chown=6552:6552 ./bin/operator /operator
ENTRYPOINT ["/operator"]
EOF
```

Step 2 Build container images.

```
[root@k8smaster1 KubeOS]# docker build -f Dockerfile1 -t os-proxy:1.0.1 .
[root@k8smaster1 KubeOS]# docker build -f Dockerfile2 -t os-operator:1.0.1 .
```

Step 3 Replicate the images.

Save the images.

```
[root@k8smaster1 example]# docker save -o os-proxy.tar os-proxy
[root@k8smaster1 example]# docker save -o os-operator.tar os-operator
```

Transfer the **os-proxy.tar** image to the **k8snode4** node to be upgraded.

```
[root@k8smaster1 example]# scp os-proxy.tar k8snode4:/var/
```

Transfer the **os-operator.tar** image to other master nodes.

```
[root@k8smaster1 example]# scp os-operator.tar k8smaster2:/var/
[root@k8smaster1 example]# scp os-operator.tar k8smaster3:/var/
```

Load the image on the **k8snode4** node.

```
[root@k8snode4 ~]# cd /var
[root@k8snode4 ~]# docker load -i os-proxy.tar
```

Load the image on **k8smaster2** and **k8smaster3**. The following gives an example of loading the image on **k8smaster2**:

```
[root@k8smaster2 ~]# cd /var
[root@k8smaster2 ~]# docker load -i os-operator.tar
```

Note: In this lab, the built container image is directly copied to each node for loading. You can also upload the built image to a private container image repository or Huawei Cloud SoftWare Repository for Container (SWR). In this case, skip this step.

Step 4 Change image repositories.

Run the following command on the **k8smaster1** node:

```
[root@k8smaster1 KubeOS]# cd /root/KubeOS/docs/example/
```

Open the **config/manager/manager.yaml** file and change the image repositories of **operator** and **proxy**.

```
...
      spec:
        containers:
         - name: proxy
           command:
             - /proxy
           image: os-proxy:1.0.1
...
      spec:
        containers:
        - command:
            - /operator
          image: os-operator:1.0.1
```

Step 5 Deploy the images.

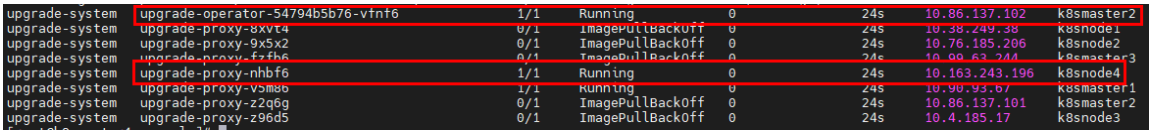
Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 example]# kubectl apply -f config/crd
[root@k8smaster1 example]# kubectl apply -f config/rbac
[root@k8smaster1 example]# kubectl apply -f config/manager
```

Step 6 View results.

```
[root@k8smaster1 example]# kubectl get pods -A -o wide | grep upgrade-system
```

operator runs on a master node, and **proxy** runs on **k8smaster1** and **k8snode4**. If the images of **operator** and **proxy** failed to load on a node, you can ignore the load failure if the node does not need to be upgraded.



6.5 Container OS Upgrade for k8snode4

Step 1 View the version that is not upgraded.

Run the following commands on the **k8smaster1** node:

```
[root@k8smaster1 scripts]# kubectl get nodes -o custom-
columns='NAME:.metadata.name,OS:.status.nodeInfo.osImage'
```

The OS of the node to be upgraded is KubeOS v1, and the OS of the node not to be upgraded is openEuler 22.03 LTS.

```
[root@k8smaster1 ~]# kubectl get nodes -o custom-columns='NAME:.metadata.name,OS:.status.nodeInfo.osImage
NAME         OS
k8smaster1   openEuler 22.03 LTS
k8smaster2   openEuler 22.03 LTS
k8smaster3   openEuler 22.03 LTS
k8snode1     openEuler 22.03 LTS
k8snode2     openEuler 22.03 LTS
k8snode3     openEuler 22.03 LTS
k8snode4     KubeOS v1
```

Step 2 Generate the SHA-256 value used for upgrade.

Generate the SHA-256 value, which is used in the **checksum** field in the YAML file in step 4.

```
[root@k8smaster1 ~]# cd /root/KubeOS/scripts/
[root@k8smaster1 scripts]# sha256sum update.img
```

```
[root@k8smaster1 scripts]# sha256sum update.img
7986f65ce2fafe8c4bf2ed1ae409a26db38b85ec7aa8a8428453d8131600462e  update.img
```

Step 3 Copy **update.img**.

Copy **update.img** to the **nginx** directory for automatic download during the upgrade of **k8snode4**.

```
[root@k8smaster1 scripts]# cp -r update.img /usr/share/nginx/repo
```

Step 4 Declare resources.

```
[root@k8smaster1 scripts]# kubectl apply -
f ../docs/example/config/crd/upgrade.openeuler.org_os.yaml
```

Step 5 Create a YAML file.

Create a YAML file used for upgrade. Change the value of the **checksum** field to the value generated in step 2.

```
[root@k8smaster1 scripts]# cat > upgrade_v1alpha1_os.yaml<<EOF
apiVersion: upgrade.openeuler.org/v1alpha1
kind: OS
metadata:
  name: os-sample
spec:
  imagetype: disk
```

```
  opstype: upgrade
  osversion: KubeOS v2
  maxunavailable: 4
  containerimage: ""
  evictpodforce: true
  imageurl: http://192.168.1.11:80/update.img
  checksum: 7986f65ce2xxxxxxxd8131600462e
  flagSafe: true
  mtls: false
EOF
```

Step 6 Run the upgrade command.

```
[root@k8smaster1 scripts]# kubectl apply -f upgrade_v1alpha1_os.yaml
```

Step 7 Check the upgrade process on the **k8snode4** node.

```
[root@k8snode4 ~]# journalctl -f -u os-agent
```

```
[root@k8snode4 ~]# journalctl -f -u os-agent
-- Journal begins at Fri 2023-12-01 02:05:23 UTC. --
Dec 01 02:05:24 localhost systemd[1]: Started Agent For KubeOS.
Dec 01 02:05:24 localhost os-agent[485]: time="2023-12-01T02:05:24Z" level=info msg="Version is: 1.0.4"
Dec 01 02:05:24 localhost os-agent[485]: time="2023-12-01T02:05:24Z" level=info msg="os-agent start serving"
Dec 01 06:19:14 k8snode4 os-agent[485]: time="2023-12-01T06:19:14Z" level=info msg="start to update to KubeOS v2"
Dec 01 06:19:14 k8snode4 os-agent[485]: time="2023-12-01T06:19:14Z" level=info msg="downloading to file /persist/update.img"
Dec 01 06:19:32 k8snode4 os-agent[485]: time="2023-12-01T06:19:32Z" level=info msg="switching to /dev/vda3 partition B"
Dec 01 06:20:04 k8snode4 os-agent[485]: time="2023-12-01T06:20:04Z" level=info msg="wait to reboot"
```

Step 8 Check the upgrade result.

Run the following command to verify that **k8snode4** has been upgraded to KubeOS v2:

```
[root@k8smaster1 scripts]# kubectl get nodes -o custom-
columns='NAME:.metadata.name,OS:.status.nodeInfo.osImage'
```

```
[root@k8smaster1 scripts]# kubectl get nodes -o custom-columns='NAME:.metadata.name,OS:.status.nodeInfo.osImage
NAME         OS
k8smaster1   openEuler 22.03 LTS
k8smaster2   openEuler 22.03 LTS
k8smaster3   openEuler 22.03 LTS
k8snode1     openEuler 22.03 LTS
k8snode2     openEuler 22.03 LTS
k8snode3     openEuler 22.03 LTS
k8snode4     KubeOS v2
```

Step 9 Clear the environment.

The **k8snode4** node is used only in this lab. After this lab is complete, delete it from the cluster.

```
[root@k8smaster1 scripts]# rm -rf system.img
[root@k8smaster1 scripts]# rm -rf system.qcow2
[root@k8smaster1 scripts]# rm -rf update.img
[root@k8smaster1 scripts]# kubectl delete -f ../docs/example/config/crd
[root@k8smaster1 scripts]# kubectl delete -f ../docs/example/config/rbac
[root@k8smaster1 scripts]# kubectl delete -f ../docs/example/config/manager
[root@k8smaster1 scripts]# kubectl delete nodes k8snode4
```

6.6 Kernel Live Upgrade

Step 1 View the kernel version.

Select a node in the Kubernetes cluster to perform kernel live upgrade. For example, select **k8smaster3**, log in to **k8smaster3**, and run the following command to view the kernel version:

```
[root@k8smaster3 ~]# uname -r
5.10.0-60.116.0.143.oe2203.x86_64
[root@k8smaster3 ~]#
```

Step 2 Download the ISO file.

Download the ISO image of the OS.

```
[root@k8smaster3 ~]# curl -O https://mirror.sjtu.edu.cn/openeuler/openEuler-22.03-LTS/ISO/x86_64/openEuler-22.03-LTS-x86_64-dvd.iso
```

Step 3 Mount the ISO file of openEuler.

```
[root@k8smaster3 ~]# mount openEuler-22.03-LTS-x86_64-dvd.iso /mnt
```

Step 4 Configure a local Yum repository.

```
[root@k8smaster3 ~]# cat > /etc/yum.repos.d/local.repo<<EOF
[local]
name=local
baseurl=file:///mnt
gpgcheck=1
enabled=1
EOF
```

Step 5 Import the GPG public key for RPM digital signatures.

```
[root@k8smaster3 ~]# rpm --import /mnt/RPM-GPG-KEY-openEuler
```

Step 6 Install nvwa.

```
[root@k8smaster3 ~]# yum install nvwa -y
```

Verify that the installation is successful.

```
[root@k8smaster3 ~]# rpm -qa | grep nvwa
```

```
[root@k8smaster3 ~]# rpm -qa | grep nvwa
nvwa-0.2-3.oe2203.x86_64
```

Enable nvwa and check its running status.

```
[root@k8smaster3 ~]# systemctl enable nvwa
[root@k8smaster3 ~]# systemctl start nvwa
```

```
[root@k8smaster3 ~]# service nvwa status
```

```
[root@k8smaster3 ~]# systemctl enable nvwa
Created symlink /etc/systemd/system/nvwa.service → /usr/lib/systemd
Created symlink /etc/systemd/system/multi-user.target.wants/nvwa.se
[root@k8smaster3 ~]# systemctl start nvwa
[root@k8smaster3 ~]# service nvwa status
Redirecting to /bin/systemctl status nvwa.service
● nvwa.service - NVWA server
     Loaded: loaded (/usr/lib/systemd/system/nvwa.service; enabled;
     Active: active (running) since Tue 2023-11-28 02:33:46 UTC; 12
   Main PID: 47276 (nvwa)
      Tasks: 8 (limit: 49040)
     Memory: 7.7M
     CGroup: /system.slice/nvwa.service
             └─ 47276 /usr/bin/nvwa -server 1

Nov 28 02:33:46 k8smaster3 systemd[1]: Starting NVWA server...
Nov 28 02:33:46 k8smaster3 systemd[1]: Started NVWA server.
```

Step 7 Install the kernel of a later version.

Run the following commands to download and install the kernel of a later version:

```
[root@k8smaster3 ~]# curl -O https://repo.huaweicloud.com/openeuler/openEuler-22.03-LTS/update/x86_64/Packages/kernel-5.10.0-60.117.0.144.oe2203.x86_64.rpm
[root@k8smaster3 ~]# rpm -ivh kernel-5.10.0-60.117.0.144.oe2203.x86_64.rpm
```

Check the kernel installation result.

```
[root@k8smaster3 ~]# ls /boot
```

```
config-5.10.0-60.116.0.143.oe2203.x86_64
config-5.10.0-60.117.0.144.oe2203.x86_64
config-5.10.0-60.18.0.50.oe2203.x86_64
dracut
efi
grub2
initramfs-5.10.0-60.116.0.143.oe2203.x86_64.img
initramfs-5.10.0-60.116.0.143.oe2203.x86_64kdump.img
initramfs-5.10.0-60.117.0.144.oe2203.x86_64.img
initramfs-5.10.0-60.18.0.50.oe2203.x86_64.img
initramfs-5.10.0-60.18.0.50.oe2203.x86_64kdump.img
loader
symvers-5.10.0-60.116.0.143.oe2203.x86_64.gz
symvers-5.10.0-60.117.0.144.oe2203.x86_64.gz
symvers-5.10.0-60.18.0.50.oe2203.x86_64.gz
System.map-5.10.0-60.116.0.143.oe2203.x86_64
System.map-5.10.0-60.117.0.144.oe2203.x86_64
System.map-5.10.0-60.18.0.50.oe2203.x86_64
vmlinuz-5.10.0-60.116.0.143.oe2203.x86_64
vmlinuz-5.10.0-60.117.0.144.oe2203.x86_64
vmlinuz-5.10.0-60.18.0.50.oe2203.x86_64
```

Step 8 Upgrade the kernel.

Run the following command to perform a kernel live upgrade. The SSH window is automatically disconnected, but the OS does not restart. Log in to the OS again.

```
[root@k8smaster3 ~]# nvwa update 5.10.0-60.117.0.144.oe2203.x86_64
```

Step 9 View the kernel version.

```
[root@k8smaster3 ~]# uname -r
```

```
[root@k8smaster3 ~]# uname -r
5.10.0-60.117.0.144.oe2203.x86_64
```

Step 10 (Optional) Clear upgrade information.

```
[root@k8smaster3 ~]# nvwa init
```

Step 11 Roll back the kernel version.

Run the **nvwa update** command again and select the kernel version before the upgrade to roll back the kernel to this version.

```
[root@k8smaster3 ~]# nvwa update 5.10.0-60.116.0.143.oe2203.x86_64
```

7 Hybrid Container Deployment

7.1 Introduction

7.1.1 About This Exercise

This lab uses Rubik to implement hybrid deployment of online and offline Kubernetes services.

7.1.2 Objectives

- Install Rubik.
- Understand hybrid deployment of services.

7.2 Rubik Installation and Deployment

Step 1 Install Rubik.

Run the following command on all Kubernetes nodes:

```
sudo yum install -y rubik
```

Step 2 Enable the QoS function.

Run the following command on all Kubernetes nodes:

```
sudo echo 1 > /proc/sys/vm/memcg_qos_enable
```

Step 3 Create a Rubik image.

Run the following command on all Kubernetes nodes:

```
cd /var/lib/rubik
docker build -f /var/lib/rubik/Dockerfile -t rubik:0.1.0-2 .
```

Step 4 Deploy Rubik.

Run the following command on the **k8smaster1** node to deploy Rubik:

```
kubectl apply -f /var/lib/rubik/rubik-daemonset.yaml
```

View Rubik.

```
[root@k8smaster1 rubik]# kubectl get pods -A | grep rubik
kube-system   rubik-agent-b92hl                          1/1     Running
kube-system   rubik-agent-lsrbt                          1/1     Running
kube-system   rubik-agent-mf74q                          1/1     Running
kube-system   rubik-agent-rf7kh                          1/1     Running
kube-system   rubik-agent-t9s5w                          1/1     Running
kube-system   rubik-agent-zkh99                          1/1     Running
```

7.3 Hybrid Deployment Test

Step 1 Create a working directory.

```
[root@k8smaster1 ~]# mkdir rubik
[root@k8smaster1 ~]# cd rubik
[root@k8smaster1 rubik]#
```

Step 2 Compile a test script.

```
[root@k8smaster1 rubik]# vim rubik_demo.sh
#!/bin/bash

with_offline=${1:-no_offline}
enable_isolation=${2:-no_isolation}
stress_num=${3:-10}
concurrency=10
timeout=30
output=/tmp/result.json
online_container=
offline_container=

exec_sql="echo \"SELECT * FROM system.numbers LIMIT 10000000 OFFSET 10000000\" | clickhouse-
benchmark -i 10000 -c $concurrency -t $timeout"

function prepare()
{
    echo "Launch clickhouse container."
    online_container=$(docker run -itd \
            -v /tmp:/tmp:rw \
            --ulimit nofile=262144:262144 \
            -p 34424:34424 \
            yandex/clickhouse-server)

    sleep 3
    echo "Clickhouse container lauched."
}

function clickhouse()
{
    echo "Start clickhouse benchmark test."
    docker exec $online_container bash -c "$exec_sql --json $output"
    echo "Clickhouse benchmark test done."
}

function stress()
```

```
{
    echo "Launch stress container."
    offline_container=$(docker run -itd joedval/stress --cpu $stress_num)
    echo "Stress container launched."

    if [ $enable_isolation == "enable_isolation" ]; then
        echo "Set stress container qos level to -1."
        echo -1 > /sys/fs/cgroup/cpu/system.slice/docker-$offline_container.scope/cpu.qos_level
    fi
}

function benchmark()
{
    if [ $with_offline == "with_offline" ]; then
     stress
     sleep 3
 fi
    clickhouse
    echo "Remove test containers."
    docker rm -f $online_container
    docker rm -f $offline_container
    echo "Finish benchmark test for clickhouse(online) and stress(offline) colocation."
    echo "==============================clickhouse
benchmark================================================="
    cat $output
    echo "==============================clickhouse
benchmark================================================="
}

prepare
benchmark
```

Step 3 Test the online service.

The **no_offline** parameter indicates that the offline Stress service is disabled, and the **no_isolation** parameter indicates that the isolation function for hybrid deployed services is disabled.

```
[root@k8smaster1 rubik]# sh rubik_demo.sh no_offline no_isolation
```

Do not interrupt the execution until the result in the screenshot is automatically printed.

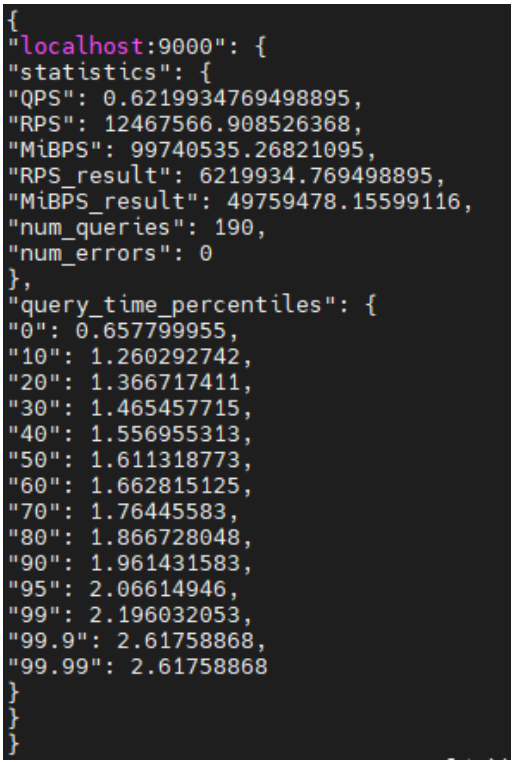
```
{
"localhost:9000": {
"statistics": {
"QPS": 1.2231429315355373,
"RPS": 24517325.18545202,
"MiBPS": 196138601.48361617,
"RPS_result": 12231429.315355372,
"MiBPS_result": 97851434.52284297,
"num_queries": 370,
"num_errors": 0
},
"query_time_percentiles": {
"0": 0.388230866,
"10": 0.620726583,
"20": 0.68645872,
"30": 0.749026271,
"40": 0.785462086,
"50": 0.827530979,
"60": 0.858735239,
"70": 0.905610762,
"80": 0.943555225,
"90": 1.002876397,
"95": 1.059682818,
"99": 1.125606613,
"99.9": 1.221566135,
"99.99": 1.221566135
}
}
}
```


Step 4 Test the offline service.

The **with_offline** parameter indicates that the offline Stress service is enabled, and the **no_isolation** parameter indicates that the isolation function for hybrid deployed services is disabled.

```
[root@k8smaster1 rubik]# sh rubik_demo.sh with_offline no_isolation
```

Do not interrupt the execution until the result in the screenshot is automatically printed.



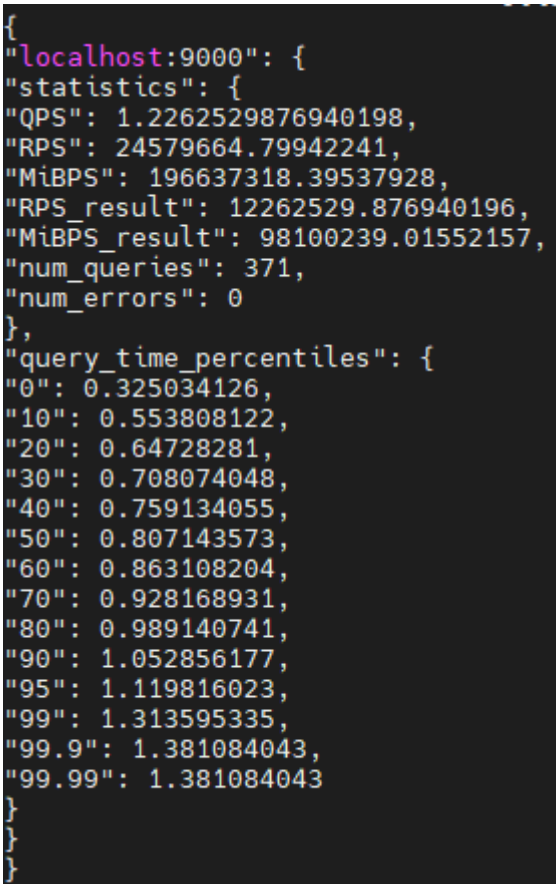
```
{
"localhost:9000": {
"statistics": {
"QPS": 0.6219934769498895,
"RPS": 12467566.908526368,
"MiBPS": 99740535.26821095,
"RPS_result": 6219934.769498895,
"MiBPS_result": 49759478.15599116,
"num_queries": 190,
"num_errors": 0
},
"query_time_percentiles": {
"0": 0.657799955,
"10": 1.260292742,
"20": 1.366717411,
"30": 1.465457715,
"40": 1.556955313,
"50": 1.611318773,
"60": 1.662815125,
"70": 1.76445583,
"80": 1.866728048,
"90": 1.961431583,
"95": 2.06614946,
"99": 2.196032053,
"99.9": 2.61758868,
"99.99": 2.61758868
}
}
}
```


Step 5 Enable the isolation function for hybrid deployed services.

The **with_offline** parameter indicates that the offline Stress service is enabled, and the **enable_isolation** parameter indicates that the isolation function for hybrid deployed services is enabled.

```
[root@k8smaster1 rubik]# sh rubik_demo.sh with_offline enable_isolation
```

Do not interrupt the execution until the result in the screenshot is automatically printed.



```
{
"localhost:9000": {
"statistics": {
"QPS": 1.2262529876940198,
"RPS": 24579664.79942241,
"MiBPS": 196637318.39537928,
"RPS_result": 12262529.876940196,
"MiBPS_result": 98100239.01552157,
"num_queries": 371,
"num_errors": 0
},
"query_time_percentiles": {
"0": 0.325034126,
"10": 0.553808122,
"20": 0.64728281,
"30": 0.708074048,
"40": 0.759134055,
"50": 0.807143573,
"60": 0.863108204,
"70": 0.928168931,
"80": 0.989140741,
"90": 1.052856177,
"95": 1.119816023,
"99": 1.313595335,
"99.9": 1.381084043,
"99.99": 1.381084043
}
}
}
```


When the isolation function for hybrid deployed services is disabled, the QPS of ClickHouse decreases from approximately 1.2 to 0.6, the service response delay (P90) increases from approximately 1.0s to 1.9s, and the QoS decreases by about 50%. When the isolation function for hybrid deployed services is enabled, the QPS and response delay (P50/P90/P99) of ClickHouse decrease by less than 2% compared with the baseline, and the QoS remains unchanged.

Huawei openEuler Certification Training

HCIE-openEuler

Lab Guide

Issue: 1.0



Huawei Technologies Co., Ltd.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Huawei Certification covers all ICT fields and follows the industry's trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talents in the digital era, and building a sound ICT talent ecosystem.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. HCIE-openEuler certification covers the Ceph cloud storage management, the cloud operating system (OS) cluster management, advanced practices of cloud native platform Kubernetes, advanced automation management, continuous integration and continuous delivery (CI/CD), intelligent operation and maintenance (O&M) platform A-Ops, and log management system EFK.

Huawei certification helps you unlock opportunities to advance your career and take one more step towards the top of the industry.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Introduction

This document is intended for candidates who are preparing for the HCIE-openEuler exam and readers who want to know how to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management on openEuler.

Description

This document consists of eight exercises, including basic usage of Git and GitLab, advanced configuration related to automatic management, AWX installation, deployment, and configuration, and Jenkins-based CI/CD.

- Exercise 1: provides a detail description of basic Git operations, including Git and global configurations and Git repository creation.
- Exercise 2: describes GitLab installation and basic use, including GitLab project creation, user creation, and code pull and push.
- Exercise 3: describes facts and magic variables, including extracting IP addresses through facts, customizing facts, and obtaining information through magic variables.
- Exercise 4: describes conditional and loop statements in a playbook. The conditional statement contains multiple types of operations, and the loop statement uses a file to create multiple users.
- Exercise 5: describes Ansible roles and how to set up an httpd cluster using roles.
- Exercise 6: describes the Ansible privilege escalation configuration and jinja2 template. This exercise, combining configurations of other exercises, illustrates the concepts presented in the matched PowerPoint documents.
- Exercise 7: describes AWX-related practices, including AWX installation and basic AWX operations.
- Exercise 8: describes Jenkins-based CI/CD process, including Jenkins installation, basic Jenkins operations, CI/CD process of Jenkins + GitLab + AWX, and CI/CD process of Jenkins + Kubernetes + Harbor.

Background Knowledge Required

This course is for Huawei certification. To better understand this course, the intended audiences are advised to meet the following requirement:

- Know Linux basics. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Lab Environment Overview

Three ECSs are needed to implement advanced automation management, and one ECS is required to host GitLab and Harbor. Additionally, some exercises will be conducted using the environments in cloud storage and cloud-native.

Allow both inbound and outbound traffic for the ECS security group.

Lab Environment Preparation

Checking Devices

Before starting the exercises, each group of trainees should apply for ECSs on Huawei Cloud according to the following table.

ECS Name	Specifications	Remarks
GitLab	4 vCPUs | 8 GiB | s7.xlarge.2	
Controller	1 vCPUs | 1 GiB | s7.small.1	
Client-1	1 vCPUs | 1 GiB | s7.small.1	
Client-2	1 vCPUs | 1 GiB | s7.small.1	

Contents

About This Document 3
Introduction 3
Description 3
Background Knowledge Required 3
Lab Environment Overview 4
Lab Environment Preparation 4
1 Basic Git Operations 1
1.1 Introduction 1
1.1.1 About This Exercise 1
1.1.2 Objectives 1
1.2 Basic Git Operations 1
1.3 Git Repository Branch Management 3
2 GitLab Installation and Usage 4
2.1 Introduction 4
2.1.1 About This Exercise 4
2.1.2 Objectives 4
2.2 GitLab Installation 4
2.3 Basic GitLab Operations 7
3 Facts and Magic Variables 15
3.1 Introduction 15
3.1.1 About This Exercise 15
3.1.2 Objectives 15
3.2 Usage of Facts 15
3.3 Customizing Facts 17
3.4 Usage of Magic Variables 18
3.4.1 Functions of hostvars 18
3.4.2 Functions of inventory_hostname 20
3.5 Usage of Lookup Plugins 20
3.5.1 Usage of the File Module 20
3.5.2 Usage of the Password Module 21
3.5.3 Usage of the env Module 21
4 Conditional and Loop Statements of a Playbook 23
4.1 Introduction 23
4.1.1 About This Exercise 23

4.1.2 Objectives 23
4.2 Usage of Conditional Statements 23
4.2.1 Comparison Operations in a Playbook 23
4.2.2 Logical Operation in a Playbook 24
4.2.3 Path Determination in a Playbook 24
4.2.4 Task Execution Result Determination in a Playbook 25
4.2.5 Usage of Block 27
4.3 Usage of Loop Statements 29
5 Usage of Ansible Roles 31
5.1 Introduction 31
5.1.1 About This Exercise 31
5.1.2 Objectives 31
5.2 Usage of Roles 31
6 Ansible Privilege Escalation and Jinja2 Template Usage 35
6.1 Introduction 35
6.1.1 About This Exercise 35
6.1.2 Objectives 35
6.2 Ansible Privilege Escalation 35
6.3 Usage of the Jinja2 Template 36
7 AWX Installation, Deployment, Configuration, and Usage 37
7.1 Introduction 37
7.1.1 About This Exercise 37
7.1.2 Objectives 37
7.2 AWX Installation and Deployment 37
7.2.1 Preparing the Ceph Resource Pool 37
7.2.2 Configuring Ceph-csi Resources 38
7.2.3 AWX Installation and Deployment 41
7.3 Basic AWX Operations 47
7.3.1 AWX Initial Experience 47
7.3.2 Interconnecting with GitLab and Performing Operations 53
7.3.3 Creating a Workflow Template 55
7.4 Performing Comprehensive AWX Practices 59
8 Jenkins Installation, Deployment, Configuration, and Usage 68
8.1 Introduction 68
8.1.1 About This Exercise 68
8.1.2 Objectives 68
8.2 Jenkins Installation and Deployment 68
8.2.1 Storage Resource Preparations 68

8.2.2 Jenkins Installation 71
8.3 Basic Jenkins Operations 74
8.3.1 Resource Preparations 74
8.3.2 Primary Operations of Jenkins CI/CD 76
8.3.3 Primary Usage of Jenkins Pipelines 87
8.4 Comprehensive Practices of Jenkins 91
8.4.1 Comprehensive Practice of Jenkins + GitLab + AWX 91
8.4.2 Dynamic Slave Implemented on Jenkins + Harbor + Kubernetes 98

1 Basic Git Operations

1.1 Introduction

1.1.1 About This Exercise

This exercise describes basic git operations, including creating and initializing a repository. This exercise is performed locally and does not involve a remote repository.

1.1.2 Objectives

This exercise mainly illustrates the content of "Basic Usage of Git" in *Introduction to the CI/CD Tool Chain and GitLab*, to enhance the understanding of trainees.

1.2 Basic Git Operations

Example: Create a Git repository **MyGit** in the local directory and initialize it. Create **README.md** of a project in the directory, with the content "this is my first repository". Create an Ansible playbook for installing the Nginx service.

Step 1 Create and initialize a Git repository.

Run the **yum install -y git** command to install Git and create a **MyGit** directory.

Run the following commands to configure Git user information:

```
[root@controller MyGit]# git config --global user.name "hcie"
[root@controller MyGit]# git config --global user.email openeuler@hcie.com
```

Check whether the configuration takes effect.

```
[root@controller MyGit]# git config --list
user.name=hcie
user.email=openeuler@hcie.com
core.repositoryformatversion=0
core.filemode=true
```

Go to the **MyGit** directory and run the following command to initialize the repository:

```
[root@controller MyGit]# git init
```

After the initialization is complete, check whether the related directory file **.git** is generated.

```
[root@controller ~]# cd MyGit/
[root@controller MyGit]# git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /root/MyGit/.git/
[root@controller MyGit]# ls -a
.  ..  .git
```

Step 2 Create related files and push them to the repository.

Create related files in the **MyGit** directory as required.

```
[root@controller MyGit]# echo "this is my first repository" > README.md
[root@controller MyGit]# cat >> nginx.yml << EOF
---
- hosts: all
  remote_user: root
  gather_facts: false

  tasks:
    - yum:
      name: nginx
      state: present
EOF
```

- Task: Run the **git status** command to check the current repository status.

```
[root@controller MyGit]# git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        nginx.yml

nothing added to commit but untracked files present (use "git add" to track)
```

Run the following command to add related files to a stage:

```
[root@controller MyGit]# git add .
```

Check the current repository status.

```
[root@controller MyGit]# git add .
[root@controller MyGit]# git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
        new file:   nginx.yml
```

Run the following command to submit files to the repository:

```
[root@controller MyGit]# git commit -m "first version"
```

Check the status of the current repository again.

```
[root@controller MyGit]# git commit -m "first version"
[master (root-commit) b068e6b] first version
 2 files changed, 10 insertions(+)
 create mode 100644 README.md
 create mode 100644 nginx.yml
[root@controller MyGit]# git status
On branch master
nothing to commit, working tree clean
```

1.3 Git Repository Branch Management

Example: Create a branch for the created repository and use Apache to replace Nginx.

Step 1 Create a branch.

Run the following command to create a branch and name it "apache":

```
[root@controller MyGit]# git checkout -b apache
```

In the current branch, run the following command to replace Nginx with httpd in the **nginx.yml** file:

```
[root@controller MyGit]# sed -i "s/nginx/httpd/g" nginx.yml
```

After the operation is complete, check the current repository status.

```
[root@controller MyGit]# git status
On branch apache
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   nginx.yml

no changes added to commit (use "git add" and/or "git commit -a")
```

Push files in the branch to the repository.

```
[root@controller MyGit]# git add .
[root@controller MyGit]# git  commit -m "apache"
[apache ba5f10c] apache
 1 file changed, 1 insertion(+), 1 deletion(-)
[root@controller MyGit]# git status
On branch apache
nothing to commit, working tree clean
```

2 GitLab Installation and Usage

2.1 Introduction

2.1.1 About This Exercise

This exercise describes how to use GitLab and how to push, update, and download code based on Git.

2.1.2 Objectives

This exercise mainly illustrates the content of GitLab installation, deployment, and basic usage in *Introduction to the CI/CD Tool Chain and GitLab*, to enhance the understanding of trainees.

2.2 GitLab Installation

Step 1 Install a container engine.

Run the following commands to configure the yum source:

```
cat > /etc/yum.repos.d/kubernetes.repo<<EOF
[kubernetes]
name=Kubernetes
baseurl=https://repo.huaweicloud.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=1
repo_gpgcheck=0
gpgkey=https://repo.huaweicloud.com/kubernetes/yum/doc/yum-key.gpg
https://repo.huaweicloud.com/kubernetes/yum/doc/rpm-package-key.gpg
EOF
```

After the configuration is complete, run the following command to install Docker:

```
yum install -y docker
```

After the installation is complete, run the following command to enable Docker automatic startup.

```
systemctl enable docker --now
```

Step 2 Install GitLab.

Before installing GitLab, use **$GITLAB_HOME** to point to the directory where related configurations, logs, and data files are located, and ensure that the directory has proper permissions.

```
[root@gitlab ~]# mkdir gitlab
[root@gitlab ~]# echo "GITLAB_HOME=/root/gitlab" >> /etc/exports
[root@gitlab ~]# source /etc/exports
[root@gitlab ~]# echo $GITLAB_HOME
/root/gitlab
```

After the configuration is complete, run the following commands to start the GitLab container:

```
sudo docker run --detach \
  --hostname gitlab.example.com \
  --publish 443:443 --publish 80:80 --publish 202:22 \
  --name gitlab \
  --restart always \
  --volume $GITLAB_HOME/config:/etc/gitlab \
  --volume $GITLAB_HOME/logs:/var/log/gitlab \
  --volume $GITLAB_HOME/data:/var/opt/gitlab \
  --shm-size 256m \
  registry.gitlab.cn/omnibus/gitlab-jh:latest
```

- Question: Based on your knowledge and analysis of container startup commands, can you see how many ports are opened in the container? Does the container automatically start after the physical node is restarted?
- Answer: Currently, three ports are developed for the container: 443, 80, and 202. The **--restart always** parameter enables the container to start with the physical node.

After the container is created, run the following command to view the default password of GitLab:

```
[root@gitlab ~]# sudo docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
```

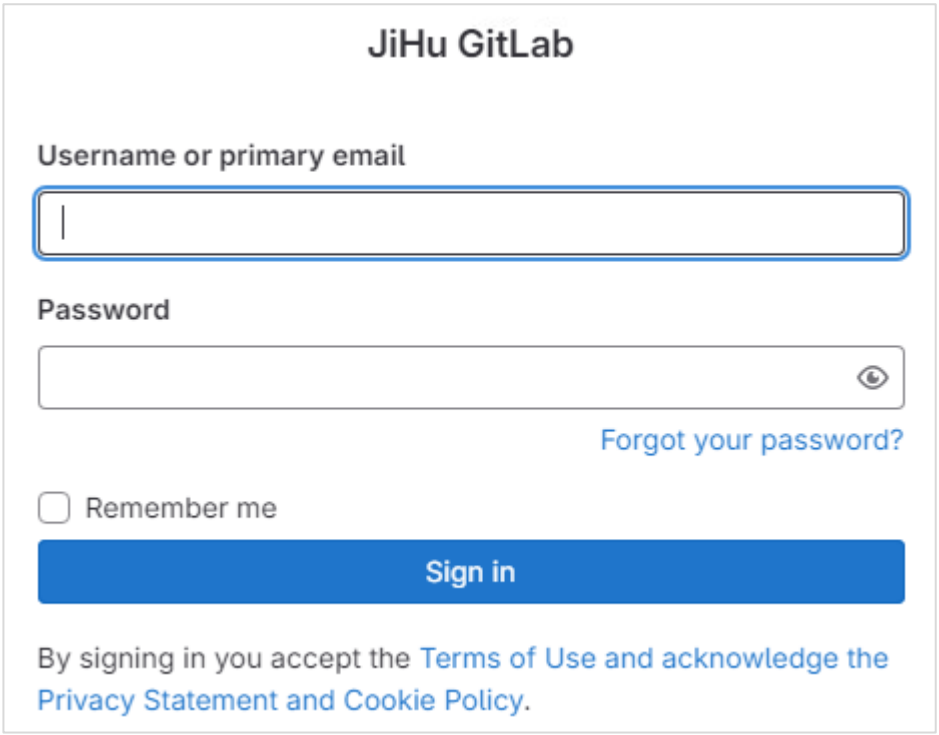
```
[root@gitlab ~]# docker ps
CONTAINER ID   IMAGE                                      COMMAND             CREATED
74dadb5046a9   registry.gitlab.cn/omnibus/gitlab-jh:latest   "/assets/wrapper"   7 seconds ago
[root@gitlab ~]# sudo docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
Password: Qz0vCKd/vCkmi9PIptaUpqXU0O9qzRHd9a2EfkMqLXA=
[root@gitlab ~]#
```

It takes a long time to start GitLab in the container. You can run the **docker logs -f gitlab** command to trace the startup process. If the following information is displayed, GitLab initialization is complete.

```
==> /var/log/gitlab/gitlab-exporter/current <==
2023-11-13_06:33:41.55440 127.0.0.1 - - [13/Nov/2023:06:33:41 UTC] "GET /sidekiq HTTP/1.1" 200 188360
2023-11-13_06:33:41.55445 - -> /sidekiq
2023-11-13_06:33:45.52130 127.0.0.1 - - [13/Nov/2023:06:33:45 UTC] "GET /ruby HTTP/1.1" 200 1068
2023-11-13_06:33:45.52158 - -> /ruby
2023-11-13_06:33:46.52040 127.0.0.1 - - [13/Nov/2023:06:33:46 UTC] "GET /database HTTP/1.1" 200 1671
2023-11-13_06:33:46.52071 - -> /database

==> /var/log/gitlab/gitlab-rails/production_json.log <==
{"method":"GET","path":"/-/metrics","format":"html","controller":"MetricsController","action":"index","
unt":0,"db_replica_count":0,"db_primary_count":0,"db_main_count":0,"db_ci_count":0,"db_main_replica_cou
ount":0,"db_ci_cached_count":0,"db_main_replica_cached_count":0,"db_ci_replica_cached_count":0,"db_repl
l_count":0,"db_ci_replica_wal_count":0,"db_replica_wal_cached_count":0,"db_primary_wal_cached_count":0,
lica_wal_cached_count":0,"db_replica_duration_s":0.0,"db_primary_duration_s":0.0,"db_main_duration_s":0
42,"mem_objects":3263,"mem_bytes":1686104,"mem_mallocs":919,"mem_total_bytes":1816624,"pid":747,"worker
tion_s":0.0,"view_duration_s":0.00039,"duration_s":0.01851}
```

After GitLab initialization is complete, manually resolve **gitlab.example.com** to the EIP of the ECS where GitLab is located. Then, check whether you can log in to the GitLab.



The default user name of GitLab is **root**, and the password is the one queried previously.

Step 3 Configure GitLab.

Run the **docker exec -it gitlab /bin/bash** command to log in to the container, modify the GitLab configuration file, and complete basic URL and SSH configurations.

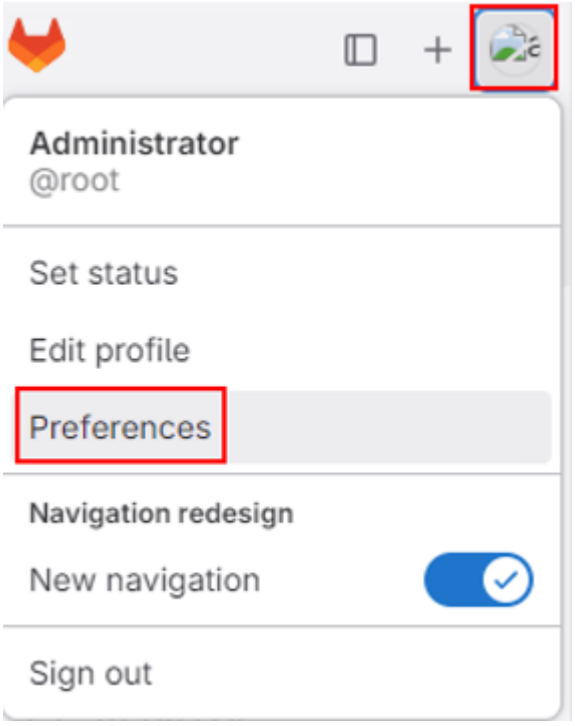
```
[root@gitlab /]# vi /etc/gitlab/gitlab.rb
.......
external_url 'http://gitlab.example.com'
......
gitlab_rails['gitlab_ssh_host'] = 'gitlab.example.com'
......
gitlab_rails['gitlab_shell_ssh_port'] = 202
```

After the modification, run the **gitlab-ctl reconfigure** command to make the configuration take effect.

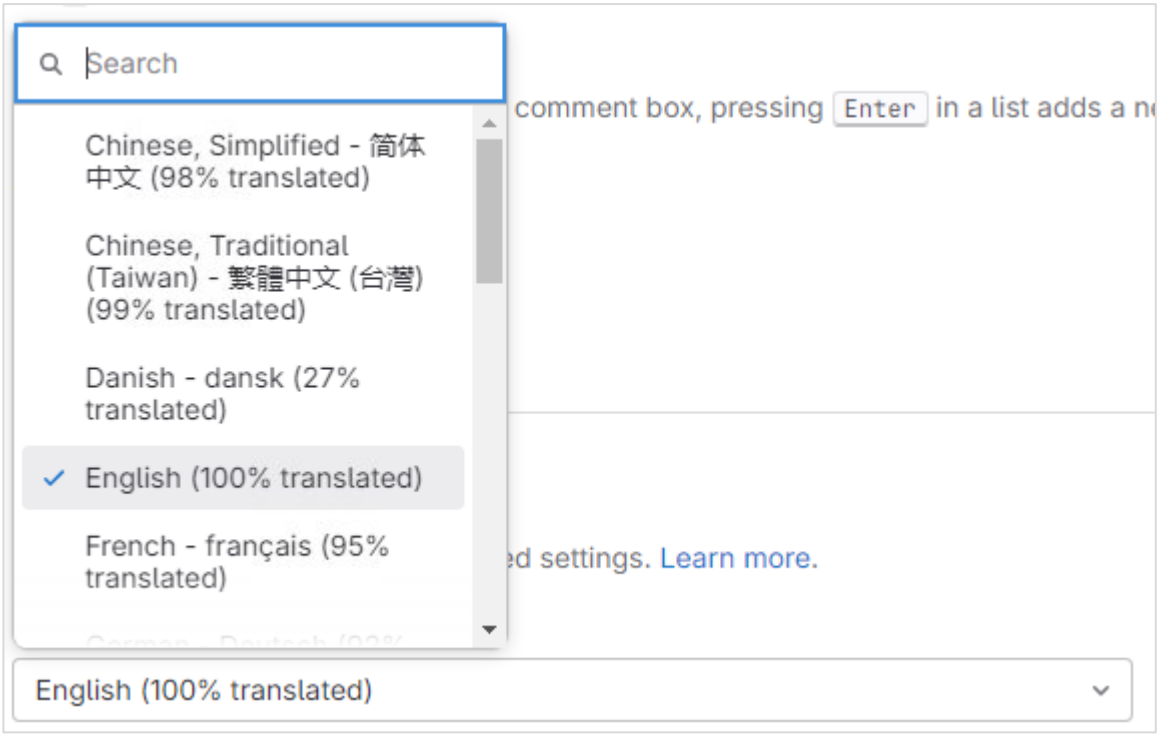
2.3 Basic GitLab Operations

Step 1 Initialize the password and switch the GUI language.

After logging in to GitLab, click your avatar in the upper left corner and select **Preferences** to switch the GUI language.

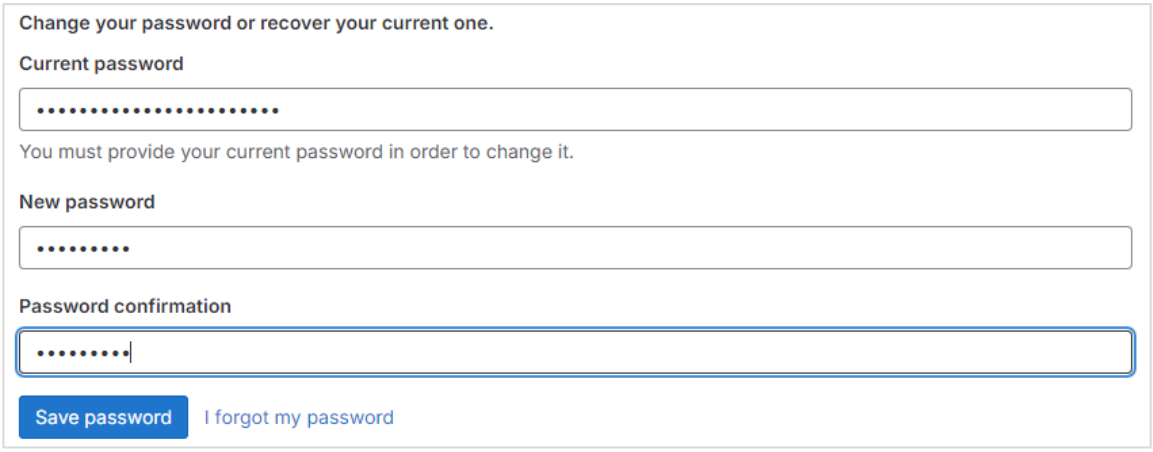


On the new page, scroll the mouse to the bottom, find **Language**, and select a desired language.



Click **Save changes** and refresh the page for the new configuration to take effect.

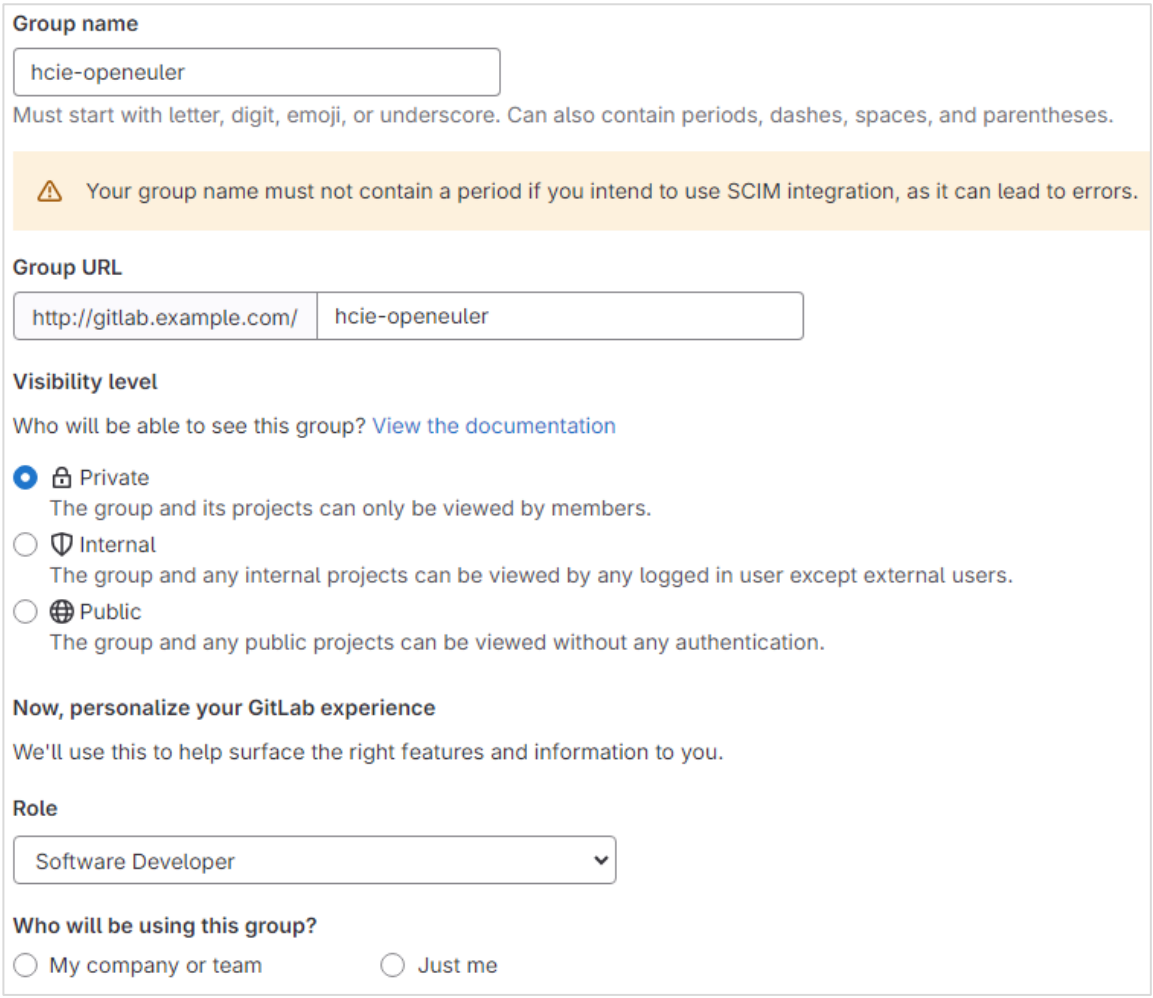
On the current page, click **Password** to set a password.



After the configuration is complete, log in to the system again.

Step 2 Create a group.

After logging in to GitLab, click **Groups** to go to the group creation page, click **New Group** to go to the group configuration page, and type information as prompted.

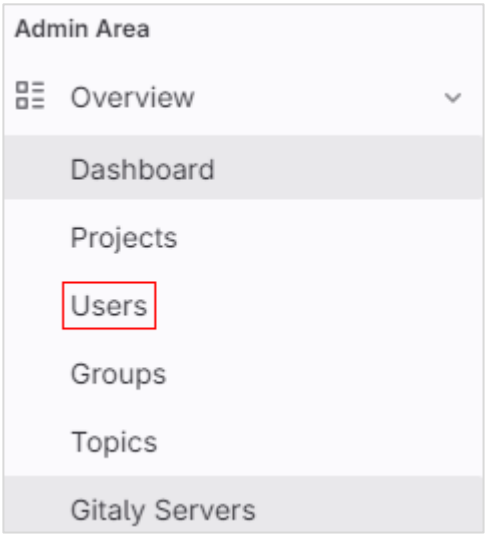


Step 3 Create a user.

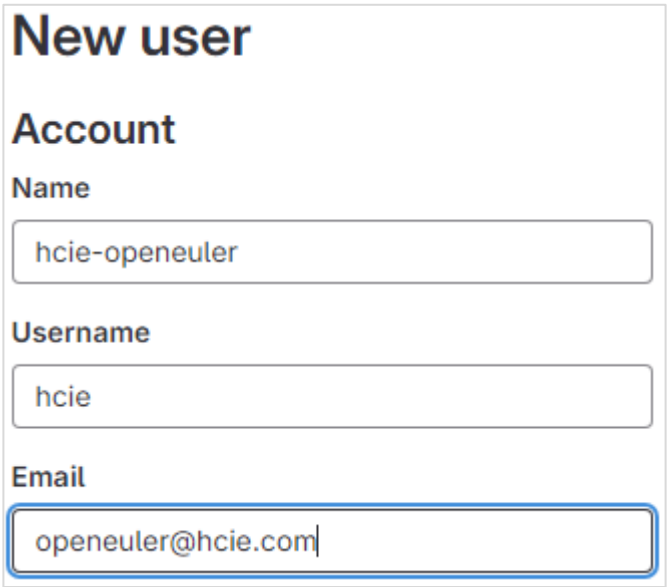
Enter **/** in the search area on the left to quickly jump to the **Admin Area**.



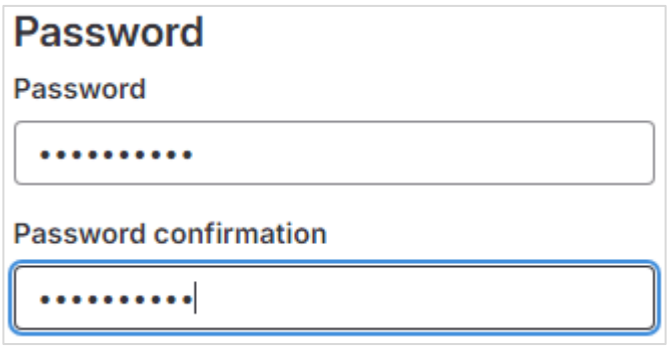
Select **Users**.



Click **New User** in the upper right corner of the page. In the displayed window, enter the user information as prompted.

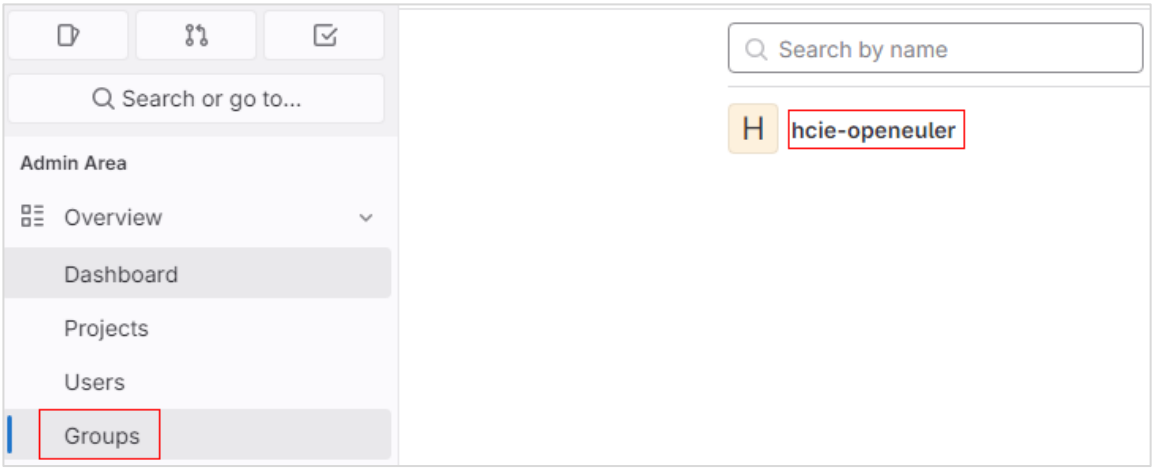


Once a user is created, the system automatically navigates to the user list page. Click the newly created user to access the user information page. To change the password, click **Edit** in the upper right corner.

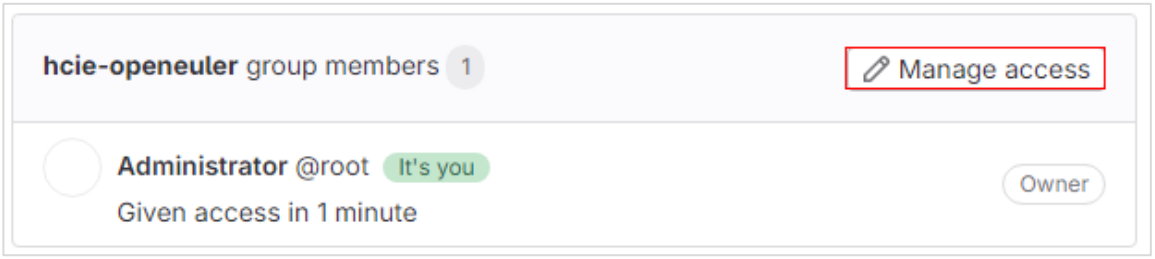


Step 4 Add the user to a group.

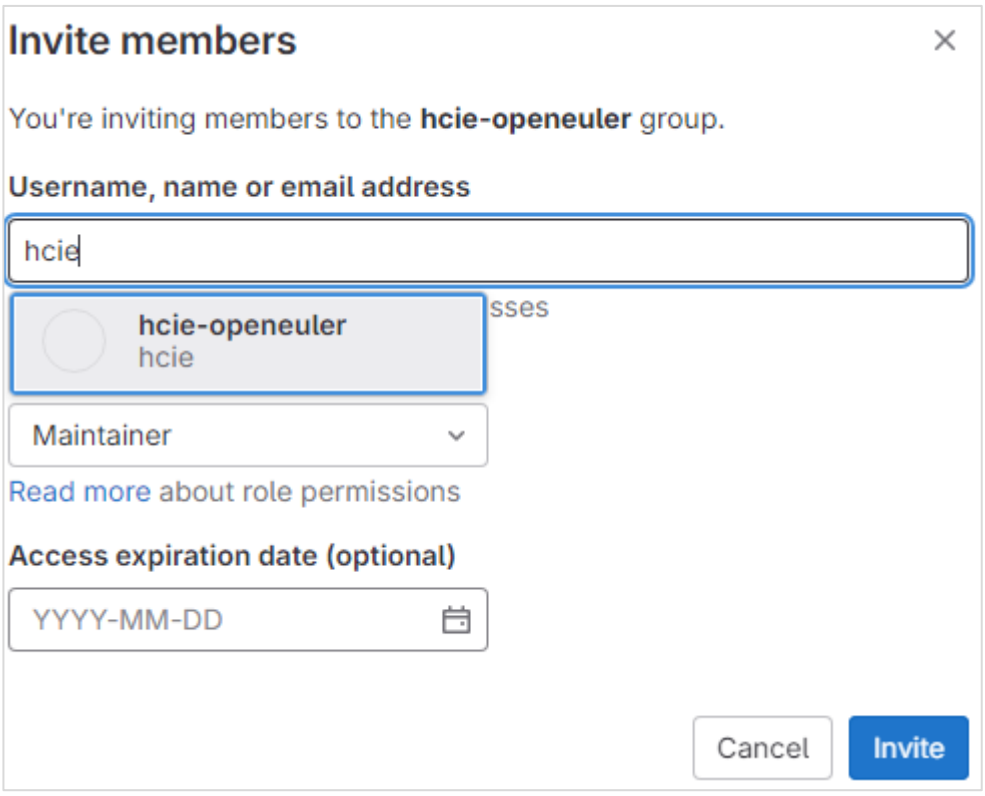
Return to the **Admin Area** page and click **Groups** to jump to the group configuration page.



Click the group name to enter the group management page, and then click **Manage access**.



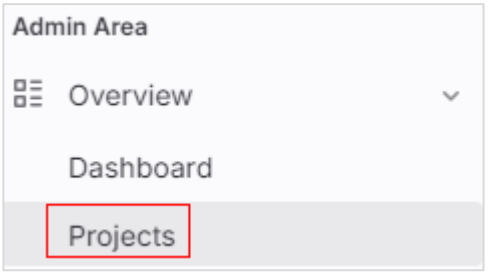
On the displayed page, click **Invite members** in the upper right corner. Enter the created user name in the **Username, name or email address** text box. The system automatically identifies and selects the user. Set a role for the member (**Maintainer** used in this exercise) and click **Invite**.

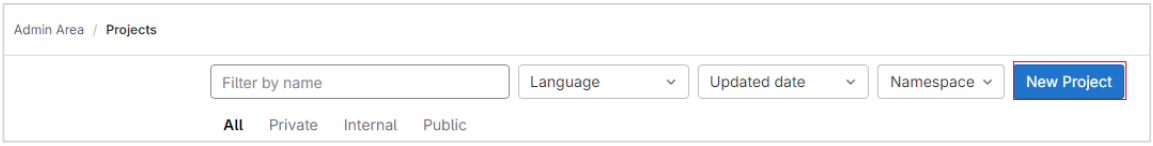


Step 5 Create a project.

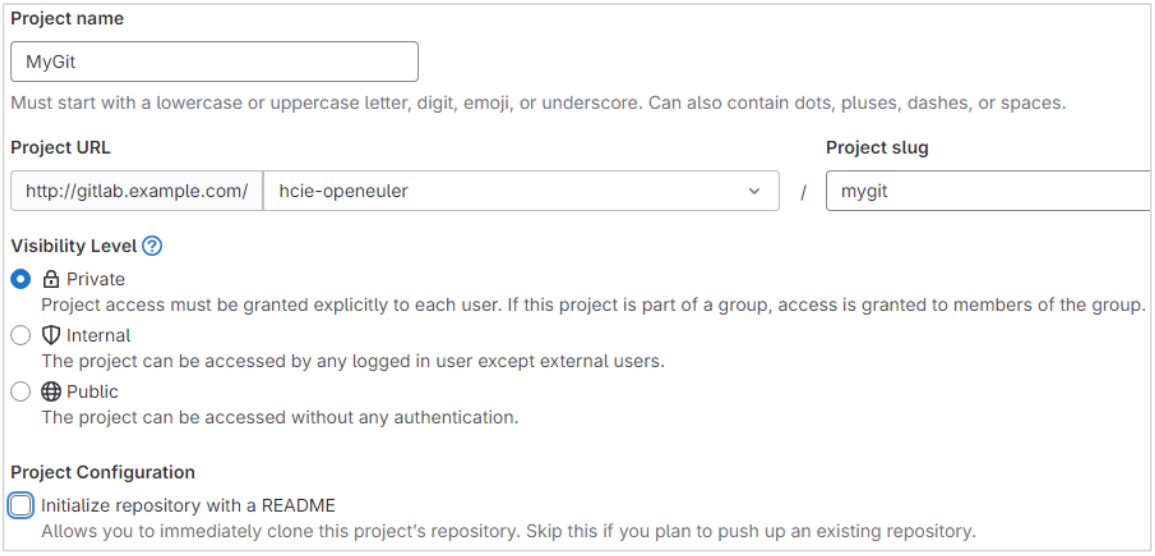
In this exercise, the code created in section "Basic Git Operations" is pushed to the project in GitLab.

Go back to the **Admin Area** page and click **Projects**. On the project configuration page that is displayed, click **New Project**.





Click **Create blank project**, and enter related configurations in the displayed page as prompted.



After a project is created, the system automatically enters the project and provides command line guidance, including Git global settings and repository creation. Because a new folder has been created in exercise 1, the following uses **Push an existing folder** to push code in the **MyGit** directory to GitLab.

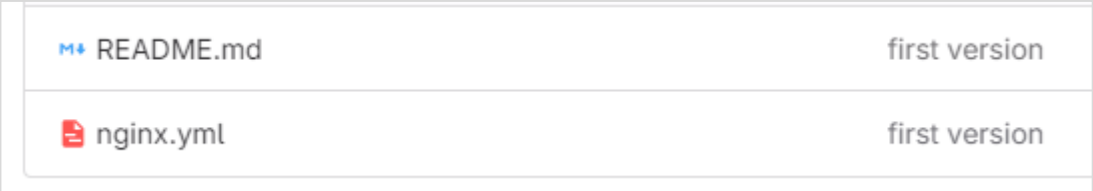
Go to the **MyGit** directory on the client and run the following commands as prompted:

```
[root@controller MyGit]# git init --initial-branch=main
warning: re-init: ignored --initial-branch=main
Reinitialized existing Git repository in /root/MyGit/.git/
[root@controller MyGit]# git remote add origin http://gitlab.example.com/hcie-openeuler/mygit.git
[root@controller MyGit]# git add .
[root@controller MyGit]# git commit -m "Initial commit"
On branch master
nothing to commit, working tree clean
[root@controller MyGit]# git push --set-upstream origin master
```

The system prompts you to enter the user name and password. In this exercise, the **root** user is used for push. Therefore, enter **root** and the corresponding password to push the code to GitLab.

```
[root@controller MyGit]# git push --set-upstream origin master
Username for 'http://gitlab.example.com': root
Password for 'http://root@gitlab.example.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 364 bytes | 364.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To http://gitlab.example.com/hcie-openeuler/mygit.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

You can view the pushed code in GitLab, as shown in the following figure.



- Task: Push the **apache** branch to GitLab.
- Reference: Run the following commands to push the **apache** branch:

```
[root@controller MyGit]# git checkout apache
Switched to branch 'apache'
[root@controller MyGit]# git push --set-upstream origin apache
```

```
[root@controller MyGit]# git push --set-upstream origin apache
Username for 'http://gitlab.example.com': root
Password for 'http://root@gitlab.example.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 297 bytes | 297.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for apache, visit:
remote:   http://gitlab.example.com/hcie-openeuler/mygit/-/merge_requests/new?merge_request%5Bsource_branch%5D=apache
remote:
To http://gitlab.example.com/hcie-openeuler/mygit.git
 * [new branch]      apache -> apache
Branch 'apache' set up to track remote branch 'apache' from 'origin'.
```

You can view the newly pushed branch in GitLab, as shown in the following figure.



Step 6 Activate the **hcie** user and push the code.

Log out from the **root** user and log in to the system again as the **hcie** user. The system prompts you to change the initial password. Perform operations as prompted. After that, the **hcie** user is activated.

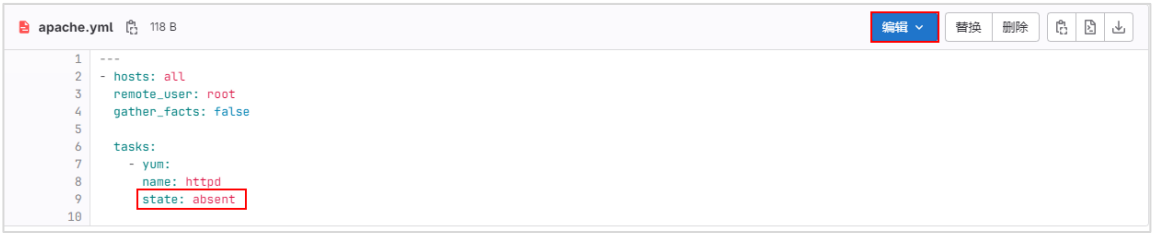
In the **MyGit** directory, go to the **apache** branch, change the **nginx.yml** file to **apache.yml**, and push the code again by using the **hcie** user and its password.

```
[root@controller MyGit]# mv nginx.yml apache.yml
[root@controller MyGit]# git add .
[root@controller MyGit]# git commit -m "second"
[root@controller MyGit]# git push --set-upstream origin apache
```

```
[root@controller MyGit]# git push --set-upstream origin apache
Username for 'http://gitlab.example.com': hcie
Password for 'http://hcie@gitlab.example.com':
Branch 'apache' set up to track remote branch 'apache' from 'origin'.
Everything up-to-date
```

Step 7 Pull code.

Manually modify the code in GitLab, as shown in the following figure.



After the modification, run the **git clone** command on the client to obtain the latest code.

```
[root@controller MyGit]# git clone http://gitlab.example.com/hcie-openeuler/mygit.git
Cloning into 'mygit'...
Username for 'http://gitlab.example.com': hcie
Password for 'http://hcie@gitlab.example.com':
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 12 (delta 2), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (12/12), done.
Resolving deltas: 100% (2/2), done.
```

- Question: Is the corresponding file in the directory is updated synchronously after the code is pulled?
- Answer: After the code is pulled, there is no corresponding file in the directory. The default branch is **master**, but the code in the **apache** branch is modified. Therefore, you need to switch to the corresponding branch to view the update details.

3 Facts and Magic Variables

3.1 Introduction

3.1.1 About This Exercise

This exercise describes how to extract facts and magic variables and how to print variable values through the debug module. Other modules are not used to call variables.

3.1.2 Objectives

This exercise illustrates the content of the first section of the *Advanced Automation Management* course and describes how to extract and use facts and magic variables for subsequent tasks.

3.2 Usage of Facts

Example: Read all IP addresses of the controlled host (server*a*) through facts.

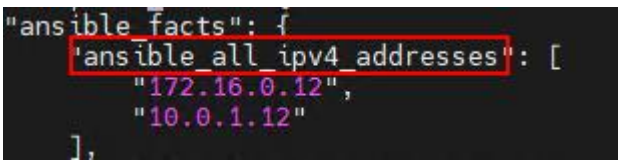
Step 1 Extract all facts.

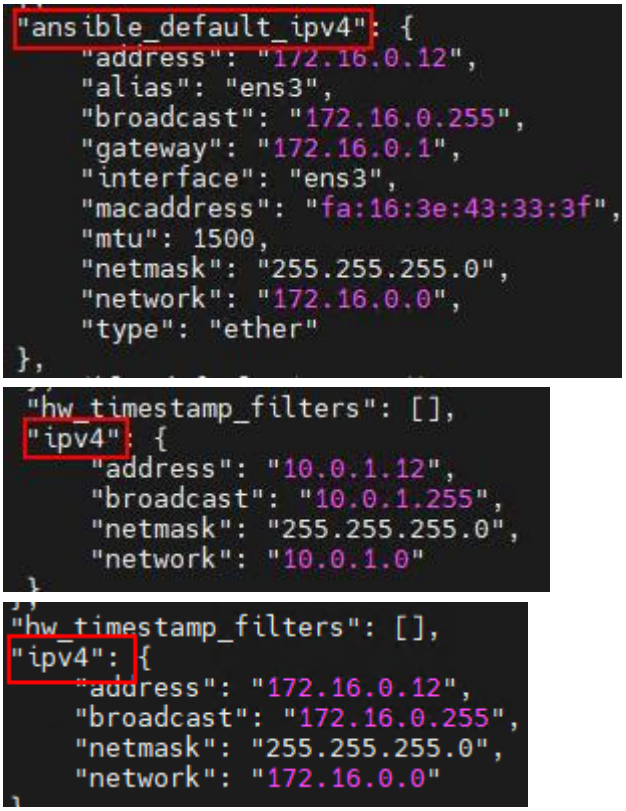
There are numerous facts. Before extracting them, run the following command to export all facts to a text file using the setup module for easier searching:

```
[root@controller yaml]# ansible servera -m setup > servera.txt
```

Step 2 Search for IP address information.

Skim the **servera.txt** file, and you can see IP addresses are displayed in **ansible_all_ipv4_addresses**, **ansible_default_ipv4**, and **ipv4**.





ansible_all_ipv4_addresses cannot map IP addresses to NICs, and **ansible_default_ipv4** cannot display all IP addresses. Therefore, only **ipv4** can be used. The value of **address** in **ipv4** is the IP address. The upper level of **ipv4** is **ansible_*xxxx***, where *xxxx* indicates the NIC name. The upper level of **ansible_*xxxx*** is **ansible_facts**. Therefore, the fact used in this example is **ansible_xxxx.ipv4.address**.

Step 3 Conduct a test.

Write a playbook as follows and invoke the fact determined in step 2 to test whether the requirements can be met.

```
- hosts: servera
  gather_facts: true

  tasks:
    - debug:
        msg: "ip address of enp4s4 is {{ ansible_enp4s4.ipv4.address}}"
    - debug:
        msg: "ip address of ens3 is {{ ansible_ens3.ipv4.address }}"
```

Execute the playbook to obtain the following information.

```
[root@controller yaml]# ansible-playbook facts.yml

PLAY [servera] *************************************************************

TASK [Gathering Facts] *****************************************************
ok: [servera]

TASK [debug] ***************************************************************
ok: [servera] => {
    "msg": "ip address of enp4s4 is 10.0.1.12"
}

TASK [debug] ***************************************************************
ok: [servera] => {
    "msg": "ip address of ens3 is 172.16.0.12"
}

PLAY RECAP *****************************************************************
servera                    : ok=3    changed=0    unreachable=0    f
```

- Question: Can **gather_facts** not be configured in the playbook in step 3?
- Answer: If **gather_facts** can collect facts by default in the configuration file, you do not need to set it; otherwise, you must set it to **true** or **yes**.
- Task: Select appropriate facts to obtain the capacity of a partition.
- Reference: Use **ansible_devices.vda.partitions.vda1.size** to read the capacity of partition **vda1**.

3.3 Customizing Facts

Example: Add a fact for server*a* and server*b* and set **usage** to **test**.

Step 1 Create required files on the controlled host.

To customize environment variables, create an **/etc/ansible/facts.d** directory on the controlled host and import the defined facts as the **x.fact** file. Create a **usage.fact** file on the controlled host as follows.

```
[usage]
name=test
```

Write a playbook to create the required directory and copy the created **usage.fact** file to the controlled hosts.

```
- hosts: all

  tasks:
    - name: mkdir dir
      file:
        state: directory
        path: /etc/ansible/facts.d
    - name: upload file
      copy:
```

```
        src: /root/yaml/usage.fact
        dest: /etc/ansible/facts.d/usage.fact
```

Step 2 Conduct a test.

Run the following command to check whether the customized facts take effect:

```
[root@controller yaml]# ansible all -m setup -a "filter=ansible_local"
```

```
[root@controller yaml]# ansible all -m setup -a "filter=ansible_local"
serverb | SUCCESS => {
    "ansible_facts": {
        "ansible_local": {
            "usage": {
                "usage": {
                    "name": "test"
                }
            }
        },
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false
}
servera | SUCCESS => {
    "ansible_facts": {
        "ansible_local": {
            "usage": {
                "usage": {
                    "name": "test"
                }
            }
        },
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false
}
```

- Question: Can **filter** of the setup module be used to accurately filter a user-defined fact?
- Answer: No. **filter** of the setup module can read only the lower layer of **ansible_facts**.

3.4 Usage of Magic Variables

This exercise shows functions and application scenarios of common magic variables.

3.4.1 Usage of hostvars

Example: Check other variables except facts of server*a*.

Write a playbook as follows:

```
- hosts: servera
  remote_user: root
```

```
  gather_facts: false

  tasks:
    - name: hostvars
      debug:
        msg: "{{ hostvars['servera'] }}"
```

The output is as follows.

```
[root@controller yaml]# ansible-playbook hostvar.yml

PLAY [servera] *****************************************************

TASK [hostvars] ****************************************************
ok: [servera] => {
    "msg": {
        "ansible_check_mode": false,
        "ansible_diff_mode": false,
        "ansible_facts": {},
        "ansible_forks": 5,
        "ansible_inventory_sources": [
            "/etc/ansible/hosts"
        ],
        "ansible_playbook_python": "/usr/bin/python3",
        "ansible_run_tags": [
            "all"
        ],
        "ansible_skip_tags": [],
        "ansible_verbosity": 0,
        "ansible_version": {
            "full": "2.9.27",
            "major": 2,
            "minor": 9,
            "revision": 27,
            "string": "2.9.27"
        },
        "group_names": [
            "client"
        ],
        "groups": {
            "all": [
                "serverb",
                "servera"
            ],
            "client": [
                "servera"
            ],
            "ungrouped": [
                "serverb"
            ]
        },
        "inventory_dir": "/etc/ansible",
        "inventory_file": "/etc/ansible/hosts",
        "inventory_hostname": "servera",
        "inventory_hostname_short": "servera",
        "playbook_dir": "/root/yaml"
    }
}

PLAY RECAP *********************************************************
servera                    : ok=1    changed=0    unreachable=0
```

- Question: Can the host in the playbook be specified as server*b*?
- Answer: Yes. Modify as required.
- Question: Why does server*a* need to be specified in **msg: "{{ hostvars['servera'] }}"**? What happens if it is not specified?
- Answer: The variables related to server*a* need to be obtained. Therefore, server*a* is specified in **hostvars**. If server*a* is not specified, variables related to all hosts in the host list are displayed.

3.4.2 Usage of inventory_hostname

Example: List the name of the controlled host where a task is being executed.

Write a playbook as follows:

```
- hosts: servera
  gather_facts: false

  tasks:
    - debug:
        msg: "{{ inventory_hostname }}"
```

The output is as follows.

```
[root@controller yaml]# ansible-playbook hostname.yml

PLAY [servera] *****************************************

TASK [debug] *******************************************
ok: [servera] => {
    "msg": "servera"
}

PLAY RECAP *********************************************
servera                    : ok=1    changed=0    unre
```

3.5 Usage of Lookup Plugins

3.5.1 Usage of the File Module

Example: Read the content in the **username** file and set it as a variable.

Write a playbook as follows:

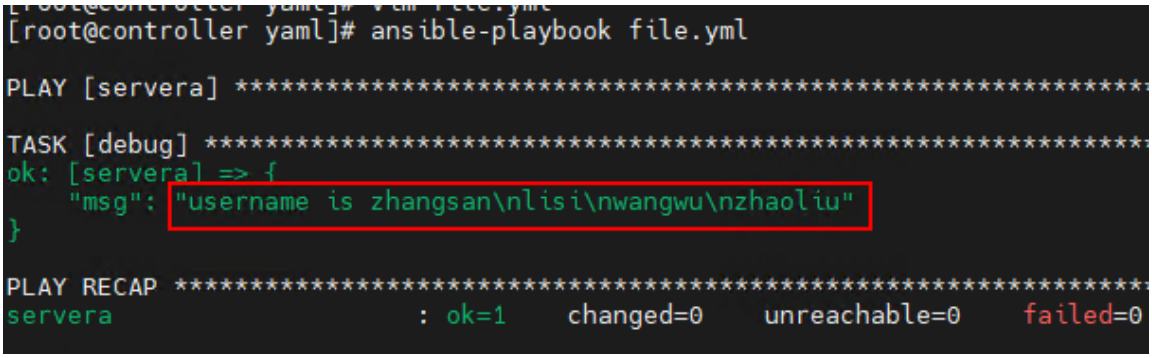
```
- hosts: servera
  gather_facts: false

  tasks:
    - debug:
        msg: "username is {{ lookup('file','/root/yaml/username') }}"
```

Write the **username** file as follows:

```
zhangsan
lisi
wangwu
zhaoliu
```

Execute the playbook. The output is as follows.



3.5.2 Usage of the Password Module

Example: Create a test user, automatically generate a password with eight characters, and save the password in the **/root/yaml/password** file.

Write a playbook as follows:

```
- hosts: servera
  gather_facts: false

  tasks:
    - user:
        name: test
        password: "{{ lookup('password','/root/yaml/password length=8')}}"
```

Execute the playbook and check the password of the **test** user.

```
[root@controller yaml]# cat password
hXQeKUm,
```

Use the password to log in to server*a* and check whether the creation is successful.

```
[root@clinet-1 ~]# cat /etc/passwd
......
test:x:1001:1001::/home/test:/bin/bash
[root@clinet-1 ~]# cat /etc/shadow
......
test:hXQeKUm,:19653:0:99999:7:::
```

3.5.3 Usage of the env Module

Example: Check the value of the environment variable **HOME** on the controlled host.

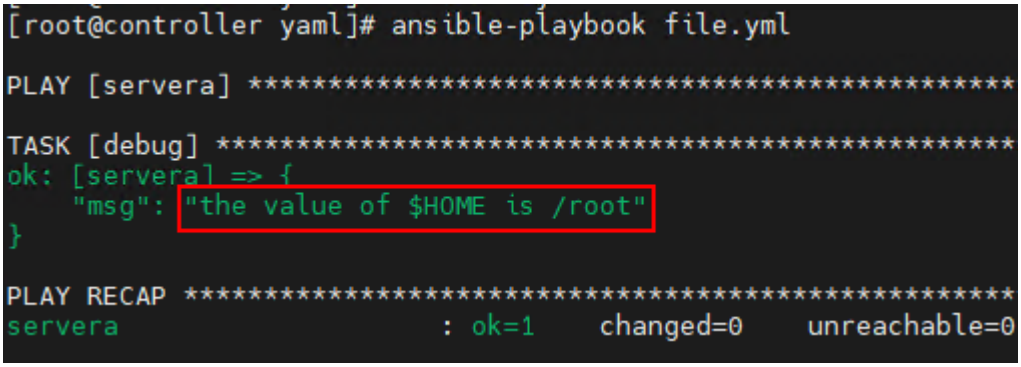
Write a playbook as follows:

```
- hosts: servera
```

```
  gather_facts: false

  tasks:
    - debug:
        msg: "the value of $HOME is {{ lookup('env','HOME') }}"
```

Execute the playbook. The output is as follows.



4 Conditional and Loop Statements of a Playbook

4.1 Introduction

4.1.1 About This Exercise

This exercise illustrates how to implement a task by comparing variable values. This exercise focuses on conditional and loop statements, and only debug or other modules with simple functions are used.

4.1.2 Objectives

This exercise uses multiple types of conditional statements to perform simple tasks and loop statements to implement specified tasks, to enhance the trainees' understanding this course.

4.2 Usage of Conditionals

4.2.1 Comparison Operations in a Playbook

Example: Print all the IP addresses of the controlled host client-1.

Write a playbook as follows:

```
- hosts: all
  gather_facts: true

  tasks:
    - debug:
        msg: ip address of Client-1 is "{{ ansible_all_ipv4_addresses }}"
      when: ansible_hostname == "client-1"
```

Execute the playbook. The following information is displayed.

```
[root@controller yaml]# ansible-playbook ip.yml

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [servera]
ok: [serverb]

TASK [debug] *******************************************************************
skipping: [serverb]
ok: [servera] => {
    "msg": "ip address of Client-1 is \"['172.16.0.12', '10.0.1.12']\""
}

PLAY RECAP *********************************************************************
servera                    : ok=2    changed=0    unreachable=0    failed=0
serverb                    : ok=1    changed=0    unreachable=0    failed=0
```

4.2.2 Logical Operation in a Playbook

Example: Print all the IP addresses of the controlled host client-1 with Ansible 2.9.27 used.

Write a playbook as follows:

```
- hosts: all
  gather_facts: true

  tasks:
    - debug:
        msg: ip address of Client-1 is "{{ ansible_all_ipv4_addresses }}"
      when: ansible_hostname == "client-1" and ansible_version.full == "2.9.27"
```

Execute the playbook. The following information is displayed.

```
[root@controller yaml]# ansible-playbook ip2.yml

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [serverb]
ok: [servera]

TASK [debug] *******************************************************************
skipping: [serverb]
ok: [servera] => {
    "msg": "ip address of Client-1 is \"['10.0.1.12', '172.16.0.12']\""
}

PLAY RECAP *********************************************************************
servera                    : ok=2    changed=0    unreachable=0    failed=0
serverb                    : ok=1    changed=0    unreachable=0    failed=0
```

4.2.3 Path Determination in a Playbook

Example: Copy the **/home/test** file of the controller to the controlled host.

Write a playbook as follows:

```
- hosts: all
  gather_facts: true

  tasks:
    - name: create a file
      copy:
        src: /home/test
        dest: /home/test
      when: "'/home/test' is exists"
```

Execute the playbook is executed. The following information is displayed.

```
[root@controller yaml]# ansible-playbook test.yml

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [serverb]
ok: [servera]

TASK [create a file] ***********************************************************
skipping: [serverb]
skipping: [servera]

PLAY RECAP *********************************************************************
servera                    : ok=1    changed=0    unreachable=0    failed=0    skipped=1
serverb                    : ok=1    changed=0    unreachable=0    failed=0    skipped=1
```

Create a **/home/test** directory on the controller.

```
[root@controller yaml]# touch /home/test
[root@controller yaml]# ls /home
openeuler   test
```

Run the playbook again, and the following information is displayed.

```
[root@controller yaml]# ansible-playbook test.yml

PLAY [all] *********************************************

TASK [Gathering Facts] *********************************
ok: [serverb]
ok: [servera]

TASK [create a file] ***********************************
changed: [servera]
changed: [serverb]

PLAY RECAP *********************************************
servera                    : ok=2    changed=1    unr
serverb                    : ok=2    changed=1    unr
```

4.2.4 Task Execution Result Determination in a Playbook

Example: Create a **/tmp/test** file on the controller. If the file is successfully created, the message "mission is success" is displayed.

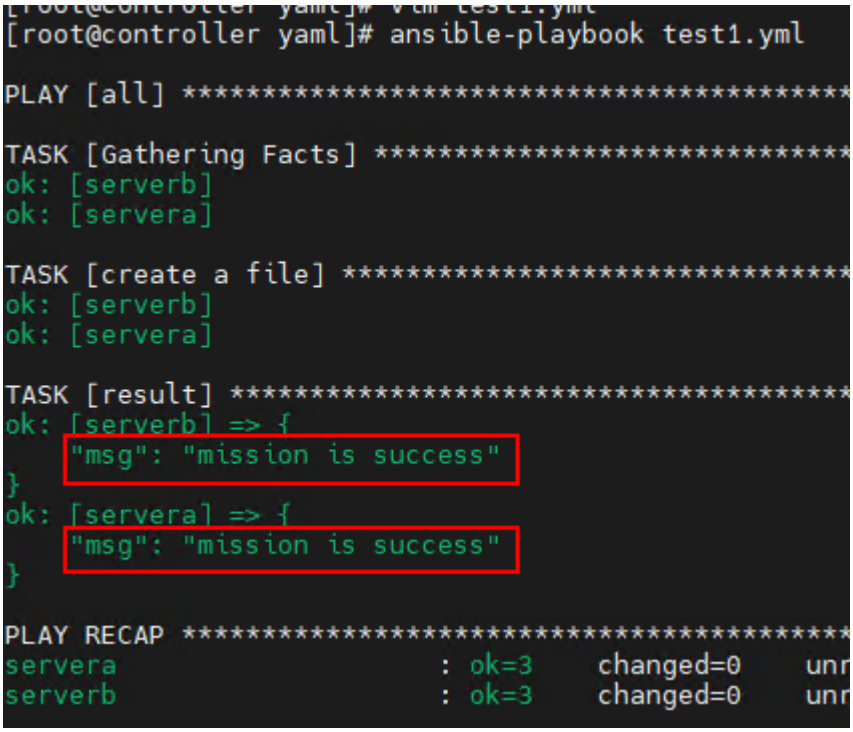
Write a playbook as follows:

```
- hosts: all
```

```
gather_facts: true

tasks:
  - name: create a file
    copy:
      src: /home/test/
      dest: /home/test/
    register: returnmsg
  - name: result
    debug:
      msg: "mission is success"
    when: "returnmsg is success"
```

Execute the playbook. The following information is displayed.



- Question: If the condition is changed to **when: "returnmsg is changed"**, can the message "mission is success" be returned?
- Answer: No. The file has been created in a previous section. Therefore, the **changed** condition cannot be met. The execution result is as follows.

```
[root@controller yaml]# ansible-playbook test1.yml

PLAY [all] *********************************************

TASK [Gathering Facts] *********************************
ok: [servera]
ok: [serverb]

TASK [create a file] ***********************************
ok: [servera]
ok: [serverb]

TASK [result] ******************************************
skipping: [serverb]
skipping: [servera]

PLAY RECAP *********************************************
servera                    : ok=2    changed=0
serverb                    : ok=2    changed=0
```

4.2.5 Usage of Block

Example 1: Install httpd on the controlled host client-1 and disable the firewalld service.

Write a playbook as follows:

```
- hosts: all
  gather_facts: true

  tasks:
    - block:
      - name: install httpd
        yum:
          name: httpd
          state: present
      - name: stop firewalld
        service:
          name: firewalld
          state: stopped
      when: ansible_hostname == "client-1"
```

Execute the playbook. The following information is displayed.

```
[root@controller yaml]# ansible-playbook httpd.yml

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [serverb]
ok: [servera]

TASK [install httpd] ***********************************************************
skipping: [serverb]
ok: [servera]

TASK [stop firewalld] **********************************************************
skipping: [serverb]
changed: [servera]

PLAY RECAP *********************************************************************
servera                    : ok=3    changed=1    unreachable=0    failed=0
serverb                    : ok=1    changed=0    unreachable=0    failed=0
```

Example 2: Install httpd-test on the controlled host client-1 and disable the firewalld service. If the installation fails, the message "installation is faild" is displayed.

Write a playbook as follows:

```
- hosts: all
  gather_facts: true

  tasks:
    - block:
      - name: install httpd
        yum:
          name: httpd-test
          state: present
      rescue:
        - debug:
            msg: "installation is faild"
      always:
        - name: stop firewalld
          service:
            name: firewalld
            state: stopped
      when: ansible_hostname == "client-1"
```

Execute the playbook. The following information is displayed.

```
[root@controller yaml]# ansible-playbook httpd.yml

PLAY [all] *********************************************************

TASK [Gathering Facts] *********************************************
ok: [serverb]
ok: [servera]

TASK [install httpd] ***********************************************
skipping: [serverb]
fatal: [servera]: FAILED! => {"changed": false, "failures": ["No

TASK [debug] *******************************************************
ok: [servera] => {
    "msg": "installation is faild"
}

TASK [stop firewalld] **********************************************
skipping: [serverb]
ok: [servera]

PLAY RECAP *********************************************************
servera                    : ok=3    changed=0    unreachable=0
serverb                    : ok=1    changed=0    unreachable=0
```

4.3 Usage of Loop Statements

Example: Create a user information file as the administrator as follows:

```
user:
  - name: zhangsan
    department: sale
    uid: 1100
  - name: lisi
    department: sale
    uid: 1101
  - name: wangwu
    department: technology
    uid: 1102
```

Create users for all hosts based on this file. **name** is the user name, **department** is the user information, and **uid** is the user ID. A user password consists of eight randomly generated digits, encrypted using **sha512** and saved in the **/root/yaml** directory. A password file is named "password-<name>", where *<name>* indicates the user name.

Write a playbook as follows:

```
- hosts: all
  gather_facts: false
  vars_files:
    /root/yaml/username

  tasks:
```

```
    - name: create user
      user:
        name: "{{ item.name }}"
        comment: "{{ item.department }}"
        uid: "{{ item.uid }}"
        password: "{{ lookup('password','/root/yaml/password-{{ item.name }} chars=digits
length=8') | password_hash('sha512') }}"
      loop: "{{ user }}"
```

Execute the playbook. The following information is displayed.

```
[root@controller yaml]# ansible-playbook user.yml

PLAY [all] *********************************************************************************************************

TASK [create user] *************************************************************************************************
changed: [serverb] => (item={'name': 'zhangsan', 'department': 'sale', 'uid': 1100})
changed: [servera] => (item={'name': 'zhangsan', 'department': 'sale', 'uid': 1100})
changed: [serverb] => (item={'name': 'lisi', 'department': 'sale', 'uid': 1101})
changed: [servera] => (item={'name': 'lisi', 'department': 'sale', 'uid': 1101})
changed: [serverb] => (item={'name': 'wangwu', 'department': 'technology', 'uid': 1102})
changed: [servera] => (item={'name': 'wangwu', 'department': 'technology', 'uid': 1102})

PLAY RECAP *********************************************************************************************************
servera                    : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
serverb                    : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

- Task: After the execution is complete, check whether corresponding password files are generated and whether users are created.

5 Usage of Ansible Roles

5.1 Introduction

5.1.1 About This Exercise

This exercise demonstrates how to create roles. You need to first install the httpd service on server*a* and server*b*, and set the service to listen to port 81 of the first NIC (default NIC) of each host. When the service accesses port 81 of the host, the message "this is xxx" is displayed, where *xxx* indicates the host name.

5.1.2 Objectives

This exercise illustrates the content of the section related to roles in the *Advanced Automation Management* course and describes how to create a role.

5.2 Usage of Roles

Step 1 Plan the design.

This exercise involves the configuration of the httpd service. For details, see the following:

- The port number needs to be set to 81 instead of the default value. To facilitate the change, the port number is implemented as a variable. If the port number needs to be changed, modify the variable file.
- The service needs to listen on the first NIC of each host. Required IP addresses can be obtained through facts. The corresponding configuration file can be implemented through the jinja2 template.
- The home page file of the service can also be obtained through facts and can be implemented through the jinja2 template.
- After the configuration file is modified, the service needs to be restarted for the modification to take effect. The handler mode can be used to trigger the service restart.
- Other procedure includes installing httpd, copying the configuration file and home page file to the specified directory, starting httpd and setting it to start upon system startup, and restarting httpd.

Step 2 Create a directory required for creating a role.

In this exercise, all role files are stored in the **/root/roles** directory. Therefore, create this directory first, use the role required for initialization in the directory, and name the role httpd.

```
[root@controller roles]# ansible-galaxy role init httpd
- Role httpd was created successfully
```

Check the directory structure of the role.

```
[root@controller roles]# tree httpd
httpd
├── defaults
│   └── main.yml
├── files
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── README.md
├── tasks
│   └── main.yml
├── templates
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml
```

In this exercise, only the **handlers**, **README.md**, **tasks**, **templates**, and **vars** directories are required. You can delete other files or directories.

Step 3 Develop an environment variable file.

Go to the **vars** directory and modify the **mail.yml** file as follows:

```
---
# vars file for httpd
PORT: 81      ##### Listening port of the httpd service
```

Step 4 Develop the Jinja2 template.

Go to the **templates** directory and create the configuration file and home page file templates.

```
[root@controller vars]# cd ../templates/
[root@controller templates]# touch roles.conf.j2
[root@controller templates]# touch index.html.j2
[root@controller templates]# ls
index.html.j2   roles.conf.j2
```

Edit the configuration file template file as follows:

```
Listen {{ ansible_default_ipv4.addresss }}:{{PORT}}
```

Edit the home page file template as follows:

```
this is {{ ansible_hostname }}
```

Step 5 Develop the **handler** file.

Go to the **handlers** directory and edit the **main.yml** file as follows:

```
---
# handlers file for httpd
- name: restart httpd
  service:
    name: httpd
    state: restarted
```

Step 6 Develop the **tasks** file.

Open the **tasks** file to write a playbook. To facilitate subsequent maintenance, each task corresponds to an independent playbook. The playbook file for installing the httpd service is **httpd-install.yml**. The content is as follows:

```
- name: install httpd
  yum:
    name: httpd
    state: present
```

Configure and start the httpd service, and set it to automatically start upon system startup. The playbook file for configuring the httpd service is **httpd-config.yml**. The content is as follows:

```
- name: config httpd
  template:
    src: roles.conf.j2
    dest: /etc/httpd/conf.d/roles.conf
  notify: restart httpd
- name: config main page
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
- name: enable httpd
  service:
    name: httpd
    enabled: yes
    state: started
```

Configure the playbook file **firewalld.yml** for configuring the firewall as follows:

```
- name: config firewalld
  firewalld:
    port: "{{ PORT }}/tcp"
    permanent: yes
```

```
    immediate: yes
    state: enabled
```

Modify the **main.yml** file in the **tasks** directory as follows:

```
---
# tasks file for httpd
- include: httpd-install.yml
- include: firewalld.yml
- include: httpd-config.yml
```

Step 7 Write the **role** file.

Write the playbook file for role execution in the **/root/roles** directory and set the file name to **httpd_role.yml**. The content is as follows:

```
- hosts: all
  gather_facts: true

  roles:
    - role: httpd
```

Execute **httpd_role.yml**. The following information is displayed.

```
[root@controller roles]# ansible-playbook httpd_role.yml

PLAY [all] *********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [serverb]
ok: [servera]

TASK [install httpd] ***********************************************************************************************
ok: [serverb]
ok: [servera]

TASK [httpd : config firewalld] ************************************************************************************
ok: [servera]
ok: [serverb]

TASK [config httpd] ************************************************************************************************
changed: [serverb]
changed: [servera]

TASK [httpd : config main page] ************************************************************************************
ok: [serverb]
ok: [servera]

TASK [enable httpd] ************************************************************************************************
changed: [serverb]
changed: [servera]

RUNNING HANDLER [restart httpd] ************************************************************************************
changed: [serverb]
changed: [servera]

PLAY RECAP *********************************************************************************************************
servera                    : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
serverb                    : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

After the test is complete, the following information is displayed.

```
[root@controller roles]# curl 172.16.0.12:81
this is client-1
[root@controller roles]# curl 172.16.0.13:81
this is clinet-2
```

6 Ansible Privilege Escalation and Jinja2 Template Usage

6.1 Introduction

6.1.1 About This Exercise

This exercise is based on the section "Usage of Roles". Change the execution user in the preceding exercise to the newly created user **zhangsan** and use the jinja2 template to customize the web home page.

6.1.2 Objectives

This exercise corresponds to the last two sections in the *Advanced Automation Management course*. It describes how to escalate privileges for non-root accounts in roles to complete tasks and how to implement specified requirements through advanced configurations of the Jinjia2 template.

6.2 Ansible Privilege Escalation

Step 1 Set **sudo** permissions for a user.

sudo permissions are not set for the user **Zhangsan** when the user is created. Add the required content to the playbook set in the section "Usage of Loop Statements".

```
- hosts: all
  gather_facts: false
  vars_files:
    /root/yaml/username

  tasks:
    - name: create user
      user:
        name: "{{ item.name }}"
        comment: "{{ item.department }}"
        uid: "{{ item.uid }}"
        password: "{{ lookup('password','/root/yaml/password-{{ item.name }} chars=digits
length=8') | password_hash('sha512') }}"
      loop: "{{ user }}"
    - name: add sudo
      lineinfile:
```

```
        path: /etc/sudoers
        line: "{{ item.name }} ALL=(ALL)    NOPASSWD:ALL"
      loop: "{{ user }}"
```

After the modification, execute the playbook again to enable the user **zhangsan** to have the privilege escalation capability.

Step 2 Configure password-free login.

Set the password-free login permission for the user **zhangsan** on the controller.

Step 3 Modify the **role** file.

Modify the playbook in step 7 in the section "Usage of Roles" as follows:

```
- hosts: all
  remote_user: zhangsan
  gather_facts: true

  roles:
    - role: httpd
      become: true
      become_method: sudo
      become_user: root
      become_ask_pass: false
```

After the modification, run the playbook to check whether it can run properly.

6.3 Usage of the Jinja2 Template

Example: Based on the chapter "Usage of Ansible Roles", change the home page. If the host name is client-1, "this is test1" is displayed. If the host name is client-2, "this is test2" is displayed.

Modify the home page file **index.html.j2** as follows:

```
{% if ansible_hostname == "client-1" %}
this is test1
{% elif ansible_hostname == "client-2" %}
this is test2
{% endif %}
```

After the modification, execute the **httpd_role.yaml** file again for the configuration to take effect.

After the execution, perform a test to check whether the requirements can be met.

```
[root@controller roles]# curl 172.16.0.12:81
this is test1

[root@controller roles]# curl 172.16.0.13:81
this is test2
```

7 AWX Installation, Deployment, Configuration, and Usage

7.1 Introduction

7.1.1 About This Exercise

In this exercise, AWX is installed and deployed based on AWX Operator, and Kubernetes and Ceph are used to ensure high availability of AWX.

7.1.2 Objectives

This exercise illustrates the CI/CD process in the *Advanced Automation Management* course and describes how to execute AWX in Kubernetes.

7.2 AWX Installation and Deployment

7.2.1 Preparing the Ceph Resource Pool

Step 1 Create a resource pool.

The AWX database stores the file system CephFS provided by Ceph, and the capacity is not less than 8 GB. Therefore, before installing and deploying AWX, create the required CephFS resource pool.

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_for_awx
[ceph: root@ceph01 /]# ceph osd pool create data_for_awx
[ceph: root@ceph01 /]# ceph fs new pool_for_awx metadata_for_awx   data_for_awx
```

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_for_awx
pool 'metadata_for_awx' created
[ceph: root@ceph01 /]# ceph osd pool create data_for_awx
pool 'data_for_awx' created
[ceph: root@ceph01 /]# ceph fs new pool_for_awx metadata_for_awx  data_for_awx
new fs with metadata pool 20 and data pool 21
[ceph: root@ceph01 /]#
```

Step 2 Create a user.

After the resource pool is created, create a user for AWX. The user should have the read and write permissions on the resource pool created in the preceding step.

```
[ceph: root@ceph01 /]# ceph fs authorize pool_for_awx client.awx / rwps -o ceph.client.awx.keyring
```

```
[ceph: root@ceph01 /]# ceph fs authorize pool_for_awx client.awx / rwps -o ceph.client.awx.keyring
[ceph: root@ceph01 /]# ceph auth ls | grep -A 4 awx
installed auth entries:

client.awx
        key: AQAHjkllPax2CBAACfRE0qQUqmZzjQtOxzQHIg==
        caps: [mds] allow rwps fsname=pool_for_awx
        caps: [mon] allow r fsname=pool_for_awx
        caps: [osd] allow rw tag cephfs data=pool_for_awx
```

7.2.2 Configuring Ceph-csi Resources

Step 1 Compile and deploy YAML files of related components.

Compile the **config-map** file based on the preceding sections. The content of the **csi-config-map.yaml** file is as follows:

```
#
# /!\ DO NOT MODIFY THIS FILE
#
# This file has been automatically generated by Ceph-CSI yamlgen.
# The source for the contents can be found in the api/deploy directory, make
# your modifications there.
#
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: "ceph-csi-config"
data:
  config.json: |-
    [
      {
        "clusterID": "d5faf9ba-56c0-11ee-9ffe-fa163edb3348",
        "monitors": [
          "192.168.0.21:6789",
          "192.168.0.22:6789",
          "192.168.0.23:6789"
        ]
      }
    ]
```

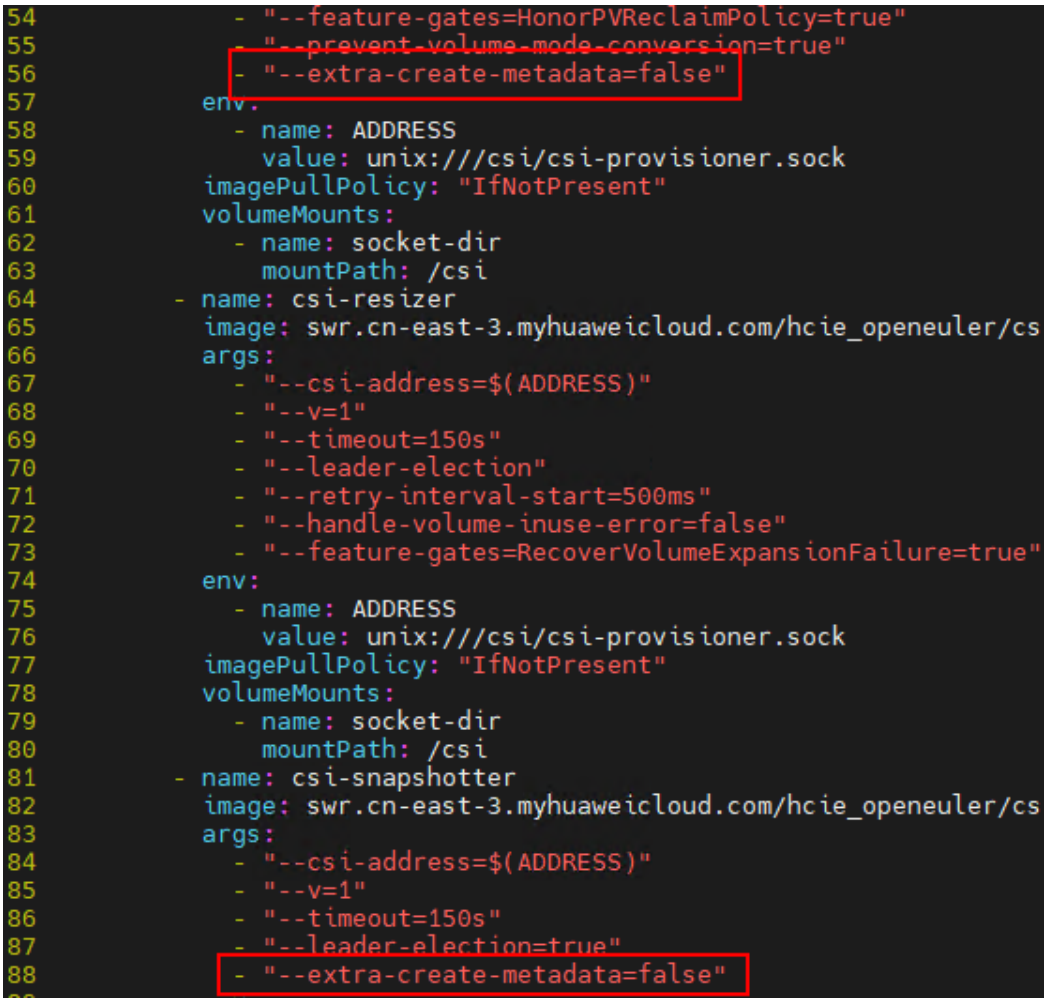
The content of **ceph-conf-map.yaml** is as follows:

```
---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    {}
metadata:
  name: ceph-csi-encryption-kms-config
---
apiVersion: v1
kind: ConfigMap
```

```
data:
  ceph.conf: |
    [global]
    auth_cluster_required = cephx
    auth_service_required = cephx
    auth_client_required = cephx

  # keyring is a required key and its value should be empty
  keyring: |
metadata:
  name: ceph-config
```

Change the 56th and 88th lines in the **csi-cephfsplugin-provisioner.yaml** file to **--extra-create-metadata=false**.



After the modification, run the following command to deploy all ConfigMaps:

```
kubectl apply -f ceph-conf-map.yaml -f csi-config-map.yaml -f csi-nodeplugin-rbac.yaml -f csi-provisioner-rbac.yaml
```

```
[root@k8smaster1 kubernetes]# kubectl apply -f ceph-conf-map.yaml -f csi-config-map.yaml -f csi-nodeplugin-rbac.yaml -f csi-provisioner-rbac.yaml
configmap/ceph-csi-encryption-kms-config created
configmap/ceph-config created
configmap/ceph-csi-config created
serviceaccount/cephfs-csi-nodeplugin created
clusterrole.rbac.authorization.k8s.io/cephfs-csi-nodeplugin created
clusterrolebinding.rbac.authorization.k8s.io/cephfs-csi-nodeplugin created
serviceaccount/cephfs-csi-provisioner created
clusterrole.rbac.authorization.k8s.io/cephfs-external-provisioner-runner created
clusterrolebinding.rbac.authorization.k8s.io/cephfs-csi-provisioner-role created
role.rbac.authorization.k8s.io/cephfs-external-provisioner-cfg created
rolebinding.rbac.authorization.k8s.io/cephfs-csi-provisioner-role-cfg created
```

Run the following command to deploy the pod required by ceph-csi:

```
kubectl apply -f csi-cephfsplugin-provisioner.yaml -f csi-cephfsplugin.yaml -f csidriver.yaml --validate=false
```

```
[root@k8smaster1 kubernetes]# kubectl apply -f csi-cephfsplugin-
service/csi-cephfsplugin-provisioner created
deployment.apps/csi-cephfsplugin-provisioner created
daemonset.apps/csi-cephfsplugin created
service/csi-metrics-cephfsplugin created
csidriver.storage.k8s.io/cephfs.csi.ceph.com created
```

Step 2 Compile the **secret** and **sc** configuration files.

Copy the **secret.yaml** file from the **example** directory and add the user information created in step 2 in section 5.2.1 to the file.

```
---
apiVersion: v1
kind: Secret
metadata:
  name: csi-cephfs-secret
  namespace: awx
stringData:
  # Required for statically provisioned volumes
  userID: awx
  userKey: AQAHjkllPax2CBAACfRE0qQUqmZzjQtOxzQHIg==

  # Required for dynamically provisioned volumes
  adminID: admin
  adminKey: AQA9UgllOALJIRAAvKYqPlEX8zZcLWgaY6F1OA==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

The value of **namespace** in the file must be the same as the namespace used by AWX Operator. Run the following command on the Ceph management node to query the user information:

```
[ceph: root@ceph01 /]# ceph auth get client.awx
```

```
[ceph: root@ceph01 /]# ceph auth get client.awx
[client.awx]
        key = AQAHjkllPax2CBAACfRE0qQUqmZzjQtOxzQHIg==
        caps mds = "allow rwps fsname=pool_for_awx"
        caps mon = "allow r fsname=pool_for_awx"
        caps osd = "allow rw tag cephfs data=pool_for_awx"
exported keyring for client.awx
```

Use the same method to find the values of **adminID** and **adminKey**.

After the modification, **secret** is not deployed because the namespace **awx** has not been created.

Copy the **storageclass.yaml** file from the **example** directory and add the resource pool information created in step 1 in section 7.2.1 to the file.

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: csi-cephfs-sc
provisioner: cephfs.csi.ceph.com
parameters:
  clusterID: d5faf9ba-56c0-11ee-9ffe-fa163edb3348
  fsName: pool_for_awx
  pool: data_for_awx
  csi.storage.k8s.io/provisioner-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/provisioner-secret-namespace: awx
  csi.storage.k8s.io/controller-expand-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/controller-expand-secret-namespace: awx
  csi.storage.k8s.io/node-stage-secret-name: csi-cephfs-secret
  csi.storage.k8s.io/node-stage-secret-namespace: awx
  mounter: kernel
reclaimPolicy: Delete
allowVolumeExpansion: true
```

Run the **ceph mon dump** command on the Ceph controller node to query the cluster ID.

```
[ceph: root@ceph01 /]# ceph mon dump
epoch 5
fsid d5faf9ba-56c0-11ee-9ffe-fa163edb3348
last_changed 2023-09-19T08:24:15.917640+0000
created 2023-09-19T07:48:14.114516+0000
min_mon_release 16 (pacific)
election_strategy: 1
```

fsName and **pool** correspond to **name** and **data pool** in the output of the **ceph fs ls** command on the Ceph controller node, respectively.

```
[ceph: root@ceph01 /]# ceph fs ls
name: fs01, metadata pool: metadata_cephfs, data pools: [data_cephfs ]
name: pool_for_awx, metadata pool: metadata_for_awx, data pools: [data_for_awx ]
name: k8s_cephfs, metadata pool: k8s_cephfs_metadata, data pools: [k8s_cephfs_data ]
```

Skip the deployment after the configuration is complete.

7.2.3 AWX Installation and Deployment

Step 1 Compile and deploy a **kustomization** file.

Create a **kustomization.yaml** file as follows:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  # Find the latest tag here: https://github.com/ansible/awx-operator/releases
```

```
  - https://gitee.com/yftyxa/awx-operator/config/default?ref=2.7.1
# et the image tags to match the git version from above
images:
  - name: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/awx-operator
    newTag: latest

# Specify a custom namespace in which to install AWX
namespace: awx
```

The namespace specified in the file must be the same as that in **secret** and **storageClass** in section 7.2.2. After the file is compiled, run the following command in the directory where the file is located:

```
[root@k8smaster1 awx1.0]# kubectl apply -k .
```

```
[root@k8smaster1 awx1.0]# kubectl apply -k .
namespace/awx created
customresourcedefinition.apiextensions.k8s.io/awxbackups.awx.ansible.com created
customresourcedefinition.apiextensions.k8s.io/awxrestores.awx.ansible.com created
customresourcedefinition.apiextensions.k8s.io/awxs.awx.ansible.com created
serviceaccount/awx-operator-controller-manager created
role.rbac.authorization.k8s.io/awx-operator-awx-manager-role created
role.rbac.authorization.k8s.io/awx-operator-leader-election-role created
clusterrole.rbac.authorization.k8s.io/awx-operator-metrics-reader created
clusterrole.rbac.authorization.k8s.io/awx-operator-proxy-role created
rolebinding.rbac.authorization.k8s.io/awx-operator-awx-manager-rolebinding created
rolebinding.rbac.authorization.k8s.io/awx-operator-leader-election-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/awx-operator-proxy-rolebinding created
configmap/awx-operator-awx-manager-config created
service/awx-operator-controller-manager-metrics-service created
deployment.apps/awx-operator-controller-manager created
```

Run the following command to check the deployment result:

```
[root@k8smaster1 awx1.0]# kubectl get pods -n awx
```

```
[root@k8smaster1 awx1.0]# kubectl get pods -n awx
NAME                                          READY   STATUS    RESTARTS   AGE
awx-operator-controller-manager-85d95854c9-8548t   2/2     Running   0          4m49s
```

After the **awx-operator-controller-manager** status becomes normal, go to the next step.

Step 2 Deploy other components.

Run the following command to deploy the **secret** and **storageClass** configured in section 7.2.2:

```
[root@k8smaster1 kubernetes]# kubectl apply -f secret.yaml -f storageclass.yaml
```

After the deployment is complete, check their status.

```
[root@k8smaster1 kubernetes]# kubectl get secret -n awx
NAME                                  TYPE
awx-operator-controller-manager-token-dq2j6   kubernetes.io/serv
csi-cephfs-secret                     Opaque
default-token-rttpm                   kubernetes.io/serv
[root@k8smaster1 kubernetes]# kubectl get sc -n awx
NAME                  PROVISIONER            RECLAIMPOLICY   VO
csi-cephfs-sc         cephfs.csi.ceph.com    Delete          Im
csi-cephfs-sc-default cephfs.csi.ceph.com    Delete          Im
harbor                storage.pri/nfs        Delete          Im
[root@k8smaster1 kubernetes]#
```

Compile the YAML file corresponding to the AWX server in the directory where the **kustomization.yaml** file is located and name the file **awx-demo.yaml**. The content is as follows:

```
---
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-openeuler
spec:
  service_type: nodeport
  # nodeport_port: 30080
```

After the compilation is complete, add it to the **kustomization** resource.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  # Find the latest tag here: https://github.com
  - https://gitee.com/yftyxa/awx-operator/config
  - awx-demo.yaml
# et the image tags to match the git version fro
images:
  - name: swr.cn-east-3.myhuaweicloud.com/hcie_o
    newTag: latest

# Specify a custom namespace in which to install
namespace: awx
```

After that, update **kustomization**.

```
[root@k8smaster1 awx1.0]# kubectl apply -k .
namespace/awx unchanged
customresourcedefinition.apiextensions.k8s.io/awxbackups.awx.ansible.com unchanged
customresourcedefinition.apiextensions.k8s.io/awxrestores.awx.ansible.com unchanged
customresourcedefinition.apiextensions.k8s.io/awxs.awx.ansible.com unchanged
serviceaccount/awx-operator-controller-manager unchanged
role.rbac.authorization.k8s.io/awx-operator-awx-manager-role configured
role.rbac.authorization.k8s.io/awx-operator-leader-election-role unchanged
clusterrole.rbac.authorization.k8s.io/awx-operator-metrics-reader unchanged
clusterrole.rbac.authorization.k8s.io/awx-operator-proxy-role unchanged
rolebinding.rbac.authorization.k8s.io/awx-operator-awx-manager-rolebinding unchanged
rolebinding.rbac.authorization.k8s.io/awx-operator-leader-election-rolebinding unchanged
clusterrolebinding.rbac.authorization.k8s.io/awx-operator-proxy-rolebinding unchanged
configmap/awx-operator-awx-manager-config unchanged
service/awx-operator-controller-manager-metrics-service unchanged
deployment.apps/awx-operator-controller-manager configured
awx.awx.ansible.com/awx-openeuler created
[root@k8smaster1 awx1.0]#
```

Wait for about two minutes until a PVC is created, which is in the **Pending** state. Edit the PVC and set its **storageClass** to **csi-cephfs-sc** created in section 7.2.2.

```
[root@k8smaster1 awx1.0]# kubectl get pvc -n awx
```




```
[root@k8smaster1 awx1.0]# kubectl get pvc postgres-13-awx-openeuler-postgres-13-0 -n awx -o=yaml > pvc.yml
```

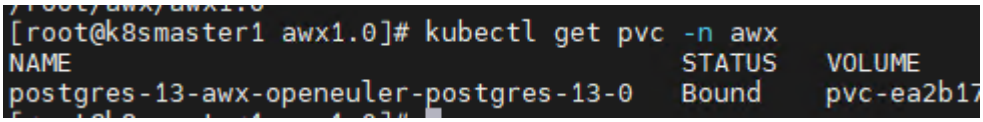
Specify the **sc** name in the **pvc.yml file** as follows:

```
[root@k8smaster1 awx1.0]# cat pvc.yml
......
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  volumeMode: Filesystem
  storageClassName: csi-cephfs-sc
.......
```

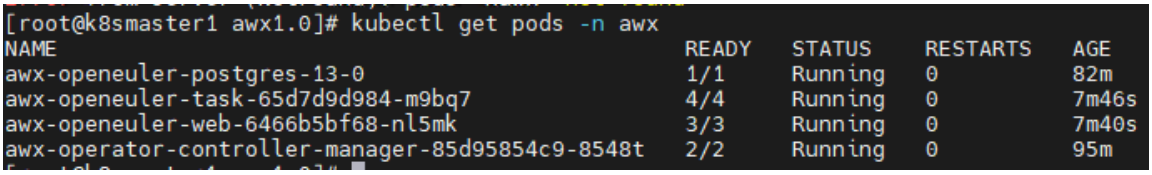
Delete the original PVC and use **pvc.yml** to deploy a PVC again.

```
[root@k8smaster1 awx1.0]# kubectl delete pvc postgres-13-awx-openeuler-postgres-13-0 --force -n awx
warning: Immediate deletion does not wait for confirmation that the running resource has been terminated. The resource may continue to run on the cluster indefinitely.
persistentvolumeclaim "postgres-13-awx-openeuler-postgres-13-0" force deleted
[root@k8smaster1 awx1.0]# kubectl apply -f pvc.yml
persistentvolumeclaim/postgres-13-awx-openeuler-postgres-13-0 created
```

Wait until the PVC status changes to **Bound**.



Run the **kubectl get pods -n awx** command to check whether the pod status is normal. If it is normal, the information similar to the following figure is displayed.



Log in to the pod and check whether the AWX service is normal.

```
[root@k8smaster1 awx1.0]# kubectl exec -it awx-openeuler-task-65d7d9d984-m9bq7 -c awx-openeuler-task -n awx -- /bin/bash
bash-5.1$ awx-manage version
```

```
[root@k8smaster1 awx1.0]# kubectl exec -it awx-openeuler-task-65d7d9d984-m9bq7 -c awx-openeuler-task -n awx -- /bin/bash
bash-5.1$ awx-manage version
23.3.1
bash-5.1$
```

If the returned information is correct, go to the next step.

Step 3 (Optional) Deploy Ingress.

If Ingress has been deployed, skip this step.

Copy the content from **https://github.com/kubernetes/ingress-nginx/blob/controller-v1.6.4/deploy/static/provider/baremetal/deploy.yaml** to the **ingress.yaml** file.

Replace the involved image with **swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/controller:v1.6.4 and swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-webhook-certgen:v20220916-gd32f8c343**.

Run the **kubectl apply -f ingress.yaml** command and wait until the deployment is complete.

```
[root@k8smaster1 awx1.0]# kubectl get pod -n ingress-nginx
NAME                                        READY   STATUS      RESTARTS   AGE
ingress-nginx-admission-create-bkjsl        0/1     Completed   0          34s
ingress-nginx-admission-patch-fp59q         0/1     Completed   0          34s
ingress-nginx-controller-6cddb7c4d4-gp2qv   1/1     Running     0          34s
```

The content of the Ingress configuration file required for compiling AWX is as follows:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: awx
  namespace: awx
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
 rules:
   - host: www.openeuler-hcie.com
     http:
       paths:
         - backend:
             service:
               name: awx-openeuler-service
               port:
                 number: 80
           path: /awx(/|$)(.*)
           pathType: Prefix
```

Query the service name and number in the file.

```
[root@k8smaster1 awx1.0]# kubectl get svc -n awx
NAME                                               TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
awx-openeuler-postgres-13                          ClusterIP   None           <none>        5432/TCP       4h39m
awx-openeuler-service                              NodePort    20.0.200.11    <none>        80:30367/TCP   3h24m
awx-operator-controller-manager-metrics-service    ClusterIP   20.0.181.71    <none>        8443/TCP       4h52m
[root@k8smaster1 awx1.0]#
```

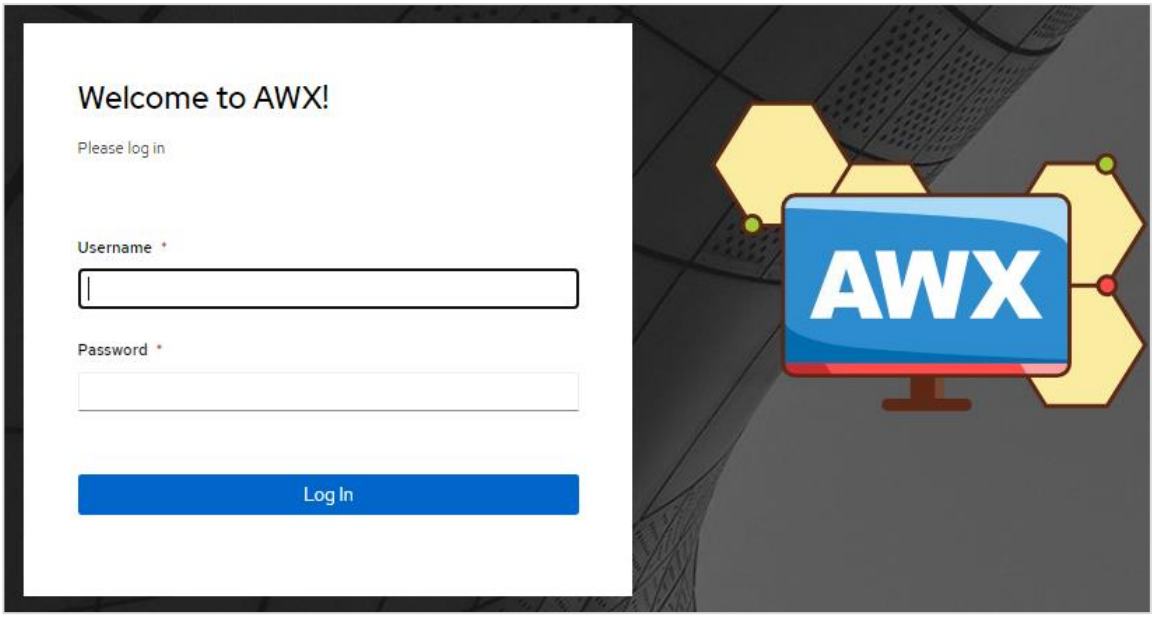
After the configuration is complete, check the **svc** corresponding to ingress-controller.

```
[root@k8smaster1 awx1.0]# kubectl get svc -n ingress-nginx
NAME                                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)                      AGE
ingress-nginx-controller             NodePort    20.0.60.240    <none>        80:32453/TCP,443:32264/TCP   136m
ingress-nginx-controller-admission   ClusterIP   20.0.7.193     <none>        443/TCP                      136m
[root@k8smaster1 awx1.0]#
```

View details about the created Ingress.

```
[root@k8smaster1 awx1.0]# kubectl describe  ingress awx -n awx
Name:             awx
Labels:           <none>
Namespace:        awx
Address:          10.0.0.12
Default backend:  default-http-backend:80 (<error: endpoints "default-http-backend" not found>)
Rules:
  Host                  Path  Backends
  ----                  ----  --------
  www.openeuler-hcie.com
                        /awx(/|$)(.*)   awx-openeuler-service:80 (30.0.93.88:8052)
Annotations:            kubernetes.io/ingress.class: nginx
                        nginx.ingress.kubernetes.io/rewrite-target: /$2
                        nginx.ingress.kubernetes.io/use-regex: true
Events:
  Type    Reason  Age                  From                      Message
  ----    ------  ----                 ----                      -------
  Normal  Sync    2m38s (x25 over 87m)  nginx-ingress-controller  Scheduled for sync
[root@k8smaster1 awx1.0]#
```

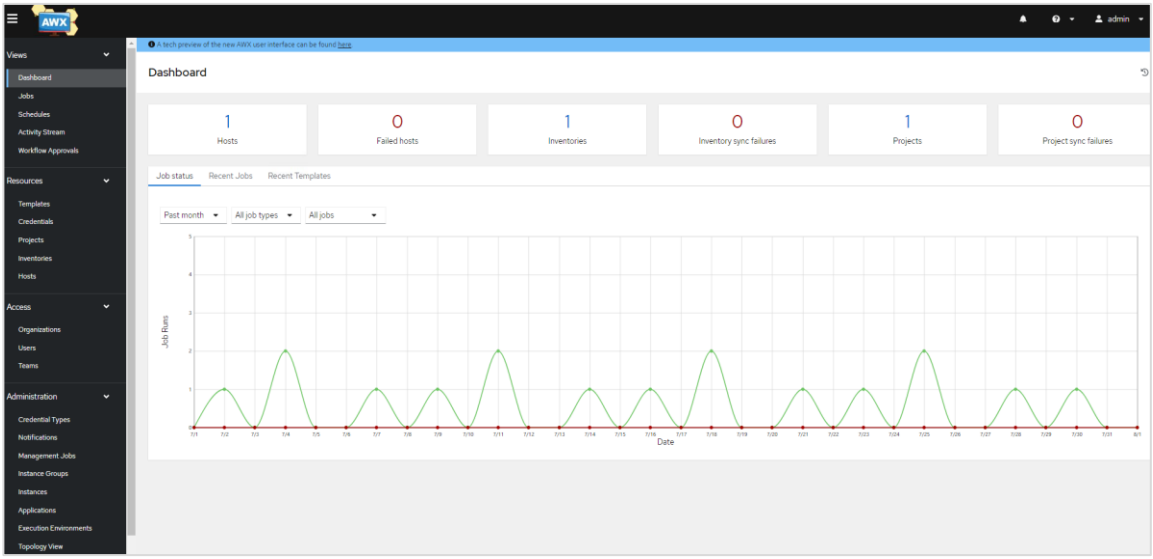
Manually bind the domain name www.openeuler-hcie.com to **10.0.0.12** or the corresponding EIP. Then, you can access AWX through www.openeuler-hcie.com:32453/awx/#/login on the client.



After the installation is complete, view the default password of the **admin** user.

```
[root@k8smaster1 ~]# kubectl get secret awx-openeuler-admin-password -n awx -o
jsonpath="{.data.password}" | base64 --decode
7yv37sFLaj3EPQ00LgwuT3sOoVQLkCgg
```

Use the password to log in to the system.



7.3 Basic AWX Operations

7.3.1 AWX Initial Experience

Example: Create a **test** file in the **/tmp** directory on the two controlled hosts (172.16.0.12 and 172.16.0.13) as user **test**.

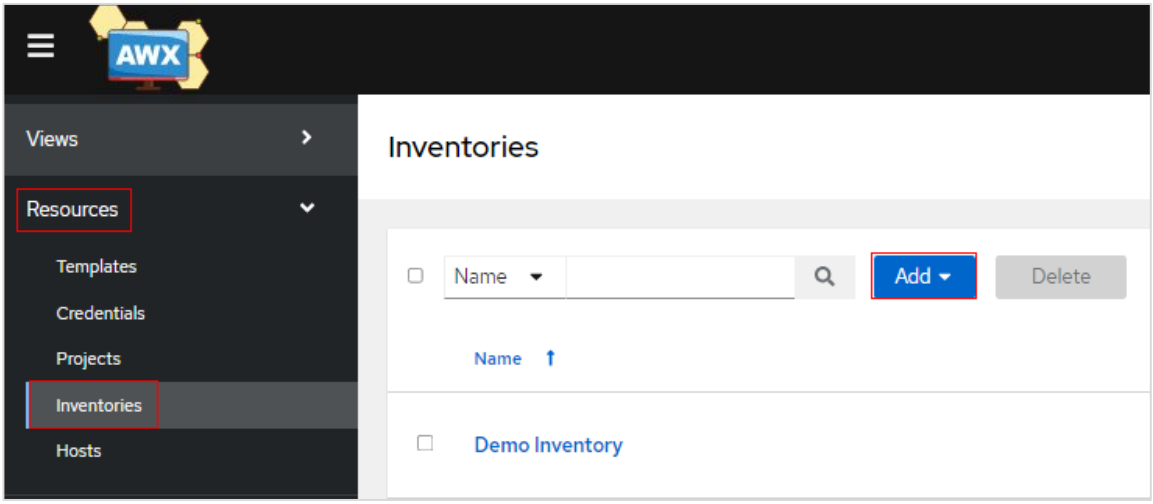
Step 1 Create user **test** on the two controlled hosts.

```
useradd test
passwd test
echo "test ALL=(ALL) ALL" > /etc/sudoers.d/test
```

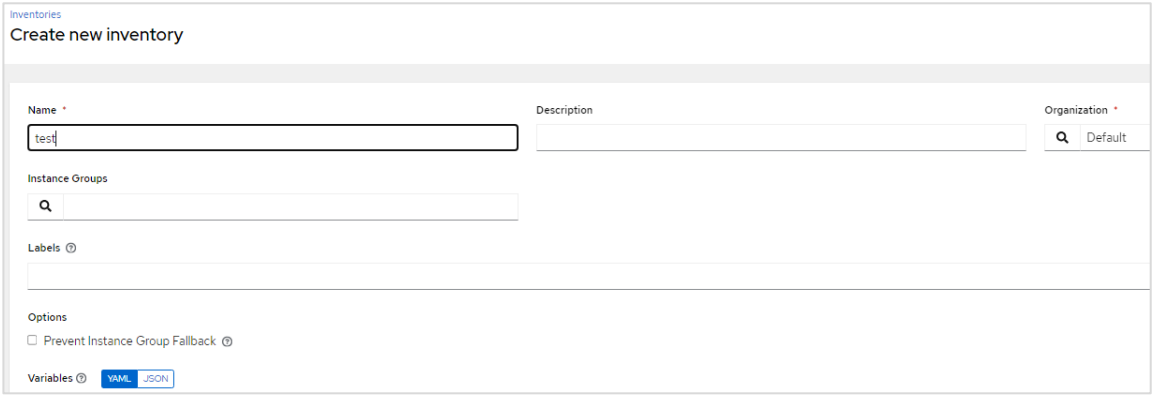
```
[root@client-2 /]# useradd test
[root@client-2 /]# passwd test
Changing password for user test.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[root@client-2 /]# echo "test ALL=(ALL) ALL" > /etc/sudoers.d/test
```

Step 2 Create an inventory in AWX.

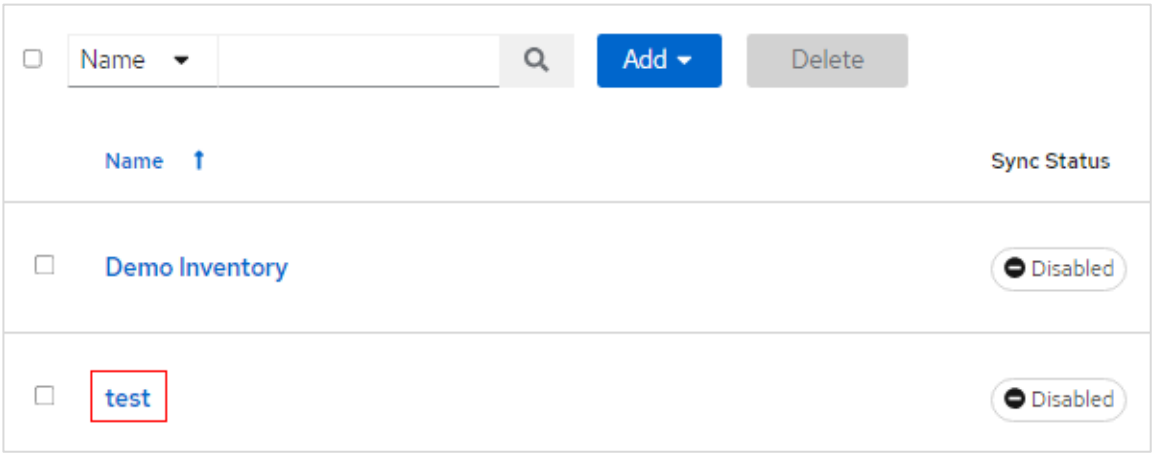
On the menu bar on the right of the AWX, click **Resources**, and then click **Inventories** to go to the corresponding page. On the new page, click **Add**.



In the displayed dialog box, click **Add inventory** and set parameters as required, as shown in the following figure.

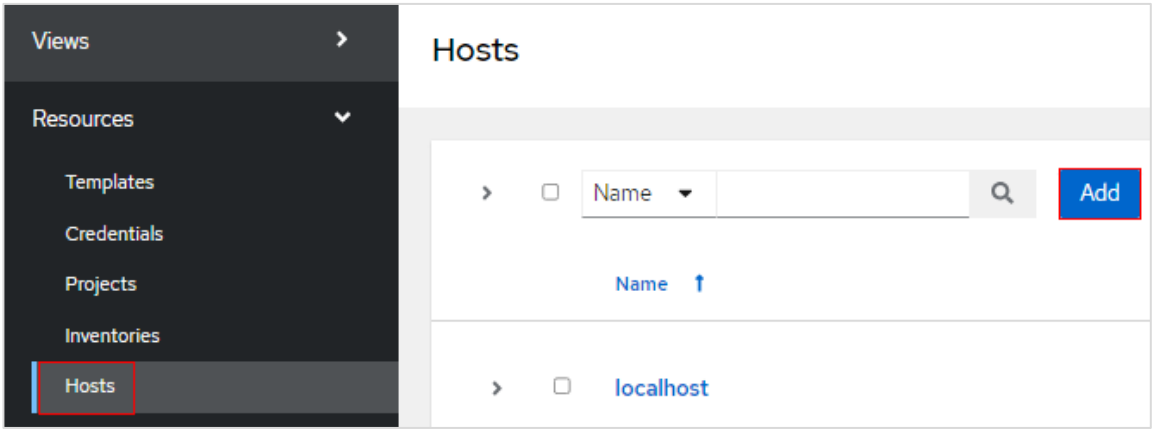


After the configuration is complete, click **Save**. Click **Inventories** again to return to the corresponding page. You can view the created inventory, as shown in the following figure.

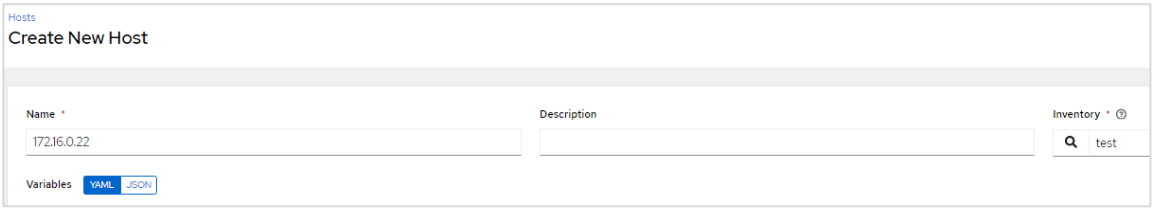


Step 3 Add hosts.

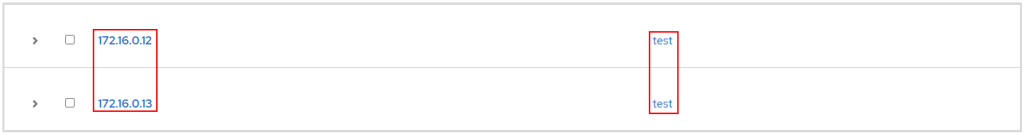
Choose **Resources** > **Hosts**, and click **Add**.



On the displayed page, enter information as prompted. Set **Name** to the IP address of the host and **Inventory** to **test** created in the previous step, as shown in the following figure.

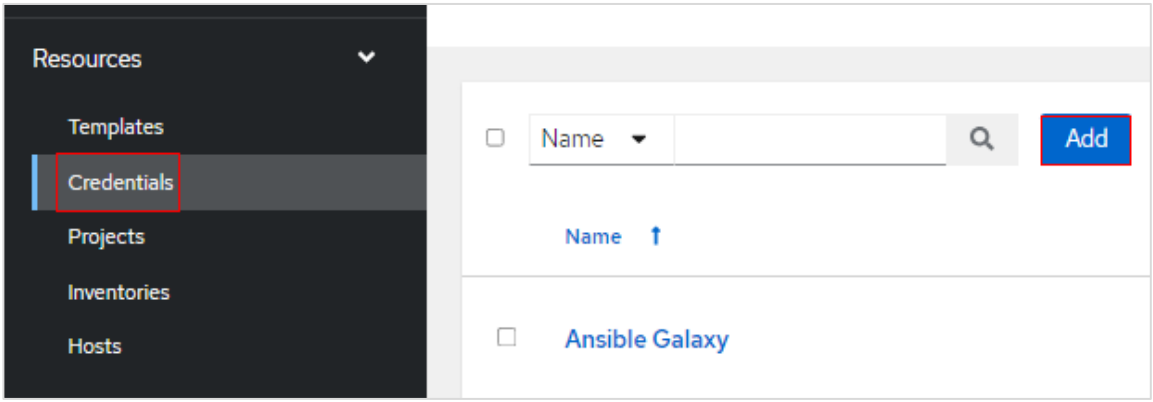


Use the same method to add another controlled host. After the configuration is complete, return to the **Hosts** page to view the added hosts, as shown in the following figure.

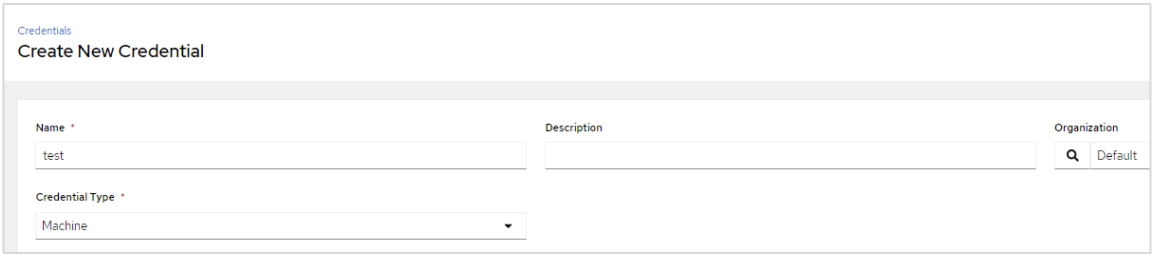


Step 4 Create a credential.

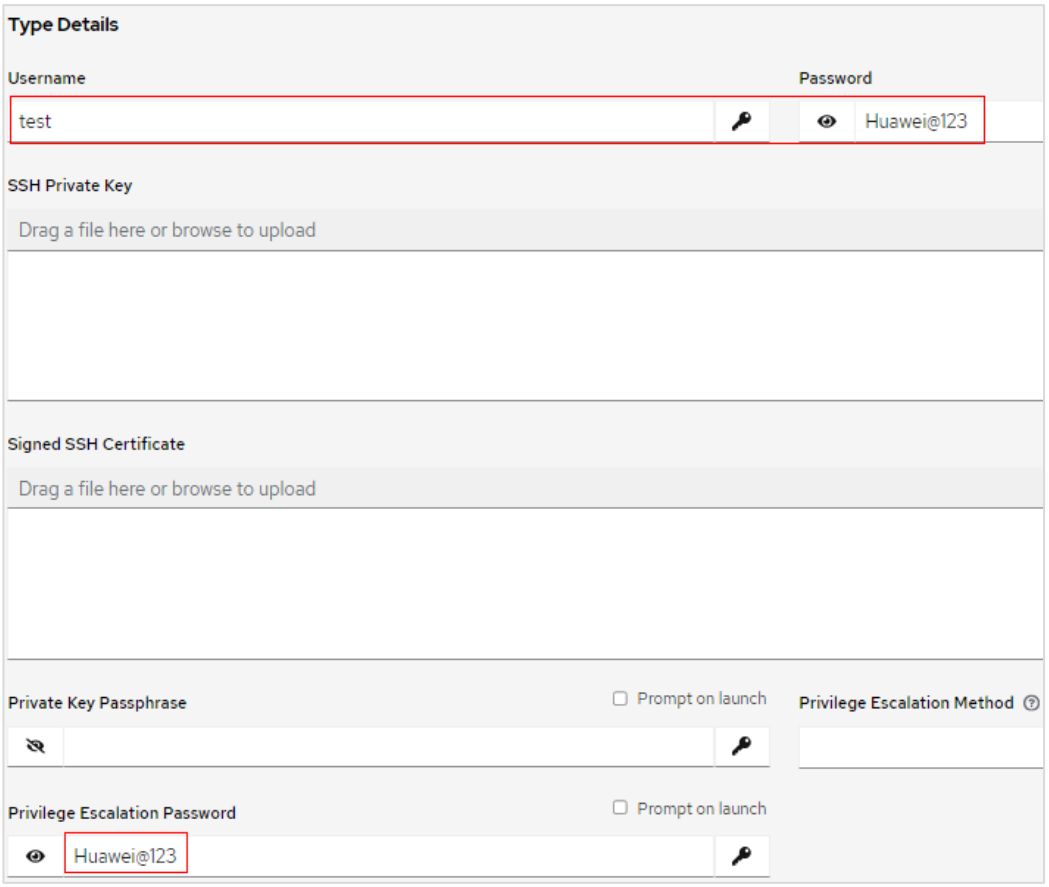
Choose **Resources** > **Credentials**, and click **Add**.



Enter the corresponding configuration as prompted. Set **Name** to **Custom**, **Organization** to **Default**, and **Credential Type** to **Machine**, as shown in the following figure.



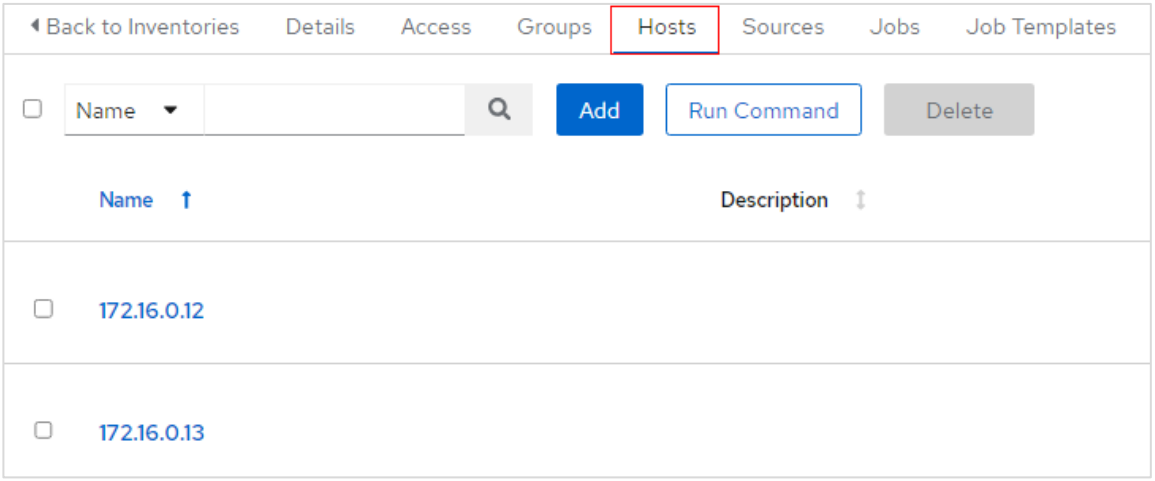
After you select **Machine**, the corresponding options are displayed in the lower part. Enter the user name and password in the corresponding dialog box based on the configuration in Step 1.



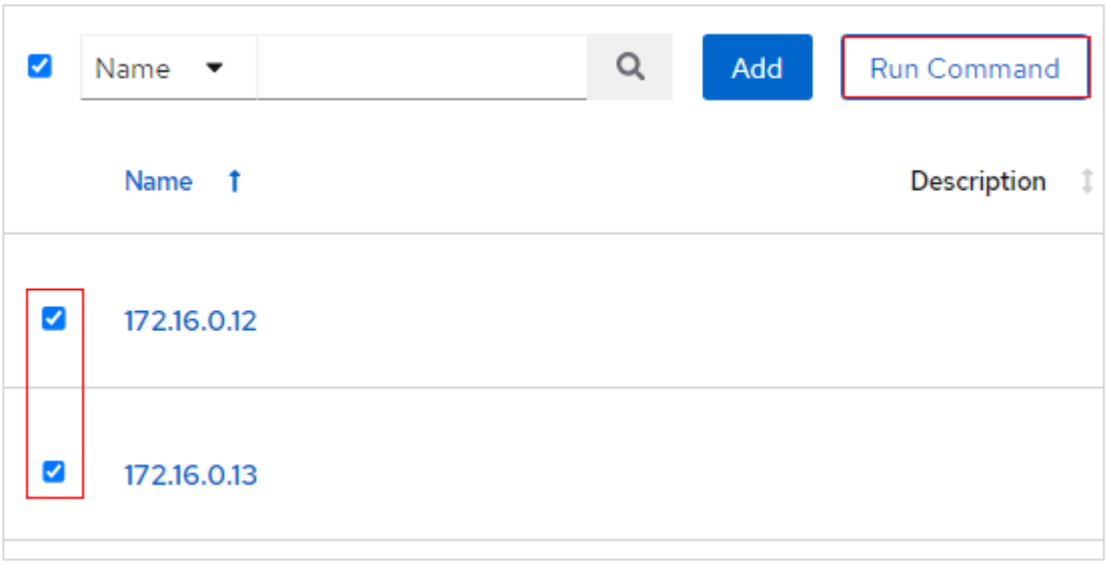
After the configuration is complete, click **Save**.

Step 5 Create a file.

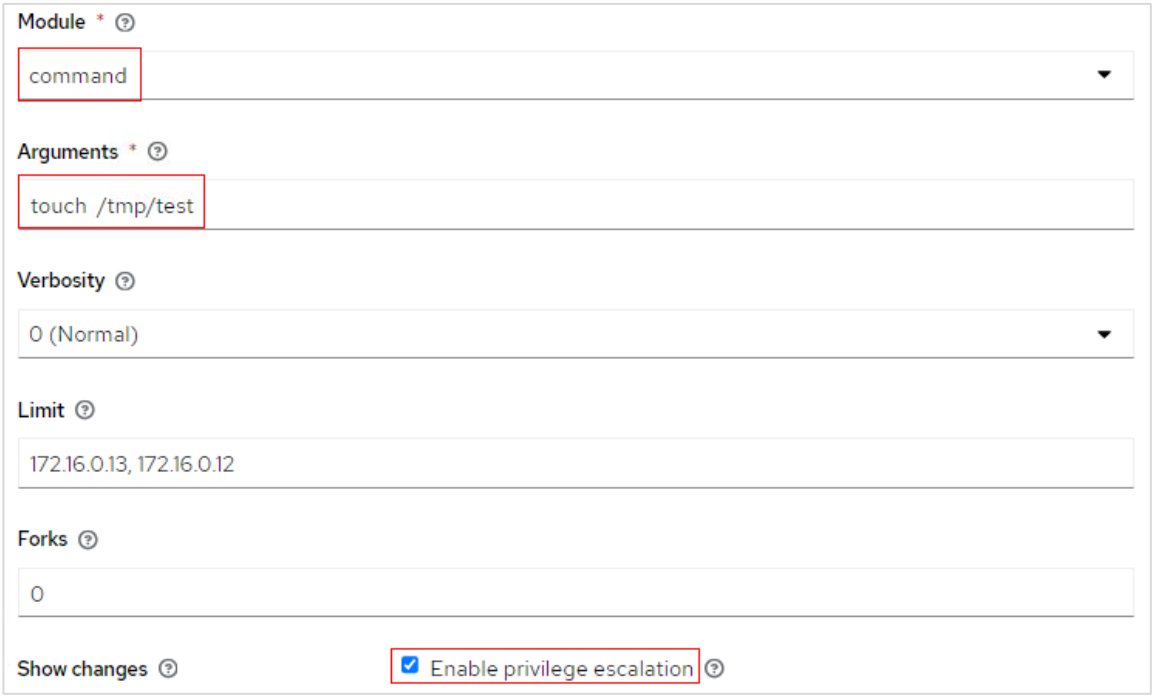
On the **Inventories** page, click **test** created in Step 2. On the displayed page, click **Hosts**, as shown in the following figure.



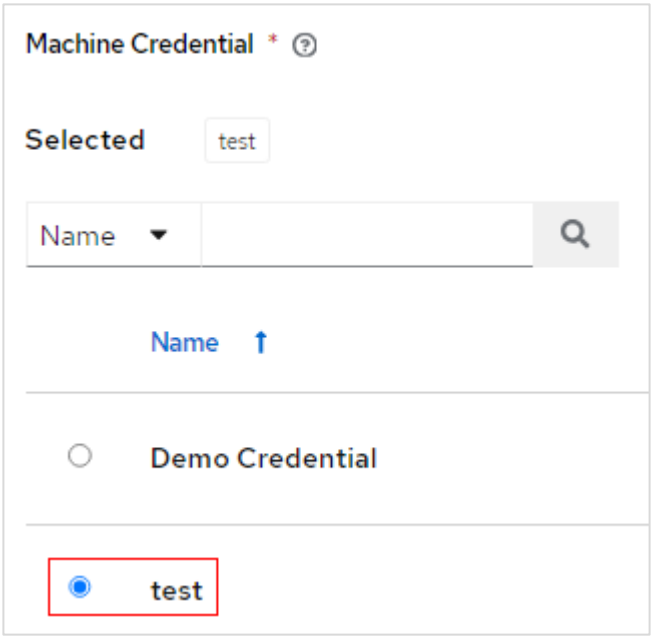
Select two hosts and click **Run Command**, as shown in the following figure.



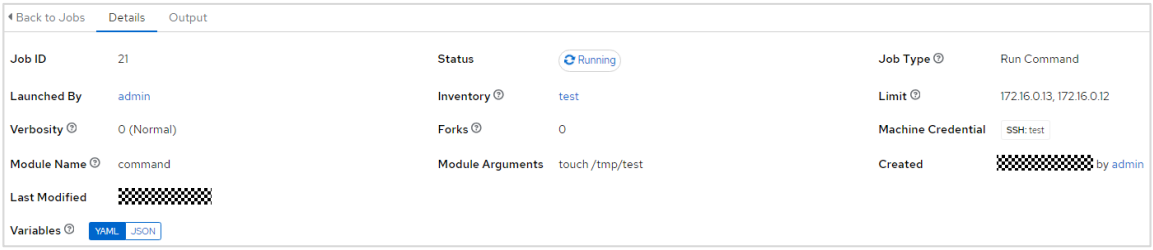
On the displayed page, set **Module**, **Parameter**, and **Verbosity** as prompted, and select **Enable Previlege Escalation**.



Click **Next** twice to go to the **Credentials** page, select the credential created in Step 4 and click **Next**.



After the configuration is complete, click **Next**. On the displayed page, click **Start**. The **Jobs** page is displayed. You can click **Details** to view the job status. On the **Output** page, wait until the task is complete. Details page:



Output page:



- Note: In the preceding figure, one host runs properly, and the other fails. If both of them run properly, the following information is displayed:

```
SSH password:
BECOME password[defaults to SSH password]:
172.16.0.13 | CHANGED | rc=0 >>

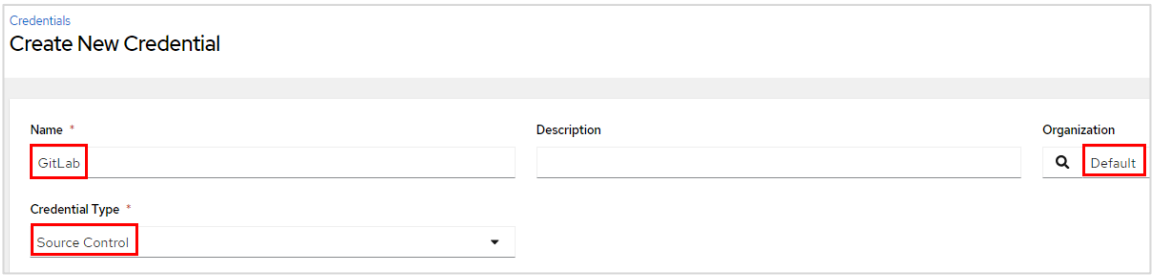
172.16.0.12 | CHANGED | rc=0 >>
```

7.3.2 Interconnecting with GitLab and Performing Operations

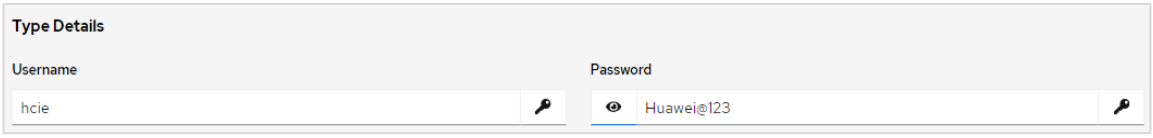
Exercise requirements: Connect the previously installed and deployed GitLab to AWX to prepare for CI/CD.

Step 1 Create a credential.

Go to the page for creating a credential by referring to Step 4 in section 7.3, input a name, set **Organization** to **Default**, and set **Credential Type** to **Source Control**, as shown in the following figure.



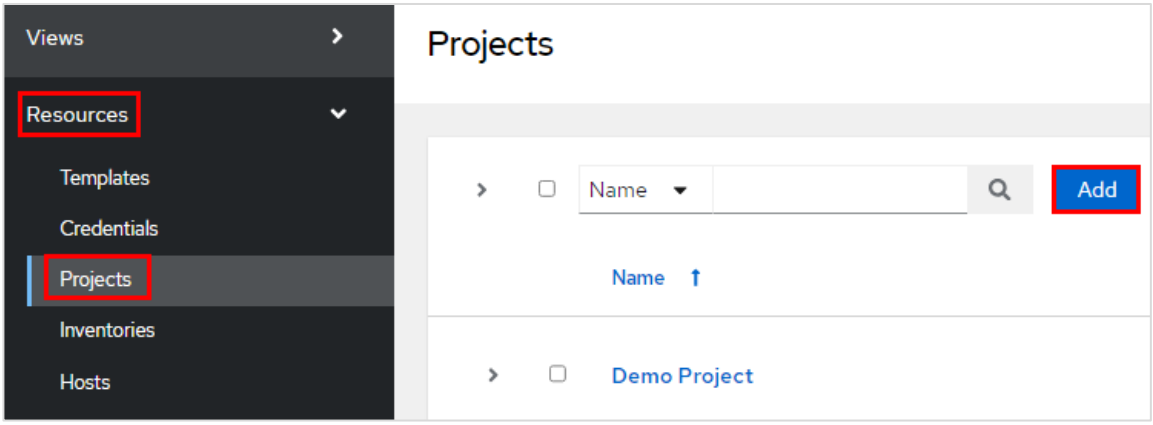
On the displayed page, enter the user name **hcie** and password **Huawei@123** created in Step 3 in section 2.3, as shown in the following figure.



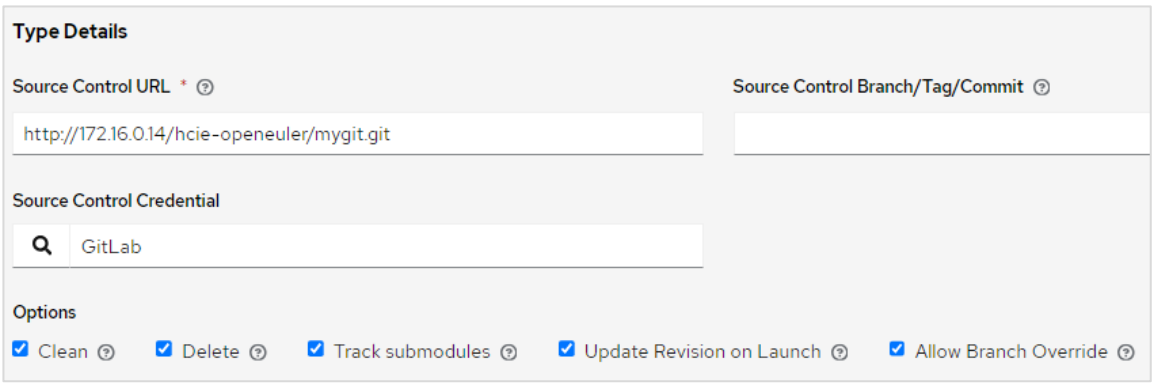
After the creation is complete, click **Save**.

Step 2 Create a project.

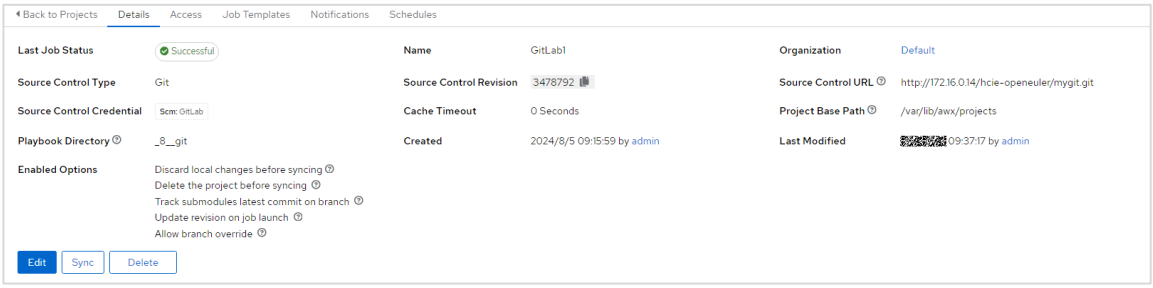
Choose **Resources** > **Projects**, and click **Add**.



Enter the following information as prompted:

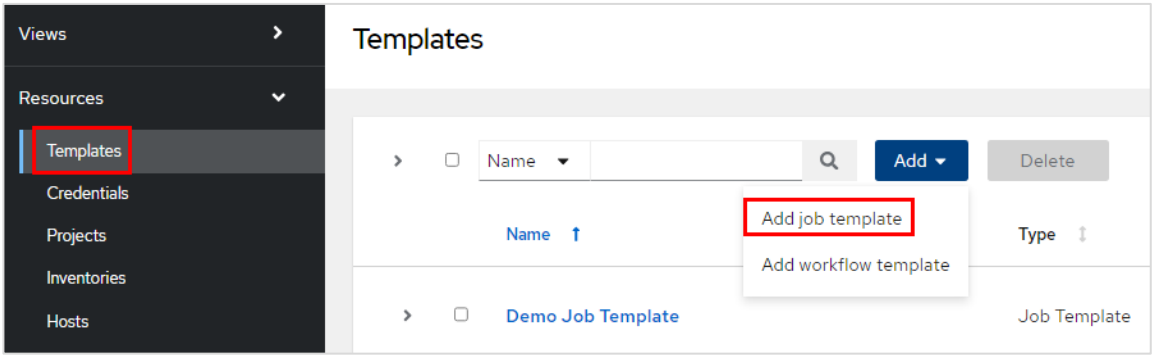


After the configuration is complete, click **Save**. The system automatically passes the code and displays the synchronization result.

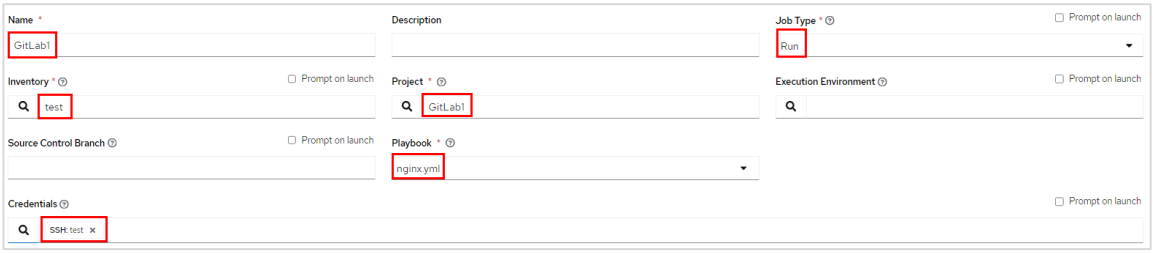


Step 3 Create a template.

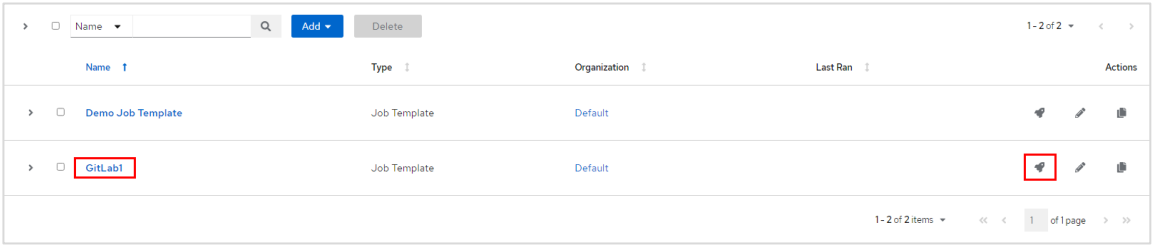
Choose **Resources** > **Templates**, click **Add**, and select **Add job template**, as shown in the following figure.



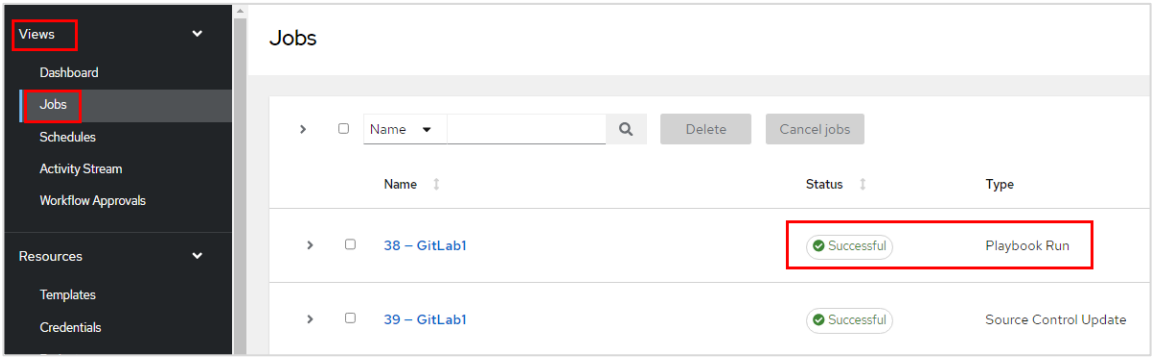
On the displayed page, enter the configuration as prompted. Select **GitLab1** created in the previous step. The playbook is automatically populated to **nginx.yml** of the project, as shown in the following figure.



After the template is created, click **Save** to return to the **Templates** page. Click **Launch Template** next to **GitLab1** to execute the template, as shown in the following figure.



After the template is started, you can view the result on the **Jobs** page under **Views**, as shown in the following figure.



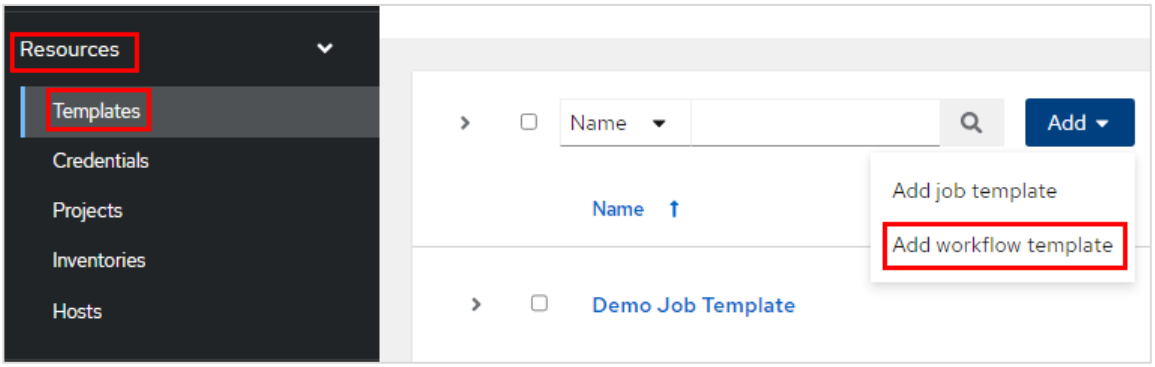
Click the job name to go to the details page and view the task details and output.

7.3.3 Creating a Workflow Template

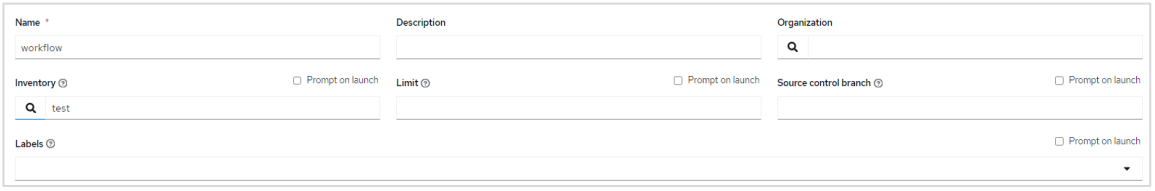
Exercise requirements: With the project and template created in section 7.3.2, pull the latest project code each time a task in the template is executed to ensure that the task to be executed is the latest.

Step 1 Create a workflow template.

Choose **Resources** > **Templates**, click **Add**, and select **Add workflow template**, as shown in the following figure.

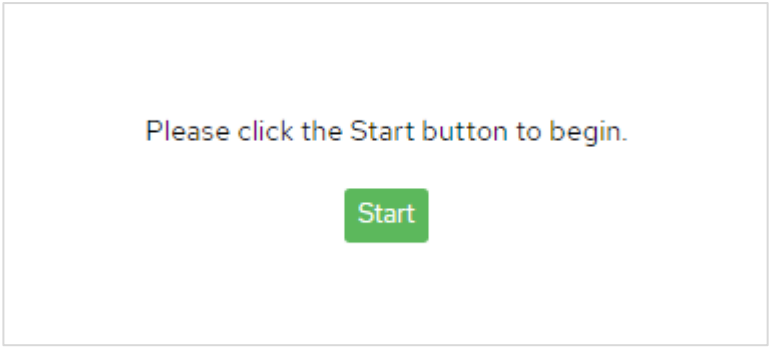


Type information as prompted on the displayed page. Set **Name** to **User-defined**, **Organization** to **Default**, and **Inventory** to **test**, as shown in the following figure.

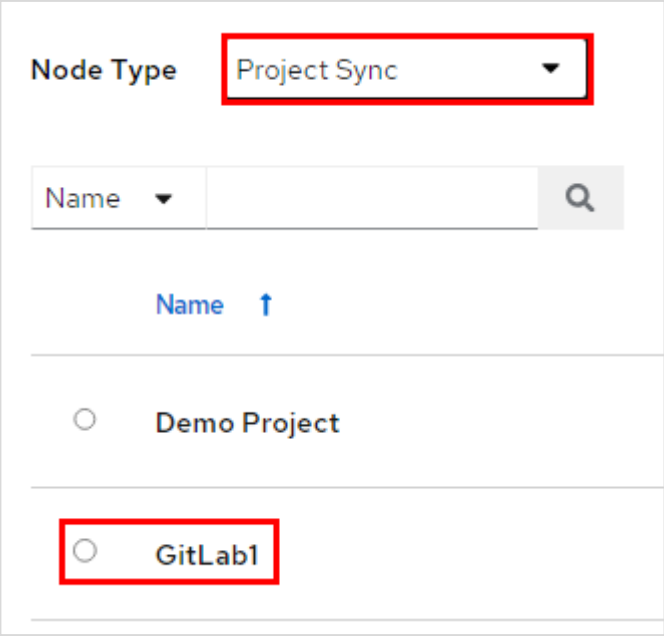


Step 2 Modify a workflow.

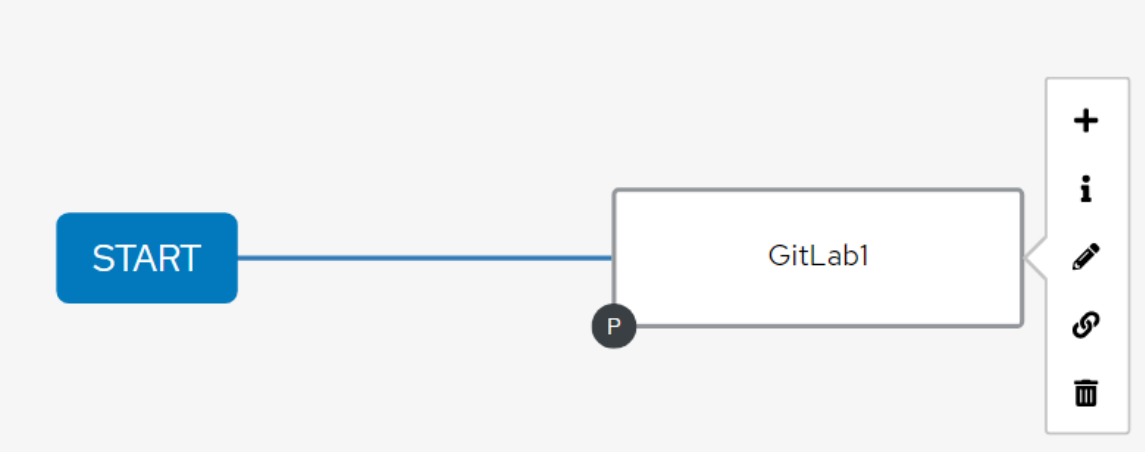
After the workflow template is created, the workflow editing page is displayed. Click **Start**, as shown in the following figure.



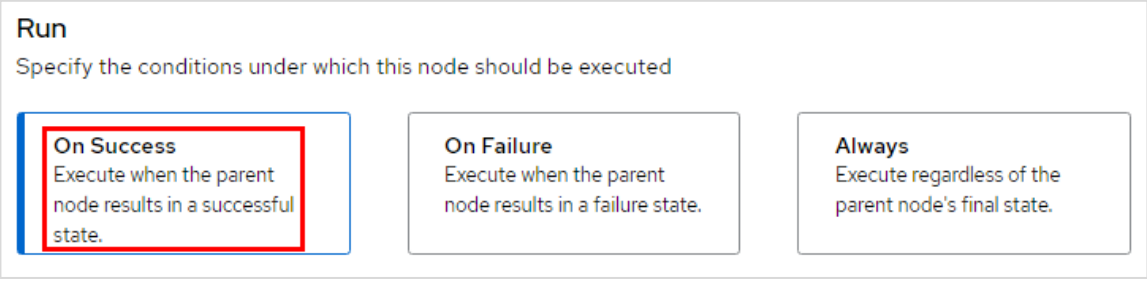
Pull the latest code as required before the template task is executed. Therefore, set **Node Type** to **Project Sync** and select the corresponding project, as shown in the following figure.



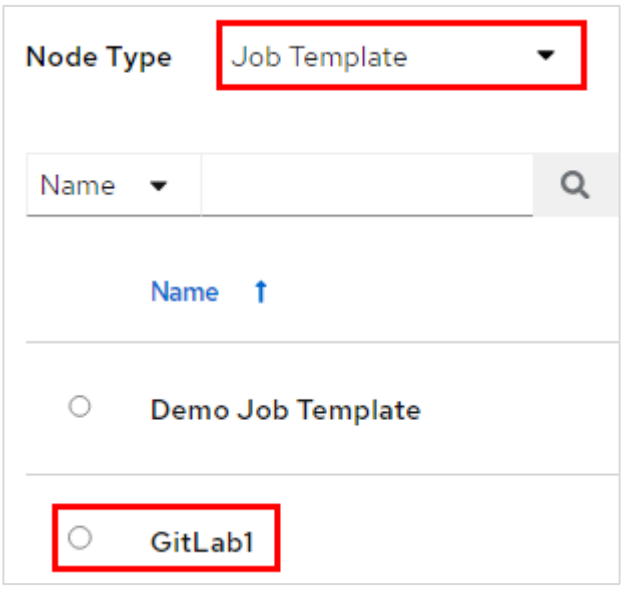
Select the first task of the workflow and click the plus sign (+) to set the next task, as shown in the following figure.



There are three execution conditions for the next task: **On Success** (used in this exercise), **On Failure**, and **Always**.



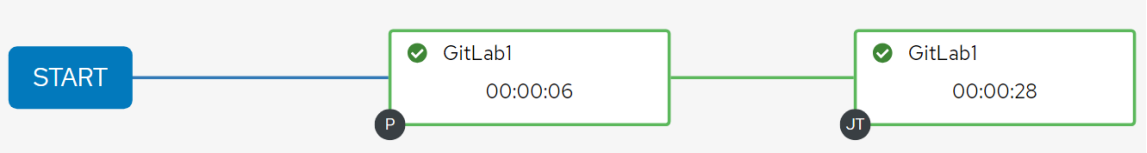
Set the new task to **Job Template** and select the corresponding task, as shown in the following figure.



Click **Save** in the upper right corner of the editing page to return to the **Templates** page. Click the start button next to the newly created workflow, as shown in the following figure.



The system automatically switches to the corresponding **Output** page. On this page, you can view the detailed execution result.

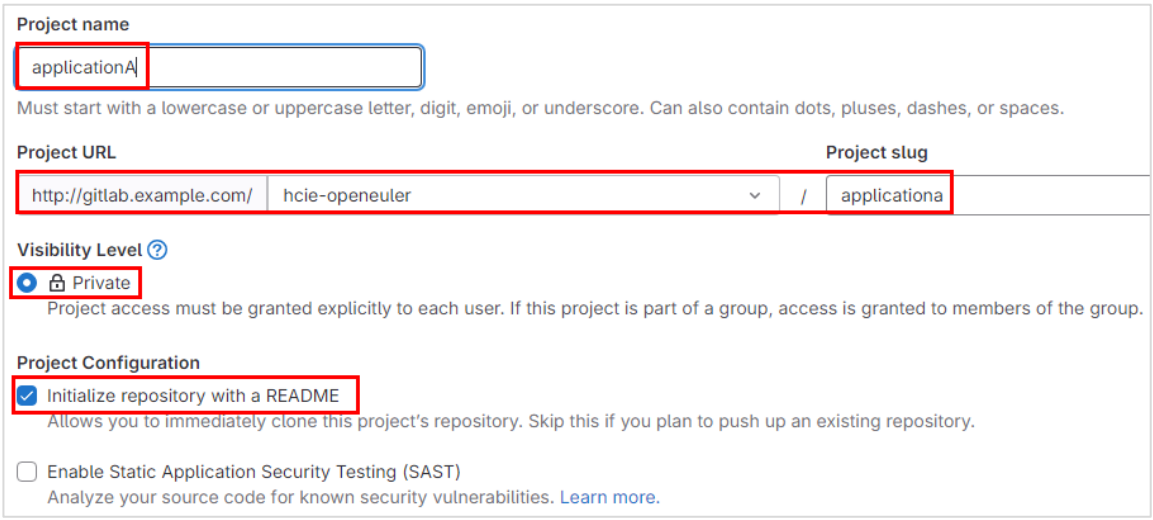


7.4 Performing Comprehensive AWX Practices

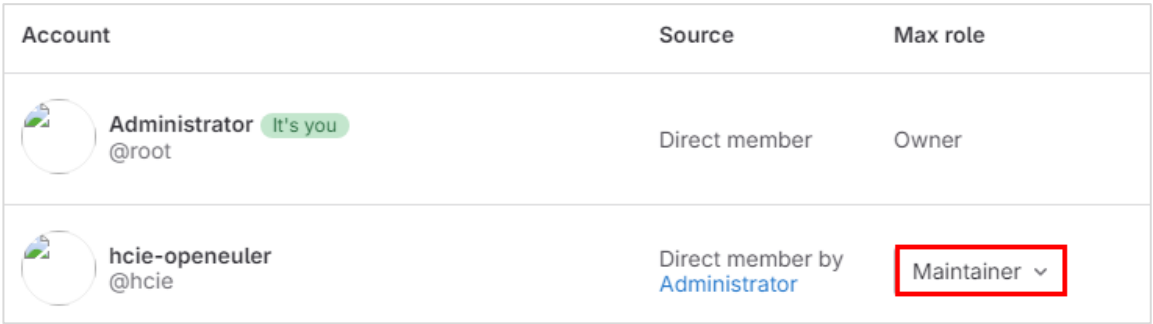
Example: Assume that project team A is developing an application in agile development mode. The development Dept. is responsible for application development, while the operations Dept. is responsible for application deployment and maintenance. To perform tests , the development Dept. needs to manually deploy the application operating environment before each application test, which is laborious. Therefore, the operations Dept. compiles a playbook to automatically deploy the operating environment and authorizes the playbook to the development Dept. through AWX. Then, the development Dept. only needs to run the script on AWX to perform tests, which greatly improves the working efficiency. In addition, the development Dept. can synchronize the modification to the operating environment during tests to the operations Dept., which updates the playbook accordingly.

Step 1 Prepare resources.

Create the repository **applicationA** on GitHub and initialize it.



Set the **hcie** user to **Maintainer** of the repository.



Pull a project on the client.

```
[root@gitlab ~]# git clone http://172.16.0.14/hcie-openeuler/applicationa
Cloning into 'applicationa'...
Username for 'http://172.16.0.14': hcie
Password for 'http://hcie@172.16.0.14':
warning: redirecting to http://172.16.0.14/hcie-openeuler/applicationa.git/
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```

Use Nginx to simulate dependencies required by the application and enable port 81 on the firewall. The YAML file is as follows:

```
---
- hosts: all

  tasks:
    - name: install nginx
      yum:
        name: nginx
        state: present
    - name: enable nginx&firewalld
      service:
        name: "{{ item }}"
        state: started
        enabled: yes
      loop:
        - nginx
        - firewalld
    - name: config firewalld
      firewalld:
        port: 81/tcp
        permanent: yes
        immediate: yes
        state: enabled
```

After the editing is complete, push the code to the corresponding repository.

```
[root@gitlab applicationa]# git add .
[root@gitlab applicationa]# git commit -m "1.0"
[root@gitlab applicationa]# git push
```

```
[root@gitlab applicationa]# git add ops.yml
[root@gitlab applicationa]# git commit -m "1.0"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

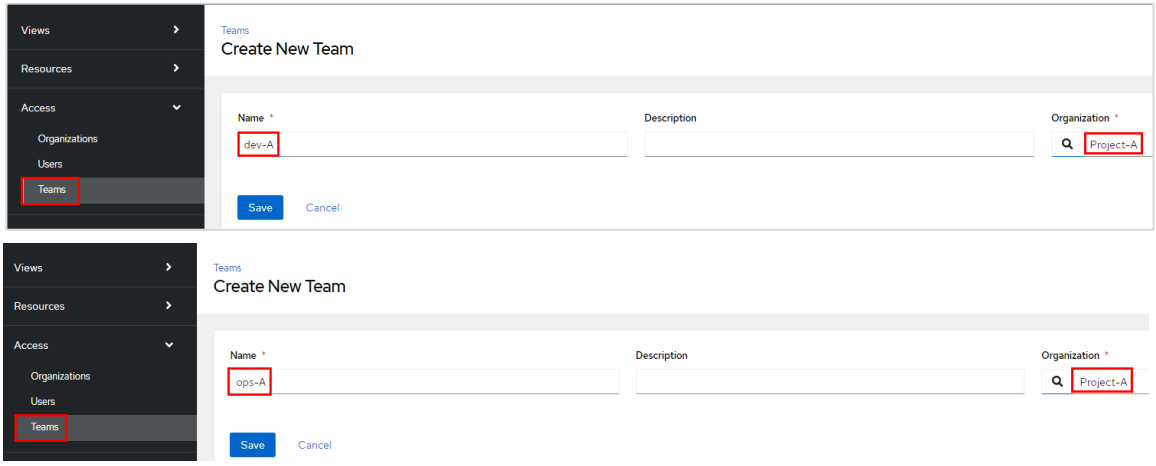
nothing to commit, working tree clean
[root@gitlab applicationa]# git push
Username for 'http://172.16.0.14': hcie
Password for 'http://hcie@172.16.0.14':
warning: redirecting to http://172.16.0.14/hcie-openeuler/applicationa.git/
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 397 bytes | 397.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To http://172.16.0.14/hcie-openeuler/applicationa
   08a96b5..f618efe  main -> main
```

Step 2 Create an organization and group and assign permissions to them.

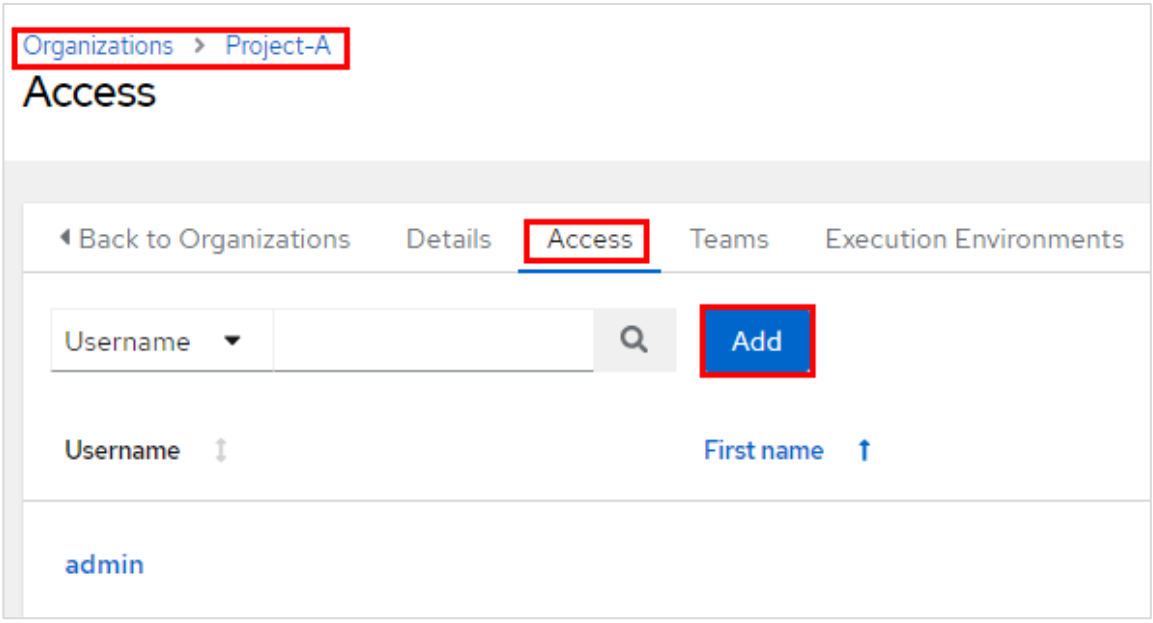
Create a dedicated organization **Project-A** for this project.



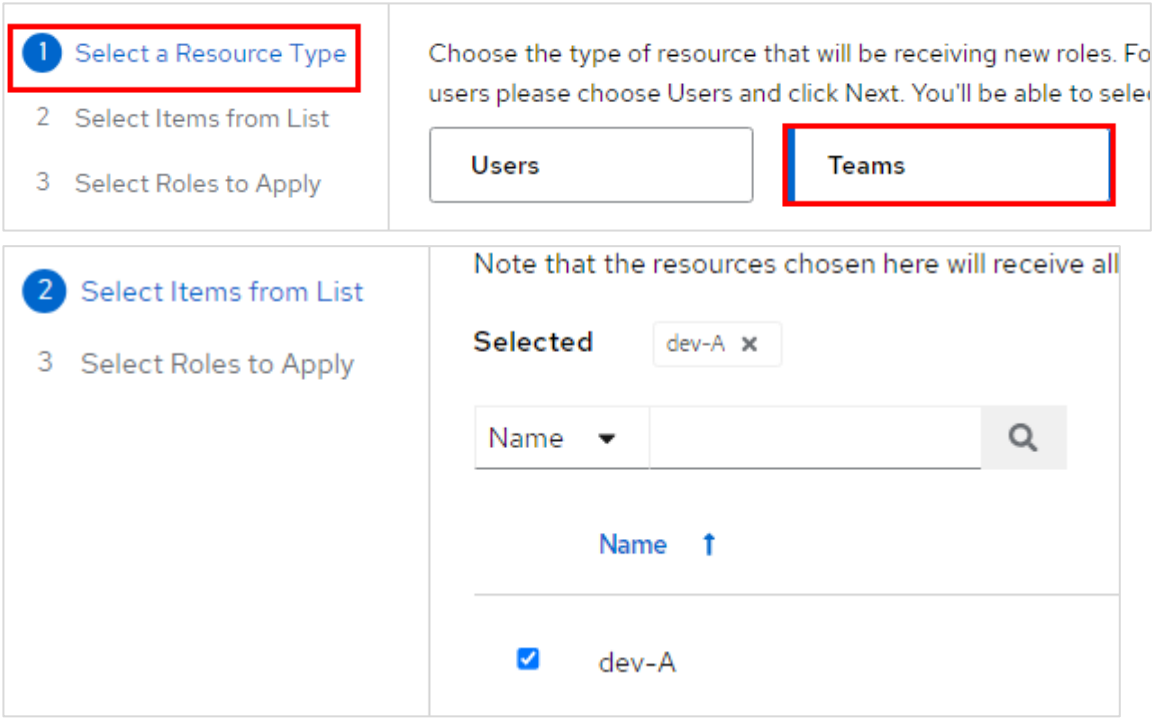
Create two groups **dev-A** and **ops-A**, which both belong to **Project-A**.

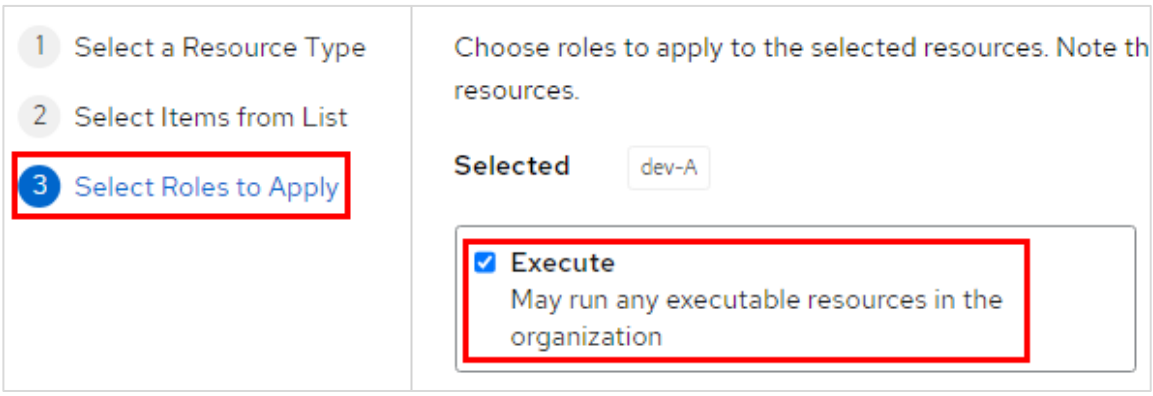


Assign permissions to **dev-A** to allow all users in this group to run all resources.

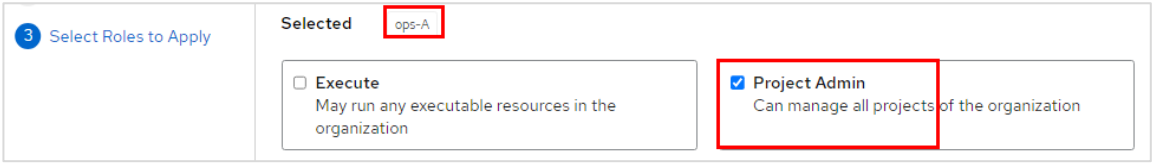


On the displayed page, select **Teams** and **dev-A**, and click **Execute**.



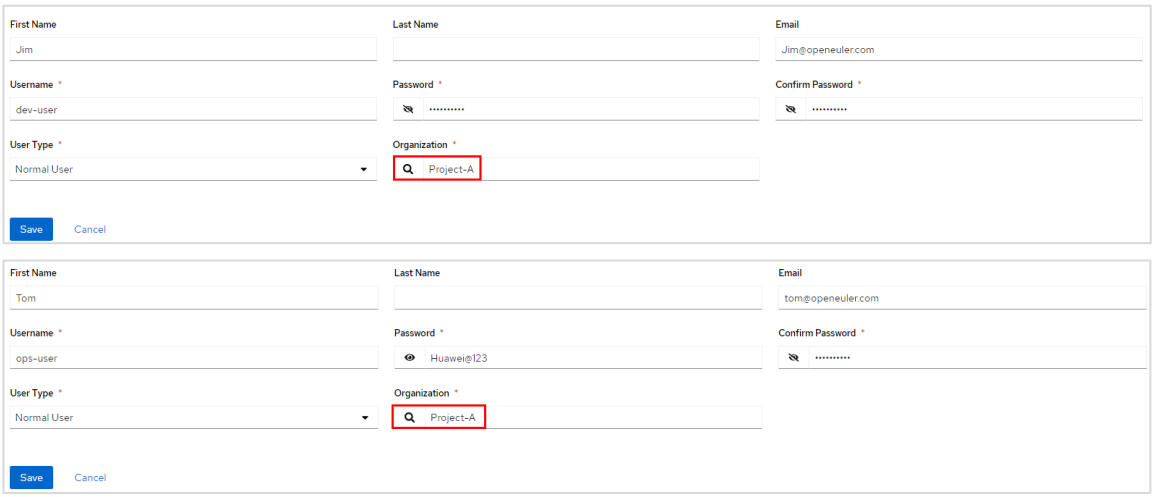


Use the same method to authorize **ops-A** so that users in it can manage all resources (as project administrators) of the organization.

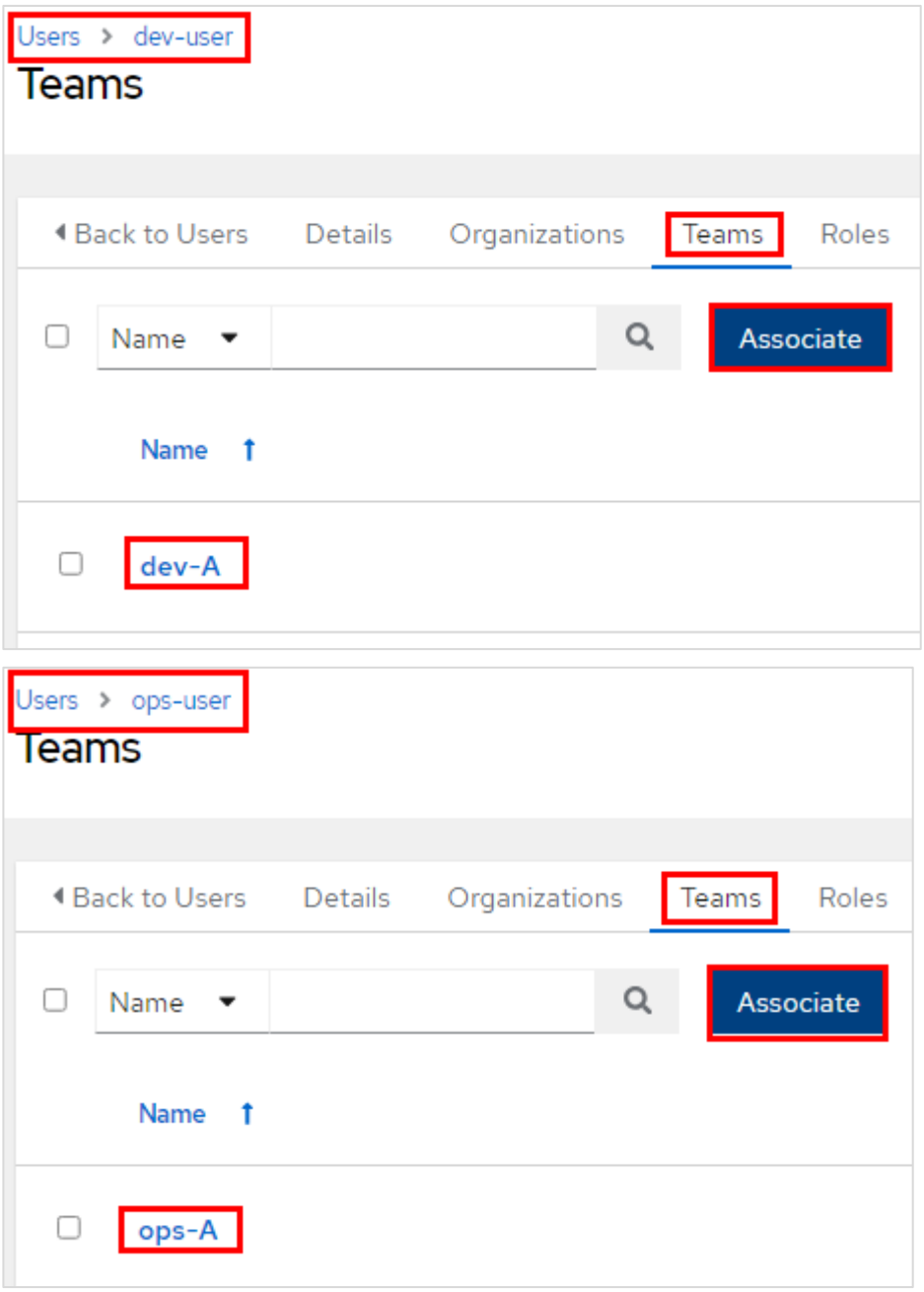


Step 3 Create a user.

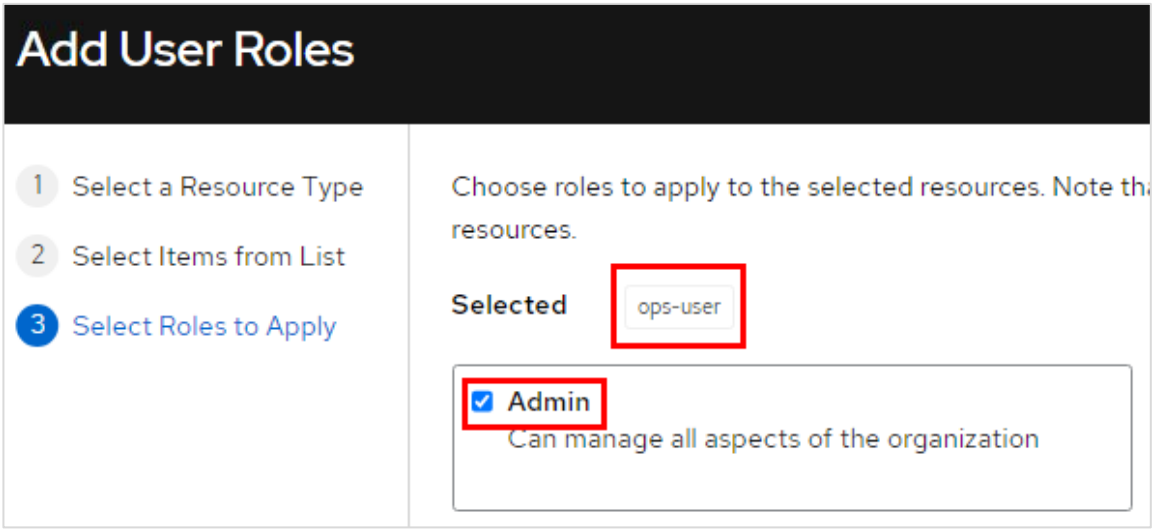
Create two users, **dev-user** and **ops-user**, for developers and O&M personnel, respectively.



After that, add the two users to the corresponding groups.



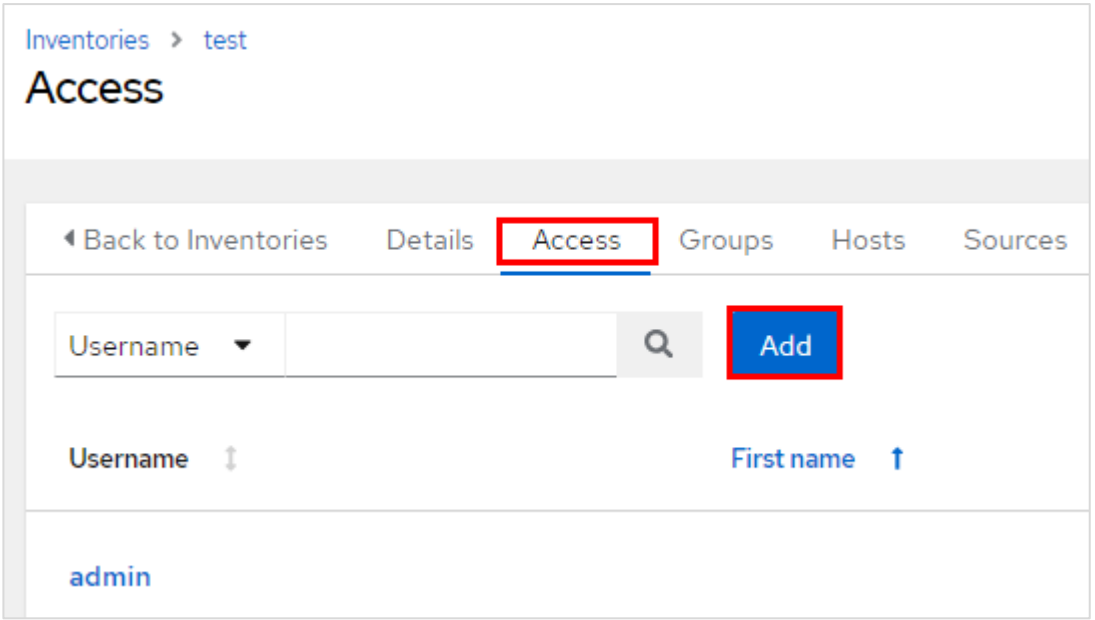
After the users are created, choose **Organizations** > **Access** to assign administrator permissions for **ops-user**.



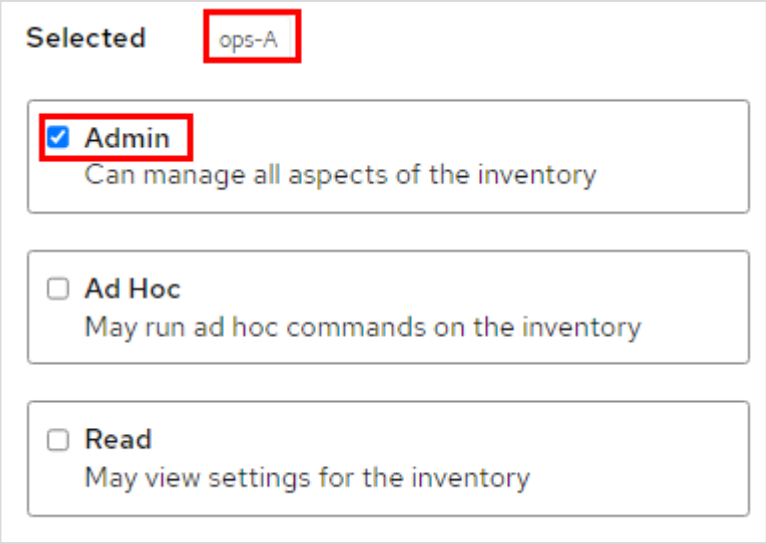
ops-user will be used to create resources for subsequent tasks.

Step 4 Add host and inventory permissions to users.

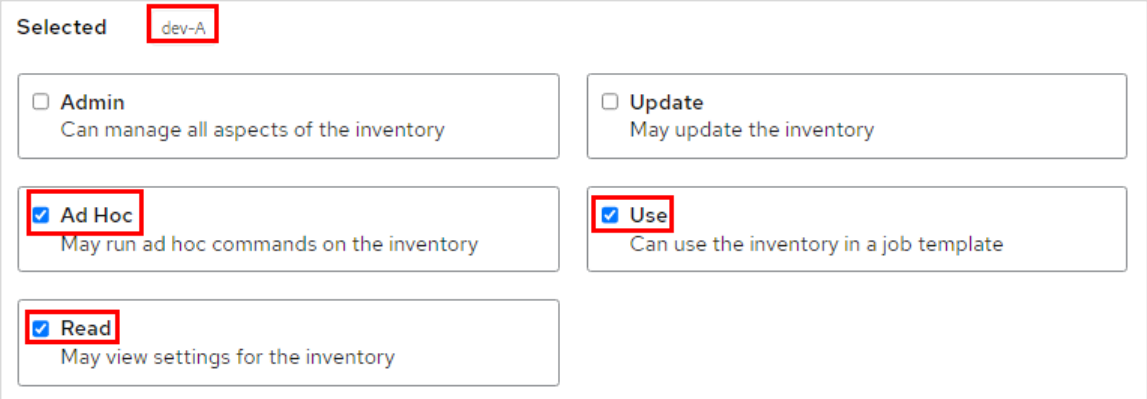
The inventories and hosts added in section 7.3.1 will be used in subsequent exercises. Therefore, assign permissions to the newly created users.



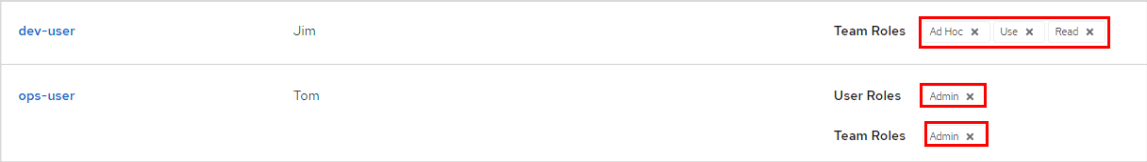
On the displayed page, choose **Groups** > **ops-A** > **Admin**.



Use the same method to add the use, temporary, and read permissions for **dev-A**.



The corresponding permissions are shown in the following figure.



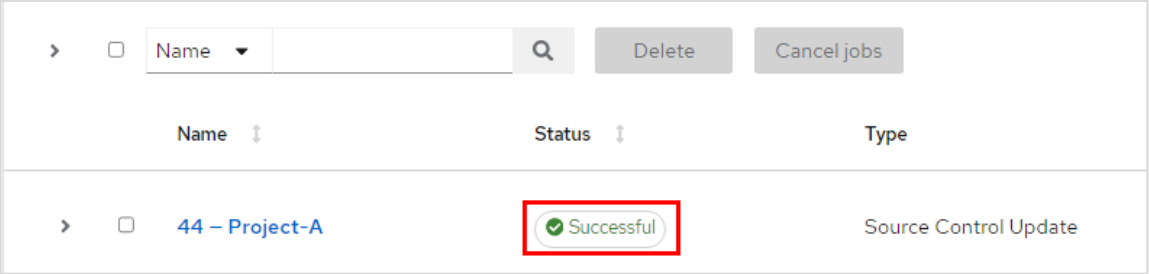
Step 5 Add credentials.

This exercise requires two credentials, one for pulling a playbook (named **GitLab-A**) from GitLab and the other for executing the playbook (named **Run-A**) on the host. For details, see the preceding steps.

Step 6 Create a project.

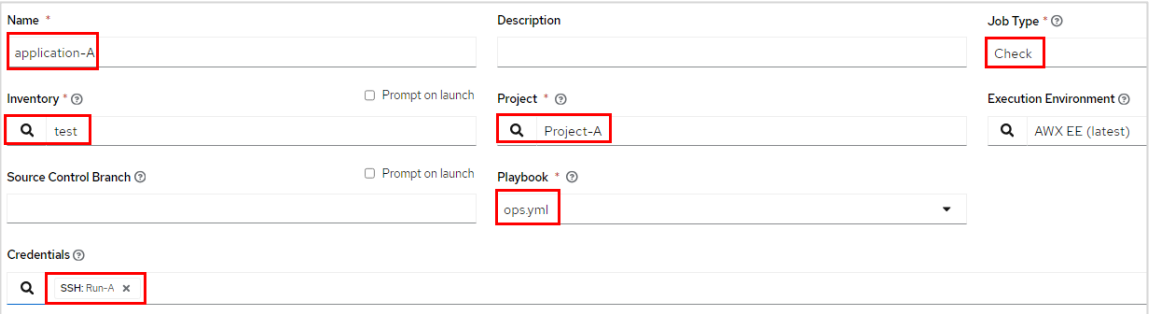
Create **Project-A** in AWX. For details, see Step 2 in section 7.3.2.

After the creation is complete, wait until the synchronization is complete on the **Jobs** page under **View**.

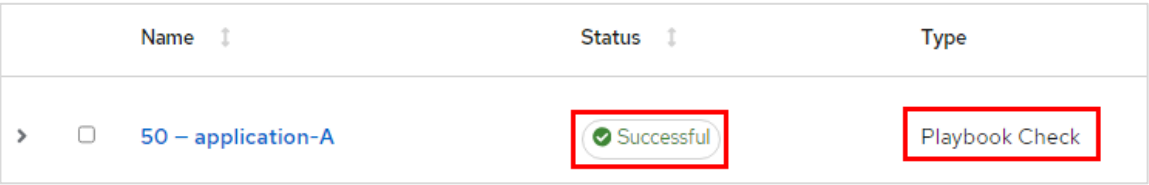


Step 7 Add a job template.

Add a job template according to the method described in section 7.3.2 and set **Job Type** to **Check**.

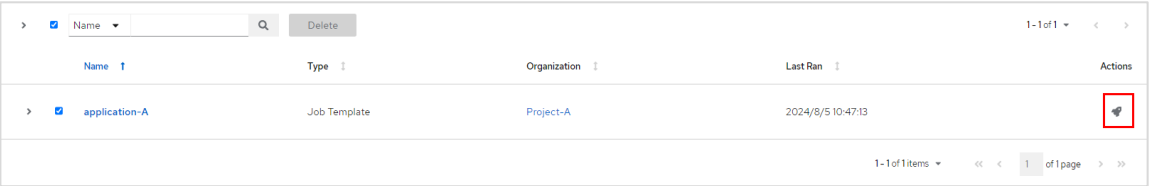


Click **Start** to check whether the job can run properly.



Step 8 Run the template before deploying services as a developer.

Log in to the AWX as **dev-user**, find the corresponding job template, and execute it.



- Question: Can developers deploy the environment? Why?
- Answer: No. When creating a job template, **ops-user** sets the template type to **Check**. And **dev-user** does not have the permission to modify the template. Therefore, **dev-user** still checks the template upon startup. **dev-user** can deploy the environment only **ops-user** changes the template type to **Run**.

8 Jenkins Installation, Deployment, Configuration, and Usage

8.1 Introduction

8.1.1 About This Exercise

This exercise finalizes the initial CI/CD setup using Jenkins and outlines two scenarios: utilizes Jenkins, GitLab, and AWX in scenario 1, while employing Jenkins, Kubernetes, and Harbor in scenario 2.

8.1.2 Objectives

The exercise illustrates the content of the *Introduction to Jenkins CI/CD* course and describes how to implement CI/CD on Jenkins.

8.2 Jenkins Installation and Deployment

8.2.1 Storage Resource Preparations

Step 1 Create a resource pool in Ceph.

The specific steps have been described in previous exercises. The following describes only key steps.

Run the following commands to create storage resources required by Jenkins in Ceph:

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_jenkins
[ceph: root@ceph01 /]# ceph osd pool create data_jenkins
[ceph: root@ceph01 /]# ceph fs new jenkins metadata_jenkins data_jenkins
[ceph: root@ceph01 /]# ceph orch apply mds jenkins --placement="2"
[ceph: root@ceph01 /]# ceph fs authorize jenkins client.jenkins / rwps -o ceph.client.jenkins.keyring
```

Step 2 Compile the PVC and **StorageClass** files.

All Jenkins-related objects are stored in the namespace named **devops**. Therefore, set the related parameter to **devops** during file compilation.

Create a **secret** file with the following content:

```
---
apiVersion: v1
```

```
kind: Secret
metadata:
  name: csi-secret
  namespace: devops
stringData:
  # Required for statically provisioned volumes
  userID: jenkins
  userKey: AQA9UgllXXXXXXXXXXXXXXWgaY6F1OA==

  # Required for dynamically provisioned volumes
  adminID: admin
  adminKey: AQA9UgllXXXXXXXXXXXXXXWgaY6F1OA==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

Create a **ceph-conf-map.yaml** file. The file content is as follows:

```
---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    {}
metadata:
  name: ceph-csi-encryption-kms-config
---
apiVersion: v1
kind: ConfigMap
data:
  ceph.conf: |
    [global]
    auth_cluster_required = cephx
    auth_service_required = cephx
    auth_client_required = cephx

  # keyring is a required key and its value should be empty
  keyring: |
metadata:
  name: ceph-config
```

Create a **csi-config-map.yaml** file. The file content is as follows:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: "ceph-csi-config"
data:
  config.json: |-
    [
      {
        "clusterID": "3568bbc0-84XXXXXXXXXXa163ef0485d",
        "monitors": [
          "192.168.0.21:6789",
          "192.168.0.22:6789",
```

```
            "192.168.0.23:6789"
          ]
        }
      ]
```

Set **--extra-create-metadata** to **false**.

The PVC file contains the following information:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins
  namespace: devops
spec:
  storageClassName: jenkins
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

The **StorageClass** file contains the following information:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: jenkins
  namespace: devops
provisioner: cephfs.csi.ceph.com
parameters:
  clusterID: d5faf9ba-56c0-11ee-9ffe-fa163edb3348
  fsName: jenkins
  pool: data_jenkins
  csi.storage.k8s.io/provisioner-secret-name: csi-secret
  csi.storage.k8s.io/provisioner-secret-namespace: devops
  csi.storage.k8s.io/controller-expand-secret-name: csi-secret
  csi.storage.k8s.io/controller-expand-secret-namespace: devops
  csi.storage.k8s.io/node-stage-secret-name: csi-secret
  csi.storage.k8s.io/node-stage-secret-namespace: devops
  mounter: kernel
reclaimPolicy: Delete
allowVolumeExpansion: true
```

Run the **kubectl create namespace devops** command to create the required namespace, and then deploy objects such as config-map, secret, PVC, and StorageClass in sequence to ensure that the PVC status is normal.

```
[root@k8smaster1 jenkins]# kubectl get pvc -n devops
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   AGE
jenkins   Bound    pvc-89ca1f24-36fd-454a-ad56-9a6599ca1e0d   5Gi        RWO            jenkins        10s
```

8.2.2 Jenkins Installation

Step 1 Implement authentication management.

Compile a file for authentication management as follows:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-admin
rules:
  - apiGroups: [""]
    resources: ["*"]
    verbs: ["*"]
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-admin
  namespace: devops
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: jenkins-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: jenkins-admin
subjects:
- kind: ServiceAccount
  name: jenkins-admin
  namespace: devops
```

After the compilation is complete, deploy it. The result is as follows:

```
[root@k8smaster1 jenkins]# kubectl apply -f jenkins.yml
clusterrole.rbac.authorization.k8s.io/jenkins-admin created
serviceaccount/jenkins-admin created
clusterrolebinding.rbac.authorization.k8s.io/jenkins-admin created
```

Step 2 Deploy Jenkins.

Jenkins is deployed in Kubernetes as a deployment. Compile the corresponding file as follows:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins
  namespace: devops
spec:
  replicas: 1
  selector:
```

```
    matchLabels:
      app: jenkins-server
  template:
    metadata:
      labels:
        app: jenkins-server
    spec:
      securityContext:
            fsGroup: 1000
            runAsUser: 1000
      serviceAccountName: jenkins-admin
      containers:
        - name: jenkins
          image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/jenkins:lts
          resources:
            limits:
              memory: "2Gi"
              cpu: "1000m"
            requests:
              memory: "500Mi"
              cpu: "500m"
          ports:
            - name: httpport
              containerPort: 8080
            - name: jnlpport
              containerPort: 50000
          livenessProbe:
            httpGet:
              path: "/login"
              port: 8080
            initialDelaySeconds: 90
            periodSeconds: 10
            timeoutSeconds: 5
            failureThreshold: 5
          readinessProbe:
            httpGet:
              path: "/login"
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 10
            timeoutSeconds: 5
            failureThreshold: 3
          volumeMounts:
            - name: jenkins-data
              mountPath: /var/jenkins_home
      volumes:
        - name: jenkins-data
          persistentVolumeClaim:
              claimName: jenkins
```

After the compilation is complete, deploy the deployment and wait until it status is normal.

```
[root@k8smaster1 jenkins]# kubectl get deploy -n devops
NAME                            READY   UP-TO-DATE   AVAILABLE   AGE
csi-cephfsplugin-provisioner    3/3     3            3           86m
jenkins                         1/1     1            1           2m26s
```

Step 3 Deploy a service.

Compile a service file as follows:

```
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  namespace: devops
  annotations:
      prometheus.io/scrape: 'true'
      prometheus.io/path:     /
      prometheus.io/port:     '8080'
spec:
  selector:
    app: jenkins-server
  type: NodePort
  ports:
    - name: http
      port: 8080
      targetPort: 8080
      nodePort: 32000
    - name: agent
      port: 50000
      protocol: TCP
```

After the compilation is complete, deploy the service and wait until it status is normal.

```
[root@k8smaster1 jenkins]# kubectl get service -n devops
NAME                          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
csi-cephfsplugin-provisioner  ClusterIP   [redacted]      <none>        8080/TCP         89m
csi-metrics-cephfsplugin      ClusterIP   [redacted]      <none>        8080/TCP         89m
jenkins-service               NodePort    [redacted]      <none>        8080:32000/TCP   10s
```

Step 4 (Optional) Use Ingress to release the service.

Compile the Ingress file as follows:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: jenkins
  namespace: devops
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
 rules:
   - host: www.openeuler-hcie.com
     http:
       paths:
```

```
          - backend:
              service:
                name: jenkins-service
                port:
                  number: 8080
            path: /jenkins(/|$)(.*)
            pathType: Prefix
```

After the compilation is complete, deploy Ingress and wait until it status is normal.

```
[root@k8smaster1 jenkins]# kubectl describe ingress jenkins -n devops
Name:             jenkins
Labels:           <none>
Namespace:        devops
Address:          10.0.0.12
Default backend:  default-http-backend:80 (<error: endpoints "default-http-backend" not found>)
Rules:
  Host                  Path  Backends
  ----                  ----  --------
  www.openeuler-hcie.com
                        /jenkins(/|$)(.*)   jenkins-service:8080 (10.244.36.[redacted]:8080)
Annotations:            kubernetes.io/ingress.class: nginx
                        nginx.ingress.kubernetes.io/rewrite-target: /$2
                        nginx.ingress.kubernetes.io/use-regex: true
Events:
  Type    Reason  Age                  From                      Message
  ----    ------  ----                 ----                      -------
  Normal  Sync    7m38s (x2 over 8m15s)  nginx-ingress-controller  Scheduled for sync
```

Manually resolve or use DNS to resolve http://www.openeuler-hcie.com/ to the ingress-controller address. Type http://www.openeuler-hcie.com:32000/ in the address box of the browser to attempt to access the ingress-controller address. If the access is normal, information about how to obtain the administrator password pops out.

Run the following command to obtain the administrator password:

```
kubectl exec -it jenkins-xxxxxx cat /var/jenkins_home/secrets/initialAdminPassword -n devops
```

See the following figure.

```
[root@k8smaster1 jenkins]# kubectl exec -it jenkins-
kubectl exec [POD] [COMMAND] is DEPRECATED and will
29bf046c35344aa28d4f76d0f1ed7549
[root@k8smaster1 jenkins]#
```

Use the obtained password to log in to Jenkins. Then, the system prompts you to install plugins. (This step is skipped in this exercise.)

8.3 Basic Jenkins Operations

8.3.1 Resource Preparations

Step 1 Prepare code.

Compile the following code on the client:

```
package main
```

```
import (
        "io"
        "log"
        "net/http"
)
func sayHello(w http.ResponseWriter, r *http.Request) {
        io.WriteString(w, "Hello, openEuler")
}

func main() {
        http.HandleFunc("/", sayHello)
        log.Println("Server is running on port 8085...")
        if err := http.ListenAndServe(":8085", nil); err != nil {
                log.Fatal(err)
        }
}
```

Step 2 Push code.

Create a repository **jenkins-test** in GitLab and push the code to the repository for Jenkins to pull. (The detailed process is not described here.)

Step 3 (Optional) Prepare the Go software package.

Go is required for subsequent application build and compilation, but the corresponding installation package cannot be downloaded from the official website. Therefore, you need to manually place the installation package in a specific directory of Jenkins and set environment variables. In this example, the software package is stored in the **/etc/jenkins/go** directory of the host and mounted to the corresponding position in the pod. This method is not recommended in real-world working scenarios.

Run the following commands on the host where the pod is running to use the Go agent of Huawei Cloud:

```
export GO111MODULE=on
export GOPROXY=https://mirrors.huaweicloud.com/repository/goproxy/
export GONOSUMDB=*
```

Create a **/etc/Jenkins/go** directory and download the Go package of the corresponding version. For example, version 1.17.3 is used in this exercise, which is obtained by running the following commands.

```
wget https://golang.org/dl/go1.17.3.linux-amd64.tar.gz
tar -xzf go1.17.3.linux-amd64.tar.gz
```

After that, add the following content to the deployment of Jenkins:

```
....
              volumeMounts:
                - name: jenkins-data
                  mountPath: /var/jenkins_home
                - name: go
                  mountPath: /etc/jenkins/go
          volumes:
```

```
      - name: jenkins-data
        persistentVolumeClaim:
            claimName: jenkins
      - name: go
        hostPath:
          path: /etc/jenkins/go/go
```

Log in to the pod and configure Go-related environment variables (method used in this exercise). You can also configure them in the YAML file.

```
[root@k8smaster1 jenkins]# kubectl exec -it jenkins-dcb97cd98-dmqd4 -n devops -- /bin/bash
jenkins@jenkins-dcb97cd98-dmqd4:/etc/jenkins/go$ echo "export GOROOT=/etc/jenkins/go" >>
jenkins@jenkins-dcb97cd98-dmqd4:/etc/jenkins/go$ export PATH=$PATH:$GOROOT/bin
```

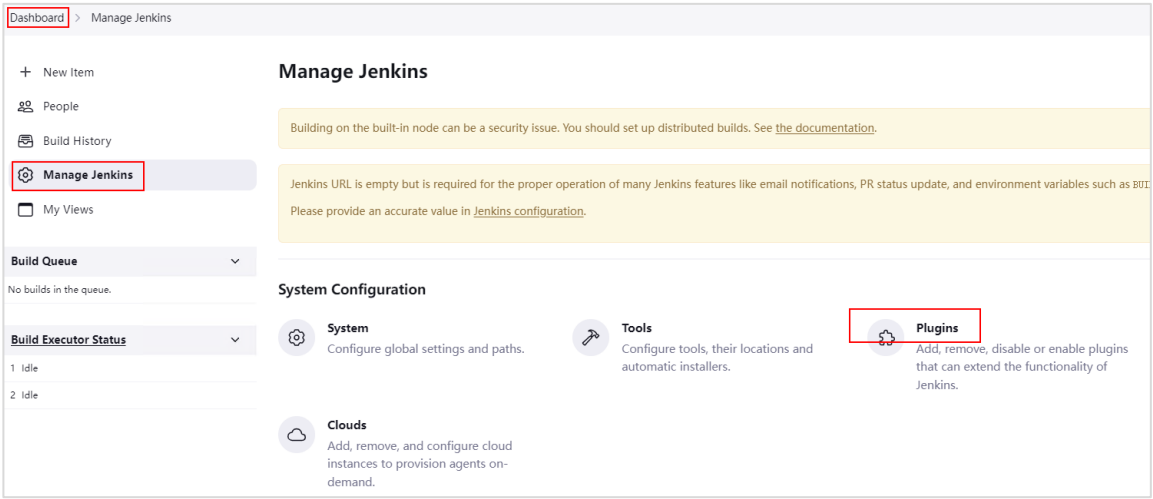
Check whether the configuration takes effect. If it does, the information similar to the following figure is displayed.

```
jenkins@jenkins-dcb97cd98-dmqd4:/etc/jenkins/go$ go version
go version go1.17.3 linux/amd64
jenkins@jenkins-dcb97cd98-dmqd4:/etc/jenkins/go$
```

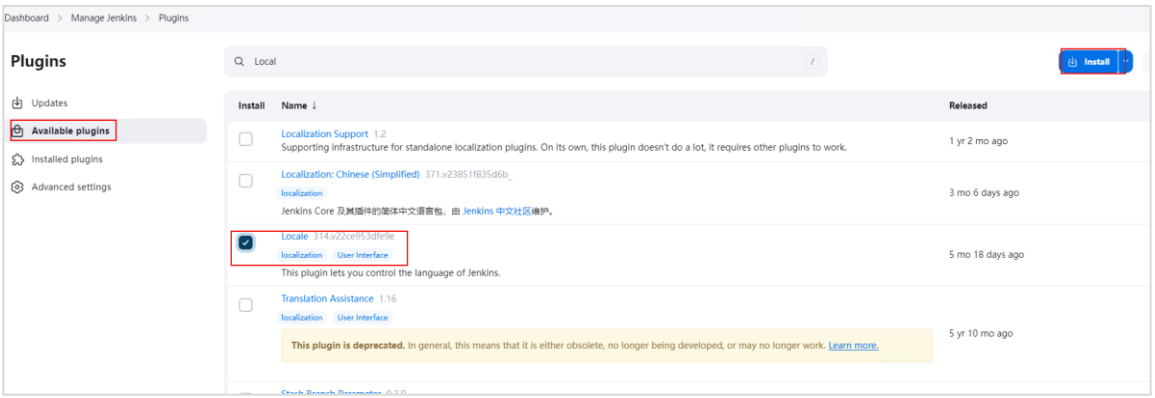
8.3.2 Primary Operations of Jenkins CI/CD

Step 1 (Optional) Initialize Jenkins.

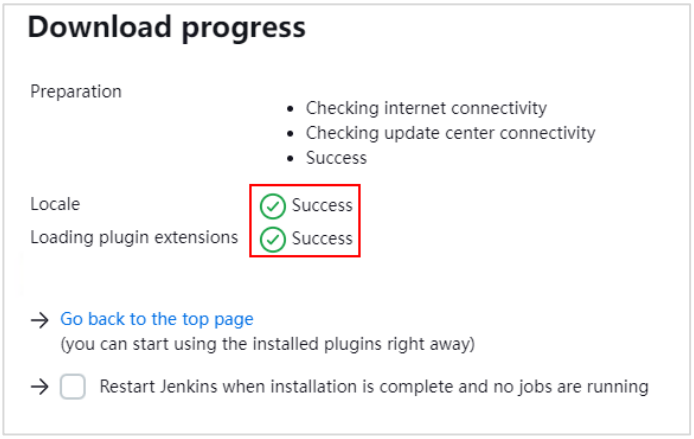
On the **Dashboard** page of Jenkins, click **Manage Jenkins** and then **Plugins** to install plugins.



On the displayed page, click **Available plugins**, type **Locale** in the search box, select the corresponding plugin, and click **Install**.

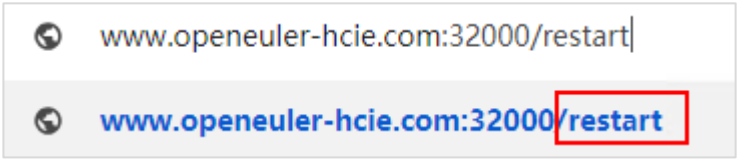


Wait until the plugin is installed.



Use the same method to install the plugin **Localization: Chinese (Simplified)**.

After the installation is complete, add **/restart** to the end of the URL for accessing Jenkins in the browser, as shown in the following figure.

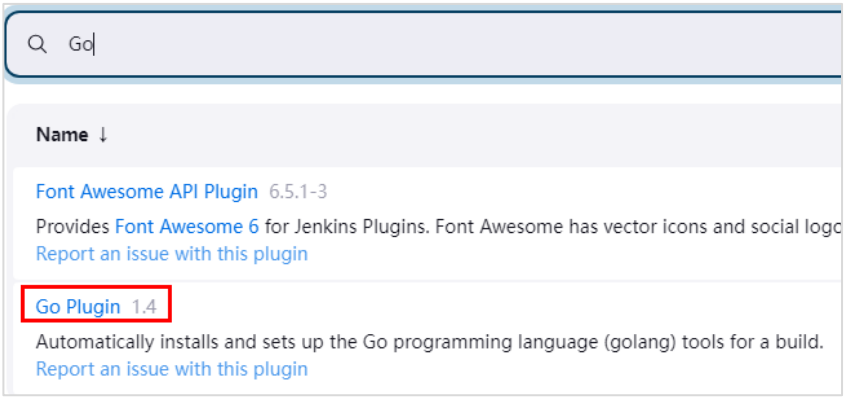


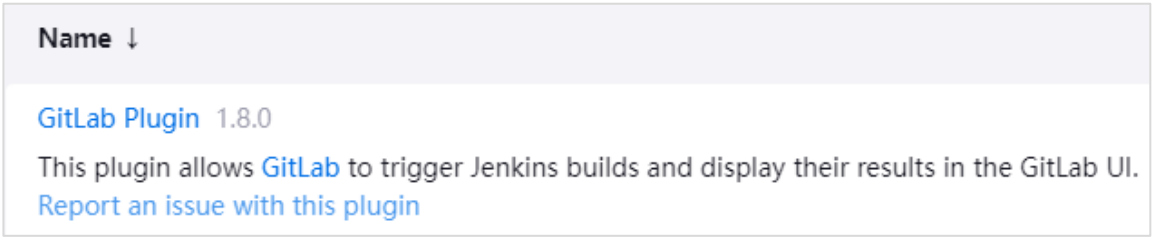
On the displayed page, click **Yes** to restart Jenkins. After the restart is complete, the login page is displayed.

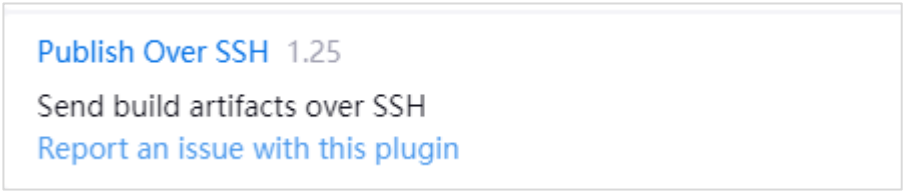
Step 2 Install required plugins.

This exercise requires three plugins: Go Plugin (search for Go), GitLab, and Publish Over SSH. Use the method in step 1 to install them.

After the installation is complete, you can view the installed plugins in **Installed plugins**, as shown in the following figures.



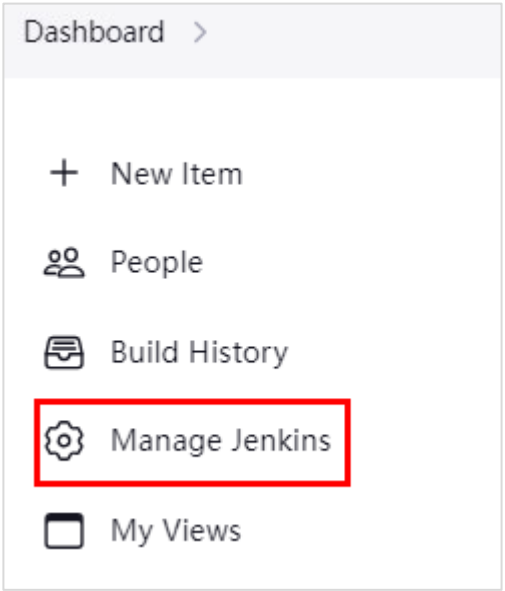


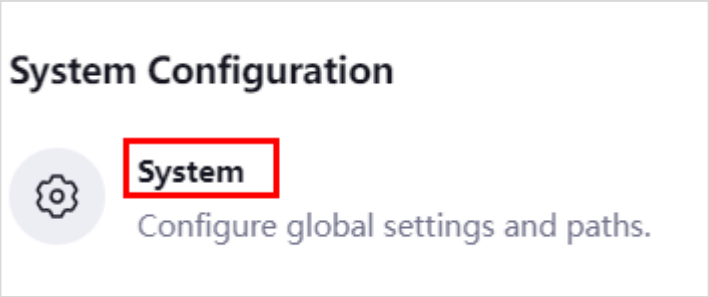


Restart Jenkins for the plugins to take effect.

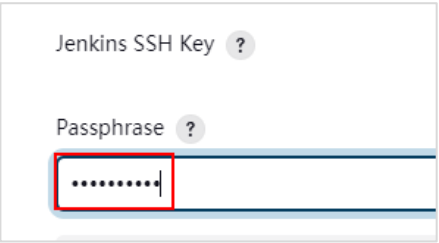
Step 3 Add a remote host.

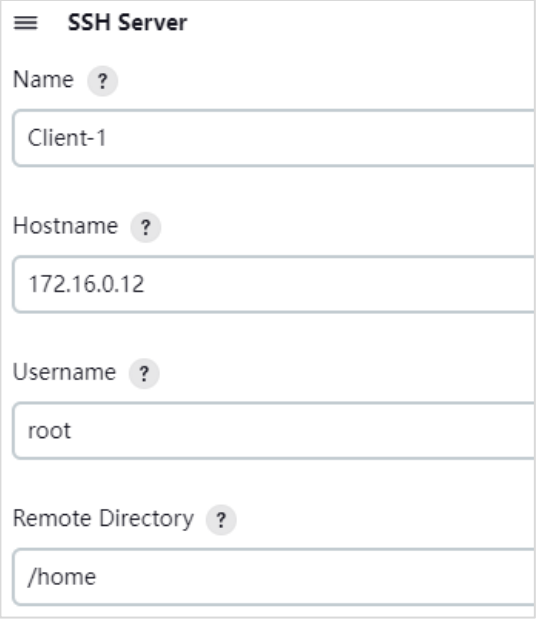
On the Jenkins home page, choose **Manage Jenkins** > **System**.





Drag to the bottom of the new page, find **Publish over SSH**, click **Add**, and enter related information. A host is used in this exercise.



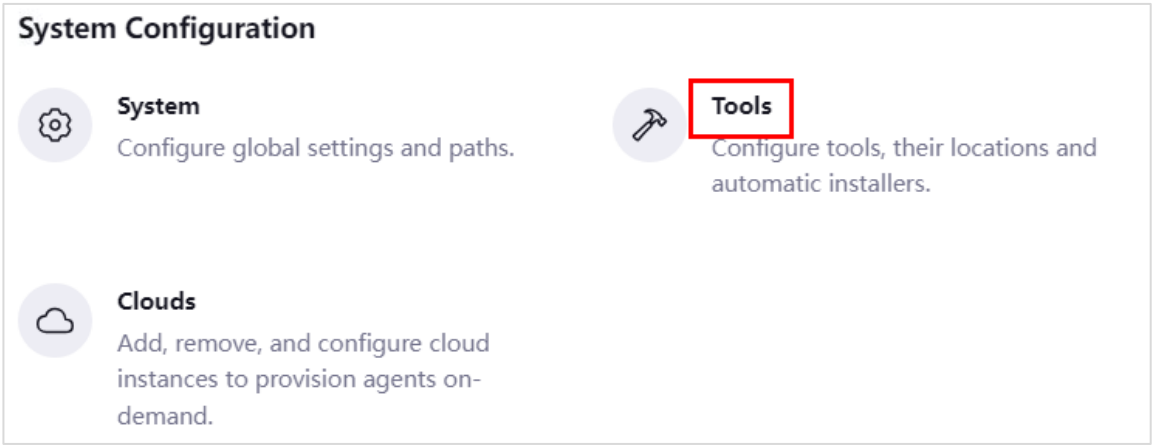


Enter the password of the host user in **Passphrase**. In this exercise, the password of the **root** user is typed.

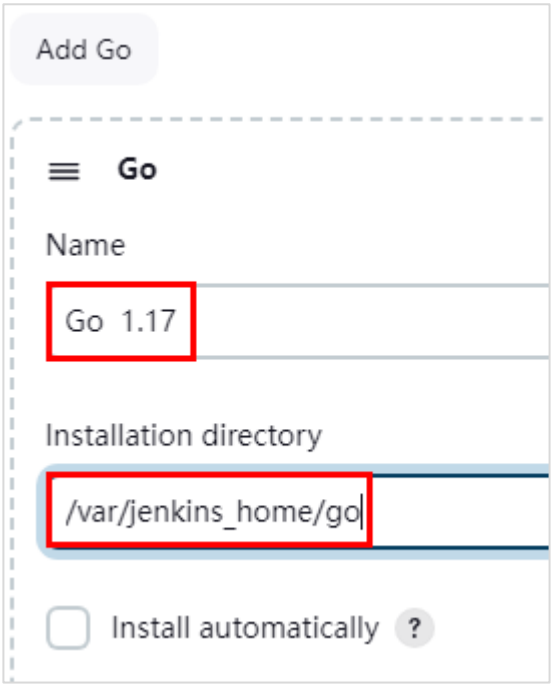
After the configuration is complete, click **Save**.

Step 4 Configure the build environment.

On the Jenkins home page, choose **Manage Jenkins** > **System Configuration** > **Tools**.



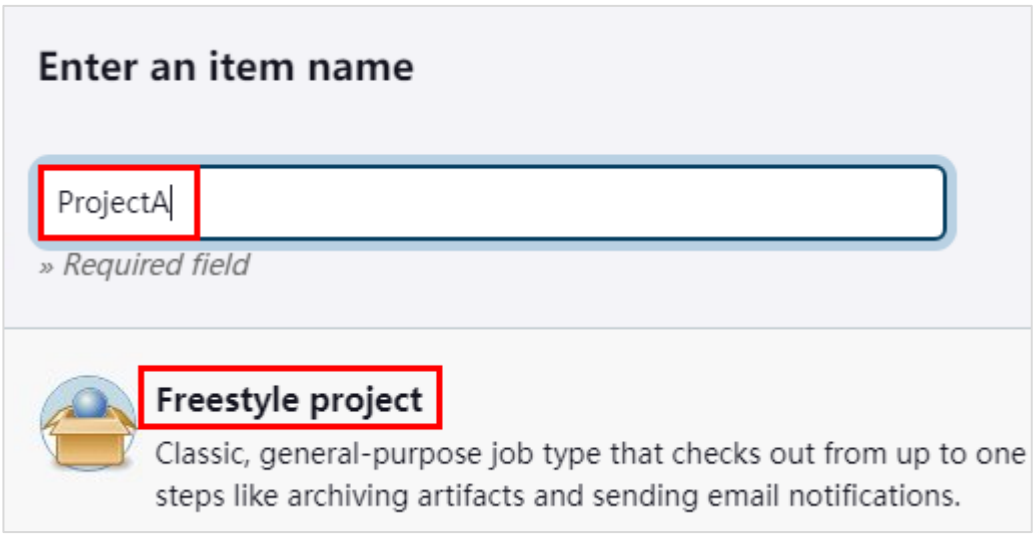
On the displayed page, click **Go Install** at the bottom, set parameters as prompted, and deselect **Automatic Installation**.



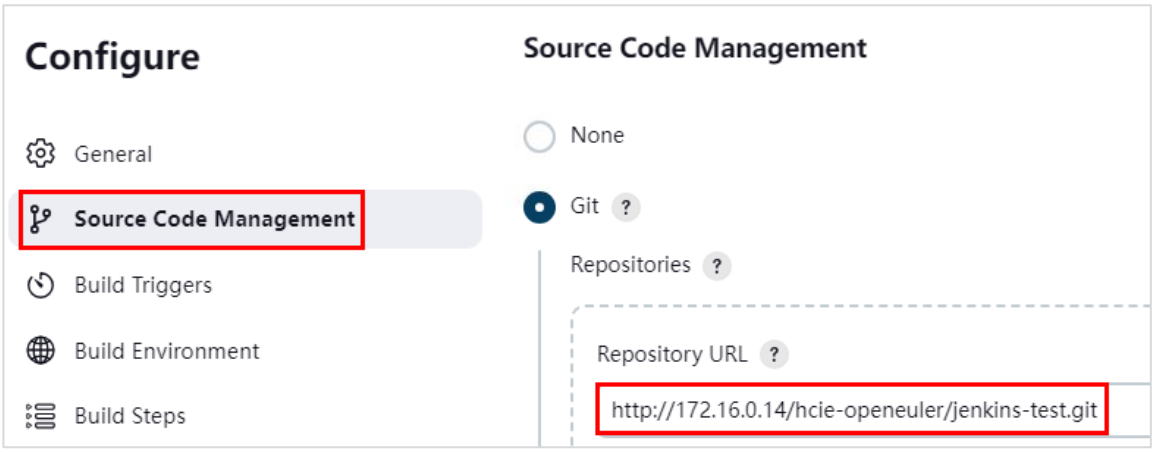
Click **Save**.

Step 5 Create a build task.

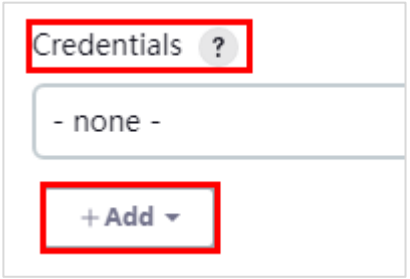
Return to the Jenkins home page and click **New Task**. On the displayed page, enter **ProjectA** and choose **Freestyle project**.



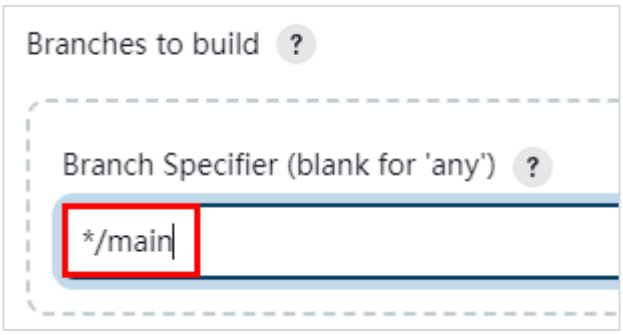
After the configuration is complete, click **OK**. On the displayed page, click **Source Code Management** and enter the code information prepared in section 8.3.1 as prompted, as shown in the following figure.



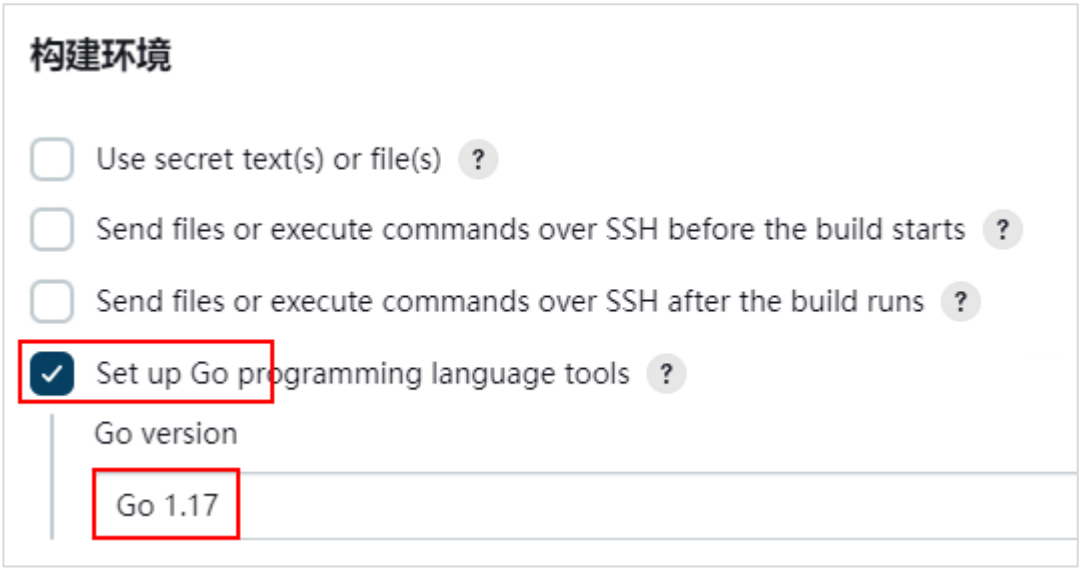
In the **Credentials** area, click **Add**. In the displayed dialog box, enter the GitLab authentication information as prompted.



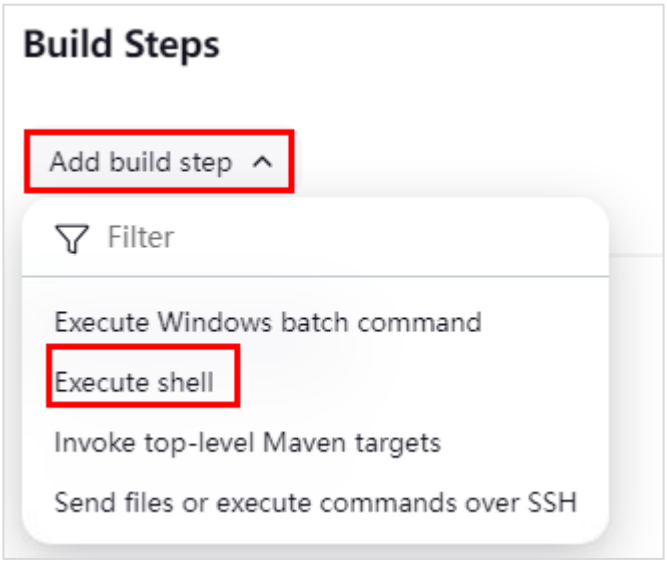
Specify the branch as ***/main**, as shown in the following figure.

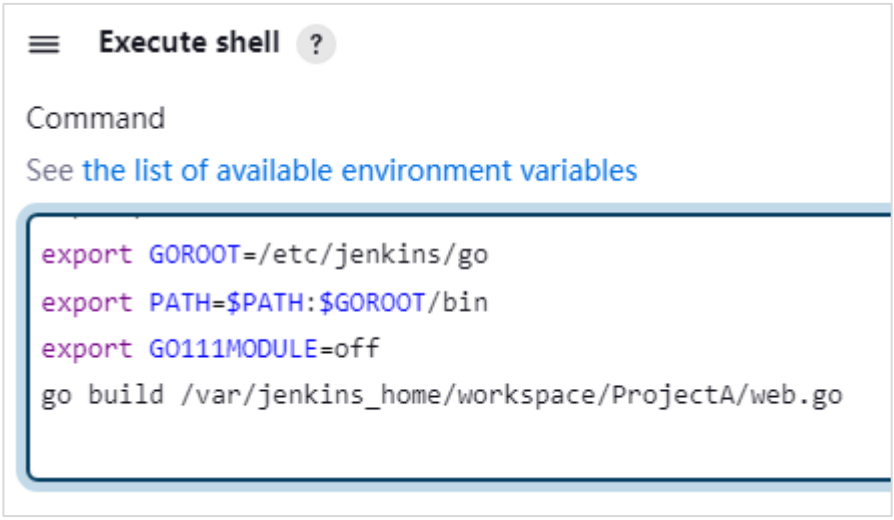


In the **Build Environment** area, select the tool corresponding to Go, as shown in the following figure.



After the configuration is complete, click **Build Steps** in the navigation tree on the left, click **Add build step**, and select **Execute shell**, as shown in the following figure.

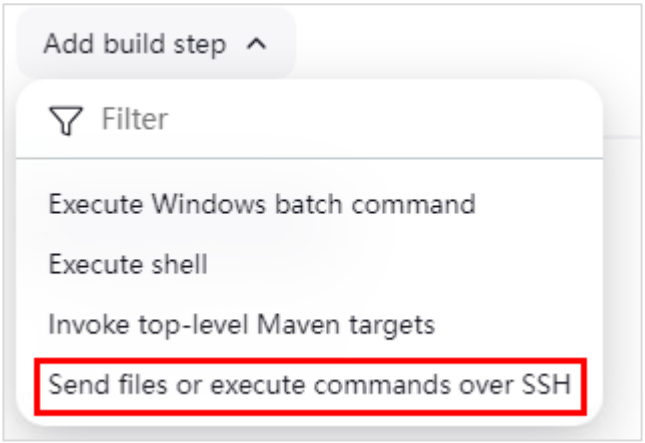


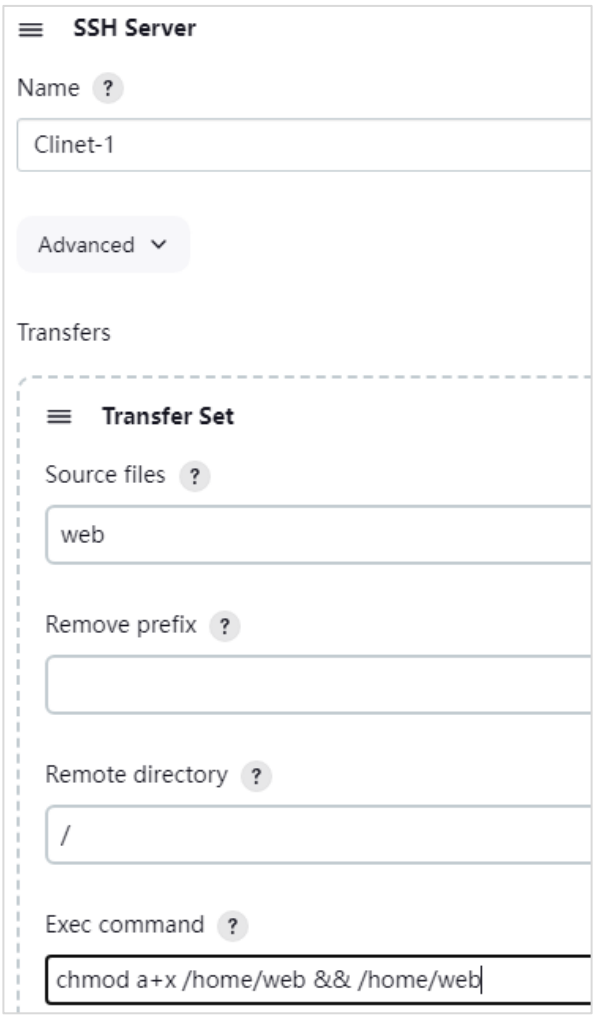


The details are as follows:

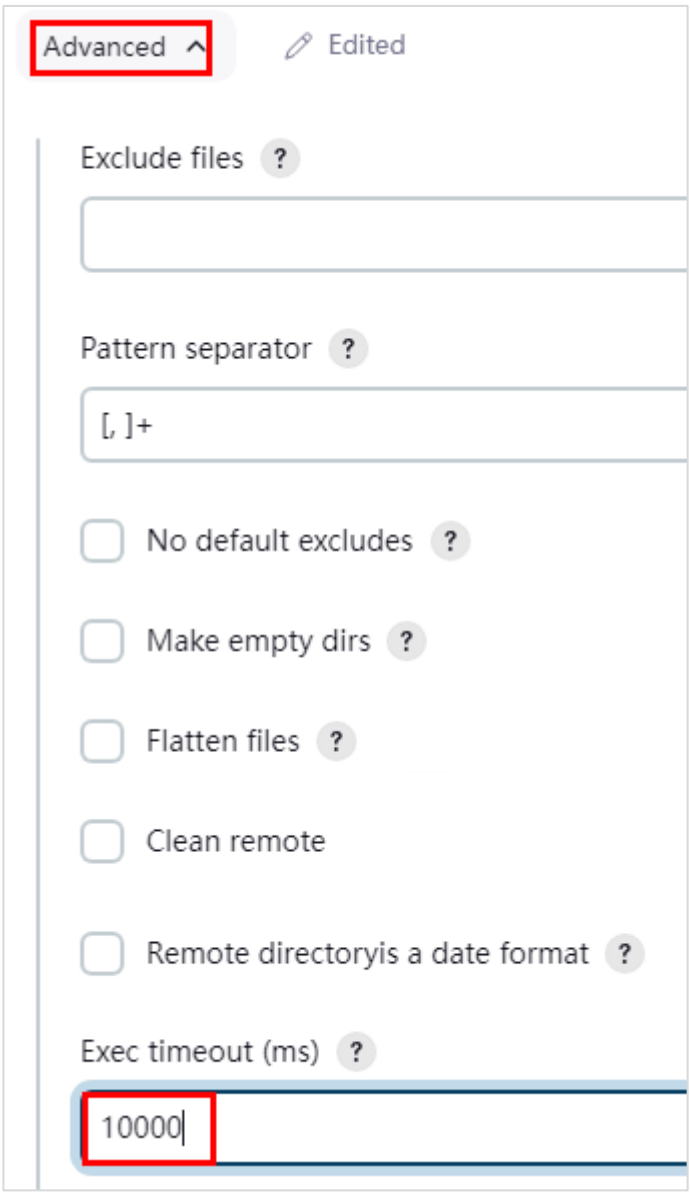
```
#!/bin/bash
export GOROOT=/etc/jenkins/go
export PATH=$PATH:$GOROOT/bin
export GO111MODULE=off
go build /var/jenkins_home/workspace/ProjectA/web.go
```

The compiled program needs to be sent to the client for execution. Therefore, **add Send files or execute commands over SSH** after the build step. The configuration is as follows:

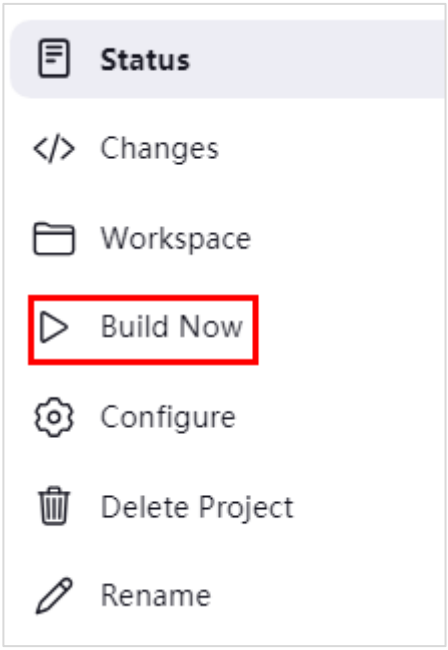




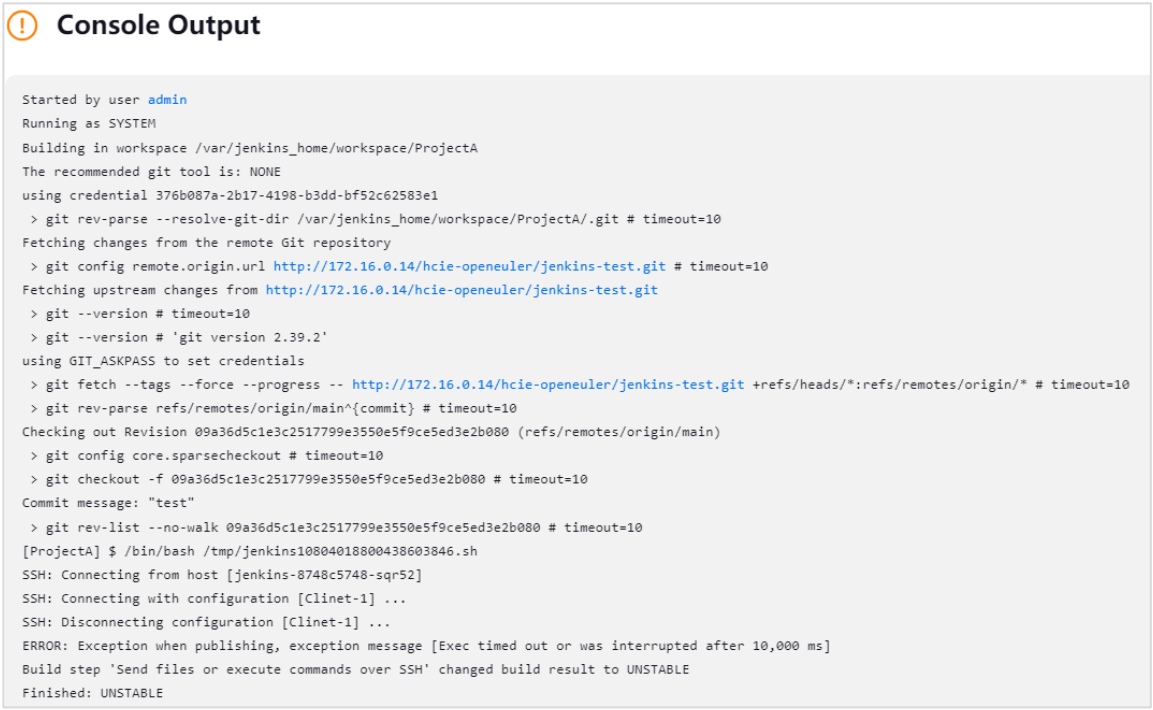
No information is returned when the application is executed over SSH. Therefore, Jenkins does not end the connection until the connection times out and considers build task "UNSTABLE". To accelerate the build time, reduce the timeout period in **Advanced** of **Send files or execute commands over SSH**, as shown in the following figure.



After the configuration is complete, click **Save**. On the displayed page, click **Build Now** and click the number corresponding to build task.



On the displayed page, click **Console Output** to view the build process, as shown in the following figure.



Log in to client-1 and check whether the task is successfully executed, as shown in the following figure.

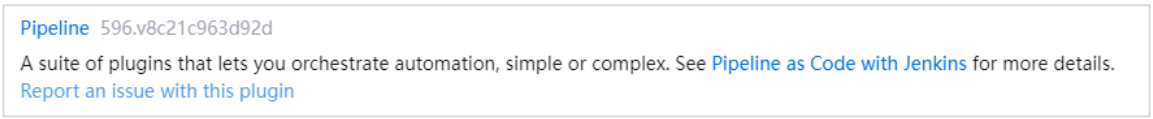


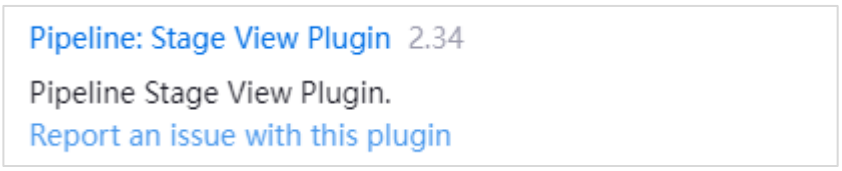
8.3.3 Primary Usage of Jenkins Pipelines

This exercise describes how to implement the content in section 8.3.2 using a pipeline.

Step 1 Install the pipeline plugin.

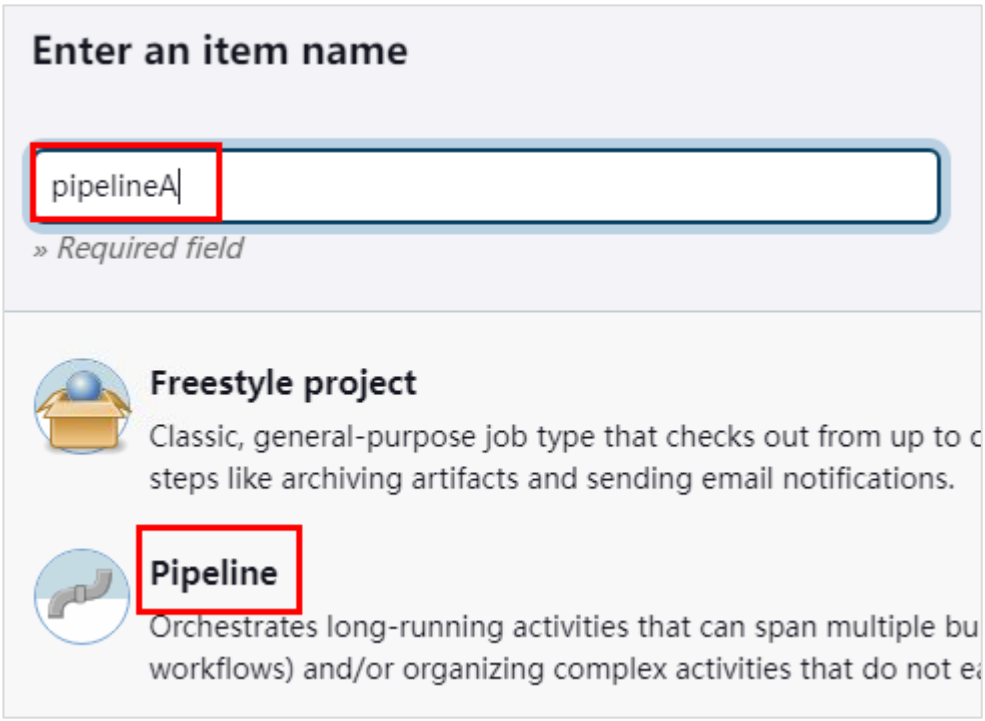
Use the preceding method to install the **Pipeline** and **Pipeline Stage View** plugins.





Step 2 Create a pipeline.

On the Jenkins home page, click **New Task**. On the displayed page, type **pipelineA** and choose **Pipeline**, as shown in the following figure.



Click **OK**.

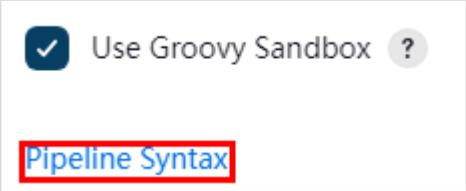
Step 3 Compile a pipeline script.

Go to the pipelineA configuration page, locate where to configure the pipeline, and select **Hello World** on the right to generate the basic syntax format, as shown in the following figure.

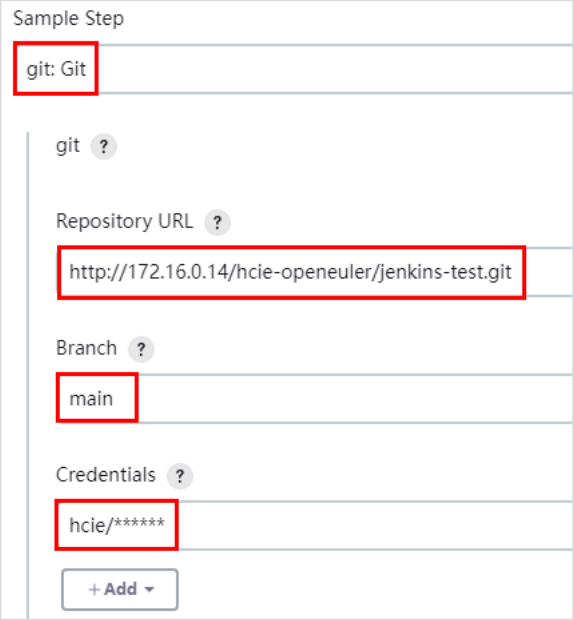


The subsequent script will be modified based on the generated script.

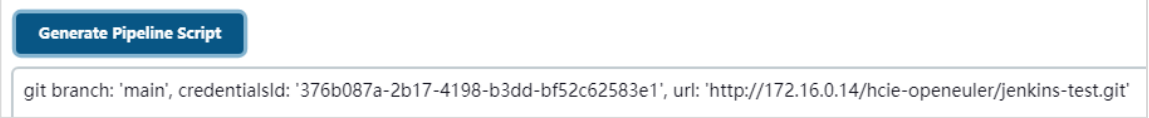
Click **Pipeline Syntax** under the pipeline.



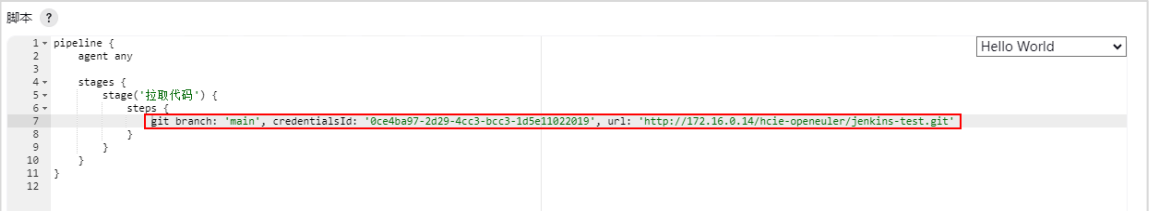
On the new page, generate code based on the information about ProjectA. For example, in the **Source Code Management** area of ProjectA, select **git:Git** and enter the corresponding information, as shown in the following figure.



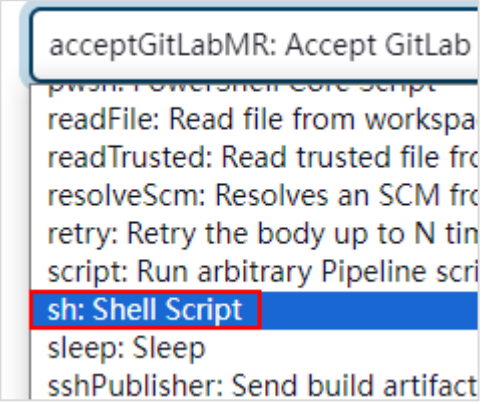
After the configuration is complete, click **Generate Pipeline Script**. The system automatically generates the corresponding script, as shown in the following figure.



Copy the generated script to the corresponding position in the pipeline, as shown in the following figure.



The subsequent steps are basically performed in a shell script. Therefore, the corresponding module is **sh: Shell Script**, as shown in the following figure.



Use the same steps to compile the code. The final code is as follows:

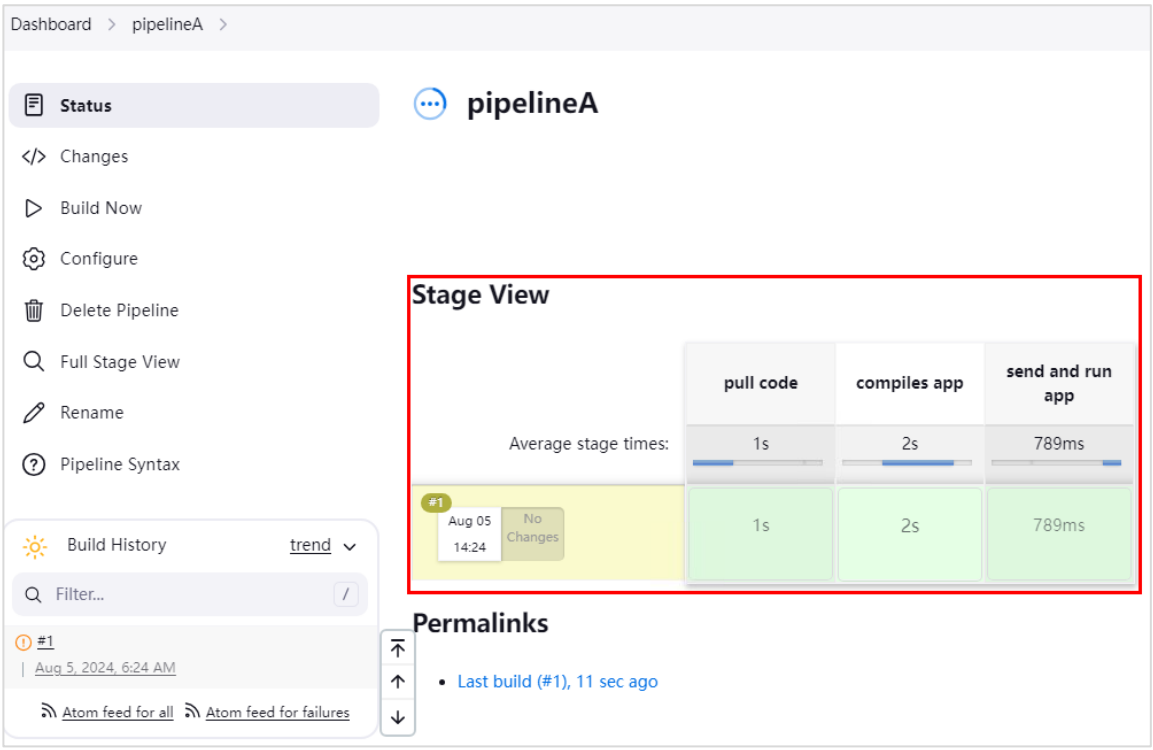
```
pipeline {
    agent any

    stages {
        stage('Pull code.') {
            steps {
                git branch: 'main', credentialsId: '0ce4ba97-2d29-4cc3-bcc3-1d5e11022019', url:
'http://172.16.0.14/hcie-openeuler/jenkins-test.git'
            }
        }
        stage('Compile an application.') {
            steps {
                sh '''#!/bin/bash
                export GOROOT=/etc/jenkins/go
                export PATH=$PATH:$GOROOT/bin
                export GO111MODULE=off
                go build /var/jenkins_home/workspace/ProjectA/web.go'''
            }
        }
        stage('Send and start the application.') {
            steps {
                sshPublisher(publishers: [sshPublisherDesc(configName: 'Client-1', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: '''chmod a+x /home/web
/home/web &''', execTimeout: 10000, flatten: false, makeEmptyDirs: false, noDefaultExcludes: false,
patternSeparator: '[, ]+', remoteDirectory: '/', remoteDirectorySDF: false, removePrefix: '', sourceFiles:
'web')], usePromotionTimestamp: false, useWorkspaceInPromotion: false, verbose: false)])
            }
        }
    }
}
```

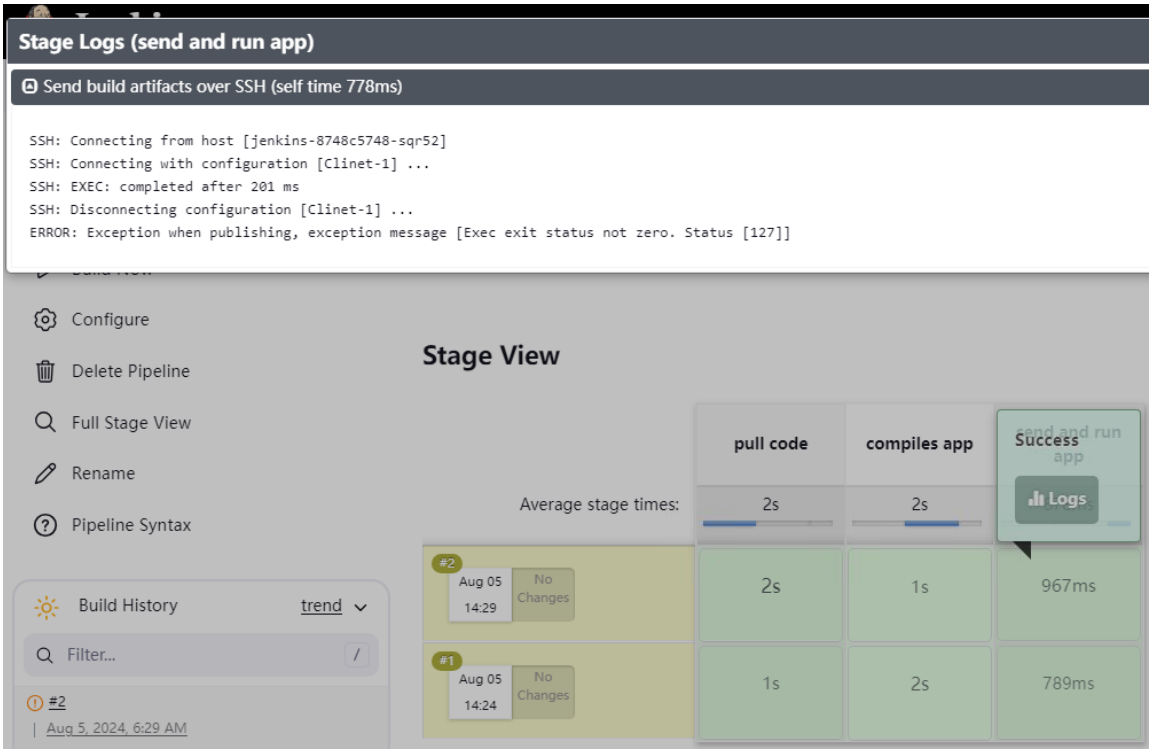
After the configuration is complete, click **Save**.

Step 4 Build an application.

After the configuration is complete, click **Build Now** to execute the task.



Click a phase to view the logs of this phase, as shown in the following figure.



- Question: What are the defects of the application deployment modes of exercises in 8.3.2 and 8.3.3?
- Answer: If the applications in sections 8.3.2 and 8.3.3 are updated, the deployment on the same host will fail after the update. This is because the running old application is not killed before the new application is deployed. As a result, the port is occupied and the new application cannot run properly. This defect will be fixed in subsequent exercise in section 8.4.

8.4 Comprehensive Practices of Jenkins

8.4.1 Comprehensive Practice of Jenkins + GitLab + AWX

Scenario description: This exercise is performed based on sections 7.4 and 8.3.3. In section 8.3.3, an application is developed using Go. Yous can access the application through port 8085 of the host where the application is located. To improve application stability, the application is deployed in a cluster. In addition, the reverse proxy function of Nginx (deployed on one client) is used at the front end to balance load. The Nginx deployment process is implemented by AWX.

Step 1 Updating code.

Modify the source code. The modified code is as follows:

```
package main

import (
	"fmt"
	"net/http"
	"os"
)

func main() {
	http.HandleFunc("/", hostnameHandler)
	http.ListenAndServe(":8085", nil)
}

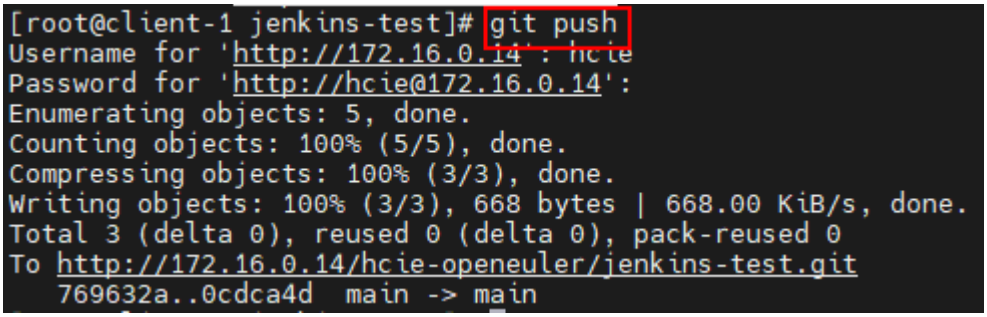
func hostnameHandler(w http.ResponseWriter, r *http.Request) {
	hostname, err := os.Hostname()
	if err != nil {
		fmt.Fprintf(w, "Error getting hostname: %v", err)
	} else {
		fmt.Fprintf(w, "Hostname: %s\n", hostname)
	}
}
```

To ensure that the Go application can be started successfully, create a shell script named **start.sh** for starting the task. The content is as follows:

```
#!/bin/bash
chmod a+x /home/web
```

```
nohup /home/web > /dev/null 2>&1 &
sleep 5
systemctl restart sshd
```

After the modification, upload it to GitLab, as shown in the following figure.



Step 2 Compile a playbook.

Obtain the code created in section 7.4 and add the reverse proxy configuration to the code. The modified playbook content is as follows:

```
---
- hosts: all
  gather_facts: true

  tasks:
    - name: install nginx
      yum:
        name: nginx
        state: present
    - name: proxy config
      template:
        src: ./proxy.conf.j2
        dest: /etc/nginx/conf.d/proxy.conf
      when: ansible_hostname == "client-1"
      notify: restart nginx
    - name: enable nginx
      service:
        name: "{{ item }}"
        state: started
        enabled: yes
      loop:
        - nginx
        - firewalld
    - name: config firewalld
      firewalld:
        port: "{{ item }}"
        permanent: yes
        immediate: yes
        state: enabled
      loop:
        - 80/tcp
        - 8085/tcp

  handlers:
```

```
    - name: restart nginx
      service:
        name: nginx
        state: restarted
```

Compile the configuration file required by the reverse proxy as follows:

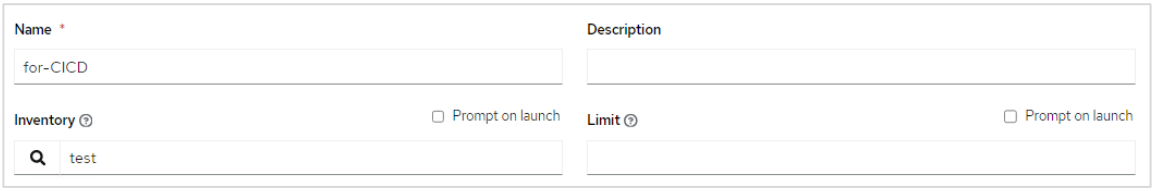
```
upstream backend {
    server {{ ansible_play_hosts[0] }}:8085;
    server {{ ansible_play_hosts[1] }}:8085;
}
server {
    location / {
        proxy_pass http://backend;
    }
}
```

Upload the modified and newly created files to GitLab, as shown in the following figure.

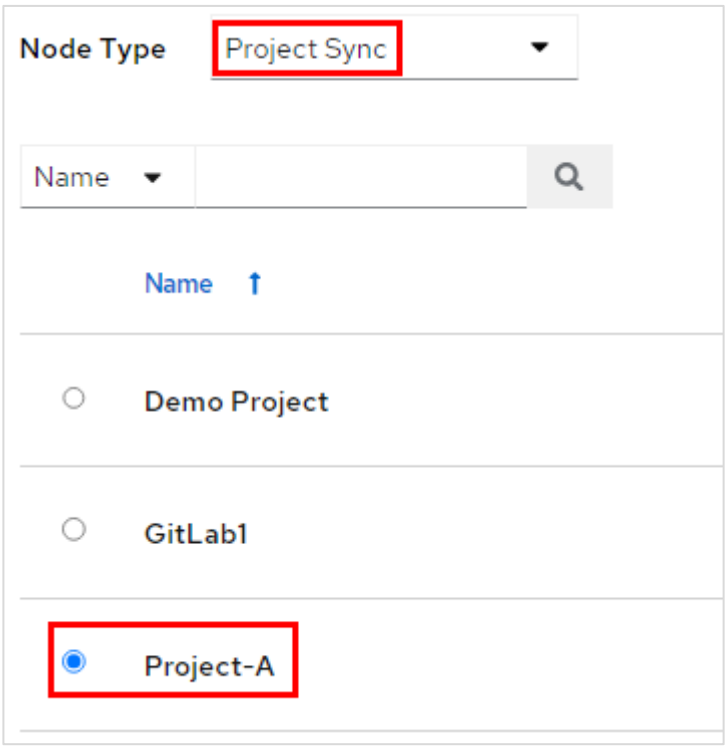
```
[root@client-1 applicationa]# git add .
[root@client-1 applicationa]# git commit -m "2.5"
[main 3b465a5] 2.5
 2 files changed, 1 insertion(+), 1 deletion(-)
[root@client-1 applicationa]# git push
Username for 'http://172.16.0.14': hcie
Password for 'http://hcie@172.16.0.14':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 404 bytes | 404.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
To http://172.16.0.14/hcie-openeuler/applicationa.git
   47fae53..3b465a5  main -> main
```

Step 3 Configure AWX and obtain corresponding API.

Create a workflow template named **for-CI/CD** based on the existing configuration. The configuration is as follows:

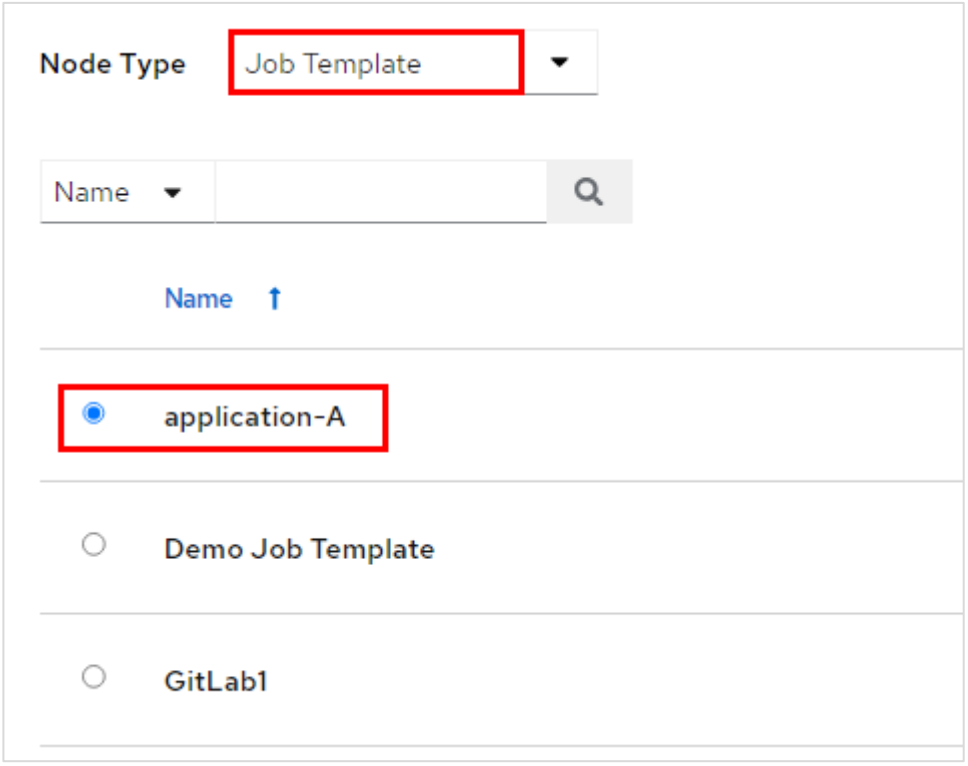


Add a specific task for the workflow template. The first node type is **Project Synchronization**, specifically **Project-A**.



- Note: **Project-A** is generated in Step 6 in section 7.4 and is used to update the code of repository (http://172.16.0.14/hcie-openeuler/applicationa.git). Check whether it meets the requirements.

The second node type is **Task Template**, specifically **application-A**. It is executed after the first node is executed successfully.



- Note: **application-A** is created in Step 7 in section 7.4. It uses **Project-A** and is also used to update the code of repository (http://172.16.0.14/hcie-openeuler/applicationa.git). Check whether it meets the requirements.

After the configuration is complete, run the following commands to obtain the execution API address corresponding to the workflow template:

```
[root@router ~]# curl -X GET --user admin:7yv37sFLaj3EPQ00LgwuT3sOoVQLkCgg http://www.openeuler-hcie.com:32453/awx/api/v2/workflow_job_templates/ -s | json_pp | grep -A 10 CI/CD | grep launch
                "launch" : "/api/v2/workflow_job_templates/14/launch/",
```

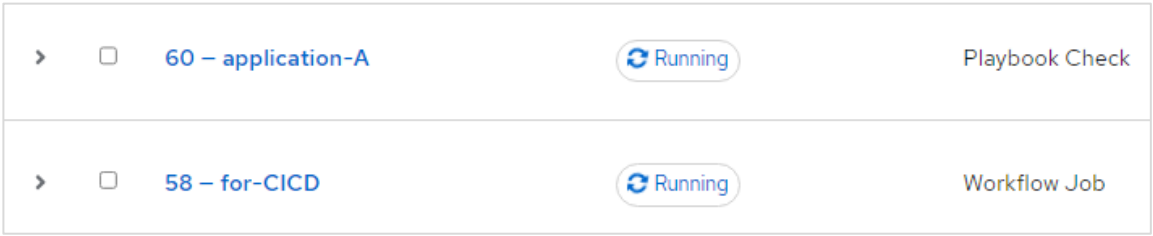


Run the following command to check whether the **for-CI/CD** workflow can be successfully executed using the API address:

```
[root@router ~]# curl -X POST --user admin:7yv37sFLaj3EPQ00LgwuT3sOoVQLkCgg http://www.openeuler-hcie.com:32453/awx/api/v2/workflow_job_templates/14/launch/ -s | json_pp
```



Log in to AWX to check whether the workflow is started successfully, as shown in the following figure.



If the workflow can be successfully completed, go to the next step.

Step 4 Add a host.

This exercise is performed on two hosts. Therefore, add client-2 by referring to step 3 in section 8.3.2.

Step 5 Roll out the Jenkins deployment service.

Create a new pipeline on Jenkins, name it for-CI/CD, and input the following pipeline script:

```
pipeline {
    agent any

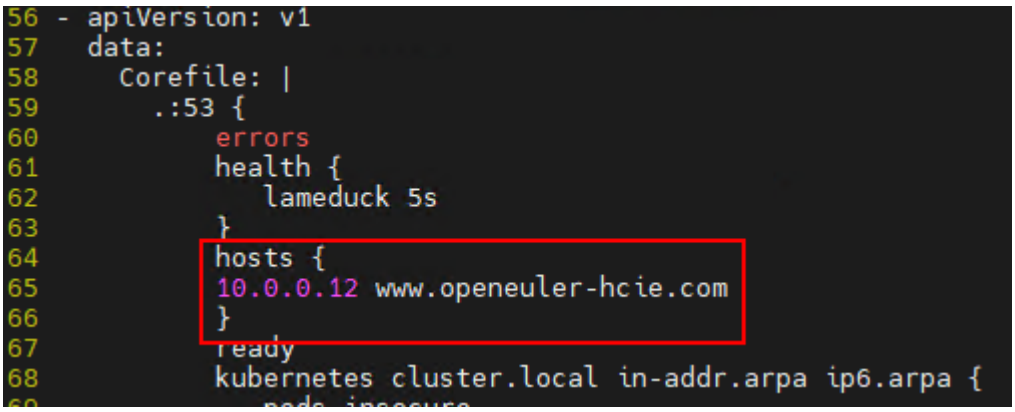
    stages {
        stage('Pull code.') {
            steps {
                git branch: 'main', credentialsId: '0ce4ba97-2d29-4cc3-bcc3-1d5e11022019', url:
'http://172.16.0.14/hcie-openeuler/jenkins-test.git'
            }
        }
        stage('Compile the application.') {
            steps {
                sh '''#!/bin/bash
                export GOROOT=/etc/jenkins/go
                export PATH=$PATH:$GOROOT/bin
                export GO111MODULE=off
                go build /var/jenkins_home/workspace/for-CI/CD/web.go'''
            }
        }
        stage ('Delete old application') {
            steps {
                sshPublisher(publishers: [sshPublisherDesc(configName: 'Client-1', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: 'rm -rf /home/web', execTimeout:
120000, flatten: false, makeEmptyDirs: false, noDefaultExcludes: false, patternSeparator: '[, ]+',
remoteDirectory: '', remoteDirectorySDF: false, removePrefix: '', sourceFiles: '')],
usePromotionTimestamp: false, useWorkspaceInPromotion: false, verbose: false),
sshPublisherDesc(configName: 'Client-2', transfers: [sshTransfer(cleanRemote: false, excludes: '',
execCommand: 'rm -rf /home/web', execTimeout: 120000, flatten: false, makeEmptyDirs: false,
noDefaultExcludes: false, patternSeparator: '[, ]+', remoteDirectory: '', remoteDirectorySDF: false,
removePrefix: '', sourceFiles: '')], usePromotionTimestamp: false, useWorkspaceInPromotion: false,
verbose: false)])
            }
        }
        stage('Stop the running application.') {
            steps {
                sshPublisher(publishers: [sshPublisherDesc(configName: 'Client-1', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: 'kill -9 $(ps -aux | grep web | awk \'{print
$2}\' | head -n 1)', execTimeout: 120000, flatten: false, makeEmptyDirs: false, noDefaultExcludes:
false, patternSeparator: '[, ]+', remoteDirectory: '', remoteDirectorySDF: false, removePrefix: '',
sourceFiles: '')], usePromotionTimestamp: false, useWorkspaceInPromotion: false, verbose: false),
sshPublisherDesc(configName: 'Client-2', transfers: [sshTransfer(cleanRemote: false, excludes: '',
execCommand: 'kill -9 $(ps -aux | grep web | awk \'{print $2}\' | head -n 1)', execTimeout: 120000,
flatten: false, makeEmptyDirs: false, noDefaultExcludes: false, patternSeparator: '[, ]+',
remoteDirectory: '', remoteDirectorySDF: false, removePrefix: '', sourceFiles: '')],
usePromotionTimestamp: false, useWorkspaceInPromotion: false, verbose: false)])
            }
        }
        stage('Send the application.') {
            steps {
                sshPublisher(publishers: [sshPublisherDesc(configName: 'Client-1', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: '', execTimeout: 120000, flatten: false,
makeEmptyDirs: false, noDefaultExcludes: false, patternSeparator: '[, ]+', remoteDirectory: '/',
remoteDirectorySDF: false, removePrefix: '', sourceFiles: 'web')], usePromotionTimestamp: false,
useWorkspaceInPromotion: false, verbose: false), sshPublisherDesc(configName: 'Client-2', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: '', execTimeout: 120000, flatten: false,
```

```
makeEmptyDirs: false, noDefaultExcludes: false, patternSeparator: '[, ]+', remoteDirectory: '/',
remoteDirectorySDF: false, removePrefix: '', sourceFiles: 'web')], usePromotionTimestamp: false,
useWorkspaceInPromotion: false, verbose: false)])
                }
            }
            stage('Send the startup script.') {
                steps {
                    sshPublisher(publishers: [sshPublisherDesc(configName: 'Client-1', transfers:
[sshTransfer(cleanRemote: false, excludes: '', execCommand: 'sh /home/start.sh', execTimeout:
120000, flatten: false, makeEmptyDirs: false, noDefaultExcludes: false, patternSeparator: '[, ]+',
remoteDirectory: '/', remoteDirectorySDF: false, removePrefix: '', sourceFiles: 'start.sh')],
usePromotionTimestamp: false, useWorkspaceInPromotion: false, verbose: false),
sshPublisherDesc(configName: 'Client-2', transfers: [sshTransfer(cleanRemote: false, excludes: '',
execCommand: 'sh /home/start.sh', execTimeout: 120000, flatten: false, makeEmptyDirs: false,
noDefaultExcludes: false, patternSeparator: '[, ]+', remoteDirectory: '/', remoteDirectorySDF: false,
removePrefix: '', sourceFiles: 'start.sh')], usePromotionTimestamp: false, useWorkspaceInPromotion:
false, verbose: false)])
                }
            }
            stage('Call AWX') {
                steps {
                    sh 'curl -X POST --user admin:7yv37sFLaj3EPQ00LgwuT3sOoVQLkCgg
http://www.openeuler-hcie.com:32453/awx/api/v2/workflow_job_templates/14/launch/'
                }
            }
    }
}
```

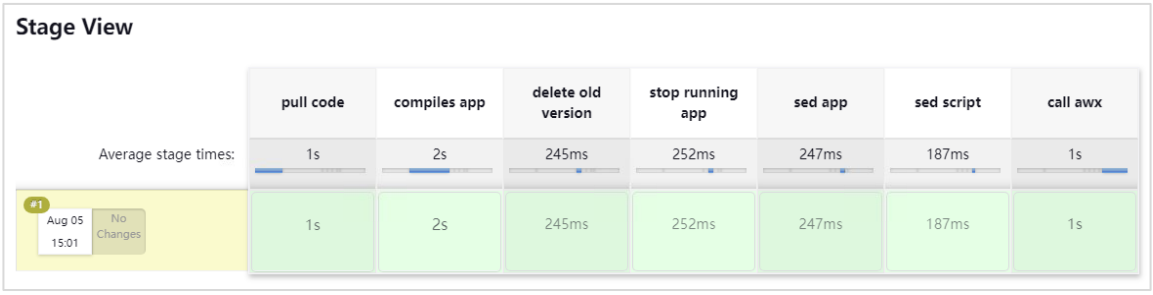
After the configuration is complete, click **Save** and execute the pipeline.

- Question: Is the pipeline successfully executed? Why?
- Answer: If the execution fails, Jenkins cannot resolve the AWX domain name. As a result, the corresponding API cannot be called. In this case, you can add DNS servers or manually add domain name entries on Kubernetes. The following describes how to manually add domain name entries on Kubernetes:

Run the **kubectl edit cm -n kube-system** command to edit the **ConfigMap** file, add the content in the red box shown in the following figure, save the file, and exit.



After the modification is complete, perform the task build again.



Step 6 Conduct a test.

Conduct a test on the client to check whether the requirements are met. If yes, the information similar to the following figure is displayed.

```
[root@router ~]# curl 172.16.0.12
Hostname: client-2
[root@router ~]# curl 172.16.0.12
Hostname: client-1
[root@router ~]# curl 172.16.0.12
Hostname: client-2
[root@router ~]# curl 172.16.0.12
Hostname: client-1
```

8.4.2 Dynamic Slave Implemented on Jenkins + Harbor + Kubernetes

Scenario description: Use the same code as that in section 8.4.1 to develop an application, deploy the developed application in Kubernetes, and release the application using the service and Ingress. This exercise is implemented based on the dynamic slave of Jenkins.

Step 1 Install Harbor.

In this exercise, Harbor is deployed in the same ECS as GitLab through containerized deployment. Run the **git clone https://gitee.com/yftyxa/harbor.git** command to obtain the online installation package of Harbor and decompress the .tar package.

```
git clone https://gitee.com/yftyxa/harbor.git
cd harbor
tar -xzf harbor-online-installer-v2.9.1.tgz
```

Go to the directory generated after the decompression, copy **harbor.yml.tmpl** as **harbor.yml**, and modify it as follows:

```
hostname: 172.16.0.14

# http related config
http:
  # port for http, default is 80. If https ena
  port: 8088

# https related config
#https:
#  # https port for harbor, default is 443
#  port: 443
#  # The path of cert and key files for nginx
#  certificate: /your/certificate/path
#  private_key: /your/private/key/path
```

Change **hostname** to the IP address of the local host, comment out all HTTPS-related configurations, and change the HTTP port number of Harbor to 8088 because GitLab has occupied port 80. Save the configuration and exit.

Run the following command to install the Harbor dependency:

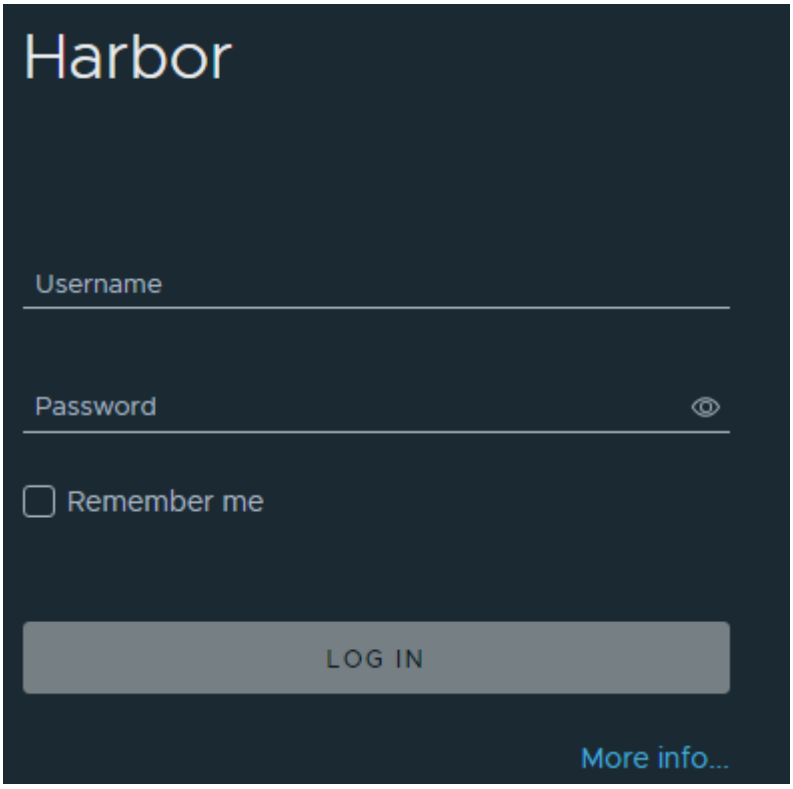
```
yum install -y docker-compose-1.22.0-4.oe2203.noarch
```

After the installation is complete, run the **sh install.sh** command to deploy Harbor. Wait until all containers are created, as shown in the following figure.

```
Status: Downloaded newer image for goharbor/nginx-photon:v2.9.1
Creating harbor-log ... done
Creating harbor-db      ... done
Creating harbor-portal ... done
Creating registry       ... done
Creating registryctl    ... done
Creating redis          ... done
Creating harbor-core    ... done
Creating nginx              ... done
Creating harbor-jobservice ... done
✔ ----Harbor has been installed and started successfully.----
```

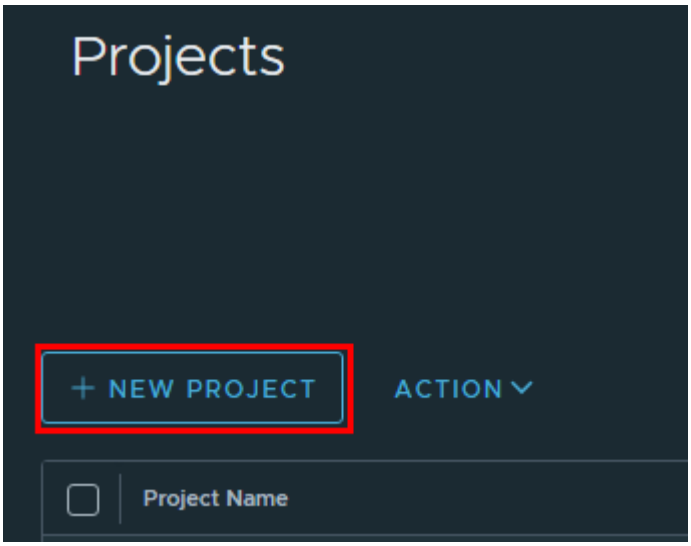
Step 2 Configure Harbor.

Use the EIP corresponding to the ECS and port 8088 to log in to the Harbor server, as shown in the following figure.

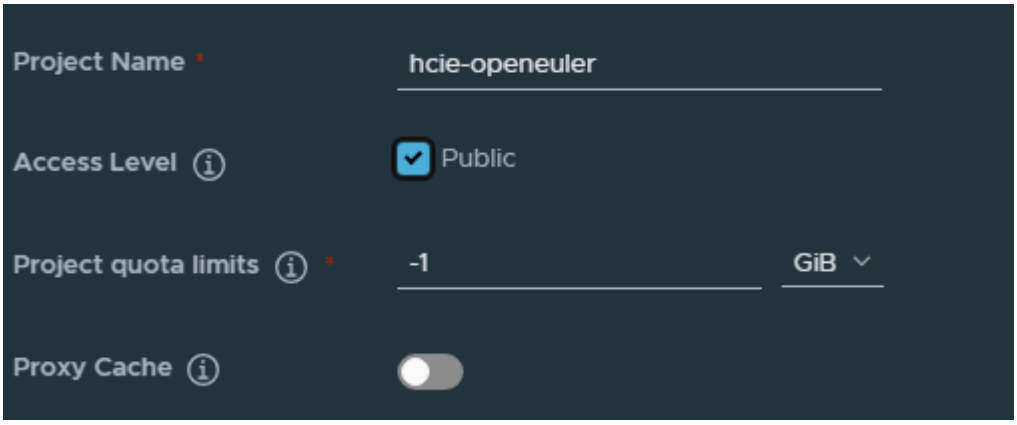


The default password is specified in the configuration file and can be changed based on the actual situation.

After logging in to Harbor, click **NEW PROJECT**.



In the displayed dialog box, enter the project name, set it to **Public**, and retain the default settings of other options, as shown in the following figure.



Step 3 Interconnect Kubernetes with Harbor.

In this exercise, Docker is used to run containers. By default, Docker uses HTTPS to connect to the image repository. Therefore, Kubernetes uses the same method to pull images. To enable Kubernetes and Docker to use the image repository created and configured in Step 1 and Step 2 in HTTP mode, add the following configuration to the Docker configuration file **/etc/docker/daemon.josn** on all Kubernetes nodes:

```
"insecure-registries": ["172.16.0.14:8088"]
```

See the following figure.

```
[root@k8smaster1 test]# cat /etc/docker/daemon.json
{
  "exec-opts": ["native.cgroupdriver=systemd"],
  "registry-mirrors": ["https://a105859804e5472ab0d034f317bc1509.mirror.swr.myhuaweicloud.com"],
  "insecure-registries": ["172.16.0.14:8088"]
}
[root@k8smaster1 test]#
```

After the configuration is complete, restart Docker for the configuration to take effect.

Step 4 Prepare a base image.

In this exercise, openeuler 22.03 is used as the base image. Therefore, run the following command to pull the image from public images:

```
docker pull swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/openeuler:22.03lts
```

After that, run the following command to label the image:

```
docker tag swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/openeuler:22.03lts
172.16.0.14:8088/hcie-openeuler/openeuler:22.03lts
```

Push the image to the Harbor repository set up in Step 1. The Harbor repository is set up in HTTP mode, but Docker requires HTTPS by default. Therefore, you need to add the following information to the Docker configuration file **/etc/docker/daemon.json**:

```
{
        "insecure-registries": ["172.16.0.14:8088"]
}
```

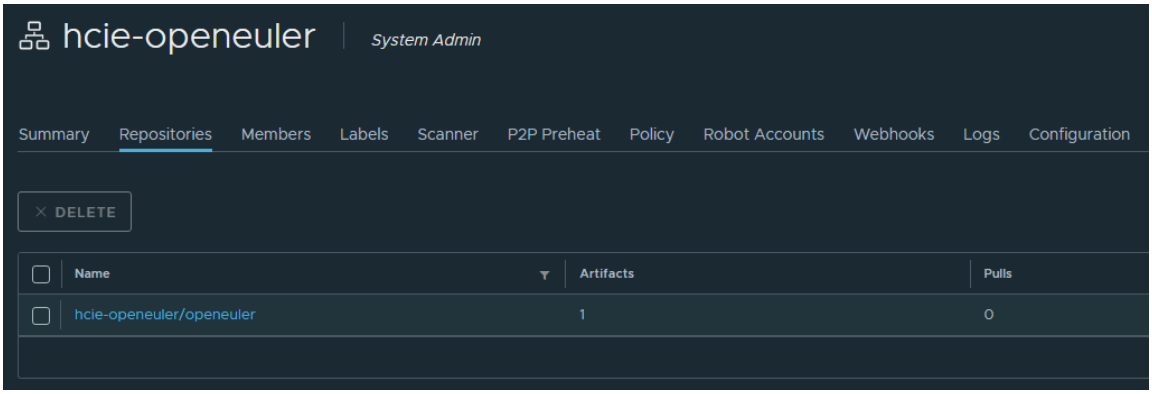
Run the following commands to reload Docker:

```
systemctl daemon-reload
systemctl restart docker
```

After reloading Docker, log in to the Harbor server and run the following commands to push the image to the Harbor server:

```
docker-compose restart
docker login    -u admin -p Harbor12345 172.16.0.14:8088
docker push 172.16.0.14:8088/hcie-openeuler/openeuler:22.03lts
```

After the upload is complete, you can view the pushed image on the Harbor page, as shown in the following figure.



Step 5 Prepare resources.

Add the Dockerfile that will be used later to the code repository. The content is as follows:

```
FROM 172.16.0.14:8088/hcie-openeuler/openeuler:22.03lts
COPY web .
RUN chmod a+x web
EXPOSE 8085/tcp
CMD ./web
```

Add a YAML file that will be used to create various objects to the code repository. The content is as follows:

```
#####Create a namespace.
apiVersion: v1
kind: Namespace
metadata:
  name: jenkins

---
#####Create a deployment.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: applicationa
```

```
  namespace: jenkins
  labels:
    app: applicationa
spec:
  replicas: 3
  selector:
    matchLabels:
      app: applicationa
  template:
    metadata:
      labels:
        app: applicationa
    spec:
      containers:
      - image: ${harborHost}/${harborRepo}/test:${tag}
        name: applicationa
        imagePullPolicy: Always

---
#####Create a service.
apiVersion: v1
kind: Service
metadata:
  name: applicationa
  namespace: jenkins
  annotations:
      prometheus.io/scrape: 'true'
      prometheus.io/path:     /
      prometheus.io/port:     '8080'
spec:
  selector:
    app: applicationa
  type: NodePort
  ports:
    - port: 8085
      targetPort: 8085
      nodePort: 32010
---
#####Create Ingress.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: applicationa
  namespace: jenkins
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
 rules:
   - host: www.openeuler-hcie.com
     http:
       paths:
         - backend:
             service:
```

```
            name: applicationa
            port:
              number: 8080
      path: /for-k8s(/|$)(.*)
      pathType: Prefix
```

After the compilation is complete, push all configuration files to the code repository http://172.16.0.14/hcie-openeuler/jenkins-test.git.

Push the Kubernetes authentication file **.kube/config** to the code repository. The final files in the code repository are as follows:



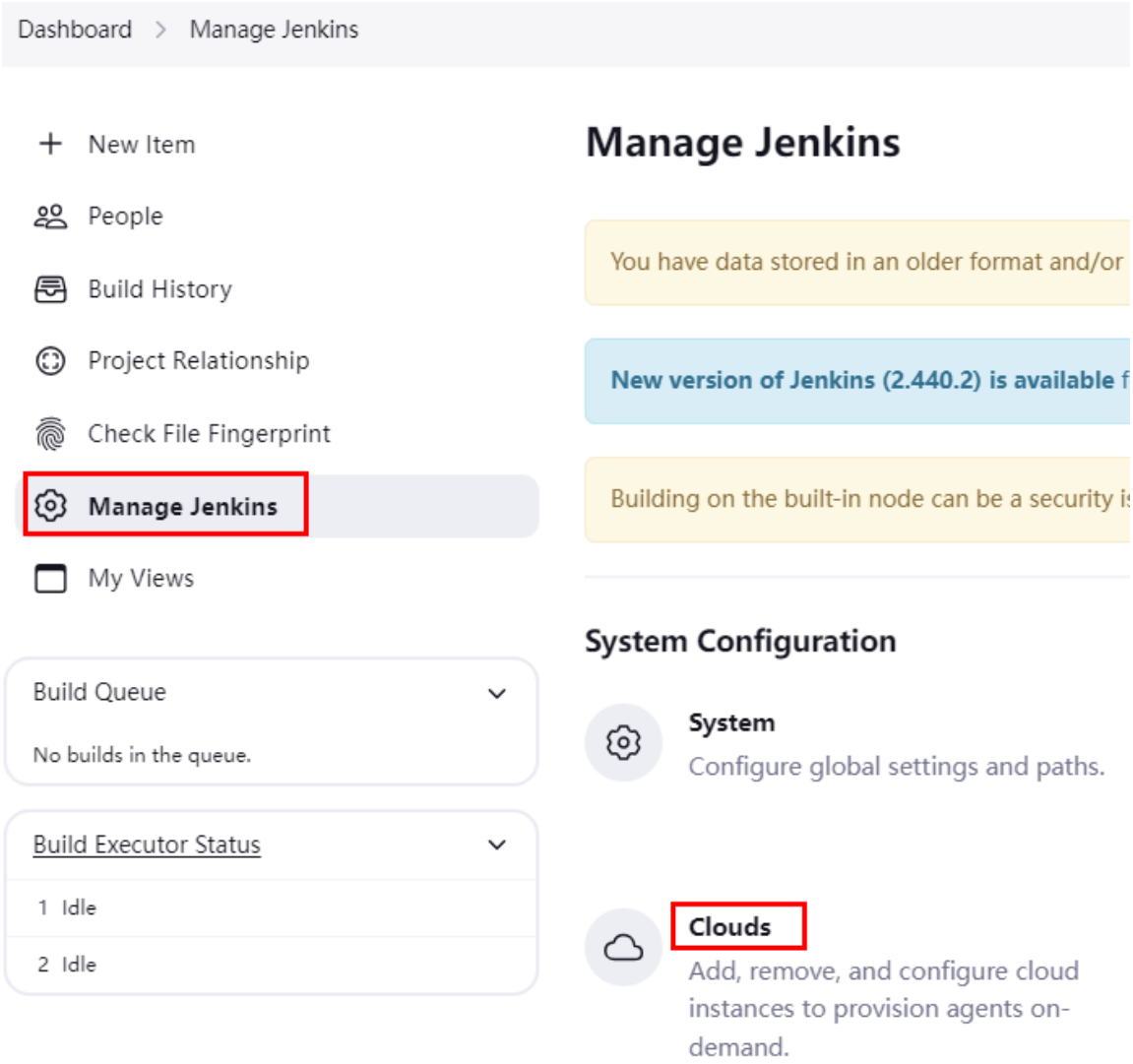
config is the Kubernetes authentication file, Dockerfile is used to generate the container image of the application, and **deploy.yml** is the application deployment file.

Step 6 Install plugins.

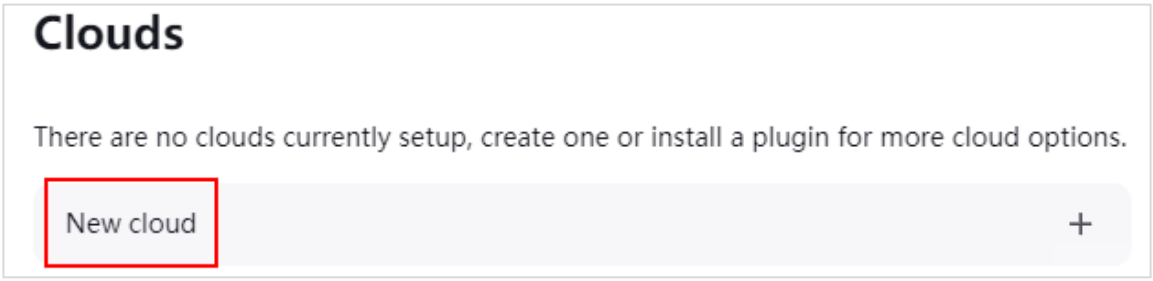
Install the Git Parameter and Kubernetes plugins and restart Jenkins for the plugins to take effect.

Step 7 Add a Kubernetes cluster.

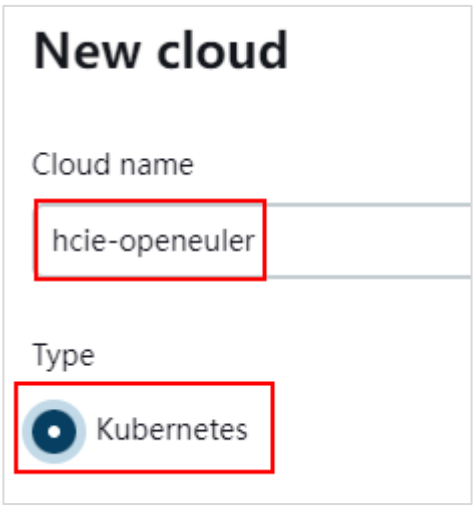
After the Kubernetes plugin is installed, click **Clouds** on the **Manage Jenkins** page to add a Kubernetes cluster, as shown in the following figure.



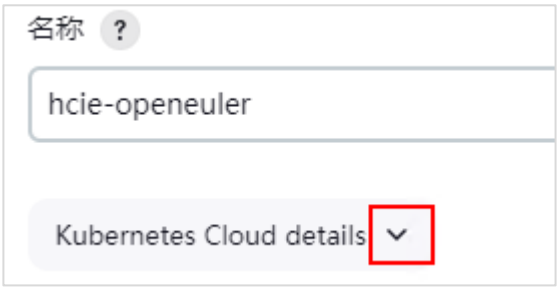
On the displayed page, click **New cloud**, as shown in the following figure.



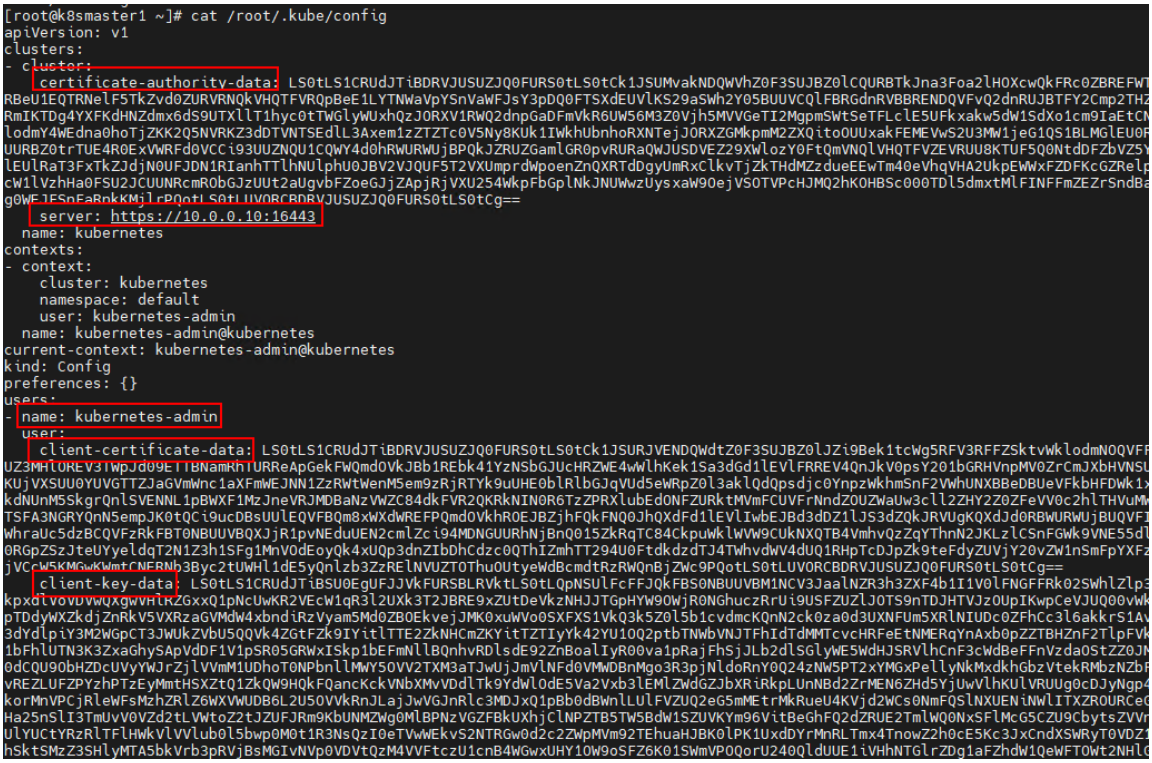
On the displayed page, enter a cloud name and set **Type** to **Kubernetes**, as shown in the following figure.



Click **Create**. On the displayed page, expand **Kubernetes Cloud detail** to supplement details, as shown in the following figure.



You need to set Kubernetes address and Kubernetes service certificate key. The corresponding parameters can be obtained by running **cat /root/.kube/config** on the Kubernetes control node, as shown in the following figure.



Save the content of **certificate-authority-data** to the **certificate-authority-data** file, the content of **client-certificate-data** to the **client-certificate-data** file, and the content of **client-key-data** to the **client-key-data** file. Run the following commands to convert these files to certificate files encrypted using Base64:

```
cat certificate-authority-data | base64 -d > ca.crt
cat client-certificate-data | base64 -d > client.crt
cat client-key-data | base64 -d > client.key
```

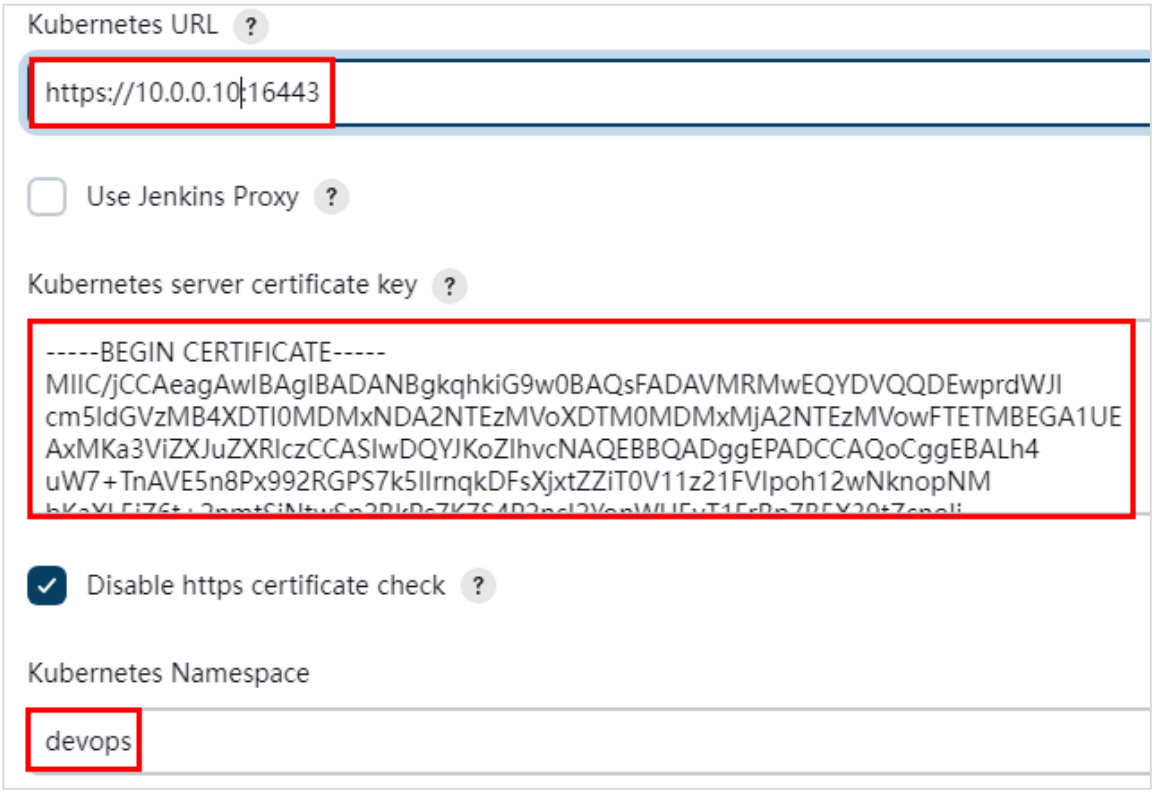
```
[root@k8smaster1 .kube]# vim certificate-authority-data
[root@k8smaster1 .kube]# vim client-certificate-data
[root@k8smaster1 .kube]# vim client-key-data
[root@k8smaster1 .kube]# cat certificate-authority-data | base64 -d > ca.crt
[root@k8smaster1 .kube]# cat client-certificate-data | base64 -d > client.crt
[root@k8smaster1 .kube]# cat client-key-data | base64 -d > client.key
[root@k8smaster1 .kube]#
```

Run the following command to generate the Client P12 authentication file **cert.pfx** using the three certificate files:

```
openssl pkcs12 -export -out cert.pfx -inkey client.key -in client.crt -certfile ca.crt
```

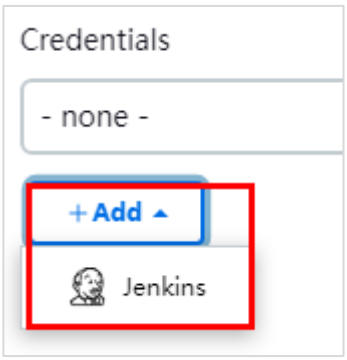
You need to enter the password when generating the authentication file. Do not lose the password because it will be used later.

When configuring **Kubernetes Cloud detail**, enter the Kubernetes URL, Kubernetes service certificate key, and Kubernetes namespace. The details are as follows:



The Kubernetes service certificate key is the content of **ca.crt**.

In the **Credentials** area, click **Add**.



The details of the added credential are as follows:

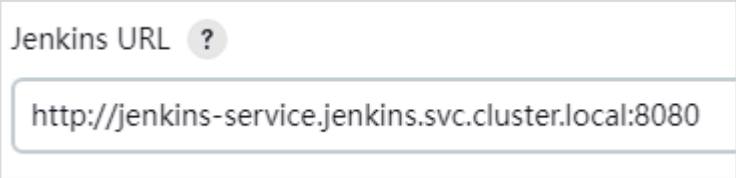


Click **Save** and select the created credential from the **Credentials** drop-down list, as shown in the following figure.

Ensure that the information is correct and click **Test Connection**. See the following figure.



In the **Jenkins URL** text box, enter the service domain name and port number of Jenkins, as shown in the following figure.



Step 8 Customize containers.

In this exercise, two containers are customized. One is used to compile the Go application, and the other is used to create the container image for running the Go program and create related Kubernetes resource objects.

Use the following Dockerfile to create the first image:

```
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/golang:1.17.3

RUN mkdir /home/jenkins \
      && useradd -u 1000 -d /home/jenkins jenkins \
      && chown 1000:1000 /home/jenkins
USER Jenkins
```

After the compilation is complete and the image is generated, push it to the private image repository installed in Step 1.

```
docker build -t 172.16.0.14:8088/hcie-openeuler/golang:1.17.3 .
docker push 172.16.0.14:8088/hcie-openeuler/golang:1.17.3
```

Use the following Dockerfile to create the second image:

```
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/jenkins-docker:1.0

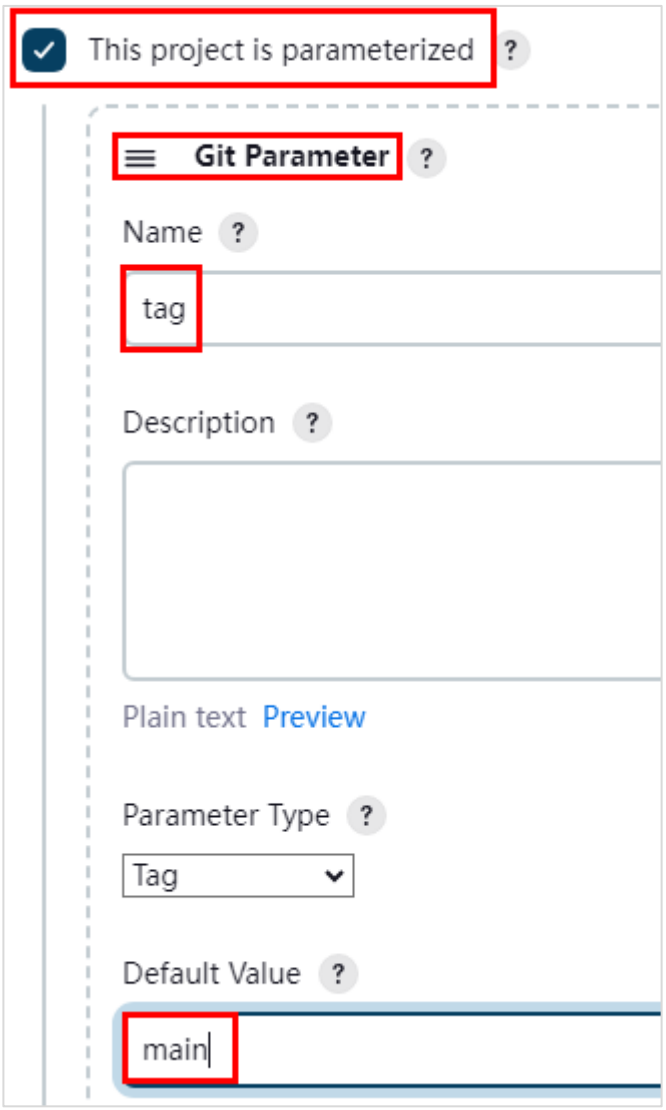
USER root
RUN echo "docker:x:987:jenkins" >> /etc/group
USER Jenkins
```

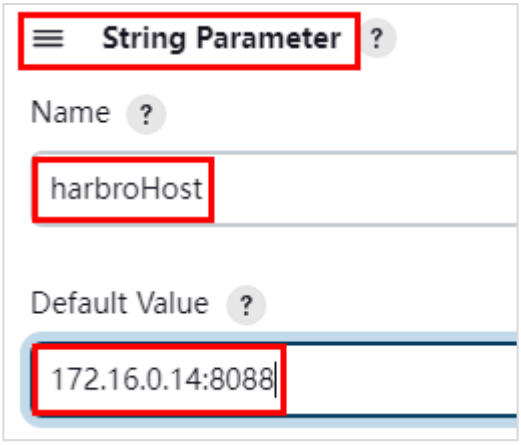
After the compilation is complete and the image is generated, push it to the same private image repository.

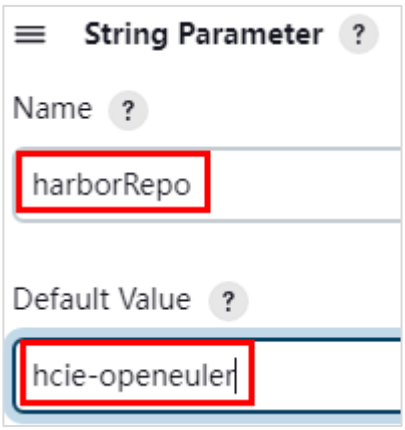
```
docker build -t 172.16.0.14:8088/hcie-openeuler/jenkins-docker:1.0 .
docker push 172.16.0.14:8088/hcie-openeuler/jenkins-docker:1.0
```

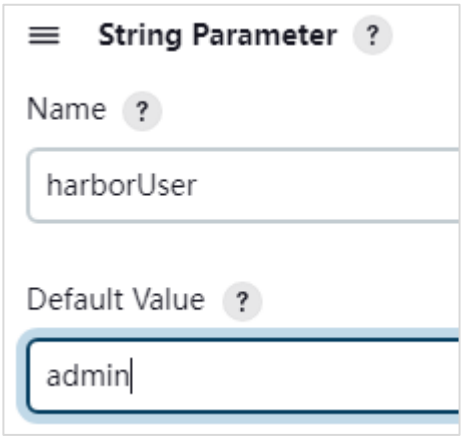
Step 9 Create a pipeline project and compile a pipeline script.

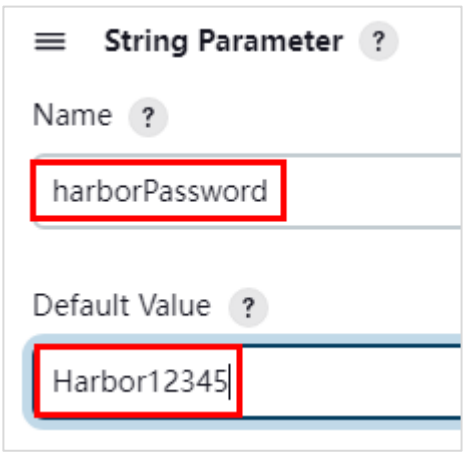
Create a pipeline project, select **Parameterized Build Process**, and set the following variables in sequence.


This project is parameterized
Git Parameter
Name
tag
Description
Plain text Preview
Parameter Type
Tag
Default Value
main


String Parameter
Name
harbroHost
Default Value
172.16.0.14:8088







Compile the pipeline script as follows:

```
podTemplate(
    containers: [
        containerTemplate(name: 'golang', image: '172.16.0.14:8088/hcie-openeuler/golang:1.17.3',
command: 'sleep', args: '99d'),
        containerTemplate(name: 'docker', image: '172.16.0.14:8088/hcie-openeuler/jenkins-
docker:1.0', command: 'sleep', args: '99d')],
        volumes: [hostPathVolume(hostPath: '/run/docker.sock', mountPath: '/run/docker.sock',
readOnly: false),
                  hostPathVolume(hostPath: '/usr/bin/kubectl', mountPath: '/usr/bin/kubectl',
readOnly: false)
  ]) {
```

```
    node(Pod_LABEL) {
        stage('Generate an application.') {
            container('golang') {
                stage('Pull code.') {
                    git branch: '${tag}', credentialsId: '0ce4ba97-2d29-4cc3-bcc3-1d5e11022019',
url: 'http://172.16.0.14/hcie-openeuler/jenkins-test.git'
                }
                stage('Compile code.') {
                    sh 'go build web.go'
                }
            }
        }
        stage('Deploy the application.'){
            container('docker') {
                stage('Generate an image and push it.') {
                    sh 'docker build -t ${HarborHost}/${harborRepo}/test:${tag} . && docker login -
u ${harborUser} -p ${harborPasswd} ${HarborHost} && docker push
${HarborHost}/${harborRepo}/test:${tag}'
                }
                stage('Deploy a deployment.') {
                    sh 'kubectl --kubeconfig ./config apply -f deploy.yml'
                }
            }
        }
    }
}
```

---------End---------

Huawei openEuler Certification Training

HCIE-openEuler

OpenStack Cluster Management

Lab Guide

ISSUE: 1.0



HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Cloud Computing Technologies Co., Ltd.

Trademarks and Permissions


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Our programs cover all ICT fields and follow the industry's trend of ICT convergence. With our leading talent development system and certification standards, we are committed to fostering new digital ICT talent and building a sound ICT talent ecosystem.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. The HCIE-openEuler certification covers Ceph management, OpenStack cluster management, advanced Kubernetes techniques, advanced automation management and CI/CD, the A-Ops intelligent O&M platform, and the EFK log management system.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

About This Document

Overview

This document is an HCIE-openEuler certification training course and is intended for trainees who are going to take the HCIE-openEuler exams or readers who want to know how to use openEuler to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management.

Description

This lab guide consists of multiple exercises, covering the setup and deployment of Ansible and Kolla Ansible. Ansible is used to deploy a OpenStack cluster in containers. During OpenStack deployment, Kolla is used to connect to an external Ceph cluster.

Background Knowledge Required

This course is for Huawei certification. To better understand this course, the intended audiences are advised to meet the following requirement:

- Have basic Linux knowledge. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
1 Lab Environment Overview 2
1.1 Node Planning 2
2 ECS Configuration 3
2.1 Basic Installation Environment Setup 3
3 OpenStack Installation 7
3.1 Kolla Installation 7
3.1.1 Installing Dependencies 7
3.1.2 Installing Kolla 8
3.2 Resource Preparations 9
3.2.1 Preparing Configuration Files 9
3.2.2 Preparing Ansible for Deployment 12
3.3 Ceph Storage Resource Preconfiguration 13
3.3.1 Creating Resource Pools 14
3.3.2 Creating and Exporting Keyring Files 14
3.3.3 Adding Ceph-related Configurations 14
3.4 OpenStack Installation 15
3.5 Post-installation Check 16
4 Resource Management 19
4.1 Compute Resource Expansion 19
4.1.1 Modifying the Kolla Host Inventory 19
4.1.2 Deploying the Node to Be Added 20
4.1.3 Verifying the Expansion 20
4.2 Storage Resource Expansion 20
4.3 Network Resource Management 23
4.3.1 Creating a Provider Network 23
5 OpenStack Management and Usage 28
5.1 Basic Skyline Dashboard Operations 28
5.2 Image Service 29
5.3 ECS Creation 31
5.4 File Storage Service 38

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

1 Lab Environment Overview

This lab uses five Huawei Cloud Elastic Cloud Servers (ECSs), one for the controller node, two for compute nodes, one for the network node, and one for the Ceph node. Ceph deployed in the cloud storage lab is used as the storage resource, including resource pools required by Glance, Cinder, and Manila. You need to apply for Huawei Cloud resources by yourself.

1.1 Node Planning

Table 1-1 Resources

Node	Specifications	Role
Controller	VPC: HCIE Business-10 Automatic IP address allocation Each ECS is bound to an EIP.	Controller node
Compute01		Compute node with two NICs
Compute02		Compute node with two NICs
Network01		Network node with two NICs
Ceph		

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

2 ECS Configuration

You need to modify the IP addresses automatically allocated to the ECSs by Huawei Cloud.

2.1 Basic Installation Environment Setup

Step 1 Change the host names.

Change the host name of each ECS according to node planning, excluding any specified exceptions. A restart is required after the change. The following uses the controller node as an example.

```
hostnamectl set-hostname  control01
```

```
[root@openstack-control-0001 ~]# hostnamectl set-hostname  control01
[root@openstack-control-0001 ~]# reboot
[root@openstack-control-0001 ~]#
```

Step 2 (Optional) Disable the firewall and SELinux.

The firewall and SELinux on the ECS are disabled by default. Ensure they are disabled.

```
setenforce 0
systemctl disable firewalld && systemctl stop firewalld
sed -i s#SELINUX=enforcing#SELINUX=disabled# /etc/selinux/config
```

Step 3 Configure time synchronization.

Install chrony and set it to automatically start upon system startup.

```
yum install -y chrony
systemctl enable chronyd --now
```

Before proceeding, verify that all ECSs share the same clock source by running the **chronyc sources** command.

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^? ntp1.flashdance.cx            2   6     1     1  -1343ms[-1343ms] +/-  132ms
^? ntp7.flashdance.cx            2   6     1     1  -1342ms[-1342ms] +/-  132ms
^? tick.ntp.infomaniak.ch        1   6     1     1  -1348ms[-1348ms] +/-   95ms
^? 119.28.183.184                2   6     1     1  -1341ms[-1341ms] +/-   38ms
```

Step 4 Configure node NICs.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Each ECS node is configured with two NICs. Ensure that the NIC names of all nodes are the identical. For example, all nodes have **ens3** for the primary NIC and **enp4s4** for the secondary.

While the primary NIC can communicate externally by default, you need to configure routing policies for the secondary NIC to enable external communication.

This example uses IP addresses of the network node, that is, 10.0.0.95 (**ens3**) and 10.0.0.3 (**enp4s1**), with a gateway of 10.0.0.1 to demonstrate the routing policy configuration. This step is necessary for all dual-NIC nodes.

Primary NIC

ip route add default via *subnet_gateway* **dev** *NIC_name* **table** *route_table_name*

ip route add *subnet_CIDR_block* **dev** *NIC_name* **table** *route_table_name*

ip rule add from *NIC_address* **table** *route_table_name*

Secondary NIC

ip route add default via *subnet_gateway* **dev** *NIC_name* **table** *route_table_name*

ip route add *subnet_CIDR_block* **dev** *NIC_name* **table** *route_table_name*

ip rule add from *NIC_address* **table** *route_table_name*

On the network node:

```
ip route add default via 10.0.0.1 dev ens3 table 10
ip route add 10.0.0.0/24 dev ens3 table 10
ip rule add from 10.0.0.95 table 10
ip route add default via 10.0.0.1 dev enp4s1 table 20
ip route add 10.0.0.0/24 dev enp4s1 table 20
ip rule add from 10.0.0.3 table 20
```

Note:

Set the parameters as follows:

NIC_name: actual NIC name

route_table_name: custom route table name using a number

Other NIC parameters: Set them according to the actual information.

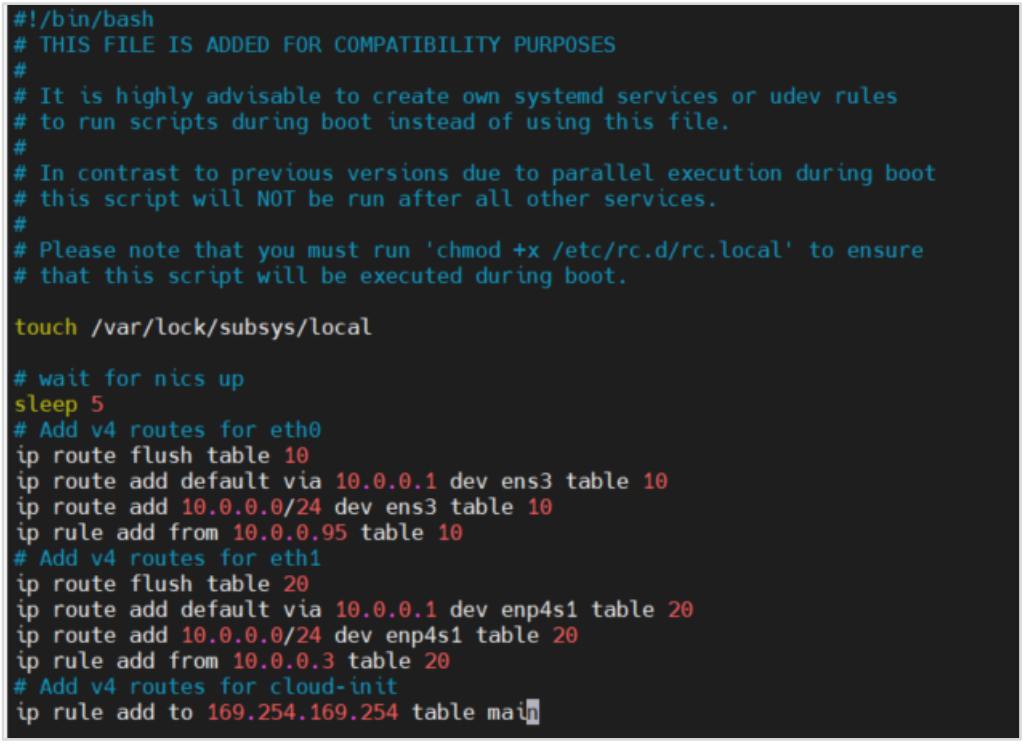
```
(venv) [root@control01 ~]# ip route add default via 10.0.0.1 dev ens3 table 10
(venv) [root@control01 ~]# ip route add 10.0.0.0/24 dev ens3 table 10
(venv) [root@control01 ~]# ip rule add from 10.0.0.95 table 10
(venv) [root@control01 ~]# ip route add default via 10.0.0.1 dev enp4s1 table 20
(venv) [root@control01 ~]# ip route add 10.0.1.0/24 dev enp4s1 table 20
(venv) [root@control01 ~]# ip rule add from 10.0.0.3 table 20
(venv) [root@control01 ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether fa:16:3e:33:33:92 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 10.0.0.95/24 brd 10.0.0.255 scope global dynamic noprefixroute ens3
       valid_lft 107999850sec preferred_lft 107999850sec
    inet6 fe80::f816:3eff:fe33:3392/64 scope link
       valid_lft forever preferred_lft forever
3: enp4s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether fa:16:3e:e7:0a:c0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.3/24 brd 10.0.0.255 scope global dynamic noprefixroute enp4s1
       valid_lft 107999850sec preferred_lft 107999850sec
    inet6 fe80::f816:3eff:fee7:ac0/64 scope link
       valid_lft forever preferred_lft forever
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Step 5 Configure permanent routing policies.

vim /etc/rc.local # To prevent route loss and network disconnection caused by ECS restart, copy the commands in the previous step to the configuration file.

```
# wait for nics up
sleep 5
# Add v4 routes for eth0
ip route flush table 10
ip route add default via 10.0.0.1 dev ens3 table 10
ip route add 10.0.0.0/24 dev ens3 table 10
ip rule add from 10.0.0.95 table 10
# Add v4 routes for ens3
ip route flush table 20
ip route add default via 10.0.0.1 dev enp4s1 table 20
ip route add 10.0.0.0/24 dev enp4s1 table 20
ip rule add from 10.0.0.3 table 20
# Add v4 routes for cloud-init
ip rule add to 169.254.169.254 table main     # Do not modify this IP address.
```



```
#!/bin/bash
# THIS FILE IS ADDED FOR COMPATIBILITY PURPOSES
#
# It is highly advisable to create own systemd services or udev rules
# to run scripts during boot instead of using this file.
#
# In contrast to previous versions due to parallel execution during boot
# this script will NOT be run after all other services.
#
# Please note that you must run 'chmod +x /etc/rc.d/rc.local' to ensure
# that this script will be executed during boot.

touch /var/lock/subsys/local

# wait for nics up
sleep 5
# Add v4 routes for eth0
ip route flush table 10
ip route add default via 10.0.0.1 dev ens3 table 10
ip route add 10.0.0.0/24 dev ens3 table 10
ip rule add from 10.0.0.95 table 10
# Add v4 routes for eth1
ip route flush table 20
ip route add default via 10.0.0.1 dev enp4s1 table 20
ip route add 10.0.0.0/24 dev enp4s1 table 20
ip rule add from 10.0.0.3 table 20
# Add v4 routes for cloud-init
ip rule add to 169.254.169.254 table main
```

Step 6 Check whether the routing policies take effect.

chmod +x /etc/rc.local # Add the execute permission to the file and restart for the routing policies to take effect.

```
chmod +x /etc/rc.d/rc.local
reboot
ip rule
ip rule show table 10
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

ip rule show table 20

```
[root@control01 ~]# ip rule show table 20
[root@control01 ~]# ip rule show table 10
[root@control01 ~]# chmod +x /etc/rc.local
[root@control01 ~]# chmod +x /etc/rc.d/rc.local
[root@control01 ~]# reboot
[root@control01 ~]# Connection to 10.0.0.3 closed by remote host.
Connection to 10.0.0.3 closed.
[root@router ~]# ssh 10.0.0.3

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
Last login: Wed Nov 29 01:46:27 2023 from 10.0.0.252


Welcome to 5.10.0-60.117.0.144.oe2203.x86_64

System information as of time:  Wed Nov 29 01:47:15 AM UTC 2023

System load:    0.00
Processes:      136
Memory used:    1.9%
Swap used:      0.0%
Usage On:       10%
IP address:     10.0.0.95
IP address:     10.0.0.3
Users online:   1


[root@control01 ~]# ip rule show table 10
32765:  from 10.0.0.95 lookup 10
[root@control01 ~]# ip rule show table 20
32764:  from 10.0.0.3 lookup 20
[root@control01 ~]# ip rule
0:      from all lookup local
32763:  from all to 169.254.169.254 lookup main
32764:  from 10.0.0.3 lookup 20
32765:  from 10.0.0.95 lookup 10
32766:  from all lookup main
32767:  from all lookup default
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

3 OpenStack Installation

3.1 Kolla Installation

3.1.1 Installing Dependencies

pip is required for Kolla installation. Install pip first.

Step 1 Update packages.

```
yum -y update
```

```
[root@control01 ~]# yum -y update
Last metadata expiration check: 2:35:39 ago on Mon [redacted].
Dependencies resolved.
================================================================================================================================================
 Package                                 Architecture                 Version                                  Repository                   Size
================================================================================================================================================
Installing:
 kernel                                  x86_64                       5.10.0-60.117.0.144.oe2203               update                       54 M
Upgrading:
 NetworkManager                          x86_64                       1:1.32.12-12.oe2203                      update                      2.2 M
 NetworkManager-libnm                    x86_64                       1:1.32.12-12.oe2203                      update                      1.6 M
 acl                                     x86_64                       2.3.1-2.oe2203                           update                       51 k
 atk                                     x86_64                       2.36.0-2.oe2203                          update                      262 k
 avahi-libs                              x86_64                       0.8-17.oe2203                            update                       62 k
 bash                                    x86_64                       5.1.8-6.oe2203                           update                      1.2 M
```

Step 2 Install dependencies for Python.

```
sudo dnf install git python3-devel libffi-devel gcc openssl-devel python3-libselinux
```

```
[root@control01 ~]# sudo dnf install git python3-devel libffi-devel gcc openssl-devel python3-libselinux
OS                                                                                    2.4 MB/s | 3.4 MB     00:01
everything                                                                             21 MB/s |  16 MB     00:00
EPOL                                                                                  4.1 MB/s | 2.6 MB     00:00
debuginfo                                                                             5.9 MB/s | 3.9 MB     00:00
source                                                                                2.9 MB/s | 1.7 MB     00:00
update                                                                                 44 MB/s |  65 MB     00:01
update-source                                                                         1.8 MB/s | 965 kB     00:00
Package python3-libselinux-3.3-1.oe2203.x86_64 is already installed.
Dependencies resolved.
================================================================================================================================================
 Package                                 Architecture                 Version                                  Repository                   Size
================================================================================================================================================
Installing:
 gcc                                     x86_64                       10.3.1-19.oe2203                         update                       29 M
 git                                     x86_64                       2.33.0-10.oe2203                         update                      6.0 M
 libffi-devel                            x86_64                       3.4.2-2.oe2203                           OS                           14 k
 openssl-devel                           x86_64                       1:1.1.1m-24.oe2203                       update                      1.8 M
 python3-devel                           x86_64                       3.9.9-27.oe2203                          update                       12 M
```

Step 3 Configure the pip environment.

Configure the pip source on the **control01** node.

```
mkdir   ~/.pip

cat >> ~/.pip/pip.conf <<EOF
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
[install]
trusted-host = https://pypitunatsinghuaedu.cn
EOF
```

Check whether the configuration takes effect.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
pip3 config list
```

```
[root@control01 ~]# mkdir ~/.pip
[root@control01 ~]# cat >> ~/.pip/pip.conf <<EOF
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/s imple
[install]
trusted-host = https://pypitunatsinghuaedu.cn^C
[root@control01 ~]# cat >> ~/.pip/pip.conf <<EOF
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/s imple
[install]
trusted-host = https://pypitunatsinghuaedu.cn
> EOF
[root@control01 ~]# cat  ~/.pip/pip.conf
[global]
index-url = https://pypi.tuna.tsinghua.edu.cn/s imple
[install]
trusted-host = https://pypitunatsinghuaedu.cn
[root@control01 ~]# pip config list
global.index-url='https://pypi.tuna.tsinghua.edu.cn/s imple'
install.trusted-host='https://pypitunatsinghuaedu.cn'
[root@control01 ~]# vim  ~/.pip/pip.conf
[root@control01 ~]# pip config list
global.index-url='https://pypi.tuna.tsinghua.edu.cn/simple'
install.trusted-host='https://pypitunatsinghuaedu.cn'
[root@control01 ~]#
```

Step 4 Install a virtual environment.

Create and activate a virtual environment. The activated virtual environment should depend on the packages installed within it.

```
python3 -m venv /path/to/venv
source /path/to/venv/bin/activate
```

```
[root@control01 ~]# python3 -m venv /path/to/venv
[root@control01 ~]# source /path/to/venv/bin/activate
(venv) [root@control01 ~]#
```

Step 5 Upgrade pip.

```
pip install -U pip
```

```
(venv) [root@control01 ~]# pip install -U pip
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: pip in /path/to/venv/lib/python3.9/site-packages (21.3.1)
Collecting pip
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/47/6a/453160888fab7c6a432a6e25f8afe6256d0d9f2cbd25971021da6491d899/pip-23.3.1-py3-none-any.whl (2.1 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 21.3.1
    Uninstalling pip-21.3.1:
      Successfully uninstalled pip-21.3.1
Successfully installed pip-23.3.1
```

Perform subsequent operations in the virtual environment.

3.1.2 Installing Kolla

Step 1 Install Ansible.

Install the Ansible version required by Kolla.

```
pip install 'ansible-core>=2.15,<2.16'
```

Step 2 Install the OpenStack package of the master branch.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Installing from source code can be time-consuming. Retry if network issues disrupt the process.

```
pip install git+https://opendev.org/openstack/kolla-ansible@s@master
```

If the installation is successful, the following information will be displayed.



Step 3 Install dependencies for Kolla and Kolla Ansible.

```
pip install ./kolla-ansible
```

After installation, proceed to Kolla Ansible preconfiguration.

3.2 Resource Preparations

3.2.1 Preparing Configuration Files

Step 1 Prepare files.

Create the **/etc/kolla** directory and copy the template files provided by Kolla to it.

```
mkdir -p /etc/kollachown $USER:$USER /etc/kolla
cp -r kolla-ansible/etc/kolla/* /etc/kolla
cp kolla-ansible/ansible/inventory/* .
```

Install dependencies for Ansible Galaxy.

```
kolla-ansible install-deps
```

Step 2 Edit the Ansible host inventory.

Multiple ECSs are used in this exercise. You need to modify the template file for multi-node deployment by referring to the following content:

```
[control]
# These hostname must be resolvable from your deployment host
controller

# The above can also be specified as follows:
#control[01:03]     ansible_user=kolla

# The network nodes are where your l3-agent and loadbalancers will run
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
# This can be the same as a host in the control group
[network]
network01

[compute]
compute01

[monitoring]
controller

# When compute nodes and controller nodes use different interfaces,
# you need to comment out "api_interface" and other interfaces from the globals.yml
# and specify like below:
#compute01 neutron_external_interface=eth0 api_interface=em1 tunnel_interface=em1

[storage]
compute01

[deployment]
localhost       ansible_connection=local
```

Keep the rest file content unchanged.

Step 3 Edit the configuration file.

Modify the **global.yml** file for installing OpenStack by referring to the following content.

```
.....
25 workaround_ansible_issue_8743: yes
.....
48 kolla_base_distro: "rocky"
49
50 # Do not override this unless you know what you are doing.
51 openstack_release: "master"
.....
60 node_custom_config: "{{ node_config }}/config"
.....
67 kolla_internal_vip_address: "192.168.1.131"
.....
137 network_interface: "ens192"
.....
166 neutron_external_interface: "ens224"
.....
172 neutron_plugin_agent: "openvswitch"
.....
311 enable_openstack_core: "yes"
312
313 # These roles are required for Kolla to be operation, however a savvy deployer
314 # could disable some of these required roles and run their own services.
315 enable_glance: "{{ enable_openstack_core | bool }}"
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
#enable_hacluster: "no"
enable_haproxy: "no"
#enable_keepalived: "{{ enable_haproxy | bool }}"
enable_keystone: "{{ enable_openstack_core | bool }}"
#enable_mariadb: "yes"
#enable_memcached: "yes"
enable_neutron: "{{ enable_openstack_core | bool }}"
enable_nova: "{{ enable_openstack_core | bool }}"
enable_rabbitmq: "{{ 'yes' if om_rpc_transport == 'rabbit' or om_notify_transport == 'rabbit' else 'no' }}"
.....
enable_cinder: "yes"
enable_cinder_backup: "yes"
.....
enable_heat: "{{ enable_openstack_core | bool }}"
enable_horizon: "{{ enable_openstack_core | bool }}"
.....
enable_manila: "yes"
.....
enable_manila_backend_cephfs_native: "yes"
.....
enable_skyline: "yes"
.....
########################
# External Ceph options
########################
# External Ceph - cephx auth enabled (this is the standard nowadays, defaults to yes)
#external_ceph_cephx_enabled: "yes"

# Glance
ceph_glance_user: "glance"
ceph_glance_keyring: "client.{{ ceph_glance_user }}.keyring"
ceph_glance_pool_name: "images"
# Cinder
ceph_cinder_user: "cinder"
ceph_cinder_keyring: "client.{{ ceph_cinder_user }}.keyring"
ceph_cinder_pool_name: "volumes"
#ceph_cinder_backup_user: "cinder-backup"
#ceph_cinder_backup_keyring: "client.{{ ceph_cinder_backup_user }}.keyring"
#ceph_cinder_backup_pool_name: "backups"
# Nova
ceph_nova_keyring: "{{ ceph_cinder_keyring }}"
#ceph_nova_user: "nova"
#ceph_nova_pool_name: "vms"
# Gnocchi
#ceph_gnocchi_user: "gnocchi"
#ceph_gnocchi_keyring: "client.{{ ceph_gnocchi_user }}.keyring"
#ceph_gnocchi_pool_name: "gnocchi"
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
504 # Manila
505 ceph_manila_user: "manila"
506 ceph_manila_keyring: "client.{{ ceph_manila_user }}.keyring"
.....
527 ########################
528 # Glance - Image Options
529 ########################
530 # Configure image backend.
531 glance_backend_ceph: "yes"
.....
574 ################################
575 # Cinder - Block Storage Options
576 ################################
577 # Enable / disable Cinder backends
578 cinder_backend_ceph: "yes"
.....
586 cinder_backup_driver: "ceph"
.....
624 nova_compute_virt_type: "kvm"
.....
639 nova_console: "novnc"
.....
695 manila_cephfs_filesystem_name: cephfs
.....
```

3.2.2 Preparing Ansible for Deployment

Step 1 Configure SSH password-free login.

Use the **ssh-keygen** and **ssh-copy-id** commands to configure password-free SSH login from the deployment node (controller node in this exercise) to the compute nodes and network node.

Step 2 Configure host name resolution.

Kolla configures host name resolution during the preconfiguration. Perform the following operations only on the deployment node **control01** unless otherwise specified. The IP addresses are the management addresses of the nodes.

```
cat > /etc/hosts <<EOF
192.168.1.131 controller
192.168.1.133 network01
192.168.1.132 compute01
EOF
```

Step 3 Verify the connectivity between hosts.

```
ansible -i multinode all -m ping
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
(venv) [root@control01 ~]# ansible -i multinode all -m ping
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
[WARNING]: Platform linux on host control02 is using the discovered Python interpreter at /usr/bin/python3, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
control02 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host control01 is using the discovered Python interpreter at /usr/bin/python3, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
control01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host control03 is using the discovered Python interpreter at /usr/bin/python3, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
control03 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host storage01 is using the discovered Python interpreter at /usr/bin/python3, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
storage01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host compute01 is using the discovered Python interpreter at /usr/bin/python3, but future installation of another Python interpreter could change this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for more information.
compute01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
(venv) [root@control01 ~]#
```

```
(venv) [root@control01 ~]# ansible -i all-in-one all -m ping
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details
[WARNING]: Platform linux on host control01 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.12/reference_appendices/interpreter_discovery.html for more
information.
control01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host compute02 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.12/reference_appendices/interpreter_discovery.html for more
information.
compute02 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host compute01 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.12/reference_appendices/interpreter_discovery.html for more
information.
compute01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
```

Step 4 Generate a password for Kolla.

The following commands generate a password in **/etc/kolla/passwords.yml** and change the value of **keystone_admin_password** to **kolla**.

```
kolla-genpwd
sed -i 's#keystone_admin_password:.*#keystone_admin_password: kolla#g' /etc/kolla/passwords.yml
```

```
(venv) [root@control01 kolla]# cat passwords.yml  |grep keystone_admin_password
keystone_admin_password: dSsVQvkLCAL2aX0NlBeJo3rWgv0J4YJvizgTjwJA
(venv) [root@control01 kolla]# sed -i 's#keystone_admin_password:.*#keystone_admin_password: kolla#g' /etc/kolla/passwords.yml
(venv) [root@control01 kolla]# cat passwords.yml  |grep keystone_admin_password
keystone_admin_password: kolla
```

3.3 Ceph Storage Resource Preconfiguration

The backend resource pools of Glance, Cinder, Cinder-backup, and Manila are previously defined in **global.yml**. This section guides you through creating corresponding resource pools in Ceph and generating their respective keyring files. These resource pools include:

Glance resource pool **images** with user **glance**

Cinder resource pool **volumes** with user **cinder**

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Cinder-backup resource pool **backups** with user **cinder-backup**

Manila resource pool **cephfs** with user **manila**

3.3.1 Creating Resource Pools

Create required resource pools.

```
##### Creating the Glance resource pool
ceph osd pool create glance
ceph osd pool application enable glance rbd
ceph auth add client.glance mon 'allow r' osd 'allow rwx pool=glance'


##### Creating the Cinder resource pool
ceph osd pool create volumes
ceph osd pool application enable volumes rbd
ceph auth add client.cinder mon 'allow r' osd 'allow rwx pool=volumes'

ceph osd pool create backups
ceph osd pool application enable backups rbd
ceph auth add client.cinder-backup mon 'allow r' osd 'allow rwx pool=backups'

##### Creating the Manila resource pool
ceph osd pool create metadata_cephfs
ceph osd pool create data_cephfs
ceph fs new cephfs metadata_cephfs data_cephfs
ceph orch apply mds cephfs --placement="2"
```

3.3.2 Creating and Exporting Keyring Files

Run the following commands to create and export keyring files:

```
ceph auth get client.glance -o ceph.client.glance.keyring
ceph auth get client.cinder -o ceph.client.cinder.keyring
ceph auth get client.cinder-backup -o ceph.client.cinder-backup.keyring
ceph auth get-or-create client.manila -o ceph.clinetmanila.keyring mgr 'allow rw' mon 'allow r'
```

3.3.3 Adding Ceph-related Configurations

Step 1 Add Glance-related configurations.

Create the **/etc/kolla/config/glance** directory and create **ceph.conf** in it with the following content:

```
[global]
fsid = 306410d2-e503-11ee-86f8-xxxxxxxxxx
keyring = /etc/ceph/ceph.client.glance.keyring
mon_initial_members = ceph-0
mon_host = 192.168.1.134
auth_cluster_required = cephx
auth_service_required = cephx
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
auth_client_required = cephx
```

In addition, copy the Glance keyring file created in section 3.3.2 to this directory.

Step 2 Add Cinder-related configurations.

Create the **/etc/kolla/config/cinder** directory and create subdirectories **cinder-backup** and **cinder-volume** in it. Then, create **ceph.conf** in the three directories with the following content:

```
[global]
fsid = 306410d2-e503-11ee-86f8-xxxxxxxxxxx
mon_initial_members = ceph-0
mon_host = 192.168.1.134
auth_cluster_required = cephx
auth_service_required = cephx
auth_client_required = cephx
```

Copy the Cinder keyring files, **ceph.client.cinder.keyring** and **ceph.client.cinder-backup.keyring** (created in section 3.3.2), to **cinder-volume** and **cinder-backup**, respectively.

Create the **/etc/kolla/config/nova** directory and copy **ceph.client.cinder.keyring** to it.

Step 3 Add Manila-related configurations.

Create the **/etc/kolla/config/manila** directory and create **ceph.conf** in it with the following content:

```
[global]
fsid = 306410d2-e503-11ee-86f8-xxxxxxxxxxx
mon_initial_members = ceph-0
mon_host = 192.168.1.134
auth_cluster_required = cephx
auth_service_required = cephx
auth_client_required = cephx
```

Create the **/etc/kolla/config/manila/manila-share** directory and copy **ceph.clinetmanila.keyring** created in section 3.3.2 to it.

3.4 OpenStack Installation

Step 1 Install dependencies for Kolla before installing OpenStack.

```
kolla-ansible -i multinode   bootstrap-servers
```

This command is required for all subsequent upgrade and reconfiguration operations.

Step 2 Check the configurations before installing OpenStack.

```
kolla-ansible -i multimode prechecks
```

Step 3 (Optional) Pull the image.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

You are advised to download the required container image to the deployment node before installation. This will ensure smooth installation and enable you to access running reports.

```
kolla-ansible -i multimode pull
```

Step 4 Install OpenStack.

```
kolla-ansible -i multimode deploy
```

Step 5 Perform post-installation configuration to generate required environment variables.

```
kolla-ansible post-deploy
```

After the command is executed, you can check the generated environment variable in **/etc/kolla/admin-openrc.sh**.

```
(venv) [root@controller ~]# ls /etc/kolla/
admin-openrc.sh          clouds.yaml  glance-api
admin-openrc-system.sh   config       globals.yml
cinder-api               cron         heat-api
cinder-scheduler         fluentd      heat-api-cfn
(venv) [root@controller ~]#
```

3.5 Post-installation Check

Step 1 Install the OpenStack client.

```
pip install python-openstackclient -c https://releases.openstack.org/constraints/upper/master
```

Import environment variables.

```
source /etc/kolla/admin-openrc.sh
```

Verify client installation and environment variable activation.

```
openstack --version
openstack availability zone list
```

```
(venv) [root@controller ~]# openstack --version
openstack 6.6.0
(venv) [root@controller ~]# openstack availability zone list
+-----------+-------------+
| Zone Name | Zone Status |
+-----------+-------------+
| internal  | available   |
| nova      | available   |
| nova      | available   |
| nova      | available   |
| nova      | available   |
+-----------+-------------+
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Step 2 Check compute resources.

The cluster should contain a compute node as planned. If the deployment is successful, the following information will be displayed:

```
(venv) [root@controller ~]# openstack hypervisor list
+--------------------------------------+---------------------+-----------------+---------------+-------+
| ID                                   | Hypervisor Hostname | Hypervisor Type | Host IP       | State |
+--------------------------------------+---------------------+-----------------+---------------+-------+
| e38060dc-55d7-4b23-b6f6-21e622e0e6cf | compute01           | QEMU            | 192.168.1.132 | up    |
+--------------------------------------+---------------------+-----------------+---------------+-------+
```

If the compute node is missing, verify that all automated deployment operations are successful. If any operation fails, redeploy the node and ensure all components are deployed.

Step 3 Check storage resources.

The backend storage of the cluster should contain a resource pool provided by RADOS Block Device (RBD) as planned. Run the following command to check the resource pool status:

```
openstack volume service list
```

If the resource pool status is normal, the following information will be displayed:

```
(venv) [root@controller ~]# openstack volume service list
+------------------+----------------+------+---------+-------+----------------------------+
| Binary           | Host           | Zone | Status  | State | Updated At                 |
+------------------+----------------+------+---------+-------+----------------------------+
| cinder-scheduler | controller     | nova | enabled | up    | 2024-04-01T05:36:44.000000 |
| cinder-volume    | compute@rbd-1  | nova | enabled | up    | 2024-04-01T05:36:36.000000 |
+------------------+----------------+------+---------+-------+----------------------------+
```

Otherwise, check whether the configuration in section 3.3.3 is correct and whether the following content in **global.yml** corresponds to the configuration in section 3.3.3:

```
########################
# External Ceph options
########################
# External Ceph - cephx auth enabled (this is the standard nowadays, defaults to yes)
#external_ceph_cephx_enabled: "yes"

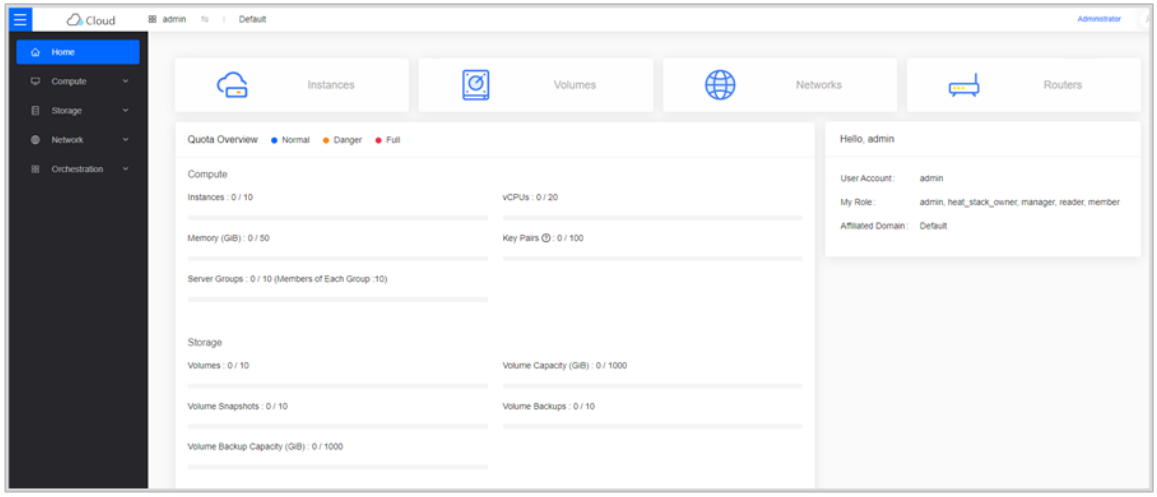
# Glance
ceph_glance_user: "glance"
ceph_glance_keyring: "client.{{ ceph_glance_user }}.keyring"
ceph_glance_pool_name: "images"
# Cinder
ceph_cinder_user: "cinder"
ceph_cinder_keyring: "client.{{ ceph_cinder_user }}.keyring"
ceph_cinder_pool_name: "volumes"
ceph_cinder_backup_user: "cinder-backup"
ceph_cinder_backup_keyring: "client.{{ ceph_cinder_backup_user }}.keyring"
ceph_cinder_backup_pool_name: "backups"
# Nova
ceph_nova_keyring: "{{ ceph_cinder_keyring }}"
#ceph_nova_user: "nova"
#ceph_nova_pool_name: "vms"
# Gnocchi
#ceph_gnocchi_user: "gnocchi"
#ceph_gnocchi_keyring: "client.{{ ceph_gnocchi_user }}.keyring"
#ceph_gnocchi_pool_name: "gnocchi"
# Manila
ceph_manila_user: "manila"
ceph_manila_keyring: "client.{{ ceph_manila_user }}.keyring"
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

After the faults are rectified, perform the deployment again.

Step 4 Check the user interface.

Access the Skyline dashboard through http://192.168.1.131:9999 to check the status of OpenStack services.



Copyright © Huawei Technologies Co.,Ltd

4 Resource Management

4.1 Compute Resource Expansion

4.1.1 Modifying the Kolla Host Inventory

Add information about the node to be added to the host inventory. For example, add information about **compute02** to the required host group.

```
[compute]
compute01
compute02

[monitoring]
controller

# When compute nodes and control
# you need to comment out "api_i
# and specify like below:
#compute01 neutron_external_inte

[storage]
controller
compute01
compute02
```

At the same time, add related information to the **hosts** file of the deployment node (controller node in this lab).

```
(venv) [root@controller ~]# cat /etc/hosts
127.0.0.1 localhost
::1         localhost localhost.localdomain
# BEGIN ANSIBLE GENERATED HOSTS
192.168.1.131 controller
192.168.1.133 network01
192.168.1.132 compute01
192.168.1.135 compute02
# END ANSIBLE GENERATED HOSTS
(venv) [root@controller ~]#
```

In addition, configure password-free login for Ansible.

```
ssh-copy-id root@compute02
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

4.1.2 Deploying the Node to Be Added

Preconfigure the node to be added.

```
kolla-ansible -i multinode bootstrap-servers --limit compute02
```

Pull the required container image to the node to be added.

```
kolla-ansible -i multinode pull --limit compute02
```

Run the **docker images** command to check the pulled image.

```
[root@compute02 ~]# docker images
REPOSITORY                                      TAG                 IMAGE ID
quay.io/openstack.kolla/nova-compute            zed-rocky-9         024108aca9a7
quay.io/openstack.kolla/neutron-openvswitch-agent   zed-rocky-9         22832ab2be30
quay.io/openstack.kolla/cinder-volume           zed-rocky-9         663699999abb
quay.io/openstack.kolla/cinder-backup           zed-rocky-9         3372c25bb663
quay.io/openstack.kolla/nova-ssh                zed-rocky-9         016b19419cd5
quay.io/openstack.kolla/openvswitch-db-server   zed-rocky-9         02062add466e
quay.io/openstack.kolla/openvswitch-vswitchd    zed-rocky-9         7ac6c7eaec4c
quay.io/openstack.kolla/nova-libvirt            zed-rocky-9         282030daf5d4
quay.io/openstack.kolla/kolla-toolbox           zed-rocky-9         b45e4e92a769
quay.io/openstack.kolla/cron                    zed-rocky-9         a10a0a9c53f9
quay.io/openstack.kolla/fluentd                 zed-rocky-9         253de28b31b1
```

Deploy the node.

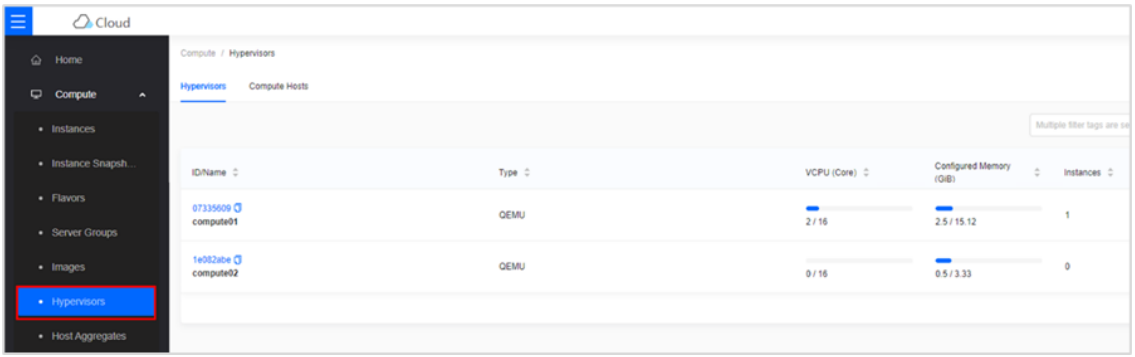
```
kolla-ansible -i multinode deploy --limit compute02
```

4.1.3 Verifying the Expansion

Check whether the node to be added is successfully deployed.

```
(venv) [root@controller ~]# openstack hypervisor list
+--------------------------------------+---------------------+-----------------+---------------+-------+
| ID                                   | Hypervisor Hostname | Hypervisor Type | Host IP       | State |
+--------------------------------------+---------------------+-----------------+---------------+-------+
| 2e2177d0-7953-4145-951a-c50cd26b6d19 | compute01           | QEMU            | 192.168.1.132 | up    |
| a82a9e58-b5f5-4718-887a-cf06dfbbee93 | compute02           | QEMU            | 192.168.1.135 | up    |
+--------------------------------------+---------------------+-----------------+---------------+-------+
```

You can also check whether the node is deployed on the **Administrator** page of Skyline.



4.2 Storage Resource Expansion

Ceph is used to provide backend storage resources for Cinder.

Step 1 Make preparations.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Create resource pool **openeuler** and user **openeuler** in Ceph.

```
ceph osd pool create openeuler
ceph osd pool application enable openeuler rbd
ceph auth add client.openeuler mon 'allow r' osd 'allow rwx pool=openeuler'
```

Export the keyring of the **openeuler** user.

```
ceph auth get client.openeuler -o ceph.client.openeuler.keyring
```

Copy the exported keyring file to the **/etc/kolla/cinder-volume** directory on each storage node.

Step 2 Register a secret.

Go to the **nova_libvirt** container on the compute node and run the following command to generate a random UUID for the secret:

```
uuidgen
```

```
(nova-libvirt)[root@compute02 /]# uuidgen
8da28902-ab96-458a-94f0-d74d78232434
```

Edit the **secret.xml** file as follows:

```
<secret ephemeral='no' private='no'>
  <uuid>8da28902-ab96-458a-94f0-d74d78232434</uuid>
  <usage type='ceph'>
    <name>client.openeuler secret</name>
  </usage>
</secret>
```

In the file, **uuid** is the generated UUID, **client.openeuler** indicates the **openeuler** resource pool user.

Define the secret.

```
virsh secret-define --file secret.xml
```

```
(nova-libvirt)[root@compute02 /]# virsh secret-define --file secret.xml
Secret 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4 created
```

Pass the key of the **client.openeuler** user to the secret. For example, the key in the following figure is **AQAy4yBmCamsLxAAG1hnu7U9nps6d0EiyspE9g==**.

```
        caps: [mon] allow r
client.openeuler
        key: AQAy4yBmCamsLxAAG1hnu7U9nps6d0EiyspE9g==
        caps: [mon] allow r
        caps: [osd] allow rwx pool=openeuler
mgr.ceph-nisorp
```

Run the following command to pass the key to the secret:

```
virsh secret-set-value --secret 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4 --base64
AQAy4yBmCamsLxAAG1hnu7U9nps6d0EiyspE9g==
```

```
(nova-libvirt)[root@compute02 /]# virsh secret-define --file secret.xml
Secret 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4 created

(nova-libvirt)[root@compute02 /]# virsh secret-set-value --secret 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4 --base64 AQAy4yBmCamsLxAAG1hnu7U9nps6d0EiyspE9g==
error: Passing secret value as command-line argument is insecure!
Secret value set
```

Check the current secret.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
virsh secret-list
```

```
(nova-libvirt)[root@compute02 /]# virsh secret-list
 UUID                                   Usage
--------------------------------------------------------------------------------
 368e464c-d5c9-4311-8d70-a9438160fbe7   ceph client.cinder secret
 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4   ceph client.openeuler secret
```

After the registration is complete, two files corresponding to the secret are generated in the **/etc/libvirt/secrets directory** in the container, as shown in the following figure.

```
(nova-libvirt)[root@compute02 /]# ls /etc/libvirt/secrets/
368e464c-d5c9-4311-8d70-a9438160fbe7.base64  7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4.base64
368e464c-d5c9-4311-8d70-a9438160fbe7.xml     7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4.xml
```

The **/etc/libvirt/secrets** directory in the container corresponds to the **/var/lib/docker/volumes/nova_libvirt_secrets/_data** directory on the host.

```
"Name": "nova_libvirt_secrets",
"Source": "/var/lib/docker/volumes/nova_libvirt_secrets/_data",
"Destination": "/etc/libvirt/secrets",
```

Copy the two generated files to the same directory on **compute01**, restart the **nova_libvirt** container, and check whether the same secret exists in the container.

```
(nova-libvirt)[root@compute01 /]# virsh secret-list
 UUID                                   Usage
--------------------------------------------------------------------------------
 368e464c-d5c9-4311-8d70-a9438160fbe7   ceph client.cinder secret
 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4   ceph client.openeuler secret
```

After confirming that the secret exists, restart the **nova_compute** container on each compute node.

Step 3 Modify the Cinder configuration file.

On each storage node, modify the **/etc/kolla/cinder-volume/cinder.conf** file as follows to add configurations for the storage pool to be added:

```
.....
11 enabled_backends = rbd-1,rbd-2
.....
77 [rbd-2]
78 volume_driver = cinder.volume.drivers.rbd.RBDDriver
79 volume_backend_name = rbd-2
80 rbd_pool = openeuler
81 rbd_ceph_conf = /etc/ceph/ceph.conf
82 rados_connect_timeout = 5
83 rbd_user = openeuler
84 rbd_cluster_name = ceph
85 rbd_keyring_conf = /etc/ceph/ceph.client.openeuler.keyring
86 rbd_secret_uuid = 7e4d77f2-6fb0-45c5-b229-0bb3bd4605c4 ### uuid is the secretuuid generated in the previous step.
87 report_discard_supported = True
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
......
```

Restart the cinder-volume container on each storage node. Once the restart is complete, verify service functionality on the controller node. The following figure shows the expected output indicating normal service operation.

```
(venv) [root@controller ~]# openstack volume service list
+------------------+-----------------+------+---------+-------+----------------------------+
| Binary           | Host            | Zone | Status  | State | Updated At                 |
+------------------+-----------------+------+---------+-------+----------------------------+
| cinder-scheduler | controller      | nova | enabled | up    | [redacted]:04.000000 |
| cinder-volume    | compute01@rbd-1 | nova | enabled | up    | [redacted]:09.000000 |
| cinder-volume    | compute01@rbd-2 | nova | enabled | up    | [redacted]:09.000000 |
| cinder-volume    | compute02@rbd-1 | nova | enabled | up    | [redacted]:02.000000 |
| cinder-volume    | compute02@rbd-2 | nova | enabled | up    | [redacted]:02.000000 |
| cinder-backup    | compute01       | nova | enabled | up    | [redacted]:11.000000 |
| cinder-backup    | compute02       | nova | enabled | up    | [redacted]:06.000000 |
+------------------+-----------------+------+---------+-------+----------------------------+
```

On the dashboard, you can check the available backend storage.



Step 4 Bring online the expanded storage resources.

Create an OpenStack volume type.

```
openstack volume type create openeuler
```

In the command, **openeuler** is a custom name.

```
(venv) [root@controller ~]# openstack volume type create openeuler
+-------------+--------------------------------------+
| Field       | Value                                |
+-------------+--------------------------------------+
| description | None                                 |
| id          | 53543b1e-4b07-4c4d-b498-05650fd0dc85 |
| is_public   | True                                 |
| name        | openeuler                            |
+-------------+--------------------------------------+
```

Associate the created volume type with the added backend storage.

```
openstack volume type set openeuler --property volume_backend_name=openeuler
```

4.3 Network Resource Management

4.3.1 Creating a Provider Network

In this exercise, you will create a provider network to connect the physical network and enable floating IP addresses for use in subsequent labs.

Step 1 Check Neutron configurations.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Before creating a network, confirm the provider network name. On the controller node, enter the **neutron_server** container and view the content of **/etc/neutron/plugins/ml2/ml2_conf.ini**.

```
[ml2]
type_drivers = flat,vlan,vxlan
tenant_network_types = vxlan
mechanism_drivers = openvswitch,l2population
extension_drivers = port_security

[ml2_type_vlan]
network_vlan_ranges =

[ml2_type_flat]
flat_networks = physnet1

[ml2_type_vxlan]
vni_ranges = 1:1000
```

The file defines a provider network named **physnet1** of the **flat_networks** type.

On the network node, enter the **neutron_openvswitch_agent** container and view the content of **/etc/neutron/plugins/ml2/openvswitch_agent.ini**.

```
[agent]
tunnel_types = vxlan
l2_population = true
arp_responder = true

[securitygroup]
firewall_driver = neutron.agent.linux.iptables_firewall.OVSHybridIptablesFirewallDriver

[ovs]
bridge_mappings = physnet1:br-ex
datapath_type = system
ovsdb_connection = tcp:127.0.0.1:6640
ovsdb_timeout = 10
local_ip = 192.168.1.133
```

The file content suggests that **physnet1** corresponds to **br-ex**. Enter the **openvswitch_vswitchd** container and run the **ovs-vsctl** command to check **br-ex** information.

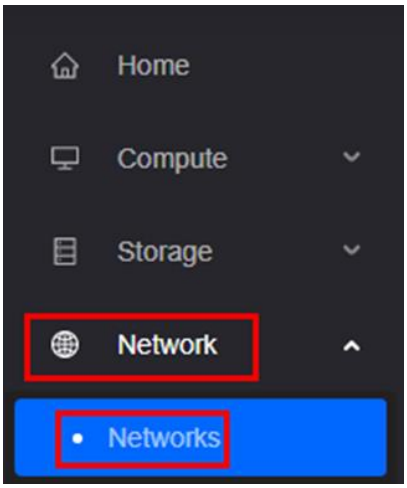
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
Bridge br-ex
    Controller "tcp:127.0.0.1:6633"
        is_connected: true
    fail_mode: secure
    datapath_type: system
    Port ens224
        Interface ens224
    Port phy-br-ex
        Interface phy-br-ex
            type: patch
            options: {peer=int-br-ex}
    Port br-ex
        Interface br-ex
            type: internal
```

br-ex connects to the physical network through the **ens224** NIC of the network node. Contact the network engineer to check the configuration of the network device interface connected to **ens224**. Based on this configuration, you can then choose the type of shared network to add. If the network type is VLAN or VXLAN, modify the **ml2_conf.ini** file in the **neutron_server** container accordingly. In this exercise, the NICs of all nodes are connected to a virtual switch without any configuration, so set the network type to **flat**.

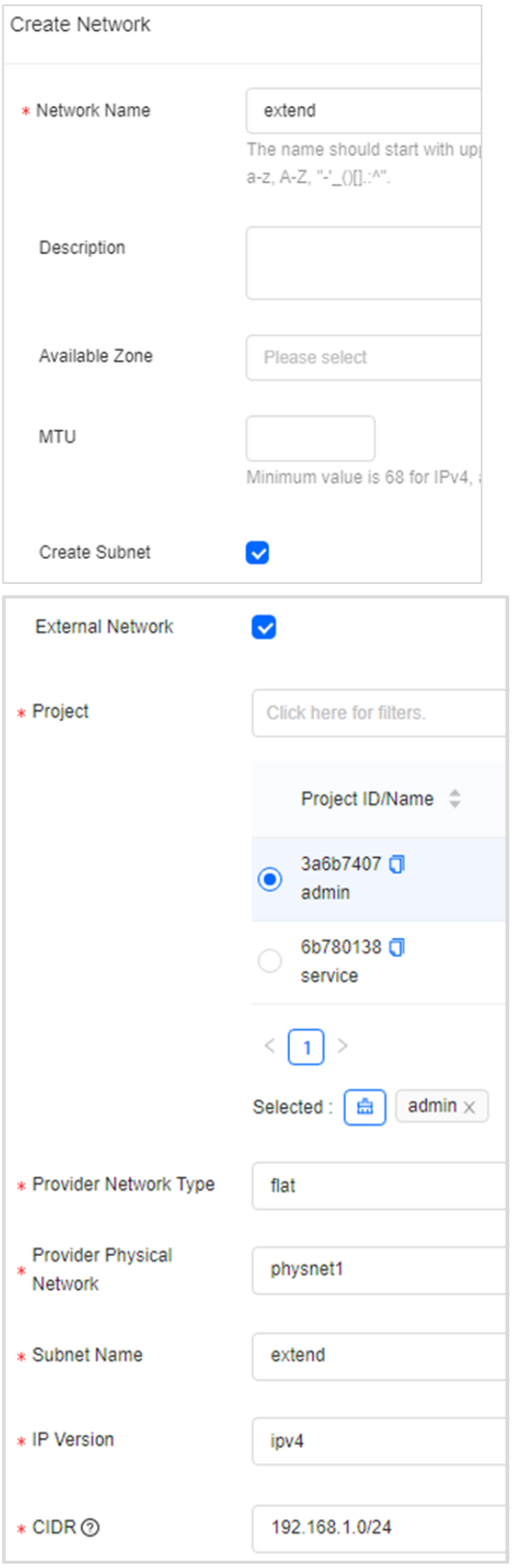
Step 2 Add a network.

Log in to the OpenStack dashboard as an administrator, click **Network**, and choose **Networks** from the expanded menu.



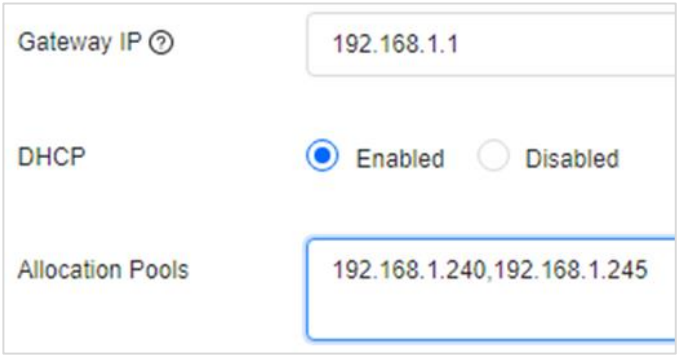
On the displayed page, click **Create Network** and enter required information.

Copyright © Huawei Technologies Co.,Ltd

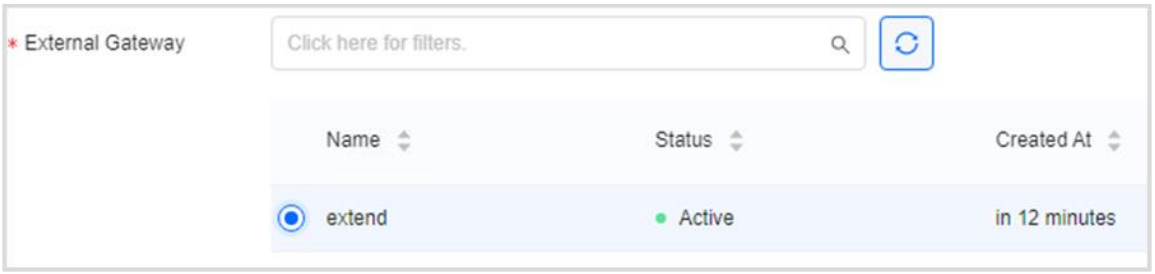


Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Click **Expand Advanced Options** to configure the subnet.



Create a router and add the subnet to the route as an external gateway.



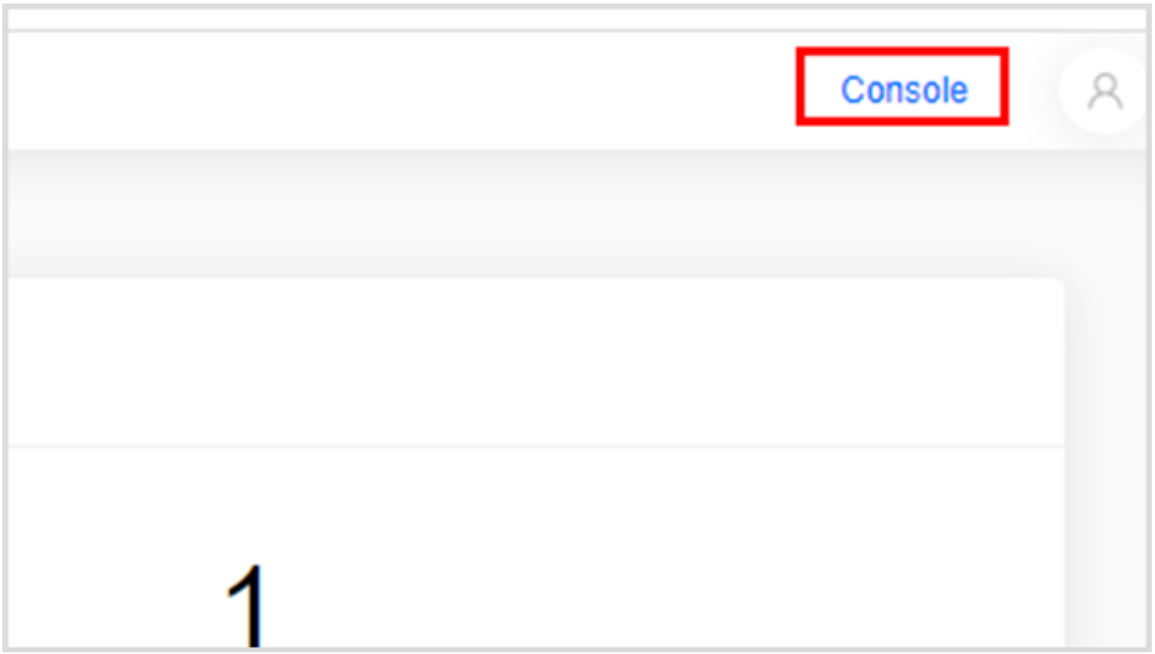
After the operation is complete, the subnet will be used as the provider network to connect to the physical network.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

5 OpenStack Management and Usage

5.1 Basic Skyline Dashboard Operations

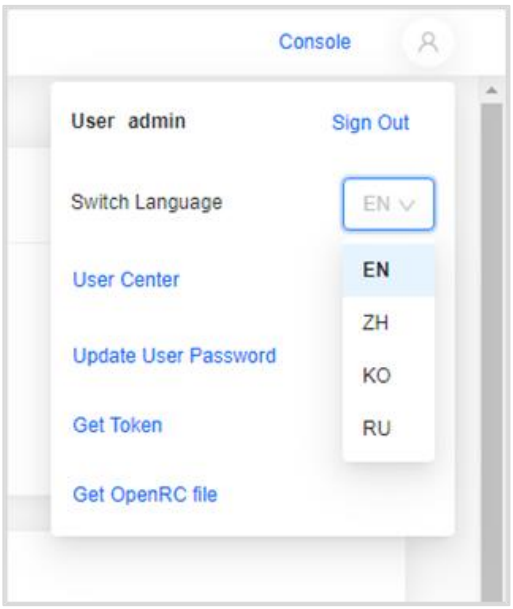
The most significant difference between Skyline and the original Horizon dashboard is the separation of tenant and administrator pages. The switch button is located in the top right corner of the main page.



Click **Console** to go to the tenant page, or click **Administrator** to go to the administrator page.

The language switcher is located in the upper right corner. You can change the language by clicking the avatar.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



5.2 Image Service

In this section, you will use Guestfish to create an image based on **openeuler-2203sp3.qcow2** and write **ceph.conf** to its **/etc/ceph** directory.

Download the **openeuler-2203sp3.qcow2** image from the openEuler official website: https://mirrors.nju.edu.cn/openeuler/openEuler-22.03-LTS-SP3/virtual_machine_img/x86_64/openEuler-22.03-LTS-SP3-x86_64.qcow2.xz

Step 1 Install Guestfish on the controller node.

```
yum install -y libguestfs-tools
```

Step 2 Edit the image.

Set the environment variable to allow the image to run directly.

```
export LIBGUESTFS_BACKEND=direct
```

Run the image.

```
guestfish -i -a openeuler-2203sp3.qcow2
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
[root@controller ~]# guestfish -i -a openeuler-2203sp3.qcow2

Welcome to guestfish, the guest filesystem shell for
editing virtual machine filesystems and disk images.

Type: 'help' for help on commands
      'man' to read the manual
      'quit' to quit the shell

Operating system: openEuler 22.03 (LTS-SP3)
/dev/sda2 mounted on /
/dev/sda1 mounted on /boot

><fs>
```

Create the **/etc/ceph** directory and the **ceph.conf** file.

```
mkdir /etc/ceph
touch /etc/ceph/ceph.conf
```

Run the **edit** command to edit **ceph.conf**.

```
edit /etc/ceph/ceph.conf
```

Write Ceph information into **ceph.conf**, for example:

```
# minimal ceph.conf for 306410d2-e503-11ee-86f8-000c29c2736c
[global]
        fsid = 306410d2-e503-11ee-86f8-000c29c2736c
        mon_host = [v2:192.168.1.134:3300/0,v1:192.168.1.134:6789/0]
```

(Optional) After saving the file and exit, re-label all files to avoid any SELinux context issues.

```
selinux-relabel /etc/selinux/targeted/contexts/files/file_contexts /
sync
```

Note: Relabeling is not mandatory because the image used in this exercise does not contain SELinux-related modules.

After the execution is complete, run the **exit** command to exit.

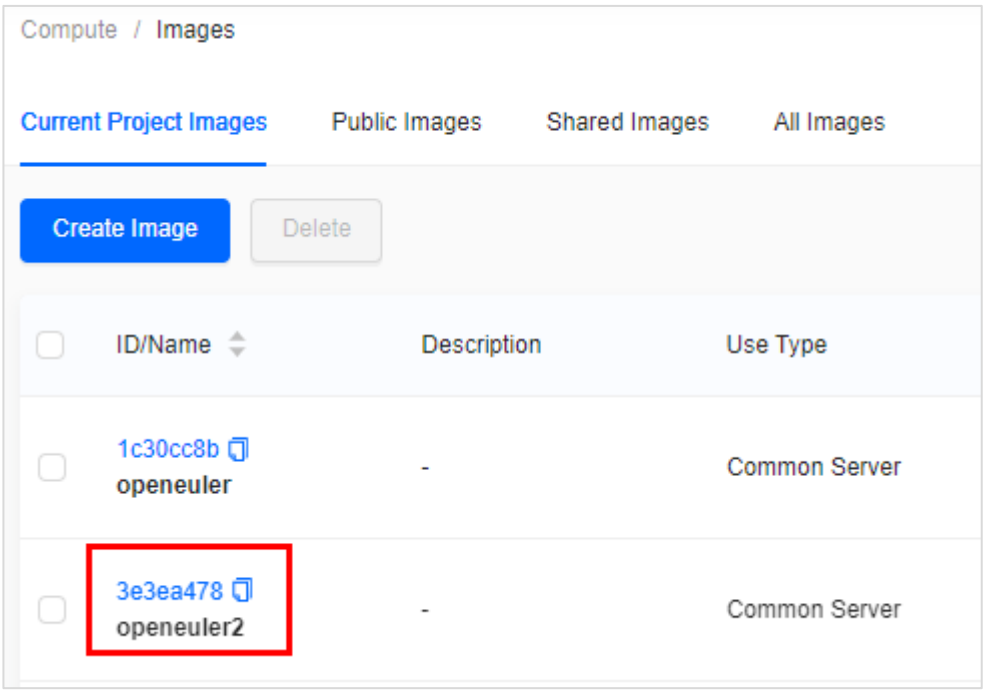
```
><fs> command "ls /etc/ceph"
ceph.conf

><fs> exit
```

Step 3 Upload the image.

Run a command or use the dashboard to register the generated image.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



5.3 ECS Creation

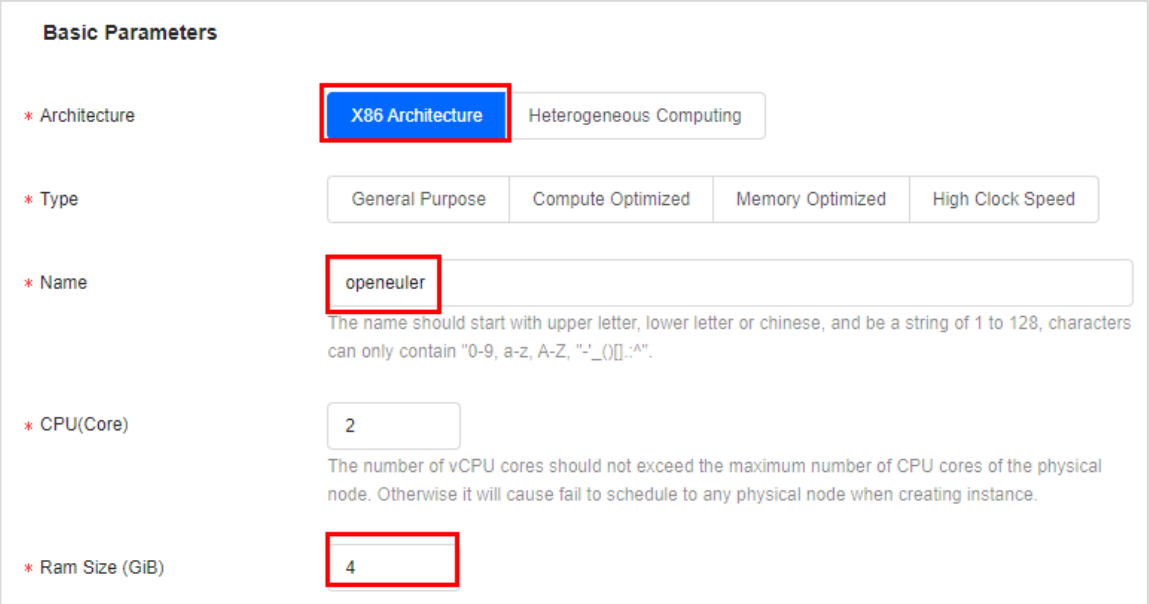
In this section, you will create **ECS1** and **ECS2** based on the image created in the previous section. The requirements for the ECSs are as follows:

1. **ECS1** and **ECS2** use **rbd-1** and **rbd-2** as their backend storage, respectively.
2. The two ECSs run on different compute nodes.
3. ceph-fuse is installed in the ECSs.

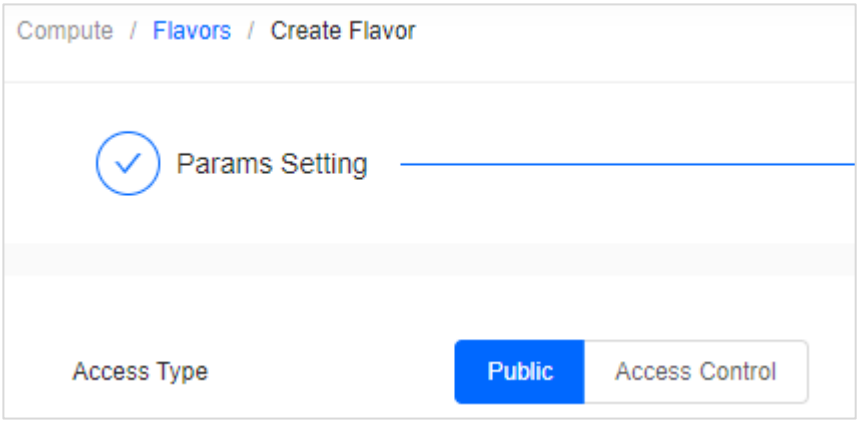
Step 1 Create an ECS flavor.

On the administrator page, choose **Compute** > **Flavor**. On the displayed page, click **Create Flavor**, and configure the flavor based on the following recommended values:

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



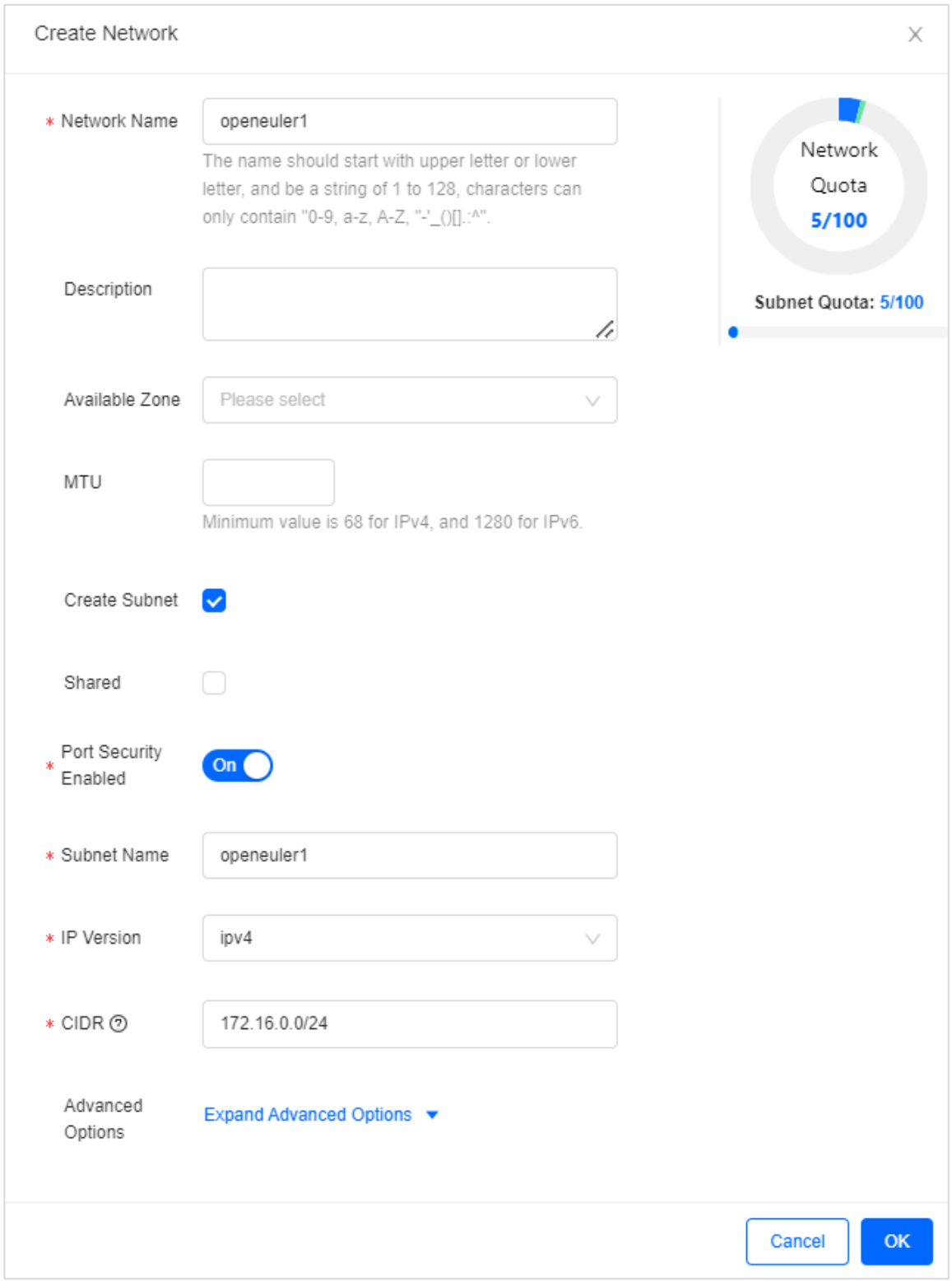
After setting the values, click **Next: Access Type Setting**, and choose **Public**.



Step 2 Create a network.

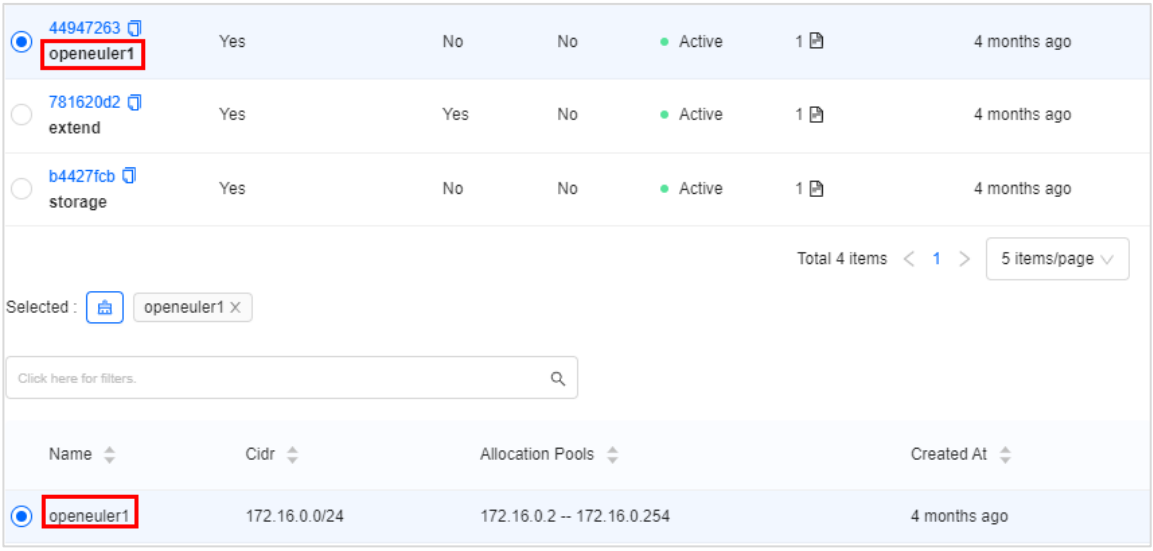
On the **Console** page, choose **Network > Networks**. On the displayed page, click **Create Network** and set the network based on the following recommended values:

Copyright © Huawei Technologies Co.,Ltd



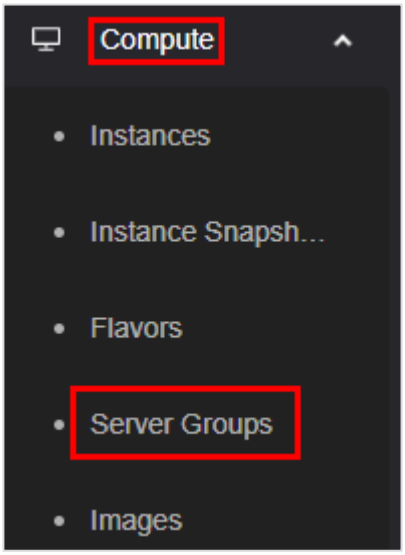
Choose **Routers** from the expanded menu. Click **More** of an existing router (if no router is available, create a router and set the external gateway by referring to step 2 in section 4.3.1) and choose **Connect Subnet**. In the displayed dialog box, select the created **openeuler1** subnet.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Step 3 Create a server group.

On the **Console** page, choose **Compute** > **Server Groups**.



On the displayed page, click **Create Server Group** and set the parameters as follows:

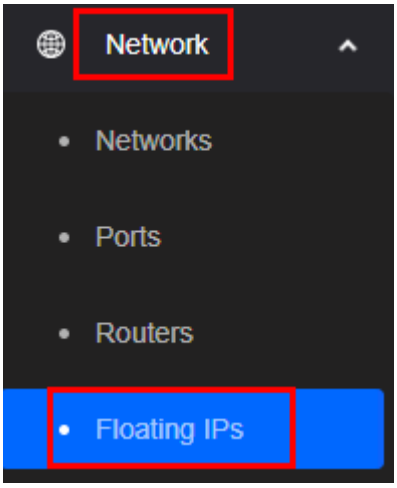
Copyright © Huawei Technologies Co.,Ltd



Step 4 Apply for floating IP addresses.

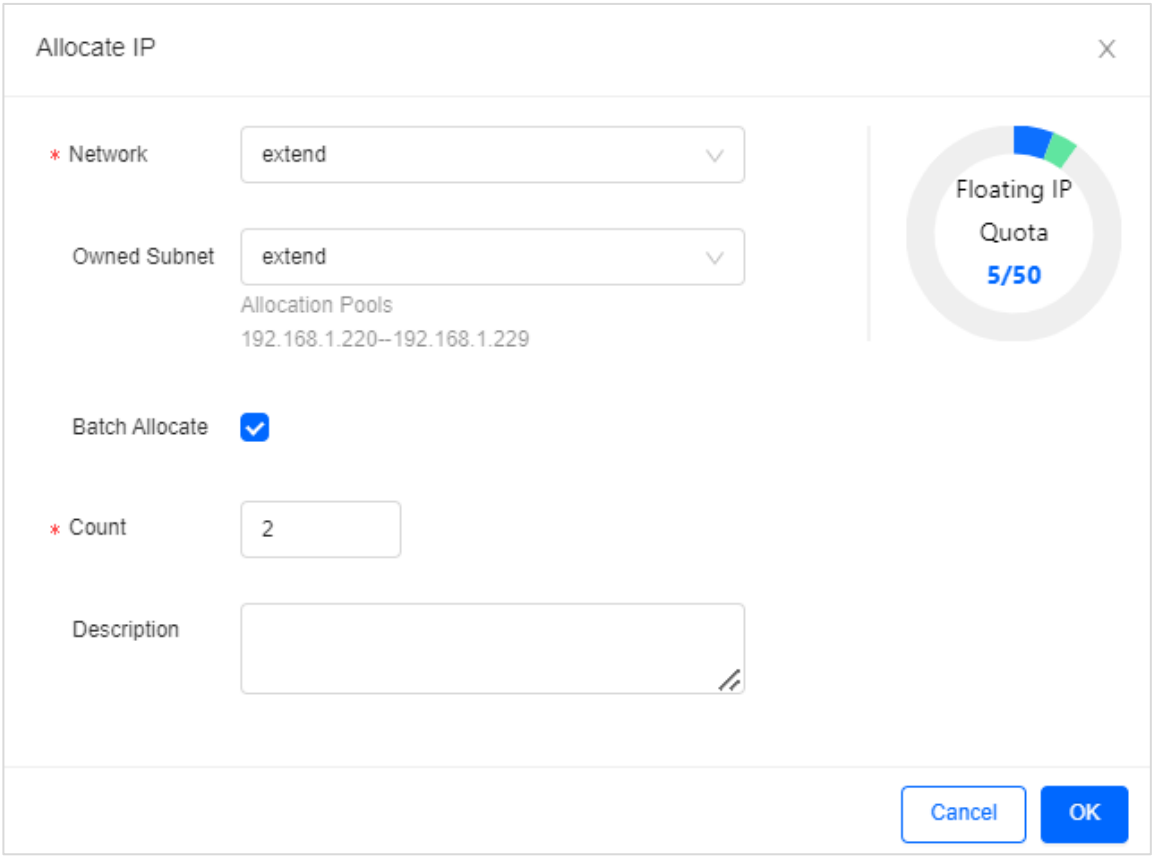
ceph-fuse needs to be installed on the two ECSs, and the ECSs need to connect to the Internet. Therefore, you need to apply for two floating IP addresses to be bound to the ECSs in advance. The procedure is as follows:

On the **Console** page, choose **Network** > **Floating IPs**.



On the displayed page, click **Allocate IP**. In the displayed dialog box, set the parameters as follows:

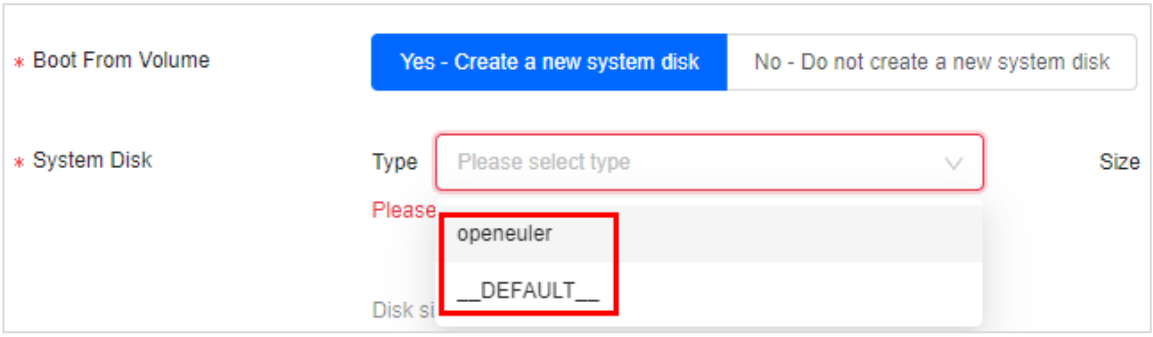
Copyright © Huawei Technologies Co.,Ltd



The network and subnet are the provider network created in section 4.3.

Step 5 Create the ECSs.

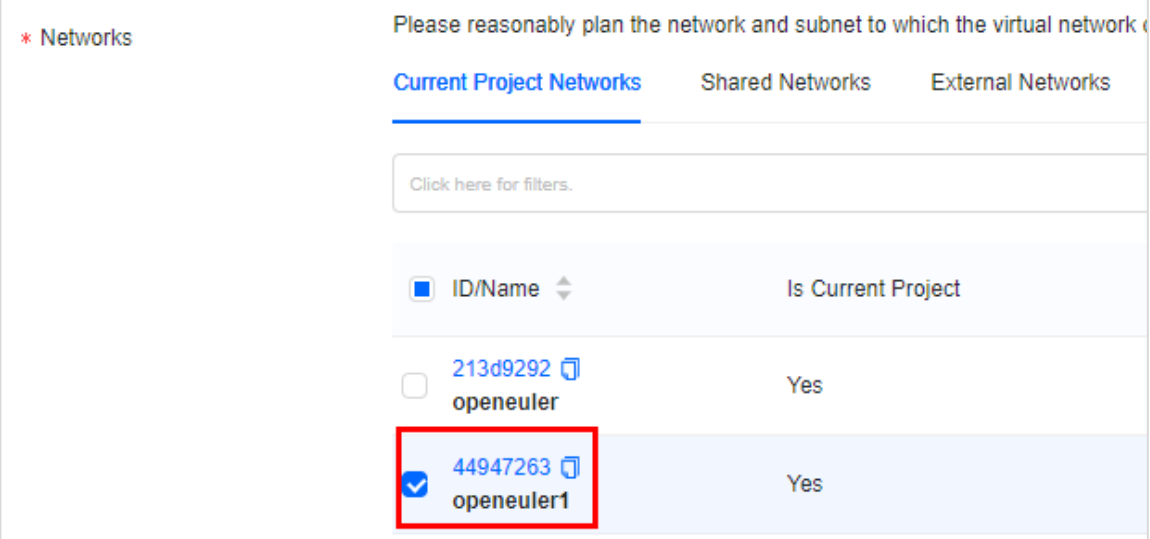
Create the ECSs on the **Console** page. Select different types for **System Disk** for the ECSs to use different backend storage.



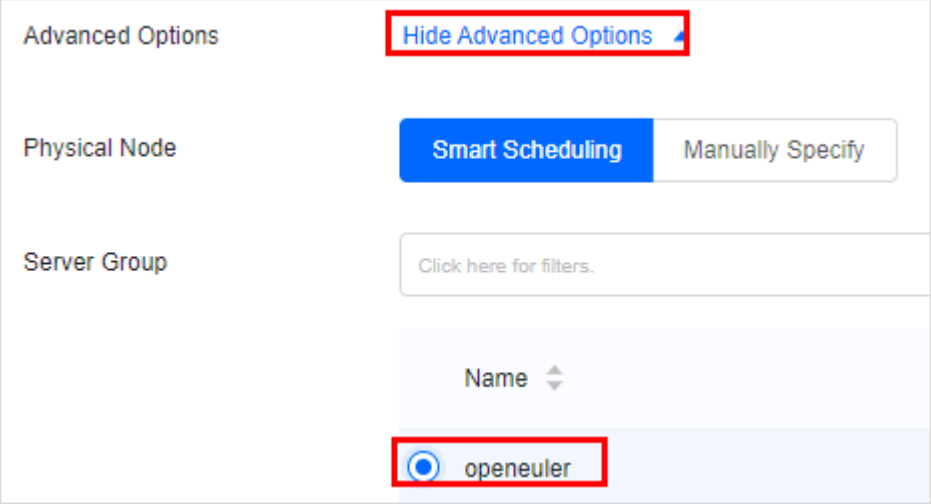
Select **_DEFAULT_** for **ECS1** and **openeuler** for **ECS2**.

Set the networks of both ECSs to use the previously created port.

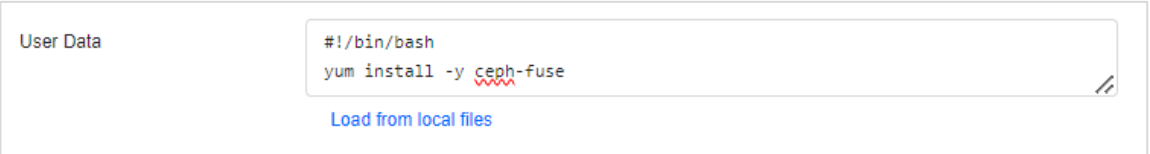
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



In **System Config**, add the two ECSs to the **openeuler** server group created in step 3.



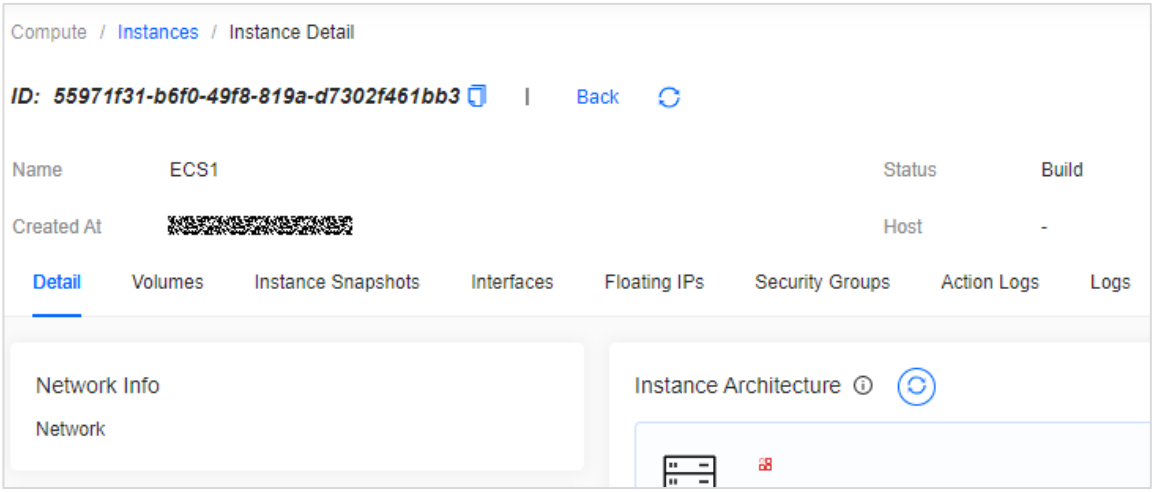
In the **User Data** dialog box, enter the following information to install ceph-fuse:



Click **Next: Confirm Config** and click **OK** to create the ECS.

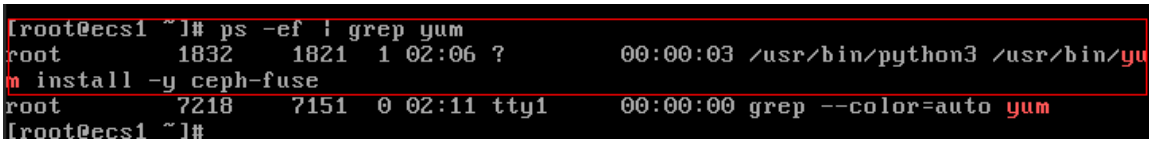
During the ECS creation, click the ECS name to go to the ECS details page.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Click **Interfaces** to go to the **Port Detail** page. Click **More** in the upper right corner, select **Associate Floating IP**, and bind the floating IP address created in step 4.

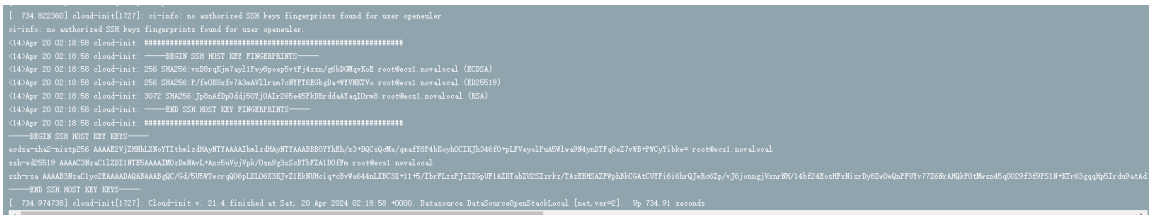
Note: Ensure that the floating IP address is bound before the ECS creation is complete. Otherwise, you will miss the time to install ceph-fuse. ceph-fuse installation may take a long time. After the ECS is created, log in and check whether the installation progress. The following figure shows an ongoing installation.



Wait for three minutes until the installation is complete.

Question: Is the password used for ECS login the same as the password set during ECS creation? Why or why not?

Answer: No. The default password of the image (**openEuler12#$**) is used for login. According to the following logs, the password is updated to the configured one after ceph-fuse is installed.



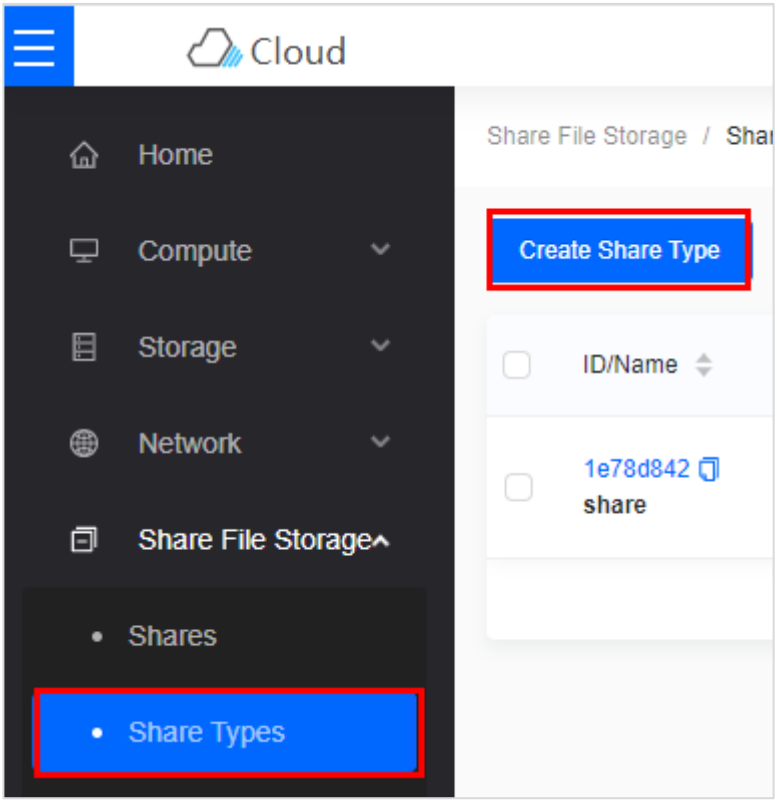
5.4 File Storage Service

In this section, you will apply for and mount file storage to **ECS1** as user **ecs**.

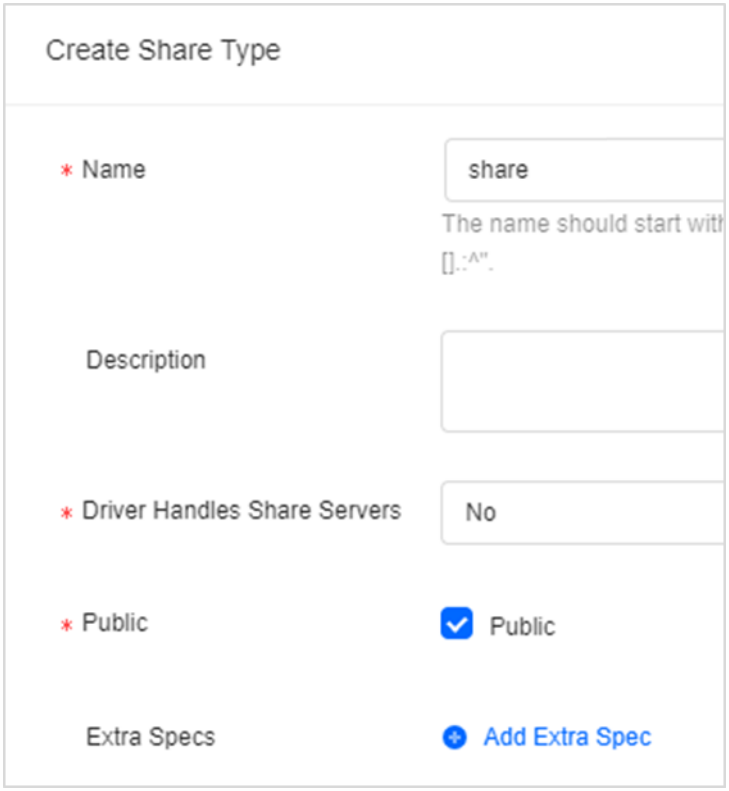
Step 1 Creates a share type.

On the **Administrator** page, choose **Share File Storage** > **Share Types**. On the page that is displayed, click **Create Share Type**.

Copyright © Huawei Technologies Co.,Ltd



In the displayed dialog box, set the parameters as follows:

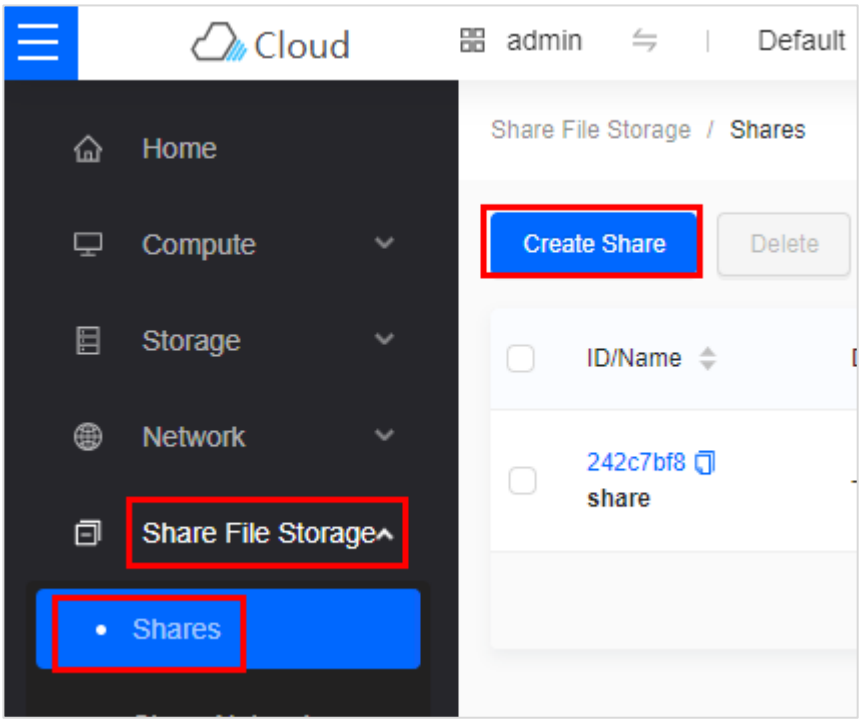


Click **OK** to complete the share type configuration.

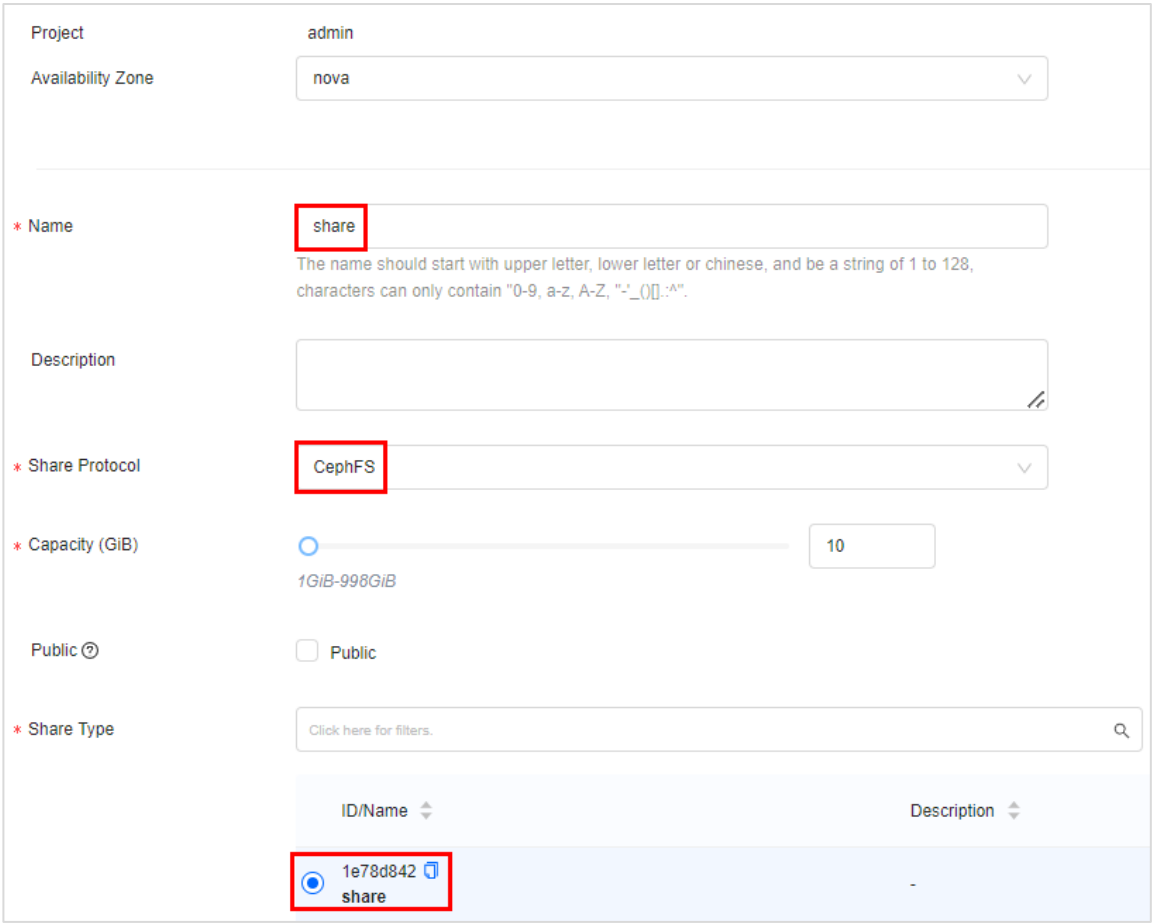
Step 2 Create shared file storage.

Copyright © Huawei Technologies Co.,Ltd

On the **Console** page, choose **Share File Storage** > **Shares**. On the page that is displayed, click **Create Share**.



Create a file share as follows:

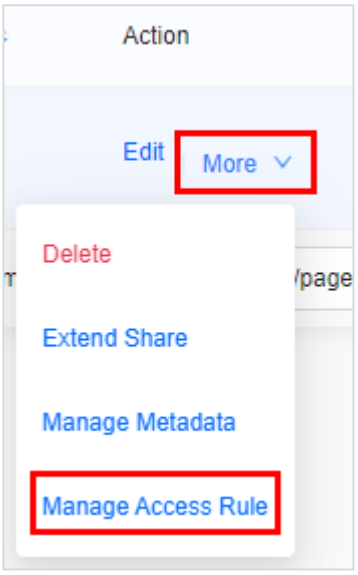


Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

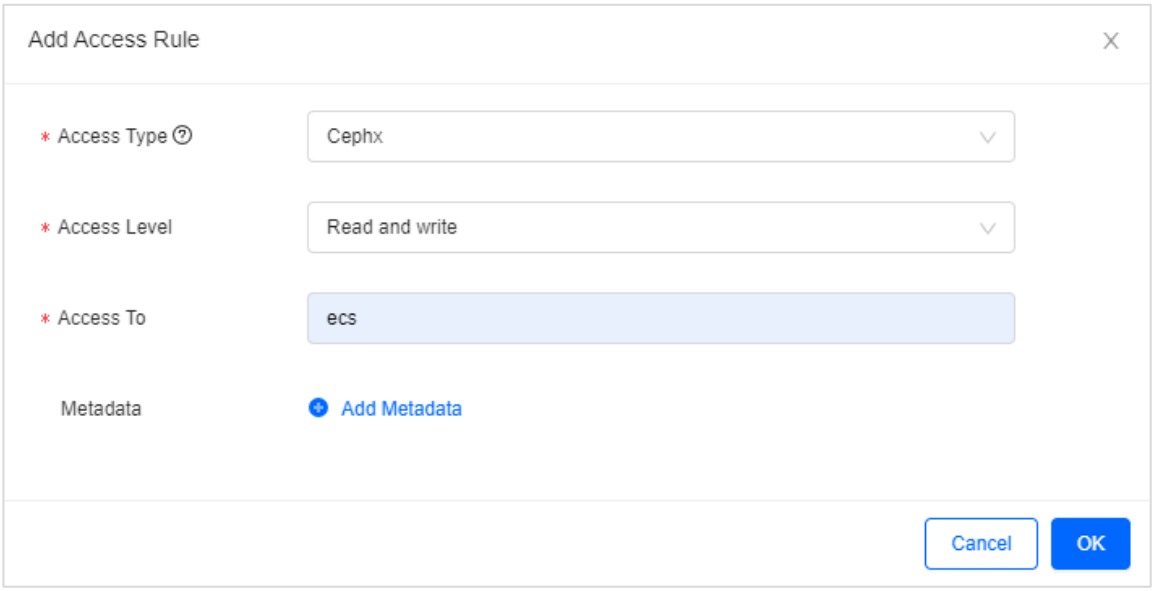
Click **OK**.

Step 3 Configure the access rule.

After the share is created, click **More** and select **Manage Access Rule**.



On the page that is displayed, click **Add Access Rule** and set the parameters as follows:



Click **OK**.

Step 4 Mount the shared file storage.

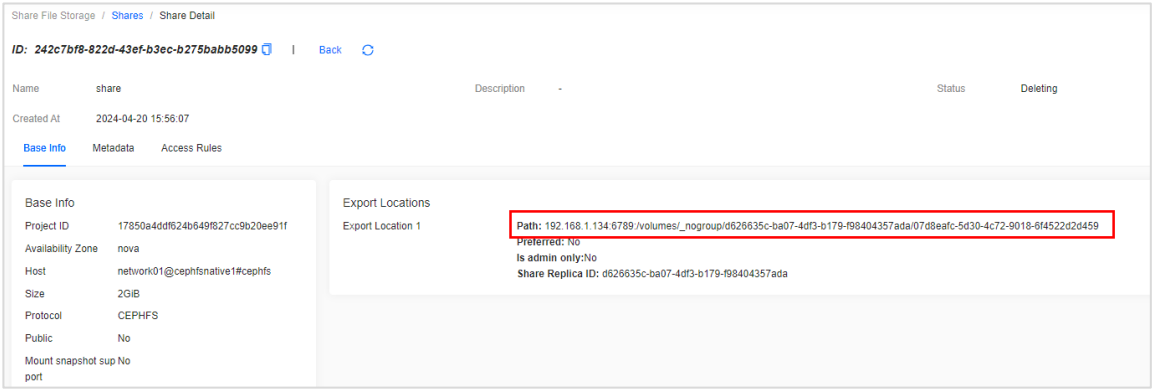
Create the **ecs** user in Ceph and export the keyring file.

```
ceph auth caps client.manila mon 'allow rwx' mgr 'allow rw'
ceph auth get client.manila -o /etc/ceph/ceph.client.manila.keyring
ceph auth get-or-create --name client.manila --keyring /etc/ceph/ceph.client.manila.keyring client.ecs > ecs.keyring
```

Save the exported **ecs.keyring** file to **ECS1**.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Click the share created in step 3 to open the share details page.



Record the path except the IP address and port number, for example, **/volumes/_nogroup/d626635c-ba07-4df3-b179-f98404357ada/07d8eafc-5d30-4c72-9018-6f4522d2d459**.

Run the following command on **ECS1**:

```
ceph-fuse --name client.ecs --keyring /etc/ceph/ecs.keyring --conf /etc/ceph/ceph.conf --client_mountpoint /volumes/_nogroup/d626635c-ba07-4df3-b179-f98404357ada/07d8eafc-5d30-4c72-9018-6f4522d2d459 /mnt
```

If the following information is displayed, the shared file storage is mounted successfully:

```
[root@ecs1 ~]# ceph-fuse --name client.ecs --keyring /etc/ceph/ecs.keyring --conf /etc/ce
2d2d459 /mnt
ceph-fuse[8171]: starting ceph client
2024-04-20T08:11:41.103+0000 7fc44e2b2080 -1 init, newargv = 0x55b5bce94740 newargc=15
ceph-fuse[8171]: starting fuse
```

Add the following content to the **/etc/fstab** file on **ECS1** to enable automatic mounting of the shared file storage during system startup:

```
name=client.ecs,keyring=/etc/ceph/ecs.keyring,conf=/etc/ceph/ceph.conf,client_mountpoint=/volumes/_nogroup/d626635c-ba07-4df3-b179-f98404357ada/07d8eafc-5d30-4c72-9018-6f4522d2d459 /mnt fuse.ceph defaults,_netdev 0 0
```

------End------

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Huawei openEuler Certification Training

HCIE-openEuler

EFK Logging

Lab Guide

ISSUE: 1.0



HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Cloud Computing Technologies Co., Ltd.

Trademarks and Permissions

 and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Our programs cover all ICT fields and follow the industry's trend of ICT convergence. With our leading talent development system and certification standards, we are committed to fostering new digital ICT talent and building a sound ICT talent ecosystem.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. HCIE-openEuler certification covers the Ceph cloud storage management, the cloud operating system (OS) cluster management, advanced practices of cloud native platform Kubernetes, advanced automation management, continuous integration and continuous delivery (CI/CD), intelligent operation and maintenance (O&M) platform A-Ops, and log management system EFK.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

About This Document

Overview

This document is an HCIE-openEuler certification training course and is intended for trainees who are going to take the HCIE-openEuler exams or readers who want to know how to use openEuler to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management.

Description

This lab guide consists of two exercises, covering the setup and implementation of log analysis and dev tools within a Kubernetes cluster using EFK.

- Exercise 1: EFK installation. By installing Elasticsearch, Fluented, and Kibana, you can get familiar with the principles of the EFK logging solution.
- Exercise 2: EFK usage. This exercise guides you through configuring and using log analysis and dev tools within the EFK stack, enabling you to understand and utilize the basic EFK workflow.

Background Knowledge Required

This course is for Huawei's certification. To better understand this course, you need to be familiar with:

- Have basic Linux knowledge. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Lab Environment Overview

This lab environment is intended for expert engineers who prepare for the HCIE-openEuler exam. All exercises are performed on Huawei Cloud. Each trainee is assigned a Huawei Cloud IAM account, which contains one Kubernetes cluster consisting of three master nodes and three nodes. Each Huawei Cloud IAM account is intended for use by only one trainee.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Lab Environment Preparation

Checking Resources

Make sure that all required resources are ready before the exercises. The following table lists the resources.

Resource Name	Quantity	Remarks
PC	1	Connect the PC to the Internet.
Huawei Cloud IAM account	1	Use the IAM account allocated by the trainer.
Kubernetes cluster	1	Use the Kubernetes cluster set up in the HCIE-openEuler Kubernetes course.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
Lab Environment Overview 3
Lab Environment Preparation 4
1 Lab Environment 6
1.1 Overview 6
1.1.1 About This Lab 6
1.1.2 Networking 6
1.1.3 Resources 7
2 EFK Installation 8
2.1 Preparations for the Backend Storage and Other Resources 8
2.2 Elasticsearch Deployment 11
2.3 Kibana Deployment 15
2.4 Fluentd Deployment 19
3 EFK Usage 34
3.1 Log Analysis 34
3.2 Dev Tools 54
4 Quiz 71
5 Acronyms and Abbreviations 73

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

1 Lab Environment

1.1 Overview

1.1.1 About This Lab

This lab utilizes the open source EFK (Elasticsearch, Fluentd, and Kibana) logging solution.

You will deploy Elasticsearch, a powerful distributed search and analytics engine perfect for handling large volumes of log data, alongside Fluentd, a versatile log collector that streamlines the aggregation and transportation of logs from diverse sources. Finally, you will use Kibana, an intuitive data visualization tool, to unlock the insights within your Elasticsearch data, empowering you to analyze and monitor logs in real-time with dynamic dashboards and visualizations.

Throughout this lab, you will learn to containerize and deploy the EFK components within a Kubernetes cluster. Upon completion, you will possess the knowledge to perform essential log analysis, implement proactive alerting mechanisms, and confidently navigate the EFK ecosystem.

1.1.2 Networking

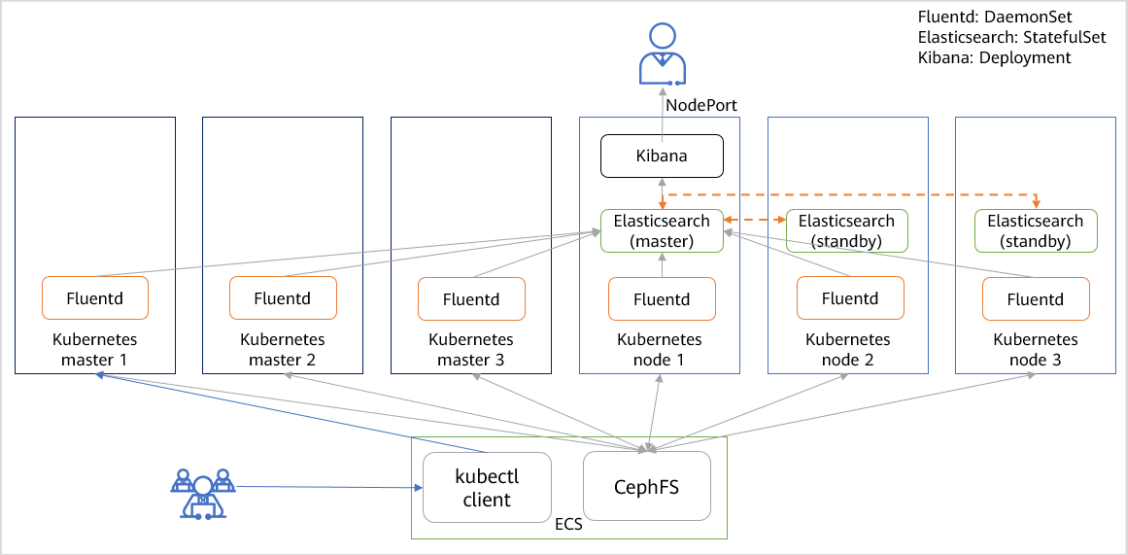


Figure 1-1 Topology for EFK logging

Copyright © Huawei Technologies Co.,Ltd

1.1.3 Resources

The following table lists the mappings among the resources, specifications, and versions.

Table 1-1 Resource mappings

Resource Name	Specifications	Software Version
ECS	c7.large.2 | 2 vCPUs | 4 GiB	openEuler 22.03 LTS
Kubernetes	Master: s7.xlarge.2 | 4 vCPUs | 8 GiB Node: s7.xlarge.2 | 4 vCPUs | 8 GiB	v1.23.1
Elasticsearch	StatefulSet	7.17.1
Kibana	Deployment	7.17.1
Fluentd	DaemonSet	3.4.1
ElastAlert	Deployment	0.2.4
PC	Local PC	Windows 10 or 11

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

2 EFK Installation

2.1 Preparations for the Backend Storage and Other Resources

Step 1 Create a resource pool in Ceph.

Create resources pools for the **efk** CephFS file system.

```
ceph osd pool create efk_data
ceph osd pool create efk_metadata
ceph fs new efk efk_metadata efk_data
ceph orch apply mds efk --placement="2"
```

Step 2 Create a namespace and directory.

To isolate resources, create a separate namespace (**logging**) for this exercise.

```
kubectl create ns logging
```

Create a directory for storing all YAML files for this exercise.

```
mkdir /root/logging
```

All subsequent files and directories will be created in **/root/logging**.

Step 3 Prepare StorageClass files.

Create directory **sc** within **/root/logging**.

```
mkdir sc
```

Copy the StorageClass YAML files to this directory.

```
cp /root/ceph/ceph-csi/deploy/cephfs/kubernetes/* ~/logging/sc/
cp /root/ceph/ceph-csi/examples/cephfs/secret.yaml ~/logging/sc/
```

Note: If you have not already, clone the Ceph CSI repository by running **git clone https://gitee.com/yftyxa/ceph-csi.git**.

Step 4 Create ConfigMaps.

You can reuse the ConfigMap YAML files created in previous exercises. Otherwise, create them using the following templates. Replace the bold text with actual configuration values. Set the namespace for these ConfigMaps to **logging**.

```
---
apiVersion: v1
kind: ConfigMap
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
metadata:
  name: "ceph-csi-config"
data:
  config.json: |-
    [
      {
        "clusterID": "88ab034c-a622-11ee-9a8c-000c2905539b",
        "monitors": [
          "192.168.1.11:6789",
          "192.168.1.12:6789",
          "192.168.1.13:6789"
        ]
      }
    ]

---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    {}
metadata:
  name: ceph-csi-encryption-kms-config

---
apiVersion: v1
kind: ConfigMap
data:
  ceph.conf: |
    [global]
    auth_cluster_required = cephx
    auth_service_required = cephx
    auth_client_required = cephx
  keyring: |
metadata:
  name: ceph-config
```

Step 5 Create a secret.

Modify the following **secret.yaml** file, replacing the bold text with your actual information. Then, deploy the secret.

```
---
apiVersion: v1
kind: Secret
metadata:
  name: efk-secret
  namespace: logging
stringData:
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
  # Required for statically provisioned volumes
  userID: efk
  userKey: AQDluLxlbrvxJxAAIe1koTjcE5fJlFIWYNC1hQ==

  # Required for dynamically provisioned volumes
  adminID: admin
  adminKey: AQBvgI5lqanTKhAAwyKQaG7i340SPTMTLBF7kQ==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

Step 6 (Optional) Deploy Ceph CSI.

You can reuse an existing Ceph CSI deployment for this exercise.

Modify the namespace information in **csi-provisioner-rbac.yaml**, **csi-nodeplugin-rbac.yaml**, **csi-rbdplugin-provisioner.yaml**, and **csi-rbdplugin.yaml** from **default** to **logging**. You are advised to use **sed** for bulk replacement and verify using **grep** to ensure no instances of **namespace: default** remain. After modifying the files, deploy Ceph CSI using the following command:

```
kubectl apply -f csi-nodeplugin-rbac.yaml -f csi-provisioner-rbac.yaml -f csi-cephfsplugin-provisioner.yaml -f csi-cephfsplugin.yaml -f csidirver.yaml
```

Step 7 Create a StorageClass.

Create a StorageClass using the following YAML file:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: efk-sc
  namespace: logging
provisioner: cephfs.csi.ceph.com
parameters:
  clusterID: 88ab034c-a622-11ee-9a8c-000c2905539b
  fsName: efk
  pool: efk_data
  csi.storage.k8s.io/provisioner-secret-name: efk-secret
  csi.storage.k8s.io/provisioner-secret-namespace: logging
  csi.storage.k8s.io/controller-expand-secret-name: efk-secret
  csi.storage.k8s.io/controller-expand-secret-namespace: logging
  csi.storage.k8s.io/node-stage-secret-name: efk-secret
  csi.storage.k8s.io/node-stage-secret-namespace: logging
  mounter: kernel
reclaimPolicy: Delete
allowVolumeExpansion: true
```

After the configuration is complete, deploy the StorageClass.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

2.2 Elasticsearch Deployment

Step 1 Create an **es** directory for storing related files.

```
[root@master01 logging]# mkdir es
[root@master01 logging]# ls
es  sc
```

Step 2 Create the service.

Create an **elasticsearch-svc.yaml** file with the following content:

```
kind: Service
apiVersion: v1
metadata:
  name: elasticsearch
  namespace: logging
  labels:
    app: elasticsearch
spec:
  selector:
    app: elasticsearch
  clusterIP: None
  ports:
    - port: 9200
      name: rest
    - port: 9300
      name: inter-node
```

Use **kubectl** to create the service resource object.

```
kubectl create -f elasticsearch-svc.yaml
kubectl get services -n logging
```

```
[root@master01 es]# kubectl create -f elasticsearch-svc.yaml
service/elasticsearch created
[root@master01 es]# kubectl get services -n logging
NAME                        TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)             AGE
csi-metrics-rbdplugin       ClusterIP   172.16.243.123   <none>        8080/TCP            20m
csi-rbdplugin-provisioner   ClusterIP   172.16.146.217   <none>        8080/TCP            32m
elasticsearch               ClusterIP   None             <none>        9200/TCP,9300/TCP   7s
[root@master01 es]#
```

Step 3 Deploy Elasticsearch.

Create an **elasticsearch-statefulset.yaml** file with the following content:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: es
  namespace: logging
spec:
  serviceName: elasticsearch
  replicas: 3
  selector:
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
    matchLabels:
      app: elasticsearch
  template:
    metadata:
      labels:
        app: elasticsearch
    spec:
      initContainers:
      - name: increase-vm-max-map
        image: busybox
        imagePullPolicy: "IfNotPresent"
        command: ["sysctl", "-w", "vm.max_map_count=262144"]
        securityContext:
          privileged: true
      - name: increase-fd-ulimit
        image: busybox
        command: ["sh", "-c", "ulimit -n 65536"]
        securityContext:
          privileged: true
      - name: fix-permissions
        image: busybox
        imagePullPolicy: "IfNotPresent"
        securityContext:
          privileged: true
        command: ["sh", "-c", "chown -R 1000:1000 /usr/share/elasticsearch/data"]
        volumeMounts:
          - name: data
            mountPath: /usr/share/elasticsearch/data
      containers:
      - name: elasticsearch
        image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/elasticsearch:7.17.1
        imagePullPolicy: "IfNotPresent"
        ports:
        - name: rest
          containerPort: 9200
        - name: inter
          containerPort: 9300
        resources:
          limits:
            cpu: 1000m
          requests:
            cpu: 1000m
        volumeMounts:
        - name: data
          mountPath: /usr/share/elasticsearch/data
        env:
        - name: cluster.name
          value: k8s-logs
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
        - name: node.name
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: cluster.initial_master_nodes
          value: "es-0,es-1,es-2"
        - name: discovery.zen.minimum_master_nodes
          value: "2"
        - name: discovery.seed_hosts
          value: "elasticsearch"
        - name: ES_JAVA_OPTS
          value: "-Xms512m -Xmx512m"
        - name: network.host
          value: "0.0.0.0"
  volumeClaimTemplates:
  - metadata:
      name: data
      labels:
        app: elasticsearch
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: efk-sc
      resources:
        requests:
          storage: 10Gi
```

Use **kubectl** to deploy Elasticsearch and wait until the deployment is complete.

```
kubectl create -f elasticsearch-statefulset.yaml
kubectl get sts -n logging
kubectl get pods -n logging
kubectl get svc -n logging
```

```
[root@master01 es]# kubectl get sts -n logging
NAME   READY   AGE
es     0/3     44s
[root@master01 es]# kubectl get pod -n logging
NAME                                        READY   STATUS             RESTARTS        AGE
csi-rbdplugin-2gzf6                         2/3     CrashLoopBackOff   10 (31s ago)    27m
csi-rbdplugin-7x5jd                         2/3     CrashLoopBackOff   10 (10s ago)    27m
csi-rbdplugin-f95pf                         2/3     CrashLoopBackOff   10 (17s ago)    27m
csi-rbdplugin-n9slr                         2/3     CrashLoopBackOff   10 (42s ago)    27m
csi-rbdplugin-provisioner-669c8649db-d9j9l  7/7     Running            0               39m
csi-rbdplugin-provisioner-669c8649db-jpk7f  7/7     Running            0               39m
csi-rbdplugin-provisioner-669c8649db-rwbln  7/7     Running            0               39m
es-0                                        0/1     Init:1/2           0               57s
```

Note: Wait until the StatefulSet and pods show a **Running** status before proceeding to the next step.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
[root@efk-terminal es]# kubectl get sts -n logging
NAME   READY   AGE
es     3/3     92s
[root@efk-terminal es]# kubectl get pods -n logging
NAME                      READY   STATUS    RESTARTS        AGE
es-0                      1/1     Running   0               3m33s
es-1                      1/1     Running   0               2m33s
es-2                      1/1     Running   0               2m30s
```

Step 4 Test the deployment.

Send a request to the RESTful API to check whether the Elasticsearch cluster is running properly. Set up a forwarding tunnel.

```
kubectl port-forward es-0 9200:9200 --namespace=logging
```

```
[root@efk-terminal es]# kubectl port-forward es-0 9200:9200 --namespace=logging
Forwarding from 127.0.0.1:9200 -> 9200
Forwarding from [::1]:9200 -> 9200
```

Open a new terminal and log in to the master node.

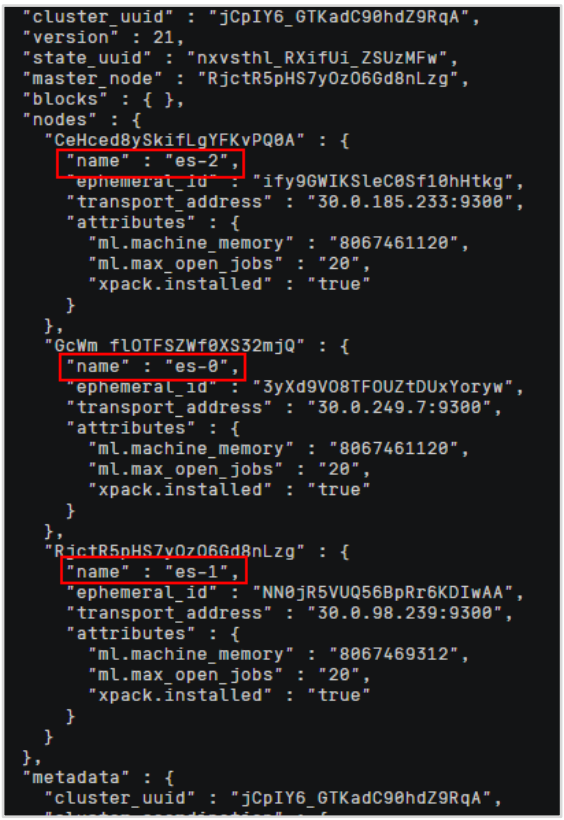
Send a request to the RESTful API of the Elasticsearch cluster and save the output to a .txt file.

```
curl -o test.txt http://localhost:9200/_cluster/state?pretty
```

```
[root@efk-terminal ~]# curl -o test.txt http://localhost:9200/_cluster/state?pretty
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  464k  100  464k    0     0  24.9M      0 --:--:-- --:--:-- --:--:-- 25.1M
[root@efk-terminal ~]#
```

View the **test.txt** file. It should contain information about the three Elasticsearch cluster nodes.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



2.3 Kibana Deployment

Step 1 Create a directory for storing Kibana YAML files.

```
[root@master logging]# mkdir kibana
[root@master logging]# cd kibana/
[root@master kibana]# pwd
/root/logging/kibana
```

Step 2 Deploy Kibana.

Create a **kibana.yaml** file with the following content:

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: logging
  name: kibana-config
  labels:
    app: kibana
data:
  kibana.yml: |
    server.name: kibana
    server.host: "0.0.0.0"
    i18n.locale: en                    # Sets the default language to English.
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
    elasticsearch:
      hosts: ${ELASTICSEARCH_HOSTS}            # Elasticsearch cluster connection address
---
apiVersion: v1
kind: Service
metadata:
  name: kibana
  namespace: logging
  labels:
    app: kibana
spec:
  ports:
  - port: 5601
  type: NodePort
  selector:
    app: kibana

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
  namespace: logging
  labels:
    app: kibana
spec:
  selector:
    matchLabels:
      app: kibana
  template:
    metadata:
      labels:
        app: kibana
    spec:
      nodeSelector:
        app: service
      containers:
      - name: kibana
        image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kibana:7.17.1
        imagePullPolicy: "IfNotPresent"
        resources:
          limits:
            cpu: 1000m
          requests:
            cpu: 1000m
        env:
        - name: ELASTICSEARCH_URL
          value: http://elasticsearch:9200        # Elasticsearch headless service name
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
        - name: ELASTICSEARCH_HOSTS
          value: http://elasticsearch:9200        # Elasticsearch headless service name
        ports:
        - containerPort: 5601
        volumeMounts:
        - name: config
          mountPath: /usr/share/kibana/config/kibana.yml       # Kibana configuration file mount path
          readOnly: true
          subPath: kibana.yml
      volumes:
      - name: config                  # Match the name in "volumeMounts."
        configMap:
          name: kibana-config                                  # Match the ConfigMap name.
```

Note: A headless service has been created when Elasticsearch is deployed. The name of the service can be used to connect Kibana to Elasticsearch. The DNS in the Kubernetes cluster automatically resolves the address.

```
[root@efk-terminal kibana]# kubectl get svc -n logging
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)             AGE
elasticsearch   ClusterIP   None           <none>        9200/TCP,9300/TCP   7d22h
```

Use **kubectl** to create Kibana resources and view the resources.

```
kubectl create -f kibana.yaml
kubectl get pods --namespace=logging
kubectl get svc -n logging
```

```
[root@efk-terminal kibana]# kubectl create -f kibana.yaml
configmap/kibana-config created
service/kibana created
deployment.apps/kibana created
[root@efk-terminal kibana]# kubectl get pods -n logging
NAME                      READY   STATUS              RESTARTS   AGE
es-0                      1/1     Running             0          51m
es-1                      1/1     Running             0          50m
es-2                      1/1     Running             0          50m
kibana-7ddbcb78f8-j6qzv   0/1     ContainerCreating   0          15s
[root@efk-terminal kibana]# kubectl get svc -n logging
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)             AGE
elasticsearch   ClusterIP   None           <none>        9200/TCP,9300/TCP   58m
kibana          NodePort    [redacted]6    <none>        5601:31298/TCP      25s
[root@efk-terminal kibana]#
```

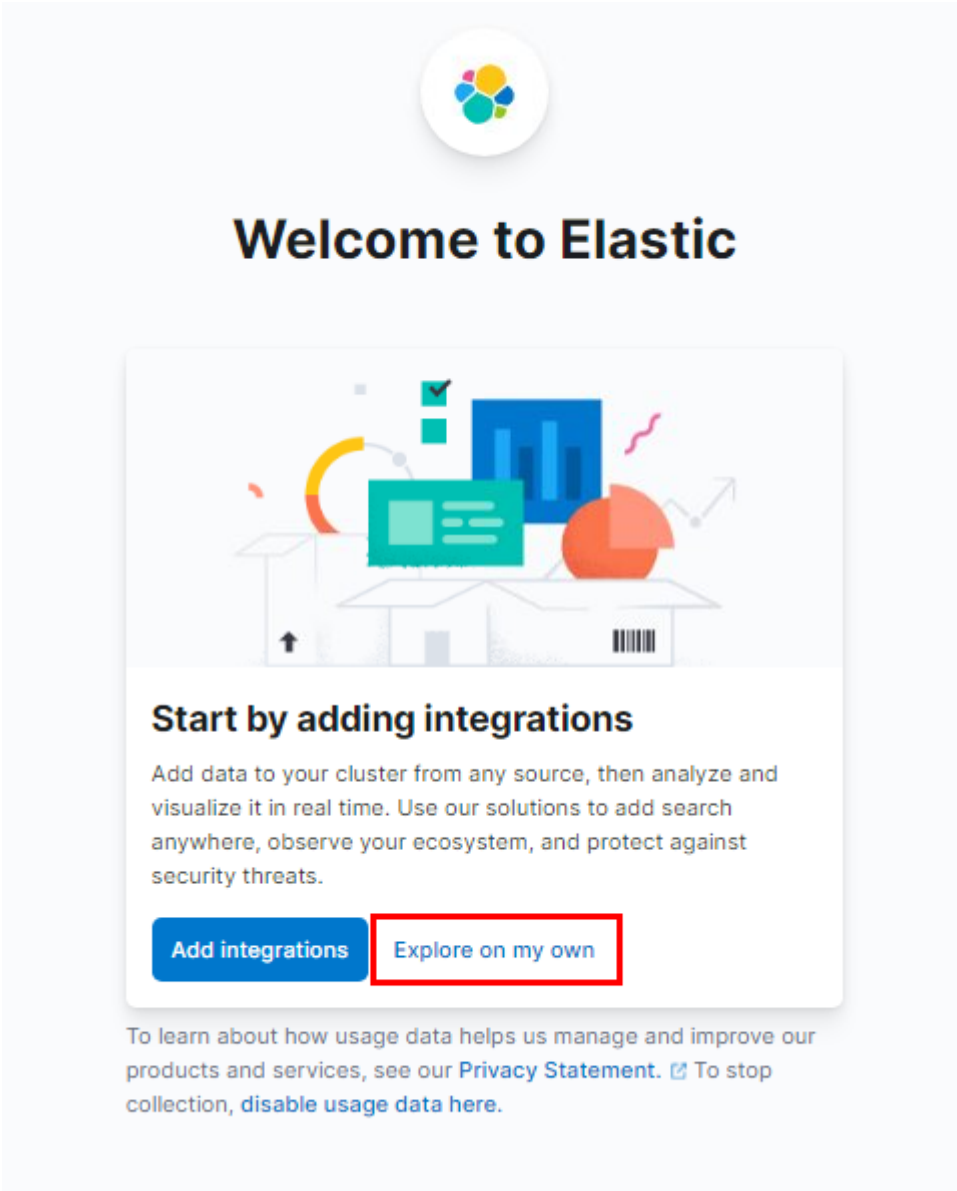
Wait for 10 to 20 seconds until the Kibana pod shows a **Running** status.

```
[root@efk-terminal kibana]# kubectl get pods -n logging
NAME                      READY   STATUS    RESTARTS   AGE
es-0                      1/1     Running   0          52m
es-1                      1/1     Running   0          51m
es-2                      1/1     Running   0          51m
kibana-7ddbcb78f8-j6qzv   1/1     Running   0          58s
[root@efk-terminal kibana]#
```

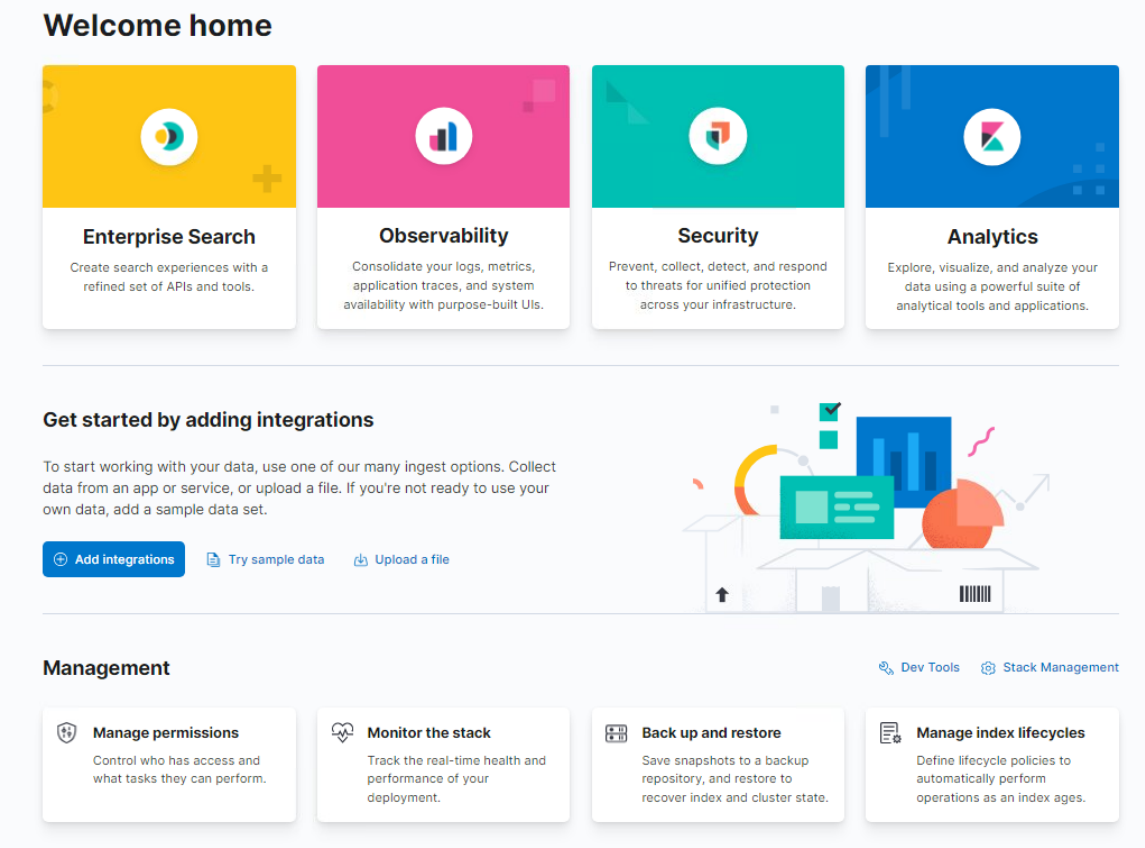
Step 3 Test the deployment.

Access Kibana through the NodePort assigned to the service. Open **http://***<EIP_of_any_node>***:31298** in a browser.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Copyright © Huawei Technologies Co.,Ltd



Note:

1. The Kubernetes nodes need an EIP for external access.
2. Port 31298 is randomly assigned by Kibana. Replace it with the actual one.
3. Add the NodePort assigned by Kibana to the inbound rules of the security group.
4. Upon first access, Kibana will not have any **Integrations** configured. Click **Explore on my own** to proceed to the homepage.

2.4 Fluentd Deployment

Step 1 Create a directory for storing Fluentd YAML files.

```
[root@master logging]# mkdir fluente
[root@master logging]# cd fluented/
[root@master fluented]# pwd
/root/logging/fluented
```

Step 2 Deploy Fluentd.

Create a **fluentd-configmap.yaml** file with the following content:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: fluentd-config
```


Copyright © Huawei Technologies Co.,Ltd

```
  namespace: logging
data:
  system.conf: |-
    <system>
      root_dir /tmp/fluentd-buffers/
    </system>
  containers.input.conf: |-
    <source>
      @id fluentd-containers.log
      @type tail                                  # Fluentd built-in input type
      path /var/log/containers/*.log              # Mounted Docker container log directory
      pos_file /var/log/es-containers.log.pos
      tag raw.kubernetes.*                        # Sets the log tag.
      read_from_head true
      <parse>                                     # Formats multiple lines into JSON.
        @type multi_format                        # Uses the multi-format-parser plugin.
        <pattern>
          format json                             # JSON parser
          time_key time                           # Specifies the "time" field for the event time.
          time_format %Y-%m-%dT%H:%M:%S.%NZ     # Time format
        </pattern>
        <pattern>
          format /^(?<time>.+) (?<stream>stdout|stderr) [^ ]* (?<log>.*)$/
          time_format %Y-%m-%dT%H:%M:%S.%N%:z
        </pattern>
      </parse>
    </source>
    # Detects exceptions in log output and forwards them as a log entry.
    # https://github.com/GoogleCloudPlatform/fluent-plugin-detect-exceptions
    <match raw.kubernetes.**>              # Matches log information with the "raw.kubernetes.**" tag.
      @id raw.kubernetes
      @type detect_exceptions              # Uses the detect-exceptions plugin to process exception
stack information.
      remove_tag_prefix raw                # Removes the "raw" prefix from the tag.
      message log
      stream stream
      multiline_flush_interval 5
      max_bytes 500000
      max_lines 1000
    </match>

    <filter **>  # Concatenates logs.
      @id filter_concat
      @type concat                    # Fluentd Filter plugin used to concatenate multiline logs split
across multiple events
      key message
      multiline_end_regexp /\n$/   # Concatenates lines using newline character "\n" as the delimiter.
      separator ""
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
    </filter>

    # Adds Kubernetes metadata data.
    <filter kubernetes.**>
      @id filter_kubernetes_metadata
      @type kubernetes_metadata
    </filter>

    # Repairs JSON fields in Elasticsearch.
    # Plugin address: https://github.com/repeatedly/fluent-plugin-multi-format-parser
    <filter kubernetes.**>
      @id filter_parser
      @type parser                    # multi-format-parser plugin
      key_name log                    # Field name to parse within the log record
      reserve_data true               # Retains original key-value pairs in parsed results.
      remove_key_name_field true      # Removes the "key_name" field after successful parsing.
      <parse>
        @type multi_format
        <pattern>
          format json
        </pattern>
        <pattern>
          format none
        </pattern>
      </parse>
    </filter>

    # Removes some redundant attributes.
    <filter kubernetes.**>
      @type record_transformer
      remove_keys
$.docker.container_id,$.kubernetes.container_image_id,$.kubernetes.pod_id,$.kubernetes.namespace_id,$.
kubernetes.master_url,$.kubernetes.labels.pod-template-hash
    </filter>

    # Keeps only the logs of pods with the "logging=true" label.
    <filter kubernetes.**>
      @id filter_log
      @type grep
      <regexp>
        key $.kubernetes.labels.logging
        pattern ^true$
      </regexp>
    </filter>

  ###### Listening configuration, generally used for log aggregation ######
  forward.input.conf: |-
    # Listens for messages sent via TCP.
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
    <source>
      @id forward
      @type forward
    </source>

  output.conf: |-
    <match **>
      @id elasticsearch
      @type elasticsearch
      @log_level info
      include_tag_key true
      host elasticsearch
      port 9200
      logstash_format true
      logstash_prefix k8s   # Sets the index prefix to "k8s".
      request_timeout 30s
      <buffer>
        @type file
        path /var/log/fluentd-buffers/kubernetes.system.buffer
        flush_mode interval
        retry_type exponential_backoff
        flush_thread_count 2
        flush_interval 5s
        retry_forever
        retry_max_interval 30
        chunk_limit_size 2M
        queue_limit_length 8
        overflow_action block
      </buffer>
    </match>
```

Create a **fluentd-daemonset.yaml** file with the following content:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: fluentd-es
  namespace: logging
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: fluentd-es
  labels:
    k8s-app: fluentd-es
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
rules:
- apiGroups:
  - ""
  resources:
  - "namespaces"
  - "pods"
  verbs:
  - "get"
  - "watch"
  - "list"
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: fluentd-es
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
subjects:
- kind: ServiceAccount
  name: fluentd-es
  namespace: logging
  apiGroup: ""
roleRef:
  kind: ClusterRole
  name: fluentd-es
  apiGroup: ""
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-es
  namespace: logging
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
spec:
  selector:
    matchLabels:
      k8s-app: fluentd-es
  template:
    metadata:
      labels:
        k8s-app: fluentd-es
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
        kubernetes.io/cluster-service: "true"
    # This annotation ensures that Fluentd is not evicted even if the node is evicted, supporting critical pod annotation-based priority schemes.
      annotations:
        scheduler.alpha.kubernetes.io/critical-pod: ''
    spec:
      serviceAccountName: fluentd-es
      containers:
      - name: fluentd-es
        image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/fluentd:v3.4.0
        env:
        - name: FLUENTD_ARGS
          value: --no-supervisor -q
        resources:
          limits:
            memory: 500Mi
          requests:
            cpu: 100m
            memory: 200Mi
        volumeMounts:
        - name: varlog
          mountPath: /var/log
        - name: varlibdockercontainers
          mountPath: /var/lib/docker/containers
          readOnly: true
          readOnly: true
        - name: config-volume
          mountPath: /etc/fluent/config.d
      #nodeSelector:
      #   beta.kubernetes.io/fluentd-ds-ready: "true"
      tolerations:
      - operator: Exists
      terminationGracePeriodSeconds: 30
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
      - name: varlibdockercontainers
        hostPath:
          path: /var/lib/docker/containers
      - name: config-volume
        configMap:
          name: fluentd-config
```

Create the ConfigMap object and DaemonSet.

```
kubectl create -f fluentd-configmap.yaml
kubectl create -f fluentd-daemonset.yaml
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
[root@efk-terminal fluented]# kubectl create -f fluentd-configmap.yaml
configmap/fluentd-config created
[root@efk-terminal fluented]# kubectl create -f fluentd-daemonset.yaml
serviceaccount/fluentd-es created
clusterrole.rbac.authorization.k8s.io/fluentd-es created
clusterrolebinding.rbac.authorization.k8s.io/fluentd-es created
Warning: spec.template.metadata.annotations[scheduler.alpha.kubernetes.io/critical-pod]: non-functional in v1.16+; use the "priorityClassName" field instead
daemonset.apps/fluentd-es created
[root@efk-terminal fluented]#
```

After the creation is complete, view the pod list to check whether the deployment is successful.

```
kubectl get pods -n logging
```

```
[root@efk-terminal fluented]# kubectl get pods -n logging
NAME                      READY   STATUS    RESTARTS   AGE
es-0                      1/1     Running   0          5m5s
es-1                      1/1     Running   0          5m
es-2                      1/1     Running   0          4m55s
fluentd-es-gkbhb          1/1     Running   0          11s
fluentd-es-h9ltp          1/1     Running   0          11s
fluentd-es-kpzgl          1/1     Running   0          11s
fluentd-es-ntqtp          1/1     Running   0          11s
fluentd-es-tg5cc          1/1     Running   0          11s
fluentd-es-z4t96          1/1     Running   0          11s
kibana-7ddbcb78f8-h57z5   1/1     Running   0          3m44s
[root@efk-terminal fluented]#
```

Note: Six Fluentd pods are distributed on the three masters and three nodes of the Kubernetes cluster.

Step 3 Test the deployment.

Create a **counter.yaml** file with the following content to create a test application:

```
apiVersion: v1
kind: Pod
metadata:
  name: counter
  labels:
    logging: "true"    # Only logs of pods with this label are collected.
spec:
  containers:
  - name: count
    image: busybox
    args: [/bin/sh, -c,
            'i=0; while true; do echo "$i: $(date)"; i=$((i+1)); sleep 1; done']
```

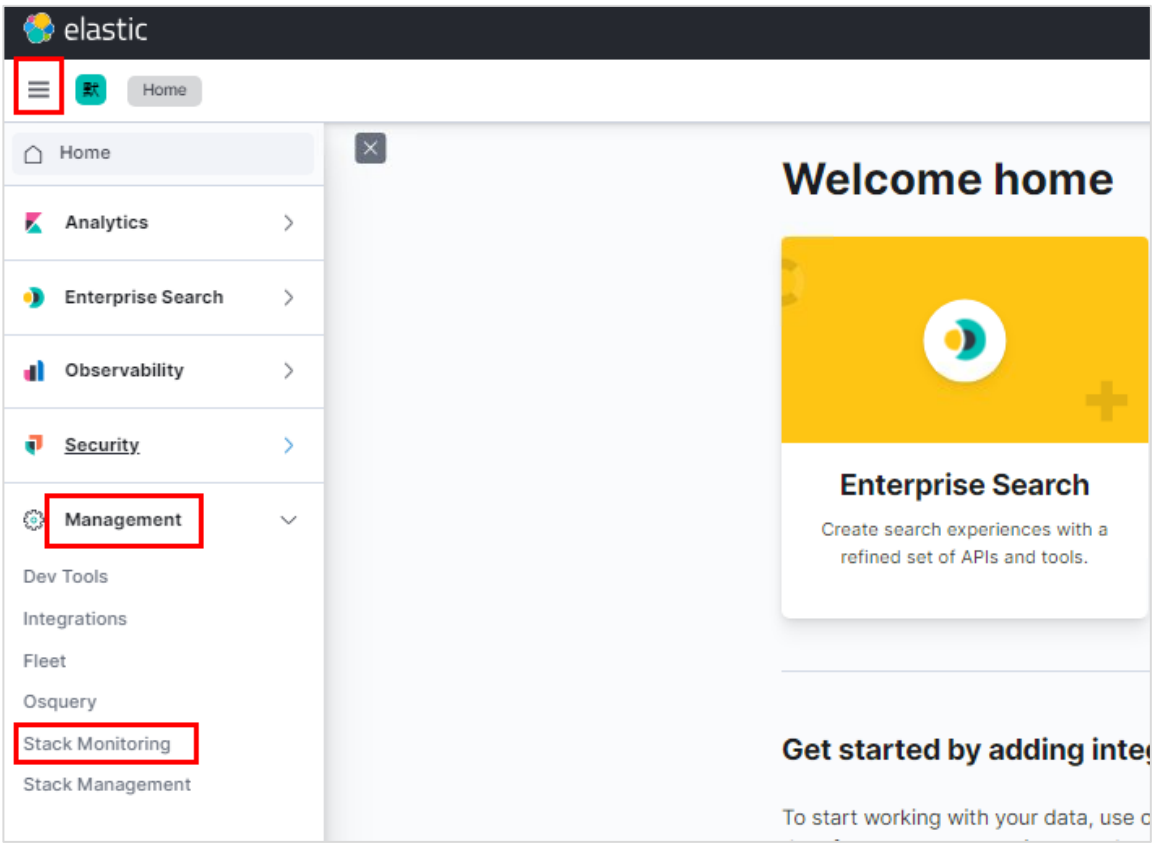
Use **kubectl** to deploy the pod and verify its status.

```
kubectl create -f counter.yaml
kubectl get pods
```

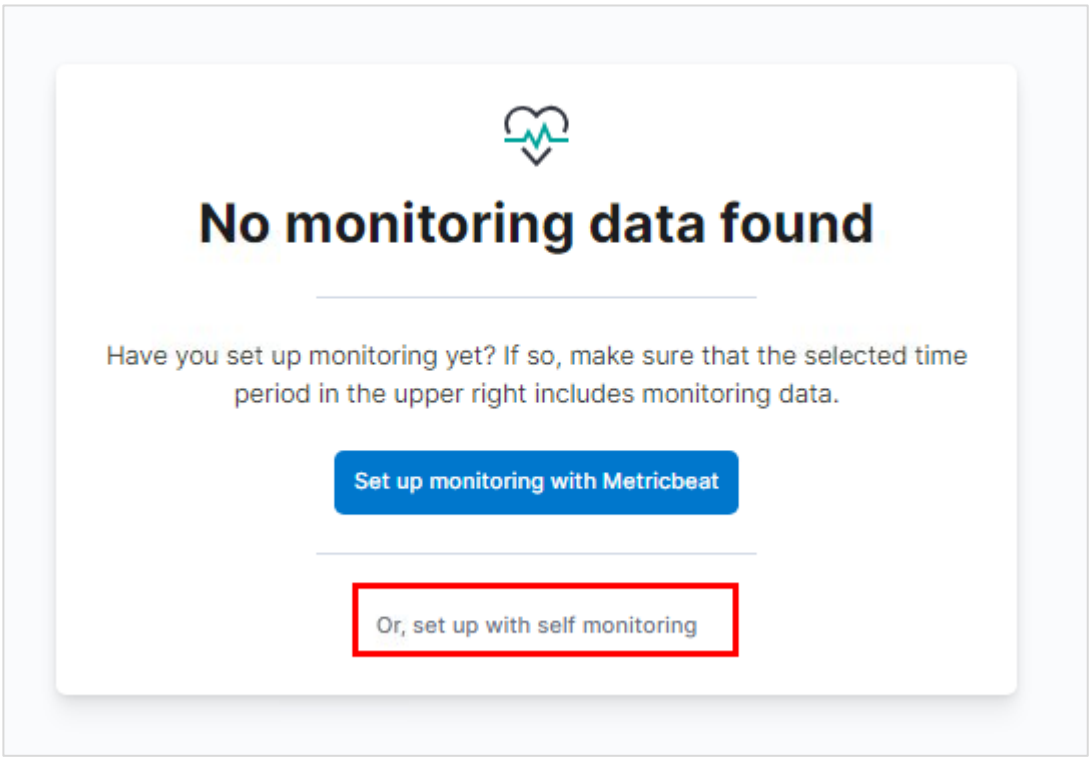
```
[root@efk-terminal fluented]# kubectl create -f counter.yaml
pod/counter created
[root@efk-terminal fluented]# kubectl get pods
NAME                                  READY   STATUS    RESTARTS        AGE
cephfs-pod1                           1/1     Running   0               165m
counter                               1/1     Running   0               5s
nfs-client-provisioner-8dd958dc8-b9s7s   1/1     Running   0               137m
rbd-provisioner-7744c688c8-mvc6b      1/1     Running   4 (7h57m ago)   3d20h
[root@efk-terminal fluented]#
```

Navigate to the Kibana dashboard and click **Stack Monitoring** in the left navigation bar.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

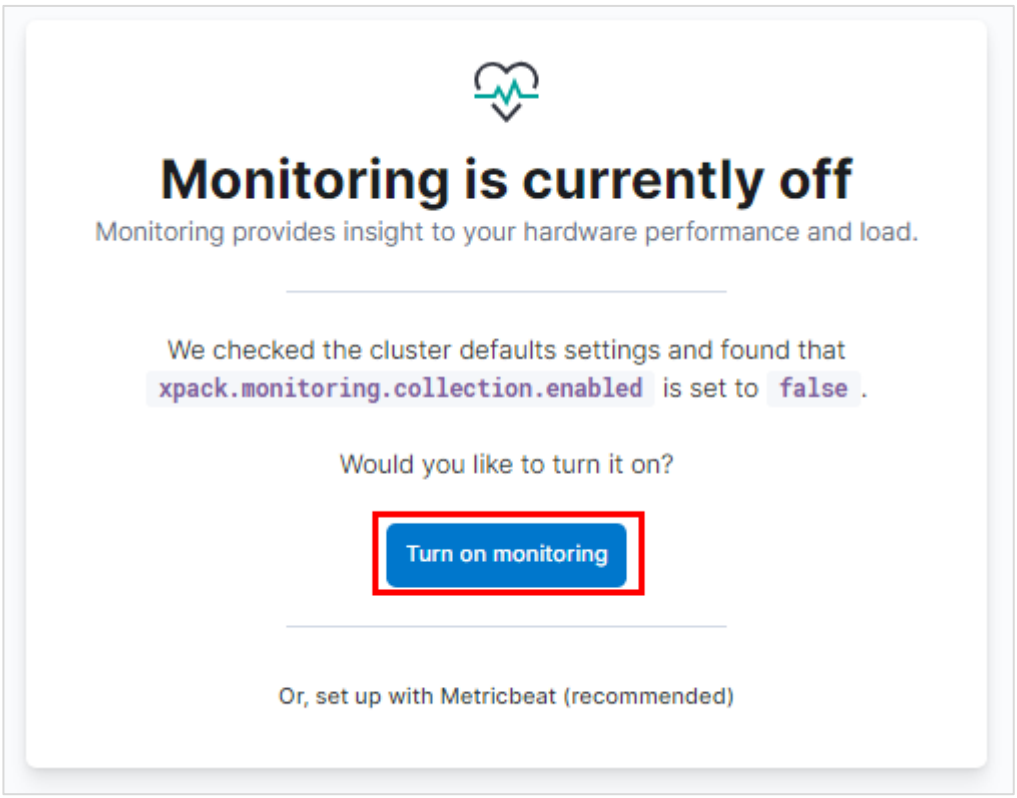


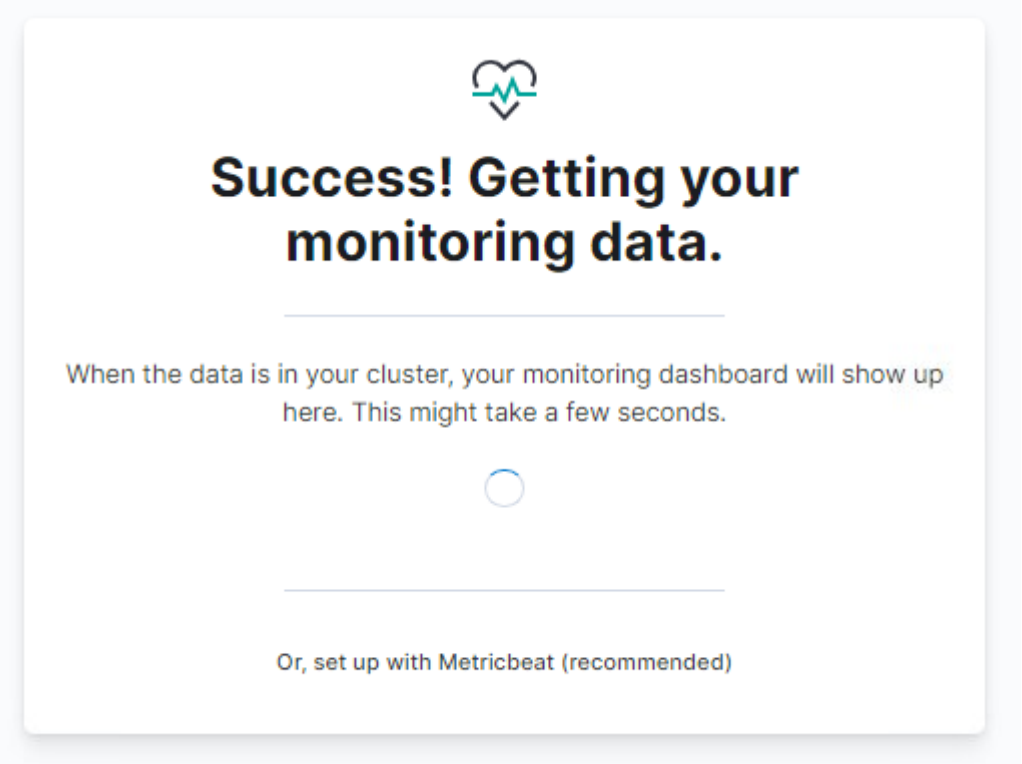
On the **Stack Monitoring** page, click **Or, set up with self monitoring**.



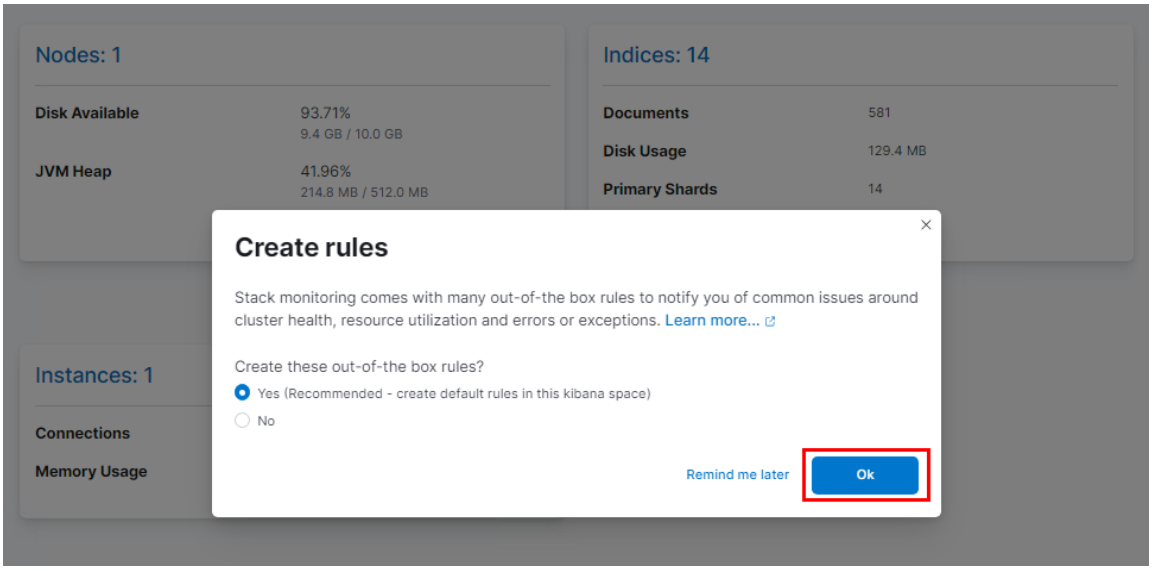
Click **Turn on monitoring** and wait for approximately 10 seconds. When a dialog box appears, click **OK** to confirm.

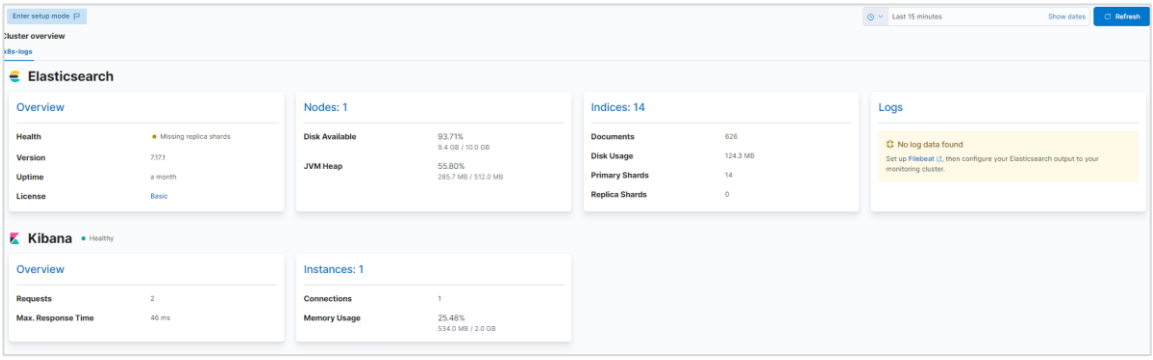
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd





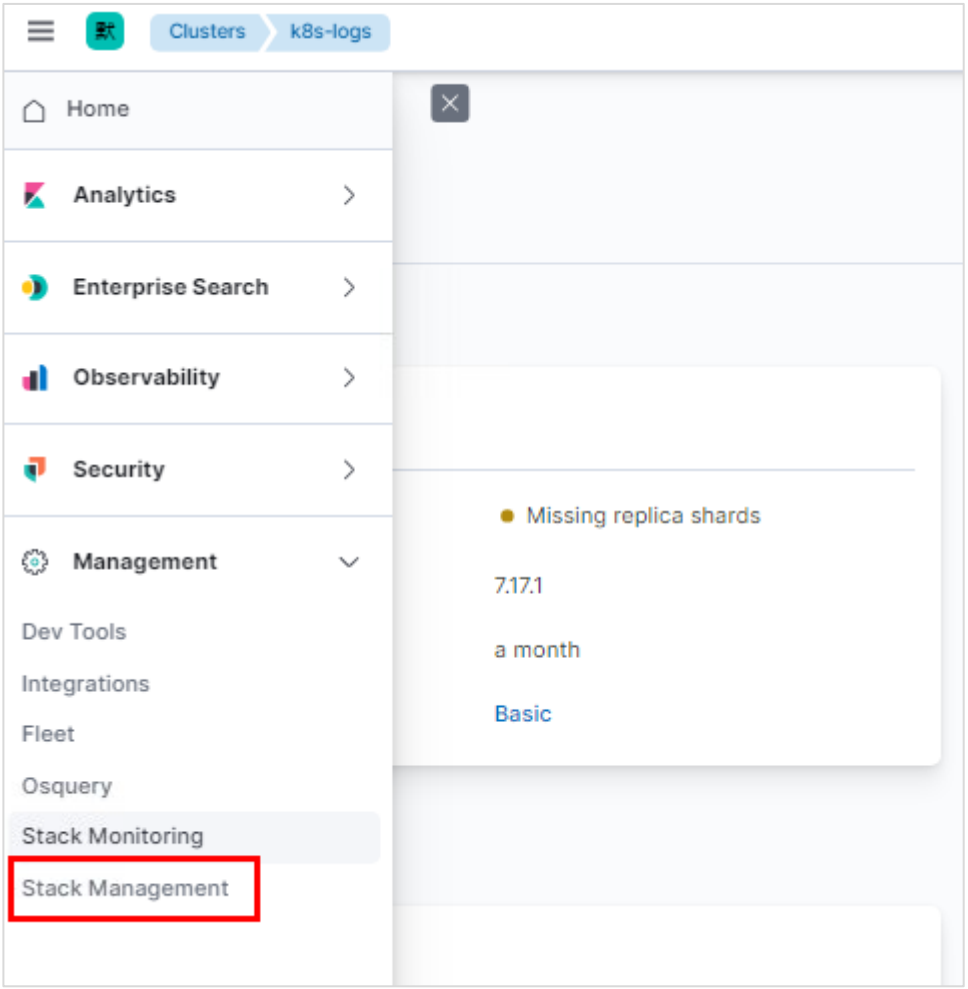
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd





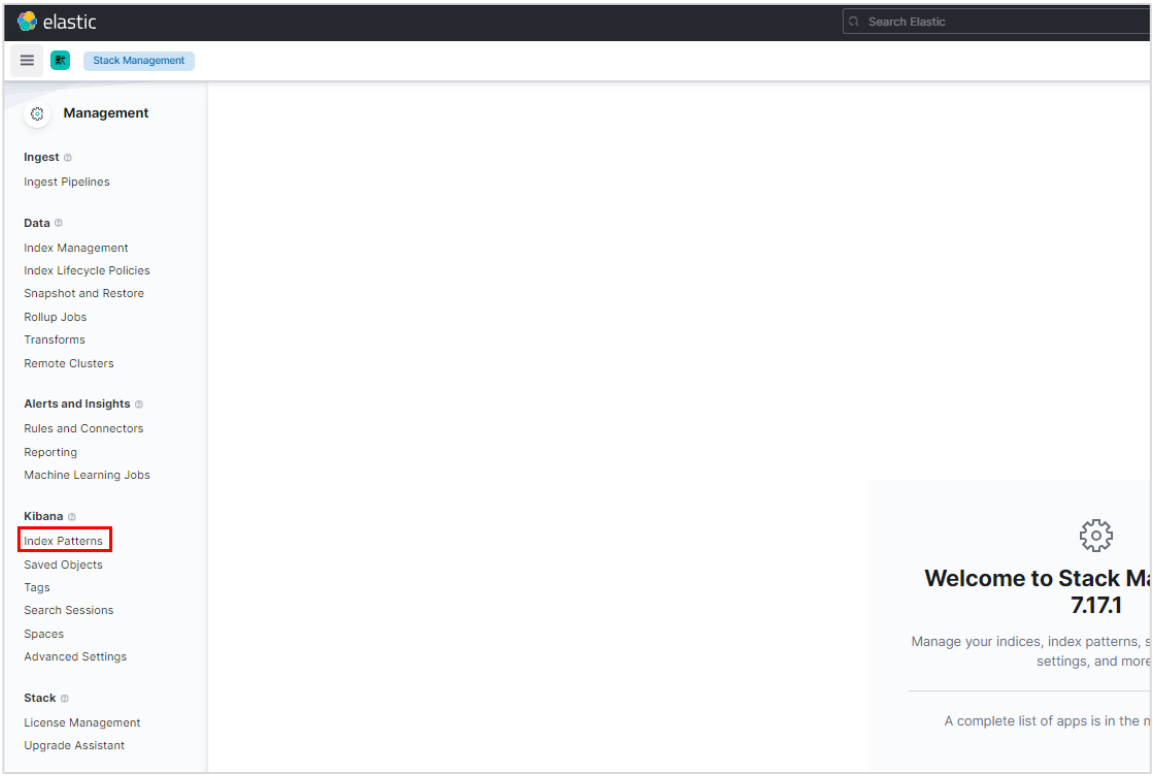
In the left navigation bar, click **Stack Management** at the bottom.

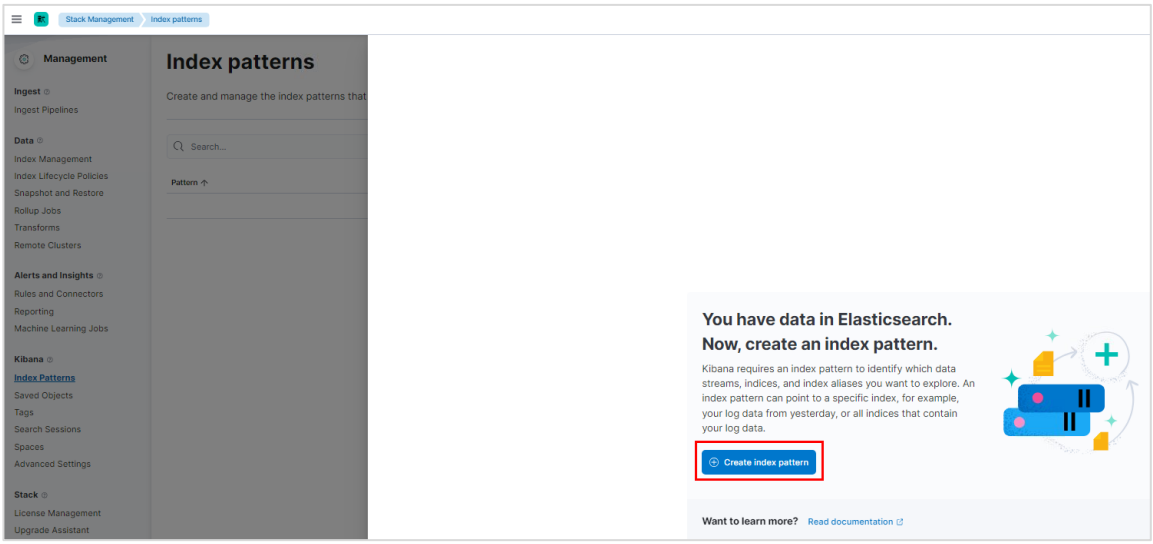
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Choose **Kibana** > **Index Patterns** > **Create index pattern**.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

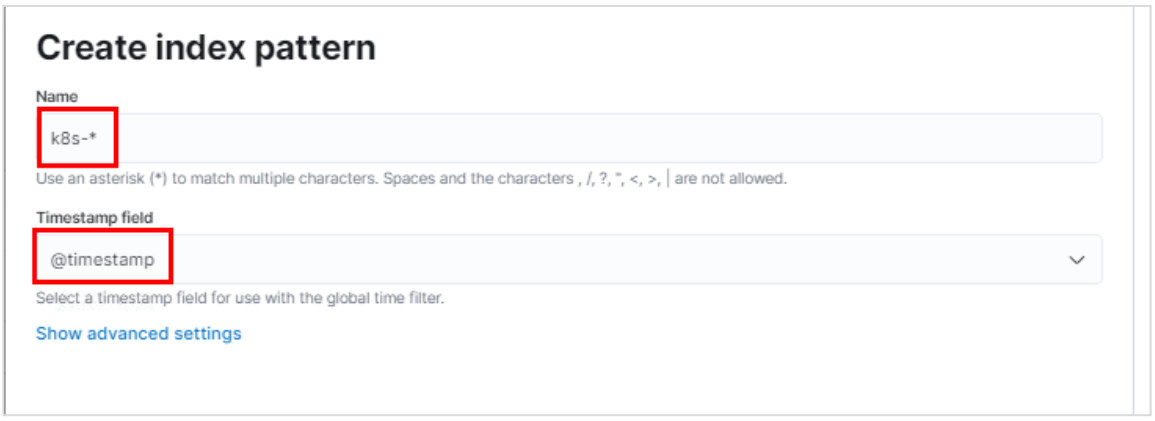


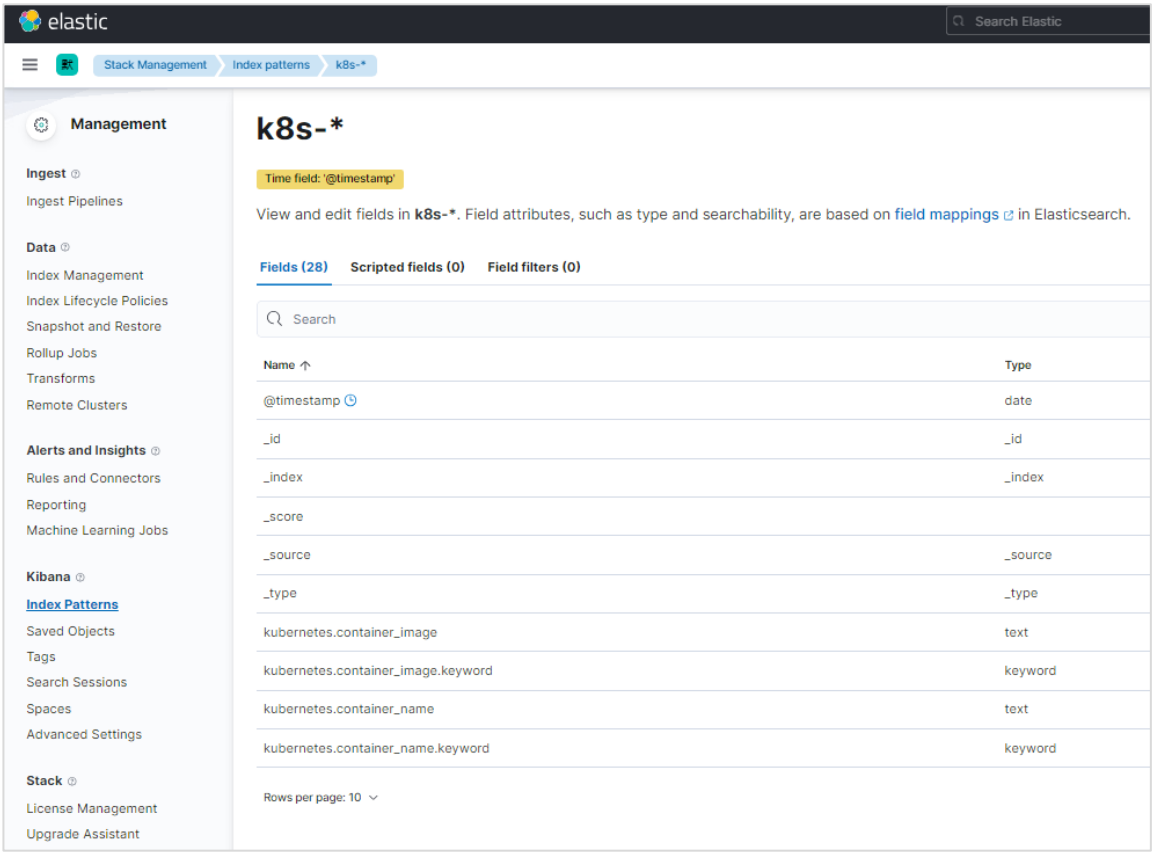


Create an index pattern using the following parameters:

- Name: **k8s-***
- Timestamp field: **@timestamp**

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd





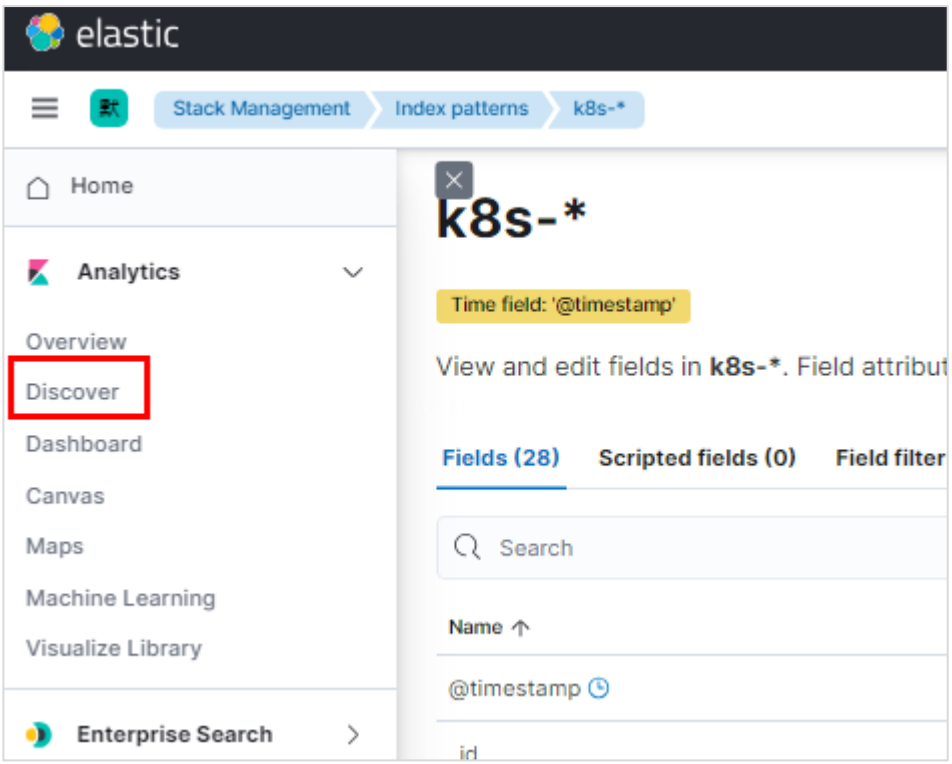
Note: The **k8s** index prefix is defined in the Fluentd ConfigMap.

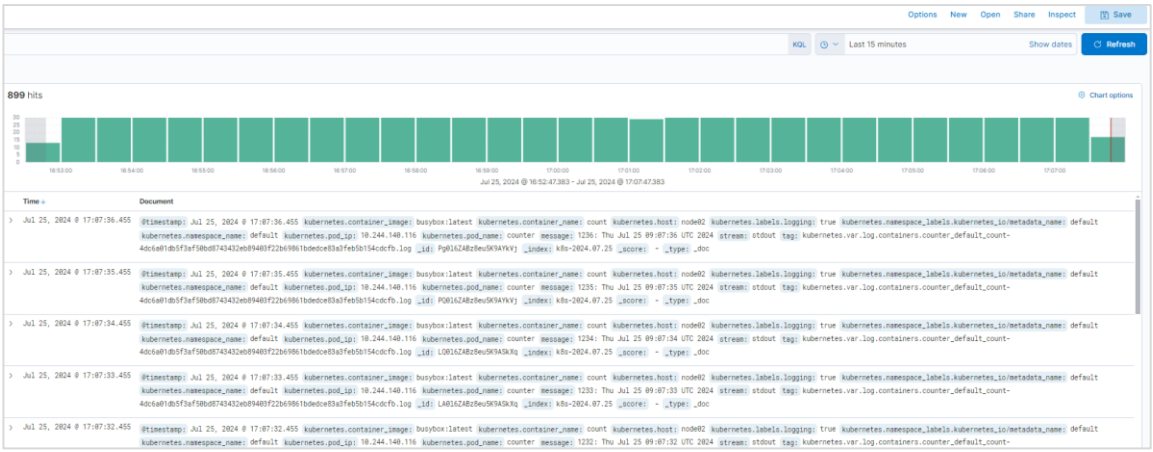
```
output.conf: |-
  <match **>
    @id elasticsearch
    @type elasticsearch
    @log_level info
    include_tag_key true
    host elasticsearch
    port 9200
    logstash_format true
    logstash_prefix k8s    # Sets the index prefix to "k8s".
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
request_timeout 30s
```

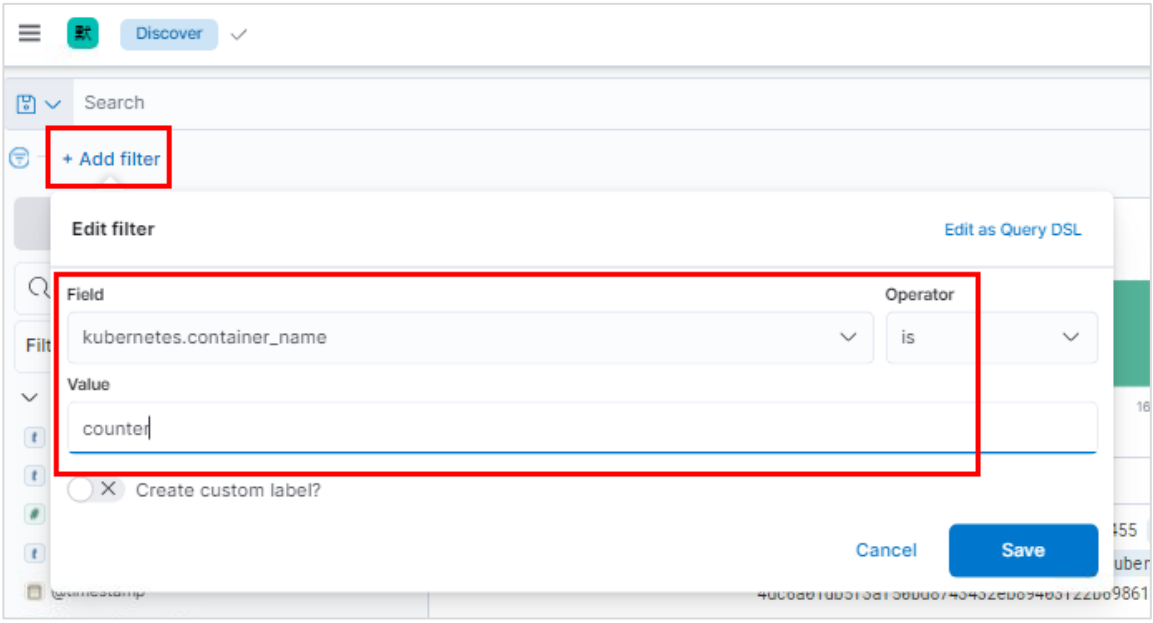
Click **Discover** in the left navigation bar. You should see the logs collected by Fluentd and visualized by Kibana through Elasticsearch.





Note: If there are numerous log entries, use the **Add filter** option at the top left to refine your search. Since this example uses a single counter pod, filtering is not necessary.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Return to the CLI and delete the test application.

```
kubectl delete -f counter.yaml
kubectl get pods
```

```
[root@efk-terminal fluented]# kubectl delete -f counter.yaml
pod "counter" deleted


[root@efk-terminal fluented]# kubectl get pods
NAME                                        READY   STATUS        RESTARTS         AGE
counter                                     1/1     Terminating   0                7m41s
csi-cephfsplugin-82h6p                      3/3     Running       9 (5h14m ago)    2d21h
csi-cephfsplugin-9n5j9                      3/3     Running       9 (5h14m ago)    2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-4lq2f   5/5     Running       15 (5h14m ago)   2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-4q2x5   5/5     Running       15 (5h14m ago)   2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-vkmlg   5/5     Running       15 (5h14m ago)   2d21h
csi-cephfsplugin-t29vq                      3/3     Running       9 (5h14m ago)    2d21h
nfs-client-provisioner-85f48d4f88-vhkxd     1/1     Running       0                56m
nginx-85b98978db-t8znx                      1/1     Running       4 (5h14m ago)    8d
[root@efk-terminal fluented]# kubectl get pods
NAME                                        READY   STATUS    RESTARTS         AGE
csi-cephfsplugin-82h6p                      3/3     Running   9 (5h15m ago)    2d21h
csi-cephfsplugin-9n5j9                      3/3     Running   9 (5h15m ago)    2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-4lq2f   5/5     Running   15 (5h15m ago)   2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-4q2x5   5/5     Running   15 (5h15m ago)   2d21h
csi-cephfsplugin-provisioner-5fcd7fb7dd-vkmlg   5/5     Running   15 (5h15m ago)   2d21h
csi-cephfsplugin-t29vq                      3/3     Running   9 (5h15m ago)    2d21h
nfs-client-provisioner-85f48d4f88-vhkxd     1/1     Running   0                56m
nginx-85b98978db-t8znx                      1/1     Running   4 (5h15m ago)    8d
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

3 EFK Usage

3.1 Log Analysis

Step 1 Create a directory for storing YAML files of the test application.

```
[root@master logging]# mkdir app
[root@master logging]# cd app
[root@master app]# pwd
/root/logging/app
[root@master app]#
```

Step 2 Create resource objects.

Create a **dummylogs.yaml** file with the following content:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dummylogs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dummylogs
  template:
    metadata:
      labels:
        app: dummylogs
        logging: "true"    # This label needs to added to pods for log collection.
    spec:
      containers:
      - name: dummy
        image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/dummylogs:latest
        args:
        - msg-processor
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dummylogs2
spec:
  replicas: 3
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
  selector:
    matchLabels:
      app: dummylogs2
  template:
    metadata:
      labels:
        app: dummylogs2
        logging: "true"    # This label needs to added to pods for log collection.
    spec:
      containers:
      - name: dummy
        image: cnych/dummylogs:latest
        args:
        - msg-receiver-api
```

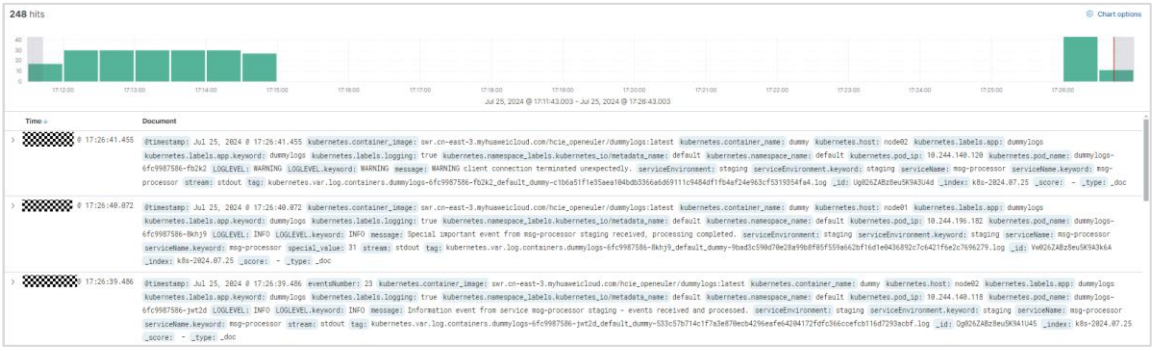
Use **kubectl** to create the two applications and verify their status.

```
kubectl apply -f dummylogs.yaml
kubectl get pods -l logging=true
```

```
[root@efk-terminal app]# kubectl apply -f dummylogs.yaml
deployment.apps/dummylogs created
deployment.apps/dummylogs2 created
[root@efk-terminal app]# kubectl get pods -l logging=true
NAME                          READY   STATUS              RESTARTS   AGE
dummylogs-6d6bb4df5-t8j4f     1/1     Running             0          6s
dummylogs-6d6bb4df5-tzxxz     1/1     Running             0          6s
dummylogs-6d6bb4df5-xxjw8     1/1     Running             0          6s
dummylogs2-86f4b48fb9-4nx9s   0/1     ContainerCreating   0          6s
dummylogs2-86f4b48fb9-gf6w4   0/1     ContainerCreating   0          6s
dummylogs2-86f4b48fb9-gq6df   1/1     Running             0          6s
[root@efk-terminal app]# kubectl get pods -l logging=true
NAME                          READY   STATUS    RESTARTS   AGE
dummylogs-6d6bb4df5-t8j4f     1/1     Running   0          9s
dummylogs-6d6bb4df5-tzxxz     1/1     Running   0          9s
dummylogs-6d6bb4df5-xxjw8     1/1     Running   0          9s
dummylogs2-86f4b48fb9-4nx9s   1/1     Running   0          9s
dummylogs2-86f4b48fb9-gf6w4   1/1     Running   0          9s
dummylogs2-86f4b48fb9-gq6df   1/1     Running   0          9s
[root@efk-terminal app]#
```

Note: After the deployment is complete, the **dummylogs** and **dummylogs2** applications start to generate logs of different levels.

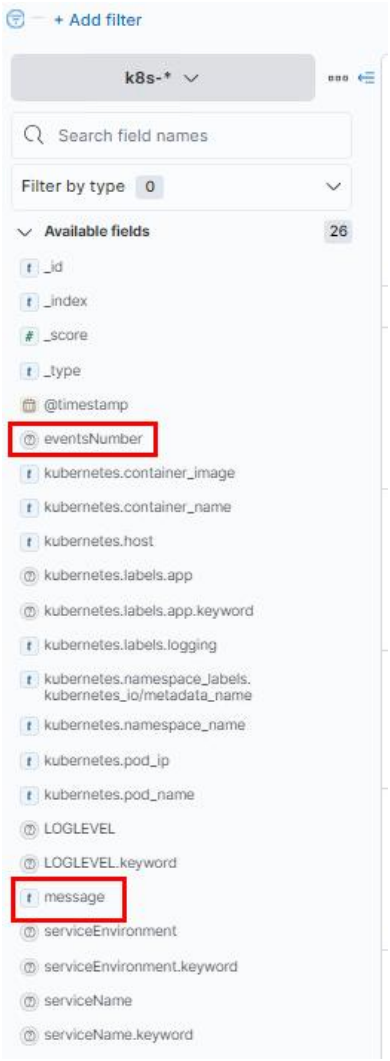
Wait for several seconds and view the log information on the Kibana **Discover** page.



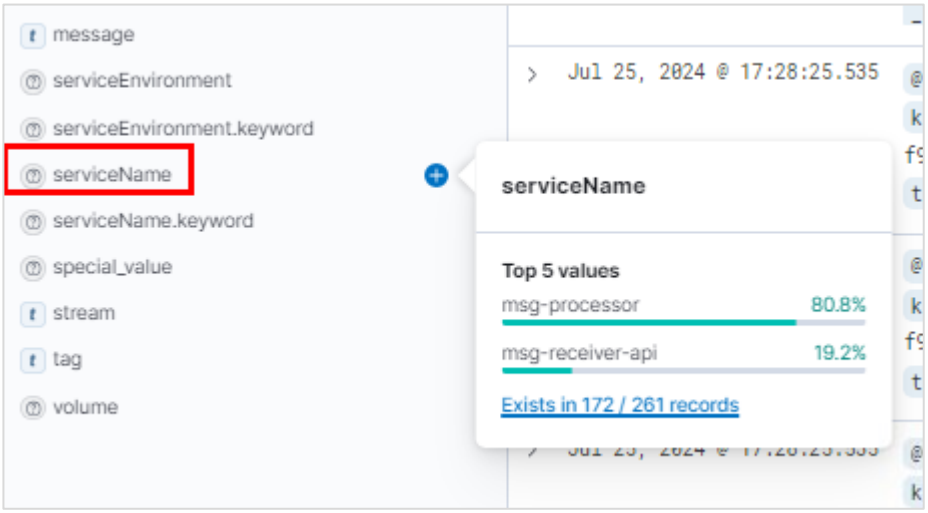
Step 3 View the logs.

The **Available fields** section on the left displays fields defined within the applications.

Copyright © Huawei Technologies Co.,Ltd

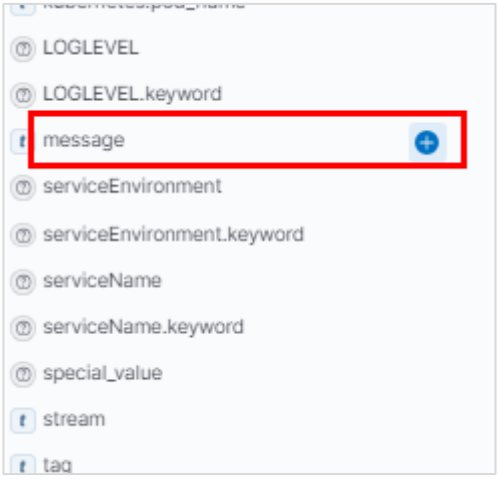


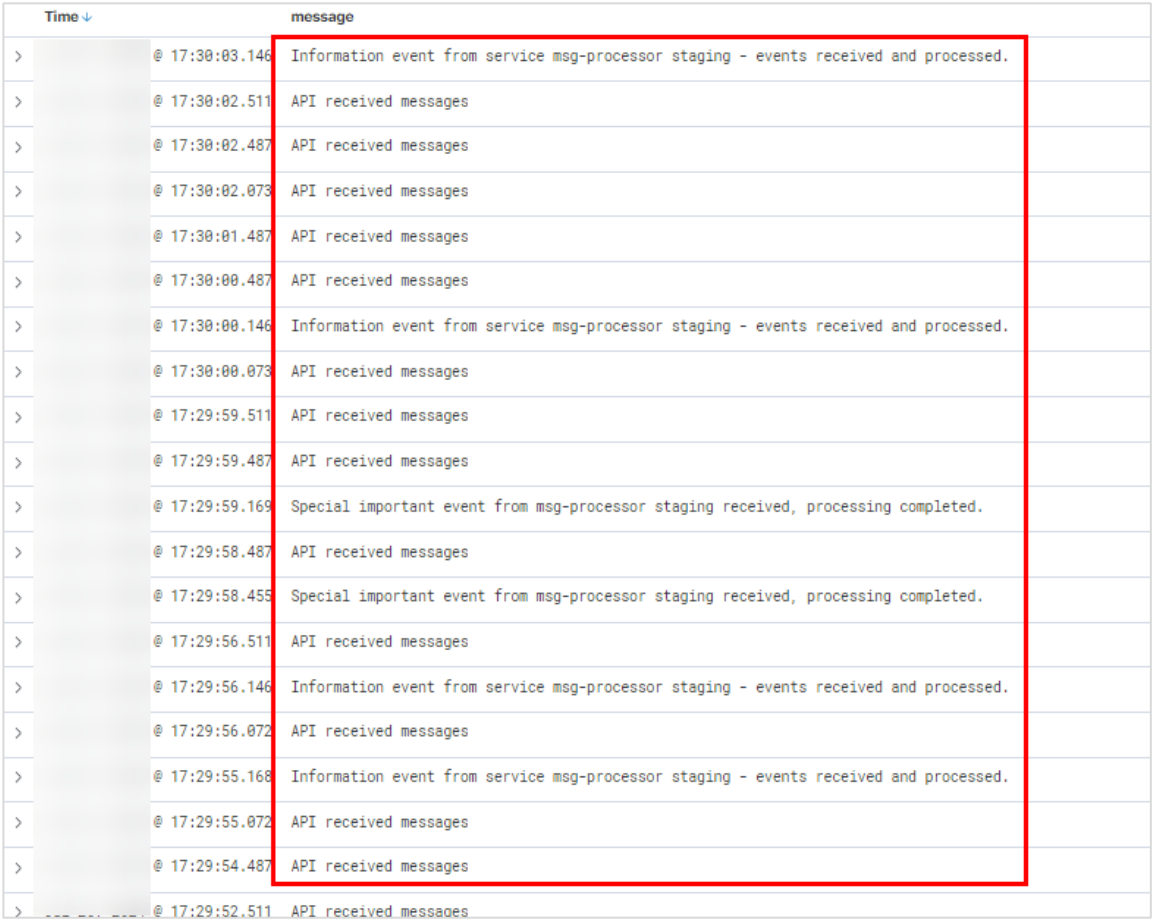
Click the **serviceName** field to see which services have been logged.



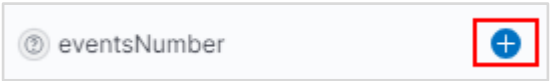
By default, the log data display includes numerous attributes, making it hard to read. You can customize this by selecting specific fields to display.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



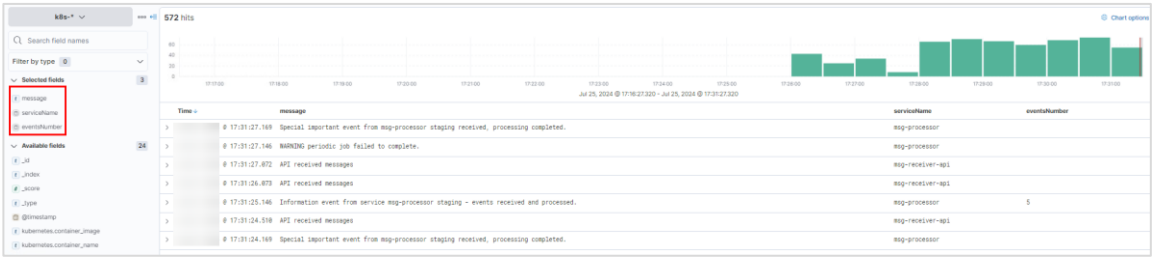


For clearer visualization, select and add the **eventsNumber** and **serviceName** fields to the display.



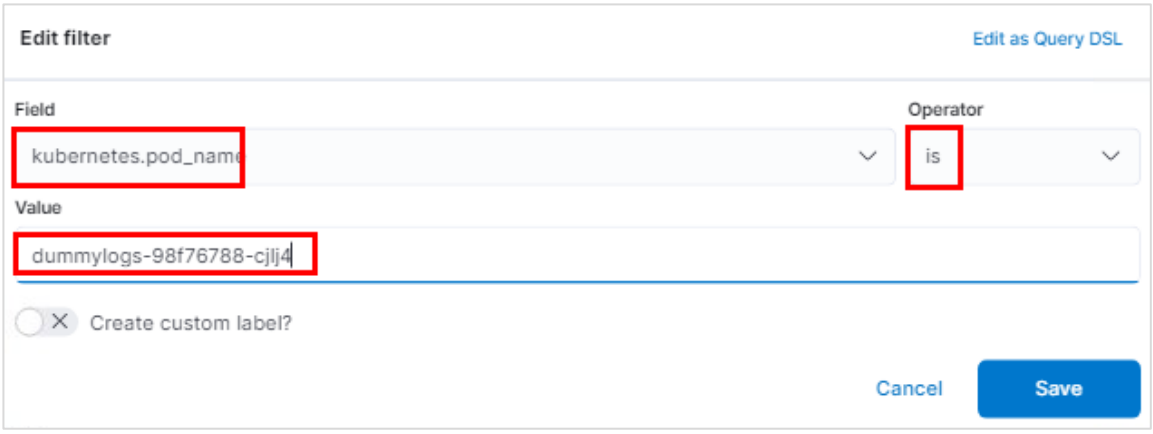


Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Step 4 Perform basic log operations.

Use the **Add filter** option to view specific log data.



Note: In the filter, replace **dummylogs-98f76788-cjlj4** with the actual log source name, as identified through **kubectl**.

Click **Save**. The system filters the logs based on your criteria and highlights keywords in yellow.

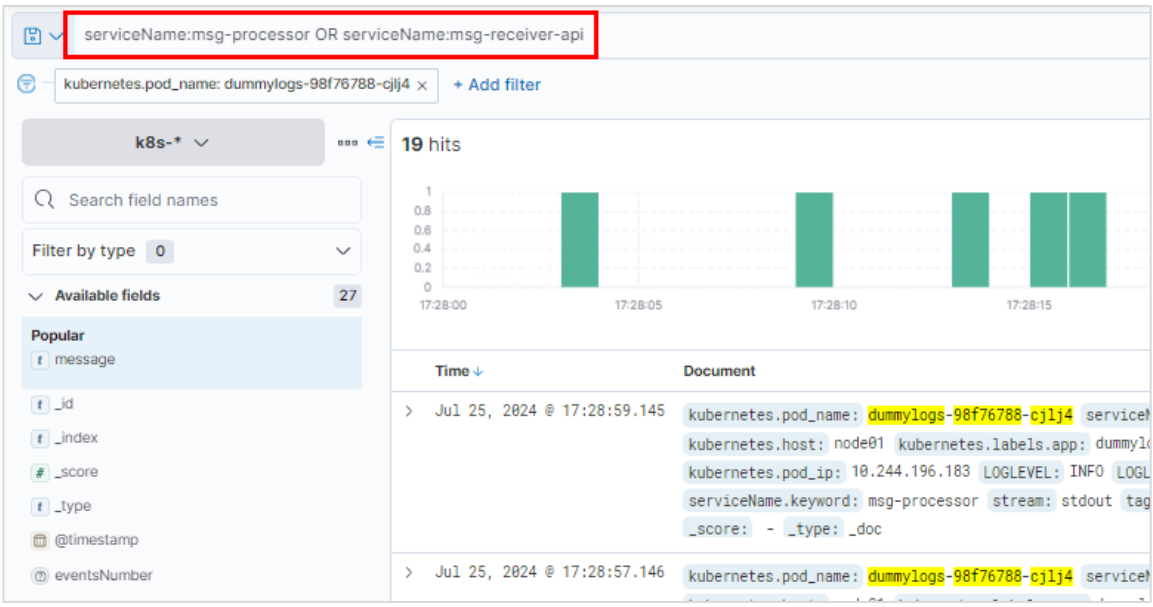


You can also utilize KQL statements for more complex filtering. For example, to view logs from the **msg-processor** or **msg-receiver-api** services, enter the following query:

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

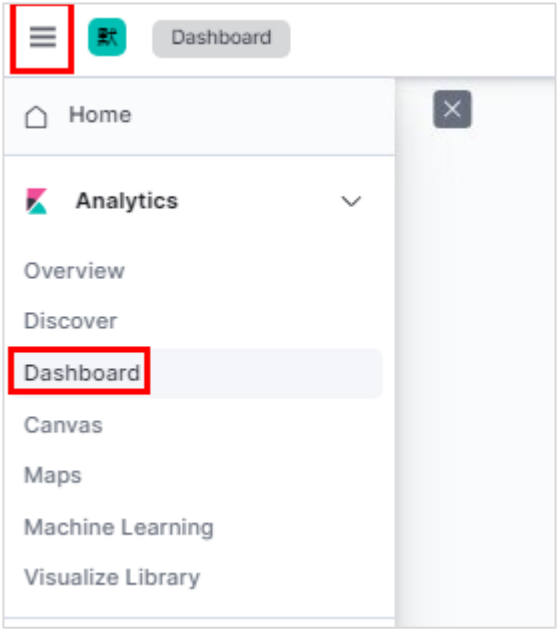
```
serviceName:msg-processor OR serviceName:msg-receiver-api
```

The system displays the corresponding logs.

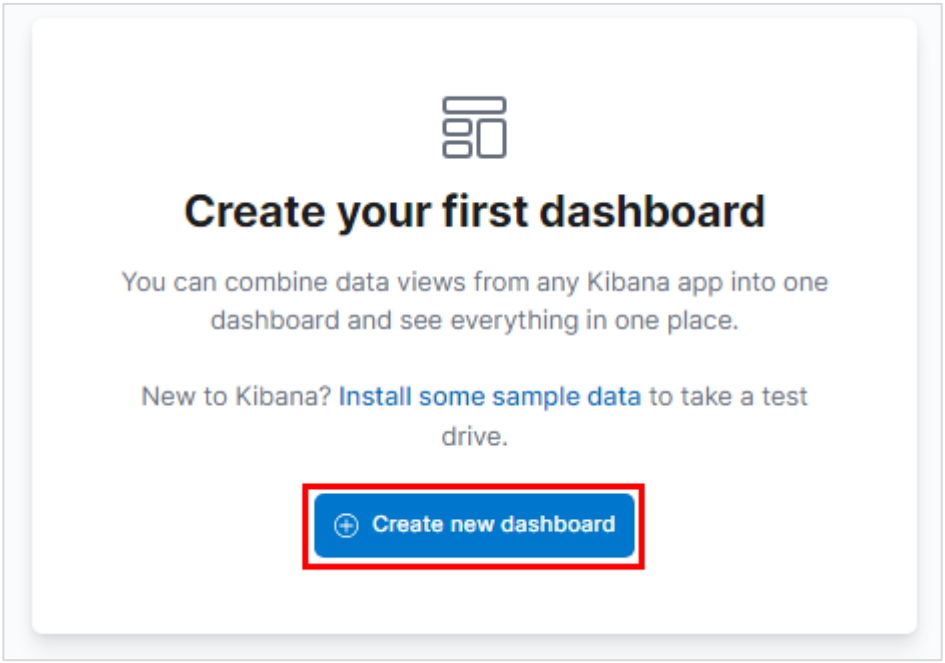


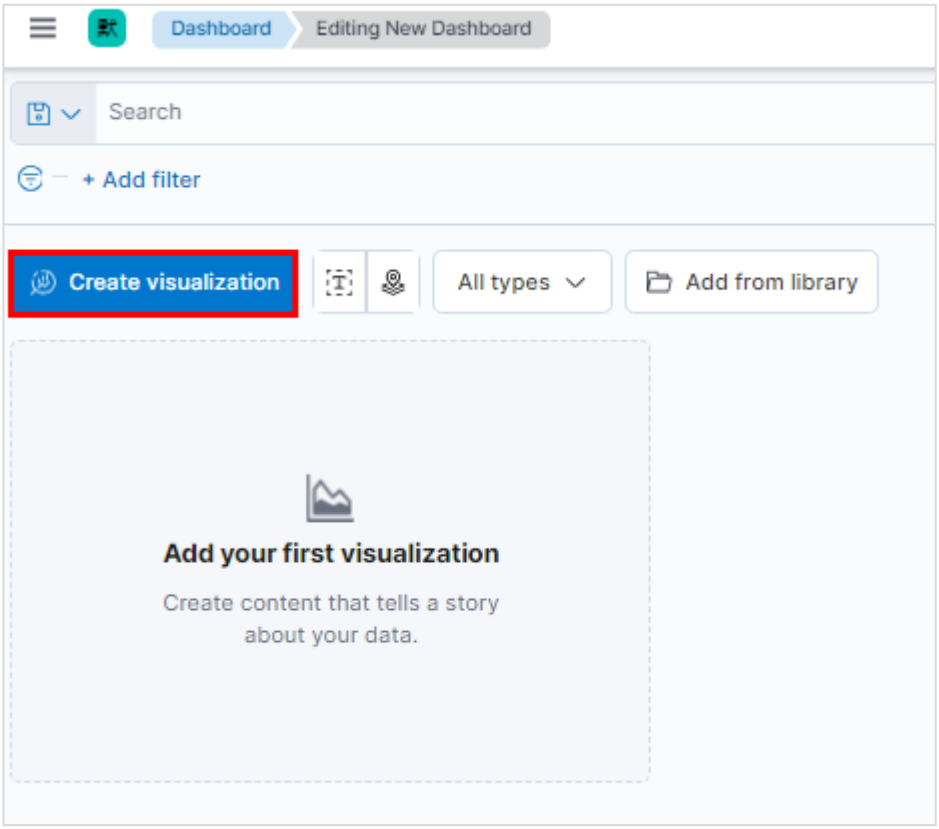
Step 5 Create a dashboard.

Click **Dashboard** in the left navigation bar, and choose **Create new dashboard** > **Create visualization**.



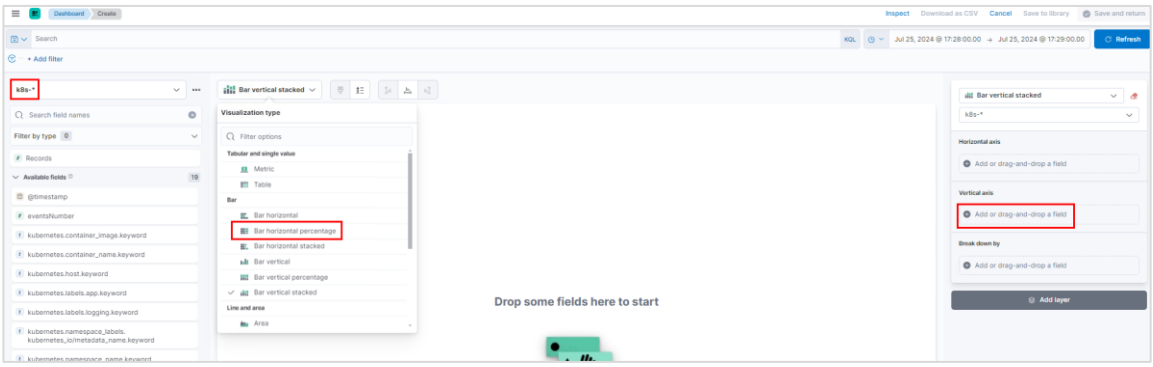
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

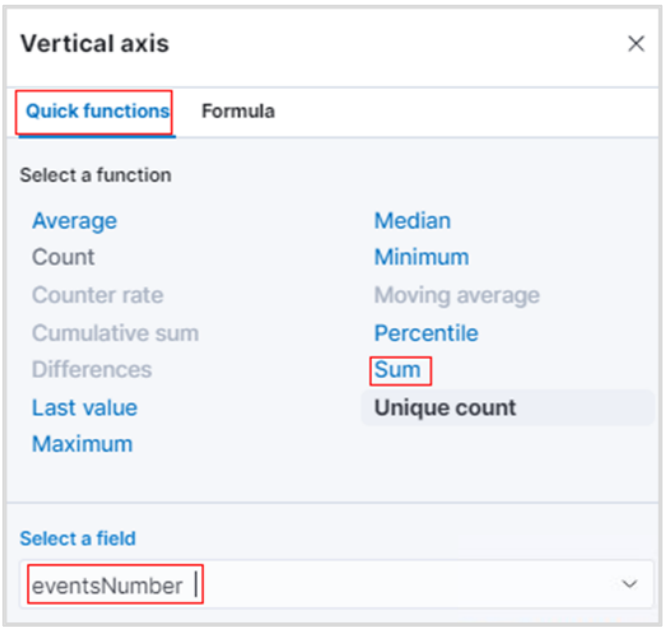




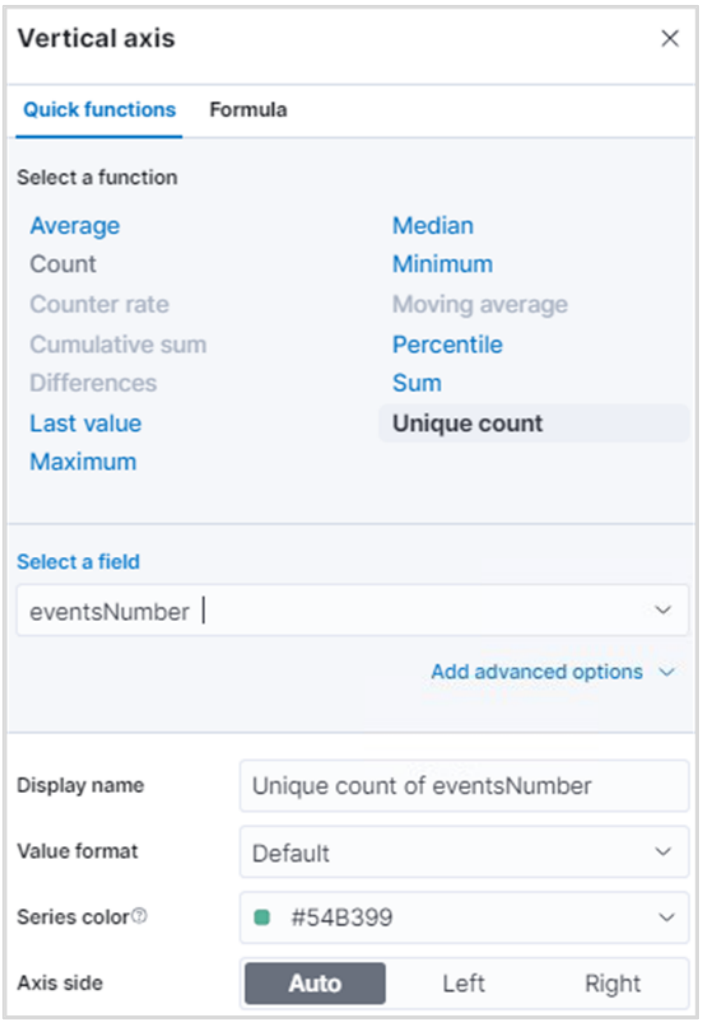
Select the **k8s-*** index and choose **Bar vertical stacked** as the visualization type. Add **eventsNumber** to the vertical axis and apply the **Sum** function for aggregation.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



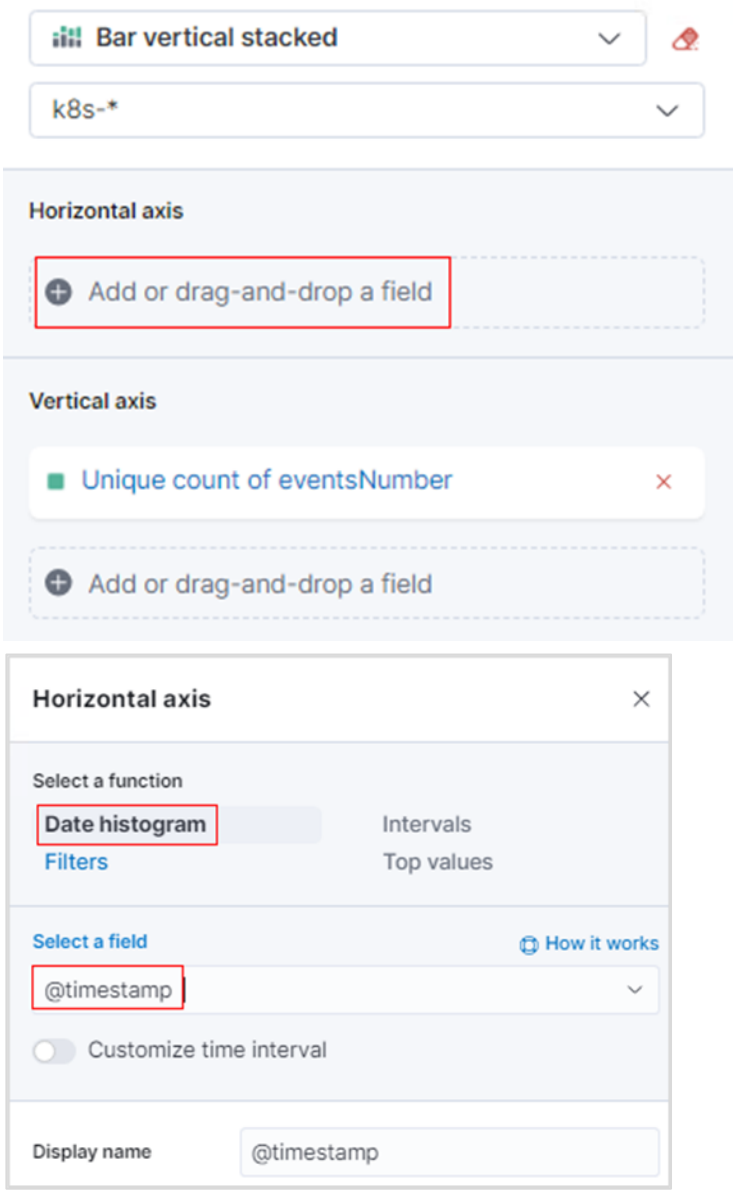


Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



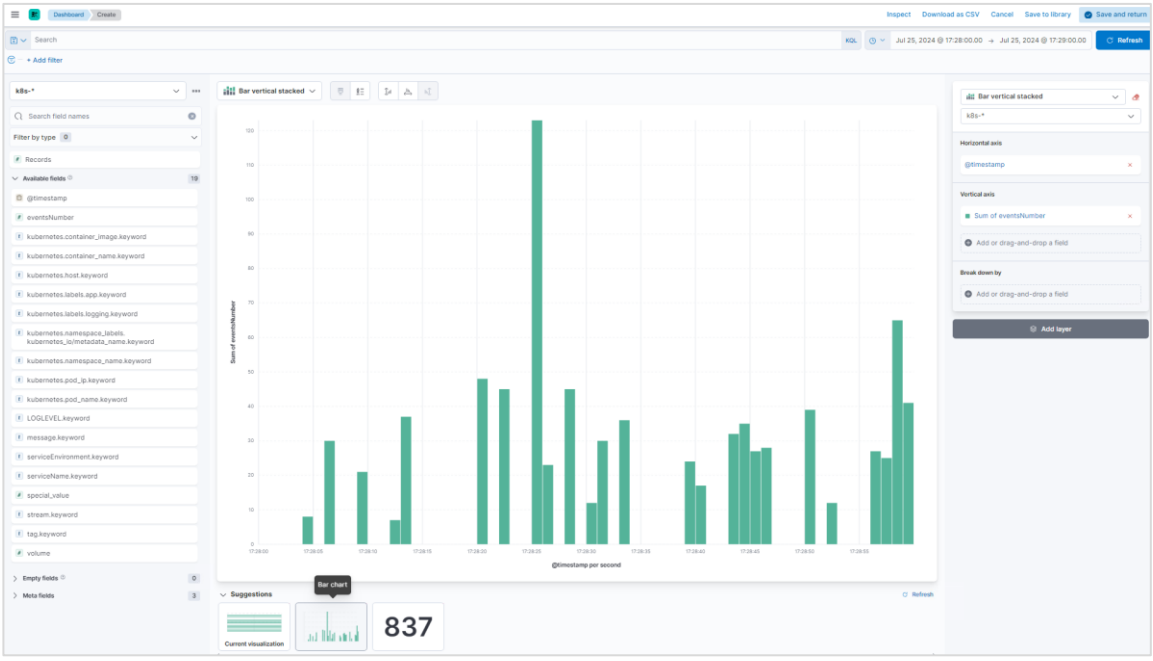
Add **@timestamp** to horizontal axis apply the **Date histogram** function.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

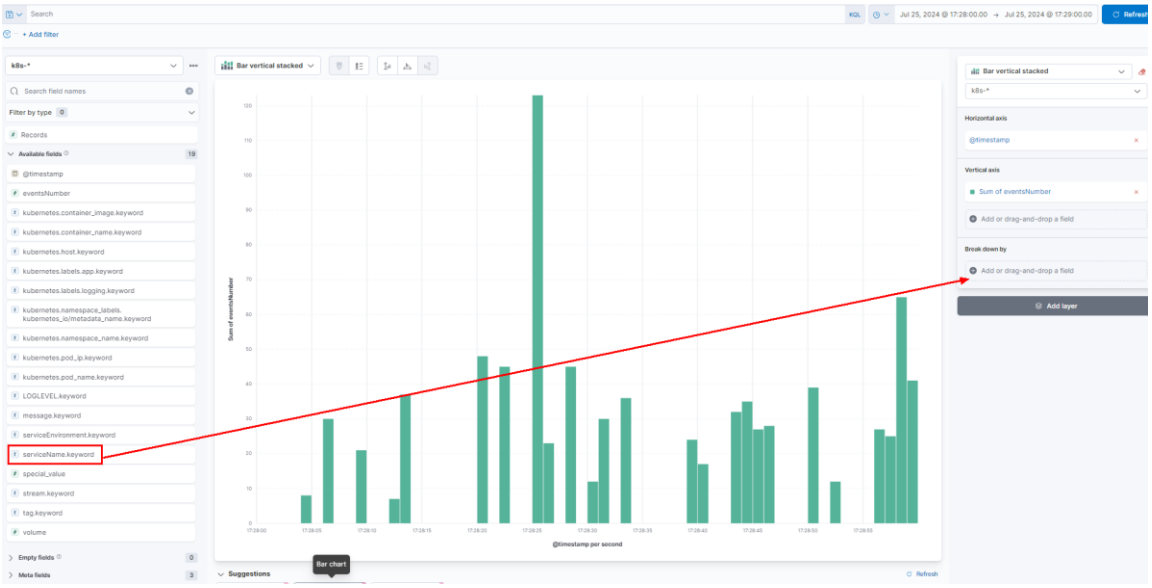


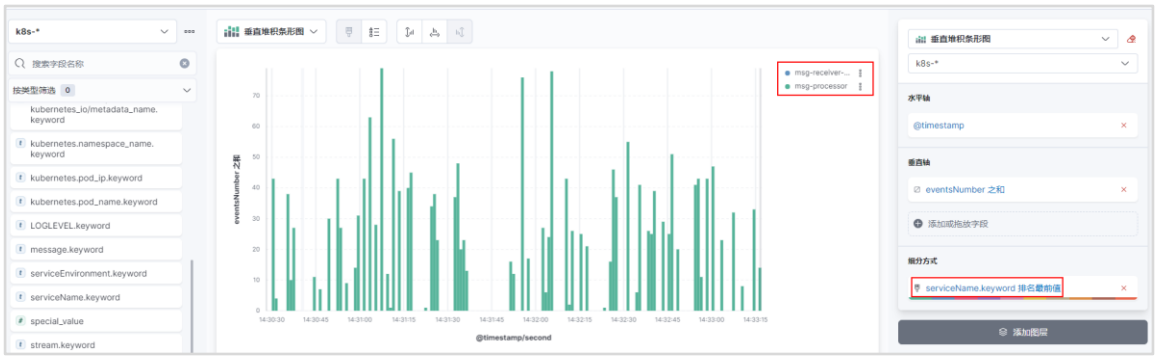
This creates a chart showing the sum of **eventsNumber** over time.

Copyright © Huawei Technologies Co.,Ltd



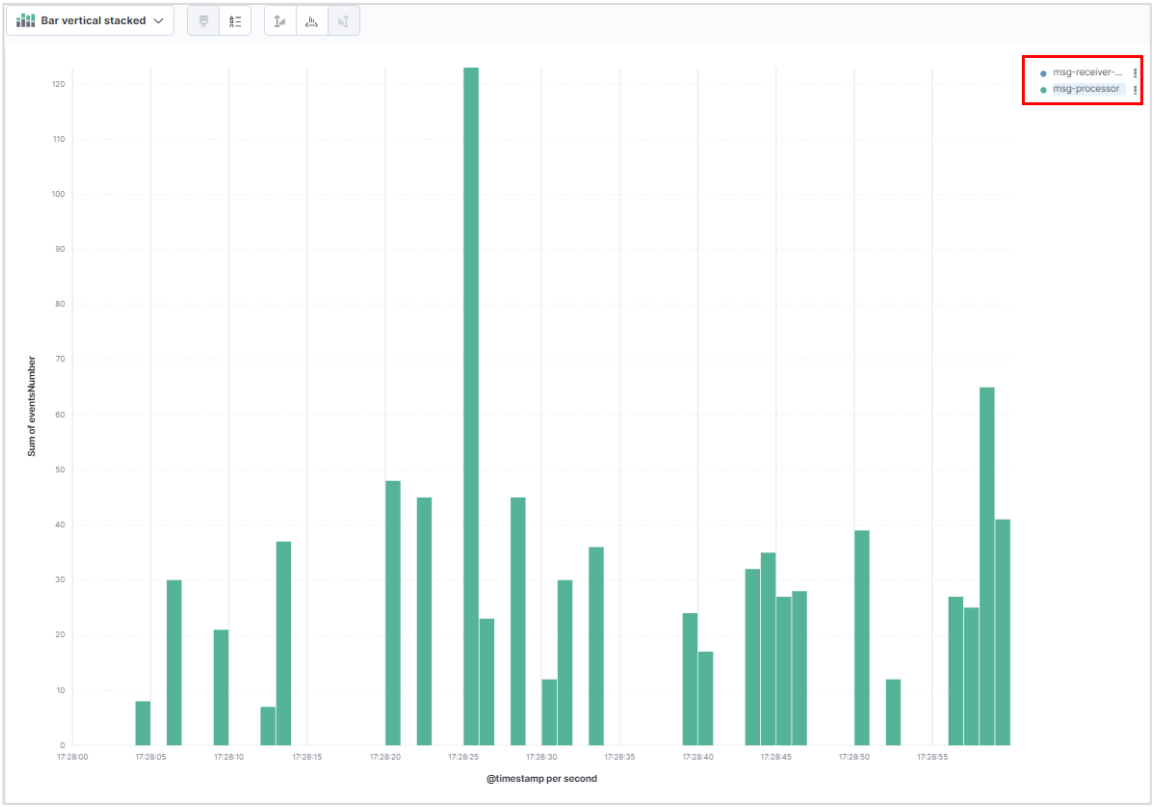
Drag **serviceName.keyword** from **Available fields** to **Break down by** on the right.

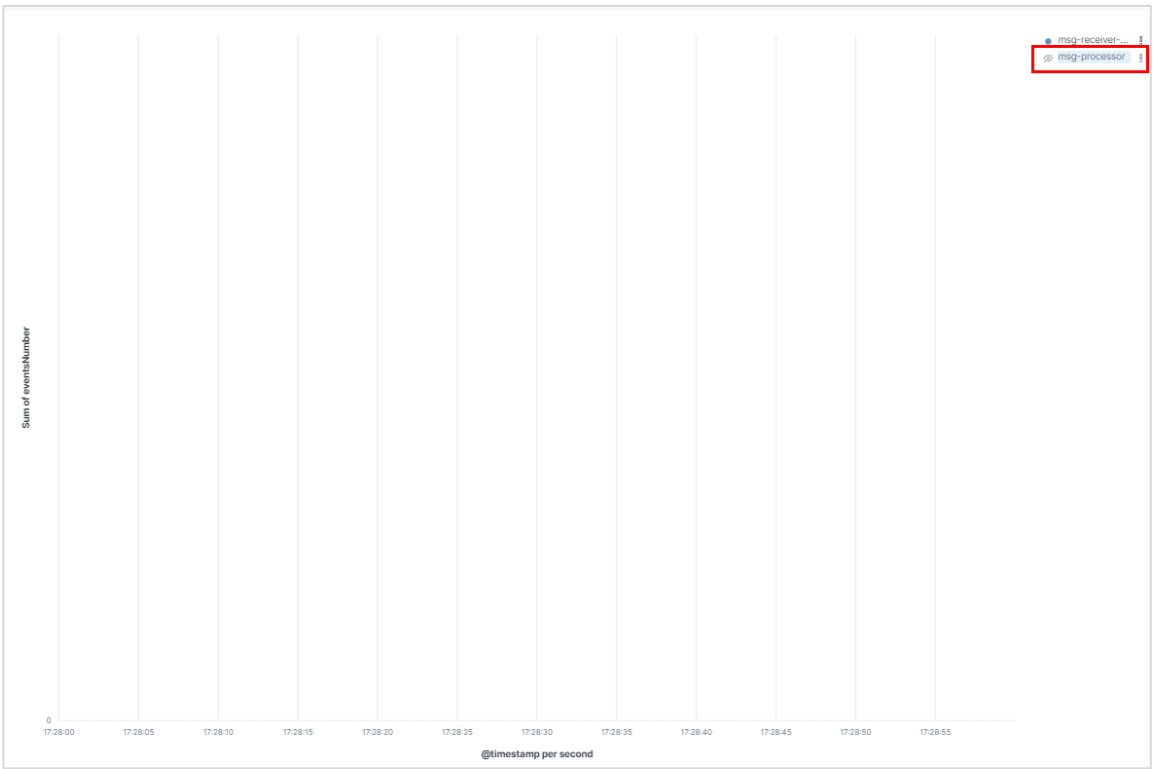




Clicking a service name within the chart will toggle its visibility.

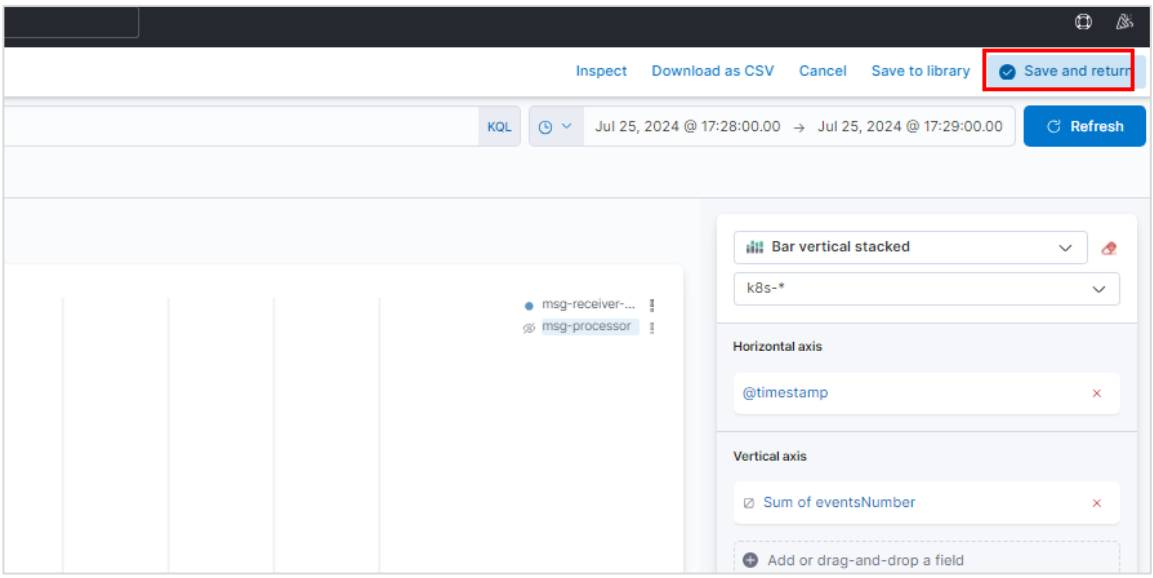
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



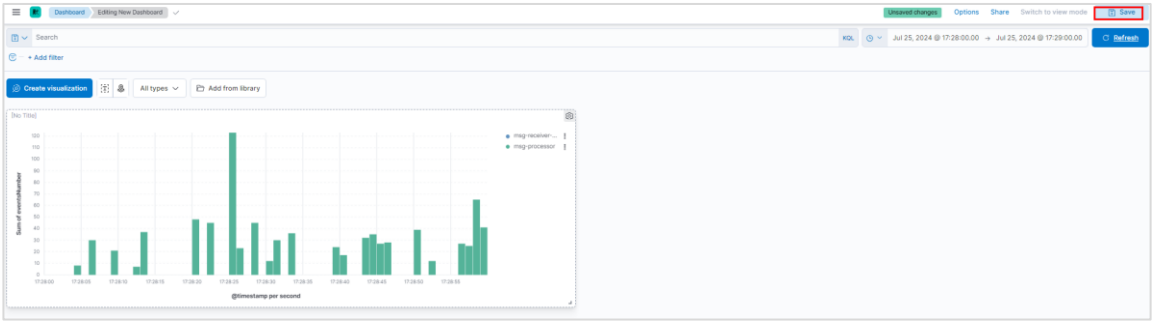


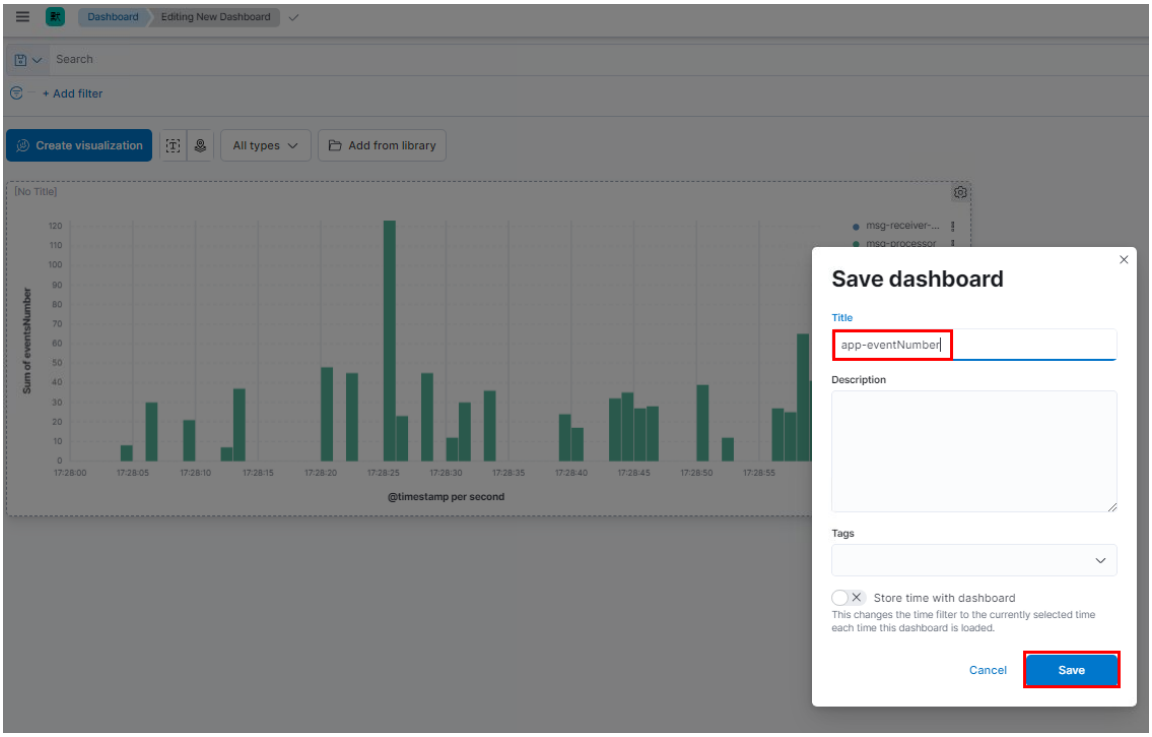
Click **Save and return** in the upper right corner. A visualized chart is displayed in the dashboard.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



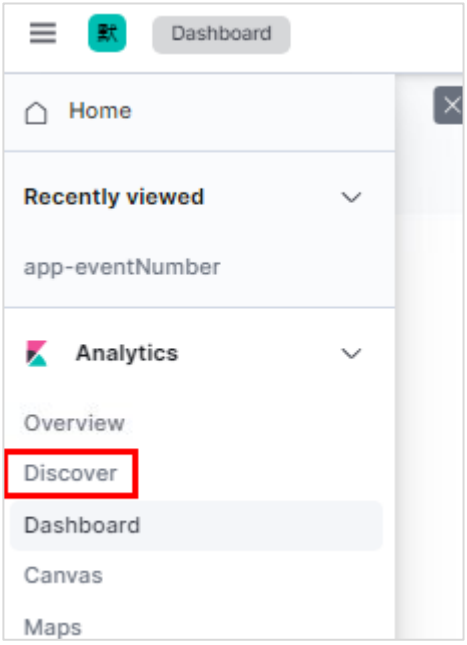
Click **Save and return** in the upper right corner, name the dashboard **app-eventNumber**, and click **Save**. This creates a dashboard with one chart.



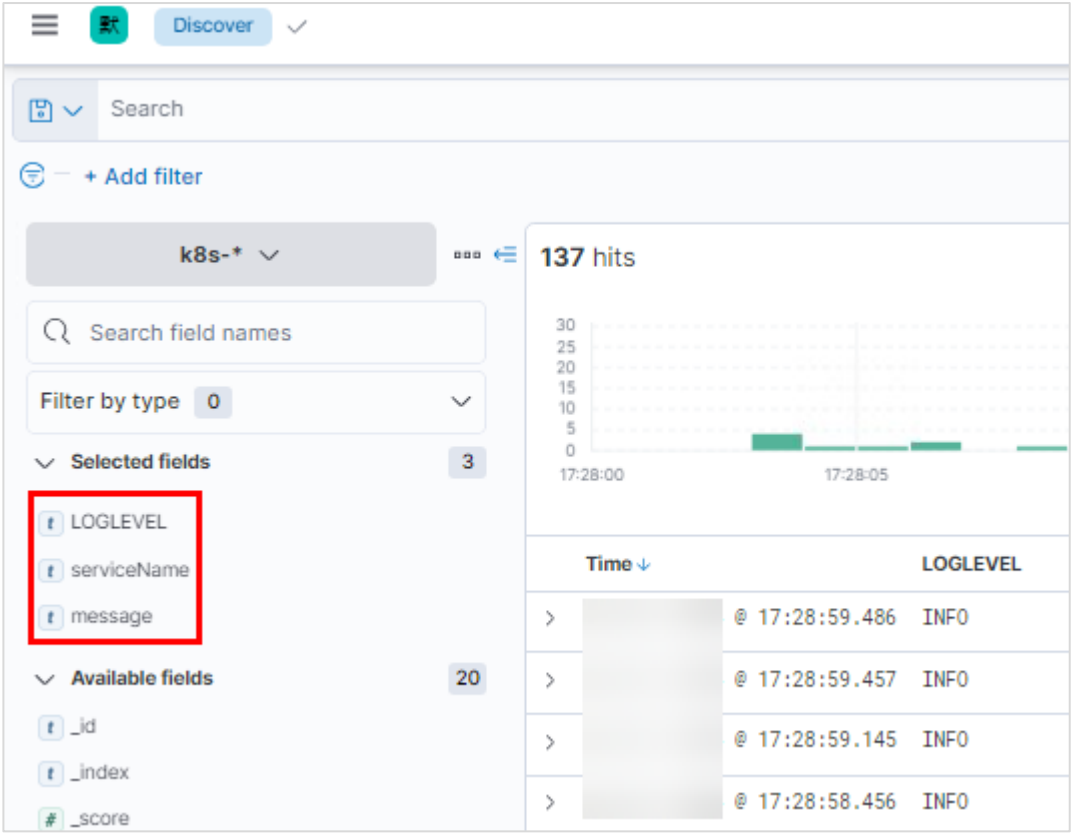


Copyright © Huawei Technologies Co.,Ltd

Return to the **Discover** page.



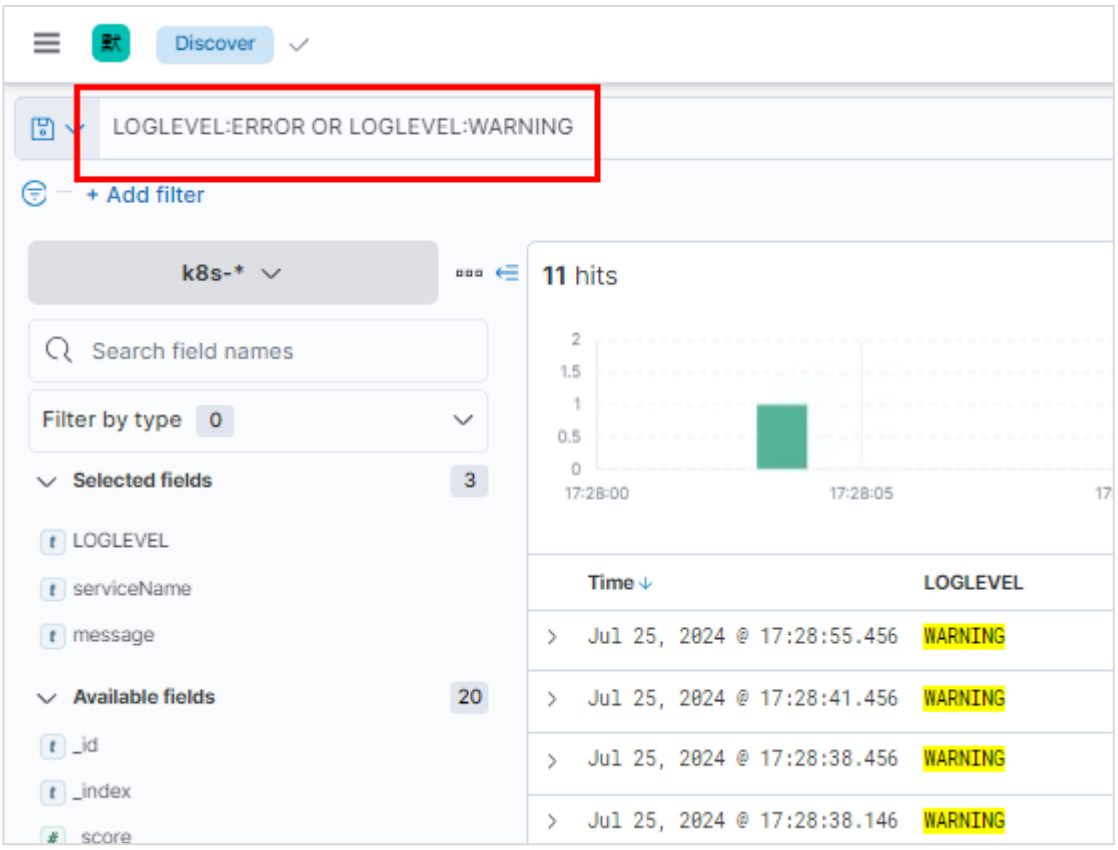
Select **LOGLEVEL**, **message**, and **serviceName** from **Available fields**.



Enter the following query to filter for error and warning logs:

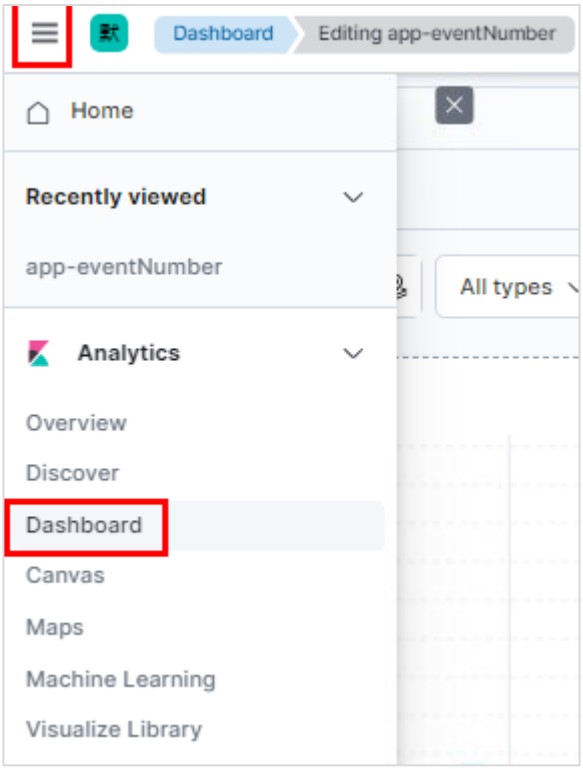
```
LOGLEVEL:ERROR OR LOGLEVEL:WARNING
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

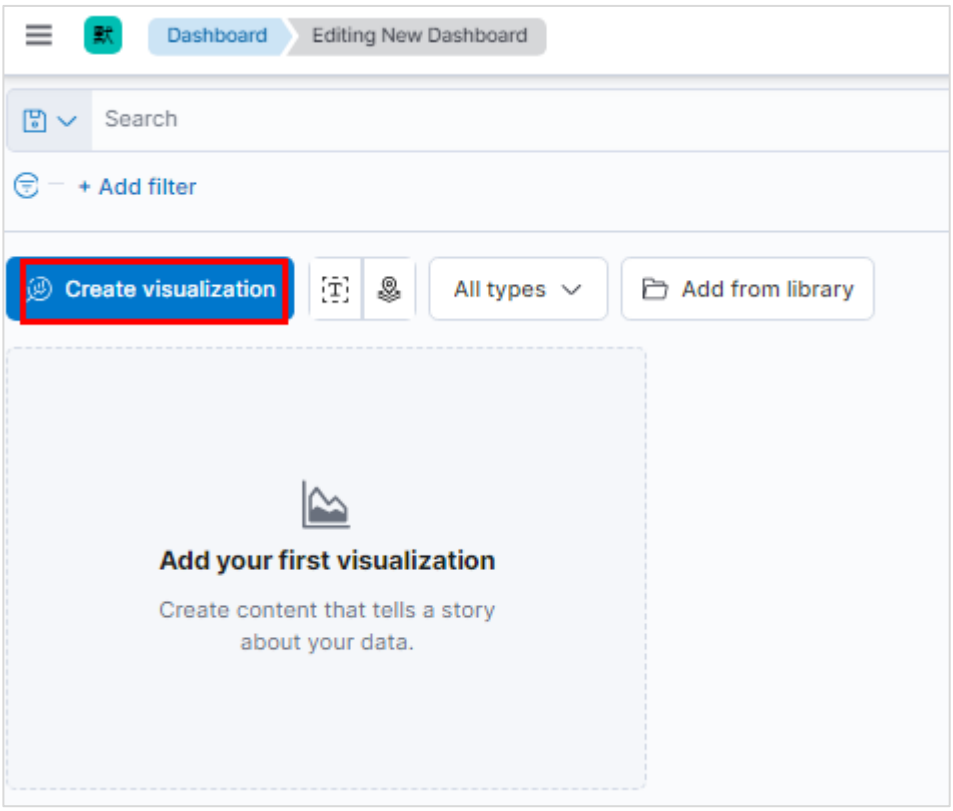


Note: Since errors are less frequent, this example application will generate approximately four error messages every 15 to 20 minutes, with the rest being warnings.

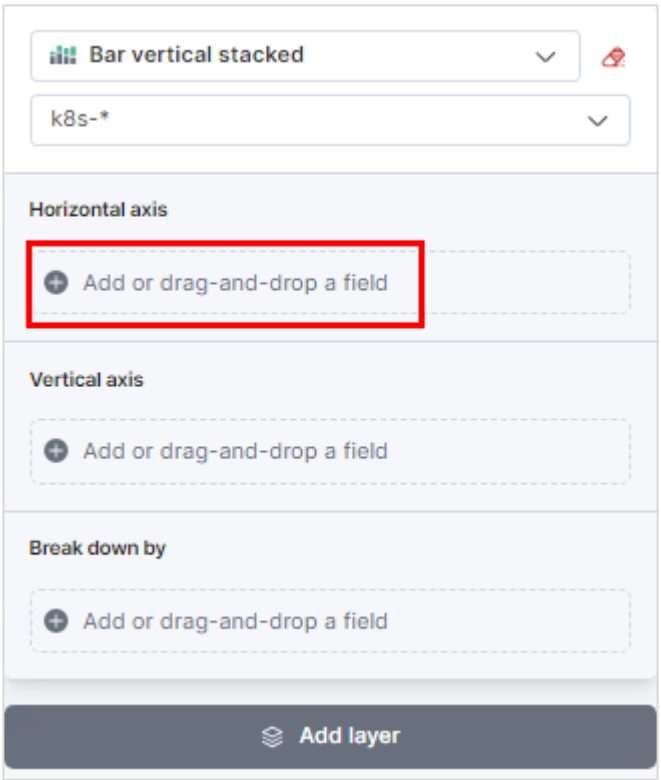
Return to the **Dashboard** page and create another visualization for error logs.



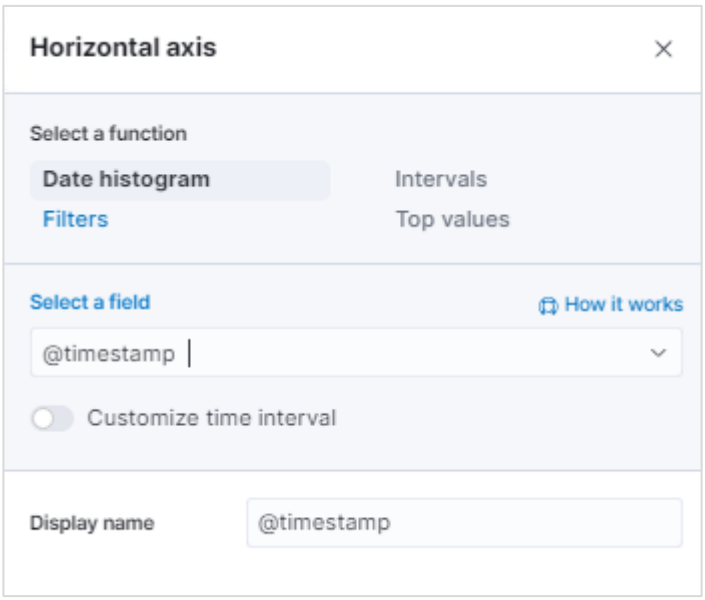
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

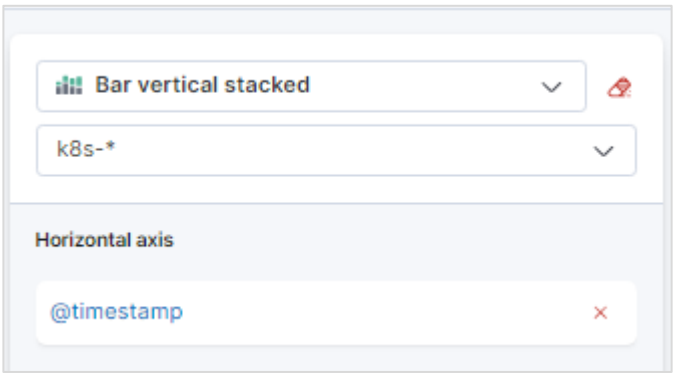


Select the **k8s-*** index and choose **Bar vertical stacked** as the visualization type. Add **@timestamp** to the horizontal axis and apply the **Date histogram** function for aggregation.

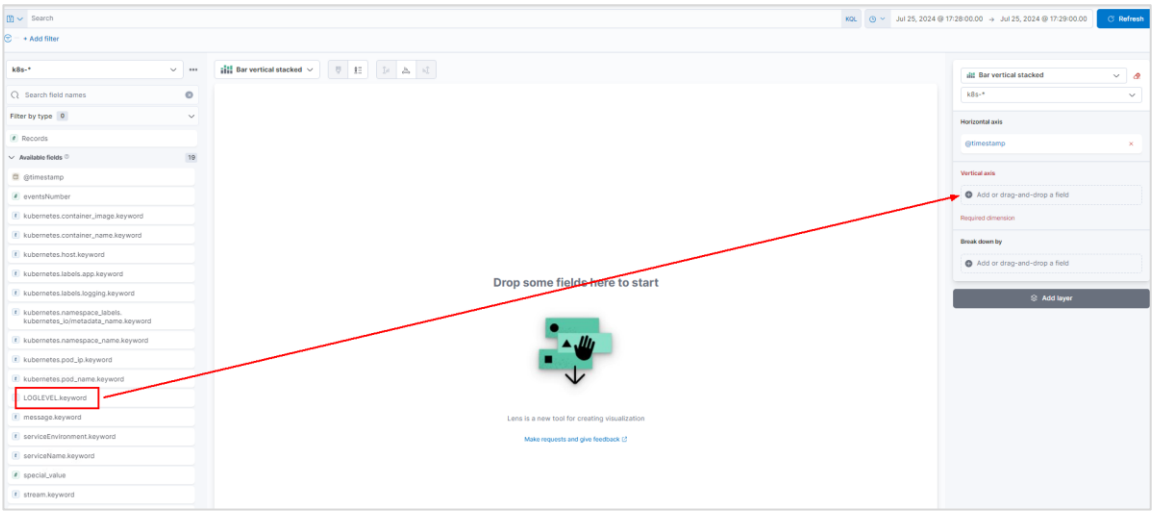


Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

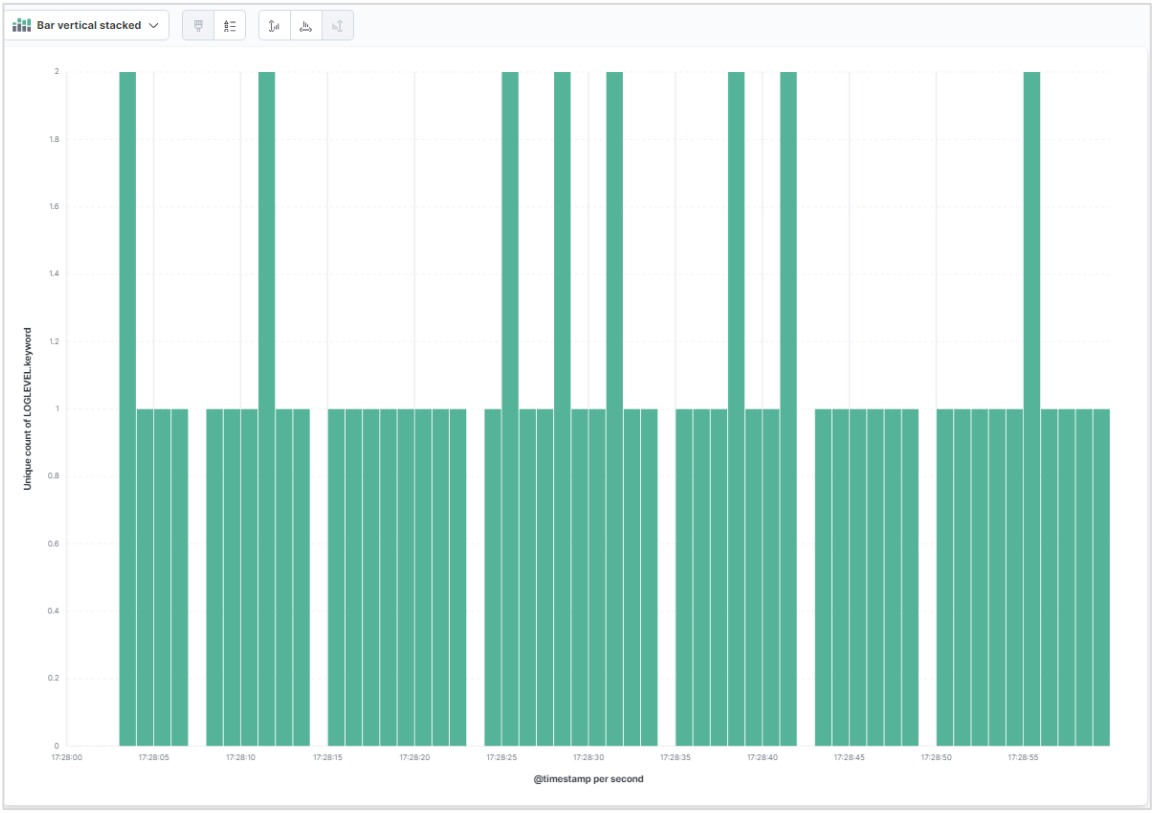




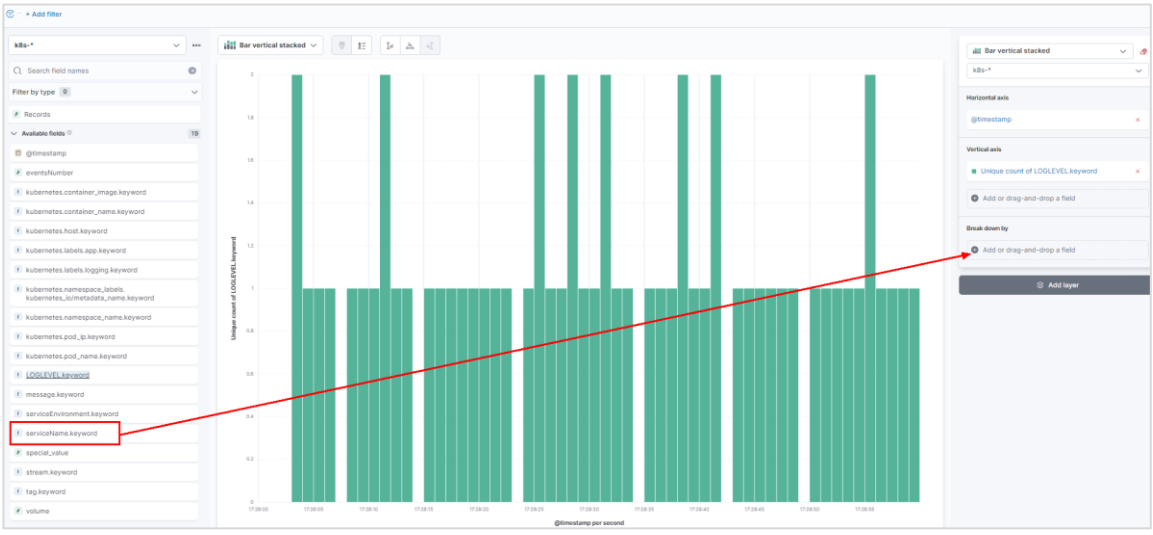
Drag **LOGLEVEL.keyword** from **Available fields** to **Vertical axis** on the right.



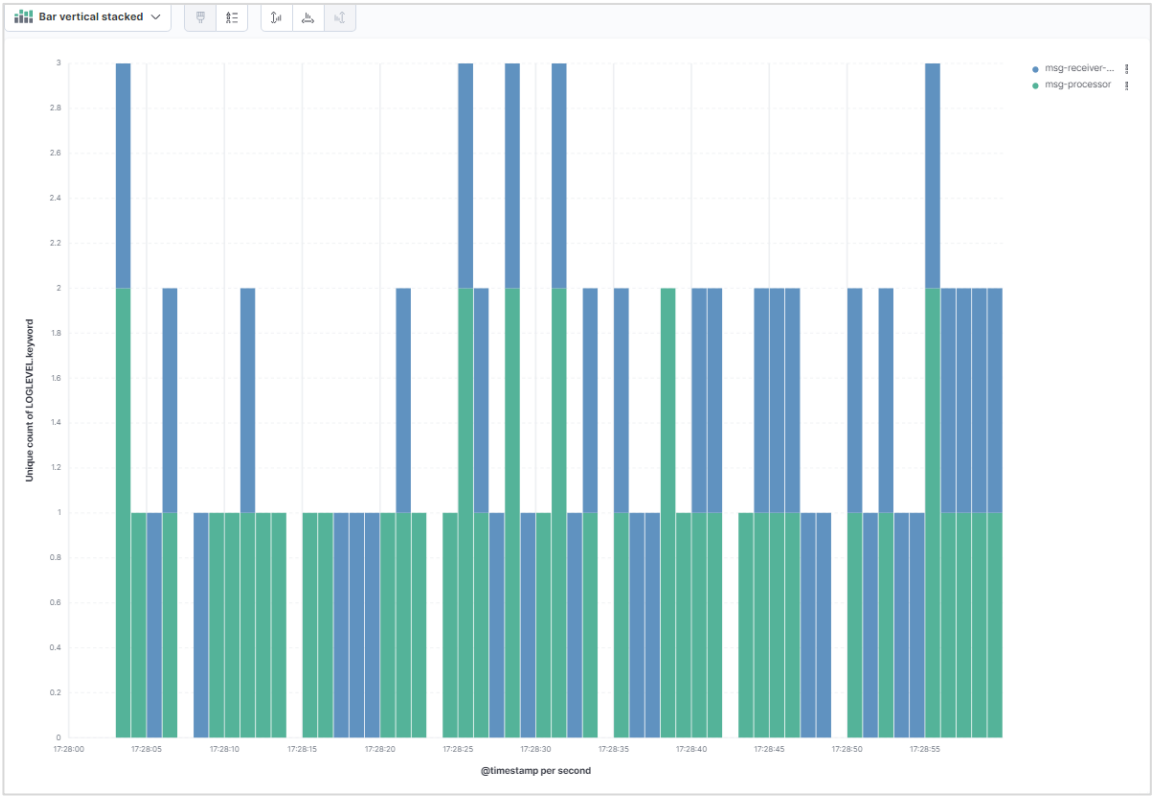
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Drag **serviceName.keyword** from **Available fields** to **Break down by** on the right.

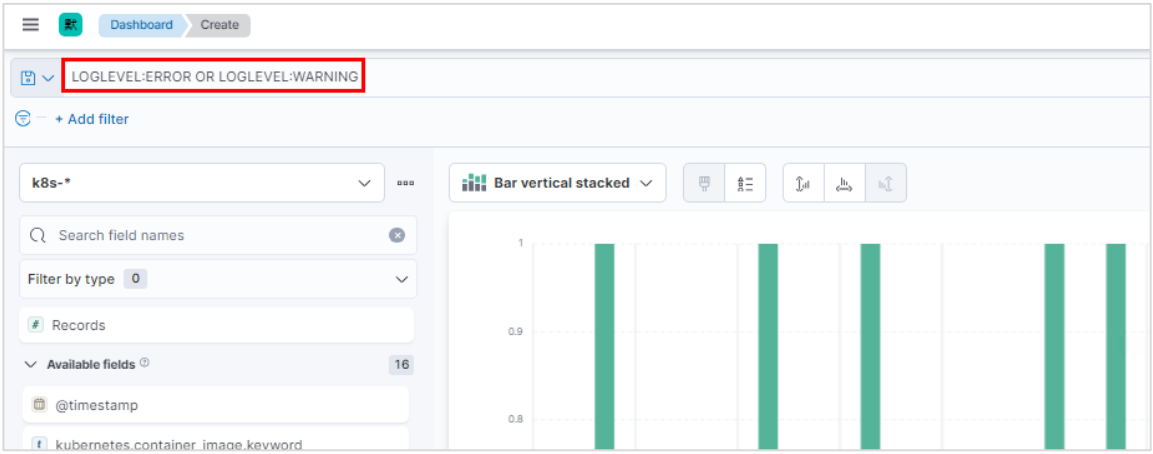


Copyright © Huawei Technologies Co.,Ltd

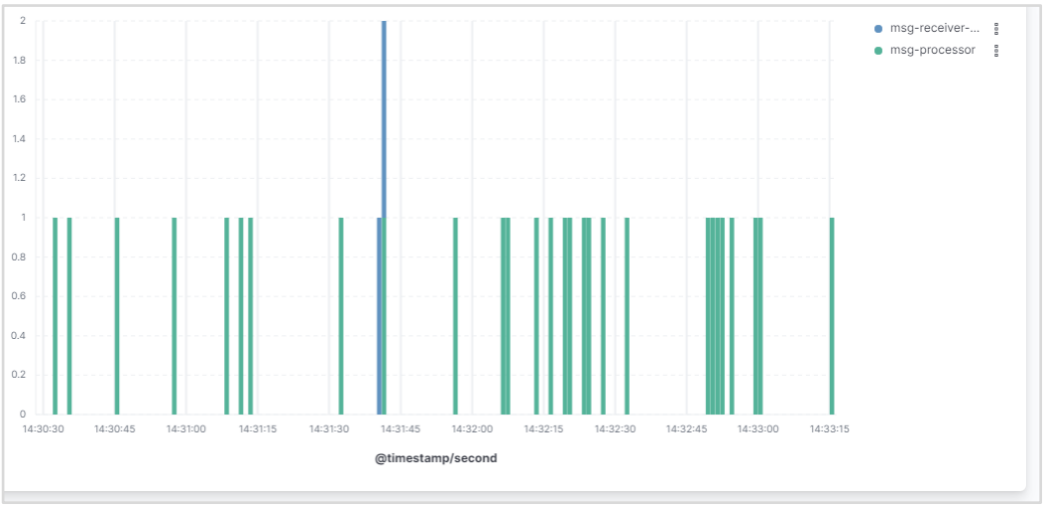


Enter the following query to filter for error and warning logs:

```
LOGLEVEL:ERROR OR LOGLEVEL:WARNING
```



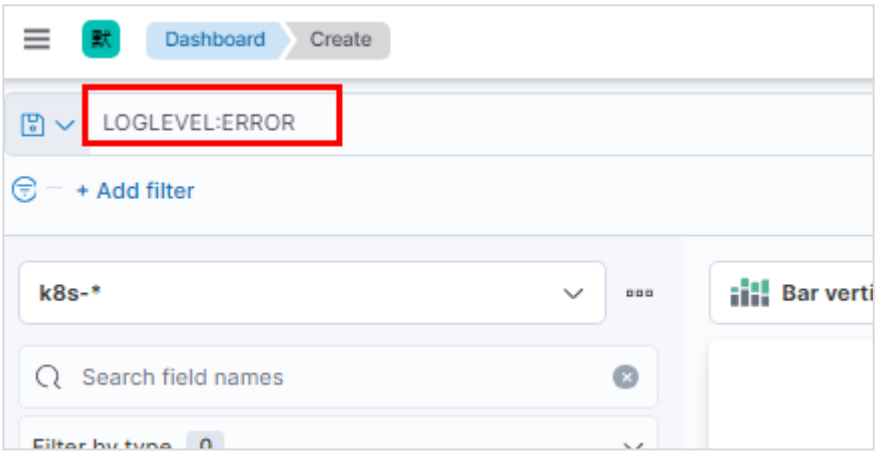
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Note: The chart reveals that **msg-processor** has more errors or warnings than **msg-receiver-api**.

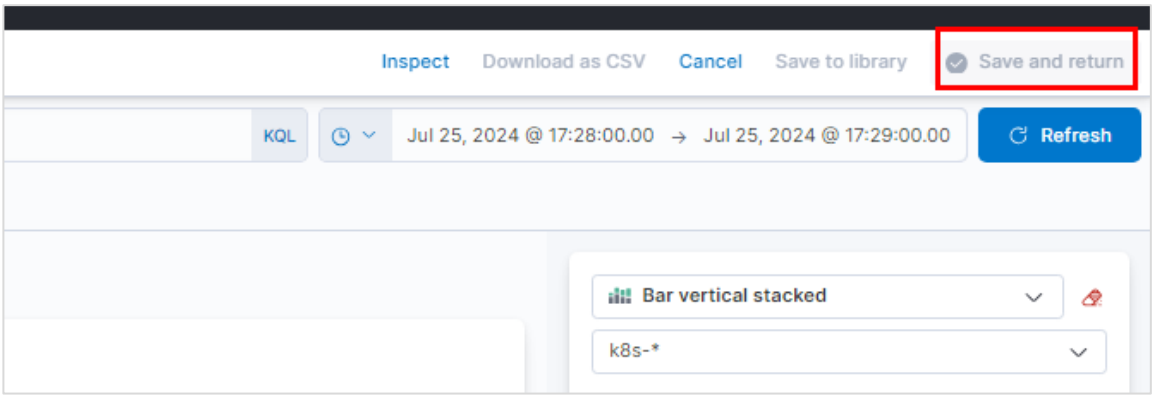
Enter the following query to filter for error logs:

```
LOGLEVEL:ERROR
```



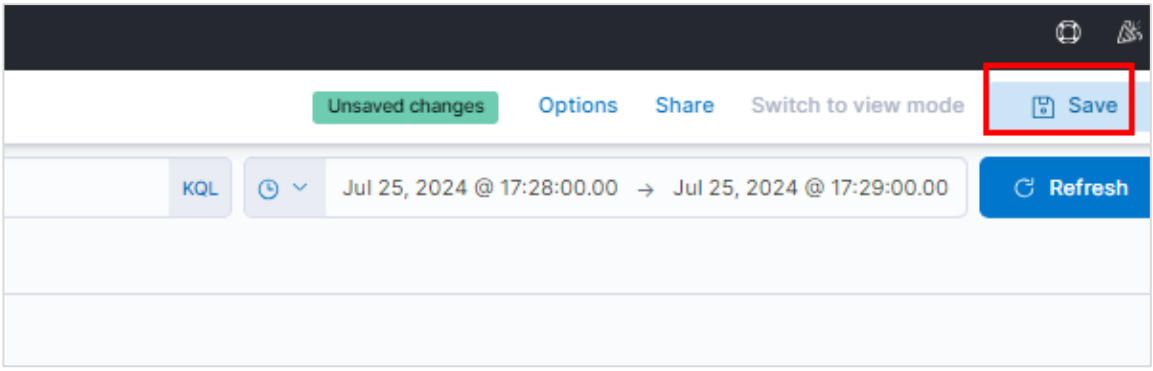
Note: The chart reveals a strong correlation between errors in both services, with similar numbers of error logs generated at approximately the same time. This connection can be used to streamline troubleshooting.

Click **Save and return** in the upper right corner. Another visualized chart is displayed in the dashboard.



Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

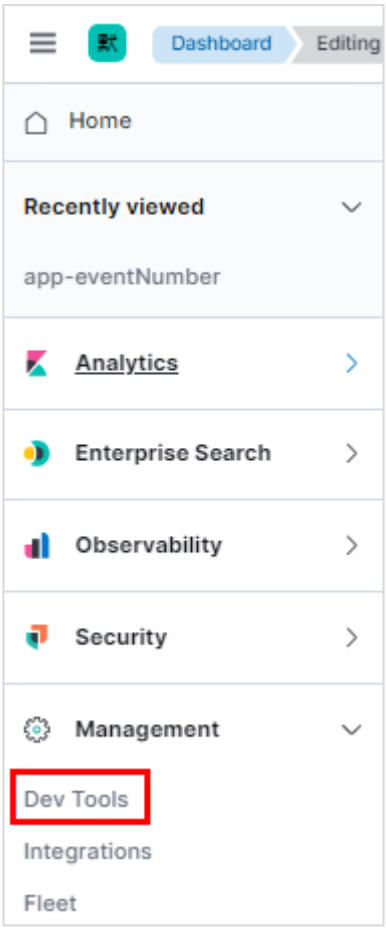
Click **Save** in the upper right corner to save the dashboard with two charts.



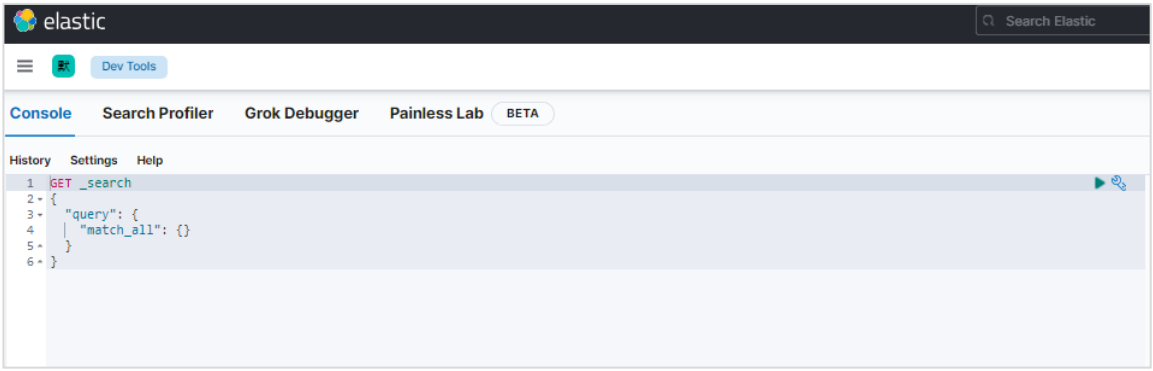
3.2 Dev Tools

Step 1 Check the Elasticsearch cluster information.

In Kibana, click **Dev Tools** in the left navigation bar to open **Console**.

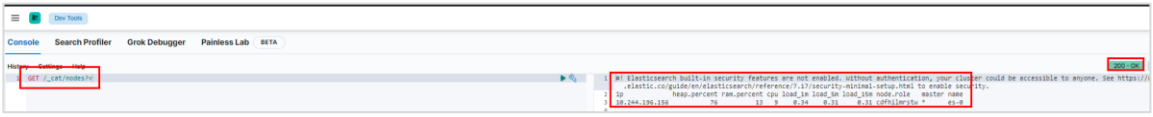


Copyright © Huawei Technologies Co.,Ltd



Enter the following command to view information about the nodes in the Elasticsearch cluster:

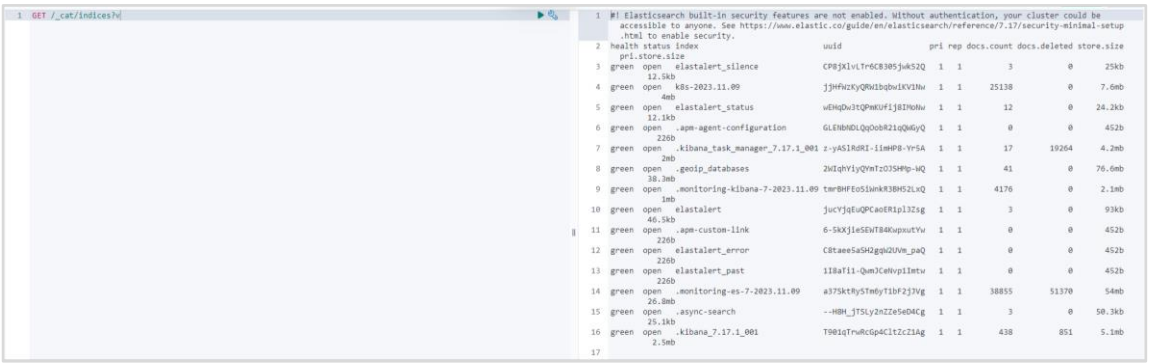
```
GET /_cat/nodes?v
```



Note: You will encounter a warning about the disabled security feature. You can ignore this message. For the purpose of this EFK setup, user authentication is not enabled on Elasticsearch, allowing password-free access to the Kibana page.

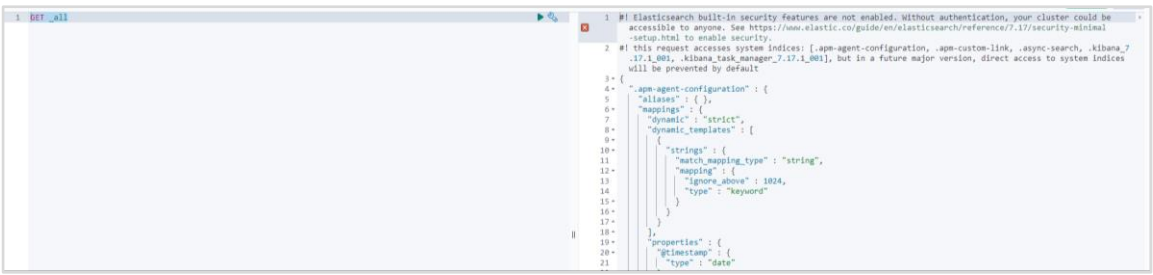
Enter the following command to obtain an overview of the indexes in the Elasticsearch cluster:

```
GET /_cat/indices?v
```



Enter the following command to obtain detailed information about indexes in the Elasticsearch cluster:

```
GET _all
```



Enter the following command to check the Elasticsearch cluster health status:

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
GET /_cat/health?v
```



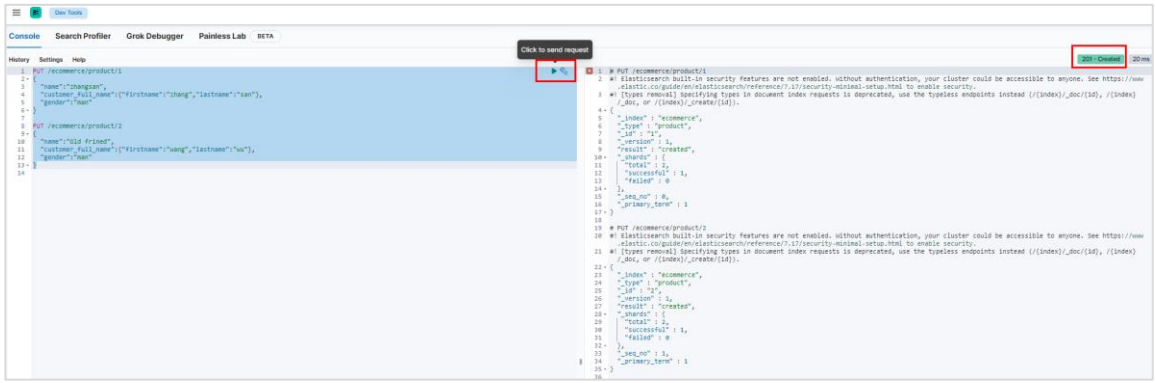
Step 2 Operate the data.

Use the **PUT** method to create index data.

```
PUT /ecommerce/product/1
{
  "name":"zhangsan",
  "customer_full_name":{"firstname":"zhang","lastname":"san"},
  "gender":"man"
}

PUT /ecommerce/product/2
{
  "name":"Old frined",
  "customer_full_name":{"firstname":"wang","lastname":"wu"},
  "gender":"man"
}
```

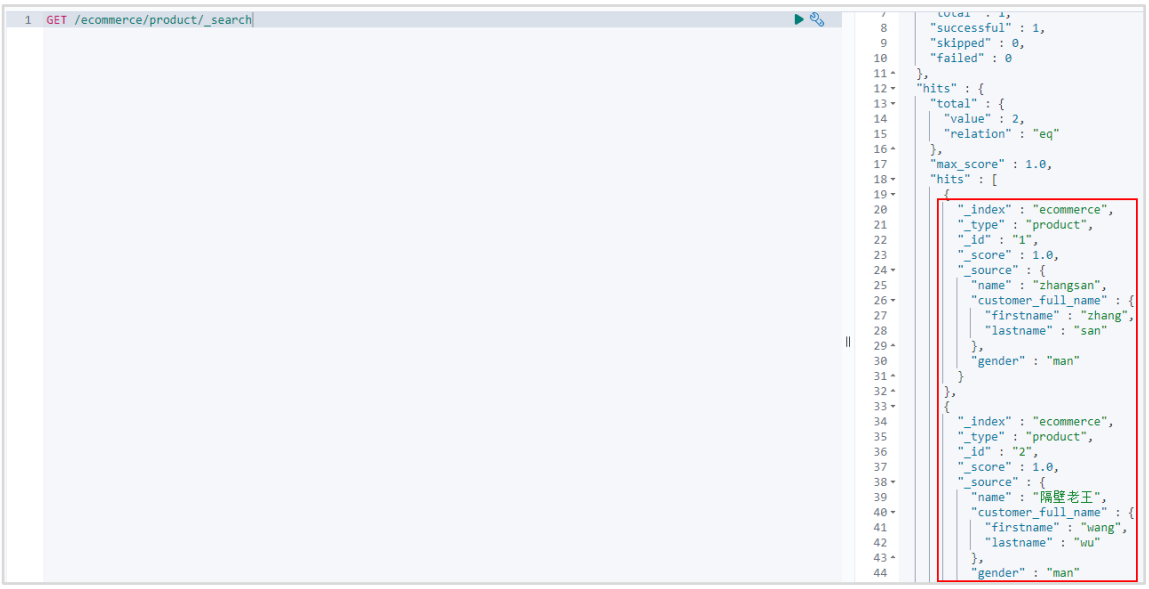
Note: To execute multiple Elasticsearch commands, select all commands and then click the run button.



Use the **GET** method to view index data.

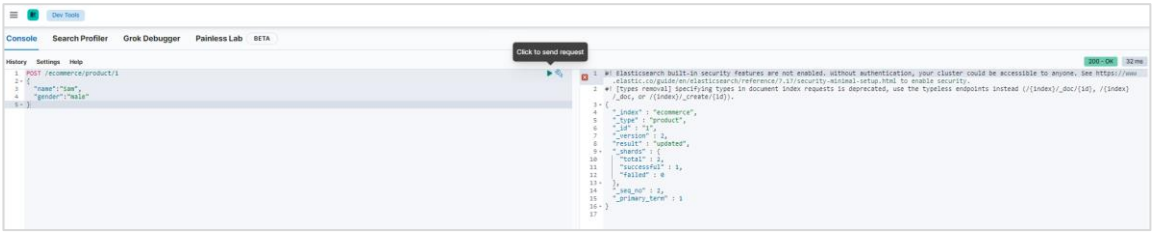
```
GET /ecommerce/product/_search
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



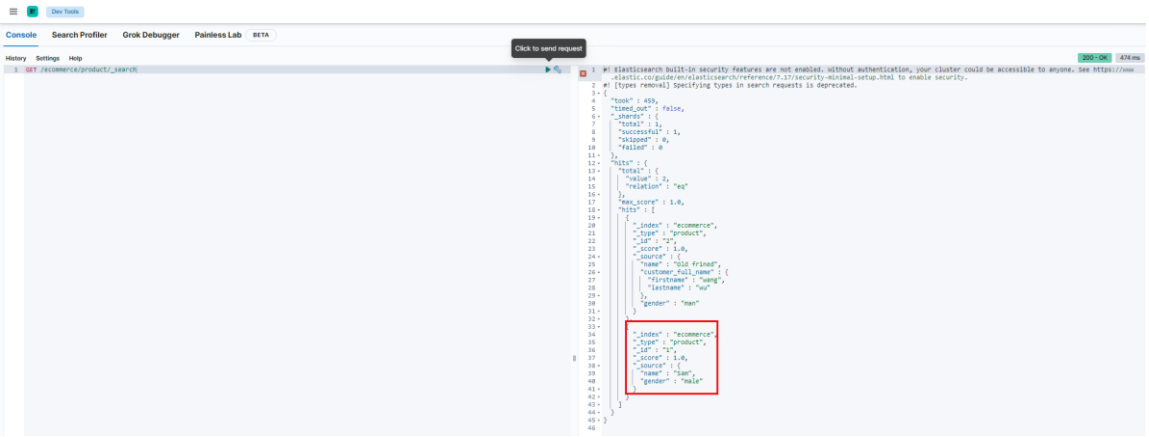
Use the **POST** method to update index data.

```
POST /ecommerce/product/1
{
  "name":"Sam",
  "gender":"male"
}
```



Use the **GET** method to view index data changes.

```
GET /ecommerce/product/_search
```

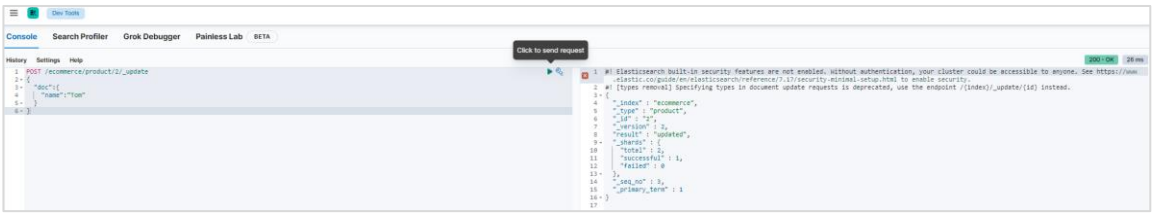


Note: Comparing the results with the previous data, you will notice that the **Product1** index data has been completely updated.

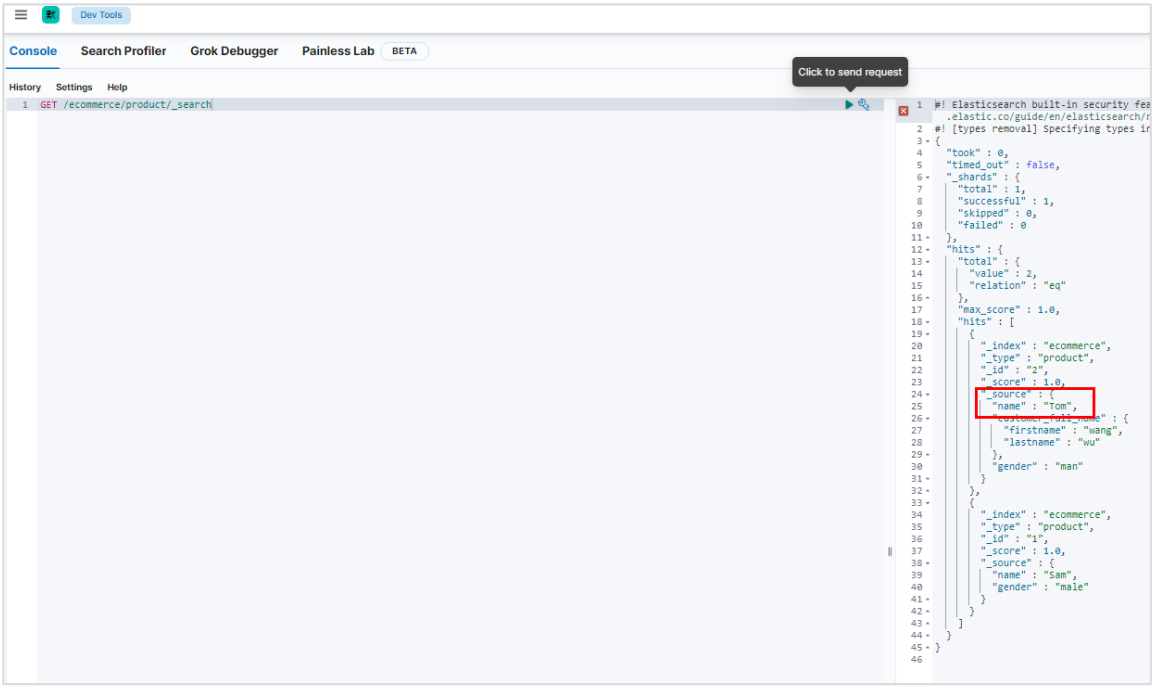
Use **POST** and **_update** to update index data.

Copyright © Huawei Technologies Co.,Ltd

```
POST /ecommerce/product/2/_update
{
  "doc":{
    "name":"Tom"
  }
}
```



Use the **GET** method to view index data changes.

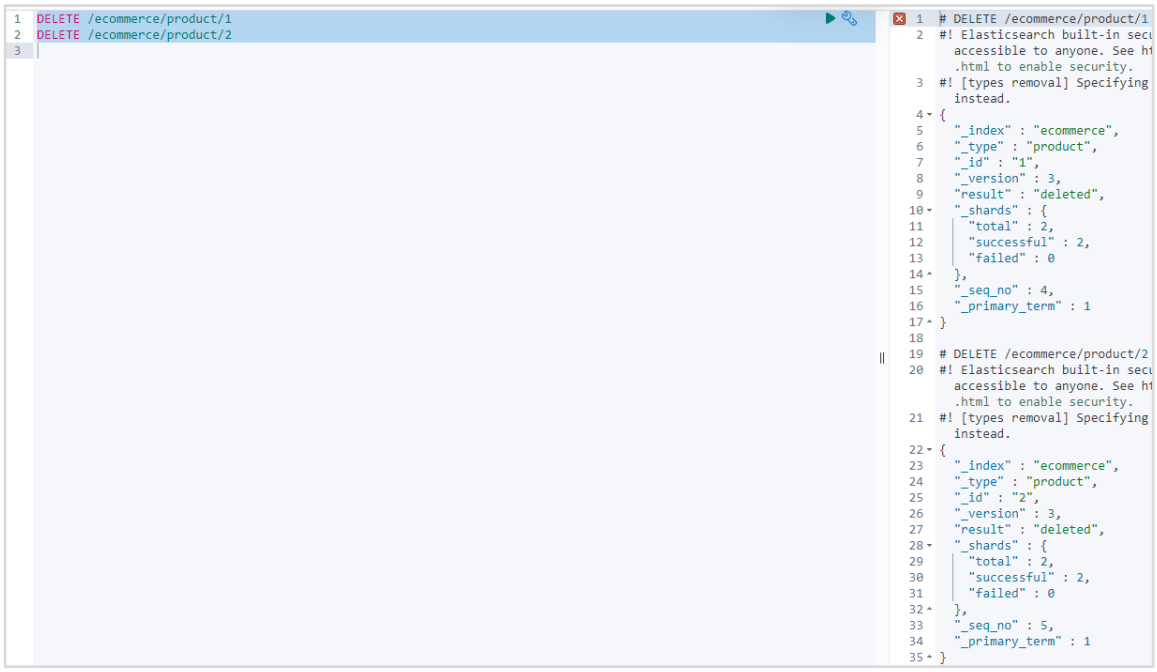


Note: Comparing the results with the previous data, you will notice that the **Product2** index data is partially updated.

Use the **DELETE** method to delete index data.

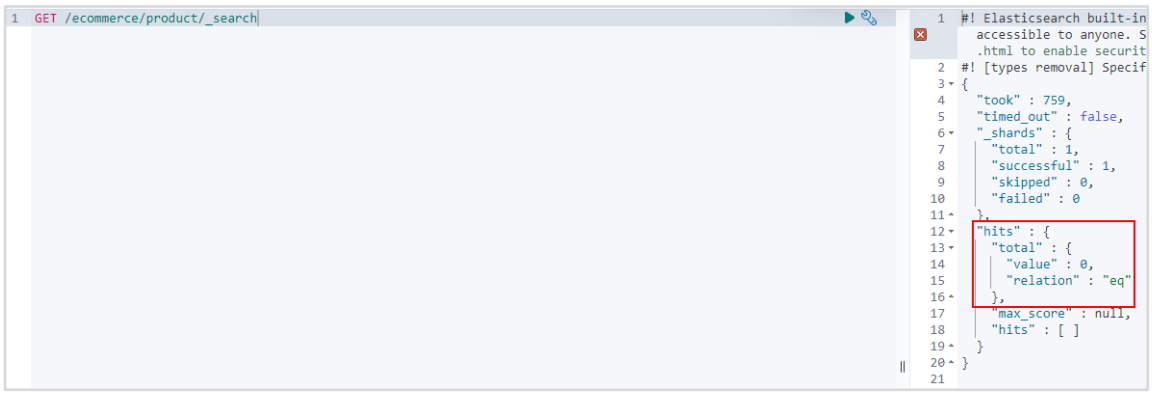
```
DELETE /ecommerce/product/1
DELETE /ecommerce/product/2
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



Use the **GET** method to view all index data.

```
GET /ecommerce/product/_search
```



Note: The **hits** command returns zero results, indicating the successful deletion of all index data.

Before proceeding with complex queries, populate the index with more data.

```
PUT /ecommerce/product/13
{
  "base_price" : 24.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Champion Arts",
  "tax_amount" : 0,
  "product_id" : 11238,
  "category" : "Women's Clothing",
  "sku" : "ZO0489604896",
  "taxless_price" : 24.99,
  "unit_discount_amount" : 0,
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
  "min_price" : 11.75,
  "discount_amount" : 0,
  "created_on" : "20XX-12-12T12:00:00+08:00",
  "product_name" : "Denim dress - black denim",
  "price" : 24.99,
  "taxful_price" : 24.99,
  "base_unit_price" : 24.99
}

PUT /ecommerce/product/14
{
  "base_price" : 11.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Elitelligence",
  "tax_amount" : 0,
  "product_id" : 6283,
  "category" : "Men's Clothing",
  "sku" : "ZO0549605496",
  "taxless_price" : 11.99,
  "unit_discount_amount" : 0,
  "min_price" : 6.35,
  "discount_amount" : 0,
  "created_on" : "20XX-12-13T13:00:00+08:00",
  "product_name" : "Basic T-shirt - dark blue/white",
  "price" : 11.99,
  "taxful_price" : 11.99,
  "base_unit_price" : 11.99
}

PUT /ecommerce/product/17
{
  "base_price" : 24.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Champion Arts",
  "tax_amount" : 0,
  "product_id" : 11238,
  "category" : "Women's Clothing",
  "sku" : "ZO0489604896",
  "taxless_price" : 24.99,
  "unit_discount_amount" : 0,
  "min_price" : 11.75,
  "discount_amount" : 0,
  "created_on" : "20XX-12-14T14:00:00+08:00",
  "product_name" : "Denim dress - black denim",
  "price" : 24.99,
  "taxful_price" : 24.99,
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

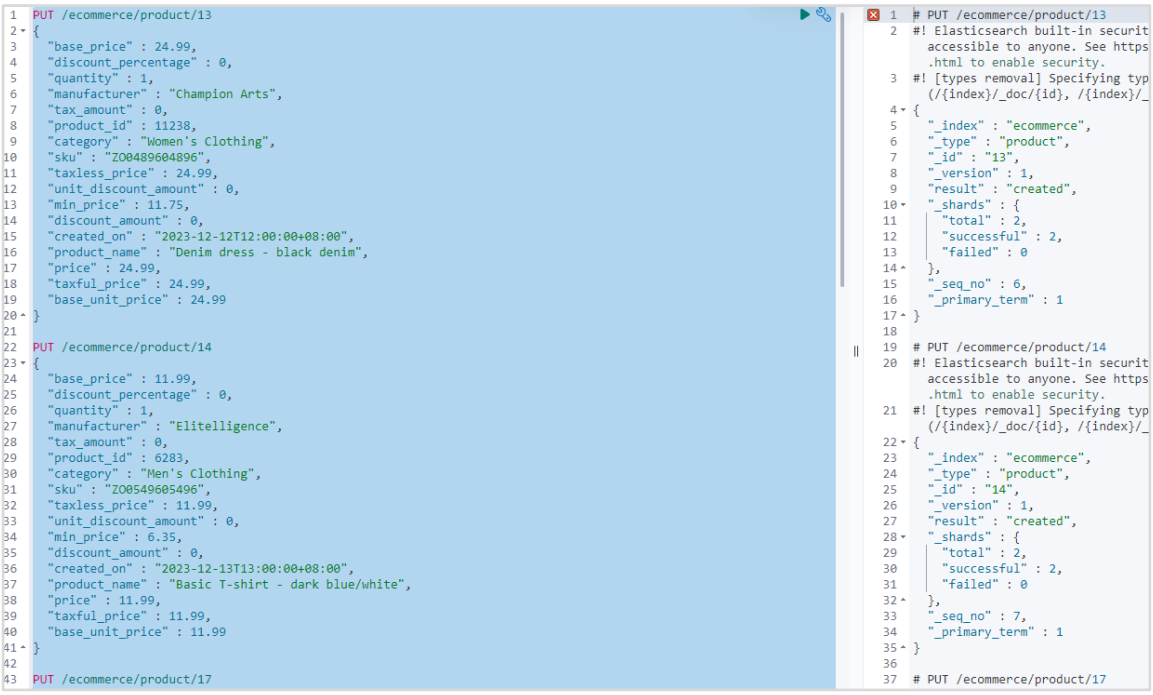
```
  "base_unit_price" : 24.99
}

PUT /ecommerce/product/15
{
  "base_price" : 99.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Low Tide Media",
  "tax_amount" : 0,
  "product_id" : 22794,
  "category" : "Women's Shoes",
  "sku" : "ZO0374603746",
  "taxless_price" : 99.99,
  "unit_discount_amount" : 0,
  "min_price" : 46.01,
  "discount_amount" : 0,
  "created_on" : "20XX-12-15T15:00:00+08:00",
  "product_name" : "Boots - Midnight Blue",
  "price" : 99.99,
  "taxful_price" : 99.99,
  "base_unit_price" : 99.99
}

PUT /ecommerce/product/16
{
  "base_price" : 74.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Primemaster",
  "tax_amount" : 0,
  "product_id" : 12304,
  "category" : "Women's Shoes",
  "sku" : "ZO0360303603",
  "taxless_price" : 74.99,
  "unit_discount_amount" : 0,
  "min_price" : 34.5,
  "discount_amount" : 0,
  "created_on" : "20XX-12-16T16:00:00+08:00",
  "product_name" : "High heeled sandals - argento",
  "price" : 74.99,
  "taxful_price" : 74.99,
  "base_unit_price" : 74.99
}
```

Note: Before using the preceding **PUT** method, replace **20XX** with the current year.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

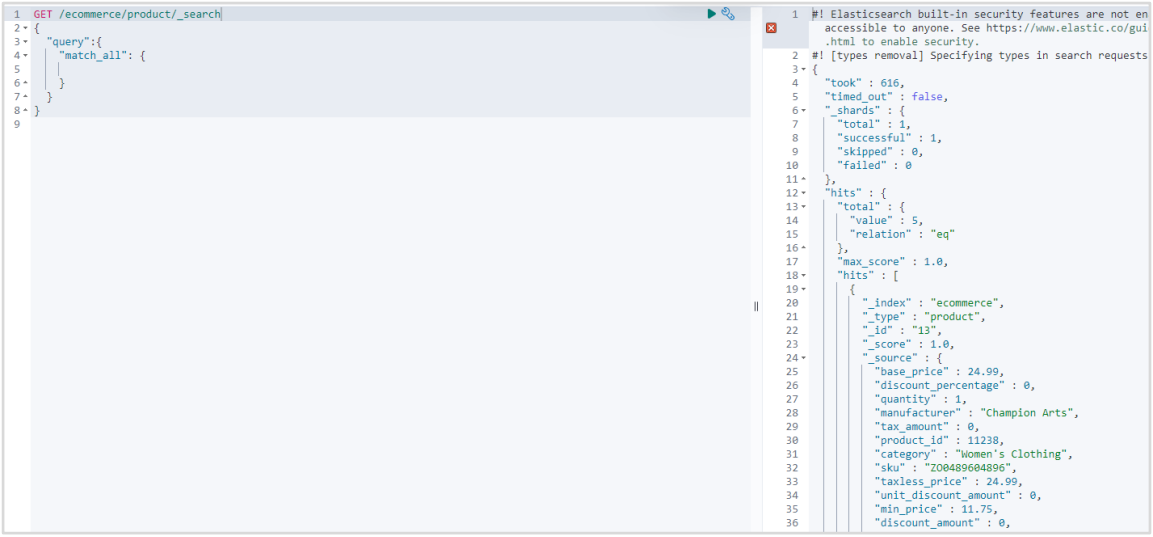


Step 3 Perform data match queries.

Use **match_all** to perform a query that matches all documents.

```
GET /ecommerce/product/_search
{
  "query":{
    "match_all": {

    }
  }
}
```

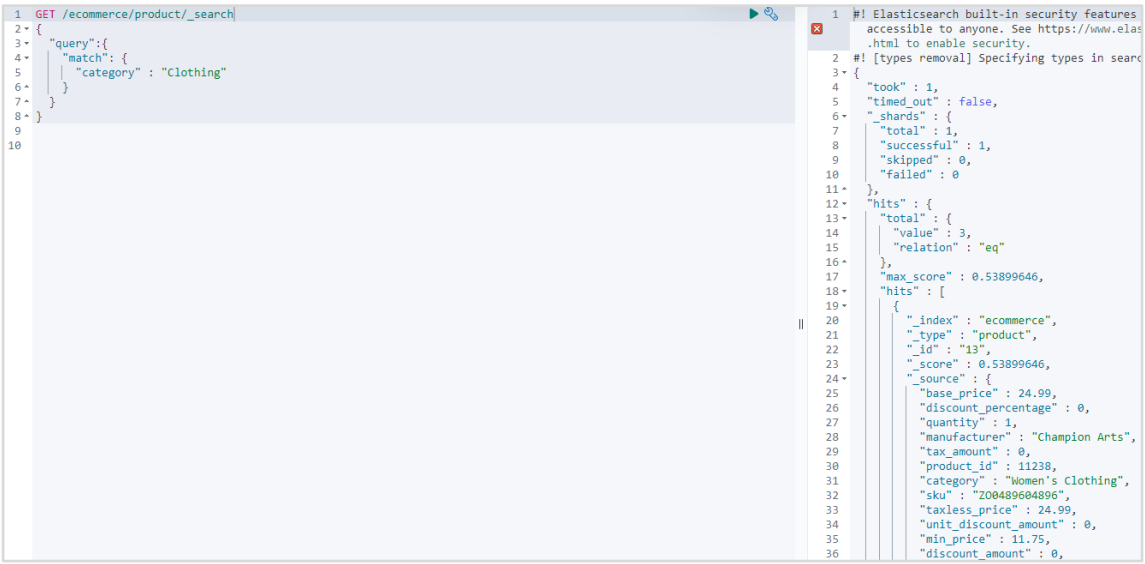


Use **match** to perform a query that matches documents based on a specific condition.

```
GET /ecommerce/product/_search
```

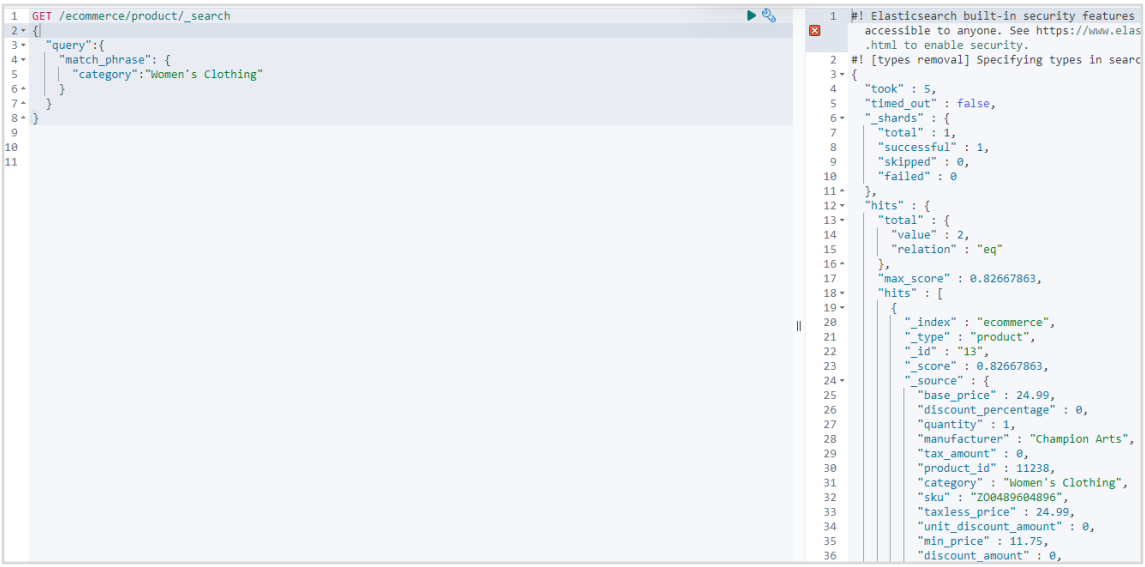
Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  }
}
```



Use **match_phrase** to perform an exact match query.

```
GET /ecommerce/product/_search
{
  "query":{
    "match_phrase": {
      "category":"Women's Clothing"
    }
  }
}
```



Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Note: You can also use **match** to search for **Women's Clothing**. However, **match** searches for **Women's** and **Clothing** separately and obtains the union set.

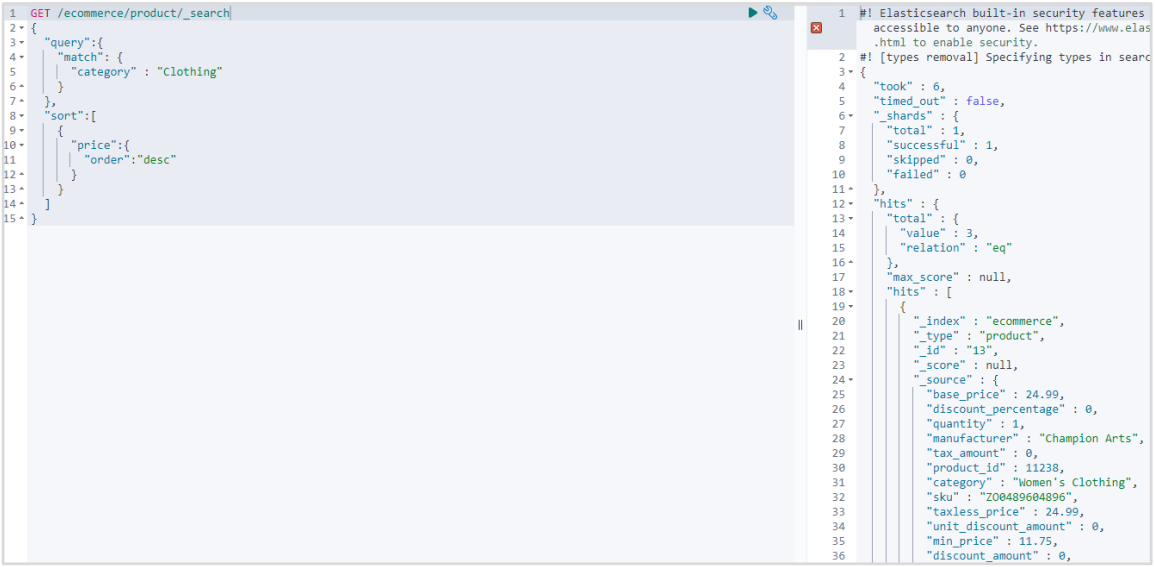
```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category":"Women's Clothing"
    }
  }
}
```

Step 4 Process data.

Sort results with **sort**.

```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
      }
    }
  ]
}
```

Note: **"order":"desc"** indicates sorting by price in descending order.

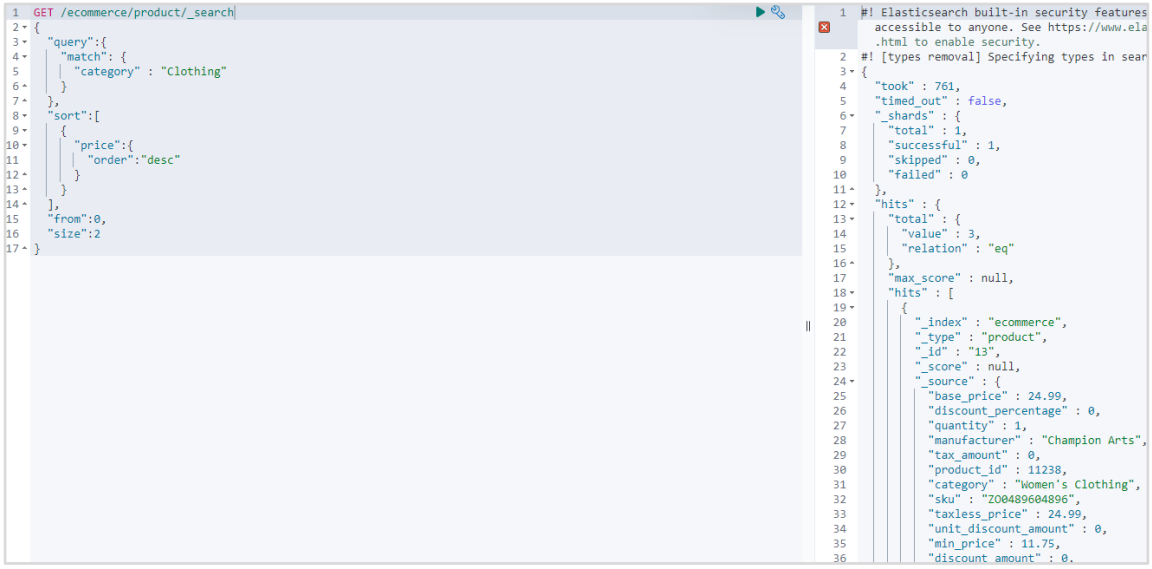


Paginate the results with **size**.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
      }
    }
  ],
  "from":0,
  "size":2
}
```

Note: **"from": 0** specifies the starting index. **"size": 2** specifies the number of results per page.



Return specific fields with **_source**:

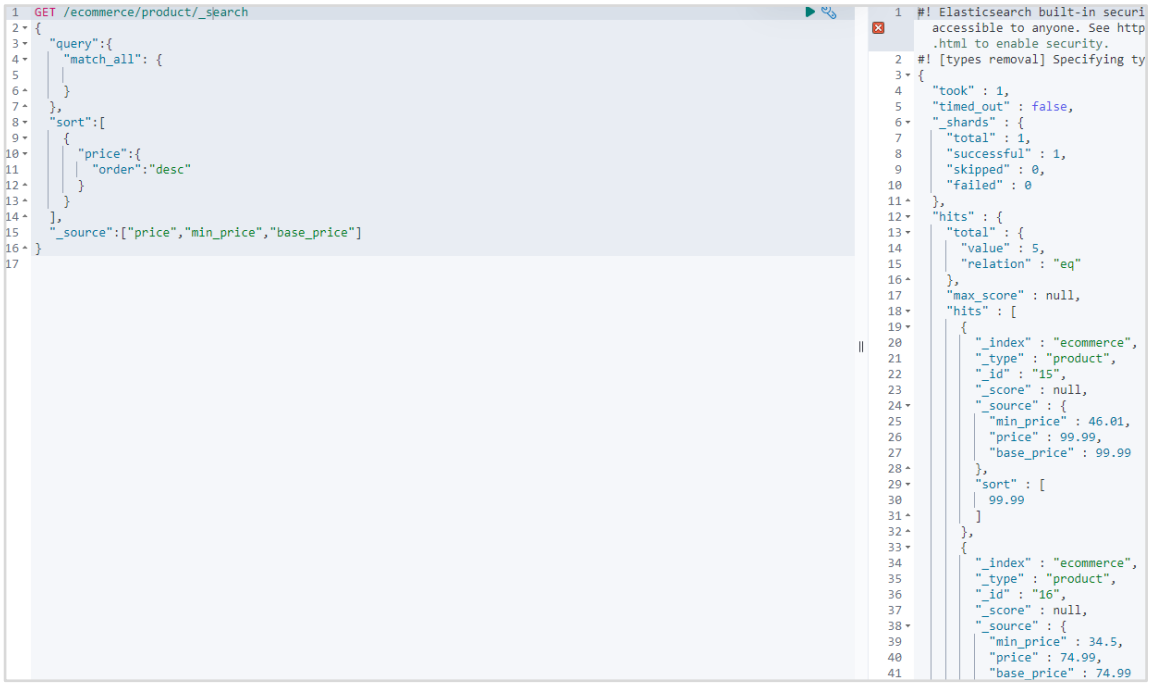
```
GET /ecommerce/product/_search
{
  "query":{
    "match_all": {

    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
```


Copyright © Huawei Technologies Co.,Ltd

```
        }
      }
    ],
    "_source":["price","min_price","base_price"]
}
```

Note: **"_source":["price","min_price","base_price"]** returns only price-related data.



Step 5 Perform a data filtering query.

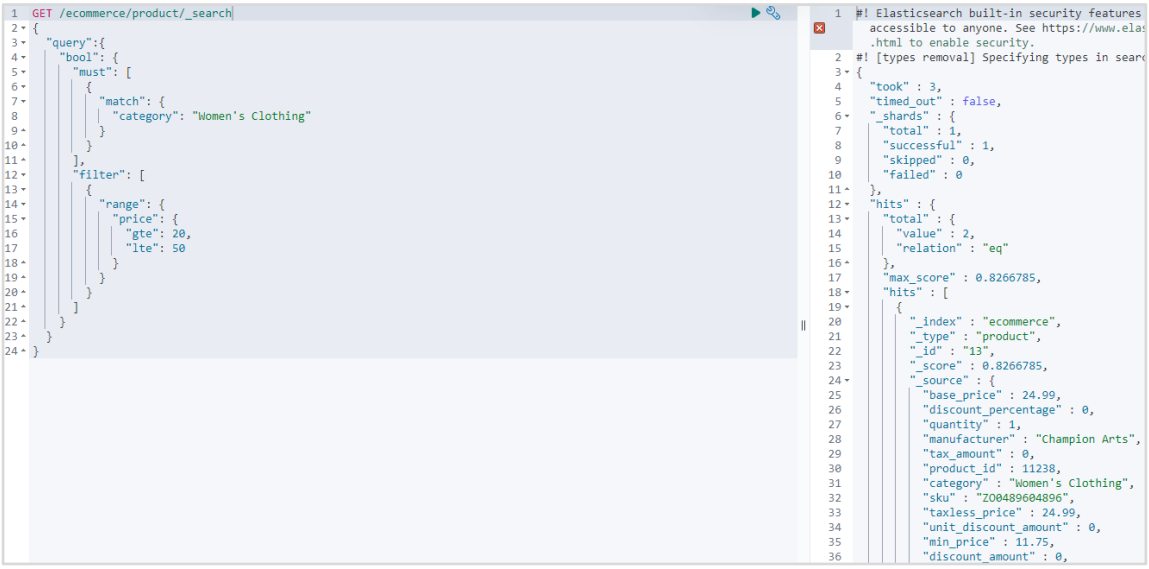
Use **filter** for conditional query.

```
GET /ecommerce/product/_search
{
  "query":{
    "bool": {
      "must": [
        {
          "match": {
            "category": "Women's Clothing"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "price": {
              "gte": 20,
              "lte": 50
            }
          }
```


Copyright © Huawei Technologies Co.,Ltd

```
        }
      ]
    }
  }
}
```

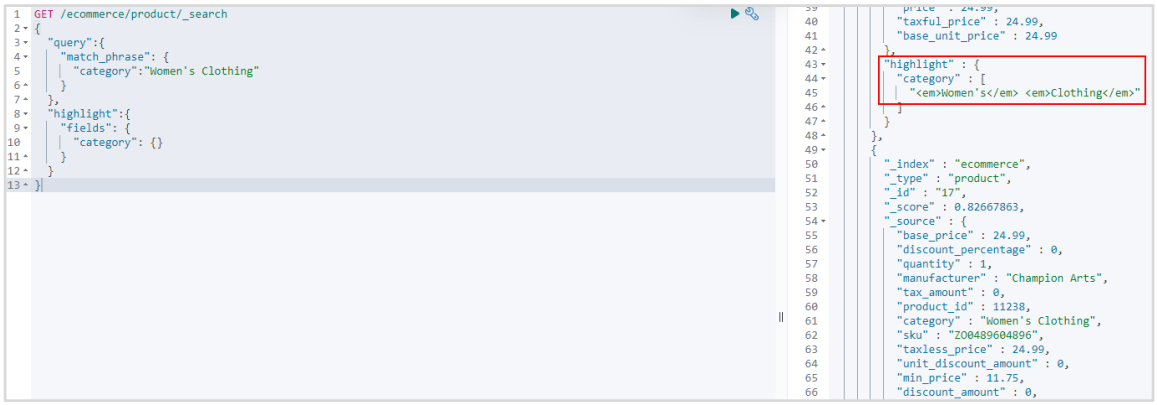
Note: The command searches for items whose name contains **Women's Clothing** and price is between 20 and 50.



Use **highlight** to highlight the query results.

```
GET /ecommerce/product/_search
{
  "query":{
    "match_phrase": {
      "category":"Women's Clothing"
    }
  },
  "highlight":{
    "fields": {
      "category": {}
    }
  }
}
```


Copyright © Huawei Technologies Co.,Ltd



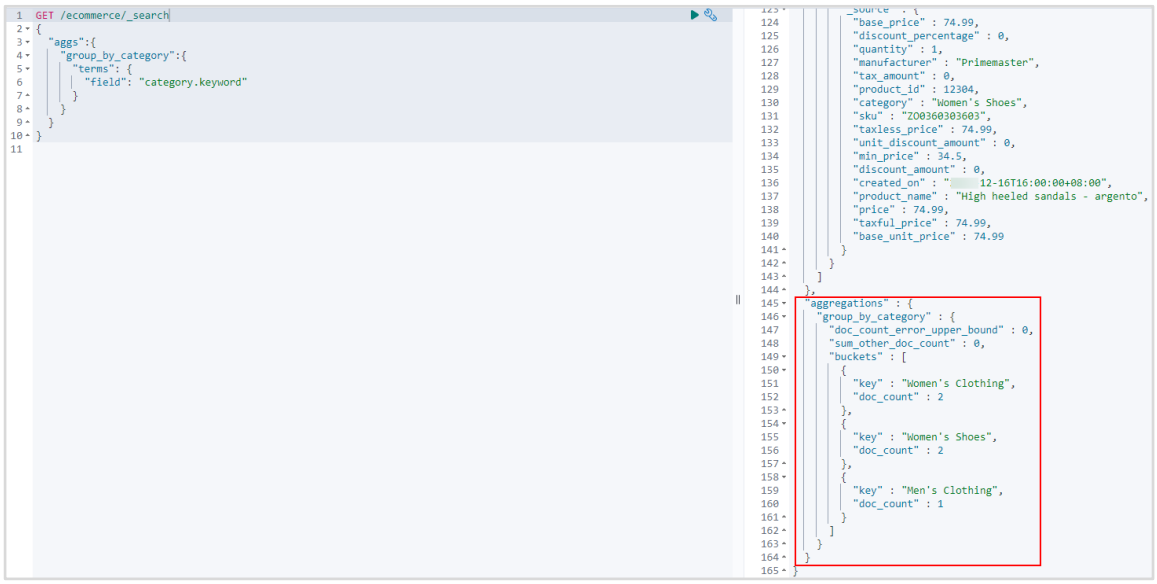
Note:

1. By default, Elasticsearch uses **** tags for highlighting. This can be customized, for example, using **** for red highlights on the web page.

2. Highlighting only applies to the keywords used in the **match** query.

Query the number of items in each clothes category.

```
GET /ecommerce/_search
{
  "aggs":{
    "group_by_category":{
      "terms": {
        "field": "category.keyword"
      }
    }
  }
}
```



Note:

1. Aggregation results are displayed in the **aggregations** section at the bottom of the query results.

2. To group and count by category (counting the number of products in each category):

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

- **group_by_category**: This is a custom name for the aggregation.
- **field**: This field requires the **.keyword** suffix. The category field is of the **text** type, which is not indexed by default. However, after text analysis, the extracted keywords are indexed. Therefore, you need to use **category.keyword** for accurate aggregation.

Query the average price and item count for each clothes category.

```
GET /ecommerce/_search
{
  "aggs":{
    "group_by_category":{
      "terms": {
        "field": "category.keyword"
      },
      "aggs": {
        "avg_price": {
          "avg": {
            "field": "price"
          }
        }
      }
    }
  }
}
```



Step 6 Perform a complex multi-conditional query.

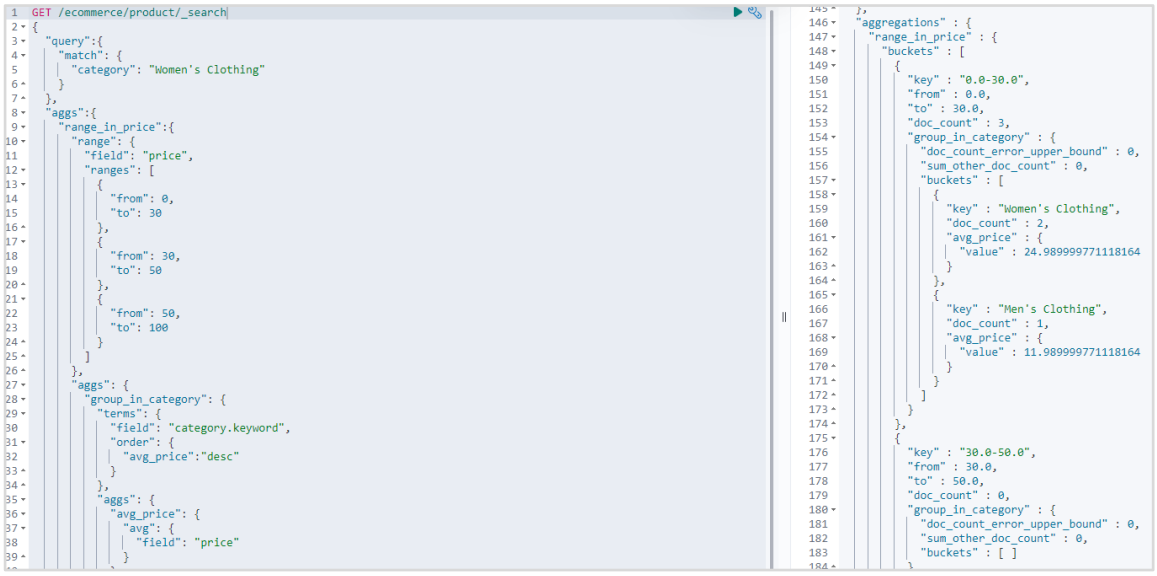
Retrieve data with **Women's Clothing** from **ecommerce**. Group this data by price, using the specified price ranges. Within each price group, further categorize the data by **category**. Finally, calculate the average price for each category within each price group and display the results in descending order of average price.

```
GET /ecommerce/product/_search
{
```


Copyright © Huawei Technologies Co.,Ltd

```
  "query":{
    "match": {
      "category": "Women's Clothing"
    }
  },
  "aggs":{
    "range_in_price":{
      "range": {
        "field": "price",
        "ranges": [
          {
            "from": 0,
            "to": 30
          },
          {
            "from": 30,
            "to": 50
          },
          {
            "from": 50,
            "to": 100
          }
        ]
      },
      "aggs": {
        "group_in_category": {
          "terms": {
            "field": "category.keyword",
            "order": {
              "avg_price":"desc"
            }
          },
          "aggs": {
            "avg_price": {
              "avg": {
                "field": "price"
              }
            }
          }
        }
      }
    }
  }
}
```

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd



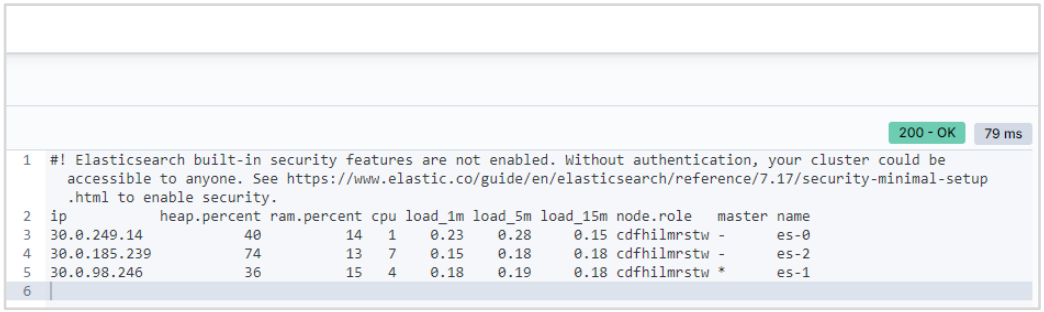
```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category": "Women's Clothing"
    }
  },
  "aggs":{
    "range_in_price":{
      "range": {
        "field": "price",
        "ranges": [
          {
            "from": 0,
            "to": 30
          },
          {
            "from": 30,
            "to": 50
          },
          {
            "from": 50,
            "to": 100
          }
        ]
      },
      "aggs": {
        "group_in_category": {
          "terms": {
            "field": "category.keyword",
            "order": {
              "avg_price":"desc"
            }
          },
          "aggs": {
            "avg_price": {
              "avg": {
                "field": "price"
              }
```

```
  },
  "aggregations" : {
    "range_in_price" : {
      "buckets" : [
        {
          "key" : "0.0-30.0",
          "from" : 0.0,
          "to" : 30.0,
          "doc_count" : 3,
          "group_in_category" : {
            "doc_count_error_upper_bound" : 0,
            "sum_other_doc_count" : 0,
            "buckets" : [
              {
                "key" : "Women's Clothing",
                "doc_count" : 2,
                "avg_price" : {
                  "value" : 24.989999771118164
                }
              },
              {
                "key" : "Men's Clothing",
                "doc_count" : 1,
                "avg_price" : {
                  "value" : 11.989999771118164
                }
              }
            ]
          }
        },
        {
          "key" : "30.0-50.0",
          "from" : 30.0,
          "to" : 50.0,
          "doc_count" : 0,
          "group_in_category" : {
            "doc_count_error_upper_bound" : 0,
            "sum_other_doc_count" : 0,
            "buckets" : [ ]
          }
```

4 Quiz

Question: Why does Kibana installed in this lab allow password-free login?

Answer: The StatefulSet YAML file for Elasticsearch does not include any security configurations. In the Dev Tools section, this lack of configuration triggers warning messages each time you run an Elasticsearch command, indicating that the security features of Elasticsearch are disabled.



200 - OK 79 ms

```
#! Elasticsearch built-in security features are not enabled. Without authentication, your cluster could be accessible to anyone. See https://www.elastic.co/guide/en/elasticsearch/reference/7.17/security-minimal-setup.html to enable security.
ip           heap.percent ram.percent cpu load_1m load_5m load_15m node.role   master name
30.0.249.14            40          14   1    0.23    0.28     0.15 cdfhilmrstw -      es-0
30.0.185.239           74          13   7    0.15    0.18     0.18 cdfhilmrstw -      es-2
30.0.98.246            36          15   4    0.18    0.19     0.18 cdfhilmrstw *      es-1
```

In a production environment, you need to configure the security feature for Elasticsearch. The procedure is as follows:

Step 1: Configure TLS on the Elasticsearch master node.

Go to the **master** directory and run the following command:

```
bin/elasticsearch-certutil cert -out config/elastic-certificates.p12 -pass ""
```

Open the **config/elasticsearch.yaml** file in a text editor. Add the following lines at the end of the file:

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

```
xpack.security.enabled: true
xpack.security.transport.ssl.enabled: true
xpack.security.transport.ssl.verification_mode: certificate
xpack.security.transport.ssl.keystore.path: elastic-certificates.p12
xpack.security.transport.ssl.truststore.path: elastic-certificates.p12
```

Save the file and start the master node by running the following command:

```
bin/elasticsearch
```

Leave this terminal running as it runs the executable file.

Step 2: Set Elasticsearch cluster passwords.

Open a new terminal and go to the master node directory. Generate random passwords for different internal stack users by running the following command:

```
bin/elasticsearch-setup-passwords auto
```

Step 3: Configure TLS on other Elasticsearch nodes.

Open a new terminal and go to the directory of another Elasticsearch node. Copy the entire configuration directory from the master node to the current node:

```
cp ../elasticsearch-7.1.0-master/config/* config/
```

Add the **node.master:false** configuration option to the **config/elasticsearch.yml** file. For details, see the Elasticsearch documentation: https://www.elastic.co/guide/en/elasticsearch/reference/7.1/modules-discovery-bootstrap-cluster.html. Alternatively, copy only the certificate file from the master node to the corresponding location on the current node. Ensure that all **xpack.security.*** keys have the same values as those on the master node.

Run **bin/elasticsearch** to start the node. You can see that the node is added to the cluster. In the master node terminal, you should see a message indicating that the node has joined the cluster.

Step 4: Configure Kibana security.

You need to set a password for the Kibana user. Retrieve the generated password from the output of the **elasticsearch-setup-passwords** command. Go to the Kibana directory and open the **config/kibana.yml** file in a text editor. Locate the following lines:

```
#elasticsearch.username: "user"
#elasticsearch.password: "pass"
```

Uncomment the **username** and **password** fields by removing the **#** symbol at the beginning of each line. Change **user** to **kibana** and replace **pass** with the password generated for **kibana**. Save the file. Now you can start Kibana securely by running **bin/kibana**.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

5 Acronyms and Abbreviations

Acronym/ Abbreviation	Full Spelling	Description
ECS	Elastic Cloud Server	A computing server consisting of CPU, memory, storage, and an operating system image, offering on-demand provisioning and flexible scalability.
DNS	Domain Name Server	A functional unit that provides domain name services on the Internet.
NFS	Networked File System	A file sharing service where a dedicated server manages storage space through a file system.
API	Application programming interface	A set of rules and specifications that allows different software applications to communicate with each other.
REST	Representational State Transfer	A software architectural style with a set of design principles and constraints.
EIP	Elastic IP	A static and public IP address.
PVC	PersistentVolumeClaim	A request for storage, which will be bound to a PersistentVolume, providing storage for a pod.
PV	PersistentVolume	A piece of network storage provisioned within the cluster.
SSD	Solid state drive	A storage device made up of solid electronic storage chip arrays.
BGP	Border Gateway Protocol	An autonomous system routing protocol that operates over TCP.

Huawei Proprietary and Confidential
Copyright © Huawei Technologies Co.,Ltd

Huawei openEuler Certification Training

HCIE-openEuler

Lab Guide

Issue: 1.0



Huawei Technologies Co., Ltd.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

 and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China

Website: https://e.huawei.com

Huawei Certification System

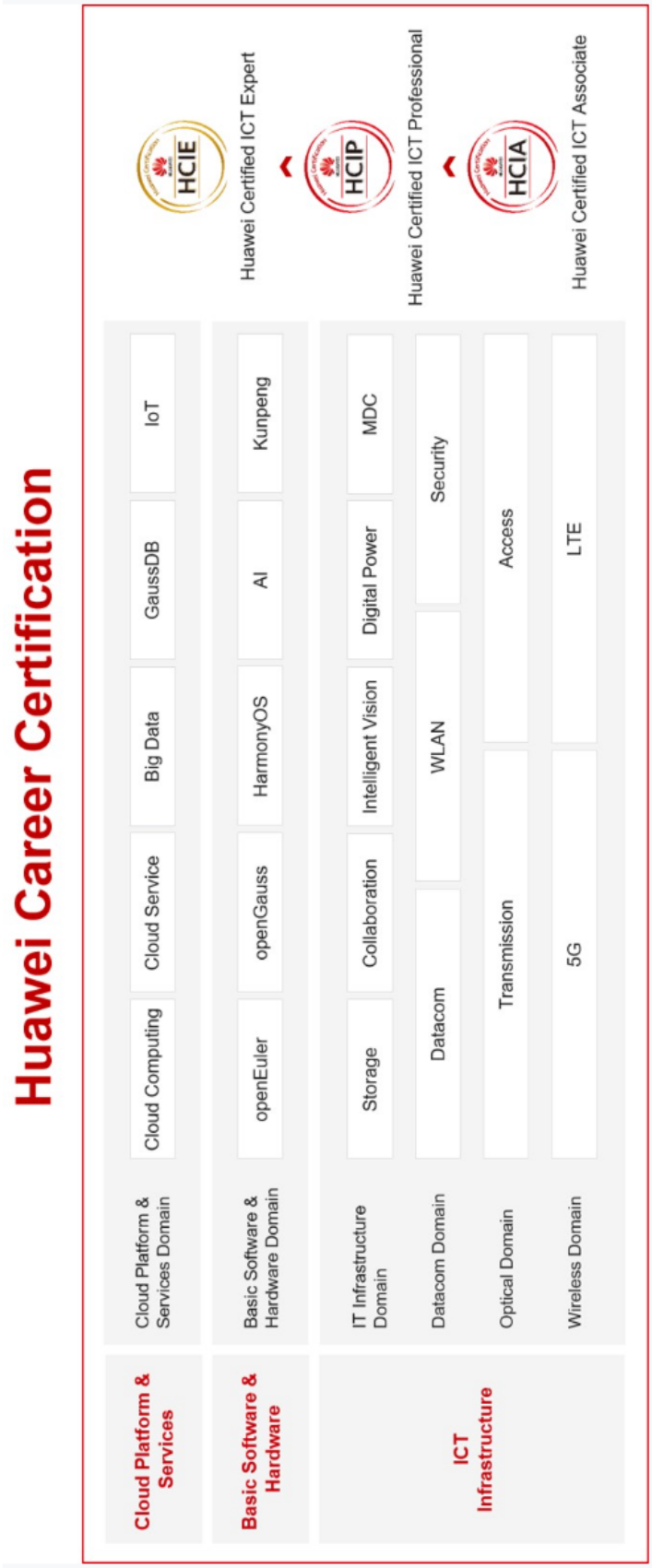
Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

Our programs cover all ICT fields and follow the industry's trend of ICT convergence. With our leading talent development system and certification standards, we are committed to fostering new digital ICT talent and building a sound ICT talent ecosystem.

HCIE-openEuler is intended for frontline engineers at Huawei regional offices or representative offices, and other personnel who want to learn openEuler technologies and usage scenarios. HCIE-openEuler certification covers the Ceph cloud storage management, the cloud operating system (OS) cluster management, advanced practices of cloud native platform Kubernetes, advanced automation management, continuous integration and continuous delivery (CI/CD), intelligent operation and maintenance (O&M) platform A-Ops, and log management system EFK.

Huawei certification helps you unlock opportunities to advance your career and take one more step towards the top of the industry.


Huawei Career Certification
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Basic Software & Hardware Domain
IT Infrastructure Domain
Datacom Domain
Optical Domain
Wireless Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
openEuler
openGauss
HarmonyOS
AI
Kunpeng
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom
WLAN
Security
Transmission
Access
5G
LTE
HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate

About This Document

Overview

This document is intended for candidates who are preparing for the HCIE-openEuler exam and readers who want to know how to implement cloud storage, cloud computing, cloud native, DevOps, monitoring, and log management on openEuler.

Description

This lab guide consists of three exercises, covering the installation and use of O&M monitoring software sysmonitor, deployment and use of the Huawei intelligent O&M platform A-Ops, and deployment and use of gala, the sub-health diagnosis tool provided by A-Ops.

- Exercise 1: installing and using sysmonitor. Use any host created during the HCIE-openEuler certification training or create an ECS to install sysmonitor and monitor the host NICs, system resources, signals, and key processes.
- Exercise 2: deploying and using A-Ops. Set up the A-Ops service on two management nodes, and deploy the agent on the production node to implement host management, vulnerability scan, and vulnerability fixes.
- Exercise 3: deploying and using gala. Deploy middleware and gala management components on one management node, and gala-gopher on different production nodes using different methods to monitor the performance of hosts, containers, and applications.

Background Knowledge Required

This course is for Huawei's certification. To better understand this course, the intended audience should:

- Know Linux basics. You are advised to complete HCIA-openEuler and HCIP-openEuler learning and pass the corresponding certification exams.

Lab Environment

Three servers are required as management nodes **aops-base**, **aops-mag**, and **gala-mag**, and their IP addresses are **172.16.0.101, 172.16.0.102**, and **172.16.0.105**, respectively. **aops-base** and **aops-mag** are the management nodes of A-Ops. **aops-base** is used to deploy basic services such as MySQL, Redis, Elasticsearch, and the zeus and hermes services of A-Ops. It is bound to an elastic IP address for browser access. **aops-mag** is used to deploy A-Ops service components. **gala-mag** is the management node of the

gala service and is used to deploy middleware such as Kafka and the gala exception detection and root cause location components. The specifications of the three management nodes are s7.2xlarge.2 (8 vCPUs and 16 GiB memory) each. The Ceph, GitLab, or Kubernetes cluster deployed in other exercises of the HCIE-openEuler training are used as production nodes to run the aops-ceres client and gala-gopher.

Lab Environment Preparation

Configure ECSs according to the following table. You can create ECSs that meet the requirements to deploy management node components, and use existing hosts as the production nodes or create production nodes based on your requirements.

Node Type	OS Version	Components to Be Deployed	Hardware Specifications
aops-base	openEuler 22.03 LTS	MySQL, Redis, Elasticsearch, aops-zeus, aops-hermes	8 vCPUs | 16 GiB
aops-mag	openEuler 22.03 LTS	gala-ragdoll, aops-apollo	8 vCPUs | 16 GiB
gala-mag	openEuler 22.03 LTS	gala-anteater, gala-spider, Kafka, Prometheus, Elasticsearch/Logstash, Pyroscope, ArangoDB, Grafana	8 vCPUs | 16 GiB | drive capacity ≥ 256 GiB
Production node	openEuler 22.03 LTS	gala-gopher, aops-ceres	Memory > 4 GiB

apos-base runs basic services such as MySQL, Redis, and Elasticsearch and A-Ops zeus and hermes services.

aops-mag runs the gala-ragdoll service for configuration tracing and the aops-apollo service for vulnerability management.

gala-mag runs middleware such as Kafka and the exception detection and root cause location components of gala.

Production node is a target node to be monitored and maintained. It typically runs service processes such as databases, Redis, and Java applications. If there are multiple production nodes, deploy gala-gopher on each of them.

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
Lab Environment 3
Lab Environment Preparation 4
1 Installing and Using sysmonitor 1
1.1 Introduction 1
1.1.1 About This Exercise 1
1.1.2 Objectives 1
1.2 Installing and Starting sysmonitor 1
1.3 Using Essential sysmonitor Functions 2
1.3.1 NIC Status Monitoring 2
1.3.2 Partition Space Monitoring 2
1.3.3 Signal Monitoring 3
1.4 Key Process Monitoring 4
1.5 Custom Monitoring 5
2 Deploying and Using A-Ops 7
2.1 Introduction 7
2.1.1 About This Exercise 7
2.1.2 Objectives 7
2.2 Installing and Configuring the A-Ops Server Framework 7
2.3 Installing and Configuring the A-Ops Client Framework 14
2.4 Using the A-Ops Framework 14
2.4.1 Essential A-Ops Operations 14
2.4.2 Host Vulnerability Scan and Fixes 16
2.5 Quiz 20
3 Acronyms and Abbreviations 23

1 Installing and Using sysmonitor

1.1 Introduction

1.1.1 About This Exercise

sysmonitor is a system O&M and monitoring utility. It detects exceptions during OS running and records them in system logs (**/var/log/sysmonitor.log**). Specifically, it can monitor the usage of system resources (such as drives, CPUs, and memory), the number of processes, threads, and handles, filesystem exceptions, as well as network interface card (NIC) and file operations (which are also recorded in logs). It can also monitor key processes and restore them when they are abnormal. In addition, sysmonitor allows users to define monitoring on custom resources.

This exercise demonstrates how sysmonitor monitors the NIC status, system resources, key processes, and custom items.

1.1.2 Objectives

- Be familiar with sysmonitor installation methods.
- Understand NIC status monitoring of sysmonitor.
- Understand system resource monitoring of sysmonitor.
- Understand key process monitoring of sysmonitor.
- Understand custom monitoring of sysmonitor.

1.2 Installing and Starting sysmonitor

Step 1 Install sysmonitor.

```
yum install -y sysmonitor-kmod
```

Step 2 Start the sysmonitor service.

The sysmonitor software package contains a configuration file for the rsyslog service. If sysmonitor is installed for the first time, restart the rsyslog service to make sysmonitor log configurations take effect. Then, start sysmonitor.

```
systemctl restart rsyslog
systemctl start sysmonitor
```

If "sysmonitor starting up" is displayed in the **/var/log/sysmonitor.log** file, the sysmonitor service has been started.

```
[root@monitor ~]# cat /var/log/sysmonitor.log  | grep "starting up"
2024-01-29T17:08:24.145626+08:00|info|sysmonitor[2151]: [------------------------sysmonitor starting up--------------------------]
2024-01-29T17:08:24.147962+08:00|info|sysmonitor[2151]: process monitor starting up
2024-01-29T17:08:24.148183+08:00|info|sysmonitor[2151]: file monitor starting up
2024-01-29T17:08:24.148228+08:00|info|sysmonitor[2151]: disk monitor starting up
2024-01-29T17:08:24.148814+08:00|info|sysmonitor[2151]: disk inode monitor starting up
2024-01-29T17:08:24.149185+08:00|info|sysmonitor[2151]: custom daemon monitor starting up
2024-01-29T17:08:24.149431+08:00|info|sysmonitor[2151]: custom periodic monitor starting up
2024-01-29T17:08:24.149594+08:00|info|sysmonitor[2151]: system resources monitor starting up
2024-01-29T17:08:24.149670+08:00|info|sysmonitor[2151]: cpu monitor starting up
2024-01-29T17:08:24.149723+08:00|info|sysmonitor[2151]: memory monitor starting up
2024-01-29T17:08:24.149775+08:00|info|sysmonitor[2151]: process count monitor starting up
2024-01-29T17:08:24.149810+08:00|info|sysmonitor[2151]: system event starting up
2024-01-29T17:08:24.157627+08:00|info|sysmonitor[2151]: signal monitor starting up
2024-01-29T17:08:24.157698+08:00|info|sysmonitor[2151]: process fd num monitor starting up
2024-01-29T17:08:24.157818+08:00|info|sysmonitor[2151]: netcard monitor starting up
2024-01-29T17:08:24.157973+08:00|info|sysmonitor[2151]: system fd num monitor starting up
2024-01-29T17:08:24.159481+08:00|info|sysmonitor[2151]: filesystem monitor starting up
```

1.3 Using Essential sysmonitor Functions

1.3.1 NIC Status Monitoring

Step 1 Set a NIC to be monitored.

In the sysmonitor network configuration file **/etc/sysmonitor/network**, add a NIC to be monitored (**ens192** for example).

```
# network event monitor, format: eth0 UP/DOWN/NEWADDR/DELADDR
ens192
```

Run the following command to reload the configuration file for the modification to take effect:

```
systemctl reload sysmonitor
```

Step 2 Perform a test.

Change the IP address of **ens192**, for example, from **192.168.1.117** to **192.168.1.118**. You can view the change record in the **/var/log/sysmonitor.log** file, as shown in the following figure.

```
Fib6 deleting fe80::20c:29ff:fe9b:acb8/128, comm: NetworkManager[1130], parent comm: systemd[1]
Fib6 deleting ff00::/8, comm: NetworkManager[1130], parent comm: systemd[1]
ens192: ip[fe80::20c:29ff:fe9b:acb8] prefixlen[64] is deleted, comm: NetworkManager[1130], parent comm: systemd[1]
Fib6 replace ff00::/8, comm: NetworkManager[1130], parent comm: systemd[1]
Fib4 deleting table=255 192.168.1.117/32, comm: NetworkManager[1130], parent comm: systemd[1]
ens192: ip[192.168.1.117] prefixlen[24] is deleted, comm: NetworkManager[1130], parent comm: systemd[1]
ens192: ip[fe80::20c:29ff:fe9b:acb8] prefixlen[64] is added, comm: NetworkManager[1130], parent comm: systemd[1]
ens192: ip[192.168.1.118] prefixlen[24] is added, comm: NetworkManager[1130], parent comm: systemd[1]
Fib6 replace fe80::20c:29ff:fe9b:acb8/128, comm: kworker/7:3[788], parent comm: kthreadd[2]
comm:systemd exe:systemd[1](parent comm:swapper/0 parent exe:[0]) send SIGTERM to comm:nm-dispatcher exe:nm-dispatcher[7862].
```

1.3.2 Partition Space Monitoring

Step 1 Prepare resources.

Add a drive to the host or use an existing drive. Create a 10 MB partition and mount the partition to **/mnt**.

```
[root@monitor mnt]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                   8:0    0   20G  0 disk
├─sda1                8:1    0    1G  0 part /boot
└─sda2                8:2    0   19G  0 part
  ├─openeuler-root  253:0    0   17G  0 lvm  /
  └─openeuler-swap  253:1    0    2G  0 lvm  [SWAP]
sdb                   8:16   0  150G  0 disk
└─sdb1                8:17   0   10M  0 part /mnt
sr0                  11:0    1  3.5G  0 rom
```

Step 2 Modify the configuration file.

Add the following content to the **/etc/sysmonitor/disk** file:

```
DISK="/mnt/" ALARM="80" RESUME="70"
```

Reload the service for the configuration to take effect.

```
systemctl reload sysmonitor
```

Step 3 Perform a test.

Run the **tail -f /var/log/sysmonitor.log** command to view the real-time logs. Open a new terminal and run the following commands to create an 8 MB file in the **/mnt/** directory and trigger an alarm:

```
cd /mnt
dd if=/dev/zero of=file1 bs=1M count=8
```

If no alarm is generated, run the following command to create a 1 MB file:

```
dd if=/dev/zero of=file2 bs=1M count=1
```

The following alarm is displayed on the terminal of real-time logs:

```
[illegible]371314+08:00|warning|sysmonitor[2151]: report disk alarm, /mnt/ used:100% alarm:80%
```

After **file1** and **file2** are deleted, the terminal displays the alarm recovery information as follows:

```
[illegible].436349+08:00|info|sysmonitor[2151]: report disk recovered, /mnt/ used:1% resume:70%
```

1.3.3 Signal Monitoring

Step 1 Check whether the signal monitoring function is enabled.

In the **/etc/sysmonitor/signal** file, check the following configuration items:

```
SIGKILL="on"
SIGTERM="on"
```

If the items are not **on**, set them to **on** and reload the service.

Step 2 Open a new terminal to monitor real-time logs in the **/var/log/sysmonitor.log** file.

Run the following command to create a background task:

```
[root@monitor mnt]# sleep 99d &
[1] 28718
```

The system returns the process ID of the task, for example, **28718**. Run the **kill -9 28718** command to kill the process. The following information is displayed in the real-time log terminal:



Information about the parent process of the signal sender is displayed in the brackets. A maximum of four levels of parent-child process relationships can be printed.

Step 3 Enable the firewalld service. The information about enabling the service is displayed in the real-time log terminal.

```
comm:systemctl exe:systemctl[32909](parent comm:bash parent exe:bash[17840]) send SIGTERM to
comm:systemd-tty-ask exe:systemd-tty-ask-password-agent[32910].(sshd[17839]<-sshd[17837]<-
sshd[1145]<-systemd[1])
```

It shows that the service is started by sshd.

1.4 Key Process Monitoring

Step 1 Install a service to be tested.

In this exercise, Nginx is used as an example. Install Nginx and start the service. Example commands:

```
yum install -y nginx
systemctl start nginx
```

Step 2 Configure the monitoring service.

Add the **nginx** configuration file to the **/etc/sysmonitor/process** directory. The file content is as follows:

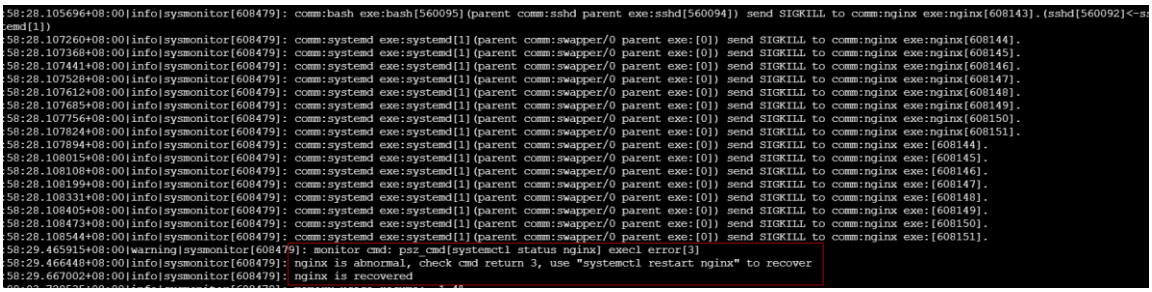
```
USER=root
NAME=nginx
RECOVER_COMMAND=systemctl restart nginx
MONITOR_COMMAND=systemctl status nginx
```

Change the permission on the file to 600 and reload the sysmonitor service. Example commands:

```
chmod 600 /etc/sysmonitor/process/nginx
systemctl reload sysmonitor
```

Step 3 Perform a test.

Run the **ps -aux | grep nginx** command to query the process ID of the Nginx main process, open a new terminal to view real-time logs, and run the **kill -9** command to kill the Nginx service. The system displays the following logs:



It shows that the Nginx service quickly recovers after it is killed.

1.5 Custom Monitoring

Step 1 Configure custom monitoring.

Create a script for custom monitoring, for example, **/usr/libexec/sysmonitor/record_top.sh**. The script content is as follows:

```
#!/bin/bash
echo `date` >> /var/log/top.log
top -b -n 1 >> /var/log/top.log
```

Step 2 Create a periodic monitoring task.

Add a **top_monitor** file to **/etc/sysmonitor.d/**. The file content is as follows:

```
MONITOR_SWITCH="on"
TYPE="periodic"
EXECSTART="/usr/libexec/sysmonitor/record_top.sh"
PERIOD="5"
```

Run the following commands to add permissions for the created script and monitoring task configuration file:

```
chmod 600 /etc/sysmonitor.d/top_monitor
chmod 550 /usr/libexec/sysmonitor/record_top.sh
```

Reload the service for the configuration to take effect.

```
systemctl reload sysmonitor
```

Step 3 Perform a test.

View the **/var/log/top.log** file. In a normal case, **top** outputs information every 5 seconds.

```
[root@gitlab sysmonitor.d]# cat /var/log/top.log
Wed D                AM UTC 20
top - 07:24:54 up 27 min,  2 users,  load average: 0.06, 0.16, 0.17
Tasks: 251 total,   2 running, 247 sleeping,   0 stopped,   2 zombie
%Cpu(s):  6.2 us,  4.7 sy,  0.0 ni, 87.5 id,  0.0 wa,  1.6 hi,  0.0 si,  0.0 st
MiB Mem :   7693.7 total,    923.2 free,   4695.1 used,   2075.4 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   2608.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
     22 root      20   0       0      0      0 S   6.2   0.0   0:00.05 ksoftirqd/2
    792 root      20   0 1865252  65096  29928 S   6.2   0.8   0:03.50 containerd
   5618 unbound   20   0 1704156 283392 147236 S   6.2   3.6   0:05.79 gitaly
   6375 systemd+  20   0 2615172 293480  73112 S   6.2   3.7   0:16.95 prometheus
  33451 root      20   0   10112   4248   3516 R   6.2   0.1   0:00.01 top
      1 root      20   0  103160  15420   9352 S   0.0   0.2   0:04.87 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
      7 root      20   0       0      0      0 I   0.0   0.0   0:00.10 kworker/u8:0-events_unbound
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
      9 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_rude_
     10 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_trace
     11 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/0
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.27 rcu_sched
     13 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     15 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     16 root      rt   0       0      0      0 S   0.0   0.0   0:00.05 migration/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.06 ksoftirqd/1
     19 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/2
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.04 migration/2
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/2:0H-events_highpri
     25 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/3
     26 root      rt   0       0      0      0 S   0.0   0.0   0:00.04 migration/3
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.07 ksoftirqd/3
     29 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/3:0H-events_highpri
     34 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     35 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 netns
     36 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     37 root      20   0       0      0      0 I   0.0   0.0   0:00.02 kworker/1:1-events
     38 root      20   0       0      0      0 I   0.0   0.0   0:00.03 kworker/2:1-cgroup_pidlist_destroy
     39 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     40 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
```

2 Deploying and Using A-Ops

2.1 Introduction

2.1.1 About This Exercise

A-Ops is a service used to improve the overall security of hosts. It provides functions such as asset management, CVE management, exception detection, and configuration tracing to identify and manage information assets, monitor software vulnerabilities, and rectify system faults on hosts, ensuring stable and secure running of hosts.

Hot patch is a technique that dynamically fixes software defects during program running. With the hot patch capability, A-Ops enables automated troubleshooting in batches and intelligent, efficient O&M.

2.1.2 Objectives

This exercise supplements the "Intelligent O&M Platform A-Ops" course. It aims to enhance understanding of hot patch usage by deploying the intelligent O&M service and using and managing hot patches.

2.2 Installing and Configuring the A-Ops Server Framework

Step 1 Configure the firewall.

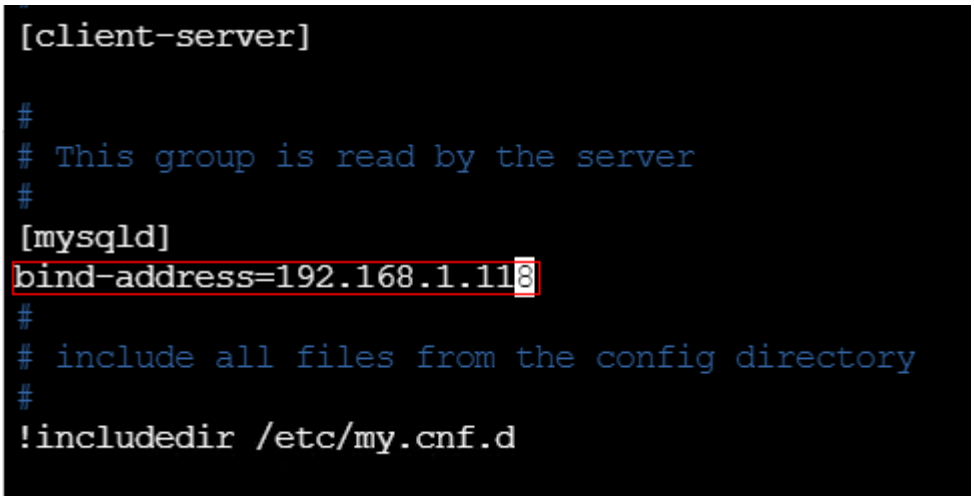
Based on the lab environment preparation requirements, create two servers **apos-base** and **apos-mag**, and enable port 11111 for the aops-zeus service and port 11116 for the aops-apollo service.

```
firewall-cmd --add-port=11111/tcp --permanent
firewall-cmd --add-port=11116/tcp --permanent
firewall-cmd --reload
firewall-cmd --query-port=11111/tcp --permanent
firewall-cmd --query-port=11116/tcp –permanent
```

Step 2 Install and configure MySQL.

Log in to **aops-base** to deploy MySQL, and add **bind-address=***local IP address* to the MySQL configuration file.

```
yum install mysql-server -y
vim /etc/my.cnf
[mysqld]
bind-address=192.168.1.118
```



Restart the MySQL service.

```
systemctl restart mysqld
```

Step 3 Create a database.

Connect to the database, set the access permission of the **root** user, and create the **aops** database.

```
mysql
show databases;
use mysql;
select user,host from user;
update user set host = '%' where user='root';
flush privileges;
create database aops default character set utf8mb4 collate utf8mb4_unicode_ci;
exit
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.01 sec)

mysql> use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select user,host from user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
4 rows in set (0.00 sec)

mysql> update user set host = '%' where user='root';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> flush privileges;
Query OK, 0 rows affected (0.01 sec)

mysql> create database aops default character set utf8mb4 collate utf8mb4_unicode_ci;
Query OK, 1 row affected (0.01 sec)

mysql> exit
Bye
[root@aops-base ~]#
```

Step 4 Install and configure Redis.

In the Redis configuration file, bind the local IP address.

```
yum install redis -y
vim /etc/redis.conf
bind 127.0.0.1   192.168.1.118
```

```
66 # IF YOU ARE SURE YOU WANT YOUR INSTANCE TO LISTEN TO ALL THE INTERFACES
67 # JUST COMMENT THE FOLLOWING LINE.
68 # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
69 bind 127.0.0.1 192.168.1.118
70
71 # Protected mode is a layer of security protection, in order to avoid that
```

Start the Redis service.

```
systemctl start redis
```

Step 5 Install and deploy aops-zeus.

Run the following command to install aops-zeus:

```
yum install aops-zeus -y
```

After the installation, run the following command to change **127.0.0.1** to the IP address of **aops-base** in the configuration file (local IP address in this exercise):

```
sed -i s/127.0.0.1/192.168.1.118/g /etc/aops/zeus.ini
```

Check that the modification is successful.

```
[zeus]
ip=192.168.1.118
port=11111

[uwsgi]
wsgi-file=manage.py
daemonize=/var/log/aops/uwsgi/zeus.log
http-timeout=600
harakiri=600
processes=2
gevent=100

[mysql]
ip=192.168.1.118
port=3306
database_name=aops
engine_format=mysql+pymysql://@%s:%s/%s
pool_size=100
pool_recycle=7200

[prometheus]
ip=192.168.1.118
port=9090
query_range_step=15s

[agent]
default_instance_port=8888

[redis]
ip=192.168.1.118
port=6379

[diana]
ip=192.168.1.118
port=11112

[apollo]
ip=192.168.1.118
port=11116
```

Install aops-tools and initialize aops-zeus.

```
yum install aops-tools -y
cd /opt/aops/scripts/deploy
```

```
bash aops-basedatabase.sh init zeus
```

Then, run the following command to start the aops-zeus service:

```
systemctl start aops-zeus
```

Step 6 Deploy aops-hermes.

Run the following command to install aops-hermes:

```
yum install aops-hermes -y
```

Modify the configuration file. Since the above services are deployed on **aops-base**, set the web access address of each service to the IP address of **aops-base**. In this exercise, **192.168.1.118** is used.

```
vim /etc/nginx/aops-nginx.conf
```

```
72          location /api/ {
73              proxy_pass http://192.168.1.118:11111/;
74          }
```

```
111          location /api/vulnerability {
112              proxy_pass http://192.168.1.118:11116/;
113              rewrite ^/api/(.*) /$1 break;
114          }
```

Run the following command to start aops-hermes:

```
systemctl start aops-hermes
```

Step 7 Deploy Elasticsearch.

Add an Elasticsearch repository.

```
echo "[aops_elasticsearch]
name=Elasticsearch repository for 7.x packages
baseurl=https://artifacts.elastic.co/packages/7.x/yum
gpgcheck=1
gpgkey=https://artifacts.elastic.co/GPG-KEY-elasticsearch
enabled=1
autorefresh=1
type=rpm-md" > "/etc/yum.repos.d/aops_elascticsearch.repo"
```

Run the following command to install Elasticsearch:

```
yum install elasticsearch-7.14.0-1 -y
```

Modify the Elasticsearch configuration file as follows:

```
vim /etc/elasticsearch/elasticsearch.yml
```



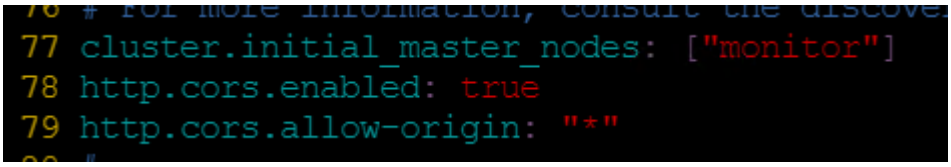
Change **node.name** to the current host name.



Change **network.host** to the IP address of the current host. In this exercise, **192.168.1.118** is used. Retain the default port number.

Add the following content to **Discovery**:

```
cluster.initial_master_nodes: ["monitor"]
http.cors.enabled: true
http.cors.allow-origin: "*"
```



Restart the Elasticsearch service.

```
systemctl restart elasticsearch
```

Step 8 Install and deploy aops-apollo.

Run the following command to install aops-apollo:

```
yum install aops-apollo -y
```

After the installation, change **127.0.0.1** to the local IP address in the configuration file. In this exercise, **192.168.1.118** is used.

```
vim /etc/aops/apollo.ini
```

```
[apollo]
ip=192.168.1.118
port=11116
host_vault_dir=/opt/aops
host_vars=/opt/aops/host_vars

[zeus]
ip=192.168.1.118
port=11111

; herems info is used to send mail.
[hermes]
ip=192.168.1.118
port=8000

[cve]
cve_fix_function=yum
# value between 0-23, for example, 2 means 2:00 in a day.
cve_scan_time=2

[mysql]
ip=192.168.1.118
port=3306
database_name=aops
engine_format=mysql+pymysql://@%s:%s/%s
pool_size=100
pool_recycle=7200

[elasticsearch]
ip=192.168.1.118
port=9200
max_es_query_num=10000000

[redis]
ip=192.168.1.118
port=6379

[uwsgi]
wsgi-file=manage.py
daemonize=/var/log/aops/uwsgi/apollo.log
http-timeout=600
harakiri=600
processes=1
gevent=100
~
```

After the modification, run the following commands to initialize aops-apollo using aops-tools:

```
cd /opt/aops/scripts/deploy
bash aops-basedatabase.sh init apollo
```

Then, run the following command to start the aops-apollo service:

```
systemctl start aops-apollo
```

2.3 Installing and Configuring the A-Ops Client Framework

Install aops-ceres and dnf-hotpatch-plugin on clients. In this exercise, you will scan and fix vulnerabilities on three Kubernetes master nodes. Therefore, install the two services on the three master nodes.

Step 1 Install aops-ceres.

As the client of A-Ops, aops-ceres exchanges data with the A-Ops management center over HTTP. Run the following command on the three master nodes to install aops-ceres:

```
yum install aops-ceres -y
```

Step 2 Install dnf-hotpatch-plugin.

The dnf-hotpatch-plugin package is used to inspect CVE vulnerabilities, install and upgrade hot patches, and query hot patch status. Run the following command to install this package on the three master nodes:

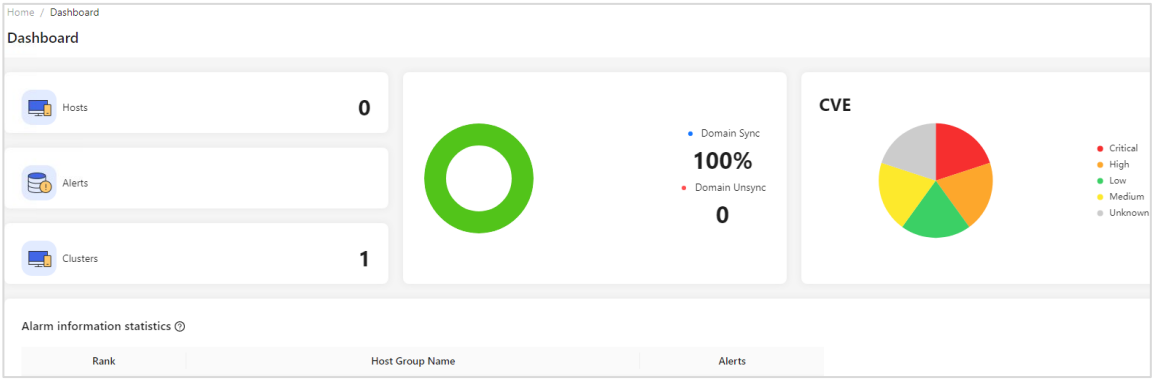
```
dnf install dnf-hotpatch-plugin -y
```

2.4 Using the A-Ops Framework

2.4.1 Essential A-Ops Operations

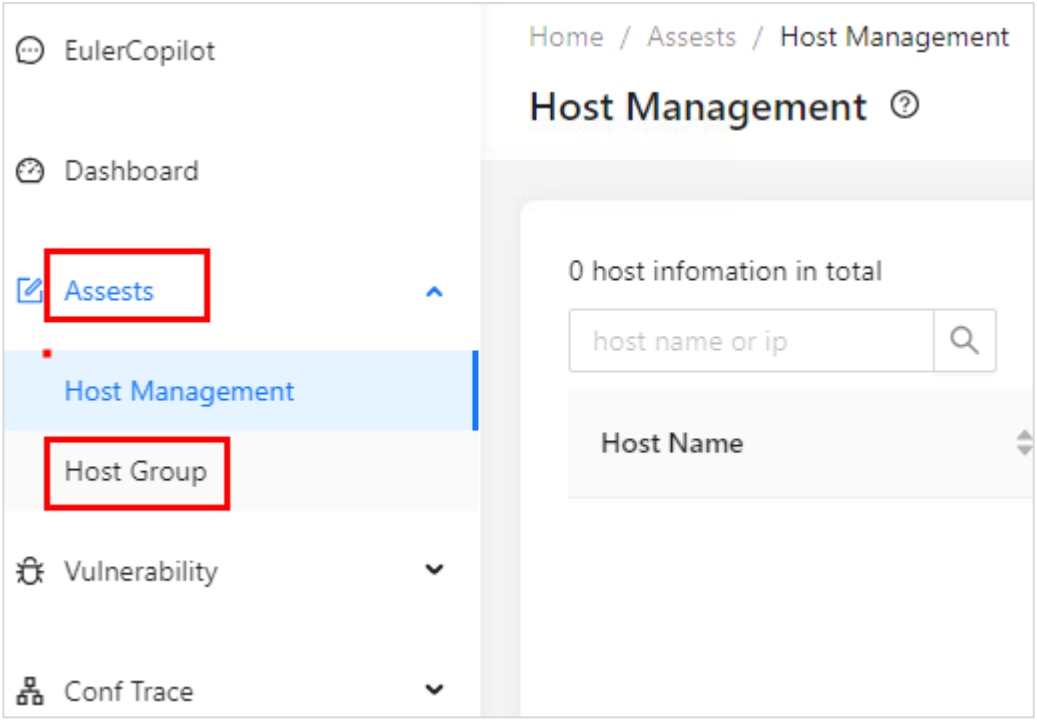
Step 1 Log in to A-Ops.

Access **http://***aops-mag_EIP*. The default user name and password are **admin** and **changeme**, respectively.

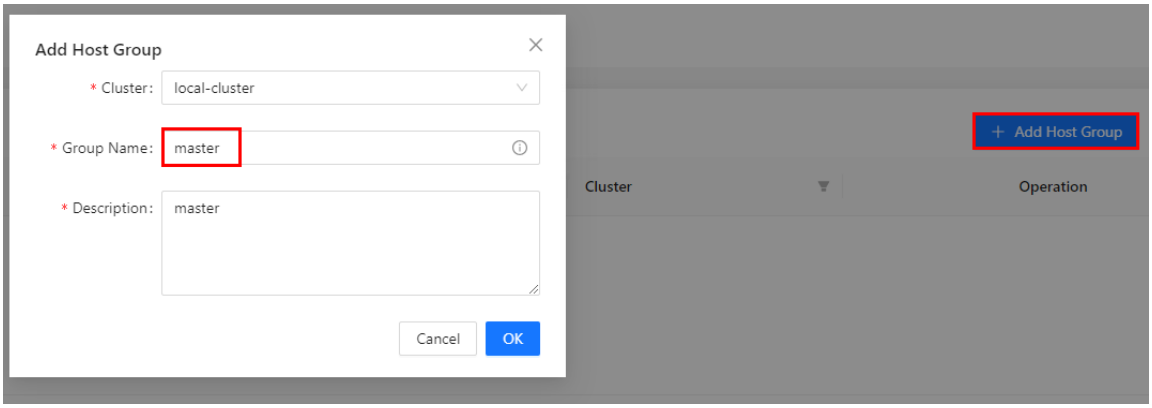


Step 2 Add a host group.

Choose **Asset** > **Host Groups** and click **Add Host Group**.



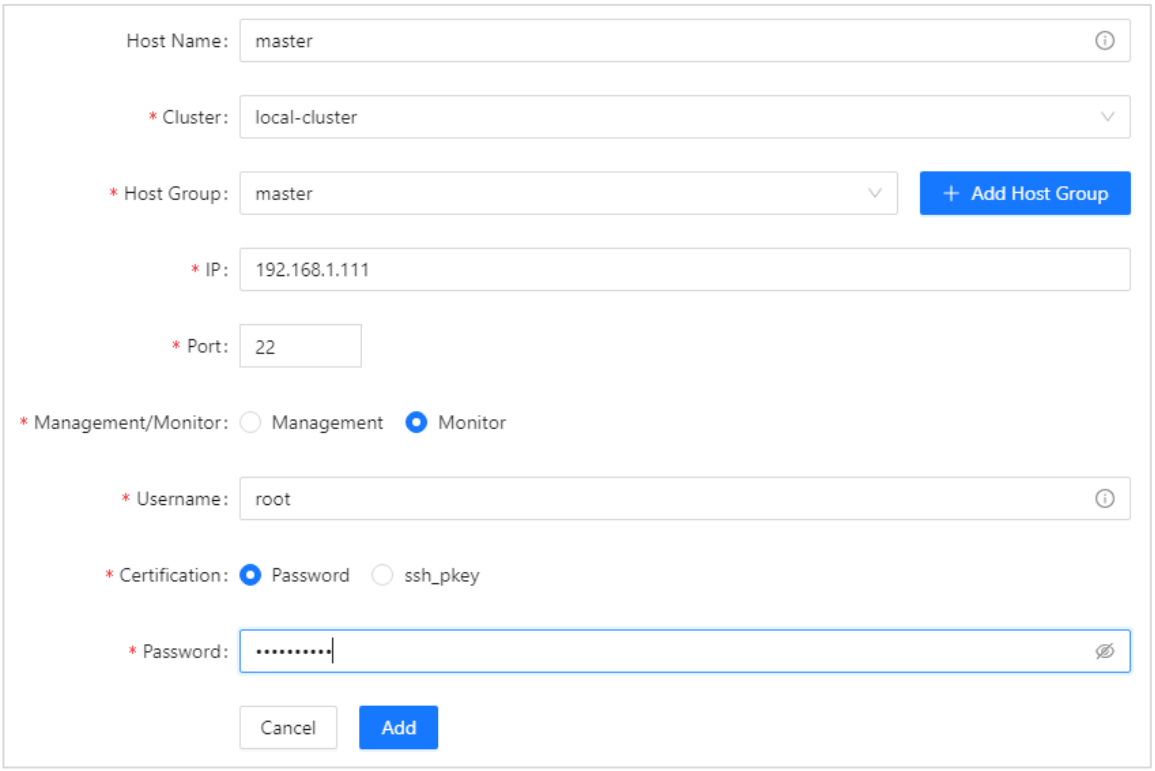
In the displayed dialog box, enter the host group name and description.



Click **OK**.

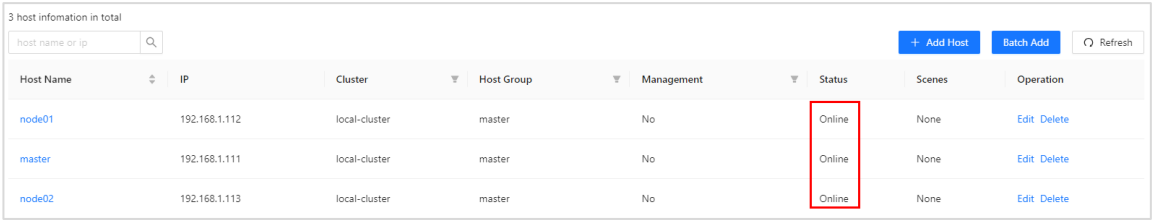
Step 3 Add a host.

Choose **Asset** > **Hosts Management** and click **Add Host** in the upper right corner. On the displayed page, enter host information as required.



Click **Add**.

Use the same method to add the other two masters for host management.

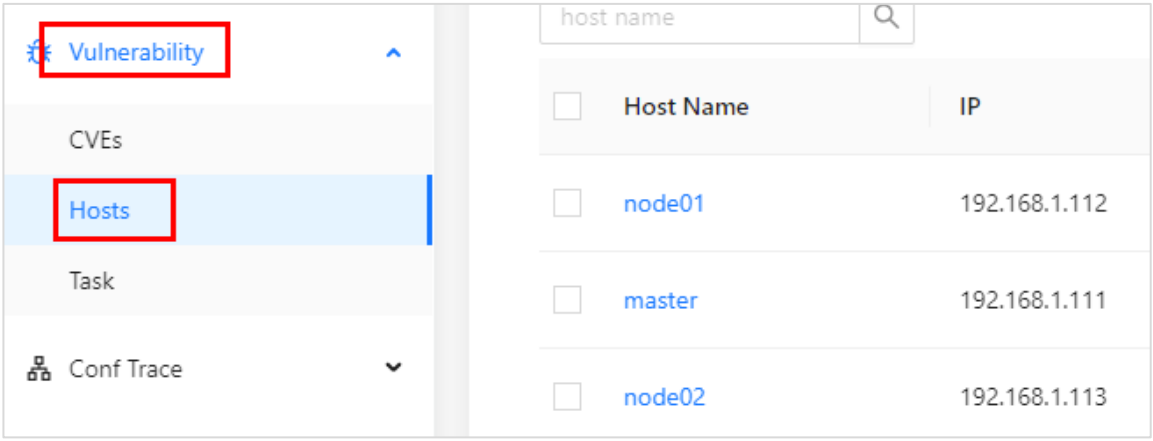


2.4.2 Host Vulnerability Scan and Fixes

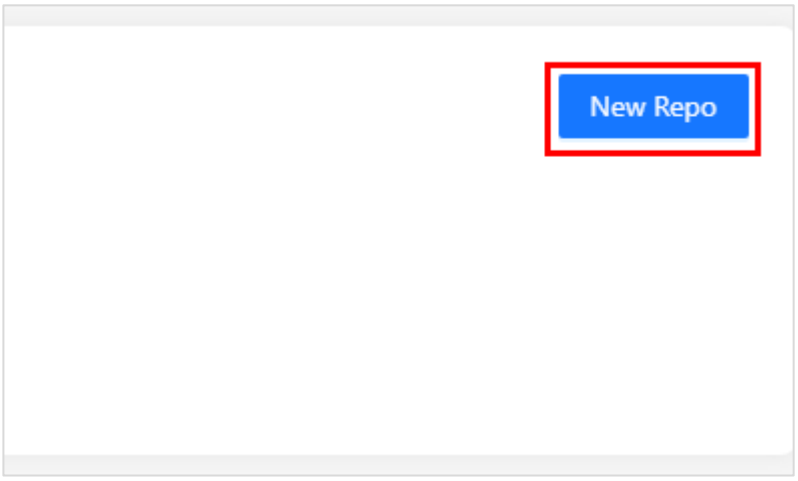
Step 1 (Optional) Add a CVE repository.

If a monitored host is not configured with the Yum repository of update packages matching the system version, deliver the Yum repository to the host as follows:

On the home page, choose **Vulnerability** > **Hosts**.

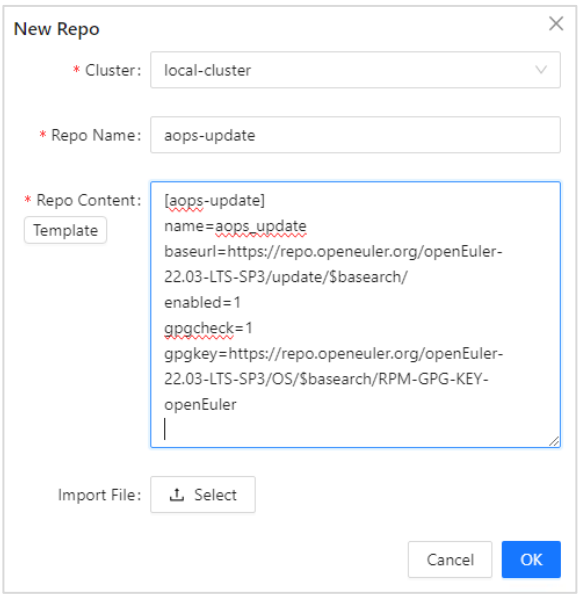


Click **New Repo** at the bottom of the page.

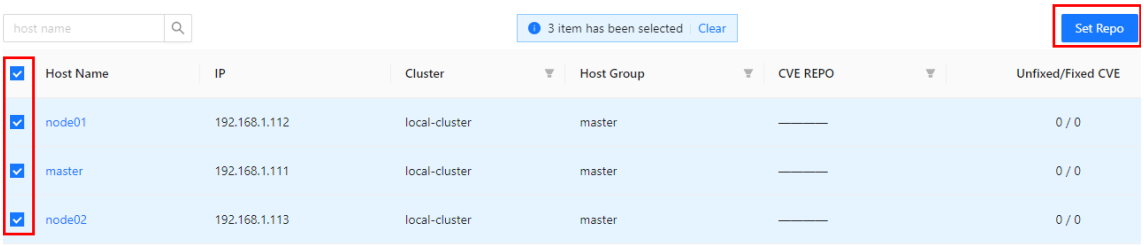


On the displayed page, enter the repository name and other information as follows:

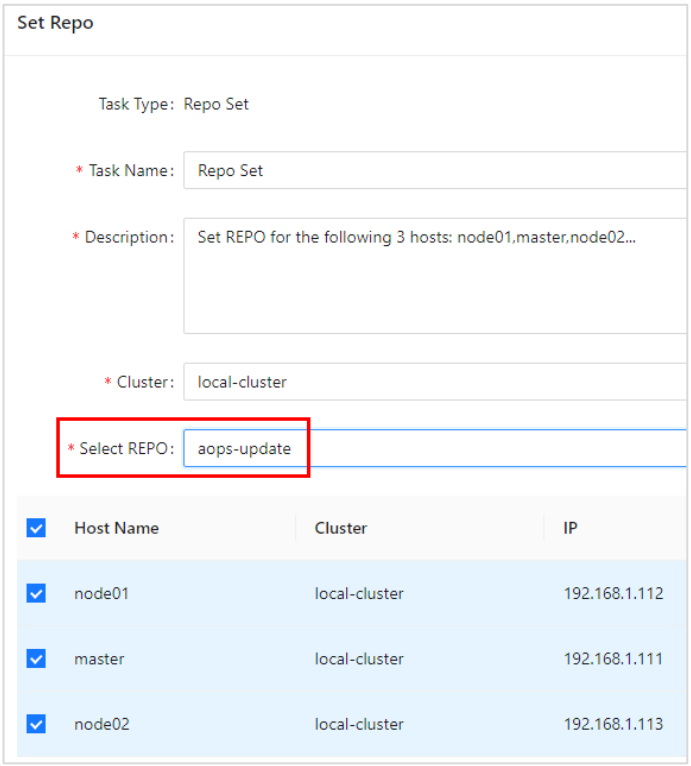
```
[aops-update]
name=aops_update
baseurl=https://repo.openeuler.org/openEuler-22.03-LTS-SP3/update/$basearch/
enabled=1
gpgcheck=1
gpgkey=https://repo.openeuler.org/openEuler-22.03-LTS-SP3/OS/$basearch/RPM-GPG-KEY-openEuler
```



Select all hosts and click **Set Repo**.

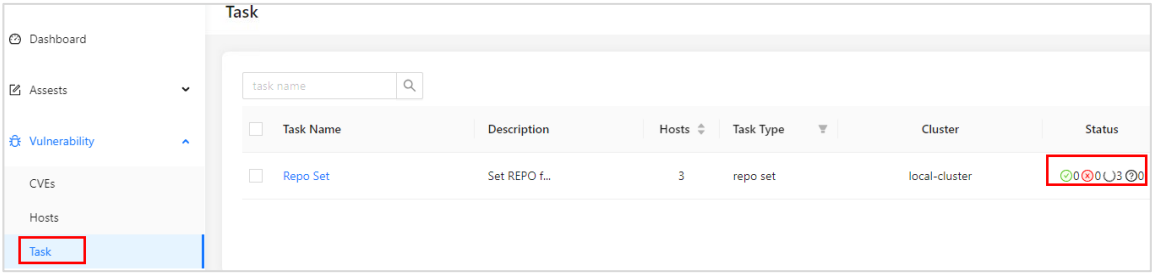


In the displayed dialog box, select the specified repository.



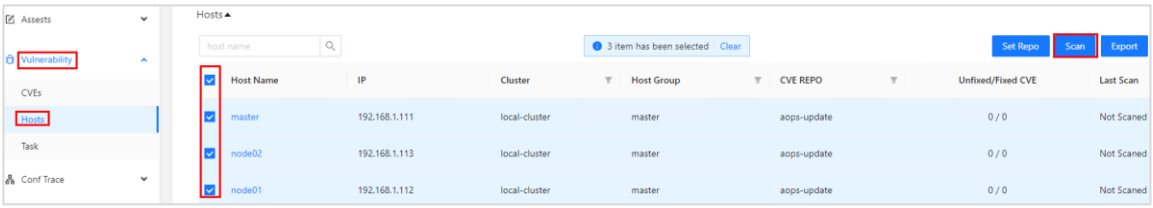
Z

Click **Execute**. Then you can check the execution status in the **Tasks** page.



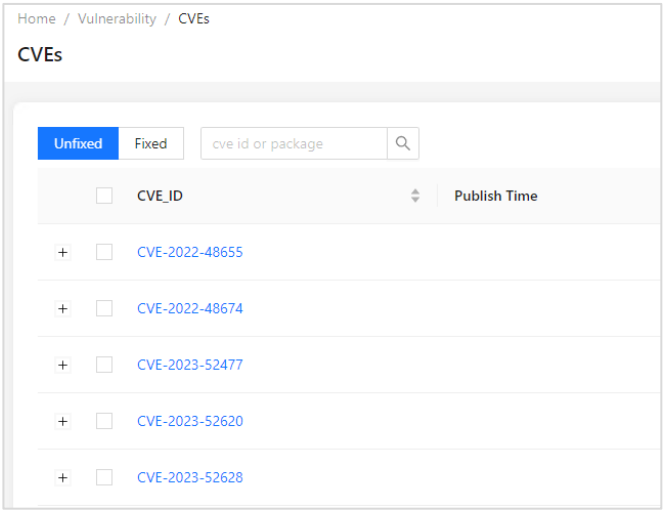
Step 2 Scan vulnerabilities.

Choose **Vulnerability** > **Hosts**, select the monitored hosts, and click **Scan**.



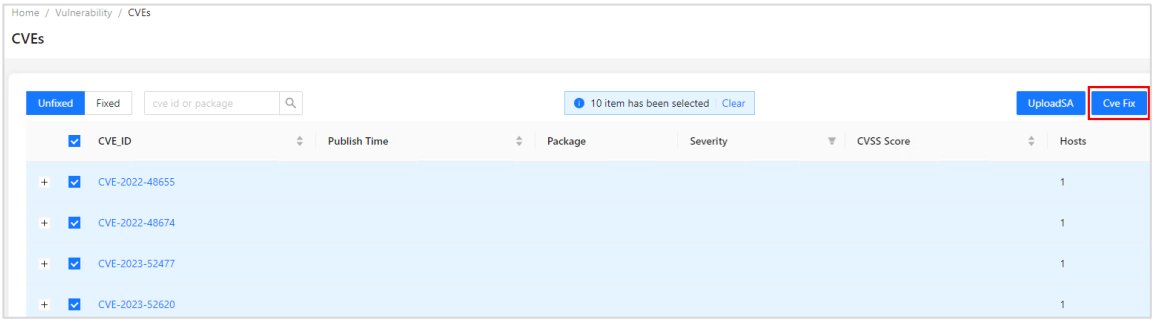
Step 3 Query vulnerabilities.

Click **CVEs** to check the vulnerabilities to be fixed.

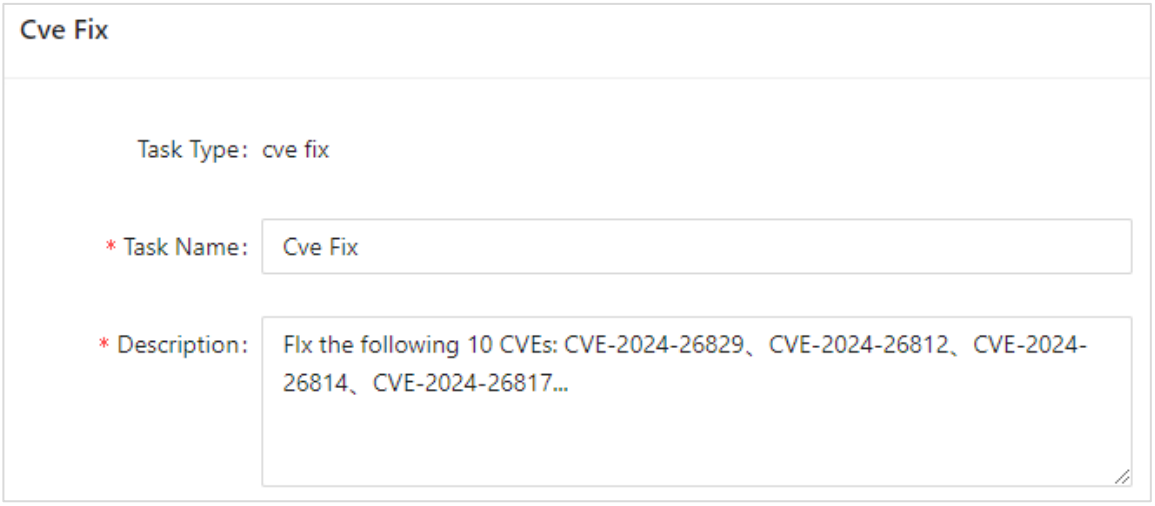


Step 4 Fix vulnerabilities.

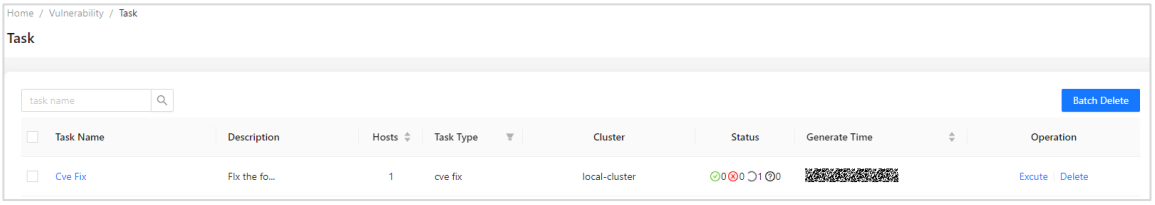
Select all CVEs and click **CVE Fix**.



In the displayed dialog box, enter the task name and description, or use the automatically generated content, and then click **Execute**.



On the **Tasks** page, check the task execution status.



After the execution is complete, check the host details.



The CVEs are still unfixed because the cold patch mode is used. The CVEs can be fixed only after the host is restarted and the new kernel takes effect.

2.5 Quiz

Question: After A-Ops is set up, you can access the login page. After you enter the correct user name and password, a wrong request message is displayed in the upper right corner. How to rectify the fault?

Answer: Check the zeus logs in the **/var/log** directory and ensure that database initialization has been performed after zeus is installed. Then check whether the status of each service, network, and database connection are normal.

```
]
(Background on this error at: https://sqlalche.me/e/14/2j85)

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/usr/lib/python3.9/site-packages/zeus/manage.py", line 85, in <module>
    app, config = main()
  File "/usr/lib/python3.9/site-packages/zeus/manage.py", line 80, in main
    init_database()
  File "/usr/lib/python3.9/site-packages/zeus/manage.py", line 65, in init_database
    init_user()
  File "/usr/lib/python3.9/site-packages/zeus/manage.py", line 39, in init_user
    raise sqlalchemy.exc.SQLAlchemyError("create tables fail")
sqlalchemy.exc.SQLAlchemyError: create tables fail
unable to load app 0 (mountpoint='') (callable not found or import error)
*** no app loaded. going in full dynamic mode ***
2023-12-11 07:50:02,927 INFO account/add_user/77: add user succeed
2023-12-11 07:50:02,928 INFO manage/init_user/58: initialize default admin user succeed
WSGI app 0 (mountpoint='') ready in 1 seconds on interpreter 0x55dbf8f9e5f0 pid: 4704 (default app)
--- no python application found, check your startup logs for errors ---
[pid: 4706|app: -1|req: -1/1] 172.16.0.102 () {42 vars in 714 bytes} [Mon Dec 11 07:50:58 2023] POST /m
anage/account/login => generated 21 bytes in 0 msecs (HTTP/1.0 500) 2 headers in 83 bytes (0 switches o
n core 0)
--- no python application found, check your startup logs for errors ---
[pid: 4706|app: -1|req: -1/2] 172.16.0.102 () {42 vars in 714 bytes} [Mon Dec 11 07:52:38 2023] POST /m
anage/account/login => generated 21 bytes in 0 msecs (HTTP/1.0 500) 2 headers in 83 bytes (0 switches o
n core 0)
[root@aops-mag uwsgi]#
```

3 Acronyms and Abbreviations

CVE: Common Vulnerabilities and Exposures. It is a program that assigns unique identifiers to publicly disclosed vulnerabilities.

LTS: long-term support, a software version, for example, Ubuntu 14.04 LTS.

SP: service pack, an integration of product updates. It may contain updates to system stability, security, and other aspects.